

Hardware Verification

A Holistic Guide

PARTS I–VII — COMPLETE DRAFT

Marco Paci · ChipsIT

2026-08-27

Draft for internal review — citations verified against full-text corpus

Hardware Verification: A Holistic Guide

Marco Paci · ChipsIT

First internal edition, August 2026.

© 2026 ChipsIT. All rights reserved.

Purpose of this edition. This edition is prepared for internal use: as a reference for engineers doing verification work, and as teaching material for engineers moving into verification from design. It is not a vendor document and recommends no commercial tool.

Sources. Every numbered reference in this book resolves to a document held in full text, and every claim carrying a reference was checked against the page cited rather than against recollection of it. Where the literature does not settle a question, the book says so. Where a figure comes from a single source, that source and its date are named, because a statistic without a date is not a statistic.

Example systems. The reference SoC and the flagship SoC used throughout are constructed teaching examples with fixed, published parameters. They are not any company's product, and no claim about them should be read as a claim about silicon. Claims about real organisations are a separate matter and are made only where a published source states them; each is cited to that source.

Trademarks. AMBA is a registered trademark of Arm Limited (or its subsidiaries or affiliates) in the US and/or elsewhere; Arm publishes trademark usage guidelines at arm.com. AXI, AHB, APB and ACE are protocol designations within the AMBA family and are used here descriptively. IEEE is a registered trademark in the U.S. Patent and Trademark Office, owned by The Institute of Electrical and Electronics Engineers, Incorporated. Verilog is a registered trademark of Cadence Design Systems, Inc. Other product, organisation and standard names appearing in this book may be the trademarks of their respective owners. They are used editorially, in the owner's favour, with no intent to infringe.

No endorsement. None of the organisations whose specifications, publications or results are cited in this book has reviewed, approved, sponsored or endorsed it. That includes Arm, IEEE, Accellera, RISC-V International, and every company named in a cited case study. Where this book characterises a published result, the characterisation is the author's and any error in it is the author's.

Quotation from standards. Short passages of normative text are quoted, with attribution to the document, its issue and its clause, for the purpose of identifying and explaining the requirement under discussion. Those quotations rest on the statutory right of quotation for purposes of criticism, review, illustration and teaching — Article 5(3)(d) of Directive 2001/29/EC as implemented in national law — and **not** on any specification licence: the AMBA specification licence grants copying for the purpose of developing products that comply with the specification, which is not the purpose of this book, and the specifications' proprietary notices reserve reproduction to the publisher's written permission. Accordingly no standard is reproduced in whole or in substantial part, and **no figure or table from any standard is reproduced**; where this book needs a diagram of something a standard specifies, the diagram is drawn from scratch. Reproducing a figure or a table would require written permission, which attribution does not substitute for.

Assistance disclosure. This book was written with substantial assistance from large language models. The systems used were Anthropic's Claude — models in the Claude Opus family for drafting, adversarial review and page-level citation checking, and a Claude Fable model for orchestration across chapters. The tools are not authors: responsibility for every claim, and for the decisions about what the book argues, rests with the named author. The editorial procedure — a closed source corpus, an independent review pass for each chapter, page-level verification of citations, and mechanical gates on numbers,

code, hedges and cross-references — is described in the preface, together with its known ceiling, so that a reader who wants to judge the method can.

Errors. Corrections are welcome and will be recorded. An error found in a book of this size is a contribution to it, not an embarrassment to it.

Contents

Figures	10
Tables	11
Preface	13
Notation and conventions	17
Part I — Foundations	21
1. Why Verification Exists	22
1.1 The problem: correctness is not the default	23
1.2 The numbers: how often the industry fails	24
1.3 The effort: who pays, and how much	26
1.4 The economics: what a bug costs, and when	28
1.5 Public history: when escapes became famous	30
1.6 The reference SoC: the running example of this book	31
2. The Verifier's Mindset	36
2.1 Designer and verifier: two minds, one specification	37
2.2 Thinking in terms of breakage	38
2.3 Independence of judgment	41
2.4 The value of doubt	43
3. The Verification Problem	47
3.1 The state-space explosion	47
3.2 Controllability and observability	52
3.3 The oracle problem	54
3.4 From exhaustive to risk-driven verification	56
4. The Verification Lifecycle	61
4.1 The shape of the lifecycle	62
4.2 Planning: the phase that decides all the others	64
4.3 From bring-up to regression maturity	67
4.4 Maturity stages as a shared language	69
4.5 Closure and sign-off	71
4.6 Roles and reviews: who checks what, and when	72
4.7 Schedule reality	74
Part II — Planning and Measurement	81
5. Verification Planning	82
5.1 From specification to feature list	83
5.2 The anatomy of a vplan row	86
5.3 Making the plan executable and traceable	91

5.4 The plan review as a rite	92
5.5 Risk: deciding how deep to go, feature by feature	93
5.6 Planning against an incomplete specification	95
6. Coverage: Theory and Practice	100
6.1 The coverage landscape: three families, one taxonomy	101
6.2 Code coverage: free, necessary, and structurally blind	102
6.3 Functional coverage: the metric you must deserve	103
6.4 What 100% means — and what it never can	105
6.5 Designing the coverage model: an engineering act	106
6.6 Worked example: a coverage model for the neural accelerator	107
6.7 When the cross explodes: shaping the crossbar model	110
6.8 Closure: the workflow from holes to done	112
6.9 Honest exclusions: the waiver as a reviewable artifact	114
7. Metrics-Driven Sign-off	118
7.1 Metrics as the project's shared truth	118
7.2 What to instrument	120
7.3 Reading bug-rate curves	122
7.4 The sign-off checklist: aggregating the evidence	123
7.5 Worked example: the dashboard, two weeks before tape-out	126
7.6 Escape analysis: the post-mortem that feeds the next plan	128
Part III — Dynamic Verification	133
8. Testbench Architecture	134
8.1 Five responsibilities, not one program	135
8.2 Layering: signal, transaction, sequence	136
8.3 Scoreboard patterns	138
8.4 Reference models: three species	141
8.5 Monitors and passive checking	143
8.6 Worked example: the crossbar environment, step by step	145
8.7 Worked example: the DMA environment and end-to-end checking	149
8.8 The environment as a design in its own right	150
9. Stimulus: Directed to Random to Portable	155
9.1 Directed tests: precision, and its ceiling	155
9.2 The constrained-random turn	157
9.3 Writing good constraints	159
9.4 Seeds and reproducibility	166
9.5 Sequences and stimulus layering	167
9.6 Graph-based and portable stimulus	169
10. UVM as a Methodology	175
10.1 The problem: reuse across projects, teams and companies	176

10.2 The component taxonomy and why it exists	177
10.3 The factory and the configuration database	179
10.4 Phasing: why a testbench needs a state machine	180
10.5 Sequences and the sequencer-driver handshake	182
10.6 The register layer	183
10.7 Reporting as infrastructure	186
10.8 Reuse in practice: the same agent, one level up	186
10.9 What UVM does not solve	187
10.10 When not to use UVM	188
11. Assertion-Based Verification	192
11.1 An assertion is a specification you can run	193
11.2 The anatomy of SVA	194
11.3 Assert, assume, cover: three claims about one property	196
11.4 Where assertions live	197
11.5 Protocol checking: the interface as a contract	197
11.6 White-box assertions and internal invariants	202
11.7 Assertion quality	204
11.8 The same text, two engines	205
12. Regression Engineering	210
12.1 What a regression is, and what earns a place in it	210
12.2 The (test, seed) matrix	212
12.3 Continuous integration for hardware	213
12.4 Failure triage at scale	214
12.5 Flakiness as a first-class enemy	216
12.6 Compute economics	219
Part IV — Static and Formal	227
13. Static Verification	228
13.1 The entry gate	229
13.2 Lint: from noise to policy	231
13.3 Clock-domain crossing	233
13.4 Structural checks and protocol checks	236
13.5 Reset-domain crossing	239
13.6 X-propagation	241
14. Formal Property Verification	247
14.1 What a proof is, and what it is over	248
14.2 Properties and assumptions	250
14.3 Bounded and unbounded proof	253
14.4 Abstraction: what you throw away	255
14.5 Convergence: the undetermined property	258

14.6 Formal sign-off	261
15. Formal Apps and Equivalence	269
15.1 The app: formal as a product	270
15.2 Connectivity checking: proving the wiring	271
15.3 Register checks: the map against its description	273
15.4 Unreachability: a proof instead of a waiver	275
15.5 Auto-generated properties: a draft, not a specification	276
15.6 Equivalence checking: the flow's quiet workhorse	277
15.7 Datapath: where the input space wins outright	279
16. Hybrid Flows	285
16.1 Dividing the work	286
16.2 Assumption and constraint duality, as a flow	287
16.3 Coverage unification	290
16.4 Choosing an engine, row by row	293
16.5 The handoff	295
Part V — Beyond RTL Simulation	299
17. Acceleration and Emulation	300
17.1 Why simulation runs out	301
17.2 Three platforms, and what each takes away	303
17.3 Use models: in-circuit and virtual	305
17.4 The transactor boundary	307
17.5 Economics: one machine, many claimants	312
18. FPGA Prototyping and HW/SW Co-verification	318
18.1 The prototype as a different instrument	319
18.2 Virtual platforms	322
18.3 Hybrid co-simulation	324
18.4 Firmware-driven verification	327
18.5 Who owns the bugs	331
19. Gate-Level, Timing and Power-Aware Verification	337
19.1 Why gate-level simulation still exists	338
19.2 Reading a gate-level failure	340
19.3 SDF back-annotation and what a timing check asserts	342
19.4 Power-aware verification	345
19.5 What to run, and when	352
20. Post-Silicon Validation	358
20.1 What changes when the design becomes a part	359
20.2 Design for debug: the budget you spend before you know what breaks	361
20.3 Bring-up: the first hours and days	364
20.4 Triage and localisation: the expensive step	367

20.5 Escapes and the feedback loop	372
20.6 Silicon debug meets the fix	375
Part VI — Specialized Domains	381
21. Analog and Mixed-Signal Verification	382
21.1 Where the digital abstraction stops	383
21.2 Real number modelling	384
21.3 AMS co-simulation	387
21.4 Behavioural models and the trust problem	388
21.5 What the digital team owns	391
22. Safety Verification	397
22.1 What changes when a standard is watching	398
22.2 Failure analysis as a verification input	400
22.3 Fault injection and fault simulation	403
22.4 Safety mechanisms and their verification	408
22.5 Tool qualification and the evidence trail	411
23. Security Verification	419
23.1 Why an adversary changes everything	420
23.2 The threat model as the plan's root	422
23.3 Security properties and how they differ	425
23.4 Information flow and its verification	427
23.5 Side channels	430
23.6 Where security verification lives in the flow	432
Part VII — The Human and the Machine	439
24. Teams, Process and Ramp-up	440
24.1 Roles, and what they actually own	441
24.2 The rites of review	442
24.3 Maturity stages as a shared language	445
24.4 Onboarding: designer → verifier	447
24.5 Process quality, and its limits	451
25. AI and ML in Verification	457
25.1 Classical machine learning, where the results are real	458
25.2 Large language models, with the evidence	461
25.3 Plausible by construction	466
25.4 The human gates	468
25.5 Governance	471
26. The Road Ahead	478
26.1 The bottleneck moves	479

26.2 Shift-left and shift-right	481
26.3 What does not change	483
26.4 Verification as a profession	485
Appendix A — Glossary	491

Figures

Figure 1.1	The four stages at which a bug can be found	28
Figure 2.1	The reconvergence model drawn for the DMA of this chapter's opening	38
Figure 3.1	The three-link chain a bug must survive to be found	52
Figure 4.1	The shape of a verification lifecycle	62
Figure 5.1	The five fields of a well-formed vplan row	86
Figure 6.1	The coverage-closure loop	112
Figure 7.1	How the sign-off checklist aggregates its evidence and what the review is then allowed to decide	125
Figure 8.1	The crossbar environment	145
Figure 9.1	The two families of stimulus constraint drawn as nested regions	159
Figure 10.1	The crossbar testbench as a UVM component hierarchy	178
Figure 13.1	Where each static check sits relative to the per-commit gate	230
Figure 14.1	The order to work an undetermined property in	259
Figure 17.1	Where the transaction boundary cuts an accelerated environment	308
Figure 18.1	A hybrid co-simulation partition	325
Figure 19.1	The reference SoC's two power domains and every signal that crosses between them	346
Figure 22.1	The reduction sequence that takes a fault list from every site crossed with every fault model down to the faults a campaign...	406
Figure 23.1	The flagship's root of trust	423
Figure 25.1	The gates a generated artifact passes before it is admitted to a suite	469

Tables

Table 1.1	The reference SoC element by element	31
Table 3.1	The per-channel programming interface of the reference SoC's DMA engine	48
Table 3.2	One DMA channel after expert abstraction	50
Table 3.3	What the 4 KB-crossing assertion of the worked example establishes about the DMA backend and what it leaves untouched	56
Table 4.1	An excerpt from the crossbar's feature list in week one	66
Table 4.2	The one-page evidence summary the crossbar's sign-off review starts from	72
Table 4.3	The review calendar in lifecycle order	73
Table 5.1	Four rows of the DMA vplan	88
Table 5.2	Five DMA features ranked on the two axes that set verification depth	94
Table 6.1	The four attributes of the accelerator's job coverage model	108
Table 7.1	Six questions to put to a bug curve that has gone flat	123
Table 7.2	The week-40 block dashboard the sign-off review reads	126
Table 8.1	The five responsibilities a testbench separates	135
Table 8.2	Scoreboard structure as a function of the ordering the design actually guarantees	140
Table 11.1	What the four property directives mean to each of the two engines that read them	196
Table 12.1	The four tiers of a regression suite	211
Table 12.2	The flagship SoC's nightly candidate list before any re-tiering	221
Table 14.1	The crossbar's formal results in the shape a sign-off package presents them	263
Table 14.2	The assumption register those results lean on	264
Table 15.1	The catalogue of formal apps	270
Table 15.2	The three equivalence flows side by side	279
Table 15.3	The two verification-plan rows a connectivity app earns at SoC top level	281
Table 16.1	What a cross-engine coverage report may and may not claim	291
Table 17.1	One verification problem per row	302
Table 17.2	A simulator, an emulator and an FPGA prototyping platform compared row by row	303
Table 17.3	The fraction of an accelerated platform's peak rate a testbench actually obtains	309
Table 19.1	The power-down and power-up sequences for the reference SoC's switched domain	347
Table 19.2	The useful combinations of the three independent choices that make up a gate-level run	353
Table 20.1	What changes when the design becomes a part	359
Table 20.2	On-die design-for-debug instruments	362
Table 20.3	The industrial path from a failed test run to a resolved bug	368

Table 20.4	Classifying a silicon escape backwards	373
Table 20.5	The three forward-looking options for an open bug once the masks exist	376
Table 21.1	Three published real number modelling results	385
Table 21.2	A model register entry laid out one field per row	390
Table 22.1	Three plan rows, in this book's five-column format	413
Table 23.1	The flagship's threat model as rows	423
Table 23.2	The four parts of an information-flow rule	428
Table 23.3	Six security questions	432
Table 25.1	What language models have been reported to do in this field	465

Preface

Why this book exists

Hardware functional verification is taught in pieces. There are good books on simulation-based methodology, good books on formal property verification, good papers on coverage, and standards documents for each of the languages involved. What there is very little of is an account of how the pieces bear on one another — of how a decision made in a verification plan determines what a proof can discharge, of what a coverage number means once formal results are merged into it, of which questions survive every instrument a project can afford. Those are the decisions a verification lead actually makes, and they fall in the gaps between the existing literature.

A second reason is narrower and more uncomfortable. Several of the figures this field repeats about itself — where verification effort goes, how often first silicon succeeds, what a bug costs at each stage — are older than the people repeating them realise, and some trace back to no primary measurement at all. A book that argues from evidence has to say which numbers survive being looked up. This one does, and it names the ones that do not.

What this book is

It is an account of hardware functional verification as a single discipline. It covers planning and measurement; simulation-based verification and the methodologies built on it; assertions; formal property verification and equivalence checking; static analysis, clock- and reset-domain crossing; gate-level and power-aware verification; acceleration, emulation and FPGA prototyping; hardware-software co-verification; analog-mixed-signal, safety and security verification; post-silicon validation; and the economics and metrics by which all of it is judged. It treats these as one subject because a project does not get to choose one of them, and because the interesting decisions are the ones about where the boundary between two of them should fall.

The organising claim is that verification is an argument about evidence, not a sequence of activities. A verification plan is a set of claims; coverage is a sample of a model somebody wrote; a proof holds under assumptions somebody discharged or did not. Every technique in this book is presented as an instrument that produces a particular kind of evidence, with a stated reach and stated blind spots, and the recurring question is which instrument answers which question, at what cost, and what remains unanswered when it has.

What it is not

It is not a manual for a methodology library, and not a tool tutorial. It names tool categories — simulator, formal tool, emulator, linter — and does not recommend products. Where a technique is inseparable from a standard, the standard is named precisely, with its issue and its clause, because that is the document a reader has to open to check the claim.

It is also not a survey. A survey reports what has been published; this book takes positions, and marks them as positions. Where the literature is divided, the division is described rather than averaged away. Where the published evidence is thin — and in several areas of this field it is remarkably thin — the book says so instead of filling the gap with confident prose.

Who it is for

Two readers, with different needs and one book.

The first is an engineer who does verification and wants the parts of the discipline they have not had occasion to use: a simulation-based verification engineer meeting formal property verification, or a formal engineer being asked about coverage closure. For that reader the chapters are self-contained enough to be entered directly, and Appendix B gives reading paths that name what each path equips you to do and what it leaves out.

The second is an engineer moving into verification from design. That reader is the reason the book develops its arguments from first principles rather than assuming a methodology background, and the reason a single worked example system recurs from beginning to end.

The example systems, and why they are constructed

Almost every example in this book runs on one of two constructed systems: a modest reference SoC, and a later, larger flagship SoC from the same fictional lineage. Their parameters are fixed and published, they do not change between chapters, and a bug found in Chapter 1 is still the same bug in Chapter 26.

The choice is deliberate and it has a cost worth stating. A constructed example cannot be looked up, and no reader can check it against a datasheet. What it buys is that the example can be complete: every parameter that an argument needs is available, the same crossbar can be examined from six different techniques' points of view, and a coverage model can be given cell by cell rather than gestured at. Examples drawn from real silicon are complete only where the owner chose to publish, which is rarely where a teaching argument needs them.

Claims about real organisations are a different kind of claim, and the book keeps the two apart. Where it reports what a named company did, that report comes from a published source — usually that company's own conference paper — and is cited to it. Where it reports what the industry does in aggregate, it names the study and its year.

The evidence discipline

The book was written against a closed corpus of documents held in full text: papers, conference proceedings, standards and books. The rule was that a claim either carries a reference into that corpus, or is presented as the author's position, or does not appear. Nothing is cited from memory of a paper.

Three habits follow from that rule and are worth naming, because they show up in the prose:

Statistics are dated. A figure about first-silicon success or about where verification effort goes is a measurement of a particular year, and the year is given. Several widely-repeated numbers in this field are older than the people repeating them realise.

Quantities are derived where they can be. Where a number in an example can be computed from the example's own parameters, the derivation is shown, so that a reader who disagrees can find the step they disagree with.

Absent evidence is reported as absent. There is no industry-wide data on several questions this book has to address. Saying so is more useful than a plausible estimate, and it tells a reader where their own measurements would be worth more than any citation.

How this book was made

It was written with substantial help from large language models — specifically Anthropic's Claude, using models in the Claude Opus family for drafting, adversarial review and citation checking, and Claude Fable and Opus models to coordinate work across chapters. The method is described here rather than buried, because a reader is entitled to weigh it.

Chapters were drafted against the corpus and the style contract, then reviewed by an independent adversarial pass whose task was to find unsupported claims, misattributed citations, arithmetic that does not hold, and internal contradictions; findings were then applied by a third pass empowered to refuse a finding it judged wrong. Citation checks were done at page level against the source document. Mechanical gates guard the classes of defect that reading does not catch: that every citation marker resolves; that no number, code block or cross-reference moves during an editorial pass without a stated reason; that no hedge is lost, because a rewrite that drops *typically* or *up to* has strengthened a claim its source does not support; that every figure and table is captioned and every reference to one resolves; and that the one term this book redefines against common industry usage — *validation* — never drifts in either direction. Eight neighbouring terms are counted rather than enforced, on the reasoning that a gate can compare occurrences but cannot read a sense, and that a count which moves deserves a person's attention rather than a build failure. Each gate was tested against a deliberate defect before being trusted, on the principle that a check nobody has seen fail is not a check.

The editorial passes were designed against the published measurements of what goes wrong when a language model edits a long document — that damage is sparse, severe and silent; that rewriting inflates certainty while appearing to preserve meaning; and that on already-clean text a model's precision at deciding what needs changing collapses. The mitigations chosen follow from those measurements rather than from intuition about them, and where the literature offers no measurement, this book's method says so rather than borrowing confidence from an adjacent result.

That procedure has a known ceiling, and the honest statement of it is this: an automated reviewer sharing an author’s blind spots will not see what the author did not see, and grounding a reviewer in sources improves how it judges a claim put in front of it far more than it improves what it thinks to check. The defects this method catches are misattribution, contradiction and arithmetic. The defect it catches least well is the fluent passage that is simply wrong and that nobody thought to question. Readers who find one have found something the process could not, and the author would like to know.

Acknowledgements

Use of generative artificial intelligence. Anthropic’s Claude was used in preparing this book, as described in “How this book was made” above: models in the Claude Opus family for drafting text, for adversarial review of drafts, and for verifying citations against the cited pages, and Claude Fable and Opus models for coordination across chapters. No text was published without being read. The systems are tools and not authors, they are not credited as authors, and the author accepts full responsibility for the whole of the content, including any error the review passes failed to catch.

Personal acknowledgements to be written by the author.

Notation and conventions

A reader who wants to check this book rather than trust it needs to know how it is written. This page states the conventions once.

Typography

Monospace marks anything that exists in a file or on a wire: signal names, identifiers, coverpoint and property names, code, file paths, and command fragments. *Italic* marks a term at the point where it is defined, and marks emphasis of a distinction. **Bold** marks the subject of a paragraph in the structured boxes described below, and nothing else.

References and how to check a claim

Citations appear as a superscript number keyed to a numbered list at the end of the chapter. Numbering is per chapter and follows first appearance, so the same source can be reference 3 in one chapter and reference 11 in another; the reference list gives the full citation each time, so no cross-chapter lookup is needed.

Each entry also carries the identifier of the source in the project's corpus. That identifier is not decoration: it means the full text of that document was held and opened, and the claim was checked against the page, not against a summary or an abstract.

Works the corpus does not hold in full text are never cited for a claim. They appear, where they are worth knowing about, under **Further reading**, marked as not held. The distinction is deliberate: a reference is a promise that somebody opened the document.

Standards

A normative document is identified the first time a chapter relies on it, by issuing body, designation, issue or version, and — where a specific requirement is asserted — clause. For example, a requirement of the AXI4 protocol is attributed to *AMBA AXI and ACE Protocol Specification*, ARM IHI 0022H.c (ID012621), with the clause that states it. The designation is given in the form the document itself prints, which is not always the form catalogues use.

Two consequences follow, and both matter for checking. First, the issue is part of the citation, because specifications change: the current issue of Arm IHI 0022 no longer contains AXI4 at all, so an AXI4 requirement can only be attributed to the last issue that specifies it. Second, where a requirement is quoted rather than paraphrased, the quotation is short and marked as a quotation, so that a reader can see where the specification's words end and this book's begin.

Pointers into this book, and pointers out of it

A section sign followed by a bare number always points inside this book: §7.3 is the third section of Chapter 7. It never means a section of anything else, so a reader who follows one never leaves the volume.

A clause of an external document is always preceded by that document’s designation, and its clause identifier carries the form that document uses — Arm IHI 0022H.c §A5.2.2, IEEE 1800-2023 §16.12. The designation is what disambiguates: without one, the pointer is this book’s. Where a document numbers its sections in the same bare style this book does, the word is spelled out instead, as in *the integration specification, section 3.2*, so that no bare §N.M in these pages can be mistaken for an outside reference.

Figures and tables

Every figure and every table is numbered by chapter — Figure 8.1 is the first figure of Chapter 8 — and carries a caption that says what the reader is looking at without requiring the surrounding paragraph. Figure captions sit below the figure, table captions above the table. Cross-references use the number, and the lists of figures and tables in the front matter give the page.

The numbers are assigned by the build from symbolic labels in the sources, so a reference in the text and the float it points at cannot drift apart when a figure is inserted or removed. A reference to a label that does not exist, and a label defined twice, both fail the build rather than producing a wrong number. An uncaptioned figure or table is reported by the build with its line number, so the guarantee in the paragraph above is **checked on every build rather than asserted**.

No figure or table in this book is reproduced from a standard or from another publication. Where a diagram of something a specification defines is needed, it is drawn for this book from the specification’s text.

The chapter apparatus

Chapters open with a statement of the problem on one of the example systems, in engineering terms, followed by a paragraph mapping the sections. Four structured elements recur:

Scenario boxes state a situation, the approach taken and the reason it is the right approach, in that order. They are constructed examples on the example systems unless a source is cited.

In practice boxes report what teams do, including the parts that are inconvenient. Each is either cited to a published source or marked as a constructed illustration; the book does not present unsourced practice as reporting.

Pitfalls list common mistakes as a symptom and a cure, on the assumption that a reader recognises the symptom before the cause.

Summary closes the chapter with the claims it established, not with a recap of its topics.

The example systems

Bare names refer to the **reference SoC**: “the crossbar”, “the DMA engine”, “the core”, “the neural accelerator” are that system’s blocks, with the parameters fixed in the front matter of Chapter 1 and unchanged thereafter. Parts of the larger, later **flagship SoC** are always qualified as such — “the flagship’s compute cluster”, “the flagship’s mesh NoC” — because several blocks exist in both generations at different scales, and an unqualified name would read as the same block twice.

A supporting cast of other devices — a safety-critical microcontroller, a secure element, an analog front end, and a set of IP blocks including a GPU, a video codec, an Ethernet controller and a flash controller — appears where a point needs a device the reference SoC does not have. These are introduced where they are used.

Vocabulary

The glossary in Appendix A is binding, not summarising: where it defines a term, this book means that and nothing else. One case is worth flagging here because the industry uses the word both ways. **Validation** in this book means checking a design on real hardware — on fabricated silicon, or in the lab on a prototype. Everything done on a model, before there is hardware, is **verification**. The distinction being drawn is hardware against model, not fabricated against programmable.

Quantities

Memory sizes and address boundaries are powers of two: 4 KB is 4,096 bytes, and the 4 KB boundary of the AXI protocol is that boundary. Data rates and clock frequencies are powers of ten: 200 MHz is 200×10^6 Hz. Large ratios are written in scientific notation, as 10^{12} , when the exponent is the point being made.

Two protocol encodings recur and are stated wherever they are used, because both are a common source of off-by-one errors: a burst length field encodes the number of beats minus one, and a coverage bin count is a count of cells in a model, not of tests.

Where a quantity in an example can be derived from that example’s published parameters, the derivation is given. Where it cannot, its source is cited. A number with neither is a defect, and finding one is worth reporting.

PART I

Foundations

Why verification exists, how a verifier thinks, and why the problem is formally impossible — so it is managed as risk.

1. Why Verification Exists

Eleven days before tape-out — the day the design database is frozen and sent for fabrication — the nightly regression of the reference SoC flags a single failing seed. The scoreboard, the testbench component that compares what the design did against what it should have done, reports a data mismatch on a DMA transfer. A two-dimensional transfer with a negative stride, whose row happens to cross a 4 KB address boundary, is split incorrectly by the DMA's transfer legalizer — but only when the second channel is competing for the AXI backend (AXI: the industry-standard on-chip bus protocol) in the same window. Two hundred and fourteen directed DMA tests had passed for months. None of them combined those three conditions, because nobody had imagined the combination. A constrained-random test, generating legal-but-unlikely descriptors by the thousands, stumbled into it in one night.

The fix takes a day. Now run the counterfactual, because the counterfactual is what this book is about. Suppose the regression had not caught it. The bug ships. First silicon arrives, boots, passes bring-up checks — the corruption needs a rare alignment and a busy second channel, so the lab doesn't see it for weeks. Then a customer's application corrupts a buffer, intermittently, under load. In the lab you cannot see inside the legalizer; observability is limited to what the silicon exposes. Root-causing takes weeks of a senior team's time. The fix requires a respin — a new mask set, re-fabrication, requalification — and the product misses its launch window. The difference between these two universes is one nightly regression. That difference, multiplied across every block of every chip, is why verification exists, why it consumes most of the engineering effort of a chip project, and why it deserves a book.

CHAPTER MAP

- §1.1 defines a bug escape and a respin, and states how the industry measures verification effectiveness — required silicon spins for ASICs, escaped bugs for FPGAs.
- §1.2 sets out the state of the numbers: first-silicon success at a two-decade low, most FPGA projects shipping non-trivial bugs, and where the effort actually goes.
- §1.3 establishes why verification dominates project effort and headcount, and why that share has been growing for twenty years.
- §1.4 develops the economics: how the cost of a bug escalates through the project lifecycle, and why verification is risk management, not a pursuit of perfection.
- §1.5 establishes what the famous escapes actually teach — that rare is not safe, that an escape's cost is set by the market rather than by the bug, and that each such case widened what verification is expected to cover.
- §1.6 introduces the reference SoC used as the running example throughout this book.

1.1 The problem: correctness is not the default

A digital design is a machine built from millions of interacting state elements, written by humans working from a specification that is itself written by humans. **Functional verification** is the set of activities that establish, before fabrication, that the design implements its specification — that it does what it is supposed to do, and does not do what it must not. It is distinct from **validation** — checking a design on real hardware rather than on a model, whether post-silicon on fabricated parts or in the lab on a prototype (Chapter 20 states the distinction in full). Much of the literature uses that word far more loosely, as an umbrella covering pre- and post-silicon work together; this chapter flags below one place where the looser sense quietly distorts a cost figure. It is distinct too from manufacturing test, which checks that each fabricated die is free of physical defects. Verification asks: *is the design right?* Test asks: *was this copy built right?*

Two terms anchor everything that follows. A **bug escape** is a design flaw that survives verification and reaches the next stage — full-chip integration, silicon, or the field. A **respin** is the consequence when an escape reaches silicon and cannot be worked around: a new fabrication cycle with corrected masks. Respins are the industry’s bluntest verification metric precisely because they are unambiguous — you either shipped first silicon or you paid for another pass. **First-silicon success**, the fraction of projects whose first fabricated silicon is production-worthy, is therefore the effectiveness headline of the whole discipline.

Why is correctness so hard to reach that a mature, forty-year-old industry still fails at it most of the time? Chapter 3 develops the theory — state spaces that dwarf the number of atoms in the universe, the impossibility of exhaustive checking, the oracle problem — but the practical shape of the difficulty is visible already in the opening scenario. The DMA bug required the *conjunction* of three individually harmless conditions. Designs fail at the intersections: between features, between blocks, between clock domains, between hardware and software. A 10G Ethernet MAC with time-sensitive-networking queues fails the same way: the timestamping logic is right, the frame-preemption logic is right, and the bug lives where a high-priority frame preempts an in-flight one in the same window the timestamp is captured. And the intersections multiply faster than the features do. By 2024, 84 percent of IC/ASIC projects incorporate one or more embedded processors, 95 percent have two or more asynchronous clock domains, 83 percent include security features, and 44 percent follow at least one safety-critical development standard ^[1]. Every one of those numbers is a layer of requirements that burdened far fewer projects twenty years ago, and every layer multiplies the intersections where bugs live.

Scenario. The reference SoC integrates a neural accelerator that shares the crossbar with two DMA channels. The accelerator team verified their block standalone: every convolution mode, every weight precision, all passing. The DMA team did the same. **Approach.** A competent verification lead still budgets integration verification for the *combination* — accelerator streaming weights while DMA channel 1 fills the next buffer — because block-level correctness does not compose into sys-

tem-level correctness. **Why.** Arbitration, ordering, and back-pressure behaviors only exist when both actors are active; no standalone environment exercises them. In practice, teams observe that bugs cluster where verified pieces meet.

1.2 The numbers: how often the industry fails

Numbers in this section come from two industry-wide surveys you will meet throughout the book: a worldwide, double-blind study of 1,886 participants covering 2014, the largest independent one of its kind at the time ^[2], and its 2024 successor (597 eligible participants, ± 4 percent margin at 95 percent confidence), published as separate IC/ASIC and FPGA trend reports ^[1, 3]. Survey data has known biases, candidly documented: regional participation shifts, for instance, can distort year-to-year language-adoption trends ^[1]. The effectiveness and effort trends, however, are consistent across a decade of editions and are the best quantitative picture the industry has of itself.

Both studies are Wilson Research Group studies, and their lineage is worth knowing, because it is the reason numbers exist here at all. The series was commissioned by its authors at Mentor Graphics and executed by an outside firm expressly to keep respondents anonymous, and it deliberately preserves the wording of the earlier Collett International Research studies of 2002 and 2004 so that trends stay comparable; after 2004 no further Collett study was conducted, and that void is what the Wilson series was created to fill ^[2]. The change of scale between the lineages is itself informative: the 2004 Collett sample was 201 eligible participants, against 1,886 in 2014 ^[2].

First-silicon success is falling

In 2014, only about 30 percent of IC/ASIC projects achieved first-silicon success ^[2]. A decade later the figure is 14 percent — the lowest recorded in twenty years, capping a decline that runs unbroken from 2012 to 2024 ^[1]. Read that number the way a project team must: *six out of seven chip projects pay for at least one extra fabrication cycle*. The respin is so common that experienced organizations budget for it — which is rational, and also a quiet admission of how far verification effectiveness lags design ambition.

What sends projects back to the fab? “Logic or functional flaws” — plain functional bugs, the kind verification exists to catch — remain the leading category of flaws contributing to respins, ahead of analog, timing, power, and physical categories (multiple flaws can contribute to one respin, so categories sum past 100 percent) [1, 2]. Digging one level deeper into the root causes of those functional flaws: design errors lead, and problems traced to changing, incorrect, or incomplete specifications are a persistent, prominent second theme [1, 2]. Hold onto that second one. A large share of “hardware bugs” are not coding mistakes at all — they are ambiguities and contradictions in what the design was *supposed* to do, discovered late. Verification, done well, is the activity that surfaces them early; this is why Chapter 5 treats the verification plan as an instrument for interrogating the specification, not just a list of tests.

Schedule tells the same story from a different angle. In 2014, 61 percent of IC/ASIC projects were behind their original schedule [2]. By 2024 the historical average of roughly 66 percent had worsened to 75 percent [1]. Verification does not bear sole blame for slips, but it sits on the critical path at the end of the project, which means it inherits every upstream delay *and* absorbs the blame for its own.

FPGAs: the illusion of a free respin

FPGA projects have no mask sets, so “respin” seems costless: fix the RTL (register-transfer level, the synthesizable description of the design), rebuild the bitstream, reprogram the part. The data punctures this comfort with its single most alarming number: asked in 2024 how many non-trivial bugs escaped into production, only 13 percent of FPGA projects answered *zero* — meaning 87 percent shipped production systems with non-trivial bugs, a result notably worse than the IC/ASIC first-silicon rate [3]. The diagnosis is cultural: many FPGA teams deliberately minimize pre-lab verification, racing to the lab to validate the design there — a strategy that worked for the simple FPGAs of the past and fails for today’s SoC-class devices [3]. And a field escape is not free even when reprogramming is: in aerospace systems, replacing or re-programming an FPGA requires removing system covers and triggers a full system revalidation, compounding cost and delay [3].

Scenario. A product team reuses the reference SoC’s I/O subsystem in an FPGA-based industrial controller. The schedule review proposes cutting simulation-based verification to three weeks — “it’s an FPGA, we’ll shake bugs out on the bench.”

Approach. The verification lead counters with the escape data [3] and a targeted compromise: full constrained-random verification for the newly modified camera-interface datapath, lab-only validation for the unmodified, previously verified UART and SPI paths. **Why.** Lab debug has a fraction of simulation’s observability — you see pins and probe points, not internal state — so bugs found there cost multiples to root-cause. Spending pre-lab effort *where the design is new* buys the most risk reduction per engineer-week; reuse of verified IP is precisely what lets some projects legitimately spend less on verification [1].

What the trend data is really saying

It is tempting to read “14 percent first-silicon success” as an indictment: twenty years of methodology — constrained-random stimulus, functional coverage, assertions, UVM (the standard testbench methodology, IEEE Std 1800.2-2020, *IEEE Standard for Universal Verification Methodology* [4], Chapter 10), formal — and the outcome got worse? The honest reading is different. Design complexity has grown relentlessly along every measured axis: gate counts, embedded processors (52 percent of designs in 2004, 71 percent by 2014 [2], 84 percent in 2024, with 58 percent of projects integrating a RISC-V core and 59 percent some form of AI accelerator [1]), clock domains, security, safety. Against that headwind, holding respin rates anywhere near flat represents enormous productivity gains in verification — gains delivered precisely by the techniques this book teaches. The 2014 data still supported an optimistic reading: designs had grown dramatically, yet required spins were not getting any worse [2]. The 2024 decline suggests the headwind is currently winning [1]. The remedy’s direction is unchanged across the decade: in 2024, projects using formal verification and mature methodologies experienced fewer spins [1]. The techniques work. There are not yet enough hands, hours, and adoption to keep pace — which is the subject of the next section.

1.3 The effort: who pays, and how much

You will hear, in nearly every verification talk and textbook, that verification consumes “70 percent of the effort.” The figure has a curious status. Exactly this claim has been called “subjective or unsubstantiated” — folklore repeated from publication to publication without primary data [2]. Yet the folklore is well rooted: verification consumes about 70 percent of design effort in the SoC era [5]; multiple chip projects cite approximately 70 percent of design time spent in verification [6]; and validation in the umbrella sense — pre- and post-silicon together, not the post-silicon activity this book means by the word — is estimated at 70 percent of overall time and resources in SoC design [7]. That third figure is a reminder to read the scope before the number: the same source puts *post-silicon* validation at half, which is the figure this chapter cites later. What does the measured data say?

Measured, the average share of total project time spent in verification was **57 percent** in 2014, essentially unchanged from 2012 — with wide dispersion in both directions, and a growing tail of projects spending more than 80 percent of their time verifying [2]. The dispersion is the insight: projects assembling pre-verified IP legitimately spend far less; projects with high proportions of newly developed IP spend far more [1]. So the honest summary is: *the majority of project time, with “how much more than half” determined mostly by how much of your design is new.* The 70 percent folklore is not wrong so much as unqualified.

Time is only one axis; headcount is the other, and its trend is steeper. Between 2007 and 2014, demand for design engineers grew at a 3.7 percent compound annual rate, while demand for verification engineers grew at **12.5 percent** — and by 2014, the mean peak number of verification engineers on a project exceeded the mean

peak number of design engineers [2]. The growth has since cooled (2016–2024: 2.4 percent for design, 3 percent for verification on IC/ASIC projects [1]) — but from that already-inverted baseline, and in some market segments, such as processors, a five-to-one ratio of verification to design engineers is not unusual [1]. The FPGA world is now living the IC/ASIC world’s 2007–2014 transition: FPGA verification headcount grew at 8.5 percent CAGR against 4.7 percent for design between 2012 and 2024 [3]. Guidance from 2006 that verification engineers may number up to twice the RTL designers [5] reads, two decades later, as conservative for some segments.

Two more findings complete the effort picture, and both should shape how you plan.

First, *designers verify too*. In 2024, IC/ASIC design engineers spent on average 49 percent of their time on verification activities — nearly half of the “design” headcount is, functionally, verification effort [1]. Add that to a dedicated verification team that equals or outnumbers the design team, and the conclusion is inescapable: on a typical project developing new IP, well over half of all engineering hours go to verification. Whatever your organization chart says, you are running a verification project with a design attached.

Second, *within verification*, the largest single consumer of an engineer’s time is **debug** — more than test planning, testbench development, or running tests — in 2014 and again in 2024, for both IC/ASIC and FPGA engineers [1, 2, 3]. The management consequence is the same in both years: debug time is unpredictable and varies wildly between projects, which is why verification schedules built by extrapolating the previous project’s effort keep failing [1]. Much of this book’s advice — self-checking testbenches (Chapter 8), assertions that catch bugs at their point of origin (Chapter 11), disciplined triage (Chapter 12) — is, at bottom, debug-time reduction.

In this scenario: staffing the reference SoC

Concreteness check. The reference SoC — one RISC-V core, a crossbar, a two-channel DMA, an I/O subsystem, a neural accelerator, and an APB peripheral set, on three clock domains and two power domains — is a small chip by industry standards. A plausible staffing: four RTL designers, five dedicated verification engineers (core and instruction-set simulator (ISS) co-simulation; crossbar and DMA; accelerator; I/O and peripherals; integration and regression infrastructure), with each designer additionally spending near half their time on block-level verification of their own RTL [1]. If the five-to-one processor-segment ratio [1] sounds implausible from here, consider what changes with an out-of-order server core: the design team grows by perhaps three times, but the state space — speculation, replay, coherence, ordering — grows combinatorially, and verification headcount must chase the state space, not the gate count. That asymmetry, effort scaling with *behavior* rather than *structure*, is the deepest reason verification’s share keeps rising, and we will meet it formally in Chapter 3.

1.4 The economics: what a bug costs, and when

Verification is an economic activity. Nobody funds it for love of correctness; it is funded because bugs found late cost more — spectacularly more — than bugs found early. The entire discipline can be read as an arbitrage on that cost curve.

The curve’s mechanism is the interplay of two quantities that move in opposite directions as the project advances: the **cost to find and fix** a bug rises, while the **time available** to deal with it shrinks [7]. Behind both sits a third quantity, **observability** — how much of the design’s internal state you can see when something goes wrong. In RTL simulation you can observe any internal signal at any time; on an FPGA prototype, observability shrinks to a few thousand pre-selected signals; on silicon, you see only what the design-for-debug hardware routes to a pin or a trace buffer [7]. Less visibility means slower root-causing, at exactly the project stage where days are most expensive. The collapse is worst where the failure leaves nothing to look at: an HBM controller whose PHY training sequence stalls mid-step yields, on silicon, a link that never comes up and a bus with no traffic on it — while in simulation the same stall is a signal you can force, step through, and print. [Figure 1.1](#) walks the four stages, each with what can still be seen there and what a fix has become by then.



Figure 1.1 — The four stages at which a bug can be found, from block RTL simulation to the field, each labelled with the observability available at that stage and the form a fix takes there. The two degrade together: as the fix moves from an edit and a recompile to a recall or a respin, observability falls from any internal signal at any time to only what the design-for-debug hardware routes to a pin or a trace buffer.

Some corpus-verified waypoints along this curve:

- **Simulation is slow but transparent.** RTL simulation runs roughly a billion times slower than the target silicon: a workload that takes seconds on the chip — booting an operating system — would take years in an RTL simulator [7]. This is why simulation-found bugs are cheap (perfect visibility, instant fix) but simulation alone cannot reach system-level behavior, and why emulation and prototyping exist (Chapter 17).
- **Post-silicon debug is measured in weeks.** On IBM’s POWER8, medium-severity post-silicon bugs took on average about 10 days to root-cause, with the 90th percentile at 20 days [7] — per bug, on one of the most instrumented and best-staffed programs in the industry.
- **A respin is refabrication plus requalification.** An escape that reaches silicon and cannot be worked around forces redesign, re-validation, and expensive refabrication [7]. The mask and fab costs are the visible part; the dominant term is usually time.
- **The market window dwarfs everything.** Missing a product’s launch window — the holiday season, a customer’s design-in deadline — can mean millions to bil-

lions of dollars of lost revenue, or missing the market entirely; and a bug that reaches the field can add recall costs and reputation damage on the same scale [7].

A widely repeated rule of thumb says the cost of a bug grows by an order of magnitude at each stage — block, chip, silicon, field. The corpus offers no measured cost-per-stage series and you should treat the “10×” as a mnemonic, not data; but every mechanism above pushes in its direction, and no mechanism pushes back.

The industry pre-pays staggering amounts to bend this curve. Beyond the engineering effort of the previous section, it pays in silicon itself: in some modern SoCs, over 20 percent of die area is devoted to design-for-debug and validation instrumentation [7] — a fifth of the product, spent on observability for bugs not yet found. And post-silicon validation as a whole is estimated in multiple studies at up to 50 percent of a design’s overall cost, measured as time [7]. Pre-silicon verification effort is what keeps that downstream machinery from becoming the primary bug-finding mechanism, a role it is economically catastrophic to play.

Scenario. During integration of the reference SoC, a formal tool proves a protocol assertion false on the crossbar: under sustained back-pressure, two write transactions from different managers, in flight to the same subordinate, can interleave their data beats — an ordering violation, since AXI4 obliges an interconnect to forward merged write data in address order. The counterexample is 11 cycles long.

Approach. The team fixes the arbiter and re-proves the property, total cost: two engineer-days. **Why.** This bug class is nearly invisible downstream: it corrupts data only under traffic patterns that directed tests and even most random runs never create, and in the lab it would present as unreproducible memory corruption. Caught at RTL by an assertion, it costs days — and the 2024 data associates exactly this maturity with fewer spins [1]; caught in the field, it is the “millions to billions” scenario [7]. Same bug, orders of magnitude apart in consequence.

Worked example: anatomy of a near-escape

Here is the opening scenario’s bug, written up the way a competent team files it. Every field of this report will get its own chapter — severity and triage (Chapter 12), escape analysis (Chapter 7), coverage crosses (Chapter 6) — but the shape is worth internalizing now: a bug report is an economic document. It records not just what broke, but what the escape *would have cost* and what process change prevents its siblings.

Bug ID:	SOC-1284
Title:	DMA ch0 2D negative-stride transfer corrupts data at 4 KB boundary when ch1 is concurrently active
Found by:	Nightly constrained-random regression (scoreboard mismatch), 11 days before tape-out
Stage found:	RTL simulation, full-chip environment
Severity:	Critical — silent data corruption, no error response raised
Root cause:	Midend transfer legalizer mis-splits a negative-stride 2D row that crosses a 4 KB AXI boundary; the malformed burst is masked unless ch1 owns the backend in the same window


```

(arbitration timing hides it otherwise).
Escape analysis: 214 directed DMA tests passed; none combined negative stride
                  × 4 KB crossing × dual-channel activity. Functional coverage
                  model had no cross of these three dimensions – the hole was
                  invisible to our own metrics.
Cost if escaped: intermittent field corruption → weeks of bring-up debug at
                  ~10 days/bug post-silicon norms, likely respin + requal
Actions:         (1) fix legalizer split logic; (2) add coverage cross
                  stride_sign × boundary_cross × channel_concurrency;
                  (3) extend scoreboard with end-to-end data check for 2D
                  descriptors; (4) sweep sibling risk: audit all legalizer
                  corner cases against the AXI 4 KB rule.

```

Note what found the bug and what did not. The directed tests — each written to check one imagined behavior — all passed, honestly. The constrained-random environment found the unimagined combination, and the scoreboard, not any human, noticed the corruption. The coverage model, meanwhile, could not even *represent* the hole. Automated checking is what makes that division of labor portable: a video codec team runs the same discipline against a bit-exact reference decoder, because “the picture looks right” is not a verdict a regression can produce at three in the morning. One bug, and it has already argued for two of this book’s parts: stimulus and checking (Part III), and measurement (Part II).

The verification gap

Economists would call the pattern of this chapter a widening gap between what projects must verify and what they can. Practitioners have given it a useful taxonomy: the **verification gap** decomposes into an *inherent* gap (complexity growth you cannot avoid — more IP, more cores, more protocols, more software content), a *transient* gap (skills and tooling that lag but catch up), and a *self-induced* gap (schedule pressure forcing shortcuts, poor environment quality, metrics collected for their own sake, best-known methods sitting unused at an estimated 20 percent productivity cost) [8]. The decomposition is a management tool: you *innovate* against the inherent gap, *invest* against the transient one, and *introspect* against the self-induced one [8]. Much of this book is aimed at the third — the gap you created and can close without waiting for anyone’s roadmap.

1.5 Public history: when escapes became famous

Most escapes die quietly — an erratum in a datasheet, a software workaround, a stepping nobody outside the team remembers. A few became public, and one became the founding legend of the discipline. The narrative in this section is public history rather than cited fact; where the corpus supports a claim, the citation says so.

In 1994, a mathematics professor doing number-theory research noticed that his brand-new Pentium machines occasionally divided incorrectly. The flaw was real: for rare combinations of operands, the floating-point divide unit returned results wrong in the low-order digits. The root cause — as it emerged publicly — was a handful of missing entries in a lookup table used by the division algorithm: a tiny, deterministic

flaw, exquisitely hard to hit by chance. Intel initially argued that a typical user might never encounter the error in the lifetime of the machine; the argument was statistically defensible and commercially disastrous. The story reached the mainstream press, and the company was eventually forced to offer replacement of the affected chips already in the market [6] — a recall whose direct cost was widely reported at the time, and whose reputational cost was larger. The case remains the canonical example of why failure costs drive verification [6].

The FDIV lesson is routinely mis-stated as “test your dividers.” The durable lessons are different. First, *rare* is not *safe*: a per-operation probability of 10^{-9} meets millions of chips executing billions of operations, and the field will find what your test-bench did not — a theme that returns as coverage philosophy in Chapter 6. Second, escape cost is set by the market, not by the bug: the flaw affected almost no real computation, and it still forced a replacement program, because trust, once questioned in public, is repriced for the whole product line. Third, the aftermath, not the bug, changed the industry: formal verification of floating-point datapaths moved into mainstream practice at processor vendors, and the case study still opens the era’s verification texts [6].

Public history since then has kept adding chapters, each extending the definition of “escape.” Processor vendors today routinely publish errata sheets — living catalogs of post-silicon bugs with workarounds, a practice so normalized that an erratum is only news when it cannot be worked around. And in 2018, the public disclosures of speculative-execution side channels showed that a design can be functionally flawless — every architectural result correct — and still leak secrets through its microarchitecture, an escape of the *specification* rather than of the implementation. The significance of these episodes for this book is structural: each famous escape widened what verification is expected to cover — correctness, then safety, then security (Chapters 22 and 23) — and the data of this chapter shows the industry absorbing exactly those layers [1].

1.6 The reference SoC: the running example of this book

Throughout this book, one design carries the examples: a microcontroller-class, single-core RISC-V SoC for low-power edge applications (*The RISC-V Instruction Set Manual, Volume I: Unprivileged Architecture*, Version 20250508, ratified, RISC-V International [9]) — small enough to hold in your head, real enough to contain every verification problem the industry’s data describes. It is a composite, technically faithful to the class of edge-AI microcontrollers shipping today, and deliberately anonymous. Here is the cast you will keep meeting:

Table 1.1 — The reference SoC element by element: the design that carries this book’s examples, described in one phrase per element with the canonical parameters — widths, counts, capacities, frequencies — that every later chapter

reuses unchanged. Read it as a cast list rather than as a specification; a bare mention of the core, the crossbar or the DMA engine anywhere in the book means the row named here, with these numbers.

Element	Description	Canonical parameters
the core	4-stage, in-order, single-issue RV32IMC core with optional custom DSP/SIMD extensions	~50k gates; OBI-like instruction/data interfaces; M/U privilege modes; CLINT-style interrupts with a fast-IRQ extension
the crossbar	full AXI4 crossbar interconnect (<i>AMBA® AXI and ACE Protocol Specification</i> , Arm IHI 0022H.c, 2021) ^[10]	5 managers (core-I, core-D, DMA×2, debug) × 4 subordinates (SRAM, boot ROM, APB bridge, neural accelerator); 32-bit address, 64-bit data; ID width 4
the DMA engine	modular multi-channel DMA: frontend (registers), midend (transfer legalizer/splitter), backend (AXI4 manager)	2 channels; 1D/2D transfers; max burst 256 beats; 16-entry × 64-bit datapath FIFO in the backend; per-channel error reporting
the I/O subsystem	autonomous I/O engine moving data between peripherals and memory without core intervention	lightweight per-peripheral DMA; UART, SPI, I2S, camera interface
the neural accelerator	fixed-point convolution engine for quantized neural networks	2/4/8-bit weights; 16-lane MAC datapath; streams weights from memory; register-programmed jobs
the memory	multi-banked tightly-coupled SRAM, word-interleaved, single-cycle	512 KB in 4 banks; logarithmic interconnect toward core and DMA
the peripherals	APB subsystem (<i>AMBA® APB Protocol Specification</i> , Arm IHI 0024, Issue E, 2023) ^[11]	UART, SPI, I ² C, GPIO, timer, event/interrupt unit
clocking	3 asynchronous domains	system @ 200 MHz, peripheral @ 50 MHz, always-on RTC @ 32 kHz
power	2 UPF (Unified Power Format, the standard power-intent description — IEEE Std 1801-2024, <i>IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems</i> ^[12]) power domains	always-on (RTC, wake-up logic) + switchable system domain; retention on the event unit
debug	RISC-V debug module over JTAG	run-control + abstract memory access

The choice is not nostalgic minimalism; it is representative by the numbers. An embedded processor (like 84 percent of 2024 IC/ASIC projects), a RISC-V one (like 58 percent), an AI accelerator (like 59 percent), multiple asynchronous clock domains (like 95 percent), security- and safety-relevant variants in later chapters (like 83 and 44 percent respectively) ^[1]. When this small chip generates a hard verification problem, it is the same problem the median project faces.

Each element is also a deliberate teaching seam. The crossbar carries protocol verification — handshakes, ordering, ID routing — and the formal methods of Part IV. The DMA carries data-mover verification: address legalization, the 4 KB boundary, error responses, descriptor corner cases. The core carries ISA-level verification against a golden ISS, privileged state, and interrupt corner cases. The accelerator

carries datapath and quantization corner cases plus memory contention with the DMA. The three clock domains carry CDC (Chapter 13); the two power domains carry low-power verification (Chapter 19); the register files everywhere carry register-model verification. And a handful of *recurring bugs* — SOC-1284 above is the first — will resurface across chapters, each time caught (or missed) by a different technique, so you can compare the techniques on constant ground.

When scale matters, the examples move up a generation. The **flagship SoC** is the reference SoC's successor: it inherits the first generation's DMA architecture, register discipline and verification assets, then adds what only scale can teach — a host cluster of four application-class RV64GC cores running an OS, whose MESI directory across four private L1s makes coherence a verification problem; a 4×4 mesh NoC and an LPDDR4 subsystem, large enough to move interconnect and memory verification to emulation and system scale; a root of trust that puts security inside the SoC; and processor trace and bus monitors, the instrumentation post-silicon bring-up debugs with. Its automotive derivative, the flagship-A, carries safety at full-SoC size. The subsystems keep standalone lives too: the **compute cluster** (8 cores sharing multi-banked memory) and the mesh NoC are block-level examples before they are the flagship's. Specialized chapters call in whatever their domain needs — an automotive brake-controller MCU, a secure element, an analog front-end. All obey the same rule: same parameters everywhere, no variants, and a bare name always means the reference SoC's.

IN PRACTICE

What do real teams actually do with the numbers in this chapter? Less than the numbers deserve. Most organizations track respins and escaped bugs *after the fact* but few feed them back systematically — escape analysis, when it happens, is often a slide in a post-mortem rather than a change to the next verification plan. Effort estimation is dominated by last-project extrapolation, which the data warns against: debug, the largest effort component, is the least predictable one ^[1]. Schedule pressure remains the standing force against verification quality — the self-induced gap ^[8] — and in most teams the verification schedule is the buffer that absorbs design slips, then takes the blame for the total. The healthiest teams we know of do three unglamorous things: they write the escape post-mortem *into* the next project's plan, they staff verification as the data says (at least one-to-one against design, before counting the designers' own verification time ^[1, 2]), and they treat "all tests pass" as the *beginning* of a sign-off argument, not its conclusion (Chapter 7).

PITFALLS

1. **Treating verification as a phase that starts when RTL is done.** Symptom: verification team "waiting for a stable drop." Cure: verification starts with the specification — plan, testbench architecture, and assertions proceed in parallel with design (Chapter 4).

2. **Equating “all tests pass” with “verified.”** Symptom: green dashboard, no coverage model, tape-out confidence by acclamation. Cure: passing tests measure only what you imagined; measure coverage and hunt what you did not imagine (Chapters 5–7).
3. **Staffing verification as the designers’ side job.** Symptom: no dedicated verification owners; designers verify their own interpretation of the spec. Cure: independent verification ownership, staffed to the industry’s ratios, with designers’ ~50 percent verification time budgeted explicitly ^[1].
4. **Counting on the lab.** Symptom: “it’s an FPGA / we’ll catch it in bring-up.” Cure: remember that 87 percent of FPGA projects ship non-trivial bugs ^[3] and that lab observability is a fraction of simulation’s ^[7]; move effort pre-lab for everything newly designed.
5. **Costing a respin as masks-plus-fab.** Symptom: risk register prices an escape at the refabrication invoice. Cure: price the schedule slip and the market window — the corpus-documented range is millions to billions ^[7].
6. **Skippping the escape post-mortem.** Symptom: bug fixed, everyone relieved, nothing changes. Cure: every escape (and near-escape) is a defect in the *process*; file the SOC-1284-style analysis and change the plan, the coverage model, or the checklist it exposed.

SUMMARY

- Verification exists because escapes are ruinous: bug cost rises and fix time shrinks at every project stage, from editable RTL to recalled product ^[7].
- The industry, measured: 14 percent IC/ASIC first-silicon success in 2024 — a twenty-year low ^[1] — and 87 percent of FPGA projects shipping non-trivial bugs ^[3].
- Functional flaws remain the leading cause of respins; their root causes are design errors and, persistently, changing or ambiguous specifications ^[1, 2].
- Verification is the majority of project effort: 57 percent of project time on average a decade ago ^[2], verification headcount at or above design headcount ^[1, 2], and debug the biggest single time sink within it ^[1].
- The classic “70 percent” figure is folklore with honest roots ^[2, 5, 6, 7]; the defensible claim is “more than half, growing with the fraction of new IP.”
- Verification is economics: you are buying risk reduction with engineer-time, and the techniques that buy it cheapest early (assertions, formal, constrained-random with strong checkers) are the subject of this book. Perfection is not on offer — Chapter 3 explains why.
- The reference SoC introduced here is the book’s running example; its parameters do not change, and its bugs will return.

FURTHER READING

From the corpus: the two 2024 Wilson Research Group trend reports — IC/ASIC ^[1] and FPGA ^[3] — are short, current, and worth reading in full; Foster’s DAC 2015 paper ^[2] adds the methodological background of the study series and the 2007–2014 trend arcs. Mishra et al.’s post-silicon tutorial ^[7] is the best single source on the economics of late-stage bugs. Bergeron’s opening chapter ^[5] and Bertacco’s introduction ^[6] give the early-2000s view of the same crisis — instructive precisely because the ratios have barely moved. The Oracle Labs “verification gap” presentation ^[8] is a compact management vocabulary for everything in this chapter.

Not in the corpus (cited here as general references only): Wile, Goss and Roesner, *Comprehensive Functional Verification* (Morgan Kaufmann, 2005), the classic full-lifecycle treatment; the Collett International Research functional verification studies of 2002 and 2004, the ancestors of the Wilson series, known today primarily through the trend baselines they established; and the contemporary press coverage of the 1994 Pentium FDIV recall, which put the direct charge at roughly half a billion dollars — a figure quoted everywhere, supported nowhere in this book’s corpus, and therefore kept out of the chapter’s argument.

REFERENCES

1. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
2. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," Proc. 52nd Design Automation Conference (DAC '15), San Francisco, CA, 2015.
3. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
4. **S9** IEEE, "IEEE Std 1800.2-2020, IEEE Standard for Universal Verification Methodology Language Reference Manual (Revision of IEEE Std 1800.2-2017)," IEEE, 2020.
5. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
6. **B3** V. Bertacco, "Scalable Hardware Verification with Symbolic Simulation," Springer, 2006.
7. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
8. **D16** R. Narayan and T. Symons, "I created the Verification Gap," Proc. DVCon U.S., 2015.
9. **S14** RISC-V International, "The RISC-V Instruction Set Manual Volume I: Unprivileged Architecture, Version 20250508 (Ratified)," RISC-V International, May 2025.
10. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
11. **S19** Arm Limited, "AMBA APB Protocol Specification, ARM IHI 0024, Issue E (ID022823)," Arm Limited, February 2023.
12. **S10** IEEE, "IEEE Std 1801-2024, IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems (Revision of IEEE Std 1801-2018)," IEEE, 2024.

2. The Verifier’s Mindset

The DMA engine is finished, and its designer built every part of it: the register frontend she specified, the midend that legalizes and splits transfers, and the AXI4 backend she tuned for full-bandwidth bursts. She verifies the block herself. In one week she has a directed test suite: register defaults and read/write paths, 1D transfers of assorted lengths and alignments, a 2D transfer, a deliberately erroneous address to check the per-channel error report, completion interrupts. Everything passes. The block is declared verified and integrated.

Three months later, a system-level constrained-random regression fails on one seed in four hundred. The scoreboard — the checker that compares the memory image produced by the DMA against a reference prediction — flags corrupted data in a 2D transfer with a *negative* row stride, in a row that happens to straddle an AXI 4 KB address boundary (Arm IHI 0022H.c §A3.4.1) ^[1], and only while channel 1 is concurrently streaming audio buffers. The midend legalizer, which must split any burst that would cross a 4 KB page, computes the split point assuming ascending addresses; with a negative stride, and only when a channel switch disturbs its internal state, it emits a mis-legalized burst and the write lands partly in the wrong page. None of the designer’s tests could have found this — not because she tested carelessly, but because of *how she thought about testing*. Her stimulus exercised the block the way it was meant to be used. Her checks confirmed her own understanding of the spec. And her judgment of what was “valid” — nobody programs negative strides across page boundaries — pruned away exactly the region of the input space where the bug lived.

This chapter is about that difference in thinking. Verification is not design performed twice; it is a different intellectual act, with its own posture toward the design, the specification, and reality itself.

CHAPTER MAP

- §2.1 establishes why the designer’s and the verifier’s ways of thinking differ structurally, not just in emphasis, and why a designer who verifies her own block verifies her interpretation, not the specification.
- §2.2 develops thinking in terms of breakage: stressing a design instead of merely interoperating with it, and hunting corner cases the specification never mentions.
- §2.3 states what independence of judgment means in practice, and what preserves it even on teams too small for a separate verification group.
- §2.4 establishes systematic doubt — of the design, of the testbench, and of the verifier’s own assumptions — as the verifier’s core professional asset.

2.1 Designer and verifier: two minds, one specification

Start with what each role is trying to do. A designer constructs a component that correctly implements the intended function, with the required performance and, ideally, without defects. A verifier starts from the opposite premise: the bugs are already in there — the job is to find them, by stressing protocols, exploring corner cases, and applying zero tolerance toward inconsistencies [2]. The designer's success is a mechanism that works; the verifier's success is a demonstration of where it doesn't. These are not two intensities of the same activity. They are different orientations toward the same specification, and each produces systematically different decisions — in stimulus, in checking, in what gets questioned and what gets trusted.

That framing is not this book's invention. It is Verilab's, set out in a DVCon paper whose device is to expose the mindset through a series of concrete choices a verification engineer has to make, rather than as an attitude to be adopted — the first of those choices being whether the environment's duty is to replicate the real world, or to stress the design beyond it [2].

The boundary between the roles, note, is porous in practice. In 2024, design engineers spent on average 49% of their time on verification activities [3] — every designer already verifies half the time. In some segments, such as processors, teams field up to five verification engineers per designer [3]. So the distinction this chapter draws is not between job titles. It is between *mindsets*, and a designer doing verification with a design mindset is precisely the situation in the opening scenario. The industry systematically undervalues this: hiring filters on languages, methodologies and tool experience — “knows UVM”, “has experience with assertions” — while ignoring the single most important determinant of verification effectiveness, the mindset of the engineer. Tools can be learned in months; the mindset usually arrives only through years of experience, mentoring, and lessons from failures [2].

Why self-verification is structurally blind

The cleanest account of why the opening scenario had to end the way it did is Bergeron's *reconvergence model*: verification is the reconciliation of two paths that share a common origin — a transformation (say, writing RTL from a specification) and an independent second path back to that origin. What you actually verify is determined by where the two paths reconverge [4].

Now trace the paths when the DMA designer verifies her own block. She read the specification once and formed an interpretation; from that interpretation she wrote the RTL — and from that *same* interpretation she wrote the tests and the expected results. The two paths reconverge not at the specification but at her interpretation of it. If that interpretation is wrong anywhere — say, she read the 4 KB rule as a constraint on *ascending* burst addresses — the verification will never highlight it: it faithfully confirms that the design matches what she believed the spec to mean [4]. The rule constrains address boundaries, not address direction: Arm IHI 0022H.c §A3.4.1 carries a Note giving the reason — the prohibition “prevents a burst from crossing a boundary between two Subordinates” [1]. Figure 2.1 sets the self-verification path beside the one that reconverges at the specification instead.



Figure 2.1 — The reconvergence model drawn for the DMA of this chapter’s opening: the same specification, the same RTL, and two ways of arriving at the tests. In the self-verification half one interpretation feeds both the RTL and the tests, so the comparison can only confirm that interpretation — including where it is wrong. In the independent half a second engineer’s reading of the specification feeds the testbench instead, and the two paths reconverge at the specification itself. One node differs between the two halves; §2.3 adds that the independent interpretation it buys pays off only if the verifier’s judgment is independent too.

Three mechanisms reduce human-introduced error in any process: automation, mistake-proofing (poka-yoke), and redundancy. Design remains too creative an act for the first two, so hardware verification rests on the third — the simplest and the most costly: a *different person* independently interprets the specification and checks the implementation against it. Redundancy is expensive — and verification, which rests on it, consumes about 70% of design effort (see Chapter 1 for the economics). The industry pays willingly, because a respin costs more than the redundancy [4].

Scenario. The crossbar team is short-staffed, and the engineer who wrote the DMA’s AXI backend offers to “help verify” the crossbar, a block she did not design. **Approach.** Accept — this is redundancy done right. She reads the crossbar spec fresh, builds her own expectation of ID-ordering rules, and her checks encode *her* reading, not the crossbar designer’s. **Why.** Independence attaches to the *artifact*, not the person. A designer is often an excellent verifier of someone else’s block: the design skills transfer, and the interpretation is finally independent. What redundancy forbids, as the reconvergence model implies [4], is only the closed loop of the opening scenario — one mind on both paths.

2.2 Thinking in terms of breakage

If the first shift is *who* interprets the spec, the second is *what you are trying to make happen*. The designer’s instinct is to make the block work; the verifier’s discipline is to make it fail. This sounds like a slogan until you watch it change concrete engin-

eering decisions. The line runs between *interoperating* with a design and *stressing* it: every testbench must interoperate at some level, but a testbench built with a design mindset quietly slides into cooperating with the very behavior it should be interrogating [2].

The most instructive case is a paradox of robustness. Consider a verification component — the reusable testbench-side model of an interface, with a driver to stimulate it and a monitor to observe it — for a protocol that requires clock-data recovery. The design mindset says: the protocol specifies CDR, so the monitor should implement CDR, as robustly as possible. The verification mindset notices that the more robust the monitor's recovery algorithm, the *less checking it performs* — a tolerant monitor gracefully adapts to data rates that are actually out of spec, masking the bug it exists to catch. The right answer is the least tolerant observer that the reference clock permits, even though no real receiver would ever be built that way. Verification's aim is not to replicate reality but to verify the design in the most thorough manner possible, “even if it means doing so in spite of reality” [2]. The same trap waits on a 10G Ethernet MAC: a receive monitor whose elastic-buffer model quietly absorbs clock drift beyond what the specification permits will accept every frame — including the ones a compliant link partner would have rejected.

The same logic recurs wherever the testbench could be *helpful*:

- **Handshakes.** A protocol says a receiver NACKs a corrupted transfer. A design-minded verification component dutifully auto-NACKs — and thereby lets a simulation that should have failed keep running, masking an unprovoked error from the DUT (the design under test). The verification-minded component logs an error and fails the test; NACK generation exists only under explicit testcase control for error injection [2].
- **Status outputs.** The I/O subsystem's FIFOs export `full` and `empty` flags, and every test wants to use them to pace stimulus. Use them unchecked and you have the blind leading the blind: the testbench regulating itself by signals that may themselves be buggy. Check the flags first — a white-box assertion (one that checks internal design state rather than only the ports) against the FIFO pointers, or an independent count of items in and out — then rely on them. The same rule bans “snooping” a DUT's internal clock as the testbench's own timebase [2].
- **Error recovery.** A testbench does not need to survive the errors it detects. Once a check fires, downstream checking is likely compromised anyway; the correct default is to stop on first error, like a mousetrap that has caught its bug and is done for that simulation [2]. Engineering testbench recovery paths is effort spent making failure comfortable.

Corner cases: “invalid” is not a stimulus filter

The designer's second reflex — after helpfulness — is validity pruning: *that input can't happen, so don't test it*. The answer is categorical: whether a scenario is valid or invalid is irrelevant; the only question is whether it *can happen*, and if it can, the design's response to it must be verified. Two reported production cases are worth retelling. In one, a designer would dismiss a zero-radius circle as meaningless input —

and in that very production system, software upstream was generating zero-radius circles routinely, unknown to that designer. In the other, a low-power mode bit was tested by the obvious sequence (write 1, check entry; write 0, check exit) and passed until tape-out — but writing 0 to the bit *when it was already 0* turned out to put the device into low-power mode [2]. Nothing in the specification suggests either test. Both bugs are invisible to a mind that tests what the design is *for*, and nearly inevitable finds for a mind that enumerates what the design can be *subjected to*.

Scenario. The inline AES-GCM crypto engine sits on the memory path. Its feature list describes encryption under a loaded key, and the team's shared assumption is that nobody starts an encryption without loading one — which describes intent, not reachability. The register interface permits it. **Approach.** Enumerate what the interface allows rather than what the block is for, and write a test for each: start an operation with an empty or stale key slot; overwrite the key while an operation is in flight; program a length field that disagrees with the number of beats actually streamed; fail the authentication tag on a message whose plaintext the engine has already pushed downstream. **Why.** Every one of these is reachable from the register interface and none appears in any feature list. On a data-mover a permitted-but-unintended input corrupts data and a scoreboard catches it; here the same class of input leaks key material or plaintext. The last case is a security failure rather than a functional one: the engine may compute exactly the right bytes and still have released them to a consumer that should never have seen them. That is a fault in *when* data moved, not in what the data was — and an oracle that compares values is not looking.

Now rerun the opening scenario with this lens. The DMA spec says 1D and 2D transfers, strides, up to 256-beat bursts, and — inherited from the AXI protocol — that no burst may cross a 4 KB address boundary (Arm IHI 0022H.c §A3.4.1) [1]. The designer's test list came from the feature list. The verifier instead interrogates each feature for its breaking points and, crucially, *crosses* them: stride sign $\times$ boundary proximity $\times$ channel concurrency $\times$ burst length. "Negative stride" is on the list not because anyone plans to use it that way but because the register field is signed, so it can happen. The cross with "row lands within one burst of a 4 KB boundary" and "other channel active" is exactly the cell where the legalizer's bug lives — and a constrained-random generator steered by that coverage model visits it as a matter of course, which is how the bug was actually caught.

Worked example — attacking the transfer legalizer. The verifier's artifacts for the midend make the contrast concrete. First, the property: the designer thinks "the legalizer splits at 4 KB", the verifier states the *prohibition* at the output and lets the tool hunt for any way to violate it:

```
// No INCR write burst emitted by the DMA backend may cross a 4 KB page,
// regardless of how the midend legalized the transfer. (FIXED bursts hold
// their address; WRAP bursts stay inside their aligned container.)
// 16-bit arithmetic on purpose: an illegal awsize (>3 on this 64-bit bus)
// must overflow the comparison and fail — not wrap silently and pass.
property p_no_4kb_crossing;
```



```

@(posedge clk) disable iff (!rst_n)
  (awvalid && awready && (awburst == 2'b01)) |->
    (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);

```

Second, the functional-coverage model that encodes the breakage hypothesis. A *covergroup* is SystemVerilog's construct for declaring which situations must be *observed* before coverage can be called closed: each *coverpoint* samples one axis of the space, its *bins* partition that axis into the cases worth counting, and a *cross* demands their combinations. This one would have exposed the directed suite's blind spot as a block of empty bins long before any failure:

```

covergroup cg_legalizer_stress @(posedge desc_accepted);
  cp_stride : coverpoint desc_stride_sign { bins pos = {0}; bins neg = {1}; }
  cp_bnd    : coverpoint bytes_to_4kb_boundary {
    bins at_bnd    = {0};
    bins in_burst  = {[1:2048]}; // crossing reachable in one burst, or
    exactly filled by one
    bins far       = {[2049:4095]}; }
  cp_len    : coverpoint desc_burst_beats {
    bins single = {1}; bins mid = {[2:255]}; bins max_beats = {256}; }
  cp_ch1    : coverpoint ch1_active { bins idle = {0}; bins busy = {1}; }
  x_attack  : cross cp_stride, cp_bnd, cp_len, cp_ch1;
endgroup

```

The property is the zero-tolerance check; the cross is the map of where to apply pressure. Note that even the checker gets the breakage treatment: the assertion's 16-bit arithmetic exists so that an illegal `awsize` overflows the comparison and fails loudly instead of wrapping past the check and passing — doubt applied to the verifier's own code. Neither artifact required more skill than the directed tests did — only a different question: not “does it transfer?” but “how could the split go wrong, and under what interference?”

2.3 Independence of judgment

Redundancy gives you an independent *interpretation*. It only pays off if the verifier also maintains independent *judgment* — the willingness to reach, and defend, conclusions the designer disagrees with. Three disciplines protect it.

Predict from the specification, not from the design. The verifier's reference for correct behavior — the expected data image in the scoreboard, the legal responses in the monitor — must be derived from the spec, never reverse-engineered from what the RTL does. The moment your checker is tuned until the design passes, the reconvergent path has quietly collapsed back onto the implementation, and you are once again verifying the design against itself. The same point holds at planning level: the verification plan must be based on the *intent* of the design, not its implementation — the corner cases that the implementation creates (like our legalizer's split arithmetic) are verified in addition, after the intent-based plan stands [5]. When a test fails, “the testbench must be wrong” and “the design must be wrong” deserve exactly equal initial credence; the spec, not seniority, arbitrates.

Trust no signal you have not checked. Independence has a micro-scale. Every time the testbench consumes a DUT output — a status flag, a ready signal, an interrupt line — without independent checking, it delegates a piece of its judgment back to the object under suspicion [2]. The DMA's per-channel error report is a case in point: a lazy environment uses it as the ground truth for “this transfer failed”, which means a bug that *under-reports* errors also silences the very tests meant to catch it. The independent environment predicts from stimulus which transfers must fail and treats the error report as one more output to be checked.

Speak as the third party in the room. Designers implement at module level; verifiers, building an environment that must exercise and check the whole block, necessarily operate one level of abstraction up. That position makes the verification engineer the natural liaison between the architect who wrote the specification and the designer who implemented it — often the first to detect that the two have diverged, and the one whose effort suffers most from inconsistency and “nice-to-have” late additions. Exercising the role takes a professional willingness to say “no, this is not the way to do it” [2] — to a designer with more years on the block, to a schedule that wants the block closed, sometimes to your own earlier plan.

Scenario. During integration, the DMA designer reviews the verifier's failing seed and replies: “negative stride across a page boundary is a software error; document it as a restriction and move on.” The tape-out review is in two weeks. **Approach.** The verifier does not argue about intent; she asks the operative question — can the programming interface express it? It can: the stride register is signed, and nothing in the frontend rejects the descriptor. So either the RTL is fixed, or the frontend must *detect and reject* the case and a test must prove the rejection. “Software will never do it” is accepted only as a documented, checked restriction — never as an untested assumption. **Why.** Restrictions that live only in someone's head are bugs waiting for a firmware revision. The verifier's judgment — reachable means testable — is precisely what the team pays independence for [2].

2.4 The value of doubt

What ties the mindset together is a disciplined, impersonal doubt. Not cynicism about designers — the doubt points equally at the testbench and at the verifier's own reasoning — but a refusal to accept comfort as evidence.

Doubt the green. A regression at 100% pass is not good news until you know the checks are real. The rule of *no coverage without checking* is the operational form: a coverage bin that was hit while nothing verified the outcome is worse than an empty bin, because it converts ignorance into false confidence. The same goes for the passing-test count as a progress metric: the verification mindset treats testcases as vehicles toward checked coverage closure, nothing more ^[2] — a growing pass rate over an unchecked design measures only the testbench's tolerance. When the DMA suite “all passed” in the opening scenario, the honest reading was: *no check we wrote has fired* — a statement about the checks as much as about the design.

Doubt the quiet. Bugs cluster where scrutiny hasn't been. A block that has never failed a test is either simple, or under-attacked; a verifier's zoom-out instinct is to ask which, and to reallocate effort accordingly. A flash controller that has never failed a data-integrity test invites both readings: its LDPC ECC pipeline may be genuinely solid, or the error-injection campaign may only ever have injected patterns the code was always going to correct. This is the regular self-audit called *zoom-out thinking*: stepping back from the testcase in front of you to re-ask what, on this project, this week, most needs breaking — design hot spots, schedule, what can move post-tapeout — because the answer changes weekly. The companion question deserves to be posted above every desk: *what are we trying to accomplish here?* — asked of every environment feature, every metric, every meeting ^[2].

Doubt your own tools. Verification engineers spend more of their time on debug than on any other activity ^[3], and a good share of what they debug is the testbench itself. Doubt therefore extends to engineering for falsifiability: environments whose messaging is rich enough to be debugged from logfiles (much of a testbench's behavior — zero-time execution, dynamic objects — never appears in waveforms), and interfaces structured so that a waveform shows *who* drove *what* ^[2]. A testbench you cannot efficiently interrogate is one whose verdicts you have no grounds to trust.

Doubt the schedule's promises. A substantial part of the “verification gap” is self-induced: schedule pressure forcing shortcuts, best-known methods skipped as low priority, and engineer productivity varying up to tenfold ^[6]. Put bluntly: every team signs up for first-time success, but success undefined is unfalsifiable — the verification plan is the definition, the specification of what “verified” will mean for this design (see Chapter 5) ^[5]. Doubt, written down and agreed before the pressure arrives, is called a plan.

Scenario. Four weeks from the coverage-closure milestone, the weekly zoom-out review notices what day-to-day work cannot: the neural accelerator has not failed a test in a month, while the DMA bug absorbed everyone's attention. Simple, or under-attacked? **Approach.** A verifier is reallocated to attack the accelerator. Her

first find is an environment defect, not a design one: the accelerator checks cannot be debugged from the logs — messages never say which job or which MAC lane a mismatch belongs to — so earlier failures had been triaged slowly and twice waived as “environment noise”. She fixes the messaging first; the reopened failures then yield a real bug in a low-precision weight mode. When the schedule pushes back, the verification plan — agreed before the crunch — arbitrates: accelerator-versus-DMA memory contention stays pre-tapeout, performance corners move to post-silicon. **Why.** Quiet blocks, opaque environments, and crunch-time triage are where escapes are born. The zoom-out cadence, logfile-first debuggability, and pre/post-tapeout prioritization are exactly the disciplines that keep doubt operative when pressure peaks ^[2].

IN PRACTICE

Real teams rarely enjoy textbook independence, and the mindset survives contact with reality in adapted forms:

- **Small teams cross-verify.** Two designers exchange blocks for verification. Interpretation independence is preserved ^[4]; what needs active management is the social side — reciprocal politeness (“I won’t press on your block if you don’t press on mine”) is the failure mode.
- **The designer is a source, not an oracle.** Good verifiers interview the designer for hot spots — “what were you least sure about?” is the highest bug-yield question in the industry — while keeping pass/fail authority, and the checkers that embody it, on the verification side.
- **Self-verification happens; contain it.** When the DMA designer must verify her own block, teams compensate at the artifact level: the verification plan (the vplan, formally introduced in Chapter 5) and the coverage model are reviewed by someone else against the spec, assertions are written from the spec text rather than from the RTL, and an independent engineer owns the scoreboard’s reference model. The interpretation loop is broken at the checks even if one person writes the stimulus.
- **Mindset is taught, not assumed.** Since design engineers already spend about half their time verifying ^[3], mature teams train the posture explicitly — review checklists that ask “what would break this?”, bug post-mortems that trace escapes to mindset decisions, mentoring pairs across the design/verification line (see Chapter 24).

PITFALLS

1. **Mirroring the design in the testbench.** Symptom: the monitor implements the same clever algorithm as the DUT, and everything passes. Cure: build the least tolerant observer the spec allows; predict from the spec, not the RTL ^[2].

2. **Validity pruning.** Symptom: “that can’t happen” removes stimulus before anyone checks whether it is reachable. Cure: if the interface can express it, drive it; validity claims become checked rejections, not silence ^[2].
3. **Trusting unchecked DUT outputs.** Symptom: tests pace themselves on `full / empty` flags or DUT clocks that no check covers. Cure: check a signal before the environment relies on it ^[2].
4. **Counting tests instead of checked coverage.** Symptom: progress reported as “412 of 415 passing”. Cure: report checked functional coverage; treat testcases as vehicles ^[2].
5. **The helpful testbench.** Symptom: auto-NACKs, silent retries, error recovery paths that keep failing simulations alive. Cure: fail fast and loud; stop on first error by default ^[2].
6. **Independence in title only.** Symptom: the checker is “fixed” until the design passes, with the designer arbitrating. Cure: the spec arbitrates; discrepancies close with a spec clarification, a design fix, or a testbench fix — each recorded as such.

SUMMARY

- Designer and verifier hold different orientations toward the same specification: constructing correct function versus proving where it breaks. The difference is a mindset, not a job title — and mindset, not tool knowledge, is the strongest determinant of verification effectiveness ^[2].
- Verifying your own design reconverges on your interpretation, not on the specification; a misread spec then escapes by construction. Independent interpretation — redundancy — is the mechanism hardware verification is built on ^[4].
- Stress, don’t interoperate: the more tolerant and “realistic” the testbench, the less it checks. Verification may legitimately ignore reality to maximize rigor ^[2].
- “Invalid” is not a stimulus filter. If a scenario is reachable, its outcome must be verified — the bugs that kill projects live outside the intended-use envelope ^[2].
- Judgment must stay independent at every scale: reference models from the spec, no unchecked DUT signal trusted, and the professional duty to say no — with the vplan, agreed in calm weather, as doubt’s written form ^[5].
- Doubt is symmetric: it covers the design, the testbench, the metrics, and the schedule. No coverage without checking; no pass rate without asking what the checks would have caught ^[2].

FURTHER READING

From the corpus. Montesano & Litterick, *Verification Mind Games* ^[2] is this chapter’s backbone and rewards a full read — seven pages of concrete mindset calls. Bergeron’s *Writing Testbenches using SystemVerilog*, Chapter 1 ^[4], develops the reconvergence model and the human factor. The *Verification Methodology Manual*, Chapter 2 opening ^[5], connects mindset to planning discipline. The 2024 Wilson Re-

search Group IC/ASIC report [\[3\]](#) supplies the workforce data behind this chapter's claims, and Narayan & Symons' *I Created the Verification Gap* [\[6\]](#) is a candid practitioner taxonomy of how teams inflict the gap on themselves.

Outside the corpus (not citable here, worth owning): Wile, Goss & Roesner, *Comprehensive Functional Verification* (Morgan Kaufmann, 2005), the broadest treatment of the verification engineer's role; Myers, *The Art of Software Testing* (Wiley, 1979), the original articulation — from software — of testing as a destructive rather than constructive act; Weinberg, *The Psychology of Computer Programming* (1971), on egoless engineering and why independent review works on humans, not just on code.

REFERENCES

1. [S18](#) Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
2. [D21](#) J. Montesano and M. Litterick, "Verification Mind Games: how to think like a verifier," DVCon, 2014.
3. [R2](#) H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
4. [B2](#) J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
5. [B1](#) J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
6. [D16](#) R. Narayan and T. Symons, "I created the Verification Gap," Proc. DVCon U.S., 2015.

3. The Verification Problem

The first question a verification plan for the reference SoC’s DMA engine must answer is what *testing all the cases* costs on a block of two channels and a few registers. The register map answers it. Each channel is programmed through five 32-bit registers — source address, destination address, byte count, and two signed strides for 2D transfers — plus a 16-bit repetition count and a byte of configuration flags; another byte of status comes back. That is 192 bits of architecturally visible state per channel. Two channels: 384 bits. The number of distinct register states is 2 to the 384th power — roughly 10 to the 115th, more configurations than there are atoms in the observable universe; strip the sixteen read-only status bits and you are still left with 2 to the 368th you can program deliberately. And that counts only the *register* values: not the FIFO in the transfer backend, not the timing of back-pressure from the crossbar, not the interleaving of the two channels, not the moment an error response arrives. So the honest answer is: longer than the remaining lifetime of the universe, using every computer ever built. Every competent verification plan is built on top of that answer. This chapter is about why the answer is what it is — and what the discipline does about it.

CHAPTER MAP

- §3.1 establishes why exhaustive verification is not merely expensive but physically impossible, computes a state space, and says what the numbers mean.
- §3.1 also establishes that formal verification changes the shape of the problem but does not abolish it.
- §3.2 establishes why a bug must be *activated*, *propagated*, and *checked* to be found, and how controllability and observability limit the first two.
- §3.3 develops the oracle problem: knowing what the design did is useless without knowing what it *should* have done, and every practical oracle is partial.
- §3.4 makes the intellectual pivot the rest of this book rests on: from pursuing completeness to managing risk — explicitly, measurably, honestly.

3.1 The state-space explosion

A synchronous digital design is a finite-state machine: a set of flip-flops and memory bits (the state), combinational logic that computes the next state from the current state and the inputs, and outputs. Finite means countable — and countable invites a dangerous thought: *if the states are finite, we can visit them all*. The arithmetic kills the thought immediately. Each state bit doubles the number of possible states; a design with n state bits has 2^n of them ^[1]. A deliberately humble example makes the point: even a 256-bit RAM — thirty-two bytes of storage — already yields about 2^{256} possible contents, and exhaustive simulation of all input combinations is “practically

impossible” for designs of any realistic size [2]. The reference SoC’s 512 KB SRAM holds over four million state bits. The core alone, at ~50k gates, carries a few thousand. Exhaustive enumeration is not a matter of budget. It is off the table before the project starts.

The consequence is plain: exhaustive testing of nontrivial designs is generally infeasible, so testing provides at best *probabilistic* assurance that the design is correct [3]. Simulation visits only a small fraction of all the possible configurations of a system, and the discovery of bugs therefore leans heavily on someone’s expertise in choosing *which* few configurations to visit [1]. Bergeron compresses the whole predicament into one sentence you should memorize: simulation can show only the presence of bugs, never prove their absence [4].

It is worth feeling how fast the wall arrives, so let us do the arithmetic properly on the smallest interesting block we have.

Worked example: the state space of one DMA channel

Take a single channel of the reference SoC’s DMA engine — the block a schedule might reasonably call “simple.” Its frontend exposes this per-channel programming interface:

Table 3.1 — The per-channel programming interface of the reference SoC’s DMA engine: each register, its width in bits, and the role it plays in programming or reporting a transfer. The widths rather than the register count are what the worked example prices — they sum to the 192 bits of architecturally visible state a single channel exposes.

Register	Width	Role
SRC_ADDR	32	source base address
DST_ADDR	32	destination base address
NUM_BYTES	32	bytes per row (1D length)
SRC_STRIDE	32	signed source row stride (2D)
DST_STRIDE	32	signed destination row stride (2D)
NUM_REPS	16	row count for 2D transfers
CFG	8	enable, 1D/2D, fixed-address flags, IRQ enables
STATUS	8	busy, done, error code from the AXI backend

That is 192 bits of architecturally visible state: $2^{192} \approx 6.3 \times 10^{57}$ distinct register configurations for *one channel*. Now put a number on “visiting” them. Suppose a block-level simulation can drive the channel into one million new configurations per second — generous by an order of magnitude for RTL simulation of this block — and suppose you run a farm of one thousand such simulations in parallel, forever. That is 10^9 configurations per second. The universe is about 4.4×10^{17} seconds old; a farm running since the Big Bang would have visited 4.4×10^{26} configurations — a fraction of the space around 10^{-31} . Not one percent. Not one billionth. A fraction whose denominator, written out in full, runs to thirty-two digits. Even the cross-product of just the two address registers — $2^{64} \approx 1.8 \times 10^{19}$ pairs — would take that thousand-machine farm almost six centuries.

And the register file is the *shallow* part of the problem:

- **Internal state.** The midend legalizer and the AXI backend hold their own state: a modest 16-entry $\times$ 64-bit datapath FIFO alone contributes 1024 bits — $2^{1024} \approx 10^{308}$ configurations, dwarfing the register space by 250 orders of magnitude. Add pointers, outstanding-transaction counters, and the splitter’s working registers.
- **Two channels.** The canonical DMA bug of this book — the 2D negative-stride mis-legalization from Chapter 1 — manifests *only when channel 1 is also active*. Bugs of this kind live in the product of the two channels’ states: $2^{384} \approx 3.9 \times 10^{115}$.
- **Sequence, not just state.** A state is only meaningful if a legal input history can reach it, and the input history space is larger still. Each cycle, the environment chooses valid/ready handshakes, response codes, and back-pressure on several AXI channels — call it ten binary choices per cycle. A thousand-cycle test is one path out of 2^{10000} . Which states are actually *reachable* is not even known a priori: computing the reachable set is precisely the job formal tools take on, and precisely where they hit their own wall (we return to this below).

The exercise scales with everything the industry does. The state space doubles with every added state bit, so as transistor budgets grow — with some fixed fraction of each new chip spent on state elements — the verification problem grows not linearly with design size but exponentially ^[1]. This is the engine behind the “verification gap” you met in Chapter 1: design capacity compounds, verification capacity does not, and the gap between the complexity of digital systems and our ability to verify them widens ^[1, 5]. A physical number sharpens this: simulation models run roughly a million times slower than the finished chip, so even months of simulation on many machines examine only a few *minutes* of the device’s operating life ^[5]. Read that as an order-of-magnitude claim, because that is what it is: where a simulator lands against silicon depends on design size, on how much of the system is modelled and on how much instrumentation is compiled in, and other chapters, from a different source, quote a far larger ratio. Nothing in the argument turns on which figure you take: on any of them, the campaign samples a vanishing fraction of the part’s operating life. Every hour your phone’s SoC runs, it traverses more machine states than its entire verification campaign did.

Abstraction shrinks the space — and it is still too big

The standard first response to these numbers is correct and instructive: *most of those states are equivalent*. For the legalizer’s control logic, a source address of `0x1000_0004` and `0x2000_0004` behave identically; what matters is the offset within the 4 KB page, the length relative to burst and page boundaries, the sign of the stride. So an expert partitions each input dimension into equivalence classes and considers the cross-product of classes instead of bits. Try it, honestly, for one channel of the DMA:

Table 3.2 — One DMA channel after expert abstraction: each input dimension partitioned into the equivalence classes a skilled verifier would treat as behaving alike, with the number of classes in each. The counts multiply, so the figure to read off the table is their product — about nine million cases, no longer cosmological and still far beyond any hand-written suite.

Dimension	Expert equivalence classes	Count
source offset in 4 KB page	aligned / near-boundary / mid-page...	~16
destination offset in page	same	~16
length	sub-beat, one beat, sub-burst, exact burst, multi-burst, crosses 0/1/2+ pages	~10
stride	positive/negative × smaller/equal/larger than row, zero	~7
repetitions	1, 2, many, max	~4
other channel	idle / active same bank / active other bank / erroring	~4
backend back-pressure	ready delay 0...7 cycles	~8
error response	none / read error / write error / both	~4

The product is about nine million cases — after aggressive, skilled abstraction that already embodies judgment about what “matters” (and is therefore already a place where bugs can hide: every class boundary you draw wrongly is a corner you will never test). Nine million is no longer cosmological, but it is far beyond directed testing: a team writing and maintaining even a thousand hand-crafted tests is exceptional, and Chapter 1’s escape data shows what the untested residue does. This double reduction — from 10^{57} raw states to 10^6 – 10^7 *meaningful* cases to some *sampled and measured* subset of those — is the actual intellectual structure of simulation-based verification. The first step is abstraction; the second is stimulus automation (constrained random, Chapter 9); the third is measurement (coverage, Chapter 6). Name the bargain exactly: since exhaustive simulation is impossible, a sense of *relative* completeness is established through a coverage metric — and defining that metric is “a selective, and thus subjective, process” [2]. Keep the word *subjective* in view. Coverage does not measure how much of the design’s behavior you verified; it measures how much of *what you decided to measure* you verified. The gap between those two is where escapes live.

Doesn't formal verification solve this?

Partly — importantly — and the shape of “partly” matters for the rest of the book. Formal property verification does not sample the state space; it reasons about all of it, using symbolic representations and mathematical proof, so a proven property holds for *every* reachable state and input sequence, not the visited ones [3]. Model checking is, in essence, an exhaustive state-space search made tractable by clever data structures — and its worst-case cost is exponential in the number of storage elements: the *state explosion problem* is the same wall wearing formal clothes [2]. In practice this restricts full proofs to small or well-abstracted targets — arbitration, interface protocols, control kernels — while simulation remains the mainstay of verification for large designs [2]. A directory-based coherent hub shows what the word *abstracted* is doing there: the MESI state of every line in every cache is a product space no tool will enumerate, so the proof runs against a model cut down to two caches and one address — small enough to converge, and still large enough to contain the livelock and starvation cases that make coherence worth proving. Bounded proofs, which explore only up to a fixed number of cycles, are honest about this: the property is guaranteed only within the bound [6]. And one experienced practitioner considers it self-evident that formal verification cannot fully investigate all the functionality of a real circuit [5].

There is a rigorous discipline — Bormann’s *complete verification* — that pushes further: a completeness checker proves that a set of operation properties leaves no verification gaps, and industrial projects using it reported that almost all such modules functioned correctly after fabrication, the few overlooked faults being traced to misapplication of the methodology [5]. It is the strongest counterexample to “completeness is impossible,” and Chapter 14 treats it with respect. But note what it took: a different way of writing specifications, a dedicated methodology, and effort concentrated on selected modules. Completeness at that standard is a *purchasable* property for critical blocks, not an ambient property of chip projects. For the SoC as a whole — hardware, firmware, three clock domains, analog at the edges — the verification problem stands.

The industrial provenance of that discipline is specific, and so is its limit. Bormann reports complete verification in regular productive use at Siemens and at Infineon from 1999, on a project list whose only named part is the Infineon TriCore2 automotive processor; almost all the circuits verified that way worked after fabrication, and in the few projects where a fault was overlooked the escape was traced to human failure in applying the methodology [5]. Read the qualifier as part of the result, not as a footnote to it.

So the state space tells us exhaustiveness is out. The next two sections sharpen the problem: even for the states you *do* visit, finding a bug requires reaching it, seeing it, and recognizing it — three separate obstacles with three separate failure modes.

3.2 Controllability and observability

A bug is not found when it exists; it is found when three things happen in sequence. First the stimulus must drive the design into the buggy condition — the bug is *activated*. Then the wrong value must travel from where it was born to somewhere your environment can see it — it *propagates*. Finally, something at that point must know the value is wrong — it is *checked*. Break any link and the bug survives the test that technically “executed” it. Simulation logs full of passing tests routinely contain activated bugs that propagated nowhere, and propagated errors that nothing checked.

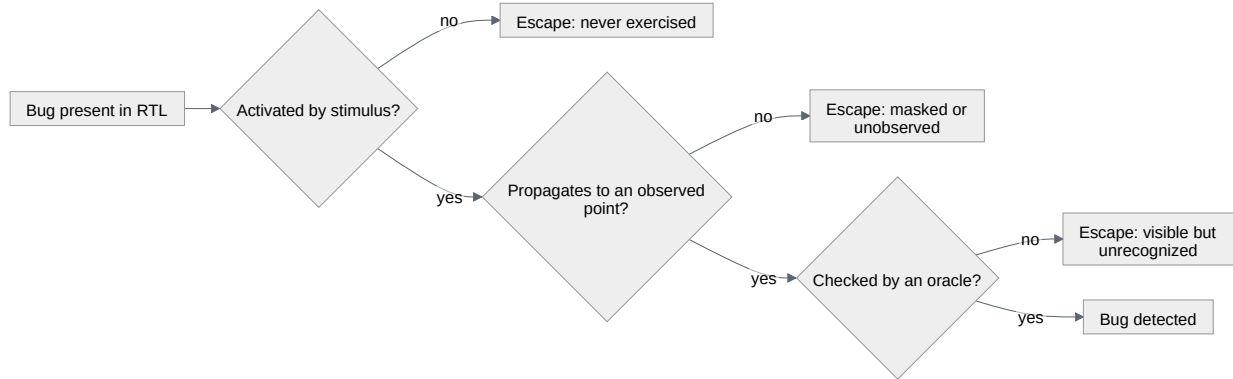


Figure 3.1 — The three-link chain a bug must survive to be found — activated by stimulus, propagated to an observed point, checked by an oracle — with the escape that follows from breaking each link. The three escapes are distinct failure modes rather than degrees of one; §3.3 takes up the third, where the wrong value is visible and nothing recognizes it as wrong.

The first two links have classical names. **Controllability** is your ability to steer the design into a given internal condition using only the interfaces you can drive. **Observability** is your ability to see the consequences of an internal event at the interfaces you can watch. Both degrade with distance: pure black-box verification — driving and observing a design only through its external interfaces, with no knowledge of the implementation — “suffers from an obvious lack of visibility and controllability”: it is often difficult to set up an interesting state combination or to isolate a piece of functionality, and equally difficult to observe the internal response [4]. This is not an argument against black-box testing (which has the virtue of checking the design against its specification rather than against its own structure); it is a statement of its cost.

The single most consequential trade-off in testbench strategy follows directly: **smaller verification targets offer greater controllability and observability** [4]. At the block level, the DMA legalizer’s inputs are your testbench’s outputs — you can present any descriptor on any cycle, stall any handshake, inject any error response. Wrapped inside the SoC, the same legalizer can only be reached through the core executing driver code through the crossbar under arbitration; most of its input space is no longer *reachable by you* in any direct sense. Choosing levels of verification — what you verify standalone, what you verify only integrated — is a planning decision: Chapter 5 assigns every plan row to a level, and Chapter 8 builds the environments that serve them. But the criterion is already clear: a feature must be verified at a level where you can both provoke it and see it fail [4].

Scenario. The reference SoC’s core has a custom SIMD extension. A bug corrupts the saturation flag — but only when an interrupt is taken in the same cycle the SIMD instruction writes it (a canonical bug of this book). **Approach.** The core team verifies this at the core level with ISS co-simulation plus *random* interrupt injection: the testbench fires IRQs at random cycles for thousands of runs, and the co-simulation checker compares architectural state instruction by instruction. **Why.** This is a controllability problem: the buggy condition is a one-cycle coincidence between two independently-timed events. At SoC level, aligning a timer interrupt with one specific instruction inside a firmware loop is nearly impossible to arrange deliberately and worse to reproduce. At core level with direct IRQ lines, randomization turns the coincidence from a lottery into a statistical certainty — and the co-sim oracle guarantees that when the flag corrupts, it is *checked* the same cycle it commits. (Compare Chapter 9 on why random stimulus finds coincidences humans do not imagine.)

Observability failures are quieter and therefore more dangerous. Consider two more of the book’s canonical bugs. The UART’s RX overrun flag is never cleared if software reads the status register in the same cycle a stop-bit error arrives: for months the condition was *activated* in regressions, but no checker ever looked at the overrun flag after that sequence — visible state, never checked. And the event unit’s retention register loses its configuration on power-domain re-entry when a wake event races isolation release: in plain RTL simulation the power-domain behavior wasn’t modeled at all, so the corruption had *nothing to propagate through* — it escaped RTL simulation entirely and was caught only by power-aware gate-level simulation (Chapter 19). Same chain, different broken links.

When a link is structurally weak, competent teams change the design rather than heroically working around it. The grey-box prescriptions are standard practice: if observability is the problem, add sample points readable through memory-mapped registers; if controllability is the problem, add provisions to force error and exception conditions that are otherwise prohibitively slow to provoke [4]. On the reference SoC, the DMA’s per-channel error code in `STATUS` and the event unit’s readback path exist precisely because someone asked, at design time, “how will verification see this?” A NAND flash controller carries an error-injection register into its LDPC ECC pipeline for the same reason: the multi-bit error patterns that exercise the uncorrectable

path cannot be produced from the flash interface at any rate a regression can wait for. Assertions serve the same end from inside: bound into the RTL, they monitor internal signals and catch violations locally, which is exactly why assertion-based verification is credited with improving observability and debugging ^[2] — the error is flagged at its birthplace, not after a thousand cycles of masking and mutation.

Finally, understand that the controllability/observability budget you enjoy in simulation is the best you will ever have. In an RTL simulator, virtually any internal signal is observable at any time; move the same design to FPGA prototyping and observability shrinks to a few thousand pre-selected signals, chosen before the bitstream is built; in actual silicon, you can observe on the order of a hundred signals routed to trace buffers and pins, with controllability limited to whatever configuration hooks the architecture defined in advance ^[7]. These limitations are one of the key factors distinguishing post-silicon validation from everything pre-silicon ^[7]. Every bug you fail to catch in simulation will have to be caught somewhere it is orders of magnitude harder to see (see Chapter 20). This asymmetry, as much as mask costs, is the economic argument for pre-silicon rigor.

3.3 The oracle problem

Suppose controllability and observability are solved: the stimulus reaches the corner, the response emerges at a monitored interface. A harder question remains, so ordinary that its difficulty goes unnoticed: **how do you know the response is right?** A simulator produces what the design *did*. Correctness is a relation between what it did and what it *should have done* — and “should” lives in a specification written in prose, in the architect’s head, or nowhere. The mechanism that pronounces pass or fail is called the **oracle**, and building one is routinely harder than building the stimulus. The asymmetry is stark: deciding how to apply stimulus is relatively easy — you control its timing and content completely — but verifying the response is difficult: you must plan how to *determine* the expected response, and then how to check the design against it ^[4].

The word “plan” is doing real work there. A testbench without a programmed oracle is a waveform generator; it finds bugs only if a human stares at the output, which does not scale past the first afternoon and vanishes entirely in overnight regressions. Everything in this book assumes **self-checking** environments: the verdict is computed, not eyeballed. The practical question becomes *which* oracle, and the honest answer is that every practical oracle is partial — it checks a projection of correctness, not correctness itself. The craft is in combining partial oracles so their blind spots do not overlap.

Reference models are the most complete oracle: an independent executable of the specification runs on the same stimulus, and outputs are compared continuously ^[4]. The catch is that reference models rarely exist — except where the specification is an executable model, which is the typical situation for processors, where the model is golden by definition ^[4]. The reference SoC’s core is verified exactly this way: an instruction-set simulator (ISS) — a fast software model of the architecture — executes

in lockstep with the RTL, and every committed instruction’s architectural state is compared. Note the projection: the ISS defines *architectural* correctness. A bug that corrupts no architectural state — a performance bug, a speculative access that violates a protocol, a security leak through timing — is invisible to it by construction. A video codec is the other classic case of a definitional oracle: the standard ships a bit-exact reference decoder, so the expected pixels are not a matter of opinion — and the projection is just as narrow, since bit-exact output says nothing about rate control, about latency, or about what the encoder does when its line buffer back-pressures.

Scoreboards are the workhorse where no reference model exists: for data-moving designs, the testbench captures each input transaction, applies the specified *transform* (for the DMA: legalize, split, re-address), stores the expected output transactions in queues, and compares actual outputs against them as they emerge, checking at end of test that nothing expected was lost [4]. The projection here: a scoreboard proves data integrity and completeness, but it is typically blind to *when* things happened (latency, throughput, starvation — an arbiter can be grossly unfair past a data-integrity scoreboard [4]) and blind to *how* (a protocol violation that still delivers correct data sails through).

Assertions are oracles for local rules: properties bound inside the design that check invariants and protocol obligations continuously, every cycle, on every test that runs [2, 4]. They are the sharpest partial oracle — instant detection, at the point of origin — over the narrowest scope: each assertion checks one rule.

Offline and human comparison survives at the edges: some responses are easy for a human and awkward for a program (a self-checking testbench verifies pixels well, but a human recognizes “a filled red circle” instantly), and some flows dump outputs for post-simulation comparison against reference files [4]. Use sparingly: verdicts that arrive after simulation ends arrive far from the state that caused them, and the rule is to detect errors as early as possible, while the model is still near the failing state, because that is when diagnosis is cheap [4].

Worked example: partial oracles on the DMA backend

One rule from the AXI protocol: no burst may cross a 4 KB address boundary (Arm IHI 0022H.c §A3.4.1 [8], whose words are “A burst must not cross a 4KB address boundary.”) — the rule at the heart of this book’s canonical DMA bug. As an assertion bound into the DMA’s AXI write channel — the same property Chapter 2 arrived at, qualifiers and all:

```
// No INCR write burst emitted by the DMA backend may cross a 4 KB page,
// regardless of how the midend legalized the transfer. (FIXED bursts hold
// their address; WRAP bursts stay inside their aligned container.)
// 16-bit arithmetic on purpose: an illegal awsize (>3 on this 64-bit bus)
// must overflow the comparison and fail – not wrap silently and pass.
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && awready && (awburst == 2'b01)) |->
      (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
```

```
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

Now ask, deliberately, what this oracle does and does not establish:

Table 3.3 — What the 4 KB-crossing assertion of the worked example establishes about the DMA backend and what it leaves untouched, one question per row answered for that single property. Only the first row is a yes, and the vacuity row is the one to dwell on: a backend that hangs forever satisfies the property without ever exercising it.

Question	Does <code>a_no_4kb_crossing</code> answer it?
Did any issued burst cross a 4 KB boundary?	Yes — every INCR burst, every cycle, every test; FIXED and WRAP bursts cannot cross by construction.
Did the split bursts carry the right data?	No — it never looks at data.
Do the split bursts add up to the programmed transfer?	No — it sees bursts one at a time, not their sum.
Does a legal transfer ever complete at all?	No — a backend that hangs forever satisfies it <i>vacuously</i> : the implication never triggers, so the property never fails.
Was the <i>read</i> channel also legal?	No — separate assertion needed.

The canonical bug — negative-stride 2D rows mis-split at the boundary — illustrates the division of labor. If the legalizer emits a burst that *crosses* the boundary, the assertion fires on the spot. But in the actual bug the emitted bursts were individually legal and the *data* landed wrong: only the scoreboard’s byte-accurate memory model, comparing final memory contents against the transform of the programmed descriptor, could catch it. Assertion and scoreboard each covered the other’s blind spot. That is the pattern to internalize: not “which oracle is best,” but “which projection of correctness is each oracle responsible for, and is any projection unowned?”

One failure mode deserves its own warning because it defeats *all* oracles at once: the **common-mode error**. Every oracle encodes someone’s reading of the specification. If the scoreboard author misreads the same ambiguous sentence the designer misread — is the stride applied before or after the first row? — model and design agree, the test passes, and the bug is structurally invisible to the environment. This is not a rare pathology: specification issues (changing, incorrect, incomplete specs) are among the leading root causes of the functional flaws that force respins [9]. The defenses are organizational, not technical: independent interpretation (the verifier reads the spec, not the RTL — Chapter 2), review of the oracle itself, and treating every model/design disagreement as a specification question first (whose resolution belongs in the spec, not just in the code). A passing regression tells you the design and the testbench agree; it is silent on whether either of them is right.

3.4 From exhaustive to risk-driven verification

Assemble the chapter’s three results. You cannot visit all the states — the space is finite yet unvisitable, astronomically beyond the reach of any farm you can build. You cannot reach or see everything even in the fragment you do visit — controllability

and observability are bounded and shrink at every stage toward silicon. And you cannot fully check even what you see — every oracle is a partial projection of a specification that is itself imperfect. Only one conclusion is available, and it should be drawn without flinching: verification is a process that is never truly complete; it can show the presence of errors, not their absence; and given enough time, an error *will* be found. The question thus becomes: **is the error likely to be severe enough to warrant the effort spent identifying it?** [4]. That sentence is the intellectual pivot of this book. The moment you accept it, verification stops being a doomed pursuit of completeness and becomes something achievable and respectable: the engineering of *known, bounded, deliberately accepted risk*.

The governing analogy is statistical hypothesis testing [4], and it is worth taking seriously rather than decoratively. You cannot prove the hypothesis “my design is correct”; you can only subject it to trials capable of falsifying it, and calibrate your confidence to the power of those trials. The formal side says the same thing: the weakness of all observation-based quality assurance is that errors are found only with certain *probabilities* [5]. Two consequences follow, and they organize everything the industry does.

First: effort must go where risk is, not where habit is. Risk is the product of two estimates — how likely a bug is to exist, and how much its escape would cost. Likelihood concentrates in novelty and intersections: new logic over silicon-proven reuse, corner-dense arithmetic over regular datapath, cross-domain seams (clock, power, block boundaries — where, in practice, teams observe bugs clustering) over interior logic, volatile spec areas over frozen ones. Cost concentrates where escapes are expensive to find and fix downstream: unworkarounds, safety paths, anything whose post-silicon observability is near zero [7]. Verification effort is monstrously expensive — roughly 70 % of ASIC R&D cost by the classic “verification gap” complaint, as stated in 2009 [5], with verification headcount growing at 12.5 % CAGR against 3.7 % for design across 2007–2014 [10] — and the 14 % first-silicon success rate of 2024 [9] shows that spending *more* is not, by itself, working. The discipline’s answer is to spend *aimed*: rank the features by risk, and let the ranking drive depth, technique, and order. That ranking, made explicit, reviewable, and traceable, is precisely the verification plan (Chapter 5).

Second: techniques must be matched to the shape of each risk. The tools you will meet in Parts III–V are not competitors; they are partial answers with different profiles along this chapter’s three axes. Random simulation covers the easy-to-reach majority of scenarios quickly but leaves hard-to-activate corners — and it is exactly those corners that root-cause the industry’s bug escapes [11]. Directed tests reach a known corner precisely but cost engineer-days each, which is why their automated generation is a research field of its own [11]. Formal proof is exhaustive with respect to the properties you write, on the blocks where it converges [2, 3]. Each buys down a different slice of risk; the plan assembles the portfolio.

Scenario. The verification lead of the reference SoC allocates the project’s finite effort across the chip. **Approach.** The crossbar’s ordering rules go to formal prop-

erty verification: control-dominated, protocol-defined, and a write-interleaving bug would be a chip-killer with no workaround — formal’s per-property exhaustiveness is worth the setup cost on a block that size [2]. The DMA gets a constrained-random environment with a byte-accurate scoreboard plus protocol assertions — its risk lives in descriptor corner cases no one can enumerate by hand. The core gets ISS co-simulation with random interrupt injection, because its oracle problem (every instruction, every architectural register) is only tractable against an executable golden model [4]. The UART — reused, silicon-proven — gets a thin directed suite, including one test written from a firmware bug report about the overrun flag. The event unit’s retention behavior is explicitly deferred to power-aware gate-level simulation, and that deferral is *written in the plan* as an accepted RTL-simulation gap. **Why.** Every allocation is a risk decision: highest depth where novelty $\times$ escape-cost is highest, reuse credit where silicon history exists, and every known blind spot named, owned, and scheduled rather than silently hoped away.

The pivot also redefines what “done” means, which is the subject of Part II. If completeness is impossible, *done* cannot mean “verified everything”; it means “reached the agreed evidence threshold on every planned item, and the risks we chose not to retire are enumerated and accepted by people entitled to accept them.” Coverage supplies the evidence — remembering that a coverage model is a selective, subjective definition of relative completeness [2], so the model itself must be reviewed, not just filled. Sign-off (Chapter 7) is then a risk-acceptance decision, not a certificate of correctness. You are never finished; you stop, on purpose, in the open. Teams that pretend otherwise still stop — the tape-out date sees to that — but they stop with their residual risk unexamined, which is how 86 % of projects met their respin in 2024 [9].

IN PRACTICE

Real projects live the pivot imperfectly. The most common failure is not rejecting risk-driven verification but doing it *implicitly*: nobody ranks anything, so the schedule does the ranking — whatever was verified when the tape-out date arrived was, retroactively, the “high-priority” scope. Mature teams differ less in tooling than in explicitness: they keep a risk register next to the vplan, re-rank when the spec changes, and when schedule pressure forces cuts they cut *named items in review* — converting “we ran out of time” into “we accepted risk X, signed by Y.” Oracle quality gets the least attention in practice: teams review RTL rigorously and merge scoreboard code unreviewed, though a checker bug silently converts an entire regression into decoration. And nearly every team over-trusts activation: regressions are routinely credited with “covering” scenarios whose results nothing checked. A blunt weekly question — “*if this feature broke today, which check would fail?*” — exposes more verification holes than most coverage reports.

PITFALLS

1. **Promising completeness.** The plan says “fully verified” and management hears “no bugs possible.” Cure: report coverage of a *stated model* plus a list of exclusions; never let the word “complete” appear without its object [2].
2. **Counting passing tests as progress.** Ten thousand green tests with weak oracles measure agreement, not correctness. Cure: track coverage and bug-discovery curves, and audit what each test actually checks.
3. **Activation without a checker.** A scenario is marked covered because stimulus reached it, but no oracle owned it. Cure: every vplan item names its oracle — the check, not just the stimulus (Chapter 5).
4. **Verifying at the wrong level.** A block corner is deferred to SoC tests where it is uncontrollable and unobservable. Cure: choose the verification level per feature from controllability/observability analysis, not from schedule convenience [4].
5. **Oracle derived from the design.** The model was written by watching RTL waveforms, so it faithfully reproduces the design’s misunderstandings. Cure: derive oracles from the specification independently; treat every mismatch as a spec question first [9].
6. **Uniform effort.** Every block gets the same depth regardless of novelty or escape cost, over-verifying reused IP while the new accelerator starves. Cure: explicit risk ranking drives the budget, and the ranking is reviewed like code.

SUMMARY

- The state space of even a trivial block is beyond enumeration — one DMA channel’s 192 bits of architecturally visible state give $\sim 10^{57}$ configurations; simulation since the Big Bang would visit a $\sim 10^{-31}$ fraction [1, 2].
- Simulation can show the presence of bugs, never their absence; testing yields probabilistic assurance, not proof [3, 4].
- Finding a bug requires activation, propagation, and checking; controllability and observability bound the first two and both collapse progressively from simulation to FPGA to silicon [4, 7].
- Every practical oracle — reference model, scoreboard, assertion — checks a projection of correctness; combine them so blind spots do not overlap, and guard against common-mode spec misreadings [2, 4, 9].
- Formal verification is exhaustive per property, not per design: it relocates the wall (state explosion, bounded proofs, chosen properties) rather than removing it [2, 5, 6].
- The pivot: replace “have we verified everything?” (always no) with “is the residual risk known, bounded, and accepted?” — effort allocated by likelihood $\times$ escape cost [4].
- “Done” is a risk-acceptance decision made in the open; coverage is its evidence and the plan is its contract (Chapters 5–7).

FURTHER READING

From the corpus. Bertacco’s opening chapters remain the clearest short account of why scalability is *the* verification problem ^[1]. Kern & Greenstreet’s survey is the standard map of formal verification’s foundations and limits ^[3]. Yuan’s introduction walks the whole technique landscape — simulation, coverage, model checking, symbolic methods — with unusual candor about each one’s wall ^[2]. Bergeron’s early chapters develop black/white/grey-box verification, levels of verification, and self-checking strategy in depth ^[4]. Bormann’s dissertation is the serious counterpoint: how far “complete verification” can actually be pushed, and at what methodological price ^[5]; Schwarz shows the same formal machinery applied across the hardware/firmware boundary ^[6]. Jayasena & Mishra survey automated directed-test generation — the modern response to the corner-case residue ^[11] — and Mishra’s post-silicon tutorial quantifies how observability decays after tape-out ^[7].

Not in the corpus (cited here on reputation, not verified for this book): Wile, Goss & Roesner, *Comprehensive Functional Verification* (Morgan Kaufmann, 2005), chapter 1, for an industrial framing of the verification problem and the “verification cycle”; Meyer, *Principles of Functional Verification* (Newnes, 2003), for an accessible treatment of self-checking and the limits of simulation.

REFERENCES

1. **B3** V. Bertacco, "Scalable Hardware Verification with Symbolic Simulation," Springer, 2006.
2. **B6** J. Yuan, C. Pixley, and A. Aziz, "Constraint-Based Verification," Springer, 2006.
3. **P20** C. Kern and M. R. Greenstreet, "Formal Verification in Hardware Design: A Survey," ACM Transactions on Design Automation of Electronic Systems, vol. 4, no. 2, pp. 123-193, 1999.
4. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
5. **T2** J. Bormann, "Complete Functional Verification," Ph.D. dissertation, University of Kaiserslautern, Germany, 2009 (English translation of the German dissertation 'Vollständige funktionale Verifikation', April 2009).
6. **T1** M. Schwarz, "Formal Hardware/Firmware Co-Verification of Optimized Embedded Systems," Ph.D. dissertation (Dr.-Ing.), Technische Universität Kaiserslautern, Germany, 2021.
7. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
8. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
9. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
10. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," Proc. 52nd Design Automation Conference (DAC '15), San Francisco, CA, 2015.
11. **P10** A. Jayasena and P. Mishra, "Directed Test Generation for Hardware Validation: A Survey," ACM Computing Surveys, vol. 56, no. 5, Article 132, 2024.

4. The Verification Lifecycle

A verification program must state, formally and repeatedly, how much of its work is done and when the rest will be finished. In the fourth month of the reference SoC project, three of its four teams answer that question in terms that cannot be checked: the core team reports being about 80% done, unchanged from the previous month; the DMA team's is a count of tests running; the I/O team's is a delivery date of *soon*. None of the three names a quantity measured against a plan, so none of them is checkable, and none constrains the schedule built around them. The crossbar's verification team answers in terms that can be checked: the crossbar is at stage 2 of 3, the environment runs all five managers against all four subordinates, nightly regression is at 340 seeds, functional coverage is at 71% against a reviewed plan, and the bug-open curve peaked two weeks ago. Stage 3 entry — coverage closure and the sign-off review — is gated on the two checklist lines still open: two ordering properties proven only up to a bound, and the starvation metric not yet wired into the dashboard. At the current closure rate, the sign-off review is six weeks out.

The difference is not talent or tooling. It is that the crossbar team runs verification as a *lifecycle*: a sequence of phases with defined entry and exit criteria, measurable maturity stages, scheduled reviews with named owners, and a plan that connects the first feature list to the final sign-off signature. This chapter walks that entire cycle — using the crossbar itself as the running example, from its feature list in week one to its sign-off review — and then confronts the uncomfortable industry data about what schedules actually do.

CHAPTER MAP

- §4.1 sets out the phases and milestones of a verification lifecycle, and how the verification cycle interlocks with the design cycle it runs beside.
- §4.2 establishes planning as the highest-leverage phase, and what a plan must produce for the rest of the cycle to be manageable.
- §4.3 develops the phases that follow planning: bring-up, where progress is measured in working demonstrations rather than in infrastructure, and the long middle phase, where regression, coverage and metrics over time become the instruments of management.
- §4.4 develops a three-stage maturity model (bring-up → mature → verified) that gives teams a shared, checkable language for how done a block is.
- §4.5 establishes what closure must converge before a block can be called done — coverage against the plan, code coverage analyzed, sustained regression health, bug metrics — and what the sign-off review adds that no metric can: a judgment, and a record of who accepted the residual risk.
- §4.6 states who reviews what, and when: the review calendar that keeps human judgment in the loop.

- §4.7 treats schedule reality: what industry data says about where effort goes, why most projects run late, and how to plan against the crunch instead of into it.

4.1 The shape of the lifecycle

Verification is a process, not a pile of testbenches [1]. Like any process it has a shape: phases that follow one another, milestones that gate the transitions, and feedback loops that send you backward when reality disagrees with the plan. The shape is remarkably stable across companies and design domains, even when the vocabulary differs; Figure 4.1 draws it in the terms this book will use.



Figure 4.1 — The shape of a verification lifecycle: six phases from the specification to sign-off, and the three feedback loops that send work backwards — bugs and specification changes into planning, coverage holes from closure back into the regression phase, and escapes found after sign-off into the plan of the next project. The loops matter as much as the phases, and a cycle of this shape runs once per verification level rather than once per chip: the crossbar’s own cycle ends at the sign-off review of §4.5, and its result is what the SoC-level cycle consumes.

Six phases, three loops:

1. **Plan.** Extract what must be verified from the specification, decide how, estimate the effort, and get the result reviewed. The verification plan can start as soon as the architectural specification is complete — it does not wait for RTL — and is augmented later with implementation-specific items once the micro-architecture exists [1]. Ideally the implementation side of the plan is finished before RTL coding begins and before any testbench code is written [2].
2. **Build.** Construct the verification environment: interfaces, drivers, monitors, scoreboard, coverage infrastructure. This overlaps heavily with RTL development — the environment should be ready to receive the first RTL drop, not start when it arrives.
3. **Bring-up.** Make the environment and the design talk: first transaction in, first response checked, first smoke tests passing in a repeatable, self-checking way.
4. **Regress and cover.** The long middle. Constrained-random stimulus ramps up, the coverage model fills in, regressions run at a steady cadence, and the bug curve rises, peaks, and — you hope — falls.
5. **Close.** Convert “we ran a lot” into “we covered what we planned”: chase coverage holes, disposition every open bug, justify every exclusion.
6. **Sign-off.** A formal review in which the team presents the evidence against the plan and the organization accepts the residual risk. Sign-off is a decision, not a discovery (see Chapter 3): you never prove the design correct, you decide the evidence is sufficient.

The loops matter as much as the phases. A bug found in the middle phase may reveal a feature missing from the plan — back to planning. A specification change re-opens the feature list. And an escape found after sign-off (in the lab, or in a customer system) must flow back into the plan of the next project, or you will pay for the same lesson twice.

Milestones, not percentages. Each phase transition should be gated by a milestone that is a *demonstration*, not a state estimate. “Agent coding 80% done” tells you nothing checkable; “a single packet goes through the DUT and the scoreboard checks it” or “register reset-value test passes” is either true or false [3]. Typical lifecycle milestones — the vocabulary varies, the pattern does not: **plan review** (the plan exists and stakeholders signed it), **first transaction** (environment and DUT communicate, self-checking), **feature complete** (all planned stimulus and checkers implemented), **RTL freeze** (the design stops moving except for bug fixes), **coverage closure** (every plan item covered or consciously excluded), **verification sign-off** (the review that gates tape-out).

One lifecycle per level. The cycle above does not run once per chip; it runs once per verification *level*. A design decomposes into physical and logical partitions — blocks, cores, subsystems, the full ASIC, the board — and each level of granularity suits a different verification objective: smaller partitions offer far better controllability and observability, larger ones verify integration and system behavior [1]. So the crossbar runs its own complete lifecycle at block level, while the SoC-level lifecycle consumes its results: block-level plans and their outcomes roll up into the system-level plan, and system verification tracks progress against milestone views built on those per-IP results [4]. Some tests are deliberately planned to port across levels — block to system, simulation to lab — and this must be decided during planning, because portable tests are constrained to the features common to both environments [2].

Scenario. The crossbar — 5 managers (core instruction, core data, two DMA ports, debug) × 4 subordinates (SRAM, boot ROM, APB bridge, neural accelerator), 32-bit address, 64-bit data, ID width 4 — gets its own block-level lifecycle: its own plan, environment, regression, and sign-off review. The SoC-level plan does *not* re-verify AXI protocol compliance on every port; it assumes the block-level sign-off and focuses on integration: address-map consistency, arbitration under real traffic from the actual core and DMA, and contention between the DMA and the neural accelerator on the SRAM banks. **Approach.** The SoC lifecycle gates its own bring-up milestone on the crossbar reaching stage 2 (defined in Section 4.4): no point wiring a full-chip environment around an interconnect that cannot yet route ID-tagged responses correctly. **Why.** Levels exist to put each class of bug where it is cheapest to find. A protocol bug found at block level costs hours; the same bug found in full-chip simulation costs days, because controllability and observability have collapsed [1].

A last structural point: the verification lifecycle and the design lifecycle share one anchor, the specification. The specification is the golden reference — when the test-

bench and the RTL disagree, the specification arbitrates, which is precisely why it must exist before the implementation and be independent of it ^[1]. When you find the specification itself is ambiguous, the lifecycle has surfaced a real project bug: changing, incorrect, and incomplete specifications are consistently reported as a leading concern among verification engineers and project managers, and one of the recurring root-cause categories for the functional flaws that escape to silicon ^[5].

4.2 Planning: the phase that decides all the others

Chapter 5 dissects the verification plan as an artifact — feature extraction, plan structure, traceability. Here we care about planning as a *lifecycle phase*: when it happens, what it must produce for the rest of the cycle to work, and what it costs.

Start with why the plan deserves the effort. The verification effort for a modern design runs 100% to 200% of the RTL design effort ^[1]. No team would spend the design half of that budget without a specification document; the verification plan is exactly that — the specification document of the verification effort ^[1]. And it is the tool that converts the unanswerable question into a manageable one: knowing *where you are* in verification is hard, but the planning process creates the instrument that lets a verification manager estimate the effort and time remaining to the degree of certainty required ^[1].

Planning is staged, like design itself. A design is specified as requirements, then architecture, then implementation; the verification planning process should follow the same refinement — either as separate cross-referenced documents or as successive refinement of one document ^[2]. In lifecycle terms:

- **Verification requirements** — identified as early as possible, ideally while the architectural design is still being carried out, and reviewed as part of the project’s technical assessment reviews ^[2]. Three questions structure them: what the design *does* (the intended function, traceable to the specification), what the design *must not do* (the errors the environment must be able to detect — a checker only finds the failures it was told to look for), and what is explicitly *out of scope* (conditions the design is not expected to survive) ^[2]. The requirements review deliberately includes stakeholders from inside and outside the project team — design, verification, and software engineers — because completeness is precisely the property no single perspective can check ^[2].
- **Environment requirements** — the decisions that are cheap now and expensive later: the response-checking strategy (with random stimulus the expected response must be *computed*, not hard-coded ^[2]), the coverage sampling interfaces, end-of-test conditions (an apparently simple question — “when is a test done?” — that hides real bugs if answered lazily ^[2]), reproducibility mechanics (any simulation must be trivially re-runnable with the same configuration and seed ^[2]), and which tests must port across environments.
- **Implementation plan** — who builds what, in what order, with what estimate.

How far ahead that staging can reach has a documented industrial answer. On IBM’s POWER8 processor, preparation for bring-up began at high-level design: the post-silicon tool team engaged early enough that the design team built the requirements into the chip — hardware irritators, non-functional running modes that stress a unit in ways no application ever would, and checkers embedded in the processor itself — with bare-metal exercisers as the primary test vehicle. IBM counted any bug first found by an operating-system test or by real software as an escape [6].

Two planning practices deserve lifecycle-level emphasis because they directly shape the schedule.

First: audit your reuse and write down your assumptions. A reuse assessment identifies, for every piece of design and verification collateral, whether it exists and to what extent it is reused, modified, or new — because the recurring failure mode is assuming “reuse” means 100% reuse, and ending up with an under-resourced, under-scoped project [7]. A dual-role USB controller is where this bites hardest: the compliance suite and the device-side stack arrive as genuine reuse, while the host-side link training and the U0-U3 power-state transitions are new work that the word “reuse” on a schedule slide quietly hides. Alongside the assessment, every verification assumption (“feature X is verified at SoC level, not here”; “the standard protocol driver is adequate because we won’t implement the custom extension”) must be documented and reviewed face to face with the designers — usually over several review rounds — because unspoken assumptions are how design features fall through the cracks [7].

Second: plan in deliverables with a definition of done. Break the work into deliverables that group everything related to one feature — testbench infrastructure, stimulus, checks, coverage — each with an effort estimate and a *definition of done* that is an action, not a state: “completely coded, committed, running and passing in regression”, “a single packet goes through the DUT”, never “80% coded” [3]. Then review the resulting schedule with stakeholders (will it meet the larger team’s needs? can the design team meet the verification team’s input dates?) and aim for linear progress: evenly sized deliverables, evenly spaced dates, later deliverables building on earlier ones [3]. This is what makes verification status legible from outside the team — at any moment you can answer *what have we verified, what is left, are we on schedule, when will we finish* [3].

Modern practice pushes one step further: the plan should be *executable* — a machine-readable structure whose items link directly to the coverage points, checks, and test results that discharge them, so that progress reporting is automatic and the plan stays alive instead of rotting in a document repository [7]. An executable plan also makes reviews auditable, because review records attach to specific plan objects [7]. Chapter 5 returns to the mechanics; what matters here is the lifecycle consequence — with an executable plan, every later phase (regression, closure, sign-off) reads its progress *from the plan*, and the plan reads its status from the regression database, automatically.

What does this cost? A meaningful verification planning process adds 10–20% to the verification front end and returns a 10–40% saving on the overall verification sched-

ule, plus at least a 20% reduction in implementation and closure effort ^[7]. Treat the exact numbers as a single organization’s reported experience; the direction — planning is the highest-ROI phase of the lifecycle — is consistent with everything this chapter cites.

Worked example — the crossbar’s plan, week one. The verification lead reads the interconnect chapter of the SoC specification and the AXI4 protocol requirements (Arm IHI 0022H.c) ^[8], and produces a first feature list. An excerpt, already in vplan form:

Table 4.1 — An excerpt from the crossbar’s feature list in week one, already in verification-plan form: each row states what must be shown for one sub-feature and, in the last column, the evidence that will be accepted as closing it. That last column is the load-bearing one: naming it in week one is the closure planning that makes §4.5 possible. The final row carries what the team has decided *not* to verify, with the assumptions section reviewed with design as its evidence.

ID	Feature	Sub-feature	What must be shown	Closure evidence
XB-01	Address decode	Legal routing	Every manager reaches every subordinate it is allowed to reach, per the address map	Functional coverage: manager × subordinate cross
XB-02	Address decode	Decode error	Access to an unmapped address returns DECERR without hanging the port	Directed test + error-response coverage
XB-03	ID routing	Response routing	Read data returns to the issuing manager with the issuing ID, under arbitrary interleaving	Scoreboard check + ID coverage
XB-04	Ordering	Same-ID order, no write interleave	Transactions with the same ID complete in issue order; write data beats of one burst never interleave with another burst’s on the same port (write-data interleaving is deprecated in AXI4 and later, and an interconnect that combines writes from different managers must forward the write data in address order — Arm IHI 0022H.c §A5.2.2 ^[8])	Protocol assertions (formal + sim)
XB-05	Arbitration	Fairness under load	No manager is starved when all five managers target one subordinate	Latency metric + starvation assertion
XB-06	Back-pressure	Stall robustness	Correct behavior with arbitrary ready de-assertion on every channel	Constrained-random with stall injection; assertion density on handshakes
—	<i>Out of scope</i>	—	Cache-coherent extensions (not implemented); APB-side peripheral behavior (verified in the peripheral subsystem plan)	Assumptions section, reviewed with design

Each row names its closure evidence *now*, in week one — this is the closure planning that makes Section 4.5 possible: completion criteria and the coverage that correlates to each plan item are defined at planning time, not discovered at the end ^[7]. Alongside the feature list, the lead produces the deliverable schedule (environment skeleton: 2 weeks; single-path bring-up: 1 week; all-paths random with scoreboard: 3

weeks; protocol assertion bind-in: 1 week; coverage model: 2 weeks) and the assumptions list. The whole package goes to the plan review — Section 4.6 says who sits in the room, and what that room adds to this list.

4.3 From bring-up to regression maturity

Bring-up: make it real, then make it right

Bring-up is the phase with the highest anxiety-to-progress ratio: the environment exists, the first RTL drop exists, and nothing works. The discipline that shortens it is the same deliverable thinking from planning — drive toward *demonstrations*. The first milestone is not “environment complete”; it is “one read transaction from the core-data manager to SRAM, driven, monitored, and checked by the scoreboard”. Then one write. Then one transaction per manager-subordinate pair. Each step is a definition-of-done that either passes in a committed, repeatable simulation or does not [3].

Two practices distinguish teams that exit bring-up quickly:

- **Self-checking from the first transaction.** The temptation is to eyeball waveforms for a few weeks and “add checking later”. Resist it. The response-checking strategy was designed during planning [2]; bring it up *with* the stimulus, because an unchecked passing test is a false sense of progress, and the checker itself needs debugging time too. Most bugs found in bring-up are environment bugs — that is the phase doing its job.
- **Regression from the first passing test.** As individual self-checking tests come alive, they go straight onto the master regression list, which runs at a regular interval — nightly is the classic cadence — so that anything the design or environment breaks is caught within a day [1]. A regression suite exists to guarantee backward compatibility: the change that fixes today’s bug routinely breaks yesterday’s verified functionality, and only re-running everything catches it [1].

The classic bring-up failure is architectural big-bang: months of up-front testbench infrastructure work with no running demonstration, which both hides schedule slip-page and makes progress invisible to outsiders [3]. Build the simplest environment that can pass the first deliverable, then refactor under the protection of the regression suite.

Scenario. The crossbar environment comes up in week 3 with one manager agent and one subordinate memory model. The first three failures all look like environment bugs: the driver violates a handshake it was supposed to test, the scoreboard mispredicts response ordering for the boot-ROM port, and the address map in the environment disagrees with the one in the RTL — which turns out to be a *real* discrepancy against the specification, caught because two teams implemented the same table independently. **Approach.** File all three through the same bug-tracking flow, including the environment bugs: errors found at any stage, in the design itself or in the verification environment, are trackable issues [1]. **Why.** The environment

is a design too — in practice often larger than the DUT — and untracked environment bugs are how a testbench silently stops testing what you think it tests. And the address-map discrepancy is bring-up’s first real return on investment: an integration bug found at the cheapest possible moment.

The long middle: regression and coverage maturity

After bring-up, the lifecycle enters its longest phase, and its character changes: from building things to *running and reading* things. Constrained-random stimulus takes over the bulk of stimulus generation; the coverage model — planned in week one — gets implemented and starts reporting; and the regression becomes the project’s heartbeat.

Regression engineering gets its own chapter (see Chapter 12), but the lifecycle-level mechanics are these. The suite grows until it no longer fits in a night, at which point you split it: a basic-functionality subset every night, the full suite over the weekend; random tests re-run with fresh seeds, with seeds ranked by their incremental coverage contribution so the most valuable ones run first [1]. That threshold arrives at very different dates for different IPs — a tile-based GPU’s shader-cluster regression stops fitting in a night long before a peripheral’s does, and its split falls naturally by workload class: short kernels nightly, full frame renders at the weekend. Mature environments run several regressions per day — four a day, five days a week is a documented cadence, each run producing pass/fail plus coverage that is snapshotted into a tracking database, after which the bulky raw results can be discarded [4].

What turns this machinery into *management* is metrics over time. The metrics were chosen during planning — a successful metrics-driven process requires organizing their execution, classification, and reporting in the planning phase, precisely because metrics are expensive to build and maintain and every measurement should have a stated purpose [9]. Now they pay off as trend lines: bug-open and bug-closure rates plotted over time, percentage of tests written, coverage progression, regression runtime per block — the charts that let a lead spot anomalies and answer “are we on schedule? are we converging?” at a glance [4, 9]. Even indirect signals earn their place: how often code is checked in correlates with the stability and maturity of design and testbench [9]. For an interconnect like the crossbar, the plan’s metric set includes bus-specific ones: functional coverage of operation mixes, plus utilization, queuing, back-pressure, and latency [9] — which is how feature XB-05 (starvation) gets measured rather than argued about.

Expect the middle phase to be dominated by debug. Across the industry, verification engineers spend more of their time debugging than on any other single activity [5, 10] — and debug effort is unpredictable and varies enormously between projects, which is exactly why schedules extrapolated from the previous project keep failing [5]. The lifecycle answer is not to schedule debug precisely (you cannot) but to keep the debug loop short: fast regressions, good failure bucketing, reproducible seeds.

The middle phase is also where the plan’s multi-engine choices come home. The crossbar plan assigned feature XB-04 — same-ID ordering and write-interleaving

rules — to protocol assertions run in both simulation and formal, and it is formal that pays off:

```
Bug #217 – crossbar: write-data interleaving at a subordinate port
Found by:    formal protocol assertion (write-interleave rule), 11-cycle trace
Phase:       regression/coverage (week 14); RTL freeze candidate build
Symptom:     with WREADY back-pressure on the SRAM port and two managers
              holding writes in flight to it, the crossbar interleaves the
              write data beats of the two bursts – AXI4 obliges it to forward
              merged write data in address order, one burst at a time.
Root cause:  write-arbiter grant not held for full burst when the
              downstream stalls exactly on the first data beat.
Escape risk: silent data corruption; wrong data written, no error response.
Disposition: RTL fix + directed sim test derived from the trace + the
              assertion stays enabled in every future regression.
```

An 11-cycle counterexample for a bug that constrained-random had not hit in six weeks of nightly regression: this is the sim/formal division of labor working as planned (see Chapters 14 and 16), and it only happened because the *plan* put the ordering rules in formal’s column back in week one. Note also the last line of the report: every bug’s fix enters the regression as a permanent test or assertion — the loop from the lifecycle diagram, closed in practice.

4.4 Maturity stages as a shared language

“How done are you?” is the question the lifecycle must answer continuously, for people who do not live inside the testbench: program managers, the SoC integration team, other block teams that consume your results. Percentages fail at this. “80% done” is uncheckable, coverage percentages without a reviewed model are noise, and outsiders cannot tell healthy coverage asymptotes from wishful ones — the difference between an honest curve and “we’ll suddenly get more productive next week” is invisible from outside [3].

What works — and what has become an industry-standard pattern, visible publicly in the staged checklists that open-source silicon projects publish for every block (OpenTitan is the best-known example) — is a small number of named **maturity stages**, each with a written checklist of *demonstrable* exit criteria. A block is *at* stage N while it works through that stage’s checklist, and *exits* it — entering stage N+1 — when every line is ticked. Any three-stage variant looks essentially like this:

- **Stage 1 — Bring-up complete.** The verification plan exists and has passed review. The environment instantiates the current RTL, drives real transactions, and checks them automatically. A smoke suite passes and runs in regression. In lifecycle terms: the plan and bring-up phases have exited.
- **Stage 2 — Verification mature.** All planned stimulus and checkers are implemented; the functional coverage model is implemented and reporting; regression runs at its target cadence with a managed pass rate; most planned features have been exercised; the bug-open curve has peaked and closure is outpacing discov-

ery. The block is trustworthy enough for others to build on — this is the gate the SoC lifecycle waits for.

- **Stage 3 — Verified / sign-off ready.** Coverage is closed against the plan, with every exclusion written down and reviewed; the full regression has been passing over a sustained window; every open bug has a disposition; the sign-off review has accepted the evidence.

Three properties make the model work as a *language*, rather than yet another status report:

The criteria are demonstrations, one checkbox at a time. Exactly like milestone definitions of done ^[3], each checklist line is either objectively true or objectively false — “coverage model implemented and reporting in nightly regression” rather than “coverage well underway”. A stage claim can therefore be audited in an hour by someone outside the team.

The stages compose upward. Each block reports one stage; the SoC dashboard is a table of blocks × stages; program-level milestone views aggregate per-IP plan results into a single picture ^[4]. When the question from this chapter’s opening is asked, “stage 2, six weeks to sign-off” is a complete, comparable answer — the same words mean the same thing for the crossbar, the DMA, and the core.

The stages tie metrics to process maturity. Using metrics to make a process quantitatively managed is the core idea hardware verification inherited from the Capability Maturity Model tradition in software engineering ^[9]. The stage model is that idea made small and practical: each stage is a bundle of metrics thresholds plus reviewed artifacts, and moving between stages is a measured event, not a feeling.

Worked example — the crossbar’s stage-2 exit checklist (excerpt, week 16 — the program review that opens this chapter).

```
[x] Verification plan reviewed; all review actions closed
[x] All 5x4 manager/subordinate paths driven and checked in regression
[x] Protocol assertion set bound on all 9 ports; enabled in sim and formal
[x] Functional coverage model implemented for XB-01..XB-07; reporting nightly
[x] Nightly regression >= 300 seeds, pass rate >= 95%, no test disabled
[x] Bug-open curve past peak (trailing 3-week closure rate > discovery rate)
[ ] Formal ordering proofs (XB-04): 2 properties still bounded-only -> action
[ ] Starvation metric (XB-05) wired into regression dashboard -> action
```

Two open boxes means the crossbar has *not yet exited* stage 2 — it cannot claim stage 3 entry — and everyone can see exactly why and what remains. That precision is the point. A stage claimed without its checklist is a percentage wearing a costume.

One warning: stages describe *verification maturity*, not RTL quality. A block can be stage 2 with fifty open bugs — the claim is that the machinery to find and track them is mature, not that the design is clean. An HBM controller can report stage 2 honestly while carrying a stack of open bugs against its refresh and thermal corner cases: the training-sequence stimulus, the coverage on those corners, and the nightly regression that keeps surfacing them are exactly what stage 2 claims — and surfacing them is what the machinery is for. Conflating maturity with cleanliness produces the worst kind of dashboard: green process indicators over a red design.

4.5 Closure and sign-off

Closure is where the lifecycle’s early investments are either redeemed or exposed. If the plan named its closure evidence per feature (as the crossbar plan did in Section 4.2), closure is a checklist-driven convergence. If it did not, closure becomes an argument about what “done” should have meant — conducted at the worst possible time, under the worst possible pressure.

The closure phase converges several strands of evidence simultaneously; Chapters 6 and 7 treat each in depth, so here is the lifecycle view:

- **Functional coverage against the plan.** Every plan item reaches its target, or is *excluded with a written reason* that survives review. Honest exclusions are a feature of closure, not a failure — the plan itself declared some conditions out of scope from day one ^[2]; what is forbidden is the silent exclusion.
- **Code coverage analyzed.** Not necessarily 100% — analyzed, with unreached code either explained (unused configuration, defensive logic) or exposed as a stimulus hole.
- **Regression health over a sustained window.** All tests passing, repeatedly, across seeds — not a single lucky green run.
- **Bug metrics.** The open-bug curve at or near zero for the block, the discovery rate flat despite continued stimulus. The classical ship criterion bundles exactly these strands: the design can ship when the RTL passes all the planned tests *and* you are satisfied with the coverage and bug-rate metrics — and not before ^[1].

Then comes the **sign-off review** — the lifecycle’s last gate and its most human one. Its function is not to generate new evidence but to *judge* the existing evidence against the plan, in front of the people who will live with the consequences. The review walks the plan, not the testbench: every feature row, its closure evidence, its exclusions and their reasons; the open-bug dispositions; the assumptions list from planning, re-read line by line, because an assumption that was reasonable in week one (“the custom protocol extension will not be implemented”) may have quietly rotted. An executable plan makes this audit mechanical rather than archaeological — results and review records are already linked to plan objects ^[7].

Worked example — the crossbar sign-off summary (the one-page table the review starts from).

Table 4.2 — The one-page evidence summary the crossbar’s sign-off review starts from, one row per strand: plan features against the plan written in §4.2, functional and code coverage, formal results, regression history, bug dispositions, the assumptions re-read at sign-off, and escapes handed to later levels. What the table admits is the point — three formal properties proven only to a bound, fourteen coverage exclusions, one waived bug and one assumption escalated upward are the residual risk the organization is asked to accept, and a summary showing none of it would be an unexamined design rather than a clean one.

Evidence	Status	Notes
Plan features XB-01..XB-07	7/7 closed	XB-04 closed by formal proof (unbounded) + sim assertions; XB-07 added at plan review, closed by directed test + coverage bin
Functional coverage	100% of model	14 exclusions, all reviewed (list attached)
Code coverage	98.7% stmt / 96.2% branch	Residue: defensive decode default — reviewed
Formal	41 proven, 3 bounded $\geq$ 40 cycles	Bounded set reviewed and accepted as residual risk
Regression	21 consecutive clean nightlies, 5 clean weekend fulls	~9,400 seeds cumulative
Bugs	63 filed / 63 dispositioned	58 fixed+regressed, 4 not-a-bug (spec clarified), 1 waived (doc erratum)
Assumptions	9/9 re-reviewed at sign-off	1 escalated: APB bridge (Arm IHI 0024E ^[11]) timeout behavior — moved to SoC-level plan
Escapes to later levels	1	Address-map bug #003 also existed in SoC integration spec — fed back

Look at what the table admits: three formal properties only bounded, fourteen coverage exclusions, one waived bug, one assumption escalated upward. A sign-off review that shows no residual risk is not a clean design; it is an unexamined one. The review’s real output is a *decision* — the organization accepts this residue — and a record of who accepted it.

And the industry data says you should expect that residue to bite occasionally, no matter how disciplined the closure: in 2024 only 14% of IC/ASIC projects achieved first-silicon success, the lowest value in twenty years ^[5], and in the same year 87% of FPGA projects reported non-trivial bug escapes into production ^[12]. Sign-off is risk management’s final act, not correctness’s. That is not cynicism; it is the reason the lifecycle’s last loop exists — every escape must flow back into the planning of the next block, the next level, the next project.

4.6 Roles and reviews: who checks what, and when

Everything in this chapter so far — plans, stages, closure evidence — is machinery. Reviews are where human judgment inspects the machinery, and they are the part most often skipped under pressure, which is exactly backwards: reviews are cheap, and they catch the class of error no tool can, the error of *intent*.

Two principles govern the cast. First, **the team owns the plan** — an RTL designer’s responsibility is not to write RTL but to produce a working design, so the entire design team contributes to and reviews the verification plan ^[1]. Second, **completeness needs outsiders**: the requirements review deliberately pulls in stakeholders from inside and outside the project team — design, verification, and software engineers together — because each perspective sees holes the others cannot ^[2]. Reviews then recur at prescribed stages across the whole flow — planning, implementation, debug, and closure — with representatives from the design, test, systems, and verification teams ^[7].

The review calendar, in lifecycle order:

Table 4.3 — The review calendar in lifecycle order, from the requirements review held when the architectural specification is complete to the sign-off review that gates tape-out: each row gives when the review happens, who is in the room, and the question that review exists to answer. The third column is the one to read closely — the reviews that judge completeness deliberately seat people from outside the verification team, and the sign-off row seats everyone in the rows above it plus management.

Review	When	Who	What is being judged
Requirements / plan review	Architectural spec complete; part of the project’s technical assessment reviews ^[2]	Design, verification, software, system stakeholders	Is the feature list complete? Are the must-not-do’s and out-of-scopes right?
Assumptions review	During planning; re-run whenever the spec moves	Verification team with the designers, face to face, usually several rounds ^[7]	Every unstated “someone else covers this” made explicit and agreed
Schedule / deliverables review	Plan approved, before build	Verification lead with consuming teams ^[3]	Do the deliverables and dates meet the larger team’s needs? Can design meet verification’s input dates?
Environment and code review	During build and continuously	Verification peers	Is the testbench itself trustworthy? (It is a design too.)
Coverage-model review	Before the long middle	Designers + verification	Does the model measure the <i>intent</i> , not the implementation? (see Chapter 6)
Progress / triage review	Weekly through the middle phase	Lead + team, escalation to program	Trend lines: bug curves, coverage, regression health — on schedule? converging? ^[4]
Coverage-closure review	Entering closure	Verification + design leads	Every hole chased or excluded with a written reason; no silent exclusions
Sign-off review	End of closure, gating tape-out (or block release)	All of the above plus management	The full evidence package against the plan; explicit acceptance of residual risk

The progress review deserves a historical footnote, because it *is* coverage — the oldest form of it. A testcase status table with owner and completion columns, updated by engineers and reviewed at regular meetings to decide whether the project is on

schedule and where resources should move, is a functional coverage model tracked through managerial procedure instead of tooling [2]. Modern dashboards automate the collection; the meeting, and the judgment exercised in it, remain.

Who reviews what is as important as when. Designers review the verification team's checkers and coverage model for *intent* — do these checks encode what the design is supposed to do? — but do not own them; the verifier's independence of judgment (see Chapter 2) is the whole value proposition, and the specification, not the designer, arbitrates when testbench and RTL disagree. Verification peers review each other's environment code, because a testbench bug that silently stops testing manufactures false confidence [1] — which is worse than a design bug, since nothing downstream is looking for it. Leads review trends, not transactions. And the sign-off review is the one room where all three altitudes meet.

Scenario. At the crossbar's plan review, the firmware engineer — invited under protest ("it's an interconnect, what would software know?") — asks how the debug manager port is verified while the core is halted. Silence. The feature list has core-I, core-D, and DMA traffic in every cross, but no scenario where the debug port is the *only* active manager issuing abstract memory accesses into a quiesced system. One row is added to the plan (XB-07), one directed test and one coverage bin come out of it, and in week 16 that test catches an arbiter assumption that would have made post-silicon debug intermittently unreliable — the tool you need most precisely when everything else is broken. **Approach.** Keep the outsiders in the review, especially the ones with no stake in the block's internals. **Why.** In practice the people closest to the design are the least likely to notice what is missing from it — which is why requirements reviews mandate stakeholders from outside the project team [2].

4.7 Schedule reality

Everything above describes the lifecycle as it should run. Here is what the industry data says about how it actually runs — numbers a verification lead needs, because a plan built against optimistic averages is a plan built to fail.

Verification dominates project effort. Figures like "70% of a project's overall effort is spent in verification" have circulated for years [10]; the measured 2014 industry mean was 57% of total project time, with a rising share of projects spending more than 80% [10]. The spread is wide and structural: projects at the low end lean on pre-verified IP reuse, projects at the high end carry high proportions of newly developed IP [5] — which is why the crossbar (configured from a mature in-house pattern) and the neural accelerator (new) sit at opposite ends of the reference SoC's effort table despite comparable gate counts.

Headcount tells the same story. The mean peak number of verification engineers on a project overtook design engineers years ago [10]. Between 2007 and 2014 demand for verification engineers grew at a 12.5% compound annual rate against 3.7% for designers [10]; the growth has since cooled (roughly 3% vs 2.4% between 2016

and 2024), but in some segments — processors — a 5-to-1 ratio of verification to design engineers is not unusual [5]. And the boundary itself is soft: design engineers spend about half their own time on verification — 47% in 2014, 49% in 2024 [5, 10]. Add it up and verification is not *a* task on the critical path; for most of the project it *is* the critical path.

Most projects run late anyway. Around two-thirds of IC/ASIC projects historically finished behind their original schedule — 67% in 2007 and again in 2012, 61% in 2014 [10] — and in 2024 the figure rose to 75% [5]. Meanwhile first-silicon success fell to 14%, the worst in twenty years, with logic and functional flaws still the leading cause of respins [5]. The uncomfortable synthesis: the industry spends more than half its effort on verification, is late doing it, and still ships bugs. This is not an argument that verification fails; it is the measure of how hard the problem from Chapter 3 is in industrial practice.

Where the time actually goes. Within the verification team, debug is the single largest consumer of engineer time — larger than testbench development, test writing, or test planning [5, 10]. Worse for the planner, debug effort is unpredictable and varies significantly from project to project, which quietly invalidates the most natural estimation method: extrapolating from the last project [5]. Effort is also back-loaded. Planning and environment build spend at a modest, steady rate; the long middle burns compute and debug time; and closure concentrates the highest-pressure work — the last coverage holes, the intermittent failures, the hardest bugs — into the weeks when the tape-out date is least movable. That concentration is the **crunch**, and every practitioner recognizes it.

The crunch has a failure mode with a name: the guillotine. When the ship decision arrives on a fixed date and simply cuts off whatever verification was still in flight, what gets shipped is “whatever happened to work” — and the features that fall in the basket may be the ones the market needed [1]. The defense was built back in Section 4.2: a plan that defines first-time success and *prioritizes* features makes the inevitable end-of-project cuts a conscious decision about named risks, instead of an accident of which tests happened to be written first [1].

A useful lens for everything in this section is the **verification gap** — the distance between the verification a project needs and the verification it gets. One practitioner taxonomy decomposes it into contributors (complexity, skills, schedule, quality, metrics, reuse) and, more usefully, into tractability classes: gaps that are *inherent* (complexity you must simply absorb), *transient* (skills that can be developed), and *self-induced* [13]. The self-induced ones are the lifecycle’s business. Aggressive schedules and milestones create pressure that forces shortcuts — “pay me now or pay me later” [13]. Sloppy environment engineering shows up as slow edit-compile-run loops and weak regression management; best-known methods simply not being in use is estimated to cost around 20% of productivity on its own [13]. And individual skill spread is brutal, up to 10× in productivity across engineers [13], which makes team composition a first-order schedule variable that no amount of methodology hides.

So plan against the reality, not the brochure: schedule in demonstrable deliverables with linear progress so slippage is visible in weeks, not quarters [3]; buy schedule

with planning up front, where the leverage is (10-20% front-end cost against a 10-40% return on the verification schedule in one reported experience) [7]; keep the maturity stages honest so nobody discovers at RTL freeze that “80% done” meant stage 1; and protect the regression cadence and debug loop, because that is where the majority of all verification hours will be spent whether you planned for it or not [5, 10].

Scenario. Week 18 of the reference SoC: RTL freeze slips three weeks because the neural accelerator’s datapath is being reworked, and the program manager proposes to “absorb” the slip by compressing verification closure — the tape-out date does not move. **Approach.** The crossbar lead does not argue with adjectives; she brings the plan. The first-time-success list is re-reviewed with the stakeholders: XB-01 through XB-07 stay non-negotiable; the stretch goals (full QoS characterization, exhaustive sparse-ID stress) are explicitly moved out of the sign-off gate and into an erratum-risk register that management signs. The stage checklists do not change. **Why.** Compression under pressure is inevitable; *silent* compression is optional. A prioritized plan converts the guillotine into a negotiation, and the signature on the risk register converts “we ran out of time” into an accountable decision [1, 13].

IN PRACTICE

What real teams do, including the messy parts:

- **The plan is written, reviewed — and then abandoned** in perhaps half of real projects: the document is a snapshot from month one while the design moved on. Teams that avoid this either adopt genuinely executable plans wired to the regression database [7] or enforce a cheap rule: every specification change and every not-a-bug dispute must land as a plan diff, reviewed in the weekly meeting.
- **Stage models get gamed.** Checklists claimed without evidence, “temporary” test disablement to keep the pass rate green, coverage closed by deleting bins. The countermeasure is social, not technical: stage transitions are *reviewed* events with an outsider in the room, and a disabled test is treated as an open bug against the environment.
- **Regression cadence is the first casualty of compute budget fights** — and the most expensive one to lose, since slow feedback multiplies the cost of the debug hours that already dominate the schedule [5, 10]. Mature teams track regression turnaround as a first-class metric and treat a runtime spike like a failing test [9].
- **Sign-off dates are negotiated backwards from tape-out**, not forwards from evidence. Expect it. The realistic goal is not to prevent the negotiation but to walk into it with a prioritized plan and a residual-risk list, so the compression is explicit and signed [1].
- **The review calendar decays under load** — first the coverage-model review is skipped, then assumptions are never re-read. The teams that keep discipline are usually the ones that were burned once: a single escaped assumption (“the other

team covers this”) is exactly how design features fall through the cracks ^[7], and one such gap costs more than every review meeting of the project combined.

PITFALLS

1. **Reporting percent-done.** Symptom: “80% done” three months in a row. Cure: maturity stages with demonstrable checklists; report the stage and the open checklist lines ^[3].
2. **Big-bang environment development.** Symptom: months of infrastructure work, nothing running end to end. Cure: deliverable-driven bring-up; first checked transaction is milestone one ^[3].
3. **Deferring checking.** Symptom: waveform eyeballing “until the env stabilizes”. Cure: self-checking from the first transaction; the response strategy was planned before the first test ^[2].
4. **Assuming reuse means 100%.** Symptom: under-scoped project discovered at bring-up. Cure: a written reuse assessment — exists / reused / modified / new — for every piece of collateral ^[7].
5. **Inventing closure criteria at closure.** Symptom: month-long arguments about what “done” means, at the worst time. Cure: closure evidence named per feature in the plan, in week one ^[7].
6. **Treating sign-off as proof.** Symptom: shock when silicon comes back with a bug despite “100% coverage”. Cure: sign-off is documented risk acceptance — keep the residual-risk list explicit and feed every escape back into the next plan ^[5, 12].

SUMMARY

- The lifecycle is six phases — plan, build, bring-up, regress-and-cover, close, sign-off — with feedback loops, run once per verification level, with block lifecycles rolling up into the SoC’s.
- Milestones and stage transitions must be demonstrations (“this test passes in regression”), never state estimates (“80% coded”) ^[3].
- Planning is the highest-leverage phase: the plan is the specification of an effort that costs 100–200% of RTL design ^[1], and disciplined planning measurably buys back schedule ^[7].
- A three-stage maturity model — bring-up complete, verification mature, verified — with written exit checklists gives every block the same auditable answer to “how done are you?”; open-source silicon projects publish exactly this pattern.
- Reviews are the human redundancy layer: requirements with outsiders, assumptions face-to-face with designers, closure and sign-off against the plan — at prescribed stages from planning to closure ^[2, 7].
- The data is unforgiving: verification consumes over half of project time (57% measured in 2014) ^[10], most projects run late (75% in 2024) ^[5], debug dominates

engineer time and resists estimation [5, 10]. Plan against these numbers, not against optimism.

- Sign-off is risk acceptance, not proof — 14% first-silicon success and 87% FPGA escape rates say the residue is real [5, 12]; make it explicit, signed, and fed back into the next cycle.

FURTHER READING

Corpus sources.

- [2] *Verification Methodology Manual for SystemVerilog*, ch. 2 — the planning process as staged refinement: requirements, environment, implementation; the fullest treatment of what a plan must contain.
- [1] Bergeron, *Writing Testbenches Using SystemVerilog*, ch. 2, ch. 3 and ch. 7 — issue tracking (what counts as a trackable issue); the plan's role, first-time success, levels of verification; simulation and regression management.
- [7] "Best Practices in Verification Planning" — executable plans, reuse assessment, assumption reviews, and the cost/benefit data quoted in this chapter.
- [3] "Proven Strategies for Better Verification Planning" — deliverables, definitions of done, linear progress; the antidote to percent-done scheduling.
- [4] "Smarter Verification Management" — regression data tracking, trend charts, multi-engine and milestone rollups at SoC scale.
- [9] "Metrics in SoC Verification" — the metrics landscape, its CMM lineage, and planning-phase metric design.
- [10] Foster, "Trends in Functional Verification: A 2014 Industry Study" (DAC 2015) — the effort, headcount, schedule, and respin baseline.
- [5, 12] Wilson Research Group 2024 IC/ASIC and FPGA trend reports — the current numbers: 14% first-silicon success, 75% behind schedule, 87% FPGA escape rate.
- [13] "I Created the Verification Gap" — an honest practitioner taxonomy of why the gap between needed and delivered verification exists.

Not in the corpus (cited here as pointers only):

- The OpenTitan project's public block-level verification stage checklists (V1/V2/V3) — the best openly available worked example of the maturity-stage pattern of Section 4.4.
- Wile, Goss, Roesner, *Comprehensive Functional Verification* — the classic end-to-end treatment of the verification cycle at industrial (IBM) scale.
- Carter & Hemmady, *Metric-Driven Design Verification* — book-length treatment of the metrics-driven process sketched in Sections 4.3 and 4.5.

REFERENCES

1. B2 J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.

2. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
3. **D7** P. Marriott, J. Vance, and J. McNeal, "Proven Strategies for Better Verification Planning," DVCon 2022 Workshop, 2022.
4. **D26** D. Zhang, "Smarter Verification Management with vManager Platform," Proc. DVCon China, 2021.
5. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
6. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
7. **D9** B. Ehlers, C. Vargas, and P. Carzola, "Best Practices in Verification Planning," Proc. DVCon U.S., 2013.
8. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
9. **D20** A. Meyer and H. Foster, "Metrics in SoC Verification: Not just for coverage anymore," Proc. DVCon U.S., 2012.
10. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," Proc. 52nd Design Automation Conference (DAC '15), San Francisco, CA, 2015.
11. **S19** Arm Limited, "AMBA APB Protocol Specification, ARM IHI 0024, Issue E (ID022823)," Arm Limited, February 2023.
12. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
13. **D16** R. Narayan and T. Symons, "I created the Verification Gap," Proc. DVCon U.S., 2015.

PART II

Planning and Measurement

The plan as a contract and measurement as ground truth: without these two, verification is opinion.

5. Verification Planning

Verification planning begins before any testbench exists, when the specification is the only artifact available. In the second week of the reference SoC project, the specification of the DMA engine's midend says: "*The midend legalizes each transfer against the AXI address rules, splitting any burst that would cross a 4 KB boundary.*" That sentence has to be interrogated, and three questions follow from it. Does the split hold for 1D transfers only, or also for 2D? For 2D the boundary check applies per row — does it still hold when the stride is negative, so consecutive rows walk *down* through memory toward the boundary instead of up? Does the answer change when the other channel is active and the two legalizers share the splitter state? The specification is silent on all three. Recording them as a plan row is the method this chapter sets out, and three months later that row corners the worst bug in the block — a 2D negative-stride transfer mis-legalized at the 4 KB boundary, corrupting data only while channel 1 runs. The constrained-random environment will *find* that bug; the scoreboard will *catch* it; but it was the plan that made both point their instruments at the right corner of the input space. No directed test would have been written for it: the question did not exist until the plan asked it.

Chapter 4 placed planning in the lifecycle: when it happens, what it must produce, what it costs. This chapter is about how it is actually done — the method by which a specification becomes a feature list, a feature becomes a measurable row, and a document becomes an instrument.

CHAPTER MAP

- §5.1 extracts features from a specification adversarially — by interface, by function, by architecture — and establishes why the designer interview and the *must-not-happen* list are non-optional parts of extraction.
 - §5.2 gives the anatomy of a vplan row — feature → attributes → measurable pass/fail criterion → method assignment — and the one-row-one-metric discipline that keeps closure honest.
 - §5.3 states what makes a plan *executable* and *traceable*, so that progress is read from the plan instead of asserted in meetings.
 - §5.4 sets out the plan review that actually challenges the plan, and who must be present.
 - §5.5 shows how risk assessment sets the verification depth of each feature; §5.6 treats planning against a specification that is incomplete or still moving.
-

5.1 From specification to feature list

The specification is the golden reference: when the testbench and the RTL disagree, the specification arbitrates — which is why it must exist before the implementation, and why a “specification” that merely describes what the RTL happens to do is unverifiable, since it changes every time the implementation does ^[1]. Two documents usually feed the plan: the architectural (functional) specification, from which planning can begin, and the implementation specification, which later contributes implementation-specific items ^[1]. For the DMA engine, the first defines transfers, channels, and error semantics; the second reveals that a midend splitter exists at all.

Feature extraction is the act of enumerating, from these documents, everything that must be shown to work. It is the step with the highest cost of omission in all of verification, because of a brutal asymmetry: if, and only if, a behavior is in the plan will it be verified ^[1]. Coverage will not save you — coverage measures progress against the model you wrote, and the model comes from this list (see Chapter 6). A feature that extraction misses is invisible to every metric downstream. The 4 KB-boundary row in this chapter’s opening is the positive case; every escaped bug post-mortem that ends with “we never thought of that” is the negative one.

Reading the spec adversarially

A designer reading a specification asks “how do I build this?” A verifier must ask “how does this break?” (see Chapter 2). Without that shift, planning reads the specification with a design mindset, lists a scenario per specified feature, adds error injection only for the errors the design explicitly detects — and finds the obvious bugs while leaving the obscure ones, which are every bit as critical, unhunted ^[2]. The discipline that replaces it is systematic interrogation: three passes over the design, each with its own question set ^[1]:

Interfaces first. For every interface, enumerate the features it implies: what transactions, what ranges of values, what sequences and densities, what protocol violations the design must sustain, what interactions exist between this interface and the others ^[1]. On the DMA frontend this yields: every register readable and writable per its access policy — read-only, read-write, write-one-to-clear; a configuration written while the channel runs is rejected or deferred, never half-applied; back-to-back writes to the trigger register start exactly one transfer. On the backend: legal AXI on every channel, correct accumulation of the two AXI error responses — SLVERR, a subordinate signalling failure, and DECERR, no subordinate at that address — and behavior under arbitrary `ready` back-pressure.

Functions second. Follow the data through the specified transformations: relevant configurations, possible transformations and their sequences, the *sensitive values* that trigger each one, where transformed data must end up, how ordering is affected, what error-detection mechanisms exist, how they report, and what happens to erroneous data [1]. For the DMA midend, “sensitive values” is the load-bearing phrase: transfer lengths of 0, 1, one beat less than max burst (256 beats), exactly max burst, one more; source and destination alignments that do and do not straddle a 4 KB frontier; 2D strides positive, zero, and negative.

Architecture third. From knowledge of the chosen implementation, list the conditions that push the design toward its limits: can a buffer overflow or underflow, where are the resource bottlenecks, can multiple requests collide on them, can one transformation path block another [1]. This pass is where the two-channel structure of the DMA stops being a bullet point and becomes a feature generator: two channels contending for one backend port, one channel’s error not perturbing the other’s stream, splitter state shared or not shared between channels. The opening’s killer question — negative stride, near the boundary, *while channel 1 is active* — belongs to this pass. It could not come from the functional spec alone, because the sharing of midend state between channels is an implementation fact.

Alongside the three passes, classify what you are collecting. A practical taxonomy from one published planning workshop sorts scenarios into four bins: *isolated features* (one behavior at a time — the backbone of bring-up and debug), *mixed features* (the key combinations — where interconnect and multi-channel bugs live), *legal exceptions* (abnormal but supported cases — soft reset mid-transfer, error responses, FIFO-full — which must appear in the design spec), and *illegal scenarios* (cases the design does not support — where the spec must say what “unsupported” means: ignored, or recovered by reset) [3]. The last two bins are chronically under-populated in first-draft plans, and they are where a planning rule of thumb applies with full force: a verification effort that does not inject errors is simply incomplete [2].

Interviewing the designer

The specification, however good, does not contain the whole feature list. Team members beyond the plan’s author — system architects and the RTL designers themselves — contribute features that are invisible in the document: some are not obvious to anyone unfamiliar with the design’s purpose, and others only come into existence when a particular implementation is chosen [1]. One methodology makes the point structurally: implementation-specific verification requirements create corner cases *that only the designer is aware of* — the specification never mentions the FIFO the designer inserted, yet its full and empty conditions must be verified — and it recommends the designer capture such requirements as coverage properties in the RTL itself [4]. The same advice governs coverage attributes: the most effective way to identify them is a brainstorming session among several verification engineers and designers familiar with the specifications [5].

Treat the interview as extraction, not as review. Useful questions, on the reference SoC’s DMA: *What was the hardest part to get right?* (the splitter’s resumption ad-

dress arithmetic — instant plan priority). *What state is shared between the channels?* (the answer decides whether “channel isolation” is one feature or ten). *What did you assume the software will never do?* (each answer is either an out-of-scope entry, reviewed and signed, or a new feature — never an unexamined hope). The interview also runs in reverse: planning early enough lets you request design changes that make hard features verifiable — pre-loadable counters, sample points, error-injection hooks (see Chapter 3’s observability discussion) [1].

What must not happen

A specification describes intended behavior. Verification is concerned with detecting errors — and it can only detect errors it is looking for [4]. This is why a feature list that contains only affirmative sentences is half a list. One methodology elevates the negative half to a rule: define what the design *must not do*, as requirements that enumerate the relevant and probable errors — there are infinitely many ways to fail, so judgment selects which failures the environment must be able to see [4]. The checker architecture of Chapter 8 is downstream of this list: a scoreboard detects exactly the wrongs someone decided to look for, and correctness at the end of the project is inferred from the *failure to find inconsistencies* — nothing stronger [4].

Must-not-happen features are also where the DMA’s canonical bug was pre-caught. “Data is delivered uncorrupted, in order, exactly once” is the affirmative feature; its negative twin — *no transfer shall ever write a byte to an address outside its legalized destination window, under any channel interleaving* — is what the scoreboard’s address check implements, and it is that check, not any directed expectation, that eventually fired on the negative-stride bug. The move fits any block whose correctness is numerical rather than structural: on the radar front-end, “the FFT pipeline computes the transform” says nothing about the fixed-point contract, so the negative list carries it — *no FFT stage shall saturate without raising its saturation flag* — because an unreported saturation turns a target into noise and leaves no trace anyone can see. Symmetrically, the third list — what is explicitly *out of scope* — bounds the effort: conditions the design need not survive are written down, so that “we do not verify descriptor chains longer than memory” is a reviewed decision, not an accident [4]. One caution from the same source: “if it is not specified, do not verify it” applies to genuine don’t-cares — an *incomplete* specification is not a don’t-care but a defect to be fixed [4]. Section 5.6 returns to that boundary.

Worked example: the DMA feature list, first cut

After the three passes, the designer interview, and the negative list, the DMA plan’s feature list opens like this (excerpt; the full list runs to roughly sixty entries):

```
DMA-F01 Frontend: register access per access policy (R0/RW/W1C), reset values
DMA-F02 Frontend: config write during active transfer rejected, error flagged
DMA-F03 Midend: 1D transfer legalized into AXI bursts, max 256 beats
DMA-F04 Midend: burst split at 4 KB boundary, all alignments, len 0/1/max±1
DMA-F05 Midend: 2D transfer, per-row legalization, stride > 0, = 0, < 0
DMA-F06 Midend: 4 KB split correct under 2D negative stride [arch pass + interview]
DMA-F07 Midend: channel 0/1 legalizer state fully independent [interview]
DMA-F08 Backend: AXI protocol compliance under arbitrary back-pressure
```

```

DMA-F09 Backend: SLVERR/DECERR reported in the issuing channel's status only
DMA-N01 MUST NOT: write outside legalized destination window (any interleaving)
DMA-N02 MUST NOT: raise done interrupt before last write response accepted
DMA-N03 MUST NOT: propagate channel A error into channel B stream
DMA-X01 OUT OF SCOPE: descriptor fetch from uncached device memory (not supported;
        software constraint documented, reviewed with design in week 3)

```

Note the shape: affirmative features (F), must-not-happen features (N), out-of-scope entries (X) with a review date; provenance tags where a feature came from the architecture pass or the interview rather than the spec text. DMA-F06 exists because someone asked a question the specification did not answer. That is feature extraction working.

5.2 The anatomy of a vplan row

A feature list is a promise; the vplan turns each promise into something checkable. The cleanest definition: the verification plan defines *what* must be verified and *how* it will be verified — it is the specification document of the verification environment [1, 5]. The partition of “what” from “how” is explicit: what must be verified lives in the coverage requirements; how, in the design of the three aspects every dynamic environment is built from — stimulus generation, response checking, coverage measurement [5]. A well-formed vplan row is that partition in miniature. Five fields, each answering one question:

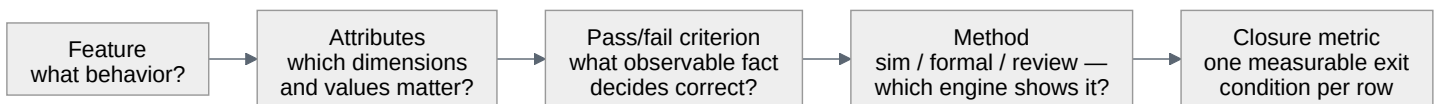


Figure 5.1 — The five fields of a well-formed vplan row, in the order they are written and with the question each one answers: the behavior, the dimensions and values that matter, the observable fact that decides correct, the engine that will show it, and the single measurable condition that closes the row. The arrows are ordered rather than decorative: the attributes belong to a stated feature, and the closure metric is chosen for a stated criterion and a chosen method. The last field is where the one-row-one-metric rule bites: a row that needs two kinds of evidence is two rows.

Feature. One behavior, stated as *conditions plus expected result*, never as an implementation description; labeled with a unique identifier; cross-referenced to the specification section that defines it [1]. Identifiers are permanent: when a requirement is deleted, its identifier retires with it and is never reused, so that an old regression report can never silently point at a new requirement [4]. The label earns its keep at debug time — put it in the checker’s error message, so a failure names the requirement it violated [1].

Attributes. An attribute, in Piziali’s vocabulary, is a parameter or dimension of *the coverage model* — a configuration mode, an opcode, a control-field value, a packet length [5]. The distinction between the model and the device it abstracts is deliberate, and Chapter 6 depends on it. Listing a feature’s attributes and *their values that matter* is what turns prose into a verifiable space. The value-selection rules differ by kind: for a control field, enumerate every value; for a data field, choose values by

how the design processes them [5]. One published example shows what this looks like in practice — a maximum-packet-length requirement becomes packet lengths at MAXFL-4, MAXFL-1, MAXFL, MAXFL+1, MAXFL+4 and 65535, crossed with the configuration values that change the limit [4]. The pattern to internalize: *boundary, both neighbors of the boundary, and the far extreme, crossed with every configuration that moves the boundary.*

Pass/fail criterion. The row must state how the response will be judged correct — expected values, timing, protocol — and, most powerfully, *which errors to look for*: explicit acceptance criteria let an engineer who is not the block’s expert build a reliable check [1]. “Transfer completes” is not a criterion. “Every destination byte equals its source byte, in order, exactly once, and the done interrupt follows the last accepted write response” is. Notice that a pass/fail criterion is a checker specification: writing this field well is where the scoreboard of Chapter 8 is actually designed.

Method. Not every row is a simulation row. One methodology’s response-checking guideline doubles as a method-selection guideline: failures visible at signal level are most efficiently caught by assertions; failures visible per output transaction, by a scoreboard; failures that are statistical — performance, fairness — by offline analysis of a trace [4]. Extend the same logic across engines: exhaustive, local, control-dominated properties go to formal; data-transport correctness at scale goes to constrained-random simulation with a scoreboard; and some rows are discharged by *review* — a signed inspection of code or configuration where neither simulation nor proof is the right instrument (see Chapters 14 and 16 for the sim/formal division of labor). Others are discharged by an instrument the plan does not own: a USB 3.x dual-role controller’s plan sends its link-training and low-power rows — the U0 through U3 transitions — to the specification-mandated compliance suite, so the method field reads *compliance suite* and the closure metric is the suite’s own item list. Reusing someone else’s exhaustive suite is a method decision, and the row’s remaining job is to record which features it discharges and which it leaves open. A published RISC-V processor verification plan is a working illustration of a dual assignment: formal verification carries architectural compliance while simulation carries the complex behaviors — custom loop hardware, interrupts, debug mode [6]. The scope assessment that one published industrial methodology prescribes makes this an explicit planning artifact: for each part of the effort, decide directed versus constrained-random versus formal, and decide which kinds of coverage will measure it, before implementation begins [7].

Closure metric — one per row. The last field states the evidence that closes the row: a functional coverage group at 100%, a proven property, a passed directed test, a signed review. The discipline this book insists on is **one row, one metric**: if a row needs two different kinds of evidence, it is two rows. The rule sounds bureaucratic and is not; closure is computed row by row (see Chapter 7), and a row whose exit condition is “coverage *and* the assertion *and* the stress test looks fine” cannot be automatically evaluated, so it gets closed by fatigue instead. One published RISC-V plan shows the pattern in its row format: each verification goal is a row carrying its own pass/fail criterion (“check against reference model”), its own test type (con-

strained-random), and its own coverage method (functional coverage) — one goal, one measurable discharge [6].

Worked example: four rows of the DMA vplan

Feature DMA-F06 from Section 5.1 needs two different kinds of evidence, so it becomes two rows — F06a and F06b. A second feature is shown alongside them: the must-not-happen entry DMA-N01, whose always-on check runs in the same simulations that fill F06a’s cross, and which splits into two rows for the same reason. Four rows, two features, four metrics — which is exactly what the one-row-one-metric rule forces you to notice:

Table 5.1 — Four rows of the DMA vplan, carrying the five fields of the row anatomy across two of the features listed in §5.1. DMA-F06 splits into a constrained-random row closed by a covergroup and a formal row closed by a proven property; the must-not-happen entry DMA-N01 splits into the always-on scoreboard check and the injection test whose failure condition is the *absence* of the error message. Two features, four rows, four metrics — the arithmetic the one-row-one-metric rule forces you to notice.

ID	Feature & attributes	Pass/fail criterion	Method	Closure metric
DMA-F06a	2D legalization at 4 KB boundary. Attributes: stride sign $\{+,0,-\} \times$ the row’s relation to the next boundary {more than a burst of headroom, exactly a burst, less than a burst, straddling it} $\times$ burst length $\{1, 255, 256\} \times$ other-channel {idle, active}	Every AXI burst issued lies within one 4 KB frame; the union of issued bursts equals the transfer’s exact byte set	Constrained-random sim, scoreboard address/data check	Covergroup <code>cg_2d_4kb</code> (below) at 100%
DMA-F06b	Splitter never emits a boundary-crossing burst (any input)	AXI 4 KB rule (Arm IHI 0022H.c §A3.4.1) [8] as a safety property on the backend’s AXI write-address channel	Formal proof, backend standalone	Property <code>p_no_4kb_crossing</code> proven unbounded
DMA-N01a	No write outside the legalized destination window, any channel interleaving	Scoreboard flags any write beat whose address $\notin$ expected window, with feature ID in the message	Constrained-random sim (always-on check)	Check enabled in 100% of regression runs
DMA-N01b	That check demonstrably fires	Injection test drives one write beat past the window’s edge; the failure condition is the <i>absence</i> of the error message	Directed sim, error injection	Injection test passes

Splitting N01 rather than joining its two exit conditions with a semicolon is the one-row-one-metric rule under pressure: “check enabled everywhere *and* proven to fire”

is two kinds of evidence, and a tool can evaluate each of them separately but neither of them together. The split also gives a home to another planning rule — never trust a checker you have not seen fail. Every planned check earns a companion row for the test that injects the error it is supposed to catch, and the absence of the error message is that test’s failure condition ^[1].

Row DMA-F06b points at an artifact the reader has already met. It is the property of Chapters 2 and 3, unchanged:

```
property p_no_4kb_crossing;
  @(posedge clk) disable iff (!rst_n)
    (awvalid && awready && (awburst == 2'b01)) |->
      (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

What the plan decides here is not the property’s text but its *binding point*. The invariant is a claim about what leaves the block, so the row binds it where bursts become observable — the backend’s AXI write-address channel — and proves it on the backend standalone. Binding the same text inside the midend would prove a claim about an intermediate representation and leave the backend’s own address arithmetic unproven: the row would close, and the block would still be wrong. A vplan row names the interface, not just the property.

The closure metric of DMA-F06a is a covergroup, and writing it forces the plan to say exactly what its boundary attribute *means*. It is not a raw address. For each descriptor row the monitor computes two byte counts — the *headroom*, from the row’s start address to the next 4 KB frontier in the direction the stride sign selects, and the bytes one maximum-length burst of this row would move (its burst length × 8 on the 64-bit bus) — and classifies the row from the pair. A row that reaches past the frontier is *straddle*, whatever its bursts do; a row that stays inside its frame is classified by headroom against burst bytes. Four cases, mutually exclusive, covering every row exactly once:

```
covergroup cg_2d_4kb @(posedge clk iff midend_txn_done);
  cp_stride_sign : coverpoint txn.stride_sign
    { bins pos = {POS}; bins zero = {ZERO}; bins neg = {NEG}; }
  // txn.bnd_rel: per-row relation, computed by the monitor from the row's
  // headroom (bytes to the next 4 KB frontier, in the stride's direction)
  // and this row's burst bytes. The four values partition every row once.
  cp_boundary_dist : coverpoint txn.bnd_rel
    { bins gt_burst = {GT_BURST}; // stays in frame, more than a burst of headroom
      bins eq_burst = {EQ_BURST}; // stays in frame, exactly a burst of headroom
      bins lt_burst = {LT_BURST}; // stays in frame, less than a burst of headroom
      bins straddle = {STRADDLE}; // reaches past the frontier – must be split
    }
  cp_len : coverpoint txn.burst_len
    { bins one = {1}; bins max_m1 = {255}; bins max = {256}; }
  cp_other_ch : coverpoint other_channel_active;
  x_worst : cross cp_stride_sign, cp_boundary_dist, cp_len, cp_other_ch;
endgroup
```

The industry’s accumulated DMA post-mortems live in the bin named `straddle` crossed with `neg` and `other_channel_active = 1` — two of the cross’s 72 combinations, not three, and the missing one is instructive: the generator’s solver budget caps a row at 256 beats, so a straddling row is split into pieces none of which can be a maximum-length burst, which leaves the 1-beat and the 255-beat cells. That exclusion follows from a budget the team chose rather than from the model itself — lift the budget and the cell comes back — which is exactly why it belongs on the row as a recorded waiver rather than in a footnote. When those two cells fill *and* the scoreboard stayed silent, row DMA-F06a is closed by evidence. When they fill and the scoreboard fires — as they did on SOC-1284, the 2D legalization escape of Chapter 1 — the plan has done its job even more directly.

Two choices in that snippet are deliberate. `cp_other_ch` is the one coverpoint without explicit bins, because a one-bit flag’s automatic bins already name both of its values and label the cross correctly. And *at 100%* means 100% of the cells the model can legitimately reach, which is a smaller and more carefully defined set than the cross’s 72. In this model the gap is entirely of one kind: the six maximum-length straddle cells are perfectly possible and are excluded because the team chose a budget that forecloses them. Naming the kind matters, because a coverage model can also contain cells that are impossible by construction — where one axis is classified against another and a pair can contradict itself — and the two are not the same claim. The difference surfaces when someone later asks whether the hole can be opened: an impossible cell cannot be, a budgeted one is one constraint away. Either way the exclusion is explicit, with its justification recorded on the row (see Chapter 6 for how waivers are written). The model also obeys Chapter 2’s rule of pairing: this coverage means something only because the scoreboard check of DMA-N01a runs in the same simulations — coverage without checking certifies stimulus, not correctness [2].

One thing about this model deserves saying out loud, because the reader has already met a smaller version of it. The covergroup in Chapter 2 crosses the same feature into 36 cells; this one makes 72. Neither is wrong, and the difference is the more useful lesson. The stride axis gained a bin because a zero stride — a descriptor whose rows all start at the same address — turned out to be its own legalization path rather than a degenerate case of the other two. The boundary axis changed more deeply: Chapter 2 bins the headroom to the next frontier in *absolute bytes*, against fixed thresholds, while this model bins it *relative to what one burst of this row would move*. That is not a cosmetic refinement. A fixed threshold is a buried guess about burst length, and it cannot name the case that matters most — headroom exactly equal to one burst — because that case moves as the burst does. A coverage model is not written once from the specification and frozen; it gains a bin on the day someone learns something about the design. Which is why a closed row should record *which version of the model* closed it. Otherwise a row that read 100% last quarter is quietly answering a coarser question than the one now being asked, and nothing in the report says so.

One structural warning as the rows multiply. Keep the plan in the *DUT’s* vocabulary — features by design functionality, scenario descriptions, kinds and quantities of cov-

erage — and out of the testbench’s: no component inventories, no test lists, no hard-coded bin values that change with every environment refactor [3]. A plan written in testbench vocabulary has to be rewritten every time the testbench is; a plan written in DUT vocabulary survives, and the testbench is derived from it. Assign each row to the verification level — block, subsystem, chip — where its behavior is fully contained and observable (see Chapter 3 on controllability, Chapter 8 on environment levels): the bulk lands at block level, and a crowd of rows piling onto one deeply buried unit is the classic sign that the unit needs its own environment [1].

5.3 Making the plan executable and traceable

A plan that lives in a document repository decays; the industry’s recurring complaints — plans that take too long to write, contain nothing the team uses (“nobody reads them”), and rot as the design changes — are documented [3]. One published methodology traces all three barriers to a single missing property: the plan is not connected to anything [7]. The cure has a precise definition, introduced in Chapter 4 and taken up here as mechanics. An *executable* verification plan is a machine-readable structure that defines the project’s scope, lists the features in it, attaches the testcases, checks and coverage objects to each feature, and links design requirements down to those objects; what makes it “executable” is that the regression list is generated *from* the plan, and every run’s pass/fail and coverage results are annotated *back onto* the plan’s objects [7]. What-was-verified versus what-was-not is then computed, not remembered, and closure reporting costs nothing because the mapping from results to features already exists [7].

Traceability is the same linkage read in the other direction, and it is built from the identifiers of Section 5.2. Each requirement’s unique ID cross-references three things: the specification clause it came from, the testcases and properties that discharge it, and the coverage points that measure it [4]. One audit rule makes the payoff concrete: annotate every feature with the testcases that verify it — a feature with no cross-reference *is not being verified*, and the plan is the only place that fact is visible before tape-out rather than after [1]. One published industrial flow adds the practical detail that generic planning tools rarely provide out of the box: a requirement-tag field on every testcase and coverage object, linked into the program’s requirements-management system, plus a per-row *target configuration* field (RTL, gate-level, mixed-signal) so the regression list extracted from the plan knows where each test must run [7].

Concretely, on the reference SoC the DMA plan is a version-controlled structured file (the format matters less than the parseability [7]); DMA-F06a names covergroup cg_2d_4kb and the scoreboard check; the nightly regression publishes per-row status; and the specification paragraph on midend legalization carries the tag [DMA-F03..F07] in its margin. When the specification changes, the tags say which rows are stale. When a row closes, the dashboard of Chapter 7 says so without a meeting. Chapter 4 described this machinery’s lifecycle consequence — every later phase reads progress from the plan; here the point is methodological: *executable* and

traceable are properties you design into the plan's rows on day one, not export formats you bolt on at the end.

5.4 The plan review as a rite

The plan review is the moment the plan stops belonging to its author. Everything about how it is run follows from one fact, met in Section 5.1: completeness is the property no single perspective can check — and a designer's responsibility is a working design, not RTL, so the entire design team, not the plan's author, must contribute to the plan and answer for its completeness [1].

Who attends. Chapter 4 placed the review in the calendar and named the roster — design, test/product, systems, and verification peers from outside the block, convened at prescribed stages of planning, implementation, debug and closure [7]. What matters here is why each seat exists: each catches a different class of hole. The designer catches features the spec never wrote down (Section 5.1's interview, now on the record). The architect catches misread intent. The software or systems engineer catches use cases nobody inside the block imagined — Chapter 4's crossbar review, where the firmware engineer's question about the debug port added feature XB-07, is the pattern. The outside verification peer catches method errors: unmeasurable pass/fail criteria, coverage that certifies stimulus rather than behavior, missing error injection.

What gets challenged. A review that walks the feature list nodding is a signing ceremony, not a review. One published workshop gives the challenge procedure a name — weakness analysis — and three blades, applied row by row [3]:

- *Correctness*: is this row real? Does the DUT actually implement this option, is the behavior fully specified, do the RTL parameters allow it, is it a valid use case — do we care? [3] Rows that fail this test are deleted, and deleting rows is a review success: unverifiable and irrelevant rows cost real schedule.
- *Precision*: what exactly is being verified, and in what context? What are the feature's dependencies, its disqualifying conditions, its value ranges and event timing — and are there different goals in different contexts? [3] "Verify 2D transfers" fails precision; DMA-F06a's attribute list passes it.
- *Completeness*: for each influencing variable, how can it change and what does the change do to the DUT; for each scenario, what do you expect *and what do you not expect*; which feature combinations produce unique outcomes that need their own rows? [3]

Two more challenges belong in every review. First, the assumptions: every "verified elsewhere," every "software will never do that," reviewed face to face with the designers — usually over several rounds — because unspoken assumptions are how features fall through the cracks [7]. Second, the designer's own "that can't happen." Chapter 2 established the mindset; at plan review it becomes procedure: whether a case is *valid* is irrelevant — the only question is whether it can occur, and if it can, the recovery is a feature [2].

Scenario. At the DMA plan review, the block’s designer objects to row DMA-F06a: “negative-stride 2D descriptors that straddle a boundary can’t happen — the driver library always allocates row-aligned buffers.” The verification lead asks two questions. *Is the driver library the only software that will ever program this block?* (No — the register interface is documented, and the neural-accelerator team writes its own descriptors.) *Does the RTL check the condition, or assume it?* (Assume.) **Approach.** The row stays, its risk rank rises — an unchecked assumption reachable by third-party software is precisely where escapes breed — and a new must-not-happen row is added for the frontend: descriptors the design cannot honor are rejected with an error, not silently mangled. The objection and its resolution are recorded on the row. **Why.** Those records are linked directly to the plan object, where audits and quality reviews can reach them ^[7] — which in practice makes them the institutional memory that survives both engineers’ departure; and the exchange itself is the review doing what no coverage metric can — testing the plan against knowledge that exists only in people.

The review marks a status change. Before it, the plan is a draft owned by one person; after it, it is the block’s contract, and changing it requires the same audience (see Chapter 4 for where the reviews sit in the calendar, and Chapter 24 for review culture at team scale).

5.5 Risk: deciding how deep to go, feature by feature

A plan that assigns every feature the same depth has not been planned; it has been transcribed. Depth — how much stimulus, how many crosses, which engines, how much of the schedule — is a per-feature decision, and risk is the input that decides it.

The base mechanism is priority, and it should be written into the rows. Three priority tiers are the canon: *must-have* features define first-time success, gate the project’s completion, and get thorough verification across all configurations; *should-have* features get their basic operation verified, with depth added only as time allows; *nice-to-have* features are verified “as time allows” — which today’s design schedules almost guarantee means never ^[1]. One methodology operationalizes the same idea as requirement ranking — resources go to the highest ranks first, and the tape-out decision is taken when the most important requirements are met — plus requirement *ordering*, because dependencies dictate sequence: register access is verified before the configurations that use it ^[4]. The point of the tiers is not optimism about the bottom one; it is that when the schedule guillotine falls (see Chapter 4), the cut line was drawn deliberately, in daylight, months earlier ^[1].

Priority ranks features by consequence. Risk adds the second axis: likelihood of a bug — and bug risk and complexity risk are among the published axes along which features are isolated for analysis in the first place ^[3]. In practice the risk drivers a plan review scores are: novelty (new logic versus silicon-proven reuse — the reuse assessment of Chapter 4 feeds this directly ^[7]), structural complexity (shared state,

concurrency, arbitration), specification quality (Section 5.6), controllability and observability at the chosen level, and the cost of escape (data corruption silently, versus a flag readable by software). Where consequence and likelihood are both high, the plan buys depth in every currency at once: more attributes and deeper crosses in the coverage model, a second engine on the same behavior (DMA-F06a and F06b — simulation for the transport, formal for the invariant), tighter closure criteria, review of the checker itself.

Walk one feature through the five drivers and the verdict stops being an assertion. DMA-F06: *novelty* — a new 2D negative-stride path, not silicon-proven reuse (high); *structural complexity* — splitter resumption arithmetic held in state shared between two channels (high); *specification quality* — the document is silent on exactly this case (high); *controllability and observability* — both good at block level, since descriptors are programmed directly and every issued burst is visible on the backend interface (low, the one driver arguing for less); *cost of escape* — silent data corruption that no status flag reports (worst available). Four drivers high, and the favourable one only makes depth affordable rather than unnecessary: high consequence $\times$ high likelihood, which buys every currency just listed at once. Run the same five questions on DMA-F01 and every answer inverts — reused register logic, generated with its own tests from one description through a register abstraction layer (RAL — see Chapter 10), a fully specified access policy, complete observability, a consequence software can read back — so its row buys almost nothing, deliberately. Table 5.2 ranges both features and three others across the same two axes; DMA-F06 and DMA-F01 are its endpoints, not its typical rows.

Table 5.2 — Five DMA features ranked on the two axes that set verification depth — the consequence of an escape and the likelihood of a bug — with the depth each ranking buys. The rows are deliberately unequal, and that is what a risk assessment produces: DMA-F06, novel and with a silent-corruption consequence, buys a full cross in constrained-random simulation and a formal property besides, while the reused register logic of DMA-F01 buys generated tests and a review of the generator’s configuration and nothing more.

DMA feature	Consequence	Bug likelihood	Planned depth
DMA-F01 register access	medium	low (generated RAL)	Generated register tests; review of the generator config
DMA-F03 1D legalization	high	medium	CR sim + scoreboard; boundary-value coverage
DMA-F06 2D neg-stride @ 4 KB	high (silent corruption)	high (novel, shared state, spec silent)	Two rows: full cross in CR sim (F06a) + formal safety property (F06b)
DMA-N01 window violation	high (silent corruption)	medium	Two rows: always-on scoreboard check (N01a) + its injection test (N01b)
DMA-F09 error routing	medium	medium	CR error injection; per-channel status coverage

The unit of ranking is not always the module. On the GPU the same five questions are asked per workload class: the tile binner handling one full-screen triangle is silicon-proven, while the same binner under thousands of sub-pixel triangles straddling tile edges is novel, poorly observable and expensive to escape — so depth is bought

per workload class, and one module appears in rows of three different depths. That is only possible because the rows were written against behaviors rather than against blocks.

Depth can also be staged in time. One published RISC-V coverage plan from 2020 makes maturity explicit in the metric itself: a *compliance basic* coverage level gives stimulus-effectiveness insight while the design is immature and changing, and the *compliance extended* level becomes the closure goal once the design is stable ^[6] — the plan’s exit criteria tighten as the design earns it. And depth interacts with method economics: the 2006 estimate is that a directed-test plan stays manageable when testcases number in the low hundreds, while a project with over a thousand identified testcases would need more than a year of hand-writing — which is the arithmetic that pushes high-risk, high-cardinality features toward constrained-random automation with coverage closure rather than enumeration ^[1]. The same arithmetic appears on the coverage side: in the same 2020 plan, hand-writing the coverage model for a 120-instruction ISA costs roughly 3,600 lines of SystemVerilog at 10–30 lines per instruction, and a 300-instruction extension another ~9,000 — the published conclusion is that generation, not typing, is the only sane source for that class of model ^[6]. Risk assessment, in other words, does not only set depth; it selects the technology that makes the chosen depth affordable.

That plan is not an anonymous example, and it repays opening. It is the verification plan for the OpenHW Group’s CV32E40Pv2 — a 32-bit in-order RISC-V core that began as ETH Zurich’s RI5CY and is developed onward by Dolphin Design, with Imperas supplying both the reference model and the generated coverage model — and it is public, published in the `core-v-verif` repository as files a reader can inspect row by row ^[6]. The arithmetic just given is that programme’s own, and so is the conclusion it drew from it ^[6].

5.6 Planning against an incomplete specification

Everything so far assumed a specification worth interrogating. Reality is documented in the 2024 industry data: changing, incorrect, or incomplete specifications remain a common concern among verification engineers and project managers, and design errors have historically been the leading root cause of the functional flaws behind respins, with recent signs of improvement ^[9] — and in practice the two are entangled, because a designer implementing an ambiguous sentence is manufacturing a design error with a specification-shaped cause. The planning literature is equally blunt: the quality of the design documentation is the primary input to planning; the worst case is a design feature with no documented description, where the verification engineer must do original research before a meaningful plan is possible; and frequent or late design changes can invalidate large portions of planning work outright ^[7]. The plan cannot pretend otherwise. It has to be built to absorb this.

The method is the published “detail levels” guidance, and it is the opposite of both denial and paralysis: with a complete, detailed specification, invest in a complete, detailed plan up front — coverage and assertions specified in the rows; with a min-

imal or in-progress specification, write a less detailed plan now, fully elaborate only the first few deliverables, add detail to later rows *as the information is finalized*, and put explicit planning-and-architecture time inside each later deliverable’s estimate [3]. Change handling follows the same shape: absorb small specification changes into future rows; for significant ones, add new deliverables rather than silently stretching old ones, and review the change’s impact with the team so the cost is communicated in a form everyone already understands [3]. Two disciplines keep this honest. First, the boundary from Section 5.1: an unspecified genuine don’t-care is excluded from verification, but an *incomplete* specification is a defect — the plan’s job is to surface it for fixing, not to paper over it [4]. Second, never let the plan quietly become the spec: a specification that is a consequence of the implementation is unverifiable [1] — so every hole the plan finds goes back through the specification’s owner, in writing.

Scenario. The neural accelerator enters planning with a specification that is solid on the datapath — 2/4/8-bit weight modes, the 16-lane MAC array, quantization and saturation semantics — and hollow on system behavior: the memory-streaming interface section is headed “TBD pending crossbar arbitration experiments,” and job-abort semantics are one sentence old. **Approach.** The verification lead writes the plan anyway, in three zones. Zone 1, fully elaborated rows for the stable datapath (attribute-complete: weight width $\times$ saturation boundary $\times$ accumulator sign crosses), scheduled as the first deliverables. Zone 2, *stub* rows for the streaming interface: feature named, spec reference marked TBD(§4.2), pass/fail and attributes deliberately blank, each stub carrying a planning-time allowance in its estimate and a named specification owner. Zone 3, a spec-hole log — every question the extraction passes raised and the spec could not answer (“what does the engine do if the crossbar stalls the weight stream mid-row?”), filed as specification defects with dates, reviewed weekly with the architect. The plan review covers zones 1 and 3 now; zone 2 rows are review-gated on their spec sections landing. **Why.** The alternative failure modes are both worse and both common: planning nothing until the spec is “done” (it never is — and verification’s questions are among the forces that finish it), or planning fiction against guessed behavior that a late change then invalidates wholesale [3, 7]. The zoned plan converts spec incompleteness from a schedule surprise into a tracked, owned, visible work item — and the spec-hole log makes the verification team what it should be: the first, most systematic reader the specification ever gets.

The deeper lesson of the incomplete-spec case is that planning is not a phase that consumes a finished specification; it is the activity that *tests* the specification, years before silicon tests the design. Every question in Section 5.1’s passes that the document cannot answer is a specification bug found at the cheapest possible moment — by a reader with a highlighter, before anyone has built anything wrong.

IN PRACTICE

Real plans are messier than this chapter’s excerpts. Teams that do this well share a few habits. The first draft of the feature list is timeboxed to days, not weeks: the pub-

lished guidance is low effort for fast results, with more detail added later where it proves necessary ^[3] — and the deeper reason is that completeness comes from the review, not from the author polishing alone. The plan lives in version control next to the RTL. It is kept in a parseable format, because every derived artifact — regression list, dashboard, traceability report — is generated from it ^[7]. Rows get owners the way code does. The must-not-happen list and the out-of-scope list are reviewed *harder* than the feature list: an omission in the first is an escape nobody was watching for, and a wrong entry in the second is an escape somebody signed for. And teams accept the front-loaded cost knowingly, at the 2013 numbers Chapter 4 quantified: 10–20% added at the front, 10–40% of the overall schedule and at least 20% of implementation and closure effort returned ^[7] — one organization’s figures, but the sign of the trade is consistent across everything this chapter cites.

PITFALLS

1. **Transcription planning.** *Symptom:* the plan is the specification’s headings with “verify that” prepended. *Cure:* run the three extraction passes and the designer interview; count features tagged [interview] and [arch pass] — zero means extraction never happened.
2. **Unmeasurable rows.** *Symptom:* pass/fail criteria like “works correctly under stress.” *Cure:* every row states the observable fact that decides it and the single metric that closes it; a row that cannot name its metric is split or deleted.
3. **Affirmative-only plans.** *Symptom:* no must-not-happen rows, no error injection, checkers that have never been seen to fail. *Cure:* a negative twin for every data-integrity feature; an injection test per checker whose failure condition is the *absence* of the error message ^[1].
4. **Testbench-vocabulary plans.** *Symptom:* rows named after agents and sequences; every environment refactor forces a plan rewrite. *Cure:* DUT functionality and scenario descriptions only; the testbench is derived from the plan, never the reverse ^[3].
5. **The signing-ceremony review.** *Symptom:* a one-hour walkthrough, no rows changed, no assumptions challenged. *Cure:* weakness analysis row by row — correctness, precision, completeness — with designers, systems and outside peers present and the objections recorded on the rows ^[3, 7].
6. **Uniform depth.** *Symptom:* every feature gets the same coverage template regardless of novelty or consequence. *Cure:* rank consequence × likelihood per feature; spend crosses, engines and schedule where both are high, and write the cut line for the rest in advance ^[1, 4].

SUMMARY

- A behavior is verified only if it is in the plan; every downstream metric measures against the feature list, so extraction is the highest-cost-of-omission step in verification ^[1].

- Extract adversarially in three passes — interfaces, functions, architecture — then interview the designer for the corner cases only the implementation knows, and write the must-not-happen and out-of-scope lists explicitly [1, 4].
- A vplan row is feature → attributes → measurable pass/fail criterion → method → one closure metric. One row, one metric: rows that need two kinds of evidence are two rows.
- Match method to failure signature: assertions for signal-level wrongs, scoreboards for transaction-level wrongs, offline analysis for statistical ones, formal for local invariants, review where no engine fits [4].
- An executable plan generates the regression list and absorbs the results; traceability ties spec ↔ feature ↔ test/coverage by permanent IDs. A feature with no cross-reference is not being verified [1, 4, 7].
- The plan review is a rite with a roster — design, systems, test, outside peers — and a procedure: challenge each row’s correctness, precision and completeness, and every “that can’t happen” [2, 3, 7].
- Risk (consequence × likelihood) sets per-feature depth; priority tiers pre-draw the schedule cut line; an incomplete spec is planned in zones, with its holes filed as specification defects [1, 3, 4].

FURTHER READING

Corpus sources.

- [1] Bergeron, *Writing Testbenches Using SystemVerilog*, ch. 3 — the fullest textbook treatment of feature extraction, prioritization, and testcase derivation; the source of this chapter’s extraction questions.
- [4] *Verification Methodology Manual for SystemVerilog*, ch. 2 — planning as rules and recommendations: must-not-do requirements, unique identifiers, ranking, response-checking selection, coverage translation.
- [5] Piziali, *Functional Verification Coverage Measurement and Analysis*, ch. 2 and 4 — the plan as the verification environment’s specification; attributes and attribute-value selection; the coverage plan (continued in Chapter 6).
- [7] “Best Practices in Verification Planning” — the executable-plan definition, template discipline, assumption reviews, and the planning ROI data.
- [3] “Proven Strategies for Better Verification Planning” — feature isolation, scenario classification, weakness analysis, and planning under incomplete specifications.
- [6] “Planning for RISC-V Success” — a published vplan row format with per-goal pass/fail and coverage method; staged coverage levels; the arithmetic for generated coverage models.
- [2] “Verification Mind Games” — outside-the-box planning: why design-mindset plans miss the obscure bugs (the mindset itself is Chapter 2’s subject).
- [9] Wilson Research Group 2024 IC/ASIC report — specification issues as a persistent root-cause concern behind functional flaws.

Not in the corpus (pointers only):

- Haque, Michelson, Khan, *The Art of Verification with VERA* — origin of the interface/function/corner-case extraction ordering that Section 5.1's three passes adopt.
- The open-source RISC-V verification working groups' published planning methodology and plan templates — large, real, openly readable vplans in spreadsheet form, useful as models for your own.
- Wile, Goss, Roesner, *Comprehensive Functional Verification* — planning at industrial scale, including the specification-review culture this chapter compresses into Section 5.4.

REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. **D21** J. Montesano and M. Litterick, "Verification Mind Games: how to think like a verifier," DVCon, 2014.
3. **D7** P. Marriott, J. Vance, and J. McNeal, "Proven Strategies for Better Verification Planning," DVCon 2022 Workshop, 2022.
4. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
5. **B5** A. Piziali, "Functional Verification Coverage Measurement and Analysis," Springer, 2004.
6. **D6** D. Graham, A. Sutton, S. Davidmann, P. Gouedo, X. Aubert, and Y. Pruvost, "Planning for RISC-V Success: Verification Planning and Functional Coverage," Proc. DVCon Europe, 2023.
7. **D9** B. Ehlers, C. Vargas, and P. Carzola, "Best Practices in Verification Planning," Proc. DVCon U.S., 2013.
8. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
9. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.

6. Coverage: Theory and Practice

The neural accelerator’s regression has run clean for two weeks: every test passes, and the code coverage report on the accelerator RTL reads 100% — every line, every branch executed. Neither fact rules out the job that breaks the accelerator, and an audio keyword-spotting model supplies it: 2-bit weights, a tensor one pixel wide, launched while the DMA engine streams camera frames through the same SRAM banks. The accelerator’s weight prefetcher, starved by the contention, wraps its buffer pointer one entry too early — but only when the weight stream ends mid-beat, which only happens when a degenerate geometry meets the narrowest precision. The output feature map is subtly wrong. Nothing in two weeks of green regressions ever created this situation, and no verification metric then in place could have revealed that it was never created: *what the tests never did* was the question none of them could answer. Code coverage said “every line executed”; it cannot say “every configuration exercised,” because the missing scenario was not a line of RTL but a *combination* — precision $\times$ geometry $\times$ contention — living in the specification, not the code structure. This chapter builds the instrument that measures exactly that, and the discipline of trusting it only as far as it deserves.

CHAPTER MAP

- §6.1 states what code, functional and assertion coverage each measure, and what each is structurally blind to.
- §6.2 sets out what each code-coverage sub-metric can see and what it cannot, and fixes the family’s role from its structural blindness: computed from the implementation against the implementation, its holes are meaningful and its fullness is not.
- §6.3 establishes the opposite economics — a model specified by hand from the specification’s intent, in the covergroup, coverpoint, bin and cross vocabulary the language provides — and why a coverpoint left to its automatic bins records something that means nothing.
- §6.4 establishes that “100% coverage” is a statement about the *model* of the design, never the design itself, and connects it to Chapter 3’s impossibility results.
- §6.5 designs a functional coverage model as a deliberate engineering act: from features to attributes to coverpoints, with bins that encode intent rather than decorate a report.
- §6.6 establishes, by setting the accelerator’s model beside a dishonest twin that reaches 100% on the same feature, that a coverage report cannot be judged without reading the model behind it.

- §6.7 states what to do when a cross explodes combinatorially: shape it in the model, with each exclusion carrying its reason in specification vocabulary, so that the reachable space is a claim a reviewer can hold and a checker can falsify.
- §6.8 sets out how closure actually proceeds: analyzing holes, choosing between targeted tests and constraint tuning, and knowing when to stop.
- §6.9 excludes what cannot or need not be covered — honestly, reviewably, and without quietly redefining success.

6.1 The coverage landscape: three families, one taxonomy

Simulation gives you an existence proof: *this* stimulus produced *this* response, and the checkers accepted it. Coverage is the complementary instrument — it records which situations your tests created, so you can reason about the ones they did not. Chapter 4 placed coverage closure in the lifecycle; this chapter opens the instrument.

Practitioners sort coverage into three families. **Code coverage** measures which structures of the RTL source — lines, branches, expressions, transitions — the simulations executed. **Functional coverage** measures which *specified behaviors* occurred, using models the engineer writes deliberately. **Assertion coverage** measures the activity of assertions: which were evaluated, triggered, completed. What separates them matters more than tooling: who defines the metric, and from what source.

Piziali's taxonomy — the founding one, and the frame this chapter builds on — makes the difference precise with two axes. A coverage metric has a *kind* — **implicit** (inherent in a representation, like the statements of a hardware description language) or **explicit** (invented and defined by the engineer) — and a *source* — the design's **implementation** or its **specification** ^[1]. The two axes define four coverage spaces: code and structural coverage are implicit-implementation; functional coverage built from the specification is explicit-specification; functional coverage of microarchitectural detail (a pipeline's occupancy, an implementation FIFO's high-water mark) is explicit-implementation; and the implicit-specification quadrant — metrics inferred automatically from a natural-language spec — had no practical implementation as of 2004 ^[1]. The taxonomy is not academic bookkeeping. It tells you, for any metric someone shows you, *whose judgment it embodies*: implicit metrics come for free and embody nobody's; explicit ones cost effort and are exactly as good as the engineering behind them. A **coverage space** is a multi-dimensional region described by attributes and their values — what the industry loosely calls a *coverage model* ^[1].

Apply the quadrants to the opening scenario and the diagnosis writes itself. The accelerator's line coverage is implicit-implementation: free, automatic, green. A model of job attributes — precision, geometry, DMA contention — would have been explicit-specification, and nobody had written one. A model of the weight prefetcher's buffer occupancy would have been explicit-implementation: also absent, and where the bug physically lives. Three quadrants were empty, and the one filled was the one that embodies nobody's judgment.

All of this is baseline practice by 2024: adoption of code coverage, functional coverage and assertions in the IC/ASIC market matured rapidly between 2007 and 2014 and has since largely stabilized within the margin of error — with the exception of constrained-random simulation, which kept climbing [2]. Coverage is no longer a differentiator; whether a team’s coverage *means* anything is. That is the rest of this chapter.

6.2 Code coverage: free, necessary, and structurally blind

Code coverage instruments the RTL and records which of its structures execute during simulation. Its purpose is refreshingly humble: to determine whether you forgot to exercise some code in the design [3]. Because the metric is implicit in the language, tools extract it automatically — no modeling effort, no judgment, no maintenance. That economy is why every flow enables it, and why every flow must know what each sub-metric can and cannot see.

Line and statement coverage report which source lines executed at least once. They catch dead configuration branches, unexercised error legs, whole features no test ever reached — a DMA error-reporting path nobody triggers shows up as a block of red lines. They catch nothing about *why* or *in what context* a line ran: a statement executed once, under the friendliest possible conditions, is “covered.”

Branch coverage reports whether each decision point took both directions. It is strictly stronger than statement coverage and still combinatorially weak, because covering decisions individually says nothing about **path coverage** — the sequences of decisions taken together. A suite can hold 100 percent statement coverage while covering only 75 percent of control paths, and paths grow exponentially with the number of control-flow statements [3]. On the DMA midend, the branch that legalizes a 4 KB-crossing transfer and the branch that handles a negative stride may each be covered while the *path* taking both in one transfer — the one hiding this book’s canonical corruption bug — was never walked.

Condition and expression coverage look inside Boolean expressions: did each controlling term independently cause the decision? Take a branch guarded by `parity == ODD || parity == EVEN`: statement coverage reads 100 percent while expression coverage sits at 50 percent, because no testbench ever set `EVEN` — an oversight statement coverage cannot report. Reaching 100 percent expression coverage is itself extremely difficult [3].

Toggle coverage reports which bits of which signals changed value (0→1 and 1→0). It is the coarsest of the family and survives because it approximates “did any activity reach this structure at all” — useful for integration sanity and power-intent connectivity — and because it is the only code metric that sees inside structures the others treat as one object.

FSM coverage extracts state machines and reports visited states and traversed arcs. The gap between the two is the lesson: a five-state machine with 14 possible transitions reaches 100 percent state coverage from a single sequence covering only

36 percent of the transitions. And because the transitions are extracted from the implementation, the tool cannot know whether an extracted transition was intended, or an intended one missing — the graph itself needs review against the specification [3]. Some transitions may be unreachable given the protocols feeding the FSM; determining which is a job for a property checker [3] — a thread Part IV picks up.

Now the structural blindness, which is why this chapter does not end here. Code coverage is computed **from the implementation, against the implementation**, so it cannot represent — even in principle — anything the designer did not write. The sharpest formulation: the code in an empty module is always 100% covered [3]. If the accelerator’s designer never implemented the 2-bit weight unpacking mode, no code metric will report it missing; every test of the modes that *do* exist passes and the report glows green. What 100% code coverage means is precisely this: the entire design implementation was executed. It provides no indication, in any way, of the correctness or completeness of the verification suite; low numbers are a reliable alarm, but a high number is by no means an indication that the job is over [3].

That asymmetry fixes code coverage’s role: a cheap automatic *backstop*, run from the first regression, whose every hole is a missing test, dead code, or a piece of the implementation nobody understood. Its holes are meaningful; its fullness is not.

Scenario. The crossbar’s ID field is 4 bits wide, and in this regression the five managers only ever index IDs 0 through 4, because the sequence library pins one ID per manager; the debug manager, ID 4, issues nothing at all. So ID bits 3 and 2 never change value. Line coverage on the ID-decode logic reads 100%: the decoder executes every cycle. Toggle coverage reads 50% — four of the eight bit transitions missing — and refuses to budge. **Approach.** The team traces both stuck bits, and the answer it reaches for first is wrong. Bit 3 looks dead by construction — five managers, none indexing past 4 — but nothing in the design says so: the field is 4 bits, AXI4 permits a manager any of the 16 values, and the interconnect widens the field precisely so that managers need not agree on which IDs they use (Arm IHI 0022H.c §A5.1 “AXI transaction identifiers”, §A5.2.3 “Interconnect use of transaction identifiers”) [4]. What the regression has established is a property of its own stimulus, not of the crossbar. Bit 3 is therefore *unreached*, not unreachable, and excluding it means recording a decision about the sequence library, with an owner and an expiry (Section 6.9) — not asserting a structural fact. Bit 2 is set only by the debug manager, whose transactions the environment never routed through the crossbar: a testbench wiring bug line coverage could not see. **Why.** Different code metrics have different blind spots, and the cheap ones earn their keep by disagreeing with each other. The second lesson is narrower and costs more when missed: the moment a hole is called *unreachable*, someone must be able to name the clause that makes it so.

6.3 Functional coverage: the metric you must deserve

Functional coverage inverts the economics. Where code coverage is inferred from the design automatically, functional coverage is specified entirely by the engineer,

from the specification’s intent, independent of the code’s structure — so it requires up-front effort: someone has to write the model [5]. That effort buys the only metric family able to measure what the opening scenario lacked: *whether the specified situations occurred*, including situations the implementation silently omits.

SystemVerilog is today’s lingua franca for these models (IEEE 1800-2023, Clause 19). A **covergroup** is the container: a user-defined type that encapsulates the specification of a coverage model, defined once and instantiated as many times as the environment needs [5]. Inside it, a **coverpoint** designates one variable or expression to observe and partitions its values into **bins** — counters, each representing a value or set of values whose occurrence you care about. A **cross** enumerates the Cartesian combinations of two or more coverpoints, because most interesting bugs — as Chapter 2’s breakage thinking predicts — live where attributes interact. Bins can also declare intent negatively: `ignore_bins` removes values from the space, while `illegal_bins` goes further — an illegal value does not merely fail to count, it raises a runtime error taking precedence over any other bin that would have claimed it [5]. Keep that asymmetry for Section 6.9: *ignore* means “not my job,” *illegal* means “must never happen” — the second is a checker wearing coverage syntax.

One language default deserves early suspicion. A coverpoint written with no bins gets them automatically: one bin per value for small domains, otherwise the range sliced uniformly into at most `auto_bin_max` bins — default 64 [5]. Automatic bins are the coverage equivalent of a meeting with no agenda: something will be recorded, and none of it will mean anything. Slicing the accelerator’s 9-bit tensor-width field into 64 uniform bins makes each bin eight values wide, so the boundary value 16 — the MAC lane count, where the behavior changes — lands in an anonymous bucket labeled `auto[16:23]`, indistinguishable from widths 17 through 23. Widths 16 and 17 are exactly the two the datapath treats differently, and the tool has just merged them. Section 6.6 returns to this as the *dishonest bins* problem.

Functional coverage then divides usefully in two: **data/stimulus coverage** — which values and configurations were processed, best captured with covergroups — and **protocol/activity coverage** — which sequences and control scenarios occurred, best expressed as `cover property` statements over temporal sequences [6]. “A 2-bit-weight job ran while the DMA contended for SRAM” is data coverage; “a write response arrived while two same-ID writes were outstanding under back-pressure” is activity coverage, and forcing it into a covergroup produces sampling contortions a three-line SVA sequence expresses directly.

Assertion coverage is the third family, and the smallest in concept: measure the assertions themselves. At minimum, which were evaluated and which vacuously satisfied (IEEE 1800-2023 §16.14.8) ^[5] — an assertion whose antecedent never occurred has verified nothing, a direct echo of Chapter 3’s observability argument. Beyond that, `cover property` directives are declared functional coverage points in their own right, and designers should mark implementation corner cases with them ^[6] so the unverified ones announce themselves rather than living in someone’s memory of the plan. The coverage of checker and coverage assertions alike is itself measurable and analyzable ^[1]. This is the connective tissue between this chapter and Chapter 11: the assertions are checkers, their coverage is the proof that the checkers were awake.

The three families are not rivals; they are three projections of the same question asked from different sources of truth. Code coverage asks the implementation, “was all of you exercised?” Functional coverage asks the specification, “did everything you promise actually happen?” Assertion coverage asks the checkers, “were you there when it did?” A verification effort needs all three, and Part IV adds a fourth voice — formal’s coverage models ^[7] — to the conversation (see Chapter 16).

6.4 What 100% means — and what it never can

Every coverage percentage is a fraction, and the denominator is the part people forget. When the accelerator’s functional coverage reads 100%, the denominator is not “everything the accelerator can do” but “everything the model’s author thought to enumerate.” Reaching 100% functional coverage indicates the completeness of the test suite *against the model* — it says nothing about the completeness of that model, and (unless bins encode invalid values) nothing about correctness ^[3].

This is Chapter 3 arriving to collect its debt. There we established that the reachable state space is finite yet unvisitable — astronomically beyond enumeration — and that exhaustive verification is therefore impossible; the honest response was to partition behavior into equivalence classes and sample each. A functional coverage model is that partition, made executable: bins are equivalence classes with a counter attached. So it inherits the partition’s fundamental property — it is a lossy, chosen projection of an intractable space. “100%” measures progress across the projection, never across the space.

The loss has a name and a measure: **fidelity**, the degree to which a coverage model captures the actual behavioral requirements. A control register with 18 specified values, modeled as 18 bins, has no abstraction gap; a 32-bit address bus whose 2^{32} values are grouped into 16 ranges has a substantial one. Fidelity is also lost by omitting *relationships*: if bit CR3 may only be 1 when CR7 is 1 and the model never encodes that dependency, it is blind to exactly the kind of coupling where bugs live ^[1]. Fidelity matters because of the bug’s *footprint* in the coverage space: some bugs occupy a huge region — any test in the right mode trips them — while others occupy a point: one instruction, one fault type, one interrupt, coincident. A high-fidelity model is far more likely to distinguish, and therefore demand, the bug-exposing scenario; a low-fidelity one files it with its harmless neighbors ^[1]. You cannot afford high fidelity

everywhere — that way lies Section 6.7’s intractable full cross — so fidelity is *spent*, like any budget, where the risk analysis of Chapters 3 and 5 says the bugs are.

One more deflation, the most important in the chapter. Coverage measures **sampling, not correctness**. It records that a situation occurred; whether the design behaved correctly in it is the business of the checkers — scoreboards, reference models, assertions (Chapters 8 and 11). Coverage methodology openly *assumes* an orthogonal mechanism detects device misbehavior ^[1]. Break that assumption and coverage becomes vanity: a covergroup counting jobs whose outputs nobody compares is a tally of unwitnessed events — Chapter 2’s unread-log problem wearing a metrics dashboard. Pin this above the desk: the actual goal of verification is to remove bugs, and coverage closure is only a well-defined and measurable *derivative* of that goal ^[8]. A derivative can be optimized directly, and that is the pathology: stimulus tuned to tickle bins while checkers sleep converges on 100% faster than honest verification, having discarded the part that was slowing it down. The cure is structural, not moral: review coverage models *together with* the checkers watching the same events, and make “who detects it if this scenario misbehaves?” a mandatory review question for every coverpoint.

6.5 Designing the coverage model: an engineering act

A coverage model is not a transcription task. Chapter 5 ends with a verification plan: features extracted from the specification, each with a measurable pass/fail intent. This chapter’s job begins there — turning a feature into attributes, bins, crosses and sampling events a simulator can count. The work has two phases: **top-level design**, which writes the model’s semantic description and identifies the attributes and their relationships, and **detailed design**, which answers *what* to sample, *where*, and *when* to sample and correlate attributes ^[1]. Writing the semantic description first — one sentence of the form “record all valid operating conditions of X defined by attributes A, B, C” ^[1] — forces the author to say out loud what the model claims to measure, the only thing a reviewer can hold it to.

The judgment concentrates in three places.

Choosing attributes. Attributes come from the specification’s parameters (the accelerator’s weight precision), from the environment’s degrees of freedom (DMA contention), and — with restraint — from the implementation’s own choices (the prefetch buffer’s occupancy). The first two make specification coverage; the third is implementation coverage in Section 6.1’s taxonomy, legitimate but owned by a different question (“did we stress this microarchitecture?”) ^[1]. An attribute no checker observes and no risk motivates does not belong in the model.

Choosing bins. Bins are where breakage thinking (Chapter 2) becomes code. The question is never “what values can this field take?” but “at which values does the *behavior* change?” — boundaries, alignment points, wrap-arounds, the value equal to a hardware resource count, the value one past it. Uniform slicing is what the tool does when the engineer abstains [5]; deliberate bins are unequal by design, dense where behavior bends and coarse where the specification is indifferent. If the only honest name for a bin is `auto[128:191]`, it is not measuring anything anyone asked for.

Some cliffs sit in the data rather than the configuration. On a video codec the control flow bends on the picture itself — an all-skip macroblock, a motion vector pinned at the search-window limit, a coefficient block that trips escape coding — so the monitor must *compute* the bin from the stream rather than read it from a register. Those are exactly the frames on which a bit-exact reference model diverges.

Choosing sampling. *Where* determines truth: sample attributes at the interface or unit whose behavior they describe — job attributes at the accelerator’s register interface, contention where the SRAM arbitration happens — not wherever a handle was convenient. *When* determines meaning: sample when the values are jointly valid and stable. The accelerator model below samples at job completion with the contention flag *latched during* weight streaming; sampling at job launch would record whether the DMA happened to be busy at an irrelevant instant, and every bin it filled would be a small lie. Correlation time is a first-class design decision [1].

Scenario. The DMA vplan hands over feature DMA-F06a (Chapter 5): “2D transfers are legalized correctly across the 4 KB AXI boundary for all stride signs.” The boundary the row names is AXI4’s, Arm IHI 0022H.c §A3.4.1 “Address structure” [4]. Its closure metric is the covergroup `cg_2d_4kb`. **Approach.** Every bin sits on a behavioral cliff rather than a round number: stride sign — `pos`, `zero`, `neg` (**3** bins); the distance from a row’s start address to the next 4 KB boundary, classified against the burst size — `gt_burst`, `eq_burst`, `lt_burst`, `straddle` (**4** bins); burst length at its extremes — 1, 255, 256 (**3** bins); and whether the other DMA channel is active (**2** bins, one per value of a one-bit flag — the one place automatic bins tell the truth). The cross is $3 \times 4 \times 3 \times 2 = 72$ combinations, each nameable in specification vocabulary. **Why.** This book’s canonical DMA bug lives in the `neg` $\times$ `straddle` $\times$ other-channel-active corner — two of those 72 points, not three: the generator’s solver budget caps a row at 256 beats, so a straddling row splits into pieces none of which can be the 256-beat maximum — which makes that point of the corner unreachable by a choice the team recorded rather than by luck of stimulus, and so a waiver to write rather than a hole to chase (Section 6.9). A model with those bins *demand*s the bug’s scenario; 64 automatic bins over the stride field would have filed it with the crowd.

6.6 Worked example: a coverage model for the neural accelerator

The accelerator’s canonical parameters (Chapter 1): a fixed-point convolution engine with configurable 2/4/8-bit weights, a 16-lane MAC datapath, weights streamed from

memory, jobs programmed through registers. The vplan feature: *quantized convolution jobs execute correctly for all weight precisions and tensor geometries, including under memory contention with the DMA engine.*

Top-level design first. Semantic description: *record every completed convolution job as the combination of weight precision, tensor width class, input channel class and whether the DMA contended for the weight-stream SRAM banks during execution.* All four attributes correlate at one instant, job completion:

Table 6.1 — The four attributes of the accelerator's job coverage model, with where each value is read from and when it is captured. Three are read off the job registers at launch and the contention flag is latched across the weight-streaming window, so that all four are jointly valid at a single sampling instant, job completion — the choice §6.5 calls the one that determines meaning.

Attribute	Source	Sampled
weight precision (2/4/8-bit)	job registers	at job launch
tensor width (1-256)	job registers	at job launch
input channels (1-512)	job registers	at job launch
DMA contention	SRAM arbiter	latched during weight streaming

Detailed design, in SystemVerilog:

```
typedef enum logic [1:0] {PREC_2B, PREC_4B, PREC_8B} prec_e;

typedef struct packed {
    prec_e      prec;      // weight precision
    logic [8:0]  tensor_w; // tensor width in pixels, 1..256
    logic [9:0]  in_ch;    // input channels, 1..512
} job_attr_t;

covergroup cg_accel_job with function sample(job_attr_t job,
                                             logic dma_contended);

    option.per_instance = 1;

    // Enum: automatic bins give one named bin per precision -- acceptable
    // here because every enum value is individually meaningful.
    cp_prec : coverpoint job.prec;

    // Width bins follow the datapath, not the number line: behavior bends
    // at the 16-lane boundary and at the extremes.
    cp_width : coverpoint job.tensor_w {
        bins single      = {1};           // degenerate geometry
        bins sub_lane    = {[2:15]};      // partial lane utilization
        bins lane_exact  = {16};          // exactly one full MAC pass
        bins lane_plus1  = {17};          // full pass + 1-pixel remainder
        bins multi_lane  = {[18:255]};    // steady-state streaming
        bins max_width   = {256};          // architectural maximum
        // Legal range is 1..256 in a 9-bit field: the register checker must
        // reject the rest, so reaching the datapath is a bug, not a hole.
        illegal_bins out_of_range = {0, [257:511]};
    }

    cp_ch : coverpoint job.in_ch {
        bins one_ch      = {1};
        bins few_ch      = {[2:15]};
    }
}
```

```

    bins accum_16 = {16};           // accumulator reuse boundary
    bins many_ch  = {[17:511]};
    bins max_ch   = {512};
    // Legal range is 1..512; same contract as cp_width above.
    illegal_bins out_of_range = {0, [513:1023]};
}

cp_dma : coverpoint dma_contended {
    bins quiet      = {1'b0};
    bins contended  = {1'b1};
}

// The cross that would have caught the opening scenario:
// precision x width-class x contention = 3 x 6 x 2 = 36 combinations.
x_prec_width_dma : cross cp_prec, cp_width, cp_dma;

// Channels interact with precision (accumulator packing), not with
// contention: a second, smaller cross instead of one giant one.
x_prec_ch : cross cp_prec, cp_ch;
endgroup

```

Read the model as an argument, because that is what it is. The width bins encode a claim: the datapath treats widths 18 and 200 equivalently (both steady-state multi-lane streaming) but treats 16 and 17 as different worlds — the reviewer can agree or produce a counterexample from the spec. The `illegal_bins` encode a claim of a different kind: those values must never reach the datapath, so if one does the simulation stops rather than quietly counting it (Section 6.9). Illegal values are excluded from coverage [5], so `cp_width` still offers 6 bins and `cp_ch` 5. The crosses encode a second claim: precision interacts with geometry and with contention (the weight unpacker’s timing depends on both), while channel count interacts with precision only — so instead of one cross of $3 \times 6 \times 5 \times 2 = 180$ combinations we maintain 36 ($3 \times 6 \times 2$) + 15 (3×5) = 51, each defensible. The sampling encodes a third: `dma_contended` is latched across the whole weight-streaming window and sampled once, at job completion, when all four attributes are jointly valid. The opening scenario’s bug lives in bin `x_prec_width_dma: (PREC_2B, single, contended)` — a bin this model *demands*, and without which no green report is possible.

Now the counterexample. Here is the same feature “covered” dishonestly:

```

covergroup cg_accel_job_vanity with function sample(job_attr_t job,
                                                    logic dma_contended);
    cp_prec  : coverpoint job.prec;
    cp_width : coverpoint job.tensor_w {
        bins lo_half = {[1:128]};
        bins hi_half = {[129:256]};
    }
    cp_dma   : coverpoint dma_contended;
    // no cross
endgroup

```

Everything else is held constant — same attributes, same sampling call, same instant — so the sins here are exactly the two this model commits. This `covergroup` compiles, runs and reaches 100% before the first coffee of the first regression, and every one of its numbers is a small deception. `lo_half` lumps the degenerate width 1, the

lane boundary 16 and the remainder case 17 into one 128-value bin: an abstraction gap that files the bug-exposing scenarios with their harmless neighbors ^[1]. And there is no cross, so “2-bit weights” and “contention” are each ticked without ever having occurred *together*.

A third way to fake it needs no change to the bins at all: keep the honest model and replace its `sample()` call with `@(posedge clk iff job_done)`, reading contention live at that edge — which measures whether the DMA happened to be busy the instant the job retired, not whether it contended during streaming, and fills the bin either way. Nothing in the report distinguishes any of these from the honest model. That is the uncomfortable truth about functional coverage: the metric is exactly as good as the model, and the model can only be judged by *reading it* — which is why coverage model review, against the vplan and the checkers, is a gate in every serious flow.

6.7 When the cross explodes: shaping the crossbar model

Crosses are where coverage models die of ambition. Take the crossbar: 5 managers (core-I, core-D, two DMA channels, debug) $\times$ 4 subordinates (SRAM, boot ROM, APB bridge, the accelerator). Route coverage alone — did every manager reach every subordinate? — is 20 combinations, trivially worth having. Now add what protocol verification cares about: burst type (FIXED, INCR, WRAP: 3), direction (2), a modest 6 buckets of burst length, 4 transfer sizes on the 64-bit data bus. The full cross is $5 \times 4 \times 3 \times 2 \times 6 \times 4 = 2,880$ combinations; fold in the 16 transaction IDs and it is 46,080. Long before that the model has stopped being an argument anyone can review — and many of its bins are *unreachable by construction*: the instruction port never writes, the boot ROM accepts no writes, the debug manager issues only single-beat INCR accesses.

Unreachable cross bins are not harmless padding. They make 100% unattainable, which forces late bulk exclusions, which destroys the audit trail — or they normalize a dashboard that reads 61% forever, teaching everyone to stop looking (Chapter 7 returns to that pathology). The engineering answer is the **shaped cross**: state the reachable space explicitly, in the model, with the reasons attached.

```
typedef struct packed {
    logic [3:0] mgr_id;    // 5 managers, IDs 0..4
    logic [1:0] sub_id;    // 4 subordinates
    logic [1:0] burst;     // AxBURST
    logic      is_write;   // direction
} xbar_txn_t;

covergroup cg_xbar_route with function sample(xbar_txn_t t);
    cp_mgr : coverpoint t.mgr_id {
        bins core_i = {0}; bins core_d = {1};
        bins dma[]  = {[2:3]}; bins dbg = {4};
    }
    cp_sub : coverpoint t.sub_id {
        bins sram = {0}; bins rom = {1}; bins apb = {2}; bins accel = {3};
    }
    cp_burst : coverpoint t.burst {
        bins fixed = {2'b00}; bins incr = {2'b01}; bins wrap = {2'b10};
    }
endcovergroup
```

```

    // 2'b11 is reserved in AXI4: a manager must never issue it.
    illegal_bins reserved = {2'b11};
}
cp_dir : coverpoint t.is_write {
    bins rd = {1'b0}; bins wr = {1'b1};
}

x_route : cross cp_mgr, cp_sub, cp_burst, cp_dir {
    // Instruction port is read-only by construction.
    ignore_bins ifetch_wr = binsof(cp_mgr.core_i) && binsof(cp_dir.wr);
    // Debug manager issues only INCR accesses.
    ignore_bins dbg_burst = binsof(cp_mgr.dbg) &&
        (binsof(cp_burst.fixed) || binsof(cp_burst.wrap));
    // Writes to the boot ROM must be rejected: observing the error
    // response is a required event, so it lives in its own
    // error-response coveragegroup rather than as a hole in this cross.
    ignore_bins rom_wr = binsof(cp_sub.rom) && binsof(cp_dir.wr);
}
endgroup

```

Each `ignore_bins` line ^[5] carries its justification as a comment, in spec vocabulary, reviewable in the same diff as the model. The model’s reserved `AxBURST` encoding is the protocol’s own: Arm IHI 0022H.c §A3.4.1 “Address structure”, Table A3-3 “AxBURST signal encoding”, page A3-50 ^[4]. The shaped cross states the *intended* reachable space, and that statement is falsifiable: if a “read-only” instruction port ever does produce a write, the protocol checkers scream, which is where that requirement belongs. Note the boot-ROM line — shaping split one lump into an ignored stimulus combination *and* a positively-required error-response coverage point elsewhere. Shaping is not deletion; it is relocation of intent to where it can be honestly measured.

Count what shaping bought. Unshaped, this cross is 5 managers $\times$ 4 subordinates $\times$ 3 burst types $\times$ 2 directions = **120** combinations. The exclusions remove 12 (`ifetch_wr` = $1 \times 4 \times 3 \times 1$), 16 (`dbg_burst` = $1 \times 4 \times 2 \times 2$) and 15 (`rom_wr` = $5 \times 1 \times 3 \times 1$) — 43 in all, but `ifetch_wr` and `rom_wr` overlap in 3, `dbg_burst` and `rom_wr` in 2, and the first two are disjoint, so their union is 38. What remains is **82** defensible combinations rather than 120 — in a model that never had to grow to the 2,880 of the full six-axis cross. That is a model a reviewer can hold in their head, the real tractability criterion. On the compute cluster’s 8-core interconnect or the mesh NoC the discipline is the same: cross only the two- and three-way interactions the risk analysis names, and let assertions own the rest.

On the GPU the axes stop being structural altogether. A shader cluster’s interesting space is a space of *occupancy shapes* — thread groups in flight $\times$ divergence class $\times$ memory-access pattern — a cross small enough to write down in full. The difficulty is that which of its cells are reachable is a property of the workload rather than of the configuration: a kernel without branches sits in one divergence bin for its whole run, and no biasing of the generator will move it out of that bin. So the model must be crossed against workload class, and a hole then reads “we have never run a shape of program that could produce this”, not “we have never randomized into it” — a distinction that changes who fixes it.

Past a certain point shaping stops being enough and the metric changes engine. On a directory-based coherent hub the interesting space is not routes but *protocol states* — combinations of MESI state, directory entry and outstanding snoop across several caches, a reachable set nobody enumerates by hand. The question becomes “over which combinations did we prove the invariant, and what did the assumptions exclude?” — formal’s own notion of a coverage model [7], and Chapter 16’s subject.

6.8 Closure: the workflow from holes to done

Coverage closure is where the metric meets the schedule: it is reached when the entire coverage space spanned by the model has been observed [8] — with, in any honest flow, the exclusions accounted for alongside. Chapter 4 placed it on the lifecycle timeline; Figure 6.1 draws the loop itself. Traditionally it is manual: run the regression across seeds, merge the coverage databases, analyze what is still empty, modify constraints or write directed tests, rerun — until the target is reached, the merge-and-analyze step alone consuming significant time and compute [9]. Coverage also accumulates across simulators, emulators and formal tools, and a standard API and interchange format exists to keep it mergeable across dynamic and static tools and across a project’s lifetime [10] (Chapter 16 covers unified coverage in depth).

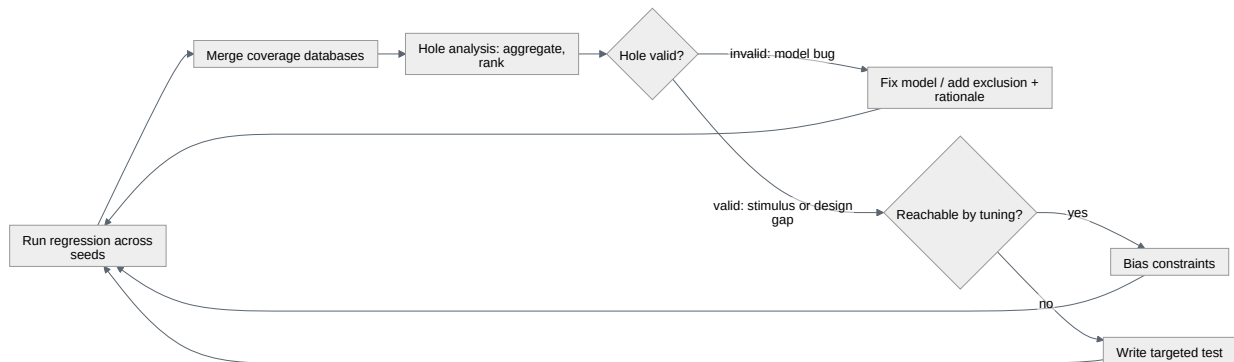


Figure 6.1 — The coverage-closure loop, from a seeded regression through merge and hole analysis to the action each hole earns. The branch that carries the discipline is the validity question: an invalid hole is a defect in the model, answered with an exclusion and its rationale — the path §6.9 exists to keep honest — while a valid hole is answered with biased constraints or a targeted test. Every branch returns to the regression, so nothing is closed without a rerun.

Reading holes. A coverage hole is required stimulus or device behavior not yet observed [1]. Raw hole lists are stupefying; the analysis techniques, first reported by IBM’s Haifa laboratory in 2002, compress them into meaning: **aggregation** coalesces many point-holes into few larger regions; **partitioning** groups holes that share semantics; **projection** finds attribute values *never* observed in any combination; and Hamming distance — the number of attribute values in which two holes differ — measures proximity [1]. Fifty scattered holes that aggregate into “no 2-bit-weight job has ever run under contention, at any geometry” become one finding, one root cause, one action. A projected hole outranks any single missing cross product: it says a whole dimension of the stimulus is broken.

Classifying holes. Every hole is one of two things, and the distinction drives everything downstream. A **valid hole** is an intended observation that has not happened — on the input side, usually a stimulus weakness; on the output or internal side, possibly a design error suppressing the behavior. An **invalid hole** is a bug in the coverage model itself: a combination that cannot or should not occur, written into the model by misunderstanding — and its fix is a restriction on the model (an exclusion with a reason), not a test ^[1]. Section 6.9 is about keeping that second path honest.

Classification also decides what the stimulus must be *capable* of. On an inline AES-GCM crypto engine, a hole in the failed-authentication bin is never a constraint-tuning problem: the environment must be able to corrupt a tag after the engine has computed it, truncate a ciphertext mid-stream, or present a key slot that does not match the one the message was encrypted under — manipulations a well-behaved driver has no path to perform at any seed. If it cannot, the hole marks a gap in the testbench’s capability, not in its randomization. Which fault classes were injected is a coverage model in its own right — one teams routinely forget to write, then find missing at sign-off.

Filling holes: tune or target. The first lever is **generation feedback**: bias the constraint distributions so the generator’s probability mass lands on what is missing — input holes respond directly; output and internal holes must first be traced back to the input scenario parameters that provoke them ^[1]. The second is the **targeted test**: encode the missing scenario directly. Tuning is cheap per hole and preserves the randomness that finds unplanned bugs (Chapter 9); targeting is precise but brittle. Mature teams tune while holes come in families and target when the survivors become singular — knowing the endgame inverts the economics: as only hard-to-reach points remain, progress decelerates sharply ^[3], and as of 2012 manually closing the last 20% of coverage costs up to 80% of a verification team’s time and resources ^[11].

That cost is why closure automation is a research and product area rather than a convenience. The published spectrum, in increasing ambition: automated constraint regeneration, where uncovered bins are read back from the merged database and turned mechanically into constraints for the next run ^[9]; solver-level distribution, where the constrained-random engine distributes its solutions to satisfy the declared coverage model, converging measurably faster than undirected constrained random ^[8]; and learning-based coverage-directed generation — Bayesian networks, Markov models, inductive logic programming, evolutionary methods — which learns the mapping from generator directives to achieved coverage and inverts it ^[11]. One instance: a deep reinforcement-learning agent generating stimulus on the fly for a compression-encoder DUT hit 133 of 136 hard functional bins in 500 training episodes where uniform-random stimulus plateaued at 28 ^[12]. Chapter 25 asks when such techniques transfer and when they are demo-ware; the structural point belongs *here*: every one optimizes the derivative, not the goal ^[8]. Automation makes Section 6.4’s checking question *more* urgent — a machine-optimized 100% arrives faster and with even less human contact with the scenarios.

That last family has an older and better-attributed instance than the reinforcement-learning one. On the Storage Control Element of an IBM z-series system, a Bayesian network trained on the correlation between test-generator directives and observed coverage synthesized new directive files and closed all 223 tasks that a 160-test training set had left uncovered, within 250 further test-cases, where an expert's hand-written directive file reached only two-thirds of them in over 400 [13]. A different lever acts on the regression rather than on the generator: instead of biasing what is produced, learn which already-generated tests are worth simulating. On the Radar Signal Processing Unit of Infineon's AURIX TC3XX — a large, highly configurable block whose tests are described by 300 configuration fields — classifiers trained on previously simulated runs nearly closed the same coverage model while simulating 18% fewer tests [14].

When to stop. Stop when the *reviewed* model is full, the exclusion file is current, and the closure history is boring — no model refinements for a stable period. Note what evolves during closure: not only the stimulus but the model. **Coverage model feedback** is part of the method: models start exploratory — coarse, low-fidelity, goals around 80% — and are refined toward production models with goals of 95 to 100 percent as understanding matures [1]; discovering mid-closure that a model needs new attributes is the process working, not failing. “100% and holding, on a model that stopped changing, with checkers that were awake” is the honest sentence. What it feeds is Chapter 7's subject.

6.9 Honest exclusions: the waiver as a reviewable artifact

Every real project excludes coverage. The crossbar's instruction-fetch port never issues a write, by construction of the port; the accelerator's register file has reserved fields; parameterized RTL carries branches this configuration can never take. Exclusion is not the failure mode. *Unreviewable* exclusion is: the bulk waiver applied at 11 pm before a milestone, the exclusion file nobody diffs, the tool-generated “excluded: unreachable” annotation that quietly includes twelve bins that were merely hard.

The discipline has four rules, none of them expensive:

1. **Every exclusion carries its reason, inline.** Not “waived” but “instruction port is read-only by construction — see integration spec section 3.2.” Section 6.7's shaped cross shows the pattern: the justification travels in the same artifact as the exclusion, so the review is a code review.
2. **Exclusions are classified by kind**, because kinds differ in owner and expiry: *structural* (tied parameters — expires if the parameterization changes), *specification* (combinations the spec forbids — expires if the spec revs), and *risk-accepted* (reachable but not worth the cost — expires never, which is why it needs the most senior signature). The third is the honest name for what bulk waivers hide.
3. **The model distinguishes “not my job” from “must never happen.”** Excluding a forbidden combination with `ignore_bins` rather than `illegal_bins` [5] is a double loss — the space shrinks *and* the watchdog is dismissed. The invalid-hole rule of Section 6.8 points the same way: a hole caused by a coverage-model mis-

understanding becomes a restriction on the model, stated as such ^[1] — and if the underlying condition is one the design must actively reject, the restriction belongs in a checker too.

4. **Exclusions are versioned and re-derived, not archaeological.** The exclusion file lives with the RTL and the vplan, is re-audited when either changes, and appears in the sign-off review as a first-class document. For code coverage, formal unreachable analysis can *prove* many structural exclusions rather than asserting them (see Chapter 15) — the strongest possible answer to “who says this is unreachable?”

The test of an exclusion regime is the auditor’s question: for any uncovered bin, can the team produce within a minute either a plan to cover it or a written, attributed, still-valid reason not to? Anything else is 100% by redefinition — moving the goalposts and mowing the old penalty box.

IN PRACTICE

Real closure looks like this. Code coverage is on from the first smoke test, but nobody triages it until the design stabilizes — early holes are churn. Functional coverage models are written alongside the testbench (writing them *after* stimulus exists invites modeling what the stimulus already does — measuring the tests with the tests), and they earn a formal review against the vplan and the checker inventory: every coverpoint names the feature it measures and the checker that catches misbehavior in it. During the long middle, the regression dashboard merges code, functional and assertion coverage across the farm, which the cross-tool interchange format enables ^[10]; a weekly triage meeting walks the aggregated holes, not the raw bin list, assigning each family a verdict: tune, target, fix the model, or exclude with a reason. Teams that skip the verdict step meet it later as an undifferentiated 4% nobody can explain at sign-off. The endgame is a grind and is staffed as one ^[11]: the last holes are singular, expensive and frequently *informative* — a hole that resists three targeted tests is, disproportionately often, sitting on top of a bug or a spec misunderstanding. And the messy truth: every real project ships with an exclusion file that was argued about, and the arguing is the value. A waiver nobody contested is a waiver nobody read.

PITFALLS

1. **Treating 100% code coverage as done.** Symptom: green structural metrics presented as verification quality. Cure: code coverage gates the *floor* (holes must be explained); functional coverage against the vplan is the ceiling ^[3].
2. **Automatic bins on meaningful fields.** Symptom: bins named `auto[112:127]` in the report. Cure: deliberate bins named in spec vocabulary; auto bins only where every value is equally (un)interesting ^[5].

3. **Coverage without checkers.** Symptom: coverpoints sampling events no scoreboard or assertion observes. Cure: ask “who fails if this sampled scenario misbehaves?” of every coverpoint [1, 8].
4. **The full cross.** Symptom: one cross with thousands of mostly-unreachable bins, stuck at 61% forever. Cure: shaped crosses with inline-justified `ignore_bins`; several small defensible crosses over one aspirational one.
5. **Sampling at the wrong moment.** Symptom: bins fill but targeted replays of “covered” scenarios find nothing resembling them. Cure: sample when attributes are jointly valid; latch condition flags across the window they describe.
6. **Bulk late waivers.** Symptom: coverage jumps 8% the week before sign-off on a single exclusion-file commit. Cure: exclusions classified, reasoned, attributed and reviewed as code — no orphan waivers.

SUMMARY

- Code, functional and assertion coverage answer different questions from different sources of truth — implementation, specification, checkers — and none substitutes for another [1].
- Code coverage is automatic and structurally blind: it can prove you are not done, never that you are; an unimplemented feature is invisible to it [3].
- A functional coverage model is an executable set of equivalence classes (Chapter 3): “100%” measures progress across *your model* and says nothing about that model’s completeness or fidelity [1, 3].
- Coverage measures sampling, not correctness: closure is a measurable derivative of bug-removal and can be gamed precisely because it is one — checkers keep the derivative attached to the goal [1, 8].
- Model design is engineering: attributes from the spec, bins at the behavioral cliffs, crosses shaped to the reachable space, sampling when attributes are jointly valid [1].
- Closure is a loop — merge, analyze, classify, tune or target — whose endgame is expensive enough (up to 80% of effort for the last 20%) to justify automation, which raises the stakes on honest checking [9, 11].
- Exclusions are legitimate only as reviewable artifacts: classified, reasoned, attributed, versioned — never a substitute for a checker on forbidden behavior [1, 5].

FURTHER READING

Corpus sources. Piziali’s *Functional Verification Coverage Measurement and Analysis* [1] is the theory backbone — taxonomy, model design method, fidelity and hole analysis in one place. Bergeron’s *Writing Testbenches using SystemVerilog* [3] gives the sharpest short treatment of what 100% does and does not mean. The VMM book [6] codifies the data-versus-activity split and the coverage/assertion guidelines. IEEE 1800-2023 Clause 19 [5] is definitive for covergroup, bin, cross and option semantics; the UCIS standard [10] for coverage interchange (with Chapter 16). On closure auto-

mation: a review of ML-based coverage-directed generation [11], automated constraint regeneration [9], solver-level coverage-driven distribution [8], deep-RL closure [12], and coverage models for formal verification [7].

Non-corpus references (not citable above, listed for the curious): the *Coverage Cookbook* (practical recipes for model construction); O. Lachish, E. Marcus, S. Ur, A. Ziv, "Hole Analysis for Functional Coverage Data," DAC 2002 — the original hole-analysis paper; H. Foster, A. Krolnik, D. Lacey, *Assertion-Based Design* — the companion discipline to Section 6.3's assertion coverage.

REFERENCES

1. **B5** A. Piziali, "Functional Verification Coverage Measurement and Analysis," Springer, 2004.
2. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
3. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
4. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
5. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
6. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
7. **D11** X. Feng, X. Chen, and A. Muchandikar, "Coverage Models for Formal Verification," Proc. DVCon U.S., 2017.
8. **D10** M. Teplitsky, A. Metodi, and R. Azaria, "Coverage Driven Distribution of Constrained Random Stimuli," Proc. DVCon U.S., 2015.
9. **D15** M. Kodi, S. S. Patil, and R. Nair, "Fully Automated Functional Coverage Closure," Proc. DVCon U.S., 2019.
10. **S3** Accellera Systems Initiative, "Unified Coverage Interoperability Standard (UCIS), Version 1.0," Accellera, June 2012.
11. **P19** C. Ioannides and K. I. Eder, "Coverage-Directed Test Generation Automated by Machine Learning – A Review," ACM Transactions on Design Automation of Electronic Systems, vol. 17, no. 1, Article 7, 2012.
12. **D1** E. Ohana, "Closing Functional Coverage With Deep Reinforcement Learning: A Compression Encoder Example," Proc. DVCon U.S., 2023.
13. **P16** S. Fine and A. Ziv, "Coverage Directed Test Generation for Functional Verification using Bayesian Networks," Proc. 40th Design Automation Conference (DAC '03), Anaheim, CA, pp. 286–291, June 2003.
14. **P4** N. Masamba, K. Eder, and T. Blackmore, "Supervised Learning for Coverage-Directed Test Selection in Simulation-Based Verification," arXiv preprint arXiv:2205.08524v3, October 2022.

7. Metrics-Driven Sign-off

Two weeks before tape-out of the reference SoC, eleven months into its verification campaign, the project has to answer a one-line question about every block on the die: is it done? Intuition and the instruments do not answer it the same way. Intuition ranks the neural accelerator as the risk, because it is the newest block and the team verifying it is the least experienced, and the DMA engine as safe — quiet for three weeks, no new failures, the regression dashboard a wall of green. The metrics review says almost exactly the opposite. All the accelerator’s plan rows are discharged but one — a contention cross its block-level bench cannot reach; its coverage is otherwise closed, and its bug discovery curve flattened *while* stimulus was still being added. The DMA’s curve flattened too — but in the same three weeks, no new stimulus went in, the last severe bug arrived late and in a corner nobody had modeled, and the coverage crosses around that corner sit at 71%. Intuition was measuring team seniority and recent silence; the instruments were measuring the project, and the instruments were right. This chapter is about building those instruments, reading them honestly, and turning them into the one decision that cannot be un-made: sign-off.

CHAPTER MAP

- §7.1 establishes why metrics — not intuition — are the only shared truth a verification project of SoC scale can have, and what that truth costs to maintain.
- §7.2 sets out what to instrument: per-feature status, coverage trends, bug curves, regression health, and where the engineers’ time actually goes.
- §7.3 reads a bug-rate curve correctly, and in particular tells *flat because clean* from *flat because the stimulus stopped asking new questions*.
- §7.4 shows how a sign-off checklist aggregates evidence — plan, coverage, static and formal results, bug dispositions, reproducibility — into a defensible decision.
- §7.5 shows the checklist applied under real pressure, where identically flat bug curves carry a different verdict for every block in the room — signed off, conditionally signed off against a dated waiver, and held.
- §7.6 establishes how escape analysis turns escaped bugs into the next project’s plan, without turning the post-mortem into a trial.

7.1 Metrics as the project’s shared truth

Chapter 4 introduced sign-off as the lifecycle’s last gate: a review that judges evidence against the plan (see Chapter 4). This chapter is about the evidence itself — where it comes from, how it can lie to you, and how to assemble it into a decision. The discipline has a name, **metrics-driven verification (MDV)**: run the project on quantitative measurements collected continuously from the verification process,

rather than on status estimates collected from people. MDV is today the universally accepted baseline methodology for verification management ^[1]; the interesting questions are no longer *whether* to measure but *what*, and how to keep the measurements honest.

The case for metrics is a case against intuition — but a precise one. Gut feel, impressions and intuition genuinely work on small projects, where a few people can hold the entire design in their heads; on an SoC integrating many complex IP blocks, no single person has a view that spans the project, and the intuition of one or even several people may stop portraying its real state ^[2]. The reference SoC sits exactly at that threshold: ten to fifteen engineers, each knowing one or two blocks deeply and the rest by rumor. The intuition that ranked the DMA as safe was not wrong out of carelessness — it was wrong *structurally*, because it was one hop from the work, and one hop is enough. In large environments, what is not measured is not known ^[2].

Two corollaries follow, and both are warnings.

First, **no single metric portrays the project**. Every measurement gives one view with significant limitations; only a wide range of metrics, planned together and *correlated* during analysis, builds a picture that is not distorted by the drawbacks of any one of them ^[2]. Coverage percentage without the bug curve rewards stimulus that tickles bins without checking anything. A green pass rate without coverage rewards a regression that stopped asking hard questions. A falling bug count without regression health may just mean the regression is broken. Sign-off is precisely the act of reading these instruments *against each other*.

Second, **metrics are expensive, so measure only what you will act on**. Functional metrics must be architected, written and maintained like any other verification code, and their simulation and maintenance cost is real ^[2]. A coverage metric nobody looks at after each regression is probably irrelevant, and reaching 100% on an irrelevant metric may look like progress but is not productive work — data is not information ^[3]. The pathology has a name, the **metrics gap**: a metrics explosion in which teams instrument for the sake of instrumenting and lose track of what matters and what does not ^[4]. The test for every proposed metric is one question: *what decision changes if this number moves?* If nothing changes, do not build it.

One reframing before the instrument panel. Metrics feel like bureaucracy to engineers and like control to managers; they are neither. They are how a distributed team maintains **shared, defined and agreed goals** — a common understanding of what “done” means, fixed when the plan is written and tracked automatically from then on ^[5]. With an executable plan (see Chapter 5), each row links to the coverage points, checks and results that discharge it, and status is *computed*, not reported — which is what makes the sign-off meeting of Section 7.5 a review of evidence rather than a negotiation between personalities.

7.2 What to instrument

A verification management platform — whatever its implementation, from a commercial tool to an in-house database — exists to answer, at any moment and without a meeting: *what have we verified, what is left, and is the machine that produces the evidence healthy?* Modern practice organizes the collection around a closed regression loop — plan, run, collect results, triage failures, merge coverage, analyze against the plan, react — with every lap of the loop depositing data [5]. One team rebuilt that entire stack in-house around a single golden coverage model, on the argument that the loop — not any one tool — is the methodology [1]. That team was Graphcore, and it measured the payoff: with the coverage model pulled out of SystemVerilog into an in-house Python implementation, coverage could be collected in an offline generation step with no simulator at all, cutting collection over the same 200 seeds from 40,493 s to 3,718 s — a 10.9× speedup its authors qualify carefully, since a model that is not timing-accurate can carry only coverage that does not rely on cycle-accurate information [1]. Out of the many things such a loop *can* record [2], five families earn a permanent place on the project dashboard.

1. Per-feature status. The master instrument, because it is denominated in the only currency that matters: the plan. For each vplan row — feature, its checks, its coverage, its closure evidence (see Chapter 5) — the dashboard shows discharged / in progress / blocked / waived. Requirements traceability runs through this instrument: from specification requirement to plan row to coverage point to test result, so that a change in the specification is visible as a hole in the evidence, automatically [5]. Everything else on the dashboard is a *leading indicator*; per-feature status is the truth the others approximate. Which indicator best approximates it is a per-block judgement: on a flash controller, whose job is surviving corrupted input, that role falls to error-injection coverage — which of the planned bit-error patterns, burst lengths and worn-block conditions the regression actually injected and checked against the data-integrity oracle. A coverage percentage blind to the injection axis is nearly mute about that block.

2. Coverage trends. Not the coverage *number* — the coverage *curve*: percentage over time, per block and per coverage category, merged across simulation, formal and any other engine that discharges plan items [5]. A single snapshot (“the DMA is at 96%”) is nearly information-free: the same number is alarming if it was 96% three weeks ago with stimulus running since, encouraging if it was 78% last Monday. Plotted over time, coverage stops being a scoreboard and becomes a trajectory [2]. Chapter 6 owns what the coverage should *be*; this chapter cares how it *moves*.

3. Bug curves. Bug-open and bug-closure rates plotted over time are among the simplest trend reports in the discipline [2], and Section 7.3 is devoted entirely to reading them, because they are also the most misread instrument on the panel.

4. Regression health. Pass rate over time, per block and per configuration; but also the health of the machine itself: turnaround time from check-in to result, farm utilization, simulation time per block — and anomalies in these. A sudden spike in a block’s regression runtime is actionable the week it appears (a recent change made

the block pathologically slow) and archaeology a month later [2]. Regression health also includes *test effectiveness*: ranking tests by coverage contributed and bugs found, so that the regression can be reordered or re-weighted instead of growing monotonically until it no longer fits in a night [2]. Late in a project, pruning the *coverage collection* pays too: the overhead scales roughly linearly with the number of bins, and fully-hit crosses keep charging it every run — one team cut a seeded regression from 46.6 to 38.2 hours of compute by dropping coverpoints already hit in the previous run, at the cost of slightly lower reported coverage on the pruned points [1]. That trade — compute bought with a bounded, deliberate loss of measurement — belongs in the sign-off record, not in a script nobody reads. Chapter 12 treats the regression factory in depth.

5. Debug-time share. The quiet one — in practice, the instrument most often missing from a dashboard. In the 2024 industry data, verification engineers spend more of their time on debug than on any other activity, and debug effort is unpredictable and varies significantly between projects [6, 7]. That variance is why extrapolating debug effort from the last project is, in practice, the estimation method that fails most reliably (see Chapter 4). Track where the hours go — even coarsely, from triage annotations in the bug tracker — and you gain the one early-warning signal the other four instruments miss: a project whose *evidence* metrics look flat because the whole team is underground in one pathological debug. On the dashboard that looks like nothing happening; in the timesheet it is everything happening in one place.

Scenario. Week 38 on the reference SoC. The DMA’s coverage curve has been flat for three weeks and its bug curve flat for the same three weeks. Read together with instrument 5, the mystery dissolves: the block’s two engineers logged those weeks almost entirely against triage and debug of one severe find from week 35 — a transfer aborted mid-descriptor and then resumed, which the midend restarts from a stale pointer while the other channel is holding the backend FIFO. **Approach.** The lead reads the flat curves as *starved*, not *clean*: no new stimulus went in, so no new questions were asked. **Why.** Metrics record what the process did, not what the design is. A flat week of a two-person team debugging one bug is a flat curve with zero evidentiary value about the rest of the block — and only the debug-time instrument distinguishes it from a genuinely quiet, genuinely clean week.

Two disciplines keep the whole panel honest. **Count only clean runs:** coverage collected from a simulation that failed, or that ran on a stale build, is contamination, not evidence — only error-free verification tasks contribute [3]. And **version everything the numbers depend on:** which RTL revision, which testbench revision, which tool versions, which seed, which configuration produced each data point [2] — without which a trend drawn across incompatible baselines is fiction. Both rules sound pedantic until the sign-off meeting, where every number on the screen is only as strong as its provenance.

7.3 Reading bug-rate curves

Plot cumulative bugs found against time and every healthy project draws the same S-shape: slow start during bring-up (the environment can barely ask questions yet), a steep middle as constrained-random stimulus and a maturing checker set rake through the design, then a flattening tail as discoveries thin out. The derivative — bugs found per week — rises, peaks, and decays. Verification managers stare at this tail like sailors at a barometer, because “the curve has gone flat” is the closest thing the discipline has to a natural stopping signal: the classical ship criterion is precisely that the RTL passes all planned tests *and* you are satisfied with the coverage and bug-rate metrics [8].

Note what that shape is, and is not, before leaning on it. It is received practice rather than a measurement: no bug-rate curve for a named project appears anywhere in this book’s cited sources, so the S describes what teams report seeing, not a fitted curve. And the published points that do exist sit on the rising limb, never on the tail — a formal deployment on a live Intel graphics project caught around 15 bugs across designs in its first two weeks and 84 within eight weeks, on totals the paper itself states differently in its abstract and in its conclusion [9].

The trap is that the flat tail is **ambiguous by construction**. A bug curve records discoveries, and a discovery needs three things to happen at once: stimulus that reaches the buggy condition, a checker that catches the misbehavior, and an engineer who triages the failure into the tracker. The curve goes flat when bugs run out — or when *any one of those three supply lines* runs out. Flat because clean and flat because blind produce the same picture. This is the general limitation of all progress metrics — they are snapshots and trends of what has happened, like stock-market indices or batting averages, and they cannot be used to predict the future [8] — but the bug curve is where the limitation does the most damage, because it is the metric managers most want to extrapolate.

The most common blindness is **stimulus saturation**. Constrained-random coverage closure is non-linear: it rises steeply at first and then exhibits diminishing returns later in the project [1]. The mechanism — this reading is ours — is that a fixed constraint set keeps re-sampling the same region of the state space: the bugs reachable inside it are found in the steep phase, and after that more seeds mostly re-confirm what is known. The curve flattens not because the design is clean but because the regression has memorized its own answers. Chapter 6’s closure discipline is the antidote; here the point is that a saturated regression manufactures exactly the flat, reassuring bug curve a naive reader takes as cleanliness.

The curve’s shape also depends on what the block is made of. An HBM controller barely draws an S: discovery is dominated by the PHY training state machine, and late finds cluster in retraining corners — after a thermal event, after a refresh collision, from an interrupted sequence. A flat tail there says nothing until you know how many training entries the regression attempted in the flat window; between training experiments that curve is flat by construction.

So never read the bug curve alone. Interrogate a flat tail with corroborating instruments — this is the correlation discipline of Section 7.1 [2] applied to one decision:

Table 7.1 — Six questions to put to a bug curve that has gone flat, with the answer pattern each returns when the flatness is genuine and the pattern it returns when the flatness is an instrument failure instead. The two columns describe the same picture read two ways, which is why the curve is never read by itself.

Question to the flat curve	Flat because clean	Flat because blind
Was new stimulus added during the flat window — new sequences, new constraints, new <i>irritators</i> (background traffic generators whose only job is to perturb the DUT)?	Yes — and it found nothing	No — same tests, more seeds
Is functional coverage still moving?	Plateaued <i>at target</i> , holes dispositioned (see Chapter 6)	Plateaued <i>below</i> target, or bins nobody reviewed
What is the fresh-seed yield (failures per thousand new seeds)?	Measured, and ~zero	Not measured, or seeds re-explore the same space
Did checker strength change?	Checkers stable or strengthened	Checkers waived/disabled to “stabilize” regression
Is the triage queue empty?	Yes — failures were dispositioned	A backlog hides undiagnosed discoveries
Where were the <i>last</i> bugs found?	Scattered, low severity, in modeled corners	Clustered, severe, in a corner the plan never modeled

The last row deserves its own paragraph, because it is the one that holds back the DMA in Section 7.5. Bugs cluster: a severe bug found late, in a corner that the coverage model did not describe, is not one bug — it is a *sample* from a region of the state space your instruments were not watching. The honest response is to treat it as evidence about the model, not only the design: extend the coverage model and the stimulus around the find and let the curve earn its flatness again, rather than fix the bug and declare the tail restored. (This reading is practitioner judgment; the corpus supports the ingredients — clustering of discoveries by stimulus category is exactly the kind of correlation a metrics database exists to expose [2] — but the inference rule is ours.)

One more curve belongs next to it: **bug-open vs bug-closure** [2]. The gap between them is the debug backlog, and its trend is a schedule instrument — a discovery curve that flattens while the open-bug count stays high means finding has paused while fixing catches up. The two lines must *meet*, not merely both go flat.

7.4 The sign-off checklist: aggregating the evidence

Chapter 4 framed the sign-off review as the lifecycle’s most human gate: a room full of people accepting a residual risk on the record. What that room needs is not a feeling and not a slide but an **evidence aggregation** — every claim of doneness traced

to an artifact a skeptic could re-derive. The checklist is that aggregation made explicit. Its shape varies by team; its content, at minimum, does not:

1. **Every plan row discharged or dispositioned.** Each feature row of the vplan (see Chapter 5) points to the coverage, checks and results that discharge it — or carries a written disposition explaining why it will not. With an executable plan and requirements traceability, this item is a database query, not an interview ^[5]. A plan row nobody can trace is an undischarged row, whatever anyone remembers.
2. **Coverage closed or waived — with waivers as first-class artifacts.** Functional coverage at target and code coverage analyzed, per Chapter 6’s closure discipline; what sign-off adds is the record. Every remaining hole carries a **waiver** with the same five fields — *the hole, the reason, the risk argument, an owner, an expiry*. Chapter 6’s three kinds of exclusion — structural, specification-driven, risk-accepted — gain one sign-off rule: a risk-accepted waiver taken to make a date is not a don’t-care, so its expiry is a real date and its signature a senior one. Only a genuine don’t-care writes “none” there, and a waiver with an empty expiry is permanent by accident.
3. **Static verification clean.** Lint, CDC and RDC sign-off with their own waiver files reviewed, not just present (see Chapter 13). Static results age well only if the waivers were honest.
4. **Formal results with their assumptions on the table.** Proofs discharge plan rows only together with their assumptions and constraints; a full proof under an over-constrained environment is a beautifully formatted blind spot (see Chapter 14). The sign-off review reads the assume list, not just the green checkmarks.
5. **Open-bug disposition.** Every open entry in the tracker classified. Severities on this project read S1 (blocks tape-out), S2 (fix, or waive with an argument) and S3 (documentation or cosmetic); Chapter 12 covers triage. Three dispositions are available: *fix before tape-out*, *waive as known errata* (with the software workaround named and the documentation owner assigned), or *defer* (with the argument why it cannot manifest in this product’s use cases). An open bug with no disposition is an unread instrument — and disposition is what the ship criterion of Section 7.3 calls *satisfaction with the bug-rate metrics*, written down.
6. **Regression health over a sustained window.** All planned tests passing across seeds and configurations over a *window*, not a snapshot: a single green run proves the weather, not the climate. And the evidence itself audited — only error-free runs on the tagged release candidate contribute ^[3].
7. **Reproducibility.** Every number on the dashboard traceable to RTL revision, test-bench revision, tool versions, configuration and seed ^[2] — which is what makes any result regenerable on demand. This is the item teams discover they need the day an escape analysis (Section 7.6) asks “what exactly did we run?” and nobody can answer.

These items are inputs, not verdicts: they converge on one review, and [Figure 7.1](#) traces the path from each of them to the decision that review records.

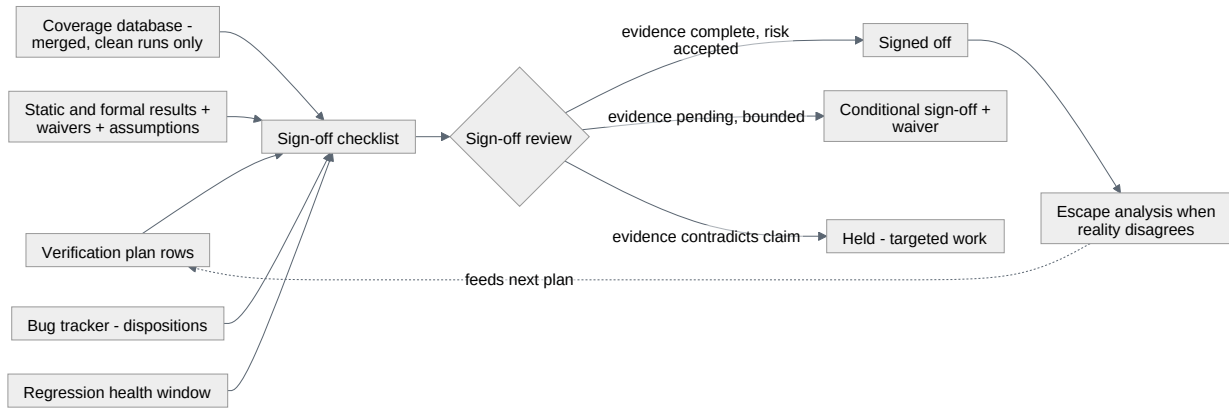


Figure 7.1 — How the sign-off checklist aggregates its evidence and what the review is then allowed to decide. Each input on the left is one checklist item reduced to the artifact a skeptic could re-derive it from; the review has three outcomes rather than two, and the middle one is a decision taken now against named evidence arriving by a named date. The dashed edge is what makes sign-off part of a loop rather than an endpoint: escape analysis returns to the plan (§7.6).

Two properties make such a checklist worth the ceremony. It is **written before the end** — the closure evidence per row was named at planning time (see Chapter 4), so the checklist is the plan’s last column, not a document invented under deadline pressure. And it is **decision-oriented**: each item resolves to signed off, conditionally signed off, or held, where *conditional* means the decision is taken now against named evidence arriving by a named date. A conditional sign-off without a condition you can test is sign-off with a guilty conscience.

The *currency* of the evidence changes with the block. A 10G Ethernet MAC with time-sensitive-networking queues signs off substantially on a compliance suite — an externally defined test set whose verdict is non-negotiable and comparable across vendors, and insufficient alone, because conformance says nothing about your own queue-shaping configuration space. Such blocks carry two evidence columns: the suite’s verdict, and the coverage model measuring everything the suite never reaches.

Worked example — checklist excerpt (the crossbar, week 40).

#	Item	Evidence	Status
1	All 41 plan rows discharged	vplan report r2841, auto-linked results; features XB-01..XB-07, 7/7 closed	PASS
2	Functional coverage 100% of the model, closed-or-waived	merge of sim+formal, tag xbar_rc3 ; 14 exclusions, all reviewed (true don’t-cares, expiry “none”)	PASS
3	Code coverage analyzed	98.7% statement / 96.2% branch; residue = defensive decode defaults, dispositioned in review	PASS
4	Static clean	lint 0 issues; CDC report waiver file v7 re-reviewed	PASS
5	Formal protocol properties	41 proven, 3 bounded ≥ 40 cycles — bounded set reviewed and accepted as residual risk; assume list reviewed against integration constraints	PASS

#	Item	Evidence	Status
6	Open bugs dispositioned	1 open S3 (documentation mismatch), waived as errata, doc owner assigned	PASS
7	Regression window	21 consecutive clean nightlies + 5 clean weekend fulls, ~9,400 seeds cumulative, tag xbar_rc3	PASS
8	Reproducibility	all results regenerable from tag + seed manifest	PASS

The evidence column above is the crossbar sign-off summary of Chapter 4, read item by item — same review, same numbers, one artifact at two distances. The write-in-interleaving bug that formal found in week 14 appears nowhere on the checklist, and that is the point: it was found, fixed, regressed, and the ordering property that caught it now sits among row 5’s unbounded proofs as permanent evidence. Row 5’s three *bounded* proofs are not a blemish either: a bounded proof holds only up to a depth, and the checklist exists to make the room say out loud that it accepts what lies beyond it. Sign-off does not certify that the block never had bugs, nor that every question was answered exhaustively; it certifies that the machinery that finds them ran to agreed completion and its residue was explicitly accepted.

7.5 Worked example: the dashboard, two weeks before tape-out

Week 40 of the reference SoC. The block sign-off review covers three blocks: the crossbar, the neural accelerator, and the DMA engine. The dashboard in front of the room — one row per block, the instrument families of Section 7.2, with debug-time share carried in the fresh-stimulus column — looks like this (arrows are three-week trends):

Table 7.2 — The week-40 block dashboard the sign-off review reads: one row per block, one column per instrument family, arrows giving the three-week trend. The bug-discovery column is flat for all three blocks and the review still reaches a different verdict on each, so the columns that do the separating are the plan rows, the direction of the coverage, and whether fresh stimulus went in at all.

Block	Plan rows	Func cov	Code cov (stmt)	Open bugs S1/S2/S3	7-day pass rate	Bug discovery (3 wk)	Fresh stimulus (3 wk)
the crossbar	41/41	100% → (closed-or-waived)	98.7% →	0 / 0 / 1	100% →	flat	yes — new irritator sequences, zero finds
the neural accelerator	37/38	98.9% ↑	97.8% ↑	0 / 0 / 2	99.8% ↑	flat	yes — quantization corners, zero finds
the DMA engine	44/47	96.0% →	98.2% →	0 / 1 / 3	99.2% →	flat	no — team in debug (Section 7.2 scenario)

The pass-rate column is a seven-day snapshot; the window each block’s *checklist* reads is longer — the crossbar’s, above, is 21 nightlies. Three flat bug curves, three different truths. The review walks the plan, block by block.

The crossbar: signed off. The checklist excerpt above is its evidence, and the flat discovery curve is corroborated against Section 7.3's table: fresh stimulus went in and found nothing, coverage at target, triage queue empty, last finds months old and scattered. What made it the easy case happened in week one — the plan named closure evidence per row, and formal took the protocol rows as far as they go, three of its properties only up to a bound, on the record. The review spends eleven minutes on it. Sign-off, minuted, with the S3 errata disposition attached.

The neural accelerator: conditional sign-off, with a waiver. One plan row is not discharged: the cross of 2-bit weight mode $\times$ concurrent DMA traffic on the shared SRAM banks. The block-level bench cannot generate real DMA contention; the SoC-level environment can, and that regression is still accumulating the cross. The dashboard otherwise argues *clean*: coverage rising to target, discovery flat while quantization-corner stimulus was still going in, both dispositioned bugs S3 (a performance-counter off-by-one on job restart, waived as errata with a software workaround; a register-description mismatch). The 99.8% pass rate gets its own question, because checklist item 6 asks for planned tests passing *repeatedly*: the 0.2% is a known test-bench race on job restart, filed against the environment, not the design — the review confirms that disposition rather than reading the number as noise. Then it decides now rather than deferring: conditional sign-off on a waiver recording the hole (the undischarged cross), the reason (no block-level bench can generate real contention), the risk argument (the 2-bit datapath is identical to the verified 4-bit path except for the unpack stage, and contention exercises arbitration already proven on the crossbar), the owner (the SoC-level verification lead), and the expiry — the cross closes in SoC regression by week 41, or the block returns to this room. That is the entire anatomy of a defensible conditional: the hole named, the reason recorded, the risk argued, an owner assigned, an expiry dated.

The DMA engine: held. Start with the master instrument. Three of forty-seven plan rows are undischarged — the worst plan status in the room, and by checklist item 1 that alone holds the block. All three were opened in week 35 in reaction to the find below, and none carries evidence yet. But the more instructive failure is what the bug curve says.

On paper the block reads like the middle child: 96% coverage, 99.2% pass rate, one open S2 with a fix under review, discovery curve flat. But the corroboration of Section 7.3's table fails on four of six axes. No new stimulus went in across the whole trend window — the two engineers were underground on one bug from week 35, and the regression has been re-running the same questions since. Fresh-seed yield is therefore unmeasured. Coverage is plateaued *below* target, and in the region of the last find it sits at 71%. And the last find is the disqualifier.

Week 35, from fresh seeds in the SoC-level regression: a transfer aborted mid-descriptor and then resumed corrupts data, because the midend restarts the interrupted 2D row from a pointer it never invalidated and re-issues bursts it had already committed — reproducible only when the other channel is holding the backend FIFO at the instant of the abort. The plan had nothing on it: abort-and-resume was not a row, not a coverpoint, not a cross, because the midend rows model *legalization* and

every one of them assumes a transfer that runs to completion. Contrast the DMA's 2D-legalization rows, where the plan does point. A transfer with negative stride mis-legalized at the 4 KB AXI boundary, corrupting data only while channel 1 is simultaneously active, is precisely the scenario one of those crosses demands: constrained-random plus the scoreboard reaches it where the directed legalizer tests never would, and the plan says in advance where to look. That failure is SOC-1284, the escape of Chapter 1, and it arrives after this review. The week-35 find had no such cross behind it: the dashboard was green *around a region it could not see*, and the 71% is not the hole, because the hole has no bins at all.

A late, severe find in an unmodeled corner is a sample, not a unit (Section 7.3): the review reads it as evidence about the instruments, not about one descriptor. The block is held two weeks, with the work aimed at the blind spot rather than the bug: the three open rows are to be discharged and widened into a family for abort and resume as a *class*, with a cross of abort point $\times$ transfer geometry $\times$ other-channel state; constrained-random abort injection while both channels contend for the backend; and a checker on the invariant the bug violated, that a resumed transfer issues exactly the bytes the aborted one had not yet committed. The curve must earn its flatness under the *new* instruments before the checklist is re-walked.

The meeting ends the way sign-off reviews should: three minuted decisions, each carrying its named residual risk, nobody deferring to the seniority of whoever spoke last. The one-line question the chapter opened with — *is it done?* — gets the only honest answer metrics can give: *the crossbar is; the accelerator is, contingent on named evidence by a named date; the DMA is not, and here is exactly what “yes” will look like in two weeks.*

7.6 Escape analysis: the post-mortem that feeds the next plan

An **escape** is a bug found on the wrong side of a boundary that was supposed to catch it: a block bug found at SoC level, an RTL bug found in gate-level simulation, a silicon bug found in the lab. Escapes are not an anomaly of bad teams — they are the statistical norm of the discipline. In the 2024 industry data, only 14% of IC/ASIC projects achieved first-silicon success, the lowest value recorded in twenty years ^[6], and 87% of FPGA projects reported non-trivial bugs escaping into production ^[10]. If escapes were shameful exceptions, the entire industry would be ashamed. The differentiating discipline is not preventing them — Chapter 3 explained why that is not on offer — but *metabolizing* them: an escape is a perfectly targeted audit of your verification process, purchased at the worst possible price. Wasting it is the only unforgivable part.

Escape analysis is the structured post-mortem that does the metabolizing. For each escape, the team answers five questions, in order, in writing:

1. **What was the bug?** Precise mechanism, not symptom — the level of the bug report, not the level of the failing test.

2. **Why did our stimulus never reach it?** Which condition was never generated, and was that a constraint gap, a missing sequence, or a scenario the plan never imagined?
3. **Why did no checker catch it — or could none have seen it?** Distinguish *reached but unchecked* (a checker gap) from *unreachable at this abstraction* (a platform gap): the same escape indicts different machinery.
4. **Why did our metrics say we were done?** Which dashboard was green over this hole — a coverage model with no bins here, a waiver that aged badly, a bug curve read as clean when it was starved?
5. **What changes?** Concretely: new plan rows and coverage crosses for the *class* of bug (never only the instance), a checker or platform added, a sign-off checklist item amended — each with an owner, filed into the next project’s planning inputs (see Chapter 5).

The non-negotiable ground rule is that the analysis is **blameless**: the object of study is the process, not the engineer. This is not kindness — it is instrumentation hygiene. The moment a post-mortem can end a career, every future post-mortem receives laundered data, and the metrics culture of the whole project quietly corrupts. Fear-driven verification and lack of introspection are themselves among the listed contributors to the gap between what teams should verify and what they do, and the self-induced part of that gap is precisely the part a team can attack — by asking *am I creating it?* ^[4]. Escape analysis is that introspection, institutionalized. In question 4 the answer is frequently “the metrics were fine and the plan was blind” — an answer only obtainable from an engineer who is not defending herself.

Scenario — escape post-mortem, week 44. The event unit carries a canonical escape: its retention registers lose their configuration on power-domain re-entry when a wake event races the release of isolation — found in UPF-aware gate-level simulation *after* RTL sign-off (see Chapter 19); UPF is the Unified Power Format, IEEE Std 1801-2024, *IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems* ^[11]. **Approach.** The five questions, in writing. (1) Mechanism: a wake event arriving in the same cycle as isolation release corrupts the restore path of the retention flops. (2) Stimulus: RTL-level power tests stepped through save/sleep/restore sequences with directed timing; nothing ever randomized wake-event arrival against the power-sequence FSM, so the race was never generated. (3) Checkers: the RTL power tests checked configuration *after* restore completed — the race window itself was unobserved; worse, plain RTL simulation does not model loss of state in the switched domain at all, so part of the behavior was unrepresentable below power-aware simulation. Platform gap *and* checker gap. (4) Metrics: the power-management plan rows were discharged by those directed tests; the coverage model had no cross of wake-timing × power-state transition, so 100% of what was measured was green — the dashboard was faithful to a blind plan. (5) Changes: a new plan-row family “retention behavior under randomized wake timing”, the missing cross added to the coverage model, power-aware simulation pulled earlier in the schedule instead of arriving with GLS, and a new sign-off checklist item — *power-intent behavior verified on a power-aware platform, not inferred from RTL* — with the low-power methodology owner named.

Why. The engineer who wrote the directed tests executed the plan she was given; the plan could not see the race. Blameless analysis converts one late, expensive discovery into four permanent upgrades of the machinery — which is the only exchange rate at which an escape is ever worth what it cost.

Escape analysis is where the sign-off loop closes on itself: the checklist of Section 7.4 is not a static document but the accumulated sediment of every post-mortem the organization has survived. A mature team’s checklist reads like a scar tissue map — each odd-looking item (“retention verified on power-aware platform”, “fresh-seed yield reported for any block whose curve is flat”) is a former escape, filed where it can never surprise anyone again. That is what “feeds the next plan” means: not a lessons-learned document nobody reopens, but new rows in the two artifacts every future project must walk past — the plan and the checklist.

IN PRACTICE

Real metrics infrastructure is less tidy than this chapter’s dashboard. Most teams run a hybrid: a vendor verification-management platform for plan-to-result traceability and multi-engine coverage merge [5], surrounded by a thicket of in-house scripts and databases. Some build the entire MDV stack in-house for flexibility and engine independence, at a real cost — they now own that infrastructure, and must train engineers who have never seen it [1]. Either way, the weekly metrics review is the actual heartbeat of the process: dashboards that are not walked in a standing meeting decay into wallpaper within a month.

The messy parts are human. Any metric tied to a date or a bonus starts being *managed* instead of *measured* — coverage models quietly grow easy bins, failing tests get “temporarily” excluded, and the dashboard becomes a theater set (fear-driven and ulterior-motive-driven verification sit on the same list of causes as design complexity itself [4]). Waivers rot: the exclusion that was honest in March is stale in September, which is why sign-off re-reads waiver files instead of counting them. Coverage closure gets scheduled as if it were linear when everyone in the room has seen the diminishing-returns tail [1, 4]. And of Section 7.2’s five instruments, debug-time share is the one that goes missing most often — even though debug dominates verification time and varies unpredictably between projects [6], which makes it the number most likely to explain where a schedule actually went. That last step is our inference; the corpus supplies the dominance and the variance, nothing about who tracks it. The teams that get value from MDV are not the ones with the most metrics — they are the ones that collect fewer numbers and act on all of them [2, 3].

PITFALLS

1. **Green-dashboard worship.** *Symptom:* process indicators green while the design is red — status derived from the dashboard’s color, not its provenance. *Cure:* every green traces to an artifact a skeptic could regenerate; sign-off walks evidence, not colors.

2. **Reading a flat bug curve as clean.** *Symptom:* “no new bugs in three weeks” offered as closure evidence by itself. *Cure:* interrogate the flat tail — fresh stimulus? coverage at target? fresh-seed yield? checkers intact? triage queue empty? where were the last finds?
3. **Turning a metric into a target.** *Symptom:* coverage percentage tied to a milestone date; bins multiply, hard crosses vanish, the number climbs while knowledge does not. *Cure:* targets live on plan rows and dispositions; percentages are instruments, and instruments are audited (see Chapter 6).
4. **Waivers without owner or expiry.** *Symptom:* a waiver file that only ever grows, full of reasons nobody present can explain. *Cure:* every waiver carries the same five fields — hole, reason, risk argument, owner, expiry — and only a true don’t-care may write “none” in the expiry field; sign-off re-reads them all.
5. **Counting dirty runs.** *Symptom:* coverage merged from failing simulations or mixed RTL revisions inflates closure. *Cure:* only error-free runs on the tagged baseline contribute ^[3]; provenance is part of the number.
6. **Post-mortems with a defendant.** *Symptom:* escape analyses that read like alibis; the same escape class returns next project. *Cure:* blameless discipline — the process is the object of study, and every analysis must end in changed rows in the plan and the checklist.

SUMMARY

- On an SoC, intuition about verification status fails structurally, not occasionally: no one person spans the project, and what is not measured is not known ^[2]. Metrics are the team’s only shared truth — provided they are few, correlated, and acted upon.
- Instrument five things: per-feature plan status (the master instrument), coverage *trends*, bug curves, regression health, and debug-time share. No single one of them is trustworthy alone ^[2].
- A flat bug curve is ambiguous by construction: flat-because-clean and flat-because-starved look identical. Corroborate with fresh stimulus, coverage position, seed yield, checker integrity and the location of the last finds before believing it.
- A late, severe bug in an unmodeled corner is a sample from an unwatched region, not a unit. Extend the instruments and let the curve earn its flatness again.
- Sign-off is evidence aggregation: plan rows discharged, coverage closed-or-waived, static and formal clean with assumptions read, open bugs dispositioned, results reproducible from tagged baselines — resolved into signed off, conditional (on a waiver with all five fields: hole, reason, risk argument, owner, expiry), or held.
- Escapes are the industry norm — in 2024, 14% first-silicon success and 87% of FPGA projects with non-trivial production escapes ^[6, 10]. The discipline that compounds is blameless escape analysis: five questions, answered in writing, ending as new rows in the next plan and the sign-off checklist.

FURTHER READING

From the corpus. [2] is the philosophical foundation of this chapter — metrics beyond coverage for SoC verification, categorization, run-time control and reporting; [1] is its modern industrial counterpoint, an in-house MDV stack with a single golden coverage model, coverage pruning and regression diffing. [5] shows what a verification-management platform automates: traceability, multi-engine merge, and the regression round trip. [3] (Chapter 6, “Coverage-Driven Verification”) gives the methodology rules this chapter leans on — metrics that guide effort allocation, the irrelevant-metric trap, clean-run discipline; [8] states the classical ship criterion and the honest limits of trend metrics. [4] is a short, uncomfortable taxonomy of why verification falls short, including the metrics gap and the self-induced gap. The industry effectiveness data cited throughout is from the 2024 Wilson Research Group studies [6, 10] and their methodological ancestor, Foster’s DAC 2015 trend study [7]. Piziali’s coverage taxonomy [12] underpins the measurement side of this chapter and is treated in depth in Chapter 6.

Not in the corpus. Carter & Hemmady, *Metric Driven Design Verification: An Engineer’s and Executive’s Guide to First Pass Success* (Springer, 2007) — the book-length treatment of metrics-driven processes, and one of the two works [2] differentiates itself from; cited here as background only.

REFERENCES

1. **D22** S. Hristozkov, J. Pallister, and R. Porter, “No Country For Old Men – A Modern Take on Metrics Driven Verification,” Proc. DVCon Europe, 2021.
2. **D20** A. Meyer and H. Foster, “Metrics in SoC Verification: Not just for coverage anymore,” Proc. DVCon U.S., 2012.
3. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, “Verification Methodology Manual for SystemVerilog,” Springer, 2006.
4. **D16** R. Narayan and T. Symons, “I created the Verification Gap,” Proc. DVCon U.S., 2015.
5. **D26** D. Zhang, “Smarter Verification Management with vManager Platform,” Proc. DVCon China, 2021.
6. **R2** H. D. Foster, “2024 Wilson Research Group IC/ASIC Functional Verification Trend Report,” Siemens EDA white paper, 2024.
7. **P17** H. D. Foster, “Trends in Functional Verification: A 2014 Industry Study,” Proc. 52nd Design Automation Conference (DAC ’15), San Francisco, CA, 2015.
8. **B2** J. Bergeron, “Writing Testbenches using SystemVerilog,” Springer, 2006.
9. **D18** M. V. Achutha Kiran Kumar, E. Seligman, A. Gupta, Ss Bindumadhava, and A. Bharadwaj, “Making Formal Property Verification Mainstream: An Intel Graphics Experience,” Proc. DVCon U.S., 2017.
10. **R1** H. D. Foster, “2024 Wilson Research Group FPGA Functional Verification Trend Report,” Siemens EDA white paper, 2024.
11. **S10** IEEE, “IEEE Std 1801-2024, IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems (Revision of IEEE Std 1801-2018),” IEEE, 2024.
12. **B5** A. Piziali, “Functional Verification Coverage Measurement and Analysis,” Springer, 2004.

PART III

Dynamic Verification

The simulation core: testbench architecture, stimulus, UVM as a methodology rather than an API, assertions, and the engineering of regression.

8. Testbench Architecture

Six weeks into crossbar bring-up, the reference SoC's interconnect testbench is one file: 2,400 lines in a single module. It drives all five manager ports, samples the four subordinate ports, computes what the SRAM should return, and collects coverage — in the same forever loop, over the same variables. It works. It has found four bugs. Then two things happen in the same week.

First, a read-data mismatch fires at 41 μ s and nobody can say what it means. The crossbar may have returned two responses in an order the protocol permits; the checker may have assumed an order the protocol never promised; or the driver's back-pressure model may have drifted out of step with what it told the checker to expect. All three hypotheses live in the same two hundred lines and share state, so no experiment separates them. Two days in a waveform viewer later, the ticket is closed as “testbench issue” — the phrase that means *we never found out*.

Second, the DMA team asks to reuse the AXI checking against their backend. It cannot be lifted out: the checker reads variables the driver writes, so it knows only what the driver *intended* to send, never what appeared on the wires — and a checker that trusts the stimulus generator's intentions is no oracle (Chapter 3).

Both problems have one root, and it is not a coding error. Five distinct responsibilities were implemented as one program. This chapter is about the alternative, and about why that alternative is not bureaucracy: it is the only structure in which an environment can be debugged, trusted, and reused.

CHAPTER MAP

- §8.1 establishes why stimulus generation, driving, monitoring, checking and coverage are five separate responsibilities, and what breaks — concretely — when they are merged.
- §8.2 develops layering: signal to transaction to sequence, with vendor-neutral transaction-level communication between components.
- §8.3 treats the three scoreboard patterns — in-order, multiple in-order streams, unordered with matching keys — and what a reordering design costs the checker.
- §8.4 states when to build a reference model, which of its three species is actually being built, and what maintaining it costs.
- §8.5 establishes why a monitor must never drive and how passive instantiation carries a block environment into a system environment; §8.6 and §8.7 build the environment step by step on the crossbar and the DMA; §8.8 treats the environment itself as a design, with its own reviews, configurability and bring-up story.

8.1 Five responsibilities, not one program

A testbench need not be a monolithic block ^[1]. The design under verification is not implemented as one unit; there is no reason the environment around it should be. The decomposition that survived decades of practice separates five responsibilities:

Table 8.1 — The five responsibilities a testbench separates, each with the question it answers and the thing it must not know. The third column is the load-bearing one: every entry in it is an ignorance imposed on purpose, because merging two of these boxes does not merely blur the environment's structure — it loses something specific.

Responsibility	Answers	Must not know
Stimulus generation	<i>What</i> scenario should happen next?	How the protocol wiggles pins
Driving	<i>How</i> is this transaction expressed on the interface?	What the correct response is
Monitoring	What <i>did</i> appear on the interface?	What was intended
Checking	Was it <i>right</i> ?	Which test is running
Coverage	What have we <i>exercised</i> ?	Whether the test passed

The separation is not a modern refinement. By 2001 a language designed specifically for functional verification was already built on the same principle — a data model, a test generator, a reference model, a checker (split further into a data checker and a temporal checker running concurrently with the design), and coverage metrics tied back to a plan, with separation of concerns as its organizing principle rather than a coding convention retrofitted later ^[2]. Chapter 9 tells that history; what matters here is that the structure came first and the tooling followed.

Each responsibility earns its own box because each has a different failure mode and a different reason to change. Merge two of them and you lose something specific:

Checking inside the driver destroys the oracle. A driver knows what it meant to send. If the checker lives there, it compares the design against the stimulus generator's intentions rather than against the specification — and every bug in which the driver itself misbehaves becomes invisible, because both sides of the comparison move together. This is the common-mode error of Chapter 3, built into the architecture instead of merely risked.

Monitoring inside the driver destroys reuse. The moment observation is a side effect of driving, there is no way to watch an interface you are not driving. That forecloses passive reuse at the system level (Section 8.5), and it forecloses the most valuable debug configuration there is: the same environment, stimulus disabled, watching real traffic.

Coverage inside the checker destroys honesty. Sample coverage where the verdict is computed and you sample it only on paths the checker reached. Holes then look like missing stimulus when they are missing observation, and Chapter 6's central question — *what did the tests never do?* — becomes unanswerable.

The checking side deserves special respect because it is where the effort actually goes. On a finished project, the self-checking structure is the largest and most complex part of the environment, the one that cost the most authoring and maintenance, and the one responsible for declaring functional correctness — it embodies a duplication of the specified functionality of the design. That is not a utility to be bolted on during bring-up: which failure modes it detects, and how, belongs in the verification plan and must be reviewed, so that a potential failure does not go undetected. The classical list worth naming explicitly is data transformation, data ordering, protocol correctness, data losses and design state ^[1] (see Chapter 5).

Nor does every check belong in the same place. Implementation-specific failure modes such as buffer overflows are best coded as assertions directly in the RTL, and signal-level relationships belong in the interface declaration that contains the signals; higher-level failures — data transformation, ordering, end-to-end computation — are better detected behaviorally in the testbench, because a property language does not lend itself well to end-to-end response checking ^[1] (Chapter 11). What matters is that the split is deliberate and written down, so every failure mode has exactly one owner.

Scenario. The opening scenario’s driver computes, for each write it issues, the value it expects to read back, and stores it in a local array. A reviewer asks what happens if the driver mis-encodes `awsize`. **Approach.** Split it: the driver only wiggles pins; a monitor reconstructs from those pins what the crossbar actually received; the predictor computes expectations from the *monitored* request. **Why.** With the split, the mis-encoded `awsize` produces a monitored transaction that differs from the intended one and the read-back check fails. Without it, the driver’s mistake propagates identically into both the design and the expectation, and the test passes. The rule generalizes: *predict from what was observed, never from what was requested*.

8.2 Layering: signal, transaction, sequence

An environment spans three levels of abstraction, and most of its engineering value comes from keeping them apart.

At the **signal level** live pins, edges and handshakes. At the **transaction level** lives the unit of meaning: a transaction is initiated by injecting something into the running simulation and terminated some time later by observing a result — functional verification works at the level of whole transactions rather than the design’s clock cycles ^[2]. At the **sequence level** live scenarios: ordered, constrained collections of transactions that express intent (“fill the write-data channel of manager 2 while the debug port reads the ROM”). Chapter 9 takes the sequence level as its subject.

The component that converts between the first two levels is the **transactor**, or bus-functional model: repetitive physical-level operations are encapsulated into tasks, and all the tasks for one physical interface or protocol are collected into one model ^[1]. Everything above it speaks transactions; only it speaks pins.

The conversion has a precise seam, and getting it wrong is the most common source of phantom bugs during bring-up: *when* the pins are read. A clocking block makes the sampling moment explicit — with the default `1step` input skew, the value sampled is always the signal’s last value immediately before the clock edge [3], which is what a monitor wants and what a naive `@(posedge clk)` read does not reliably give.

Two shapes of communication

Between components, two communication shapes cover almost everything, and they are not interchangeable.

Operational communication is point-to-point and blocking: a producer hands a transaction to one consumer, and back-pressure is meaningful — a sequencer that cannot hand its next item to a busy driver *should* wait. **Observational** communication is broadcast and non-blocking: a monitor publishes what it saw and does not care who is listening. The standard form of the second is an analysis port whose interface is a single `write()` function that broadcasts a value to all subscribers [4]; the port may have zero, one, or many subscribers, the producer’s operation does not depend on how many are connected, and because `write()` is a void function the call always completes in the same delta cycle [5].

That asymmetry is the whole point: a monitor must never be slowed by the things watching it, or the act of measuring changes the timing of the measurement. It also makes observers pluggable — coverage collector, protocol checker and scoreboard subscribe to one stream, and a fourth costs nothing structurally.

Broadcast carries one obligation that is easy to violate and painful to debug: each subscriber receives a handle to the *same* transaction object, so every subscriber must copy before operating on it [5], and the standard states normatively that a subscriber’s `write()` implementation shall not modify the value passed to it (IEEE Std 1800.2-2020, *IEEE Standard for Universal Verification Methodology*, §12.2.10.2) [4]. A coverage collector that normalizes an address field in place will silently corrupt the scoreboard’s expectation, and the failure will look like a design bug for as long as you let it.

The library that standardizes these connections is Chapter 10’s subject; the pattern is older than any library and independent of all of them: *one publisher, many independent observers, no back-pressure on observation*.

Layer by protocol, not by pin bundle

Transactors are traditionally tied to one physical interface, which quietly prevents reuse: each environment then needs its own model with its own abstraction level. The better structure layers the models according to the layers of the protocol they implement. The layers are usually obvious in communication protocols — a USB application layers into packets, transactions and transfers (*Universal Serial Bus 3.2 Specification*, Revision 1.1, USB 3.0 Promoter Group, June 2022) ^[6] — but they exist elsewhere too, and any system-level function you can decompose into blocks gives you a corresponding decomposition of its transactions ^[1].

This matters because what is a transaction at the block level is usually not a transaction at the system level: for a MAC block, a transaction is an Ethernet frame; for the system that reassembles fragments and routes them, a transaction is an IP packet ^[1]. Layered transactors let both environments share the lower layers instead of forking them.

Scenario. The USB controller must pass a compliance suite defined outside the specification, in the USB SuperSpeed Compliance Methodology white paper, and the team also needs its own directed and random tests for the dual-role device/host switch. **Approach.** Build the harness in three layers — a signal-level transactor per port, a packet layer, and a transaction/transfer layer — and let the compliance suite drive the top layer while in-house sequences drive whichever layer they need. **Why.** The compliance suite is a fixed body of externally specified stimulus and checking; it is not an environment and it does not know about your power-state corner cases. Layering lets both consume the same transactors, so a bug found by either is reproducible by the other, and the signal layer is written and debugged once.

The third payoff is retargeting. Exchanging compact transaction messages instead of per-signal traffic is what lets an environment escape the simulator/emulator throughput bottleneck, provided timed and untimed behavior are kept apart and the two sides synchronize on transactions and events rather than clocks ^[7]. Transactors are what make that possible; an environment that reaches into signals from twenty places cannot be moved at all (see Chapter 17).

8.3 Scoreboard patterns

The word *scoreboard* is not used consistently in the industry: for some it means the entire self-checking structure — predictor, expected-data storage and comparison together. This chapter uses the narrower and more useful definition: a **scoreboard is the data structure that holds data expected to be received**, filled by a predictor and consulted by the comparison function when the output monitor reports something. Anything still in it at the end of simulation was lost in the design — which may or may not be an error ^[1].

Designing one is therefore a data-structure problem whose single driver is the look-up: because the exact order in which outputs appear may be hard to predict, the structure must be designed to minimize the look-up operation [8]. Three patterns cover nearly every design.

In-order: one queue. If the design preserves the order of its input stream, store expectations in a queue and check each observed response against the front. A queue beats a dynamic array here because it performs the same adding or removing regardless of position, whereas an array shifts its whole content on every check [8]. Lookup is $O(1)$, the failure message is unambiguous, and any reordering the design performs is reported as a data mismatch — which is either exactly what you want or exactly what you must not do, depending on the spec.

Multiple independent in-order streams: one queue per stream. Responses on several concurrent interfaces may be observed in any order while the sequence on each single interface stays in order; one queue per output interface holds that interface's predictions in the expected order and lets observations be checked in any order across interfaces [8]. The pattern generalizes past physical interfaces to any *logical* stream the specification promises to keep ordered. On the crossbar, the guarantee AXI makes is per manager port, per direction and per transaction ID: responses carrying the same ID return to their manager in order, responses with different IDs may be reordered by the interconnect, and reads and writes carry independent ID spaces. The *AMBA® AXI and ACE Protocol Specification*, Arm IHI 0022H.c §A5.1 “AXI transaction identifiers” [9], states the ordering rule in one sentence: “All transactions with a given AXI ID value must remain ordered, but there is no restriction on the ordering of transactions with different ID values.” The stream key therefore needs all three coordinates, and Section 8.6 shows what goes wrong with any fewer.

Unordered: matching keys. When the design reorders by design and no per-stream ordering survives, the checker must *find* the matching expectation rather than pop it. Searching the whole structure makes simulations crawl; the fix is an index — a hashing function computes a likely-unique key from the observed response's own values, and an associative array maps that key to the expected data's location, eliminating the search [1, 8].

Scenario. The Ethernet MAC's TSN queues reorder by design: eight priority queues with a credit-based shaper mean a low-priority frame injected first can legitimately leave last. **Approach.** Key expectations by (destination port, traffic class, stream identifier), all three of which the egress monitor reads off the frame itself, and keep one in-order queue per key — within a traffic class the shaper does not reorder — then check the *relations* the specification actually guarantees (per-class order, no duplication, no loss) rather than the exact global departure sequence. **Why.** A global in-order scoreboard would report a false failure on the first legal reordering. A fully unordered scoreboard would accept a real per-class ordering bug. The keying decision is the specification of what ordering you claim to verify — which is why it belongs in a review, not in a commit message.

Which of the three patterns applies is not the checker’s choice; it follows from the ordering the design guarantees. Table 8.2 sets them side by side, with the lookup each one forces.

Table 8.2 — Scoreboard structure as a function of the ordering the design actually guarantees: what the design preserves, the structure that matches it, the lookup that structure needs, and what the checker gives up in exchange. The last column is where the price is paid — each step down the table tolerates more of the reordering the specification permits, and checks less of it.

Design behavior	Structure	Lookup	Cost to the checker
Preserves order	One queue	Front only, O(1)	Cheapest; any reorder = failure
Per-stream order only	Queue per stream key	Front of one queue	Must <i>name</i> the stream key correctly
Reorders freely	Associative index on a content key	Hash, O(1) amortized	Ordering is no longer checked at all

Read the last column as a ledger: every step down buys tolerance of legal reordering and pays for it by giving up an ordering check.

Tagging, losses and inexact answers

Three refinements recur often enough to name. **Data tagging** exploits designs that carry part of their input untouched to an output: once you have separately established that the payload is not modified, you can encode the expected destination, position and transformation *in the payload*, so each output monitor decides correctness on its own. It yields the simplest self-checking structure of the lot, since all the intelligence sits in independent monitors — with two sharp limits. Serial numbers should not be embedded in stimulus data, because a system-level environment will need those user fields for higher-level information [8]; and when the payload is too short to hold the tag, tagging must work in concert with a scoreboard, the tag serving as a fast key into it [1].

Losses. Some designs legitimately drop data, and reproducing the discard decision means reproducing the design’s resource state and timing. Usually, do not try. On an *ordered* stream — the first two patterns only, since “ahead of” means nothing in a content-keyed index — assume a transaction was properly dropped when it sits ahead of the matched expectation, and confirm the assumption against the design’s own drop-count statistics [8]. That converts an intractable prediction into a cheap aggregate check — honest, because what remains is stated as an aggregate.

Inexact answers. A fixed-point datapath will not match a floating-point model bit for bit; the observed response is then compared against the predicted one within an acceptable margin of error or bit-error rate [8]. The radar front-end’s FFT chain is the canonical case, and the margin’s justification belongs in the plan rather than in a constant nobody remembers choosing.

8.4 Reference models: three species

Chapter 3 established that a true reference model is the most complete oracle and that it rarely exists. What that chapter left open is what you build when it does not — and the answer is three species with different economics, plus one distinction teams routinely blur.

A **transfer function** reproduces the data transformation the design performs, so that the environment can determine which output to expect; it uses the design's configuration descriptor to perform the same transformation, and it is normally used together with a scoreboard. It is not a reference model, and the difference is not academic: a transfer function does not exist a priori, is not golden by definition, and as simulations run will contain as many errors as the design itself; because it predicts less accurately, the response monitor has to perform more intelligent checks to deal with the uncertainty — whereas with a true reference model, checking is a plain observed-against-expected comparison ^[1].

Three species, then.

Species 1 — the algorithmic transfer function. You write it, in the testbench language, from the specification. The crossbar's address decoder and the DMA's descriptor *legalizer* — not its byte-level descriptor model, which Section 8.7 treats separately — are both of this kind. Cheap to build, cheap to change, never trusted absolutely: when model and design disagree, the first question is which of the two misread the specification (Chapter 3's common-mode warning applies in full).

Species 2 — the golden executable. The standard ships the model, so checking collapses to comparison: decode the conformance bitstream, compare pixels, done. The video codec's decoder is that clean case (see Chapter 3) — and the encoder beside it on the same die is where the species runs out. The standard constrains the bitstream, not the choices that produced it, so two conformant encoders emit different bitstreams from the same picture and both are correct; there is no executable to compare against. What replaces it is a conjunction of weaker claims — the output decodes, it decodes to a picture within the distortion target *the team itself* set (a quality goal of its own making, not a standard-mandated one), and it never pushes the level's buffer-occupancy constraints out of bounds — each of them checkable, none of them golden. A bit-exact model in one direction and no model at all in the other, in one block: golden is not the same as complete.

Species 3 — the co-simulated instruction-set simulator. For the reference SoC's core, the model runs alongside the RTL and the comparison happens at instruction retirement: an instruction manager processes instructions as the design retires them, and an ISA scoreboard performs a step-and-compare of PC, registers, CSRs and trap status (*The RISC-V Instruction Set Manual, Volume II: Privileged Architecture*, Version 20250508, ratified, RISC-V International) ^[10] against the golden state ^[11]. Two structural facts make this work.

First, reference models need not be cycle-accurate specifications of the hardware, only functionally accurate ^[2]. That is what makes a software model fast enough to

co-simulate, and it dictates *where* you may compare: at an architectural boundary such as retirement, never cycle by cycle.

Second — easy to get wrong, and it produces failures that look exactly like design bugs — asynchronous events must reach both sides. A RISC-V environment doing this correctly has its interrupt agent drive random external, timer and software interrupts to the core and *simultaneously notify the reference model of the same interrupts*, so design and model operate on a synchronized view of external events ^[11]. Skip that and every random interrupt produces a divergence that costs a day to diagnose and is entirely an artifact of the environment. The same rule governs error injection and any responder whose behavior the model must know about.

This is not only a teaching device. On the OpenHW Group’s CV32E40Pv2 — a 32-bit in-order RISC-V core that originated as RI5CY at ETH Zurich and whose development was continued by Dolphin Design, with the reference model and the coverage model supplied by Imperas — the environment runs the same program on the RTL and on the reference model, streams the core’s internal state and its asynchronous events over an interface named RVVI-TRACE, and compares state as each instruction retires, flagging a mismatch the moment it appears rather than at the end of the run ^[12]. Both structural facts above are visible in that arrangement: the comparison point is architectural, and the asynchronous events reach both sides.

Where concurrent integration is too hard, both sides can dump outputs for comparison in a post-processing step ^[1] — at Chapter 3’s price: a verdict that arrives after the simulation ends arrives far from the state that caused it.

The maintenance burden

Whatever species you build, you have written a second implementation of the specification and will maintain it as long as the design lives: every specification change becomes two edits, two reviews and one argument about which side was right. Budget it in the plan (Chapter 5) rather than discovering it in month four. The alternative is worse in a specific way. Without a model, expectations are hard-coded per test, which needs a known configuration and a known input stream and so works only for directed testcases; and because the response checked is crafted for that testcase, an unexpected corner accidentally created by a test aimed elsewhere goes undetected ^[1]. That is the real argument for a model: not elegance, but that a hard-coded oracle can only find the bug you were already looking for.

Scenario. The team asks whether the UART needs a reference model. **Approach.** No — Chapter 3’s risk allocation already gave that block a thin directed suite, and nothing about oracle architecture changes it. **Why.** Build models where the transformation is stateful, configurable or arithmetic, and where you would otherwise re-derive expectations by hand in every test. A small, silicon-proven block with a trivial transformation is none of those.

8.5 Monitors and passive checking

The rule is absolute and constantly violated: **a monitor never drives**.

Making it precise requires the right criterion, because signal direction is the wrong one — a stimulus task that checks feedback signals is still stimulus, and a monitor that must return flow-control is still a monitor. The distinction that holds is *who initiates*: if a task initiates the transaction under full control of the testbench, it is a stimulus generator; if it waits for a transaction initiated by the design, it is a response monitor ^[1]. A subordinate-side responder that must answer requests is therefore an agent’s driver, not its monitor, however passive it feels.

Monitors must also always be monitoring. Activate a response task late and the design finds the testbench not ready: back-pressure builds inside the design so it is never verified at full throughput, output data “spills” and leaves a gap in the stream, or the output protocol is violated outright because feedback signals are missing — a protocol error that is entirely the testbench’s fault. The structural answer is the **autonomous monitor**: an independent thread started at construction that continuously watches the interface and pushes extracted data into a FIFO the rest of the environment drains at its leisure, decoupling the timing of the transaction from the timing of the checking ^[1].

```
// Passive by construction, but only through the modport: an inputs-only
// clocking block constrains vif.cb.*, and nothing else.
interface axi_mon_if (input logic clk, input logic rst_n);
    logic          awvalid, awready;
    logic [31:0]   awaddr;
    logic [3:0]    awid;
    clocking cb @(posedge clk);
        input awvalid, awready, awaddr, awid;
    endclocking
    // Only the clocking block and reset are reachable through this modport.
    modport MON (clocking cb, input rst_n);
endinterface
```

The modport is the enforcement, and the shortcut version of this claim is false. A component holding a plain `virtual axi_mon_if` handle reaches every member of the interface, and `awvalid`, `awready`, `awaddr` and `awid` are ordinary `logic` variables: `vif.awvalid = 1'b1;` compiles and drives. A clocking block on its own constrains `vif.cb.*` and nothing else. The construct that closes the hole is a *clocking modport*, whose standard example is `modport STB (clocking sb);` — a synchronous testbench modport ^[3]. A monitor handed a `virtual axi_mon_if.MON` cannot assign to the bus at all: the compiler rejects the access, so no code review has to catch it.

Active and passive instantiation

Chapter 10 defines the agent and its active/passive switch; what matters architecturally is what that switch buys — and what it buys is a block environment that survives into a system environment. A multi-level RISC-V environment does exactly this: the vector unit’s environment actively emulates the scalar core at block level, and the

same environment is then instantiated in passive mode inside the core-level environment once the real core drives the interface ^[11]. Nothing is rewritten: the stimulus half is switched off and the observation half keeps working. An environment built without that seam is rebuilt at every integration level, which is how teams end up verifying the system with no checking at all.

Protocol monitors and functional monitors

Two kinds of passive component get called monitors, and separating them keeps ownership clear. A **protocol monitor** owns one question — *is the traffic on this interface legal?* Handshake stability, response-code legality, no burst crossing a 4 KB boundary, no reserved encodings (Arm IHI 0022H.c §A3.2.1 “Handshake process”, §A3.4.1 “Address structure” and its Table A3-3 “AxBURST signal encoding”) ^[9]. Its natural implementation is assertions attached to the interface, since signal-level relationships belong in the interface declaration that contains the signals ^[1] (Chapter 11 covers how they are written and bound). It knows nothing about intent and fires on the offending cycle. A **functional monitor** owns a different question — *what did the design do?* It reconstructs transactions from pins and publishes them; it does not judge.

The boundary between them is where a specific failure mode lives: a monitor that *infers* what it cannot observe has become a second design.

The split also survives a change in the nature of the boundary, which suggests it is architectural rather than a habit of digital design. On a mixed-signal block taken from a current LSI Corporation design — an analog driver with filtering and fault detection — the environment was built as wire UVCs whose driver and monitor functions are written in Verilog-AMS rather than in the testbench language, wrapped in the same self-checking sequences, functional coverage and regression control as a digital agent would be ^[13]. The boundary changed language; the responsibilities did not move.

Scenario. The flash controller’s LDPC ECC pipeline. A first environment gives its NAND-side monitor the job of recomputing the ECC codeword so it can report “correctable” or “uncorrectable” per page. **Approach.** Move the recomputation out of the monitor into a predictor on the other side of the analysis port; the monitor reports the raw codeword, the syndrome the design published, and the page’s status bits. **Why.** Two reasons. Debug: when monitor and design disagree, a failure inside the monitor’s own arithmetic is indistinguishable from a design bug, and it fires from a component nobody reviews. Independence: if the monitor reuses the design team’s LDPC routine — which is exactly what happens under schedule pressure — the check is common-mode and proves nothing (Chapter 3). Monitors report what they saw; predictors compute what should have happened.

The same discipline explains a subtler bring-up failure: an Ethernet MAC monitor that timestamps frames on the wrong clocking edge reports a constant one-cycle offset on every TSN measurement. Nothing in the design is wrong, every test fails, and the sampling seam of Section 8.2 is the only place to look.

8.6 Worked example: the crossbar environment, step by step

The crossbar is a full AXI4 interconnect: five managers (instruction fetch, data, two DMA channels, debug) by four subordinates (SRAM, boot ROM, APB bridge, neural accelerator), 32-bit address, 64-bit data, ID width 4. Here is how the environment gets built, in the order it should be built.

Step 0 — write the checking contract before any code. Four verdicts, each with a named owner: data integrity per ordered stream and completeness (nothing lost, nothing invented) belong to Step 4’s scoreboard; routing correctness to Step 3’s forward-path check; protocol legality on the nine interfaces to Step 2’s checkers. And three it will *not* produce: relative ordering between different transaction IDs, arbitration fairness, latency bounds. Name the owners as you name the verdicts — a promised verdict with no mechanism behind it is as dangerous as a check nobody wrote — and keep the negative list, which is the half teams skip and the half that stops a reviewer assuming a check that does not exist (Chapter 3).

Step 1 — one agent per interface. Nine agents, each owning exactly one interface [5]. Five manager-side agents are active: sequences feed a driver, a monitor watches. Four subordinate-side agents are responders — by Section 8.5’s criterion they are drivers, since their reply is under testbench control: an SRAM model with programmable latency and back-pressure, a ROM responder that returns an error response to every write, an APB-bridge responder with long variable latency, and an accelerator responder that can stall indefinitely. All nine carry a monitor, including the interfaces the environment itself drives.

Step 2 — monitors publish, subscribers consume. Each monitor broadcasts the transactions it reconstructs, to three kinds of subscriber: the per-interface protocol checker; the coverage collector, which samples the shaped route cross `cg_xbar_route` of Chapter 6 (the model, its `ignore_bins` and their justifications belong there — this environment only supplies the stream); and the checking path, which takes two edges on the manager side (requests to the predictor, responses to the scoreboard) and one on the subordinate side (the forward-path comparison). None of them can block the monitor, and none may modify the transaction it receives [4]. Figure 8.1 shows the resulting structure.

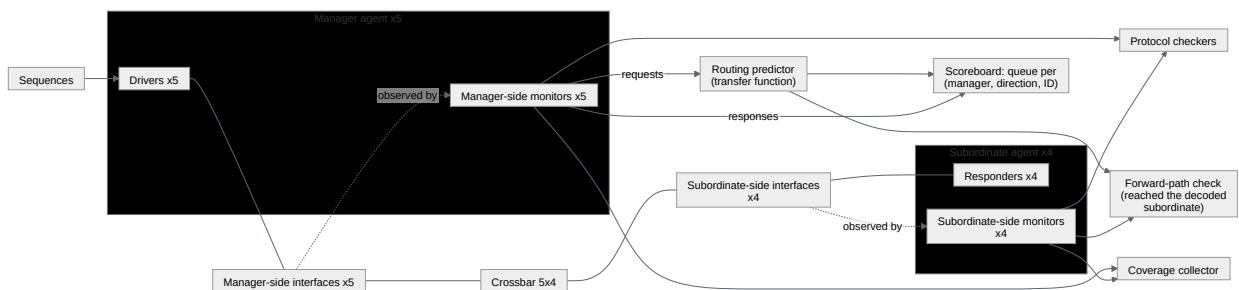


Figure 8.1 — The crossbar environment. Stimulus reaches the design through drivers and interfaces; checking is fed only by monitors, so no checker can observe a value the design never drove. The routing predictor supplies both the scoreboard, which matches responses per `(manager, direction, ID)`, and the forward-path check, which confirms the request reached the decoded subordinate.

Step 3 — the routing predictor, and the two paths it feeds. The predictor is a transfer function, not a reference model (Section 8.4): from a *monitored* manager-side request it decodes the destination using the runtime configuration descriptor ^[1] — the address map is data the test can change, never a constant in the checker — and emits one expectation record holding the originating manager, the decoded subordinate, the direction, the manager-side ID, the address and the expected beats. It is not golden: if its map and the RTL’s parameterization disagree, every access at a region boundary mismatches, and the right first move is to treat that as a specification question rather than a checker bug (Chapter 3).

That record feeds two comparisons, because the crossbar has two paths and each needs its own witness. The **forward-path check** matches it against the subordinate-side request monitor — did this address and this write data reach the decoded subordinate, and no other? — which is the verdict “routing correctness”. The **return-path check** is Step 4’s scoreboard, and it observes on the *manager* side, because read data flows subordinate → crossbar → manager. A subordinate-side response monitor watches the environment’s own responders reply; comparing predictions against that stream grades the testbench’s homework with the testbench’s own answer sheet, and never asks the question that matters — did subordinate 0’s read data reach manager 2 rather than manager 3, and in ID order?

Step 4 — the out-of-order scoreboard. A key acquires one dimension per independent source of concurrency, which is the rule Section 8.7 applies again when the flagship’s DMA grows to eight channels. Here there are three: five managers issuing at once, two independent ID spaces, and AXI’s per-ID ordering within one port and one direction (Arm IHI 0022H.c §A5.1 “AXI transaction identifiers”, §A5.2.1 “Read data ordering”) ^[9]. The stream key is the triple (manager port, direction, manager-side ID), one in-order queue per stream ^[8].

The tempting two-coordinate key — (subordinate, ID) — fails all three tests, which is where “name the stream key correctly” stops being a slogan. Manager 1 and manager 2 may both have ID 3 outstanding to subordinate 0 — the situation Chapter 10’s virtual sequence deliberately sets up — and one queue then matches one manager’s expectation against the other’s response, under some arbitrations only, so it surfaces as a mismatch that disappears on a seed change rather than as a reproducible bug. It merges `ARID` and `AWID`, which name unrelated streams, so a read expectation can be popped by a write response. And it misreads the protocol: an interconnect merging traffic from several managers must *widen* the ID toward the subordinate, precisely so that same-ID transactions from different managers are not falsely serialized. Five managers need $\lceil \log_2 5 \rceil = 3$ manager-identifying bits, so the 4-bit ID a manager issues arrives at a subordinate port as a 7-bit one; keying there is sound only with that widened ID, which obliges the predictor to reproduce the design’s ID-remap policy exactly. Keying on the manager side avoids modelling that policy at all, and respects the constraint every key must respect: **it is computable from what the observer sees.**

```
typedef enum bit {RD, WR} xbar_dir_e;
```



```

typedef struct {
    int unsigned mgr;           // originating manager port, 0..4
    int unsigned sub;           // destination decoded by the predictor, 0..3
    xbar_dir_e dir;             // read and write IDs are independent spaces
    logic [3:0] id;             // manager-side ID, as the manager issued it
    logic [31:0] addr;
    logic [63:0] beats [$];     // expected response data, one entry per beat
    time t_pred;               // when the request was monitored
    int unsigned capture;       // monotonic index, for the failure message
} xbar_exp_t;

class xbar_scoreboard;
    // One expectation queue per (manager port, direction, manager-side ID).
    local xbar_exp_t pending [int][$];
    local int unsigned n_capture;
    int unsigned n_matched, n_orphan, n_mismatch; // read by report()

    local function int stream_key(int unsigned mgr, xbar_dir_e dir,
                                logic [3:0] id);
        return (int'(mgr) << 5) | (int'(dir) << 4) | int'(id);
    endfunction

    // Called by the routing predictor when a manager-side monitor
    // reports a legal request. Never called from a driver.
    function void predict(xbar_exp_t e);
        if ($isunknown(e.id)) begin // never legal, and int'(id) would
            $error("xbar_sb: X/Z in a request ID, manager %0d", e.mgr);
            return; // silently file it under key 0
        end
        e.t_pred = $time;
        e.capture = n_capture++;
        pending[stream_key(e.mgr, e.dir, e.id)].push_back(e);
    endfunction

    // Called by a MANAGER-side monitor, so that what is checked is the
    // crossbar's return path and not the environment's own responder.
    // Zero-delay: it never blocks the monitor's thread.
    function void observe(input int unsigned mgr, input xbar_dir_e dir,
                        input logic [3:0] id, input logic [63:0] beats [$]);

        int k;
        xbar_exp_t head;
        if ($isunknown(id)) begin
            $error("xbar_sb: X/Z in a response ID, manager %0d", mgr);
            n_orphan++;
            return;
        end
        k = stream_key(mgr, dir, id);
        if (!pending.exists(k) || pending[k].size() == 0) begin
            n_orphan++; // a response nobody asked for
            $error("xbar_sb: orphan %s response, manager %0d ID %0d",
                dir.name(), mgr, id);
            return;
        end
        head = pending[k].pop_front(); // one manager, direction and ID:
        // these must return in order

        if (head.beats != beats) begin // != : an x beat never reads as equal
            n_mismatch++;
            $error("xbar_sb: %s mismatch, manager %0d ID %0d: exp %p got %p; \
predicted for subordinate %0d at %0t, capture %0d",
                dir.name(), mgr, id, head.beats, beats,
                head.sub, head.t_pred, head.capture);
        end
        else n_matched++;

```



```

endfunction

// End of test: anything still queued was never delivered.
function int unsigned leftover();
    int unsigned n = 0;
    foreach (pending[k]) n += unsigned'(pending[k].size());
    return n;
endfunction

// A silent scoreboard is a broken scoreboard: all four counts printed,
// three of them failures.
function bit report();
    int unsigned n_left = leftover();
    $display("xbar_sb: matched=%0d orphan=%0d mismatch=%0d leftover=%0d",
            n_matched, n_orphan, n_mismatch, n_left);
    if (n_left != 0)
        $error("xbar_sb: %0d expectations never answered", n_left);
    return (n_orphan == 0) && (n_mismatch == 0) && (n_left == 0);
endfunction
endclass

```

Four details carry weight. `predict` is called from the *predictor*, fed by a monitor, never by a driver — the rule of Section 8.1. Both entry points are zero-delay functions, so a checker can never back-pressure a monitor [8]. The comparison uses `!=`, not `!=`, because logical inequality on operands containing `x` or `z` yields `1'bx`, which `if` reads as false: with `!=`, a single undriven beat scores as *matched* — a green result from a comparison that could not go red. The `$isunknown` guards close the same hole one level up, since `int'(id)` is a two-state cast that would collapse an unknown ID to `0` and file a corrupt transaction under a real key. And `leftover()` exists because the interesting failure is usually silence: a response that never arrives produces no mismatch, only a queue that never empties [1]. `report()` is what makes all of it visible: counters nobody can read are the same as no counters at all.

Step 5 — what this environment cannot see. The canonical crossbar bug is write-data interleaving under back-pressure: beats belonging to two writes in flight to one subordinate appear interleaved on the wire, on a channel AXI4 obliges the crossbar to keep in address order (Arm IHI 0022H.c §A5.2.2 “Write data ordering”) [9] — interleaving write data with different IDs is deprecated in AXI4 and later rather than forbidden, and with no `WID` to tag a beat, address order is one burst at a time. The scoreboard is blind to it by construction, because a subordinate tolerant of the interleaving still stores the right bytes and returns the right responses — the violation is a relation *between* two transactions, not a property of either one’s data. It was found by protocol assertions in formal, with an 11-cycle counterexample (Chapter 14). That is the division of labor to internalize: the scoreboard owns data and completeness on the return path, the forward-path check owns routing, the protocol monitor owns legality, formal owns exhaustiveness on the legality properties that matter most — and fairness and starvation belong to none of them, which is why Step 0 listed them as out of scope.

Step 6 — make it debuggable before you need to. Every expectation record carries its originating manager, its decoded subordinate, a monotonic capture index and the time the request was monitored — which is why the mismatch message above

can say “RD mismatch, manager 2 ID 3 ... predicted for subordinate 0 at 41,320 ns, capture 8814” instead of merely reporting that two queues of bytes differ. The cost is four fields; the saving is the two days the opening scenario lost.

8.7 Worked example: the DMA environment and end-to-end checking

The DMA engine — two channels, 1D and 2D transfers, bursts up to 256 beats, a 16-entry × 64-bit datapath FIFO in the backend, per-channel error reporting — poses a different oracle problem. Nothing at any single interface tells you the transfer was correct. The question is end-to-end: *given this descriptor, do the destination bytes end up right?*

The environment therefore has a different shape. A register agent programs the frontend (Chapter 10 covers register abstraction); a byte-accurate memory model behind the backend’s AXI manager port serves reads and absorbs writes with randomized latency and back-pressure; a backend monitor publishes every burst; an error/interrupt monitor publishes completion and error events. The oracle is a **descriptor-level golden model**: a C model of the transfer semantics — source, destination, 1D/2D sizes, strides, element width — co-simulated through the language’s foreign-interface layer, which turns the programmed descriptor into the expected destination memory image.

Calling it golden is a deliberate claim, and it holds for a narrow reason: the transfer semantics of a data mover *are* the specification, and the model implements them at the specification’s own level of abstraction — bytes moved, not bursts issued. It says nothing about how the midend splits the transfer, which is exactly what makes it maintainable. This is not Section 8.4’s Species 1 legalizer model: that one predicts *how* the midend splits a transfer, and is a transfer function, as buggy as the design it mirrors; this one predicts only which bytes end up where, and is golden precisely because it refuses to predict the splitting. One IP, two models, on opposite sides of the classification — and the line between them is the level of abstraction each one commits to.

Checking then happens at two levels, and both are necessary.

Level 1, end-to-end and late. On descriptor completion, compare the memory model’s image against the golden image byte for byte, and the reported per-channel status against the model’s predicted status. This is the check that catches the canonical DMA bug — a 2D transfer with negative stride mis-legalized at the 4 KB boundary, corrupting data only when channel 1 is also active. Every burst the backend issued was individually legal; only the bytes were wrong.

Level 2, per-burst and immediate. Protocol assertions bound to the backend’s AXI channels, including the 4 KB-crossing property established in Chapter 2 and re-examined as a partial oracle in Chapter 3, plus the row-level legalization coverage model `cg_2d_4kb` named in Chapter 5’s plan row DMA-F06a and treated in Chapter 6. These fire on the offending beat.

Both levels exist because detection distance is a real cost: detect errors as early as possible, while the model is still near the failing state ^[1]. An end-to-end mismatch says 4 KB of destination memory is wrong at transfer completion — on this block, order-of-magnitude, tens of microseconds and hundreds of bursts after the mistake; the assertion says which beat. Neither substitutes for the other — the assertion cannot see wrong data, the image comparison cannot see *when*.

Two further decisions matter, because each is where teams over-specify and pay in false failures. **Do not check the split sequence:** comparing the exact burst sequence turns every legitimate change to midend arbitration into a regression failure. Check invariants instead — no burst crosses a page, the union of bursts covers the descriptor exactly once, byte counts match — and let coverage record which split shapes appeared. **Deliver asynchronous events to both sides:** when the responder injects an error response on beat 7, the golden model must be told, or it predicts a complete image while the design correctly reports a partial transfer — the data-mover form of the rule Section 8.4 drew from processor co-simulation ^[11]. And resist the bring-up temptation to let the model peek at the burst stream to “help” diagnose: that turns a golden model into a second implementation of the midend, with its own bugs and no independence left (Chapter 3).

The burden belongs in the plan: a change to legalization touches the RTL, the model and the coverage model, and the review must cover all three. The reward is reuse across a generation. The flagship’s DMA — eight channels, 40-bit physical addressing, descriptor chaining — inherits this environment rather than restarting it: the golden model gains an address-width parameter and a chaining loop, the scoreboard gains a channel dimension in its key, and the 4 KB discipline is unchanged. The canonical bug’s flagship descendant, where the split’s second half writes a line the host cluster holds Modified, needs exactly one new component — a coherence-aware memory model.

8.8 The environment as a design in its own right

An environment that will be reused, reconfigured and debugged for three years is a software design, and the properties that make it survive are not the ones that make it work on Friday.

Reuse has a precise contract. The self-checking structure belongs in a class that holds no reference to the verification environment around it, exposes a procedural interface through which stimulus, exceptions and observed responses are reported, and works with zero delay rather than as a process of its own — forking a blocking thread out of that interface on the rare occasion it needs simulation time, which is also why channels and mailboxes are the wrong interface to a checker ^[8]. The no-reference rule is the load-bearing one: a checker that reaches into the block-level environment cannot be instantiated in the system-level one, so the reuse everybody plans for at kick-off is foreclosed by an import statement in week three. Get it right and composition scales as the design does — one environment per IP, grouped into cluster environments, grouped into the top-level one ^[5].

Configuration is data. The design configuration descriptor has to reach the predictor, because the predictor performs the same transformation the design does and that transformation depends on configuration ^[1]. Once configuration is an object rather than a set of compile-time switches, it can be randomized, printed in the failure log and replayed, and the environment stops needing a rebuild per test.

Bring-up debuggability is designed in, not added. Four properties repay themselves within a week: every transaction carries an identifier and a timestamp; every failure message names expected, observed *and* the provenance of the expectation; the environment has a mode with checking disabled, to separate “stimulus cannot reach this” from “the check is wrong”; and — most neglected — the checkers are themselves tested by injecting a known-bad response and confirming the environment fails. An unfalsified checker is decoration (Chapters 2 and 3). Budget all four honestly: this will be the largest, most complex and most maintained part of the environment ^[1], it deserves the same review, coding standard, lint and regression discipline as RTL, and its schedule belongs in the plan (Chapters 4 and 5) rather than in the margin.

IN PRACTICE

Real environments are less orthogonal than this chapter’s diagrams. The most common compromise is a scoreboard that quietly knows one implementation detail — a reordering window, a prefetch depth — because predicting the legal set exactly was too expensive; teams that do this deliberately write the assumption in a comment and in the plan, and teams that do it accidentally rediscover it during the next integration, when the detail changes.

Checker-testing is the first thing dropped under schedule pressure, which is why teams keep finding regressions that were green because nothing in them could fail. The blunt remedy is two or three permanently broken RTL variants in the repository, run nightly, with the rule that if the environment does not fail on them, the environment is broken. Terminology stays messy too: “scoreboard” means the data structure to one engineer and the whole self-checking subsystem to the next ^[1]. Saying which you mean costs nothing and regularly prevents an argument about whether something is golden.

PITFALLS

1. **Checking from the driver’s intent.** *Symptom:* expectations computed from the transaction object the driver was handed, not from what the monitor saw. *Cure:* predict only from monitored transactions; the driver’s copy never reaches the checker.
2. **A monitor that drives.** *Symptom:* the monitor completes a handshake or asserts a ready signal “to keep things moving”; a real stall bug becomes invisible. *Cure:* hand monitors a modport that exposes only an inputs-only clocking block, so passivity is a compile-time property, and put every reply in a responder.

3. **One global scoreboard queue for a reordering design.** *Symptom:* failures that disappear when you change the seed, and a checker that is progressively relaxed until it checks nothing. *Cure:* name the ordering guarantee the specification actually makes, key the queues by it, and record the choice in the plan.
4. **No end-of-test completeness check.** *Symptom:* everything the design produced matched, so the test passes — and the transactions it never produced go unnoticed. *Cure:* fail on orphan responses and on leftovers ^[1], excluding only those the design's own drop-count statistics account for ^[8], and report both counts in every log.
5. **The over-specified reference model.** *Symptom:* the model predicts arbitration order, latency or burst decomposition, and legal design changes break the regression. *Cure:* predict what the specification promises; measure the rest with coverage.
6. **The reconstructing monitor.** *Symptom:* a monitor recomputes ECC, CRC or a protocol state it cannot observe, using the design team's own routine — and a regression that never goes red on an injected fault. *Cure:* monitors report, predictors compute from an independently written specification (Chapter 3), and known-bad variants prove the checkers can fail.

SUMMARY

- Stimulus, driving, monitoring, checking and coverage are five responsibilities with five failure modes. Merging any two silently deletes a capability: the oracle, passive reuse, or honest measurement.
- Layer signal → transaction → sequence, and let observation be broadcast and non-blocking so that measuring never changes what is measured ^[4, 5] — which is also what makes an environment retargetable to an emulator ^[7].
- A scoreboard is a data structure whose design driver is the lookup ^[1, 8]. Its key takes one dimension per independent source of concurrency and must be computable from what the observer sees; each step from one queue to a content-keyed index buys tolerance of reordering by giving up an ordering check.
- A predictor you wrote is a transfer function: not golden, as buggy as the design, and its disagreements are specification questions ^[1]. Only a model that *is* the specification is golden, and even then only about one projection of correctness. Co-simulation adds two conditions: compare at an architectural boundary, because models are functionally and not cycle accurate ^[2], and deliver every asynchronous event to both sides ^[11].
- Monitors never initiate and never drive, run autonomously so the design is never back-pressured by a late testbench ^[1], and observe where the design's own work actually lands: a checker fed by the testbench's own responder grades its own homework. An agent must work identically in active and passive mode ^[4, 5] — that switch is what carries a block environment into the system one ^[11].

- The environment is a design: a checker with no reference to its surroundings and a procedural interface is reusable, one without is not ^[8]. It will be the largest and most maintained thing you build ^[1]; plan, review and test it accordingly.

FURTHER READING

From the corpus. ^[1] Chapters 5 and 6 are this chapter's direct ancestors: bus-functional models, transaction-level interfaces, the verification harness, and the taxonomy of self-checking techniques — hard-coded response, data tagging, reference model, transfer function, scoreboarding — including the transfer function / reference model distinction Section 8.4 is built on. ^[8] supplies the implementation rules: encapsulation of the self-checking structure, its independence from the environment, its procedural zero-delay interface, and the scoreboard recommendations for in-order queues, per-stream queues, hashing indexes, assumed drops and margin-of-error comparison. ^[5] gives the component vocabulary and the analysis-port communication model, with ^[4] as its normative statement. ^[2] is the historical source for the decomposition and the transaction-level framing; ^[11] is a recent industrial account of a multi-level environment with ISS co-simulation and passive reuse. ^[7] shows what transaction-level layering buys on an emulator, and ^[3] Clauses 14 and 25 define the sampling and modport semantics every monitor depends on.

Not in the corpus. The UVM Cookbook (Siemens, continuously updated online) is the most widely used practical catalogue of these patterns, with worked scoreboard and agent code; treat its API detail as one instantiation of the vendor-neutral structure taught here. Mike Mintz and Robert Ekendahl, *Hardware Verification with SystemVerilog: An Object-Oriented Framework* (Springer, 2007), treats environment architecture as software design at book length. Both are background only.

REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. **P18** Y. Hollander, M. Morley, and A. Noy, "The e Language: A Fresh Separation of Concerns," Proc. Technology of Object-Oriented Languages and Systems (TOOLS 38), Zurich, Switzerland, pp. 41–50, 2001.
3. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
4. **S9** IEEE, "IEEE Std 1800.2-2020, IEEE Standard for Universal Verification Methodology Language Reference Manual (Revision of IEEE Std 1800.2-2017)," IEEE, 2020.
5. **S11** Accellera Systems Initiative, "Universal Verification Methodology (UVM) 1.2 User's Guide," Accellera, October 2015.
6. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.
7. **D33** A. Grove, "UVM hardware assisted acceleration with FPGA co-emulation," Proc. DVCon Europe, 2015.
8. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
9. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.

10. **S15** RISC-V International, "The RISC-V Instruction Set Manual Volume II: Privileged Architecture, Version 20250508 (Ratified)," RISC-V International, May 2025.
11. **P25** S. Ahmed, A. Kropotov, R. I. Genovese, B. Homs, E. Merino, F. Urbani, et al., "Verification and Validation (V&V)-in-the-Loop for RISC-V Design: The Holistic Vision of BZL," arXiv preprint arXiv:2604.27013v1, April 2026.
12. **D6** D. Graham, A. Sutton, S. Davidmann, P. Gouedo, X. Aubert, and Y. Pruvost, "Planning for RISC-V Success: Verification Planning and Functional Coverage," Proc. DVCon Europe, 2023.
13. **D19** N. Khan, Y. Kashai, and H. Fang, "Metric Driven Verification of Mixed-Signal Designs," Proc. DVCon U.S., 2011.

9. Stimulus: Directed to Random to Portable

The DMA engine’s testbench is three weeks old. It has a driver, a scoreboard and eleven directed tests, and every one of them passes. Test 1 programs a 64-byte 1D copy and compares the two buffers. Test 7 programs a 2D transfer with a positive row stride. Test 11 programs a transfer whose destination sits eight bytes below a 4 KB frontier, because someone read the AXI rule and wanted to watch the midend split a burst. The verification engineer looks at the list and asks the question that ends the directed era on every project that survives its first tape-out: *what is test 12?*

There is no shortage of candidates. Negative strides. Both channels at once. `NUM_REPS` at its maximum. A row exactly one maximum burst long, starting exactly one maximum burst below a page frontier, on channel 0, while channel 1 is draining. Each is plausible; each takes half a day to write and debug; and the list, honestly enumerated, does not have eleven more entries — it has several thousand. Writing them by hand is not merely slow: it guarantees that the bugs you find are the ones you already imagined.

The answer the industry converged on is to stop writing tests and start writing *rules* — describe what a legal descriptor is, let a solver produce them by the million, and measure what you got. That trade introduces a new class of failure, in which the stimulus looks busy and covers nothing.

CHAPTER MAP

- §9.1 establishes what directed tests are genuinely good at — bring-up, spec compliance, regression anchors — and the point at which they stop scaling; §9.2 records what the dedicated verification languages contributed to what replaced them.
- §9.3 develops constraints that separate *legality* from *distribution*, and shows why over-constraining is the quietest way to lose a verification project.
- §9.4 establishes what a seed actually reproduces, and what it does not.
- §9.5 layers stimulus from atomic transactions to virtual sequences coordinating several interfaces at once.
- §9.6 states what portable stimulus solves that constrained random does not, and where its honest limits are.

9.1 Directed tests: precision, and its ceiling

A **directed test** is stimulus whose content and order are written out by hand. Individual features are verified by individual testbenches, with the stimulus manually crafted to exercise that feature and the response checked against the symptoms that

would appear if the feature were broken ^[1]. Nothing is chosen at run time; the test does the same thing every time it runs.

That determinism buys three things no random environment supplies as cheaply.

Bring-up. The first stimulus a new block ever sees must be small enough to debug when everything is broken at once. When the DMA engine’s first descriptor fails, you need to know whether the fault is in the register decode, the midend, the backend, the scoreboard or the clock. A twelve-line directed test answers that in one waveform; a randomized 2D transfer with 47 rows and a negative stride answers nothing.

```
// Directed: the first descriptor the DMA engine ever executes.
task automatic dma_bringup_1d();
    bus_write(SRC_ADDR, SRAM_BASE);
    bus_write(DST_ADDR, SRAM_BASE + 32'h8000);
    bus_write(NUM_BYTES, 32'd64);           // 8 beats of 64 bits
    bus_write(CFG,      CFG_EN_1D_CH0);    // enable, 1D, channel 0
    do @(posedge clk); while (!(bus_read(STATUS) & ST_DONE));
    check_memory_equal(SRAM_BASE, SRAM_BASE + 32'h8000, 64);
endtask
```

Spec compliance. Some stimulus is enumerated for you by an external authority, and randomizing it is a category error. An HBM controller’s initialization and PHY-training sequence is a strictly ordered protocol with mandated wait states between steps: there is exactly one legal opening move, and the test that proves you make it is directed. The same holds for any published compliance suite — the value of running the industry’s list is that it *is* the industry’s list, not a sample from a space you invented.

Regression anchors. A short deterministic test that touches one feature and finishes in seconds is the ideal smoke test: it tells you within a minute of a commit whether the design still boots, and its failure is never ambiguous.

Where it stops. The directed approach can be managed to completion when the total number of test cases is in the low hundreds; a project with over a thousand identified test cases would need more than a year this way, and beyond that some form of testbench automation becomes necessary to complete the task within an acceptable time frame ^[1] — a schedule claim, not a feasibility one, and quite bad enough. The reference SoC’s DMA alone generated several thousand candidates in the paragraph above.

Two failure modes matter more than the arithmetic. The first is **bias**: directed test cases can only find bugs you were looking for ^[1]. Hand-written stimulus is a projection of the author’s model of the design, and where the model is wrong the stimulus is silent — Chapter 2’s opening suite contained no negative-stride case because its author’s mental model of a 2D copy went forwards.

The second is **decay**. A directed test can pass forever while ceasing to test what it was written to test: enlarge a FIFO or a memory and every test case that verified its full operation still succeeds, while now exercising only part of it. The defence is a redundant path — state through a coverage model what the test is supposed to accom-

plish and require it to fill 100% of those points, since deterministic stimulus should fill exactly what it targets ^[1]. A directed test that drops to 60% of its own goal has told you it went stale.

None of this retires the technique. In 2024, random tests are still described as covering the vast majority of easy-to-detect scenarios but unable to reach complex corner cases in reasonable time, with directed tests covering the remainder ^[2]. The mature position is a division of labour, stated crisply by one methodology: use a directed test when a scenario is too complex to describe or too unlikely to happen randomly — and note that it need not be 100% directed, since a directed sequence can be injected randomly within a stream of random ones, which may expose problems a purely directed run would not ^[3]. Keep that last clause. In a mature environment, *directed* describes a fragment of a run, not a run.

9.2 The constrained-random turn

The pressure that produced constrained random was not that directed tests were bad. It was that designs acquired bugs which reveal themselves only under convoluted scenarios — a cross of states among several state machines that is set up only by one particular transaction sequence — and beyond writing as many directed tests as possible, the only reasonable answer was random simulation ^[4].

From enumeration to description

The idea that changed the practice is small and radical: stop describing *a* run and start describing *the set of legal runs*. Constraints are formal, unambiguous specifications of design behaviour; in stimulus generation they define what input combinations can be applied and when. Because they are declarative they read like the specification rather than like a program, and because they are executable — stimuli are generated directly from them — the manual construction of a procedural testbench disappears, raising testbench writing to a higher level of abstraction much as RTL raised design above gates ^[4].

There is a second payoff. The same constraint that *generates* legal traffic on one block's input can *check* legality on a neighbouring block's output — the generator/monitor duality — which is what makes interface constraints reusable one level up the hierarchy ^[4]. Chapter 11 makes the same object an assertion; Chapter 14 makes it a formal assumption. One artifact, three hats.

The dedicated languages, and what survived them

That model was worked out in dedicated hardware verification languages before it reached a general standard, and the `e` language is its clearest surviving record. It reached engineers as a commercial product rather than as a research proposal: `e` came from Verisity Ltd., and Specman — Verisity’s flagship product, most of which, its designers noted, was itself written in `e` — is what put the language on projects [5]. In it, a data item is a struct whose fields carry declarative constraints, resolved during the run to generate a random but directed stream of data for the DUT [5]. Three of its ideas are now so ordinary that their origin is invisible:

- **A test is a layer of constraints, not a new generator.** A test specification, in its simplest form, adds a few constraints on top of an existing environment — extending an already-defined packet type with `keep i == j` and `keep kind != empty` rather than editing the generator.
- **Preference distinct from requirement.** Constraints could be marked soft: a hard constraint applied to the same field by another piece of code simply overrules them, so a general-purpose default could be overridden by a specific test without touching it.
- **Weights, possibly reactive.** A select-style constraint weighted the generator toward chosen alternatives, and those weights could be integer-valued functions of data sampled from the DUT — generation influenced by the current state of the device [5].

SystemVerilog absorbed the model into a general-purpose language, and the semantics are now normative in IEEE Std 1800-2023, *IEEE Standard for SystemVerilog* [6]. Class variables are declared `rand` or `randc`; `randomize()` generates values for all the *active* random variables of an object, subject to the *active* constraints, and returns 1 on success or 0 on failure — `rand_mode()` and `constraint_mode()` are what move a variable or a constraint block out of that set. Inheritance builds layered constraint systems, so a general-purpose model can be constrained into an application-specific one, and `randomize()` with `adds` constraints inline at the call site [6] — the `e` test-layering pattern, standardized. In the 2024 industry data, constrained-random simulation is the one simulation-based technique whose adoption is still climbing rather than having stabilized [7].

What the trade actually costs

Compared with the directed approach, a coverage-driven random approach trades longer initial testbench development for more productive feature coverage in the long run — and the source that says so immediately declines to quantify the gain, because it depends heavily on the team’s commitment and on its skill at writing generators that can be constrained easily from the outside [1]. Two commitments follow, both made on day one.

First, **you must measure**, because you no longer know what you ran. If you are not using functional coverage in tandem with a random environment, you are only doing directed test cases with random stimulus filling [1] — a sentence worth taping to a

monitor. The generator answers *what could happen*; only the coverage model answers *what did*.

Second, **the generator is an architecture, not a detail**. One able to exercise the required features does not happen by accident, and it is better to design a generator that can be constrained easily from the outside than to alter its code for each new test ^[1] — a decision that belongs with the environment architecture of Chapter 8, not inside a test file. Every forked generator is a directed test wearing a costume.

9.3 Writing good constraints

Constraints do two jobs that beginners conflate and experts keep rigorously apart: they define what is **legal**, and they shape the **distribution** within what is legal. Confusing them is the origin of most bad stimulus.

Legality is a claim about the design, not about the test

Stimulus constraints come in two families: *environment* constraints, which express the interface protocol and must be obeyed strictly, and *directive* constraints layered on top, which steer the run toward the scenarios you currently care about. The picture is three nested regions ^[4]:

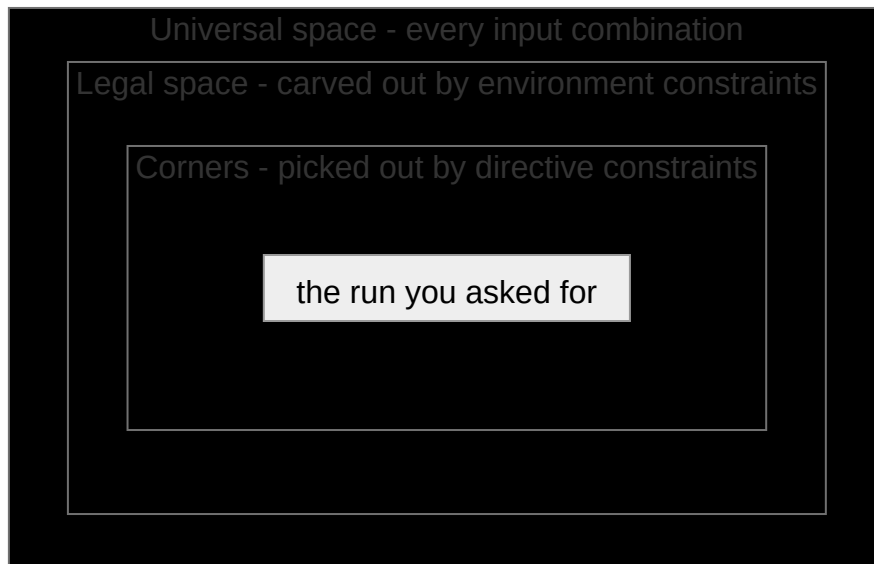


Figure 9.1 — The two families of stimulus constraint drawn as nested regions: environment constraints carve the legal space out of the universal space of every input combination, and directive constraints pick out the corners inside it that give the run you asked for. Which boundary a constraint moves is what decides where it belongs — the outer one is the design’s and lives in the generator, the inner one is a test’s and lives in a layer above it.

The outer boundary belongs to the DUT and lives in the generator; the inner one belongs to a test and lives in a layer above it.

Here is the DMA engine’s 2D descriptor generator, each constraint labelled by family. The DMA has a 64-bit data path and a maximum burst of 256 beats, so one max-

imum burst moves 2048 bytes — exactly half a 4 KB page. Keep that number; the worked example turns on it.

```
class dma_2d_descriptor;
  rand bit [31:0]   src_addr;
  rand bit [31:0]   dst_addr;
  rand int unsigned num_bytes;    // bytes per row
  rand int          src_stride;   // signed
  rand int          dst_stride;   // signed
  rand bit [15:0]   num_reps;     // rows
  rand bit          channel;
  rand bit [31:0]   dst_row[];    // the rows this descriptor will touch

  // --- legality: the frontend rejects anything else ---
  constraint c_bus_alignment {      // 64-bit data path
    src_addr[2:0] == 3'b000;
    dst_addr[2:0] == 3'b000;
    num_bytes[2:0] == 3'b000;
    src_stride[2:0] == 3'b000;
    dst_stride[2:0] == 3'b000;
  }
  // --- policy, NOT legality: an unmapped address is legal to PROGRAM.
  // Disable this one in the error-response test.
  constraint c_addressable {
    src_addr inside {[SRAM_BASE : SRAM_BASE + SRAM_SIZE - 1]};
    dst_addr inside {[SRAM_BASE : SRAM_BASE + SRAM_SIZE - 1]};
    foreach (dst_row[i]) // a negative stride must not walk the rows
      dst_row[i] inside {[SRAM_BASE : SRAM_BASE + SRAM_SIZE - 1]};
  }
  // --- policy: keep the randomized object small enough to solve fast ---
  constraint c_solver_budget {
    num_bytes inside {[8 : 2048]}; // <= one 256-beat burst
    num_reps inside {[1 : 64]};
    src_stride inside {[ -8192 : 8192]};
    dst_stride inside {[ -8192 : 8192]};
  }
  constraint c_rows { // definition, not policy
    dst_row.size() == int'(num_reps);
    foreach (dst_row[i]) dst_row[i] == dst_addr + unsigned'(i * dst_stride);
  }
endclass
```

Four details carry the lesson. The `c_addressable` comment is the most important line in the file: an address outside the memory map is legal to *program* — the register field accepts it — and what the DMA does with it (a decode error, reported per channel) is a feature to verify, not stimulus to suppress. Confining stimulus to what the design is *for* is the reflex Chapter 2 warns against, and a `constraint` block is the most convenient place in the language to commit that error permanently. Second, `c_solver_budget`'s bounds are engineering choices about solve time, not statements about the DUT; labelling them honestly lets a later test relax them without anyone wondering whether it has just generated something illegal. Third, alignment is a bit-select rather than `num_bytes % 8 == 0`: both correct, one enormously cheaper to solve. Fourth, `c_addressable` bounds `dst_row[]` and not just the two base addresses, because a negative stride otherwise walks the rows out of the map and round through zero. Deriving a field and forgetting to bound the derivation is the commonest way a generator produces addresses nobody meant to allow.

The uniform-distribution promise, and what it is not

What the solver owes you is precise: random values shall be selected to give a uniform distribution over legal value *combinations*, every legal combination equally probable, so that randomization explores the whole design space [6]. That is a strong guarantee — about the *solution space defined by your constraints*, which is not the space your coverage model measures. Uniform distribution over the solution space does not reflect the intention of covering as many functional scenarios as possible; the distribution most tools provide is derived from the constraint model and is simply not aligned with the coverage model. In one published example, packets constrained only for internal consistency were measured against a coverage model asking for the extreme edges of the address range: over a run of a thousand generated instances most of its objectives were left uncovered, and adding distribution directives closed the coverage completely [8]. The constraint model contains no directive that could steer toward the edges, because nothing in it knows the edges matter.

Scenario. The radar front-end's FFT pipeline takes 16-bit Q1.15 samples. **Approach.** Every 16-bit pattern is legal, so a bare `rand bit signed [15:0]` is a correct generator and a nearly useless one: the bugs live at the ends — saturation on the most-negative value, whose negation does not exist in the format — and uniform over 65,536 values visits each end with probability $1/65,536$. `systemverilog constraint c_dynamic_range { iq dist { 16'sh8000 := 3, // most negative 16'sh7fff := 3, // most positive 16'sh0000 := 2, [16'sh8001 : -1] :/ 46, [1 : 16'sh7ffe] :/ 46 }; }` **Why.** The legal set is unchanged; only the probability mass moved. Weighting distinguished values against broad ranges is the standard idiom for making a corner case frequent without ever making an illegal one possible.

Shaping: `dist`, ordering, and `soft`

Three constructs do the shaping, and none of them can make the solver fail.

`dist` assigns weights to values and ranges: absent any other constraints, the probability that the expression matches an item is proportional to that item's weight; `:=` applies the weight to each element of a range while `:/` divides one weight across the whole range; the weights are ratios and need not be normalized to 100; and — crucially — nonzero weights do not change the solution space and therefore cannot cause the solver to fail. Keep that opening qualifier, because it is doing work: any other constraint on the same expression renormalizes whatever is left. One asymmetry deserves care: values *absent* from the list, and values whose *total* weight is zero, are treated as a constraint forbidding them [6]. A `dist` whose ranges do not cover everything legal is silently an over-constraint.

`solve ... before` changes probabilities by ordering the choice, and the canonical arithmetic is worth internalizing. With `s -> d == 0` over a 1-bit `s` and a 32-bit `d`, `s` is true in exactly one of the $1 + 2^{32}$ legal combinations, so `d == 0` occurs with probability $2/(1 + 2^{32})$ — effectively never. Adding `solve s before d` leaves the legal set identical but makes `d == 0` occur with probability slightly over 50%. Variable ordering forces selected corner cases to occur more often and, like `dist`, cannot cause

the solver to fail because it does not change the solution space [6]. It is the cure for the “control bit implies a narrow data case” pattern that appears in every configuration descriptor ever written.

`soft` separates preference from requirement. Hard constraints must be satisfied or the solver fails; a soft constraint is discarded when no solution satisfies it alongside the active hard ones. Its purpose is layering: the author of a generic component supplies defaults that a later, more specialized constraint overrides automatically, where a hard default would force the test to disable it procedurally or extend the class just to replace it [6]. In the DMA generator, “most descriptors are small” belongs in a soft constraint; “addresses are 8-byte aligned” does not.

Over-constraining: the silent killer

There are two ways to over-constrain, and only one of them is loud.

The loud one is unsatisfiability: when the constraints admit no solution, not a single input is allowed at the current state, that state is therefore a dead-end and simulation has to be aborted; over-constraints stall the progress of simulation and lead to vacuous proofs in formal verification, which is why constraint diagnosis — finding a minimal set of constraints actually responsible for the conflict — is a tool feature and not a nicety [4]. You see it within seconds, as a failed `randomize()`. Checking that return value can be necessary for exactly this reason [6]; treating it as mandatory is the practitioner’s rule on top, because an unchecked failure leaves the object holding its previous values and the run continues, cheerfully re-sending the last legal descriptor forever.

The quiet one is worse. The constraint set stays perfectly satisfiable; it has merely deleted the feature under test. A lingering constraint can prevent generation of exactly the input sequences that would fill the coverage points that stubbornly stay empty [1] — nothing fails, nothing warns, and coverage climbs to a plateau everyone reads as “the hard part is left.”

Worked example — the line that hides the canonical bug. Week 4 of the DMA campaign. Reviewing failures, an engineer notices descriptors whose rows straddle a 4 KB frontier, remembers that no AXI burst may cross a 4 KB page (Arm IHI 0022H.c §A3.4.1 “Address structure”) [9], concludes the stimulus is illegal, and adds:

```
class dma_desc_overconstrained extends dma_2d_descriptor;
  // Week 4: "the AXI spec says no burst may cross a 4 KB page."
  constraint c_no_page_cross {
    foreach (dst_row[i])
      (32'(dst_row[i][11:0]) + num_bytes) <= 32'd4096;
  }
endclass
```

The constraint is *true of the design’s output* and *false of its input*. The 4 KB rule governs the bursts the backend may emit (Arm IHI 0022H.c §A3.4.1 states the rule of a burst, which is what makes the constraint wrong of the *input*) [9]; splitting a request that would cross a page is precisely the midend legalizer’s job — the feature the

campaign exists to verify. The engineer has taken the DUT's obligation and imposed it on the stimulus, and the tests keep passing because the DUT is no longer asked to do the thing it gets wrong.

The damage is measurable against the canonical coverage model, `cg_2d_4kb` (Chapter 5): four axes — stride sign, the destination row's relation to the next 4 KB frontier, burst length, other channel active — crossing to $3 \times 4 \times 3 \times 2 = 72$ cells. The new constraint makes the `straddle` bin of `cp_boundary_dist` unreachable, and with it every cell containing it: $3 \times 1 \times 3 \times 2 = 18$. But not all eighteen were ever reachable, and the difference is worth deriving, because it is the same derivation a waiver needs.

The two axes sample at different scopes: `cp_boundary_dist` classifies a destination row, while `cp_len` samples the length of a burst the midend actually emits, after any split. `c_solver_budget` caps a row at 2048 bytes — 256 beats on the 64-bit path, one maximum burst, half a page — so a row reaches past at most one frontier, and a straddling row splits into exactly two bursts whose beat counts sum to the row's, at most 256, each of them at least one. Neither piece can be 256. `straddle` $\times$ `max` is therefore empty for every stride sign and both channel states: **6 of the 18 were unreachable before anyone wrote the week-4 line**, and what `c_no_page_cross` costs is the other **12**. The canonical DMA bug — negative stride mis-legalized at the 4 KB boundary, corrupting data only when channel 1 is also active — sits in `neg` $\times$ `straddle` $\times$ other-channel-active, one cell per burst-length bin; the `max` one sits in the excluded column, so the constraint deletes **both** of the bug's live cells.

Two caveats belong on vplan row DMA-F06a rather than in a footnote. Those six are excluded by *policy*, not by geometry: lift the solver budget and a 4104-byte row at offset 4088 splits into 1 beat and 512, and the 512 splits again into 256 + 256 — `straddle` $\times$ `max`, reachable after all. And the whole argument is destination-side: `c_no_page_cross`, `c_addressable` and `dst_row[]` all speak about writes. Chapter 5's *100% means 100% of the cells the model can legitimately reach* is exactly this bookkeeping, and it is why that chapter insists on separating what is impossible from what was merely decided; an unrecorded exclusion is how a sign-off ends up quoting 72.

What the team sees is none of that. It is a coverage report with one clean block of cells permanently empty — a whole value of one axis missing from every combination it appears in. No test fails. The block looks like a hard corner, gets an owner and a due date, and slides.

How the hole is actually found. Three habits catch it:

1. **Read empty cells as a generator question first.** A gap in functional coverage could also be a symptom of a functional problem in the random generators or in the verification environment ^[1] — a possibility, not a verdict, but by far the cheapest one to check. Structure decides which: a whole *value* of an axis missing from every combination — a clean projection, not scattered cells — says a dimension of the stimulus is switched off, not that the corner is hard (Chapter 6).

2. **Diff the constraint set against the coverage model.** For each axis of `cg_2d_4k`, find the generator variable that moves it. For `cp_boundary_dist` there is none — only a constraint pinning it.
3. **Aim, and watch it fail.** Ask for the missing *cell* — for the relation that defines it, not for a field you hope moves it — and let the failure be the diagnosis.


```
systemverilog // aim at neg x straddle: a destination row reaching past the
frontier if (!desc.randomize() with { dst_stride < 0; (32'(dst_addr[11:0]) +
num_bytes) > 32'd4096; }) $fatal(1, "over-constrained: no descriptor reaches
past a 4 KB frontier");
```

 The second clause carries the diagnosis. Against `dma_2d_descriptor` it is satisfiable — offset 4088 with a 2048-byte row reaches 6136 — and against `dma_desc_overconstrained` it cannot be, because `c_rows` puts row 0 at `dst_addr`, so `c_no_page_cross` asserts precisely the negation of what you asked for. Aim at `dst_stride` alone and the same generator answers cheerfully with an 8-byte row, which fits inside a frame at every legal offset: a diagnostic that cannot fail diagnoses nothing.

Then shape. Deleting `c_no_page_cross` restores reachability, and the straddle case is then not even rare. But two axes remain badly served by uniformity: the maximum burst length is one value out of 256 legal ones, and the negative stride, though legal, carries no more probability mass than the positive one nobody doubts. Shaping fixes both without touching legality:

```
class dma_desc_shaped extends dma_2d_descriptor;
  constraint c_len_extremes {
    num_bytes dist { 2048 := 20, 2040 := 20, [8 : 2032] :/ 60 };
  }
  constraint c_frontier {
    (32'(dst_addr[11:0]) + num_bytes) dist {
      [0 : 4088] :/ 35, // where row 0 ends, mod 4 KB
      4096 :/ 15, // ends before the frontier
      [4104 : 6136] :/ 50 }; // ends exactly on it
    // reaches past it: must be split
  }
  constraint c_stride_sign {
    dst_stride dist { [-8192 : -8] :/ 55, 0 :/ 5, [8 : 8192] :/ 40 };
  }
  constraint c_order { solve num_bytes before dst_addr; }
endclass
```

Note what `c_frontier` constrains: not an address, but the *expression the coverage model classifies*. Shaping a raw field and hoping the interesting combination emerges is guesswork; shaping the expression that defines the bin is aim. Be exact about how much aim, though. The third range maps onto `straddle`; the first two land somewhere among the other three bins without choosing which, since those are classified against a burst's worth of headroom rather than against the frontier — and the expression names row 0, while `cp_boundary_dist` samples every row. One bin of four, on the first of up to sixty-four rows, is still aim. Claiming four would not be. `c_stride_sign` then gives the negative stride *more* mass than the positive one: the register field is signed, software rarely uses it, and unloved legal inputs are where bugs accumulate.

One honesty clause holds the subsection together. `c_len_extremes` and `c_frontier` both constrain expressions containing `num_bytes`, and `c_order` orders one of them against `dst_addr`. Nothing here breaks a rule — no `randc` variable is involved and each distributed expression contains a random one — but it buys less than it looks. Weights hold *absent any other constraints*; where other constraints make some of them unsatisfiable, an implementation is required only to satisfy the constraints [6]. And the standard says nothing about how a `dist` and a solve order interact (IEEE 1800-2023 §18.5.3 and §18.5.9): not forbidden, not defined. So the arrow from `solve num_bytes before dst_addr` to “length first, address after” is intent, not mechanism. Two distributions over overlapping expressions are a request, and you learn whether it was granted from the coverage report. With the three in place the negative-stride straddle cells fill within a night’s regression: the scoreboard reports a destination mismatch, and the seed that produced it is the bug report.

Run habit 2 once more, though, and one axis still fails it. Nothing in `dma_2d_descriptor` moves `cp_other_ch`, because “the other channel is active” is not a property of a descriptor at all — it is a relation between two streams, and no amount of shaping inside one generator reaches it. Both reachable cells of the canonical bug lie on the other-channel-active side, so this generator, however well shaped, reaches neither. Getting there needs a second descriptor on channel 1 inside the same randomization, which is what Section 9.5 is for.

Two costs: the solver’s, and yours

Constraint solving is not free. Even though constraints are concise, the complexity of solving them is easily multiplied by the hundreds or thousands of constraints and variables present when verifying a unit- or block-level design, and fast solving is critical to simulation throughput and therefore to coverage. The hardness underneath is real, and worth stating exactly as the literature states it: generic constraint solving is NP-hard; SAT is NP-complete and its solutions are *believed* to have an exponential worst case; SAT performance varies widely across problem instances that look similar; and SAT is one solving approach among several, alongside linear and integer programming, ATPG, binary decision diagrams, Boolean unification and constraint logic programming [4]. Keep the register — *believed* — and keep the plurality: which engine your tool runs is a choice of the tool’s, not a property of the problem. In practice: keep the randomized object small, prefer bit-selects to arithmetic, and treat a generator that suddenly takes 40 seconds per transaction as a bug report rather than a fact of life.

Shaping by hand has its own overhead — the reason Chapter 6’s automation research exists. Using distribution and ordering directives well demands understanding both their semantics and the solving process, because distribution constraints carry order semantics whose misunderstanding yields a distribution different from the one intended; in a large model the relations between fields are tangled, so a directive on one field can change the distribution of others; and the engineer must know the whole coverage model, since not all objectives can be met at once. Reordering the distribution *constraint statements* of a working example is by itself enough to change the resulting distribution [8] — the statements, note, not the items

inside a `dist` list. Hence a review rule: distribution constraints are read as carefully as legality ones, and whoever writes one names the bin it aims at.

9.4 Seeds and reproducibility

Once stimulus is generated, a test is no longer a thing you run — it is a thing you *sample*. The unit of work becomes a **run**: an execution with a specific seed and a specific set of source files, producing a specific set of messages and coverage metrics [3]. That definition is more careful than the shorthand “a run is a (test, seed) pair”, and the extra clause is where reproducibility is usually lost. The same seed against a different RTL revision, a different testbench commit, or a different simulator build is a different experiment.

Reproduction is therefore an infrastructure requirement, not a habit: it must be *easy*, not merely possible, which means it must be simple to know which versions of which sources, tools and libraries were used and to reissue the exact command, especially the seed — and since command-line options cannot be source-controlled, a script should set the detailed options from broad, high-level ones. One honest limit belongs on the record: a testbench reused across simulator and constraint-solver versions may generate different stimulus for the same seed [3]. A seed is a pointer into a generator’s output; change the generator’s implementation and the pointer dangles.

Random stability keeps seeds useful while the testbench changes. The RNG is localized to threads and objects, so the values one object or thread produces are independent of the others; each class instance gets its own RNG, seeded from the thread that creates it, and each new dynamic thread from its parent — hierarchical seeding — so a whole subsystem can be pinned by manually seeding one root thread. The property that pays your rent is this: stability is preserved as long as object creation, thread creation and random-number generation happen in the same order as before, so new objects, threads and draws should be added *after* existing ones, at the end of a code block, to keep previously created work reproducible [6]. This is why inserting one innocuous `$urandom` call at the top of a sequence can re-roll an entire regression, and why the fix is to append rather than insert.

When to add seeds, and when to stop. If input coverage is low and only a few runs have happened, run more seeds; if it is still low after many runs, the problem is the generator’s distribution, not the sample size — 8 of 10 opcodes seen after three runs is noise, the same result after twenty-five is a defect. Where extra seeds stop paying, constraints start: a constraint that drives an unfilled point toward certainty is more work, and likely the more productive avenue, especially once hundreds of previous runs have failed to produce the inputs that point needs [1]. Where the new constraint goes depends on what went wrong. A test that missed points it was *designed* to hit is a broken test: fix that test. Points it was never designed to hit belong in a new one, so the existing test keeps working and keeps the coverage it earns [3].

One long run or many short ones? Both produce equal-quality random data if the source is truly random and the seeds do not repeat a previous sequence, but their

project effect differs sharply: a long run is cumbersome to reproduce when the error appears near the end, exercises a single device configuration, and can park the DUT in one corner of the state space where further stimulus adds nothing, whereas many short runs reproduce faster, sweep more configurations and traverse the space more quickly ^[1]. Prefer many short runs; Chapter 12 turns this into a regression economy.

What randomness costs. It moves effort from writing stimulus to reading failures, and that side of the ledger is already the heaviest: in 2024, verification engineers spend more of their time on debug than on any other activity ^[7]. A directed failure hands you the intent; a random failure hands you a log and a seed. Two cheap practices blunt this: have every generated item print a human-readable description of itself, and record the seed in the failure report so triage begins with a command that reproduces rather than a request for one.

9.5 Sequences and stimulus layering

A generator emitting individually constrained transactions covers a surprising amount of ground and then stops, because the interesting failures are not in the transactions but in their *relationships*. A **scenario** is a sequence of random or directed stimulus that is particularly interesting to the device under test and unlikely to be spontaneously generated with individually constrained-random stimulus ^[3]. That definition is the whole argument for layering, which has three levels, each existing because the one below cannot express something.

Atomic. Generators producing individually constrained transactions, suitable wherever constraints across a *sequence* are unnecessary — a configuration descriptor is the canonical case ^[3]. For the DMA, one descriptor.

Scenario. Sequences of random transactions with defined relationships, each aimed at a particular functional corner. The implementation trick: the scenario is described and randomized as an *array* of items solved all at once, letting the solver relate past, current and future items in one constraint set ^[3]. You cannot express “the third descriptor reuses the second’s destination” by randomizing three descriptors in turn.

Scenario. A USB controller’s link power states (*Universal Serial Bus 3.2 Specification*, Revision 1.1, USB 3.0 Promoter Group, June 2022, section 7.5, “Link Training and Status State Machine (LTSSM)”) ^[10]. **Approach.** The atomic item is a packet; the scenario is a link-state excursion — U0 to U1, an inactivity timeout to U2, a remote wake back to U0 — with transfers scheduled *relative to* the transitions. **Why.** No packet-level generator, however well constrained, will spontaneously start a transfer in the same cycle the link begins a low-power entry: that is a relationship between two items and a timer, not a property of either item.

Multi-stream (virtual). The third level exists for one hard technical reason, worth stating precisely: constraints cannot be expressed *across* scenarios in different streams, because those items are generated in different generators and are not inside the object any one generator randomizes — expressing cross-stream relations

requires that the descriptors for all streams be randomized as a single object. Every virtual-sequence mechanism in every methodology is an answer to that sentence. Three arrangements are on offer, as alternatives rather than as a ladder: synchronize otherwise independent generators; let a single-stream scenario descriptor forward some of its items to other streams; or use a true multi-stream generator that randomizes one descriptor holding a sub-descriptor per stream. The first is enough when the scenario needs only the initial synchronization of normally independent streams — it will force an Ethernet MAC collision by starting all transmitters together — and it is not enough when the scenario needs the *contents* of the individual streams cross-correlated, or individual transactions within them synchronized [3].

Worked example — one object, three streams. Two canonical bugs need conjunctions no single-stream generator can request, and they need the same object. The neural accelerator’s weight prefetcher wraps its buffer pointer one entry early when the weight stream ends mid-beat *under DMA contention*, reachable only at 2-bit weights with a width-1 tensor. The DMA’s own 4 KB mis-legalization corrupts data only when channel 1 is also active — the axis Section 9.3 could not reach from inside one descriptor. Both are relations between streams, so both go into one randomization:

```
class contention_scenario;
  rand dma_2d_descriptor dma_ch0; // the descriptor under test
  rand dma_2d_descriptor dma_ch1; // the other channel: what cp_other_ch samples
  rand nn_job_descriptor nn;
  rand int unsigned skew_beats;

  function new();
    dma_ch0 = new();
    dma_ch1 = new();
    nn      = new();
  endfunction

  constraint c_channels {
    dma_ch0.channel == 1'b0;
    dma_ch1.channel == 1'b1;
  }
  constraint c_contend { // one SRAM bank, all three
    nn.wgt_addr[4:3] == dma_ch0.dst_addr[4:3];
    dma_ch1.dst_addr[4:3] == dma_ch0.dst_addr[4:3];
    // where in channel 0's stream the accelerator's weight fetch lands
    skew_beats inside [{0 : (32'(dma_ch0.num_reps) * dma_ch0.num_bytes) >> 3}];
  }
  constraint c_worst_cell {
    nn.weight_bits == 2;
    nn.tensor_width == 1;
    dma_ch0.dst_stride < 0;
  }
endclass
```

The 512 KB SRAM is word-interleaved across four banks, so `addr[4:3]` selects the bank on a 64-bit word grid: equating those bits turns “all three are busy” into “all three are contending”. `skew_beats` is the scenario’s real variable — the bug is not in the overlap but in *where* in the transfer the accelerator’s stream ends — and it is a

beat index into channel 0's own transfer rather than a fixed cycle count, because a transfer of up to 64 rows runs for thousands of cycles and a constant range would only stagger the starts.

Two clauses deserve separating out. `c_worst_cell` is a *test* layer: it belongs in the test hunting this cell, not in the environment. `c_channels` closes Section 9.3's open axis — a descriptor on channel 1 is what makes `other_channel_active` true while `cg_2d_4kb` samples channel 0's transfer. Note also what the class does *not* do: constraints state relations, not schedules. Pinning channel numbers and bank bits guarantees the three descriptors *would* contend if they ran together; making them run together is the job of the sequence that starts them, and a scenario class without that step has aimed carefully at a cell it never fills.

Two points close the section. An Ethernet MAC with TSN queues shows the layering paying off at the top: the atomic item is a frame, the scenario is a traffic profile (a burst class at a stated line occupancy), and the multi-stream descriptor is what lets a high-priority profile on one queue and a best-effort flood on another be *correlated in time* — the only way to provoke a shaper-arbitration corner. And layers must be interruptible: the scenario layer may be partially or completely bypassed by the test above it, so generators must be stoppable, at the start or mid-run, to let directed stimulus be injected, and restartable afterwards ^[3] — the machinery behind Section 9.1's closing claim. Expressing all this in a particular methodology is Chapter 10's subject.

9.6 Graph-based and portable stimulus

Everything so far assumes a testbench that owns the interfaces. At the SoC level that assumption breaks, and the diagnosis AMD published after hitting the wall is specific: at IP level the constrained-random testbench is excellent, but the initialization sequences it contains are not easily portable to firmware or post-silicon tests, and IP-level tests lack system context; at SoC level the stimulus is simple directed feature tests, complex scenarios are difficult to create by hand, and run times are long; post-silicon, tests are OS-based, debug takes longer, and failures are difficult to bring back to emulation or simulation ^[11]. That diagnosis has an owner: it is AMD's, set out in the deck that also describes the flow the company built to answer it, so read it as the experience of a team that hit this wall rather than as a survey finding. Three platforms, three incompatible stimulus formats, no reuse across them.

There is a second, narrower gap. Transaction-level sequences randomize transaction *contents*; the transaction *flow* is not itself randomized, and randomizing *between* sequences is difficult. You can express one resource-contention configuration in a class with constraints — the streams of a display controller competing for pipes, overlays and interfaces — but that captures one specific scenario, not the ability to start and stop streams while others are in progress or to schedule them arbitrarily; it does not capture the true nature of the problem, which is tasks contending for limited resources ^[12].

What PSS is. The Portable Test and Stimulus Standard (Accellera Systems Initiative), at version 3.0 in August 2024, defines a single representation of stimulus and test scenarios, usable across levels of integration and configurations, from which implementations can be generated for simulation, emulation, FPGA prototyping and post-silicon. It is a declarative domain-specific language for modelling *scenario spaces*, and its constraint and coverage syntax takes SystemVerilog as its reference [13].

A model is a set of class definitions plus rules constraining their legal instantiation. Behaviour is an **action** — atomic if it maps directly to an operation of the system, compound if it encapsulates a flow of other actions in an **activity**. Actions consume and produce passive **objects** whose type fixes the scheduling: a *buffer* is persistent data whose producer must complete before its consumer starts, a *stream* is transient data requiring producer and consumer to run in parallel one-to-one, and a *state* object may be read concurrently but written only exclusively. Actions may also claim **resources** from sized **pools**: a device with two DMA channels is modelled as a pool of two channel resources, so no more than two channel-locking actions may execute at any given time — the reference SoC’s problem exactly, and the standard’s own example of it. A **scenario** is one particular solution of all those constraints [13].

What distinguishes PSS from a constraint class is **inference**. Where the specification of intent is partial, the tool shall infer the additional actions and model elements needed to make it complete and valid, and shall infer nothing that is not required — so one partial specification expands into a variety of scenarios that all satisfy the stated intent while including other behaviours that broaden coverage (PSS 3.0 §5 and §5.3.2) [13]. Practically: you write “do a 2D transfer, then run an inference job”, and the tool works out that the transfer needs a free channel, that the inference job’s input buffer needs some action to have produced it, and that the accelerator must have been configured first.

```
// Sketch: both DMA channels and the accelerator, inside one scenario.
buffer    mem_block_s { }
resource channel_s    { }

component dma_c {
  action transfer_2d {
    lock channel_s ch;                // holds one channel for its duration
    rand int in [-8192..8192] dst_stride;
    output mem_block_s block;        // a buffer output needs no consumer
  }
}

component nn_c {
  action infer_job {
    rand int in [2,4,8] weight_bits;
  }
}

extend component pss_top {
  dma_c dma;
  nn_c nn;
  pool [2] channel_s channels;        // the reference SoC has two
  pool    mem_block_s blocks;
```

```

bind channels *;                                // every object needs a same-type pool
bind blocks  *;

action contention {
  activity {
    parallel {                                  // synchronized start, no ordering across
      do dma_c::transfer_2d with { dst_stride < 0; };
      do dma_c::transfer_2d;                    // the other channel
      do nn_c::infer_job    with { weight_bits == 2; };
    }
  }
}

```

Three things there are load-bearing. The pool depth is the model: two channels means at most two channel-locking actions may hold one at a time, each with a distinct instance, and a scenario that would over-subscribe the pool is not generated at all. Shrink `channels` to one and this activity becomes illegal rather than merely slower — capacity is a rule the generator must satisfy, not a queue it waits in. Second, every object reference is bound to a pool of its own exact type, which is what makes the model legal at all; a buffer output may have no consumer, where a stream output may not (PSS 3.0 §5.1.1, §5.1.2). Third, `parallel` is not a promise of simultaneity: what the standard guarantees is a synchronized start and the absence of any scheduling dependency across branches, and two executions with no dependency between them may or may not overlap in time (PSS 3.0 §6.3.1, §6.2.3) ^[13]. Its weaker sibling `schedule` lets the tool pick any legal order at all, including one branch entirely after another — which is how a scenario named *contention* ends up contending with nothing.

Retargeting is the other half. The abstract model says what the system does; *test realization* says how, mapping atomic actions to target code. The order in which actions execute must conform to the flow the activities dictate *and* be correct with respect to the model rules ^[13] — which is how inferred actions, called by no activity at all, get their place in the generated test. The same `contention` action becomes transaction sequences driving bus VIP in a block-level simulation and a bare-metal C program running on the core in emulation or on silicon.

AMD’s published account of doing exactly that is the concrete case. PSS-described intent was compiled into UEFI and bare-metal tests that ran unchanged on an AMD APU, first under emulation and then on post-silicon parts, reaching the hardware through Cadence Perspec and an in-house AMD runtime layer called uDREX whose job was to abstract the BIOS and provide services to the generated tests; in the lab the same flow was then used to generate fresh power-management firmware tuning scenarios ^[11]. Note what is being claimed and what is not: the account reports a flow that retargeted, and a use it was put to, with no speed-up, bug count or effort saving attached.

Honest limits. A published industrial trial from 2019 recorded six real-world observations ^[11]. One of them — that inputs and outputs for activities were not available

yet — describes a 2019 toolchain and should be checked against whatever you are offered today. The other five are properties of the approach, and still bite:

1. Generic actions and activities can be complex to create, and may need procedural description — the declarative model does not cover everything.
2. PSS tests prefer complete control of the system: a bare-metal environment with statically allocated and scheduled tests.
3. Getting to that environment needs a bare-metal library, and building one needs multi-discipline team collaboration.
4. Randomization should be carefully thought through, because test *generation* time has the potential to increase exponentially. Section 9.3’s solver cost reappears at scenario scale, before simulation even starts.
5. Constraint solver errors can be terse — and diagnosing an over-constraint across an inferred graph is harder than diagnosing one in a class.

Two more come from the standard rather than from that deck. The value concentrates in scenario composition rather than in checking: for expected-result computation, memory modelling and allocation policies, external models, software libraries or algorithmic code in other languages may need to be employed (PSS 3.0 §1.4) ^[13]. The standard permits them; it does not supply them, and specifying results checking inside the model remains a stated goal rather than a delivered construct. PSS generates the test; the oracle remains your problem (Chapter 3). And the realization code is per-platform work, so *portable* describes the model, not the effort. PSS buys reuse of intent across platforms and automated composition of multi-engine scenarios — exactly what Section 9.5’s layering runs out of at the top — at the cost of a modelling language, a bare-metal runtime, and a team that can debug a generator.

IN PRACTICE

No mature team runs one style. On a settled block you will find a handful of directed tests that never change (bring-up, one compliance run, three smoke tests), a constrained-random generator carrying most of the cycles, a growing folder of test-layer constraint files each aimed at a named coverage bin, and a short list of scenario classes for the conjunctions random cannot reach. The directed tests run on every commit, the random ones nightly across seeds, the aimed ones when the coverage report says so.

The messy parts are worth naming. Constraint files accumulate, and after a year nobody remembers which of the forty are legality and which are stale aiming; teams that stay sane require every constraint block to declare *legality* or *policy* in a comment. Then there is the temptation to fix a failing random test by constraining the failure away — that is how Section 9.3’s week-4 line gets written, and why a constraint addition deserves the same review as an RTL change.

PITFALLS

1. **Constraining the DUT's job into the stimulus.** Symptom: tests pass, a block of coverage cells never fills, and the offending constraint reads like a quotation from the specification. Cure: ask of every constraint whether it restricts the *input* the design must accept or the *output* it must produce; only the first belongs in the generator.
2. **"Invalid" used as a stimulus filter.** Symptom: the error-response rows of the plan have no owner. Cure: keep the legality constraints the interface truly enforces, mark the rest as policy, and give the error tests a build that disables them.
3. **Unchecked `randomize()`.** Symptom: a test passes suspiciously fast and the log shows the same transaction repeatedly. Cure: always check the return value; treat failure as fatal, not as a warning.
4. **A `dist` that does not cover the legal range.** Symptom: a bin outside the listed ranges never fills. Cure: values absent from a distribution list are excluded as if forbidden — leave a range open for the remainder.
5. **Shaping raw fields instead of measured quantities.** Symptom: weights added, report unmoved. Cure: write the distribution over the same expression the coverpoint samples.
6. **Seeds as the answer to every hole.** Symptom: the nightly seed count rises and coverage does not. Cure: after many runs, low coverage is a distribution or constraint problem; more samples of the same distribution change nothing.

SUMMARY

- Directed tests remain right for bring-up, mandated compliance sequences and smoke tests; they stop scaling in the low hundreds, and they only find the bugs you already imagined.
- Constrained random replaces enumeration with description: constraints are executable legality, and the same constraint can generate, monitor and assume.
- The solver guarantees uniformity over *legal combinations* — a promise about your constraint model, not your coverage model. Legality and distribution are different jobs; `dist` and `solve...before` do the second one safely, changing probabilities without changing the legal set, and `soft` lets a default be overridden rather than fought. What the weights actually produce is measured, not promised.
- Over-constraining is the quiet project-killer: satisfiable, silent, capable of deleting the exact feature under test. Read a persistently empty *block* of cells as a question about the generator before accepting it as a hard corner.
- A run is a seed *and* a source revision *and* a tool version; random stability survives only if new objects, threads and draws are appended, not inserted.
- Layering exists because constraints cannot cross generators: to relate two streams, randomize one object containing both.

- Portable stimulus reaches what layering cannot — scenario-space description and retargeting across simulation, emulation and silicon — at the cost of a modelling language, a bare-metal runtime, and solver behaviour of its own.

FURTHER READING

From the corpus: [\[4\]](#) on constraints, solvers and over-constraint diagnosis; [\[1\]](#) on the directed-versus-coverage-driven trade-off and generator architecture; [\[3\]](#) on atomic/scenario/multi-stream layering and seed discipline; [\[6\]](#) clause 18 for the normative randomization semantics; [\[5\]](#) on the language-design origins of constraint-oriented stimulus; [\[8\]](#) on why hand-written distribution directives do not scale; [\[13\]](#) for the portable-stimulus model, with [\[12\]](#) and [\[11\]](#) for industrial application and its limits; [\[2\]](#) for the 2024 state of automated directed-test generation.

Not in the corpus: the *SystemVerilog for Verification* texts cover randomization idioms at length, and Accellera publishes a PSS user guide alongside the standard.

REFERENCES

1. [B2](#) J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. [P10](#) A. Jayasena and P. Mishra, "Directed Test Generation for Hardware Validation: A Survey," ACM Computing Surveys, vol. 56, no. 5, Article 132, 2024.
3. [B1](#) J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
4. [B6](#) J. Yuan, C. Pixley, and A. Aziz, "Constraint-Based Verification," Springer, 2006.
5. [P18](#) Y. Hollander, M. Morley, and A. Noy, "The e Language: A Fresh Separation of Concerns," Proc. Technology of Object-Oriented Languages and Systems (TOOLS 38), Zurich, Switzerland, pp. 41–50, 2001.
6. [S8](#) IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
7. [R2](#) H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
8. [D10](#) M. Teplitsky, A. Metodi, and R. Azaria, "Coverage Driven Distribution of Constrained Random Stimuli," Proc. DVCon U.S., 2015.
9. [S18](#) Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
10. [S21](#) USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.
11. [D29](#) P. Gupta and M. Vax, "Test driving Portable Stimulus at AMD," Proc. DVCon U.S., 2019.
12. [D5](#) Accellera Portable Stimulus Working Group, "Portable Stimulus Standard Update: PSS in the Real World," Proc. DVCon U.S., 2022.
13. [S1](#) Accellera Systems Initiative, "Portable Test and Stimulus Standard, Version 3.0," Accellera, August 2024.

10. UVM as a Methodology

Two engineers on the reference SoC spend a month each building a block-level testbench. One verifies the crossbar — five managers, four subordinates, 32-bit address, 64-bit data, ID width 4. The other verifies the DMA engine, whose two channel backends drive two of those five manager ports. Both write a driver that turns transaction objects into AXI4 handshakes. Both write a monitor that turns handshakes back into transactions. Both write something that decides whether a response was legal. Neither can use a line of the other’s code. The transaction fields differ, one uses a mailbox where the other uses a queue, one prints `ERROR:` and the other prints `*E`, and the two testbenches end in different ways — one on a beat counter, one on a `$finish` buried inside a task.

Then integration arrives. The DMA engine’s channel-1 backend now drives crossbar manager port 3, and somebody must build an environment containing both. It can be done: roughly three weeks of rewriting. The interesting question is why it costs three weeks, when both engineers had already written a working AXI4 driver.

That question — not “how do I call `create`” — is what this chapter is about. UVM is the industry’s standardized answer to it (IEEE Std 1800.2-2020, *IEEE Standard for Universal Verification Methodology* ^[1]), and every structural decision in the library is an answer to some version of “why did that cost three weeks?”.

CHAPTER MAP

- §10.1 states the problem UVM was built to solve, and why the answer had to be a *standard*.
- §10.2 establishes why the taxonomy has the shape it has — agent, environment, test — and what each is allowed to know.
- §10.3 develops the factory and the configuration database, §10.4 phasing, §10.5 sequences and §10.6 the register layer: each buys reuse with indirection, and what that indirection costs in debug.
- §10.7 establishes why severity and message identifiers are load-bearing infrastructure: they decide what counts as a failure, and what regression triage can group by.
- §10.8 carries the block environment up into the system environment, and sets out what breaks in the move — dead stimulus, relocated configuration paths, reasigned objections and expired block-local expectations.
- §10.9 marks where the framework stops: the checking strategy, the coverage model, the debuggability and the runtime are the project’s, not UVM’s.
- §10.10 treats when a UVM environment is the wrong tool, and what to build instead.

10.1 The problem: reuse across projects, teams and companies

Verification has become a project that spans internal and external teams, and the discontinuity created by multiple incompatible methodologies among those teams limits productivity. That sentence is the whole motivation. The purpose of the standardization effort is stated just as plainly: improve interoperability, reduce the cost of repurchasing and rewriting intellectual property for each new project or tool, and make it easier to reuse verification components ^[1].

It helps to be precise about which reuse is meant, because there are three, and they get harder in order. *First-order* reuse is one verification environment reused across many testcases of one project. *Second-order* reuse is components of that environment reused in a system-level environment. *Third-order* reuse is those same components reused in a different environment, for a different design ^[2]. The opening scenario failed at second order, which is the one most block-level teams silently assume they will get for free.

For any of the three to work, a component must be not merely correct but *configurable* — able to exhibit the behaviour a block-level environment needs and, without modification, the behaviour a system-level environment needs ^[2]. Configurability is not a convenience bolted onto a testbench; it is the mechanism that makes reuse possible at all, and most of the UVM library exists to provide it.

The second thing UVM sells is vocabulary. When a colleague says “make the manager agent passive at the top level”, the sentence carries a structural consequence both of you can look up in a public document (the Accellera *Universal Verification Methodology (UVM) 1.2 User’s Guide*, 2015 ^[3]): no driver and no sequencer are instantiated, only the monitor. In-house methodologies have vocabulary too, but it does not survive a new hire, a purchased verification component, or an external partner. The user guide is candid that the architecture it recommends is one viewpoint and not the only possible structure ^[3] — what is standardized is the *library and its meanings*, not a mandatory shape.

The standardization story explains the library’s occasional inconsistency. UVM was the first verification methodology to be standardized, and the standard’s own introduction records the lineage: the Open Verification Methodology, itself a merger of three earlier methodologies; VMM; and transaction-level modelling derived from work done for SystemC. A library with that many lineages will have more than one way to do some things. The current standard is IEEE 1800.2-2020, a revision of the 2017 edition, defining a set of APIs and a base class library on top of IEEE 1800 SystemVerilog — written for implementors of that library, implementors of tools supporting it, and users ^[1]. It is a conformance document: it says what the library shall do, never what a good testbench looks like. Confusing the two is the root of most disappointment with UVM, and section 10.9 returns to it.

The adoption picture is one-sided. In the 2024 industry study UVM is the predominant standard for creating IC/ASIC testbenches ^[4] and the leading standard for FPGA testbenches, with the VHDL-oriented OSVVM and UVVM gaining traction since tracking began in 2018 and Python-based approaches newly measured ^[5]. For an

AXI4 interface on a RISC-V SoC the practical consequence is simple: a UVM agent is the artifact your suppliers, your partners and your next hire already know how to consume.

10.2 The component taxonomy and why it exists

Chapter 8 argued that the reusable unit of verification is *one interface*: the protocol is the boundary that survives integration, since AXI4 — *AMBA® AXI and ACE Protocol Specification*, Arm IHI 0022H.c, 2021 [6] — does not change when the crossbar is dropped into the SoC. UVM encodes that argument as a base class plus one flag. `uvm_agent` groups a sequencer, a driver and a monitor for one interface, and `get_is_active()` returns the mode the configuration database set for it, defaulting to active [1]. Active mode is where the agent emulates a device and drives design signals, and it requires a driver and a sequencer; passive mode disables their creation and leaves the monitor. The rule of thumb: one active agent per device that must be emulated, one passive agent per RTL device that must be watched [3]. That flag is what makes second-order reuse mechanical rather than a rewrite.

```
class axi_agent extends uvm_agent;
    `uvm_component_utils(axi_agent)

    axi_agent_cfg  cfg;
    axi_sequencer  seqr;
    axi_driver      drv;
    axi_monitor     mon;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    virtual function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        mon = axi_monitor::type_id::create("mon", this);
        if (get_is_active() == UVM_ACTIVE) begin
            seqr = axi_sequencer::type_id::create("seqr", this);
            drv = axi_driver::type_id::create("drv", this);
        end
    endfunction

    virtual function void connect_phase(uvm_phase phase);
        super.connect_phase(phase);
        if (get_is_active() == UVM_ACTIVE)
            drv.seq_item_port.connect(seqr.seq_item_export);
    endfunction
endclass
```

Three things in that fragment are load-bearing. Sub-components are built in `build_phase`, not in the constructor, because a virtual function can be overridden without restriction while a constructor cannot. They are created through `type_id::create` rather than `new`, which is what makes section 10.3's overrides possible. And the driver-to-sequencer link is made in `connect_phase`, after all building is finished [3].

The environment is the next layer: a hierarchical component that groups interrelated verification components — agents, scoreboards, or other environments. Composition, not new behaviour. Its own configuration fields are structural ones, of the same kind as `is_active`: how many manager agents, how many subordinate agents, whether a bus-level monitor exists [3]. Figure 10.1 applies that layering to the crossbar: what the test owns, what the environment owns, and which components actually reach the design.

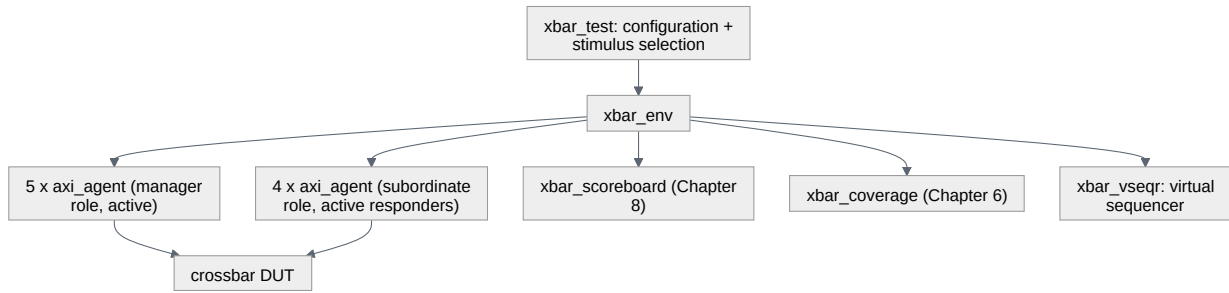


Figure 10.1 — The crossbar testbench as a UVM component hierarchy: the test carries configuration and stimulus selection, and the environment below it composes nine agents — five in the manager role, four responding — with the scoreboard, the coverage collector and the virtual sequencer. Only the agents reach the design; every layer above them is composition rather than new behaviour. Notice that the scoreboard and the coverage collector hang off the environment and not off the test, because structure a test instantiates for itself is structure the next reuse of the environment will not have.

Scenario. You must verify the crossbar: five manager ports (core-I, core-D, the two DMA channels, debug) against four subordinate ports (SRAM, boot ROM, APB bridge, neural accelerator). **Approach.** One `axi_agent` type, instantiated nine times — five configured to initiate, four to respond. **Why.** Nine instances of one class means the same driver and monitor are exercised nine ways in every regression, so a protocol bug in the agent is found by the crossbar’s own tests rather than by the DMA team three months later. It also means the ID-routing coverage model of Chapter 6 has exactly one place to sample from.

Above the environment sits the test, whose job is deliberately narrow: instantiate the top-level environment, configure it through factory overrides or the configuration database, and apply stimulus by invoking sequences. The recommended pattern is one base test that instantiates and wires the environment, with every real test extending it to configure differently or select different sequences [3]. A test that instantiates components of its own has taken structure out of the environment, and that structure will not be there when the environment is reused.

Note what is *not* in this taxonomy. The monitor broadcasts observed transactions on an analysis port and may process them itself or delegate to components connected to that port [3]. UVM supplies the pipe and takes no position on what hangs off it: the scoreboard on the far end is Chapter 8’s subject, the coverage collector on the same port implements the model of Chapter 6, and section 10.9 returns to why that boundary is the important one.

10.3 The factory and the configuration database

Everything in section 10.2 has an obvious hole. If the environment hard-codes `axi_driver`, then a test that needs a differently-behaved driver must edit the environment — and the environment is the artifact you were trying to reuse unmodified.

The verification community met this problem before UVM existed. An aspect-oriented verification language of the late 1990s — Verisity’s `e` — solved it in the language itself, letting independent groups add features to an existing environment by extending already-compiled types from separate files; its designers noted that the object-oriented alternative, the Abstract Factory pattern, was cumbersome because of the profusion of subclasses it required, and that the underlying difficulty is *influencing what class gets created* [7]. SystemVerilog has no aspect weaving, so UVM pays the cost those designers declined to pay: it builds the factory, and asks you to route every construction through it.

The mechanism is an override, by type or by instance. Components requested from the factory are resolved at construction time: the factory first searches for an instance override matching the object’s full instance name, then for a type-wide override, and failing both creates the requested type — instance overrides take precedence over type overrides [1]. Anything you swap in must be polymorphically compatible with what it replaces, including the same TLM interface handles [3]. And because resolution happens when the child is constructed, an override takes effect only if the parent registers it before building its children — which is why environment overrides belong in the test.

```
class xbar_stall_test extends xbar_base_test;
  `uvm_component_utils(xbar_stall_test)

  function new(string name, uvm_component parent);
    super.new(name, parent);
  endfunction

  virtual function void build_phase(uvm_phase phase);
    // Replace the driver of manager port 3 only; the other eight agents
    // keep the standard driver. Done before super.build_phase() builds env.
    set_inst_override_by_type("env.mgr_agent[3].drv",
                             axi_driver::get_type(),
                             axi_stall_driver::get_type());

    super.build_phase(phase);
  endfunction
endclass
```

That test provokes the crossbar’s canonical bug — write-data interleaving at a subordinate port under back-pressure — without touching one line of the environment, the agent or the driver it extends. A type override instead of an instance override would have made all nine agents stall; the choice between the two is the choice between “this scenario” and “this policy”.

Scenario. The USB controller is a dual-role part (USB 3.2 Revision 1.1 [8]): it must be verified as a device and as a host. **Approach.** One agent, one monitor, one se-

quence item; two drivers, selected by a type override in two base tests. **Why.** The pin-level protocol and the transaction abstraction are identical in both roles — only the initiative differs. Overriding the driver keeps the monitor, the coverage model and the register model shared between the two campaigns, which is exactly the third-order reuse of section 10.1 happening inside one project. The same trick supplies the flash controller’s error-injection hooks: an ECC-corrupting driver overrides the clean one in the data-integrity tests, and nothing downstream knows.

The configuration database is the factory’s twin. It lets an integrator configure an environment without knowing its implementation or its hook-up scheme ^[3], by setting typed values against hierarchical instance-path patterns:

```
uvm_config_db#(virtual axi_if)::set(this, "env.mgr_agent[3].*", "vif", mgr_vif);
uvm_config_db#(int)::set(this, "env", "num_mgr_agents", 5);
// The subordinate agents answer the crossbar's requests under testbench
// control, so by Chapter 8's criterion they are drivers, not observers.
uvm_config_db#(uvm_active_passive_enum)::set(this, "env.sub_agent*", "is_active", UVM_
ACTIVE);
```

All three are written from the test, where `this` is the top; a component deeper in the tree sets only relative to itself, which is why paths written here break the day the environment moves down a level — and it is there, at SoC level, that `is_active` becomes `UVM_PASSIVE` for all nine agents (section 10.8).

Two conventional parameters are worth adopting everywhere because tools and colleagues expect them: `checks_enable` and `coverage_enable` on monitors, which let an integrator silence a component’s checking or coverage without editing it ^[3] — useful the day the same interface is also checked by a bound assertion module (Chapter 11) and every violation is reported twice.

Now the bill. Both mechanisms replace a compile-time reference with a runtime lookup, so both defer their failures to runtime and report them somewhere other than where the mistake was made. A `set` whose path pattern has a typo silently does nothing; the component keeps its default and the failure surfaces a hundred microseconds later as a null virtual interface. An override registered after `super.build_phase` runs is simply ignored. An instance path that was correct before the environment moved one level down is now wrong, and no tool will tell you. This is the honest trade: indirection is what buys reuse, and indirection is what makes UVM harder to debug than the flat testbench it replaced. Treat the standard override- and configuration-audit facilities as first-class debug tools, and give the environment’s configuration surface one owner, the way a module’s port list has one.

10.4 Phasing: why a testbench needs a state machine

Every simulation performs the same overall sequence of steps to reach completion. The argument for formalizing those steps rather than inventing a synchronization scheme per environment is that formalization is what lets different block-level environments be combined into a system-level one, and lets different tests intervene at

the right moment without violating the sequence [2]. Phasing is second-order reuse made executable.

The standard splits phases into two families. The *common* phases are executed together by every component in the testbench, which is what makes them a synchronization mechanism; the *run-time* phases execute on a schedule that runs concurrently with the run phase. Every common phase before the run phase executes at simulation time zero [1].

Only three of them need to be understood before you can read any UVM testbench:

- `build_phase` is a **top-down** function phase [1]. Top-down because a parent decides what its children are — how many agents, active or passive — and must therefore run before they exist.
- `connect_phase` is a **bottom-up** function phase [1]. Bottom-up because wiring can only happen once both endpoints have been constructed, so the deepest, most local connections are made first.
- `run_phase` is a task phase, running in parallel with the run-time schedule from `pre_reset` through `post_shutdown`, and it starts a global timeout counter whose expiry is a fatal error [1] — the mechanism that turns a hung testbench into a failed test instead of a job that occupies a licence overnight.

Ending the test is a separate problem, and it is solved by objections rather than by a phase. A component or sequence raises a phase objection at the start of an activity that must complete before the phase may stop, and drops it at the end; a task-based phase persists only while an objection is raised, and it ends once every component has dropped its own [1]. The asymmetry matters: an active manager agent has an agenda — its reads and writes must finish — while a reactive subordinate agent has none, since it merely serves requests as they arrive, and should therefore not object at all [3].

Scenario. A crossbar test runs traffic from all five manager agents. The four subordinate agents respond. **Approach.** Each manager sequence objects while it runs; the subordinate agents never object; the environment sets a small drain time. **Why.** With no objections at all the run phase would end at time zero. With the subordinate agents objecting, the test would never end, because a reactive responder is never finished. And without the drain time, the last write response is still crossing the crossbar when the final manager objection drops, so the scoreboard never sees it — the “all objections dropped” condition is genuinely temporary. A drain time set once at the environment or test level, or a `phase_ready_to_end` implementation that re-raises while a transaction is in flight, closes that window — and the drain is also what makes vertical reuse survivable: a sub-system developer declares the drain that sub-system needs, and it still applies when that environment becomes one of six inside a larger one [3].

The recurring phasing mistake is reaching for a sibling’s handle inside `build_phase`: it reads a null, because bottom-up wiring has not happened yet, and the code belongs in `connect_phase` [3].

10.5 Sequences and the sequencer-driver handshake

A sequence is an object that contains behaviour for generating stimulus, and it is deliberately *not* part of the component hierarchy — the design decision worth understanding here. Components are structural and persist for the whole simulation; sequences are transient — created, used, and garbage collected when dereferenced — and one may exist for a single transaction or for the whole run. Making stimulus structural would have welded the scenario you want to run into the environment you want to reuse. Sequences compose instead: a parent invokes children, each is bound to a sequencer when it starts, and many may be bound to the same sequencer [3].

The handshake is a pull. The driver's `seq_item_port` connects to the sequencer's `seq_item_export`; the sequencer produces items, the driver consumes them and optionally returns responses, and the component containing both makes the connection [3]. Pull, not push, because only the driver knows when the interface is ready for the next item — its ability to synchronize to the bus and ask for stimulus at the right moment is the point of the arrangement.

Because the driver handles one item at a time while several sequences may be running, the sequencer maintains a queue of pending requests and chooses one each time the driver asks. The choice is the arbitration policy; the available modes are FIFO, weighted, random, strict FIFO, strict random, and a user-defined hook. The default is plain FIFO, which grants in arrival order; only the two strict modes grant the highest-priority request first, while the weighted and random modes grant randomly — the weighted one by weight [9]. If your test depends on a priority being honoured deterministically, choose a strict mode. Two further controls exist for scenarios that need exclusivity: `grab/ungrab`, which give one sequence exclusive use of the sequencer — the canonical interrupt-service pattern — and plain prioritization, which is less disruptive [3].

Coordination *across* interfaces is what virtual sequences are for. A virtual sequencer holds references to its sub-sequencers and executes sequences on them; it has no data item of its own and drives nothing directly. The payoff is that block-level sequence libraries, already written and already debugged, become the vocabulary of system-level scenarios [3].

```
class xbar_w_interleave_vseq extends uvm_sequence;
  `uvm_object_utils(xbar_w_interleave_vseq)
  `uvm_declare_p_sequencer(xbar_vseqr)

  function new(string name = "xbar_w_interleave_vseq");
    super.new(name);
  endfunction

  virtual task body();
    axi_wr_burst_seq wr_a, wr_b;
    axi_backpressure_seq bp;
    wr_a = axi_wr_burst_seq::type_id::create("wr_a");
    wr_b = axi_wr_burst_seq::type_id::create("wr_b");
    bp = axi_backpressure_seq::type_id::create("bp");
    fork
      wr_a.start(p_sequencer.mgr_seqr[1], this); // core-D -> SRAM, AWID 3
```

```

        wr_b.start(p_sequencer.mgr_seqr[2], this); // DMA ch0 -> SRAM, AWID 3 too
        bp.start(p_sequencer.sub_seqr[0], this); // SRAM port stalls WREADY
    join
    endtask
endclass

```

The second argument is not decoration. Passing `this` is what makes the three streams *children* of the virtual sequence rather than three unrelated root sequences: the parent's `pre_do`, `mid_do` and `post_do` hooks fire during their execution, and each inherits the parent's priority instead of the root default of 100 [1].

That is the whole reason the crossbar's canonical bug — write-data interleaving at a subordinate port under back-pressure (Arm IHI 0022H.c §A5.2.2 [6]) — needs a virtual sequence: the scenario is not “a manager does something”, it is “two managers and a subordinate do three things at once”. No single-interface sequence can state it. Two of those three carry the hazard: concurrent writes to one subordinate, and a stall on its write-data channel. The shared `AWID` carries something else — the crossbar widens IDs on the way out, so the two writes reach the subordinate under *different* IDs, which is exactly the case that breaks a scoreboard keyed on the subordinate side (Chapter 8) and exactly the case the interleaving check does not care about.

What a sequence *contains* — the constraints, the distributions, the layered scenario descriptions — belongs to Chapter 9. Here the point is only structural: UVM gives stimulus a lifetime, a binding and an arbitration policy, and stops there.

10.6 The register layer

Registers are where a verification environment stops being about protocol and starts being about the design. The register layer builds a high-level, object-oriented model of the memory-mapped registers and memories, abstracting reads and writes so that environments and tests migrate from block to system level without modification. The model — universally called the RAL, the register abstraction layer — is a hierarchy of blocks mirroring the design hierarchy; blocks contain registers, register files and memories, and registers contain fields [3]. Two ideas carry the rest.

Mirrored versus desired. The model maintains a *mirror* of what it believes each register currently holds. The mirror is not guaranteed correct, because the model's only information is the accesses it has seen: if the design changes a field on its own — setting a status bit, incrementing a counter — the mirror goes stale. After every read it is updated from the observed data; after every write it is predicted from the field's access policy, such as read-only or write-one-to-clear [3]. The *desired* value is a separate quantity: it equals the mirrored value until `set` changes it in the model alone, in zero time, and `update` then pushes it to the design, issuing bus cycles only for the registers that need updating [1] — so configuring a 40-register block costs the writes that matter rather than forty.

The most important sentence in the whole register layer is this one: a mirror is not a scoreboard — it can predict the content of registers the design does not update, but it cannot determine whether an updated value is correct [3]. Teams that forget it end

up with a register model that agrees with the design about a value both of them got wrong.

Front door and back door. A front-door access drives real bus cycles through the real path; a back-door access reaches the simulation constructs directly through a hierarchical path in the design. Back-door read and write *mimic* the front-door side effects — a back-door read sets clear-on-read bits to zero exactly as a physical read would, and a back-door write leaves read-only bits alone — whereas `peek` samples the register without modifying it and `poke` deposits a value as is, bypassing the behaviour entirely ^[1]. The redundancy is the point: when both directions share one path, a bug introduced on the way in is undone on the way out, so a reversed data bus is invisible to any write-then-read check that never leaves the front door. The cost is that identifying and maintaining those hierarchical paths is the main difficulty of back-door access ^[3] — and they break every time the design hierarchy is refactored.

Worked example: the peripherals' status register. The UART's receive-overflow flag is a one-bit, write-one-to-clear, volatile field: hardware sets it, software clears it by writing a one. Modelled honestly, it looks like this.

```
```systemverilog class uart_status_reg extends uvm_reg; rand uvm_reg_field oe; //
RX overrun error
```

```
function new(string name = "uart_status_reg"); super.new(name, 32, 0); endfunction
```

```
virtual function void build(); oe = uvm_reg_field::type_id::create("oe"); // parent
size lsb access volatile reset has_rst rand indiv oe.configure(this, 1, 3, "W1C", 1,
1'b0, 1, 0, 0); endfunction endclass ```
```

`build()` here is the generator's convention, not a UVM phase: the enclosing block calls it after creating the register, then calls the register's own `configure` to attach it to an address map, left out here to keep the fragment on the field.

Two of those arguments carry the real weight, and both are the ones people get wrong. `volatile` set to one declares that the mirrored value cannot be trusted, because the field can change in a way the model has no way to observe; the definition is borrowed from the IP packaging standard (IEEE Std 1685-2022, *IEEE Standard for IP-XACT*), where a volatile element gives no guarantee about what a read returns after a write or on the second of two consecutive reads <sup>[1, 10]</sup>. The access policy "W1C" means a written one clears the matching bit and a read has no effect. The first is a statement about the mirror: comparing a volatile field against the mirror without a fresh read is not a check, because nothing entitles the model to believe the value it last saw. The second is a statement about prediction: if the real design *also* clears on read, you have modelled the wrong register, the mirror diverges on every read, and the model rather than the design gets blamed. The `rand` qualifier on the declaration is the generator's habit, not a claim: `is_rand` is passed as zero, and for a predefined access policy outside the writable set — "W1C" is outside it — the library ignores `is_rand` and turns the field's randomization off in any case <sup>[9]</sup>. The two say the same thing, which is the part worth noticing: the passed argument records the intent, the access policy enforces it whatever the argument

said, and a reader who flipped only the argument would not have re-enabled anything.

The canonical UART bug shows what the pair buys. That bug — the overrun flag is never cleared if software reads the status register in the same cycle as a stop-bit error — is a disagreement between the two access paths: a front-door read followed by the clearing write leaves the model believing the flag is down, while a back-door peek shows it still up. Written as “read front door, peek back door, compare”, it becomes a two-line register test instead of a firmware lockup.

**Where the model comes from.** Given the number of registers in a real design and the number of details in configuring these classes, the specialization is normally done by a model generator working from a specification, which yields an up-to-date, correct-by-construction model — and generators are explicitly outside the library’s scope <sup>[3]</sup>. That gap is filled by machine-readable register description. A register description language (the Accellera *SystemRDL 2.0 Register Description Language*, 2018) provides a single source from which every view is generated, from specification through model generation and verification to maintenance and documentation; its stated motivation is the failure mode every team recognizes, the same information replicated in many places by many people with change propagation that is tedious and error-prone. Crucially it carries field behaviour directly — clear-on-read and write-one-to-clear are language properties, not comments <sup>[11]</sup> — which is what makes the `configure` call above correct by construction rather than by transcription. The IP packaging standard plays the complementary role: an XML schema for IP meta-data used in development, implementation and verification, in which each target interface carries a memory map of address blocks, registers and register files <sup>[10]</sup>.

**Scenario.** The Ethernet MAC’s time-sensitive networking queues expose several hundred per-queue gate-control and shaper registers, and the specification is still moving. **Approach.** Nobody writes that model by hand. One description generates the design’s decode logic, the driver’s C header and the register model; a field width changes in all three at once. **Why.** The failure this prevents is not a typo — it is the *silent* version skew where model, firmware header and design each encode a different revision of the map, and the mismatch is debugged three times by three teams.

Once the model exists, a library of predefined register test sequences comes free: a hardware-reset check, bit-bashing, access tests that play front door against back door, memory walking-ones, and a check of the declared hierarchical paths <sup>[1]</sup>. Start with the hardware-reset test — it debugs the model, the environment, the transactors and the design at once, at the lowest cost <sup>[3]</sup>.

Finally, choose a prediction mode deliberately. *Implicit* prediction attaches the model to a bus sequencer and updates the mirror after each operation the model itself performed — simple, and blind to any access it did not originate. *Explicit* prediction adds a predictor per interface, fed by the bus monitor, which looks up the register at every observed address and updates its mirror — more complex, but it captures all

register activity, including accesses the model did not originate [1]. *Passive* integration connects the monitor only: the model tracks and checks but cannot drive [3]. On the reference SoC the choice is forced: once firmware running on the core writes the peripherals through the crossbar and the APB bridge, an implicitly-predicted model is wrong within microseconds, because those writes never went through it.

## 10.7 Reporting as infrastructure

Messaging looks like the least interesting part of the library and carries the clearest reuse argument. The reporting classes issue reports with consistent formatting, let you configure the action taken and the file written based on severity, on message ID, or on the pair, and filter by verbosity; reports travel from the component through a report handler to a central report server. The default actions encode a policy: informational and warning messages are displayed, an error is displayed and *counted*, a fatal message is displayed and exits [1].

The counting is why this is infrastructure and not a printing convenience: a component can fail a test without knowing how pass and fail are decided — which is what lets messages from components reused from different sources be displayed and controlled consistently without modifying the reused component itself [2]. Drop the crossbar agent into the SoC environment beside a purchased memory-controller component, and one verbosity setting quiets both while one error from either fails the run.

The practical discipline is to treat message IDs as a schema, not as decoration. The regression triage of Chapter 12 groups failures by ID; an agent that reports every protocol violation under the ID `AXI4` has given the triage script exactly one bucket, while one that reports `AXI4_WSTRB`, `AXI4_ORDER` and `AXI4_4KB` has given it a first-pass classifier for free.

## 10.8 Reuse in practice: the same agent, one level up

Second-order reuse is the promise; here is the invoice. The crossbar environment of section 10.2 is now instantiated inside the SoC environment, with the real core, the real DMA engine and the real debug module (RISC-V Debug Specification v1.0 [12]) driving the manager ports and the real memories answering. Every one of the nine agents must therefore stop driving and only observe — the passive mode of section 10.2 [3]. That is one configuration field, and it works.

What breaks is everything that quietly assumed we were the one driving.

- **Stimulus becomes dead code.** The manager sequences have nothing to drive. Any checking that lived in the driver disappears with it — which is why checking belongs downstream of the monitor.
- **Configuration paths move.** `env.mgr_agent[3].*` is now `soc_env.xbar_env.mgr_agent[3].*`. Every absolute path written from the old top-

level test is wrong, and silently so. The cure is a rule: a component sets configuration only relative to itself.

- **Objections change owner.** The manager agents no longer object, so a SoC test that inherited its end-of-test from them ends at time zero; and the drain time that suited one crossbar is short when a response crosses two more layers — which is why the drain is declared per sub-system [3].
- **The virtual sequencer holds null handles.** `xbar_vseqr` points at sequencers a passive agent never built, so it must be guarded or not built at all.
- **Block-local expectations expire.** The scoreboard assumed every transaction on a manager port was one the environment generated. Now they come from firmware, and the predictor needs a different source of truth (Chapter 8). The register model has the same problem and the same fix: explicit prediction.

The agent survived because it was built around an interface, and the interface did not change. What did not survive is the code that assumed a *role*. A configurable component meets the needs of both a block-level and a system-level environment without modification [2]; every item above is a place where a team thought it had written one and had written the block-level half.

## 10.9 What UVM does not solve

---

UVM is a set of APIs defining a base class library for building modular, scalable and reusable verification components [1]. Read that as a boundary, not as modesty. Four things sit outside it, and all four decide whether your project succeeds.

**The checking strategy.** The monitor broadcasts; what listens and what it concludes is not the library's business — it observes only that different methodologies exist for the scoreboard and its reference model [3]. Nothing in UVM would have told you that write-data ordering under back-pressure was the property that mattered on the crossbar; that came from the protocol specification and a plan. The plan is a specification of both the verification process and the testbench infrastructure needed to support it [13] — the framework supplies the second half, and the second half is the easier one.

**The quality of the coverage model.** UVM decides where a coverage collector plugs in, not what it counts. A model with the wrong axes reaches 100% and means nothing (Chapter 6); the neural accelerator's canonical prefetcher bug escaped 100% code coverage entirely, and no methodology could have changed that.

**Debuggability.** Section 10.3's bill comes due here: indirection turns what would have been compile-time errors into runtime silence. A hand-written testbench of a thousand lines is easier to debug than a UVM environment of a thousand lines; it is the ten-thousand-line case where the comparison reverses.

**Performance.** Per-item object allocation, dynamic dispatch and a report server on the message path are not free. The cost is invisible on a block-level run of ten thousand cycles and highly visible in a nightly regression (Chapter 12) or at the boundary

with emulation, where transactor code must be synthesizable and a UVM component is not (Chapter 17).

The honest summary: UVM removes most of the *accidental* cost of building a testbench and none of the *essential* difficulty, which is deciding what to check and proving you checked it.

## 10.10 When not to use UVM

---

The library has a fixed cost — a sequence item, an agent, a configuration object, factory registration, a phasing discipline — that is paid before the first transaction moves. Three situations do not repay it.

**Small blocks with one narrow interface.** A timer on the APB subsystem, a two-flop synchronizer, a small FIFO: one interface, no second consumer, forty lines of directed stimulus. A module-based testbench with bound assertions (Chapter 11) and one covergroup is finished before the UVM version elaborates, and easier to read a year later.

**Formal-first blocks.** Arbiters, round-robin schedulers, handshake converters, and the crossbar's own ordering rules. When the block's specification is a set of properties and the state space is small enough to close (Chapter 14), a constrained-random generator is a strictly worse instrument than a proof: it samples where the proof concludes. Build the environment for what formal cannot reach — long traffic mixes, integration, firmware-driven scenarios — and let the proof own the protocol rules.

**Prototype and bring-up testbenches.** During bring-up the deliverable is a waveform tomorrow, not an asset for the next project. A throwaway testbench is not technical debt if you delete it; it becomes debt the day someone keeps it.

There is also a language question: VHDL-centred and Python-centred flows have their own methodologies, both gaining ground in the 2024 FPGA study <sup>[5]</sup>, and the methodology should follow the language the team actually verifies in.

The rule fits in one line: build a UVM environment when the interface will be verified more than once — a second test, a second level of hierarchy, or a second project. If none of the three is true, you are buying insurance against a risk you do not carry.

---

### IN PRACTICE

Real teams rarely write an agent for a standard protocol from scratch; they consume a verification component and spend their effort on configuration, sequences, the scoreboard and the coverage model — the parts nobody can sell them. Almost every group also grows a house layer between the library and its environments: a base test that reads command-line arguments, a standard configuration object, a logging convention. That layer is where local judgment lives, and it is the first thing to review on a new project, because it silently constrains everything built on top.

Register models are generated in continuous integration from the same description that produces the design's decode logic and the firmware header, on every commit — a model regenerated weekly is a model that is wrong for six days. And most reported “UVM bugs” are configuration-path bugs: a `set` that never matched, or an override registered too late.

---

## PITFALLS

1. **Sub-components built in the constructor.** Symptom: a factory override has no effect. Cure: build in `build_phase`, construct with `type_id::create`.
2. **Absolute configuration paths written from the top-level test.** Symptom: the environment works standalone and silently misconfigures after integration. Cure: every component sets configuration relative to itself.
3. **Sequence priorities with the default arbitration mode.** Symptom: the interrupt sequence waits behind mainline traffic. Cure: select a strict arbitration mode — the two strict modes are the only ones that grant the highest-priority request first <sup>[9]</sup>.
4. **Checking implemented inside the driver.** Symptom: all protocol checking disappears when the agent is made passive. Cure: check downstream of the monitor's analysis port.
5. **Implicit register prediction in an environment where firmware also writes registers.** Symptom: the mirror diverges quietly and the model is blamed for design bugs it did not see. Cure: explicit prediction with a predictor per bus interface <sup>[3]</sup>.
6. **Drain time increased until the test passes.** Symptom: a green regression that hides a scoreboard which stopped draining. Cure: re-raise the objection while a transaction is genuinely in flight <sup>[3]</sup>.

---

## SUMMARY

- UVM exists to make verification components reusable across testcases, across levels of hierarchy and across companies; its stated purpose is interoperability and the removal of the cost of rewriting verification IP for every new project <sup>[1]</sup>.
- The agent is the reusable unit because an *interface* is the stable boundary; the active/passive switch is what makes block-to-system reuse mechanical <sup>[3]</sup>.
- The factory and the configuration database buy reuse with indirection, and indirection is paid for in debug time — that trade is the chapter's central economic fact.
- Phasing is a synchronization contract that lets independently developed environments combine; objections, not phases, decide when a test ends <sup>[2, 3]</sup>.
- Sequences are deliberately outside the component hierarchy, so stimulus is not welded to structure; virtual sequences are how multi-interface scenarios get expressed <sup>[3]</sup>.



- A register mirror is not a scoreboard, and the model should be generated from a machine-readable register description (Accellera SystemRDL 2.0) rather than transcribed [3, 11].
- The framework does not choose your checks, your coverage model, or your risk priorities. It is not a verification plan.

---

## FURTHER READING

**From the corpus.** The standard [1] is the authority on phases, factory semantics, reporting and the register classes, and its introduction is the clearest short account of where UVM came from. The user guide [3] is the readable companion and the source of the recommended structure; the class reference [9] pins down access policies and arbitration modes. For the reasoning *behind* the structure, the methodology ancestor [2] remains the best text on the orders of reuse and on why messaging and simulation-step formalization are reuse mechanisms; the testbench-writing companion [13] frames the verification plan as the specification the framework serves. The lineage of extension-as-configuration is in the Verisity e-language paper [7]. For register generation, read the register description language [11] and the IP packaging standard [10] together — the first for behaviour, the second for the address map. Adoption data is in the 2024 trend reports [4, 5].

**Not in the corpus.** The community-maintained *UVM Cookbook* remains the most practical source of worked patterns, and the open-source design-verification style guides published by RISC-V hardware projects are a useful counterweight to vendor material.

---

## REFERENCES

1. **S9** IEEE, "IEEE Std 1800.2-2020, IEEE Standard for Universal Verification Methodology Language Reference Manual (Revision of IEEE Std 1800.2-2017)," IEEE, 2020.
2. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
3. **S11** Accellera Systems Initiative, "Universal Verification Methodology (UVM) 1.2 User's Guide," Accellera, October 2015.
4. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
5. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
6. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
7. **P18** Y. Hollander, M. Morley, and A. Noy, "The e Language: A Fresh Separation of Concerns," Proc. Technology of Object-Oriented Languages and Systems (TOOLS 38), Zurich, Switzerland, pp. 41–50, 2001.
8. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.
9. **S4** Accellera Systems Initiative, "Universal Verification Methodology (UVM) 1.2 Class Reference," Accellera, June 2014.

10. **S7** IEEE, "IEEE Std 1685-2022, IEEE Standard for IP-XACT, Standard Structure for Packaging, Integrating, and Reusing IP within Tool Flows (Revision of IEEE Std 1685-2014)," IEEE, 2022.
11. **S2** Accellera Systems Initiative, "SystemRDL 2.0 Register Description Language," Accellera, January 2018.
12. **S16** T. Newsome and P. Donahue, "The RISC-V Debug Specification, Version 1.0 (Ratified), Revised 2025-02-21," RISC-V International, February 2025.
13. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.

---

# 11. Assertion-Based Verification

The bug report opens with a scoreboard mismatch. On the reference SoC, seed 4471 of the nightly regression, a DMA channel copies 3 KB into the neural accelerator’s weight buffer and eleven bytes land at the wrong offset. Two engineers spend a day and a half on it, because the scoreboard raised its hand at the end of the transfer, some nine hundred cycles after the damage, and everything in between must be walked backwards through a waveform. The cause is four cycles wide: on the crossbar port the DMA asserted `AWVALID` with an address, the subordinate was slow to accept it, and an arbitration event made the DMA withdraw the request and re-offer it with a *different* address before `AWREADY` came back. The subordinate had already latched the first one.

Nothing about that failure was hard to detect. It was hard to *find*. The rule the DMA broke — once you offer an address, you keep offering it, unchanged, until it is taken — is one sentence of the bus specification, decidable from seven signals on one interface, in the cycle it is broken. What was missing was not a smarter test but a statement of that rule, written so a tool could evaluate it, sitting on the port where it applied.

That statement is an assertion. This chapter is about writing them: what they mean, where they live, which ones are worth having, and why the same text can be sampled by a simulator today and proved by a formal engine in Chapter 14 without changing a character.

---

## CHAPTER MAP

- §11.1 establishes what an assertion is — specification made executable — and why writing one forces a precision that reading a specification never does.
- §11.2 develops the semantics of SystemVerilog Assertions (SVA): sampled values, evaluation attempts, sequences, implication, clocking and `disable iff`, concurrent versus immediate.
- §11.3 shows why `assert`, `assume` and `cover` are three claims about one property text, and how each changes meaning between a simulator and a proof engine.
- §11.4 states where assertions belong — inline in the RTL or in a `bind`-ed checker — and who owns which kind.
- §11.5 works the crossbar’s AXI4 contract clause by clause, showing how much machinery each obligation costs and where a passing check still proves nothing.
- §11.6 turns to the invariants an implementation promised itself — absent from every specification, and richest where redundant state must agree with itself.
- §11.7 judges an assertion’s quality: vacuity, density, and the assertion that can never fail.

- §11.8 sets out what a verifier will misread when a proof engine reads the property text a simulator was reading: assumptions that stop being checked, covers that search rather than count, and liveness that becomes decidable.

## 11.1 An assertion is a specification you can run

An assertion is a positive statement about a property of a design: it states what must hold, and its falsehood indicates an error. The important word is *declarative*. An assertion does not describe how to check anything; it describes what to check and leaves the mechanism to the tool <sup>[1]</sup>. That is what separates it from a checker written by hand in procedural code, and why the same three lines are usable by a simulator, an emulator and a model checker.

The act of writing one is where the value starts. Take the sentence from the opening scenario, in the form the specification uses: *the payload must remain stable while VALID is asserted and READY is low*. Everyone nods at that sentence. Now write it.

```
// Wrong: compares this cycle's payload against the previous cycle's.
property p_aw_stable_wrong;
 @(posedge clk) disable iff (!rst_n)
 (awvalid && !awready) |-> $stable({awid, awaddr, awlen, awsize, awburst});
endproperty
```

`$stable` is true when a value is unchanged from the previous clock tick, so this property looks backwards: at the cycle where the stall is observed, it asks whether the payload was already the same one one tick ago. The obligation is forward, and it needs the non-overlapping implication:

```
// Right: the payload offered now must still be there at the next tick.
property p_aw_stable;
 @(posedge clk) disable iff (!rst_n)
 (awvalid && !awready) |=> awvalid && $stable({awid, awaddr, awlen, awsize, awburst})
;
endproperty
a_aw_stable : assert property (p_aw_stable);
```

The two differ by a cycle and by one added term, and the two edits do different jobs. The cycle shift repairs a known mistranslation: “remain stable” in protocol prose means “hold the same value into the next clock cycle”, and a bare `$stable` is a documented failure of automated specification-to-property translation. Nor does the wrong version fail quietly. A manager drives a fresh address in the same cycle it raises `AWVALID`; if `AWREADY` is low that cycle the antecedent is true, `$stable` compares against the idle cycle before and is false, and the property fires on perfectly legal traffic. What it cannot see is the withdrawal in the opening scenario: the moment `AWVALID` drops, its antecedent goes false and nothing is checked. That is the work of the added `awvalid` in the consequent — a second rule rather than a sharpening of the first: do not withdraw an offer before it is accepted. The documented fix for the stability rule is the cycle shift alone [2]; folding two obligations into one property is this book’s choice, and worth knowing you have made.

That is the discipline the chapter is about. A specification sentence can be vague and still feel complete; a property cannot. Writing one forces you to name the clock, the exact cycle relation, the condition under which the obligation starts, the reset behaviour and the polarity of every term — each a question you must answer from the specification or, when it is silent, from whoever owns it. Chapter 5 called an incomplete specification a defect to be surfaced rather than papered over; assertion writing is the most reliable machine for surfacing it, because an unanswered question shows up as a property you cannot finish typing.

Assertions are not free, though. Because the language is declarative, debugging one is harder than debugging RTL: there is no statement to step through, only a verdict. Expect to get complex properties wrong on the first attempt, review them as carefully as design code, and reuse a library rather than re-derive routine claims — “these two signals are mutually exclusive”, “a request is granted within N cycles”, “this state machine is never stuck” [1].

## 11.2 The anatomy of SVA

SVA is built in four layers, and the layering matters more than the operator catalogue.

**The Boolean layer** is ordinary SystemVerilog expressions — but over *sampled* values. A concurrent assertion is evaluated only at clock ticks, and the value it uses for a variable is that variable’s value in the Preponed region (IEEE Std 1800-2023, *IEEE Standard for SystemVerilog*): the value it had at the beginning of the step in which the clock edge occurred, before any of that edge’s updates [1, 3]. This is not pedantry. It means an assertion cannot race the RTL it watches, cannot see a glitch on a combinational path, and cannot be *raced* by the designer’s choice of blocking or non-blocking assignment — though that choice does change what the design does, and so what the assertion sees at the next tick. It also means an assertion reads a waveform the way you do — the value *before* the edge — which is why a property that looks off by one cycle in a debugger usually is not.

**The sequence layer** describes patterns over time, like a regular expression whose alphabet is Boolean expressions and whose step is a clock tick: `a ##1 b` is *b* one tick after *a*, `a ##[1:4] b` allows a window, `s [*3]` repeats. Sequences are why a temporal rule is three lines of SVA instead of the hand-written state machine protocol checking needed before the language existed.

**The property layer** turns sequences into claims, and the workhorse is implication. `antecedent |-> consequent` starts evaluating the consequent in the same cycle the antecedent completes; `|=>` starts it in the next. Two facts matter for Section 11.7: an implication whose antecedent is false succeeds *without checking anything*, and which side you make the antecedent changes what “this assertion passed” means.

**The statement layer** is the four directives — `assert`, `assume`, `cover`, `restrict` — covered next.

Two more pieces complete the picture. A concurrent assertion starts a fresh **evaluation attempt** at every tick of its leading clock, each running to its own success or failure, overlapping with the others <sup>[1]</sup>; a property with a five-cycle window has up to five in flight, which is why a rule broken under sustained traffic produces a burst of failures rather than one.

And `disable iff` is the reset guard: it discards any attempt in flight while its expression is true, ending it as *disabled* — neither pass nor failure. Two properties of it catch people out. It is asynchronous, monitored at every time step rather than only at clock ticks, which is what an asynchronous reset needs and which also makes it sensitive to zero-width glitches unless you sample it explicitly. And there may be at most one per assertion, immediately after the clocking event, guarding the whole property <sup>[1]</sup> — you cannot reset half a rule. Nearly every real property carries one, because a design in reset violates almost every rule it has, and a wall of reset-time failures is how assertion suites get switched off.

Finally, the distinction beginners trip on. **Immediate** assertions are procedural: they test a non-temporal expression when control flow reaches them, exactly like an `if`, and `x, z` or `0` counts as a failure. **Concurrent** assertions are the clocked, temporal ones <sup>[3]</sup>. If the claim is “this is true at this point in this code”, it is immediate; if it involves a clock or spans time, it is concurrent.

```
// Immediate: a sanity check at the point where the value is computed.
always_comb begin
 a_imm_cnt_sane : assert (cnt <= 5'd16)
 else $error("count %0d exceeds depth", cnt);
end

// Concurrent: a rule about the design's behaviour over clock ticks.
a_conc_no_overflow : assert property (@(posedge clk) disable iff (!rst_n)
 (cnt == 5'd16) |-> (!push || pop));
c_cnt_full : cover property (@(posedge clk) disable iff (!rst_n)
 cnt == 5'd16);
```

The immediate one is a statement about the code around it. The concurrent one is a statement about the design, and it will be checked on every test that ever runs,



whether or not anyone remembers it is there. Its rule is the one Section 11.6 writes from the other side as `a_no_overflow`; both reduce to *no push while full unless a pop retires in the same cycle*. The `cover` beside it applies Section 11.7’s discipline from the first example onwards: an overflow rule on a FIFO that never fills has never been tested.

### 11.3 Assert, assume, cover: three claims about one property

The same property text can be attached to a design in four ways — the three of the heading, which a simulator acts on, and a fourth that only a proof engine reads — and the choice states verification intent rather than syntax. `assert` makes the property an obligation on the design; `assume` makes it a constraint on the environment; `cover` monitors it for coverage; `restrict` constrains formal computation only, and simulators do not check it [1, 3]. Those are the intents; what each directive *does* depends on which engine reads it, and Table 11.1 sets the four against both — with `assume` the row where the two readings diverge.

**Table 11.1** — What the four property directives mean to each of the two engines that read them: the same property text attached as `assert`, `assume`, `cover` or `restrict`, with the simulator’s reading in the middle column and the proof engine’s on the right. The `assume` row is the one to read twice — in simulation it is checked like an assertion, in formal it is not checked at all and instead deletes traces from consideration, so an assumption written stronger than reality proves something about a design that does not exist.

Directive	In simulation	In formal verification
<code>assert</code>	fails if violated on this trace	proved, or a counterexample produced, under all stated assumptions
<code>assume</code>	checked like an assertion: a violation means the environment misbehaved	constrains the legal traces; its own correctness is <i>not</i> checked
<code>cover</code>	reports and counts each match on this trace	asks whether a matching trace exists, and returns a witness
<code>restrict</code>	ignored	constrains traces, like <code>assume</code>

In simulation `assert` and `assume` are effectively synonyms — both are checked, and the keyword only records whom you blame. In formal they are not remotely the same: an assumption is not checked, it deletes traces from consideration, and the assertions are proved only over what survives [1]. Write an assumption stronger than reality and you have proved something about a design that does not exist. That asymmetry is why the directive is reviewed alongside the property, and why Chapter 14 spends real effort on assumption hygiene.

The distinction pays off at integration. A module whose interface obligations are written down — these are my inputs’ rules, those my outputs’ guarantees — turns integration mistakes into assumption violations on the first system-level run, at the boundary where the mismatch is [1]. The mirror image is the whole trick: the rule that is an `assume` when you verify the DMA in isolation is an `assert` when you verify the crossbar that feeds it, and the same text serves both.

## 11.4 Where assertions live

---

Two questions look like one and are not: *where does the assertion text sit*, and *who wrote it*.

Assertions over internal signals require detailed knowledge of the design’s internal structure, so the designer is best placed to write them; assertions over external interfaces treat the block as a black box, derive from the interface specification rather than the implementation, and are normally written by verification engineers [4]. The split is not about seniority but about which document you are reading: the designer writes from the micro-architecture, the verifier from the specification.

The designer’s assertions belong *in* the RTL, inline, next to the logic they constrain — they are intimately tied to that code and must be maintained in step with it. Think of them as comments that can fail: an assertion carries the explanation a comment would, in both formal and natural language, and additionally reports when the explanation stops being true [4].

The verifier’s assertions must not touch the RTL at all. SystemVerilog’s `bind` construct exists for exactly this: it instantiates a module, interface, program or checker into a target scope — every instance of a module, or one named instance — without modifying the target’s source [3]. Interface assertions are attached that way, or placed inside the `interface` that carries the signals so they become part of its specification. Both consequences are organisational. Ownership becomes unambiguous: the RTL file is the designer’s, the bound checker is verification’s. And reuse becomes mechanical: a parameterised checker module is bound once to each of the crossbar’s nine ports rather than copy-pasted nine times, and can be compiled out of a gate-level run under a macro [4].

One hazard lives precisely at the inline end, and it is Chapter 2’s independence argument in miniature: an assertion written by the same person, in the same hour, from the same reading of the specification as the RTL encodes the same misunderstanding and never fires. Keeping the implementation from tainting the check is an explicit discipline, not an accident of style [4]. The division of labour above is the mitigation — inline assertions state the *implementation’s* invariants, which no specification mentions; bound assertions state the *specification’s* obligations, which no implementation should define.

## 11.5 Protocol checking: the interface as a contract

---

Protocol checking is where assertion-based verification earns its keep, for three reasons. The rules are written down by someone else, in a public specification, so you are not inventing the oracle. They are local — decidable from the signals of one interface — so they are cheap to evaluate and tractable to prove. And in practice their violations are among the most expensive to diagnose downstream, because a protocol error corrupts state that surfaces much later, somewhere else.

The useful division is between assertions on a *local* interface protocol, which are typically well suited to formal proof, and assertions on *end-to-end* behaviour relating one interface to another, which usually span too much of the design for a proof and are often easier to express in the testbench anyway. When a check needs extensive data structures or algorithms across a large part of the design, a monitor and a scoreboard are the right instrument, not a property (see Chapter 8) [4].

### Worked example: the contract on a crossbar port

The reference SoC's crossbar has five manager ports and four subordinate ports. Each port carries an AXI4 contract (AMBA® AXI and ACE Protocol Specification, Arm IHI 0022H.c, 2021 [5]), and three of its clauses need visibly different amounts of machinery. The three are Arm IHI 0022H.c §A3.2.1 (handshake process), §A5.2.2 (write data ordering) and §A5.1 (AXI transaction identifiers), the last of which requires a Subordinate to reflect a Manager's ID on the corresponding BID or RID response.

**Clause one — handshake stability.** This is `p_aw_stable` from Section 11.1: while a request is offered and not yet accepted, it stays offered and unchanged. It needs no memory; the rule relates two adjacent cycles and SVA expresses it directly.

**Clause two — one burst's write data at a time.** AXI4 deprecated write-data interleaving, which AXI3 had permitted (Arm IHI 0022H.c §A5.2.2 [5]): a manager must issue its write data in address order — with no way left to tag an interleaved beat, that means finishing the data beats of one burst before starting another's — and an interconnect that merges writes from several managers must forward their data in the order it forwarded their addresses. That second half is the crossbar's to obey, and Section 11.5 is where it matters. There is no ID on the write-data channel to check against — AXI3's `WID` is gone — so the obligation has to be reconstructed: each burst's beats counted and reconciled against the length promised by the address that opened it. That needs a counter and a queue.

**Clause three — a response only for work in flight.** A write response may be returned only for an ID with an outstanding address, and never more responses than addresses accepted. That needs per-ID bookkeeping.

Clauses two and three tip the property over into a *checker*: a small module carrying its own model of the interface, bound to the port. The escalation is normal, and it is the honest boundary of the technique — a property that quietly assumes a scoreboard behind it is not reusable, so the state goes in the file, in plain sight.

```
module xbar_port_checker #(
 parameter int unsigned IDW = 4, // AXI ID width on this port
 parameter int unsigned AW = 32 // address width
) (
 input logic clk, rst_n,
 input logic awvalid, awready, wvalid, wready, wlast, bvalid, bready,
 input logic [IDW-1:0] awid, bid,
 input logic [AW-1:0] awaddr,
 input logic [7:0] awlen,
 input logic [2:0] awsize,
```

```

 input logic [1:0] awburst
);
wire aw_hs = awvalid && awready;
wire w_hs = wvalid && wready;
wire b_hs = bvalid && bready;

// The checker's own model of the port.
logic [7:0] aw_len_q [$]; // AWLEN of each accepted address, in issue order
int unsigned id_out [1<<IDW]; // writes outstanding, per ID
logic [8:0] w_beats; // beats so far in the burst now on W (max 256)
logic len_err, id_err;

always @(posedge clk) begin
 len_err = 1'b0;
 id_err = 1'b0;
 if (!rst_n) begin
 aw_len_q.delete();
 foreach (id_out[i]) id_out[i] = 0;
 w_beats = '0;
 end else begin
 if (aw_hs) begin
 aw_len_q.push_back(awlen);
 id_out[awid]++;
 end
 if (w_hs) begin
 if (!wlast) w_beats++;
 else begin
 if (aw_len_q.size() == 0) len_err = 1'b1; // data with no address
 else begin
 if (w_beats != aw_len_q[0]) len_err = 1'b1; // wrong beat count
 void'(aw_len_q.pop_front());
 end
 w_beats = '0;
 end
 end
 if (b_hs) begin
 if (id_out[bid] == 0) id_err = 1'b1; // response nobody asked for
 else id_out[bid]--;
 end
 end
end

a_aw_stable : assert property (@(posedge clk) disable iff (!rst_n)
 (awvalid && !awready) | => awvalid && $stable({awid, awaddr, awlen, awsize,
awburst}));

a_w_burst_len : assert property (@(posedge clk) disable iff (!rst_n) !len_err);

a_b_id_outstanding : assert property (@(posedge clk) disable iff (!rst_n) !id_err);

// The covers that keep the assertions honest.
c_aw_stall : cover property (@(posedge clk) disable iff (!rst_n)
 awvalid && !awready);
c_w_overlap : cover property (@(posedge clk) disable iff (!rst_n)
 aw_hs && (aw_len_q.size() != 0));
c_same_id_pair : cover property (@(posedge clk) disable iff (!rst_n)
 aw_hs && (id_out[awid] != 0));
c_max_burst : cover property (@(posedge clk) disable iff (!rst_n)
 aw_hs && (awlen == 8'd255));
endmodule

```

Three notes on the model, because a reader will copy it. Every variable in it is written by this one block and read nowhere else except by the assertions, which sample in the Preponed region; that single ownership, rather than the choice of assignment operator, is what makes the blocking style safe here. Within a tick the statements run in the order written and the queue methods mutate immediately, so an address pushed at this edge is visible to the `pop_front` at the same edge — which is what makes a single-beat burst whose address and data arrive together check correctly. `w_beats` is deliberately one bit wider than `AWLEN`, so a manager that runs past its `WLAST` cannot wrap around onto a legal count. And because `len_err` and `id_err` are registered, an assertion samples them at the tick *after* the one that set them: the failure is reported one cycle behind the beat that caused it, with the transaction still on screen.

Two instantiation lines attach the checker to all nine ports of the crossbar:

```
bind xbar_manager_port xbar_port_checker #(.IDW(4)) u_chk (.*) ;
bind xbar_subordinate_port xbar_port_checker #(.IDW(7)) u_chk (.*) ;
```

The five manager-side instances check the five managers. The four subordinate-side ones check the crossbar itself, which plays the manager role as far as a subordinate is concerned — and that distinction carries more weight than it looks, as the coverage argument below shows. The design source is untouched, and that file is what Chapter 14 hands to a proof engine. The two `IDW` overrides are not decoration. An interconnect widens the ID it forwards, so a manager port carries the 4-bit ID the manager issued and a subordinate port carries a 7-bit one (Chapter 8 derives the three extra bits). The widening is the interconnect’s, not the checker’s: Arm IHI 0022H.c §A5.2.3 <sup>[5]</sup>, whose words are “The ID identifier at a Subordinate interface is wider than the ID identifier at a Manager interface.” `.*` connects a port only to a signal of the same name *and equivalent type*, so a checker left at its default width does not silently mis-connect on the subordinate side — it fails to elaborate. Which is the good outcome, and one you only see if you elaborate: lint alone does not resolve `bind`.

**Two ways to pass and prove nothing.** Now the trap, and it is why the four `cover` statements are in the file rather than on a wish list. They catch two different failures, and it is worth separating them.

The first is vacuity in the strict sense. `a_aw_stable` fires only when `awvalid && !awready` — an offer that had to wait. If the subordinate models in the testbench accept every address immediately, that antecedent is never true, every attempt succeeds trivially, and the assertion reports a perfect record on a rule it never evaluated. `c_aw_stall` is exactly that antecedent, promoted to a coverage question: an empty `c_aw_stall` says the stability check has not run.

The second is subtler because the assertion does run. Suppose the sequence library issues one outstanding write per manager at a time — a reasonable thing for an early bring-up sequence to do, and a thing nobody notices once it works. `a_w_burst_len` still evaluates on every run and would still catch a short or long burst; what it

never gets to see is the interleaving case it was written for, because on a manager port `aw_len_q` never holds two entries. Six weeks of nightly regression at 100% pass, and the specific hazard is untouched. `c_w_overlap` is the witness, and it reads zero. `c_max_burst` asks the same question of the beat-counting arithmetic: if nothing ever issues the 256-beat maximum, the counter's most interesting value was never reached.

Then read the covers against the binding point, because that is where this trap is usually sprung. `c_same_id_pair` fires when an address is accepted whose ID already has a write outstanding *on that port*. On a manager-side instance it therefore asks a question about one manager, and the natural-sounding premise cuts both ways: a sequence library that cycles a fresh ID per transaction leaves the cover empty, while one that pins a single ID per manager fills it on every second outstanding write. The tempting reading of the four subordinate-side instances is that the same cover now catches two managers colliding on an ID — and it cannot. An interconnect that merges traffic *widens* the ID it forwards, appending enough bits to identify the source port so that no manager need know which IDs the others use; Chapter 8 does the arithmetic, four bits in and seven out. Two managers presenting `AWID 3` reach the subordinate as two distinct IDs, so same-ID at a subordinate port still means *one manager*. What the cover witnesses there is the obligation the crossbar inherits along with the renumbering: writes a manager issued under one ID must be passed on in that order, and their responses returned in it. A real question — and not this bug's.

The bug's precondition is the other cover. The canonical crossbar bug of this book is a violation by neither manager: each finishes one burst's beats before starting the next, exactly as clause two requires. It appears on the subordinate side, where the crossbar is the one obeying or breaking AXI — its write arbiter releases a burst's grant when the subordinate stalls the first data beat, and beats of two managers' bursts arrive interleaved on a channel obliged to carry them in address order. On a subordinate-side instance `c_w_overlap` reads *a second write address accepted while an earlier burst is still unfinished on the write-data channel*, which is that precondition exactly, and `a_w_burst_len` is what turns the overlap into a failure — beats from two bursts counted as one reconcile with the length at the head of the queue only by coincidence. Notice what never enters that mechanism: an ID. `aw_len_q` is keyed by arrival order alone, which is why an ID-based reading of this hazard finds nothing to check. And it reads zero on these four instances too — the same sequence library, a different unmet condition. Filling it on a manager port takes one manager with two writes outstanding; filling it here takes two managers with writes in flight at one subordinate. A library grown out of the bring-up ladder — one manager against one subordinate, then the next pair — arranges neither.

A cover on the antecedent turns that silence into a number. When it reads zero the diagnosis forks two ways, and both are actionable: either no test drives the design into the situation, so the stimulus needs work, or the design can never reach it at all, in which case either the property is misformulated or something in the design is preventing a state you expected <sup>[1]</sup>. Section 11.7 generalises this into a rule.



**Scenario (elsewhere in the cast).** The shape recurs wherever a public specification defines an interface. A 10G Ethernet MAC with time-sensitive-networking queues has a timing obligation rather than an ordering one: the reference design budgets a bounded number of cycles between the start-of-frame delimiter and the frame’s capture timestamp, and if one hop overruns it, that hop silently consumes the network’s synchronisation budget. **Approach.** One assertion, and the cover that says whether a frame delimiter was ever seen at all:

```
systemverilog a_ts_latency : assert property (@(posedge clk) disable iff (!rst_n)
$rose(sfd_seen) |-> ##[1:2] ts_captured); c_sfd : cover property (@(posedge clk)
disable iff (!rst_n) $rose(sfd_seen));
```

**Why.** `$rose` is true at a tick where the sampled value went from 0 to 1, so the obligation starts once per frame, and the window comes from the standard rather than from the implementation. That is the test of a good protocol assertion: if you had to read the RTL to write it, it is not checking the protocol.

## 11.6 White-box assertions and internal invariants

Interface assertions check what the outside world is entitled to expect. Internal assertions check what the *implementation* promised itself — the invariants that hold because of how the block was built, that no specification mentions, and that the designer knows and would otherwise write in a comment. The standard catalogue is short: state-machine transitions, memory access correctness, queue overflow and underflow, arithmetic overflow, and signal stability — though the complete list depends on the methodology in use <sup>[1]</sup>.

Take the DMA engine’s backend, whose datapath FIFO is 16 entries of 64 bits. Four invariants cover most of what can go structurally wrong with it, and none of them is derivable from the DMA’s specification:

```
module dma_backend_invariants #(parameter int unsigned DEPTH = 16) (
 input logic clk, rst_n,
 input logic push, pop,
 // free-running binary pointers on a power-of-two depth: an index plus a
 // wrap bit, so their difference tells full from empty
 input logic [$clog2(DEPTH+1)-1:0] level,
 input logic [$clog2(DEPTH):0] wr_ptr, rd_ptr,
 input logic [3:0] state // one-hot: IDLE, ADDR, DATA, RESP
);
a_no_overflow : assert property (@(posedge clk) disable iff (!rst_n)
 push |-> (level < DEPTH) || pop);
a_no_underflow : assert property (@(posedge clk) disable iff (!rst_n)
 pop |-> (level != 0));
a_level_tracks : assert property (@(posedge clk) disable iff (!rst_n)
 level == (wr_ptr - rd_ptr));
a_state_onehot : assert property (@(posedge clk) disable iff (!rst_n)
 $onehot(state));

c_full : cover property (@(posedge clk) disable iff (!rst_n)
 level == DEPTH);
c_full_bypass : cover property (@(posedge clk) disable iff (!rst_n)
```

```

 (level == DEPTH) && push && pop);
endmodule

```

Read `a_level_tracks` twice: it is the most valuable of the four and the least obvious. The other three restate rules a careful designer already coded — `$onehot` is true when exactly one bit of its argument is set — while this one asserts *redundancy*: two independently maintained representations of the same fact, the occupancy counter and the pointer difference, must agree. In practice, redundant-state invariants are where many of the good white-box bugs are found, because a corruption that affects one representation and not the other is exactly the class that produces plausible-looking data with wrong contents, six hundred cycles from its cause. It is also the one to copy carefully: the equality holds because both operands are binary free-running pointers of the width declared above, so their difference wraps modulo the pointer range; a gray-coded CDC FIFO needs the synchronised binary shadow instead.

The two `cover` statements exist for the Section 11.5 reason — the antecedents `push` and `pop` cover themselves on any run that moves data, so the question worth asking is whether the FIFO ever reaches full. `c_full_bypass` is the sharper of the two: it witnesses precisely the escape hatch `a_no_overflow`'s `|| pop` opens, a push accepted at full because a pop retires in the same cycle. If it reads zero, that clause has never been exercised — and a rule with an untested exemption is a rule you do not yet know you have. The asymmetry with `a_no_underflow` is deliberate: this FIFO forwards a pop that arrives alongside a push at full, but does not forward a push to a pop at empty.

The internal-assertion payoff is observability, which is why Chapter 3 calls assertions the sharpest partial oracle. A bug is caught adjacent to the code that produced it, so the failure message points at the guilty lines instead of a downstream symptom <sup>[1]</sup> — the difference between the opening scenario's day and a half and a single annotated cycle.

### The property Chapter 14 will prove

One rule on the DMA's backend belongs to both worlds: its origin is the bus specification, so it is an interface obligation, while its violation comes entirely from the midend's splitting arithmetic. Chapter 2 used it to show what an adversarial reading of a specification produces, Chapter 3 as a worked case of what a partial oracle does and does not establish:

```

property p_no_4kb_crossing;
 @(posedge clk) disable iff (!rst_n)
 (awvalid && awready && (awburst == 2'b01)) |->
 (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);

```

In simulation this property is *sampled*: evaluated on the bursts the tests happened to generate, silent about the ones they did not. Chapter 14 hands the identical text to a

proof engine and gets a different kind of answer — that no legal input sequence whatsoever can produce a crossing burst, or a counterexample that does. Nothing in the property changes; the question asked of it does, which is Section 11.8’s subject. Section 11.7’s rule applies here with a twist: this is an implication, but its antecedent — an accepted INCR write address — is ordinary traffic. What needs covering is the boundary relation, which is why Chapter 5’s plan pairs the row with a coverage model rather than with a single `cover` line.

**Elsewhere in the cast.** Internal invariants are also how security properties get stated, because a security rule is usually an invariant about internal state rather than a behaviour at an interface. In the crypto engine, two assertions say more about key handling than a page of prose:

```
a_key_quiet : assert property (@(posedge clk) disable iff (!rst_n)
 key_load |-> engine_idle);
a_no_ct_unauth : assert property (@(posedge clk) disable iff (!rst_n)
 ct_valid |-> key_authenticated);

c_key_load : cover property (@(posedge clk) disable iff (!rst_n) key_load);
c_ct_valid : cover property (@(posedge clk) disable iff (!rst_n) ct_valid);
```

The first forbids a key change under an in-flight operation, which is how key material bleeds between contexts. The second is weaker than it sounds, and reading it precisely is the exercise: it forbids ciphertext being *presented* while the key is unauthenticated, and says nothing about the key the ciphertext was computed under. If authentication completes between the computation and the rise of `ct_valid`, the assertion passes. Provenance needs a tag travelling with the key material — a different invariant, over different state. Both of these are single-cycle claims over a small cone of logic, so a proof engine settles them in seconds when they hold; neither is checkable from the engine’s data interface; and both carry a cover on their antecedent, because an assertion about key loading proves nothing on a run that never loaded a key.

## 11.7 Assertion quality

An assertion suite is not measured by size. It is measured by how much of it could ever have failed, and there are three ways for an assertion to be worthless.

**It never triggers.** A property of the form `a |-> b` succeeds whenever `a` is false, and that success establishes nothing: the standard calls it vacuous and says such an evaluation is interpreted as not matching the user’s intent — neither a pass nor a failure (IEEE 1800-2023 §16.14.8) [3]. A run in which an assertion has only vacuous successes tells you one of two things, and that fork is the most useful diagnostic in this chapter: either the tests never drive the design into the situation the antecedent describes, or the design can never reach it at all — in which case the property is misformulated, or something in the design is preventing a state you expected [1]. Hence the rule: **every implication ships with a cover on its antecedent**, written at the same time and reviewed in the same review. Where the antecedent is ordinary traffic

— `push`, on a FIFO that pushes all day — a bare cover on it proves nothing; discharge the rule instead with a witness that *implies* the antecedent and isolates the interesting case, as `c_full_bypass` does above, or with the coverage model Chapter 5’s plan pairs with `p_no_4kb_crossing`. Chapter 6 reports those covers as assertion coverage; here they are a writing discipline, and the discipline comes first.

Vacuity also depends on how you phrased the rule. `ok |-> !err` and `err |-> !ok` are logically equivalent and pass vacuously under opposite conditions — the first when `ok` is false, the second when `err` is false <sup>[1]</sup>. Which term you make the antecedent therefore decides what a passing run means. Make it the rare, deliberately constructed condition; then a non-vacuous pass is evidence and an empty cover is a to-do item.

**It restates the design.** An assertion derived from the implementation agrees with the implementation, including where the implementation is wrong; keeping the code from tainting the check is an explicit obligation on whoever writes an inline assertion <sup>[4]</sup>. The review symptom is easy to spot: the property paraphrases the `always` block six lines above it, same signals, same order. The cure is to write from the specification, and from the *prohibition* rather than the mechanism — Chapter 2’s move.

**It cannot distinguish anything.** For every assertion in a review, ask the mutation question: name one change to the RTL that this assertion catches and code coverage does not. An assertion with no answer is not a check, it is a comment with a runtime cost. This is also the honest reading of assertion-count metrics — counting signals touched, or assertions relative to code size, is a legitimate structural indicator of effort <sup>[1]</sup>, but an indicator, not a goal. Two hundred properties restating a register decode and none on the ordering rules is a worse suite than twenty placed where the risk analysis said the design was novel.

## 11.8 The same text, two engines

One of the stated goals of SVA is a common semantic meaning, so that a single property can drive different classes of design and verification tool <sup>[3]</sup>. That is why the checker in Section 11.5 is a deliverable rather than a simulation artifact. But three things change when a proof engine reads what a simulator was reading, and a verifier who does not know them will misread the results.

**Assumptions stop being checks.** Section 11.3’s asymmetry — checked in simulation, never checked in formal, where it deletes traces instead — is the first of the three. The work moves with it: simulation’s setup cost is the environment, formal’s is stating the assumptions, and a missing one produces spurious failures rather than a silent gap <sup>[1]</sup>.

**Covers stop counting and start searching.** A `cover` in simulation reports how many times the scenario occurred on this trace; in formal it asks whether the scenario is reachable at all and returns a witness if it is <sup>[1]</sup>. An empty cover therefore means two very different things — “no test produced it” versus “no legal execution can produce it” — and only the second is a statement about the design.

**Liveness becomes decidable.** `req |-> ##[1:$] gnt` — a request is eventually granted — cannot fail in simulation: the evaluation attempt stays open until the run ends, and many open attempts cost simulation performance, which is why open-ended intervals should be bounded by other operators when the property is meant for simulation [4]. A proof engine decides the same text. That is the cleanest case of a rule you can *write* in one language and only *answer* in one engine, and the reason a bounded restatement (`##[1:16]`) is the usual simulation-side compromise: weaker, but falsifiable tonight.

The coherent hub gives the shape in one line. The invariant that at most one cache may hold a line in the Modified state —

```
a_single_writer : assert property @(posedge clk) disable iff (!rst_n)
 $onehot0(modified_vec);
```

— is sampled by every system-level simulation the hub ever runs and closed for good by a proof over the abstracted protocol in Chapter 14, on the hub’s canonical abstraction of two caches and one address: `modified_vec` is the per-line vector of Modified holders, and `$onehot0` is true when at most one of its bits is set. Same text, two answers, and only the second is a guarantee.

A word on generated properties, since the tooling now exists. A 2024 framework producing SVA from a full natural-language specification reported 89% of 56 generated properties correct both syntactically and functionally, on a single serial-bus design — where *correct* means the property passed a formal proof against a known-good RTL, after human engineers had removed the generations that referenced signals the tool could not map [6]. That definition deserves a second reading beside Section 11.7: a property whose antecedent the design can never satisfy also passes a proof on bug-free RTL, and would be scored correct. Encouraging, then, and not yet a trend — and measured by the criterion this chapter has just finished arguing is not evidence. The failure mode to expect is the one Section 11.1 opened with: protocol documents are written for experienced engineers and are ambiguous by machine standards, so an automated translation reaches for the intuitive operator and lands on the wrong cycle [2]. Set beside that, the corpus holds an earlier and narrower result whose designs a reader can download and re-run: on the taped-out open-source Ariane (CVA6) core and the OpenPiton manycore framework, 236 liveness and safety properties generated from 110 lines of interface annotation were run across seven forward-progress-critical modules in those two hierarchies. They proved out on the page-table walker and the translation-lookaside buffer after roughly 30 minutes of human effort spent defining the correct transactions, and the same generated set found real bugs in the memory-management unit and in an OpenPiton network-on-chip buffer, each of which then proved in full [7]. Note what changed with the narrowing: annotations describing a module’s transactions carry far less ambiguity than a protocol document written in prose, which is why the properties they yield are checkable at all. Treat generation as a drafting aid whose output enters the same review as a hand-written property.

## IN PRACTICE

Assertion suites grow in two streams that meet at integration. Inline assertions accumulate during RTL development, in the same commits as the logic, and pay off first in the designer's own unit runs. Protocol checkers are written from the interface specification during planning — Chapter 5's plan fixed the binding point for `p_no_4kb_crossing`, the backend's AXI write-address channel, before anyone had written a line of it — and bound at bring-up, where they do most of their bug-finding in the first two weeks.

The messy parts are predictable. Somebody switches a noisy assertion off with a `plusarg` and nobody switches it back on, so failure counts must be audited against the enabled list rather than the file. Assertions in gate-level or emulation runs have to be compiled out or restructured, which is why the macro guards exist. And a bound checker with its own state is software: it has bugs, it needs a review, and the first week of a new checker usually finds more defects in the checker than in the design. That is normal, and not an argument against the checker.

## PITFALLS

1. **No `disable iff`.** Symptom: a wall of failures in the first twenty cycles of every test, and a suite everyone has learned to ignore. Cure: guard every concurrent property with the reset condition, once, immediately after the clocking event.
2. **The assertion that was never in a position to fail.** Symptom: 100% pass rate beside an empty antecedent cover. Cure: write the cover with the property, and treat an empty one as a stimulus bug until proven otherwise.
3. **An implication pointing the wrong way in time.** Symptom: a property that fires on legal traffic, or one that is silent on the very scenario it was written for — two faces of the same mistake. Cure: check the cycle relation explicitly — `|->` is the same tick, `|=>` the next — and confirm on a waveform which direction in time `$stable` is looking.
4. **The property that paraphrases the RTL.** Symptom: assertion and logic use the same signals in the same order, written in the same hour. Cure: derive from the specification, and state the prohibition rather than the mechanism.
5. **Unbounded eventuality in simulation.** Symptom: a liveness property that has never failed and never will, plus a slow regression from thousands of open attempts. Cure: bound the window for simulation, send the unbounded form to formal.
6. **Assertion count as a goal.** Symptom: a metric rising steadily while the bug curve does not move. Cure: measure where the assertions are against where the plan said the risk was.



## SUMMARY

- An assertion is specification made executable: declarative, tool-evaluated, and precise in ways prose cannot be [1]. Writing one is the cheapest way to discover that a specification sentence is ambiguous.
- Concurrent assertions are clocked and use sampled values from before the edge, so they cannot race the design; `disable iff` discards in-flight attempts asynchronously and belongs on essentially every property [1, 3].
- `assert`, `assume` and `cover` are three intents over one property text. In simulation the first two are the same check; in formal an assumption is never checked and instead deletes traces [1].
- Interface assertions come from the specification and attach with `bind`, keeping verification code out of the design; internal invariants come from the micro-architecture and belong inline [3, 4].
- In practice the white-box assertions that pay best check redundancy — two representations of one fact agreeing — because that is where silent corruption hides.
- An assertion that never triggers proves nothing; the cover on its antecedent is what turns a passing run into evidence [1, 3].
- The same text is sampled by a simulator and proved by an engine, but assumptions, covers and liveness all change meaning between them (Chapters 14 and 16).

## FURTHER READING

Within the corpus, [1] is the reference treatment of SVA semantics: assertion kinds and evaluation attempts, sampled values and reset, assumptions and coverage as first-class directives, checkers and checker binding, and the fullest available discussion of vacuity. IEEE 1800-2023 [3] is definitive on the four directives, on sampling, on non-vacuous evaluation and on `bind`. For methodology rather than language — who writes what, inline versus bound, reusable checker packaging, coding rules aimed at simulation performance — the assertion chapter of [4] is the most practical corpus source, and [8, 9] place assertions among the other oracles. On automated property generation, [6] reports the state of specification-to-SVA generation as of 2024 and [2] documents why protocol documents in particular defeat it; [7] is the corpus's account of generated liveness and safety properties run against taped-out open-source silicon.

Outside the corpus: H. Foster, A. Krolnik and D. Lacey, *Assertion-Based Design*, is the standard book-length treatment of assertion methodology; the Accellera Open Verification Library remains a readable model of how a reusable checker library is packaged and parameterised.

---

## REFERENCES

1. **B4** E. Cerny, S. Dudani, J. Havlicek, and D. Korchemny, "SVA: The Power of Assertions in SystemVerilog," 2nd ed., Springer, 2015.
2. **P12** Y.-A. Shih, A. Lin, A. Gupta, and S. Malik, "FLAG: Formal and LLM-assisted SVA Generation for Formal Specifications of On-Chip Communication Protocols," arXiv preprint arXiv:2504.17226v1, April 2025.
3. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
4. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
5. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
6. **P11** W. Fang, M. Li, M. Li, Z. Yan, S. Liu, H. Zhang, and Z. Xie, "AssertLLM: Generating and Evaluating Hardware Verification Assertions from Design Specifications via Multi-LLMs," arXiv preprint arXiv:2402.00386v4, February 2026.
7. **P15** M. Orenes-Vera, A. Manocha, D. Wentzlaff, and M. Martonosi, "AutoSVA: Democratizing Formal Verification of RTL Module Interactions," Proc. Design Automation Conference (DAC '21), 2021.
8. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
9. **B6** J. Yuan, C. Pixley, and A. Aziz, "Constraint-Based Verification," Springer, 2006.

---

## 12. Regression Engineering

Month nine of the reference SoC: a nightly regression of 900 runs reports 863 passed and 37 failed. Three engineers spend the morning on those 37, and their triage arithmetic accounts for all of them. Twenty-nine failures are one bug: a scoreboard change merged the previous afternoon mispredicts response ordering on the boot-ROM port — the same misprediction bring-up filed in week three (see Chapter 4), reintroduced by a refactor and caught this time by the suite rather than by an engineer watching waveforms. Four are a real design bug in the neural accelerator’s weight prefetcher. Two are a test that has failed about once a week for two months and that everybody now calls “the flaky one”. One hit the farm’s disk quota. And one — the one that matters — is a genuine DMA legalization failure closed as a duplicate of the flaky one, because the log looks similar and the triage window is closing.

Nothing in that morning was a *verification technique* problem. The scoreboard was right, the stimulus was right, the coverage model was right. What failed was the **factory**: the machine that turns commits into evidence. One fact was reported twenty-nine times; a real find was destroyed by a signature collision with a defect the team had learned to ignore; and the loop consumed most of a working day of three engineers. Chapter 7 taught you to read the numbers a regression produces. This chapter builds the thing that produces them.

---

### CHAPTER MAP

- §12.1 tiers a regression suite and states what earns admission to each tier.
- §12.2 runs the (test, seed) matrix so that a failure is reproducible by construction, not by luck.
- §12.3 establishes how per-commit gating changes a hardware project, and what it costs.
- §12.4 triages hundreds of failures a night without losing the one that matters; §12.5 establishes why a flaky test is more expensive than a failing one.
- §12.6 fits a growing suite into a fixed night, and states what is given up in doing so.

---

### 12.1 What a regression is, and what earns a place in it

A **regression suite** is the standing proof that today’s design still does what yesterday’s did: a fix for today’s bug routinely breaks yesterday’s verified functionality, and re-running the previously verified functionality is what catches it <sup>[1]</sup>. Everything else follows from one consequence — the suite is *cumulative*, so it grows monotonically unless somebody makes it stop.

Chapter 4 established the cadence and Chapter 7 how to read the resulting metrics. The engineering question is more concrete: **given a fixed nightly window, what earns a place in it?** A tier is not a schedule; it is an admission policy with a runtime budget attached. Table 12.1 sets out four such policies, each with its trigger, its budget, and the rule that admits a test to it.

**Table 12.1** — The four tiers of a regression suite: what launches each one, the wall-clock budget it has to fit inside, and the rule a test must satisfy to be admitted to it. The last column is the load-bearing one, because a tier is an admission policy with a runtime budget attached rather than a schedule — and the campaign row is the tier teams keep reinventing without a name, a purpose-built run owned by someone and answerable to no clock. §12.6 is what happens when the nightly budget stops being enough.

Tier	Trigger	Wall-clock budget	Admission rule
<b>Smoke</b>	every commit / merge request	minutes	proves the build is alive; fixed seeds; bounded runtime; zero known failures
<b>Nightly</b>	scheduled, daily	one night	discriminates: fails when the design or environment regresses
<b>Weekend</b>	scheduled, weekly	one weekend	everything long, deep-seeded, or configuration-crossed
<b>Campaign</b>	on demand	no wall-clock budget; bounded by its written purpose	purpose-built runs that belong on no clock

A published industrial hardware CI setup groups its jobs along almost exactly these lines, down to the extended-duration *stability* configuration pinned to a specific commit hash [2]. The names differ between companies; the shape does not.

Three admission rules do most of the work. **Smoke earns its place by speed and silence:** fixed seeds (a smoke tier that randomizes is one that sometimes lies), a hard per-test runtime ceiling, and zero tolerance for a known-failing member — a gate with a permanent red light in it is decoration. **Nightly earns its place by discrimination:** a test belongs there if a plausible regression in design or environment would make it fail, which is the criterion that splits directed tests into a basic-functionality group running every night and a remainder held for the weekend [1]. **Weekend earns its place by length and breadth** — deep seed counts, exhaustive rather than sampled configuration crosses, the expensive engines. Gate-level simulation is the standard case: running the full RTL suite at gate level is prohibitive in labor or schedule terms, so only a minority is regressed there — which almost guarantees some X-optimism issues are missed [3], making *which* minority a risk decision rather than an afterthought (see Chapter 19).

**Scenario.** An HBM controller’s PHY training state machine is the block’s main source of late bugs, and one training-and-retraining soak — cold boot, thermal excursion, retrain, refresh collision during retrain — runs for four hours across six configurations. **Approach.** All six go to the weekend tier; the nightly keeps a twelve-minute *training smoke* that reaches initial calibration and stops. **Why.** The nightly tier’s budget is the scarce resource and its job is discrimination, not depth: if calibration itself broke on Tuesday you want to know on Tuesday, while

everything past calibration can wait for Sunday without changing what anyone does on Wednesday.

The fourth tier deserves a name because teams keep reinventing it without one. A **campaign** is a purpose-built run on no clock: launched for a reason, owned by someone, producing a written result, then stopping. A flash controller’s error-injection campaign is the archetype — bit-error patterns and burst lengths swept against the LDPC pipeline across simulated wear states, checked by the data-integrity oracle, ten thousand runs over a weekend, once per RTL milestone. In the nightly tier it would eat the night forever; outside the tier structure it happens when somebody remembers.

## 12.2 The (test, seed) matrix

A constrained-random test is not one experiment. It is a family of experiments indexed by seed, and what your regression actually runs is a **(test, seed) matrix**: rows are tests, columns are seeds, each cell a run with its own outcome. Almost every pathology in this chapter is a property of that matrix rather than of any test in it.

**One seed proves nothing much.** The default seed is the commonest failure of discipline in constrained-random verification: engineers keep using it until the simulation runs error-free and consider the job done — but the same seed generates the same input sequence, so nothing has been verified under different conditions <sup>[1]</sup>. A test that passes on seed 1 has established one path through its own constraint space; what makes it valuable is the *distribution* it samples, and you have sampled a distribution only once you have drawn from it repeatedly.

**Seed budgets are an allocation problem, not a constant.** On the reference SoC, a DMA descriptor test spanning stride sign, boundary distance, burst length and other-channel activity (the four axes of `cg_2d_4kb`, Chapter 5) has a large space to sample; a UART framing test with three legal configurations does not, and its seventh seed buys almost nothing. Allocate proportionally to *unclosed* coverage: groups still short of target get seeds, groups closed and stable get one seed as a change detector.

**Reproducibility is a hard requirement and harder than it looks** — Chapter 9 develops why, and what a run actually is. What regression engineering adds is the artifact that discharges it, plus two habits: display the seed at the start of every run, and put it into every output pathname, since two runs of the same code on different seeds would otherwise overwrite each other’s results <sup>[1]</sup>.

Chapter 7 established that every data point must carry its provenance for the trend lines to mean anything <sup>[4]</sup>. That artifact is the **run manifest**, emitted by every run next to its log.

```
run_id: nightly-20260312-0417
test: dma_2d_stride_stress
```

```

seed: 3417755291
tier: nightly
rtl_rev: 9f21ac4 # design repo, tagged 'regress'
tb_rev: c07e13b # verification repo
tool: sim-2025.3-p2 # patch level, not just major version
config: DMA_CH=2, SRAM_KB=512, XBAR=5x4
plusargs: +UVM_TIMEOUT=12000000,N0 +cov_categories=dma,axi
host: farm-node-118 (linux-x86_64, 64 GB)
result: FAIL
signature: SCB_DATA_MISMATCH:dma_scoreboard:beat_compare
cpu_seconds: 641

```

Every field is there because somebody once lost a day without it. The `tool` line is worth arguing about: the *major* version is common practice and is not enough, as Section 12.5 demonstrates at some expense. `signature` makes Section 12.4 possible; `cpu_seconds` makes Section 12.6 possible.

**Random stability is what makes the manifest sufficient**, and it is narrower than most engineers assume. Chapter 9 develops the mechanism and its scope; the regression-level consequence is that it says nothing about changes to the design, to scheduling freedom, or to the simulator build — which is where reproducibility actually breaks. One cheap countermeasure: beside the fresh seeds, run one *fixed* seed nightly on a handful of tests. Fresh seeds measure the design; the fixed seed measures the machine, because if its result moves while its manifest does not, something outside the manifest did.

### 12.3 Continuous integration for hardware

In software, continuous integration is unremarkable. In hardware design it is still not very mature, and the reason is not cultural: a software unit test costs milliseconds and a hardware one costs minutes to hours, so the software answer — run everything on every commit — does not transfer. What transfers is the *principle*: automatically integrate design changes, run simulations and checks, and deliver verified descriptions or bitstreams, so errors are caught early and design quality stays consistent [2]. What must be engineered is the tier that runs per commit.

**Static checks are the entry condition**, not a parallel activity: one published pipeline groups linting and structural checks — static analysis, clock-domain and reset-domain crossing, elaboration — as its own job category ahead of the simulation categories [2]. Static analysis is cheap, deterministic, and catches defects simulation catches slowly or not at all, which is what belongs in front of a gate (see Chapter 13).

The pre-CI ancestor explains what the gate is *for*. Before automated pipelines, regressions ran on a tagged view: engineers applied a `regress` tag to design and test-bench revisions once they trusted the basic functionality, because the newest file might contain code never verified or that does not compile, and a regression spent on it is wasted [1]. Per-commit CI inverts the mechanism and keeps the goal — the commit *earns* the tag by passing the fast tier.



Three further gates are cheap. **A coverage gate:** a differential between two regressions shows which bins lost coverage, and a CI run can be failed automatically when work on constraints introduces such a regression <sup>[5]</sup> — the most insidious constraint change being the one that makes the suite faster and greener by asking fewer questions. **A performance gate:** making simulation performance a criterion for revision-control check-in reduces system simulation time <sup>[4]</sup>, and a commit that halves the design’s cycles-per-second is a schedule event invisible unless something measures it. **A per-test failure threshold:** in one published pipeline the emulation jobs each carry a configurable failure threshold, and labels selectively disable job categories across the pipeline <sup>[2]</sup> — because not every test is a hard gate, and a work-in-progress branch needs an escape hatch that is explicit and recorded rather than an engineer silently skipping the tier.

What the fast tier *runs* need not be a fixed list: a dependency map from RTL directories to the testbenches that exercise them keeps a commit touching only the APB peripherals from triggering the neural accelerator’s suite, and the research direction goes further, predicting a test’s failure probability and assembling a test set from changes in the RTL <sup>[6]</sup>.

**Two things change the gate without changing its mechanism.** Approaching RTL freeze (see Chapter 4) the fast tier’s role shifts from “catch mistakes early” to “prove this fix broke nothing”, so the gate gets *stricter* and the escape hatch *narrower* — a disable label reasonable in month four is a hole in the evidence in month eleven. And the social layer, on which the corpus offers no measurements, so treat this as practitioner judgment: a broken fast tier is reverted within a stated window rather than fixed in place, since a revert costs one commit and a broken gate costs everyone downstream; ownership of a break is by *commit* rather than by seniority, which is what makes it impersonal; and a rota rather than a volunteer watches the pipeline.

Finally, the fast tier’s budget is a design constraint, defended like a timing constraint. On the flagship SoC it is twelve minutes for lint, CDC and a 60-run smoke suite; when that grew to fifteen, two tests moved to the daily tier rather than the slip being accepted, because a gate a human will not wait for is a gate that gets bypassed. Volume is real too: one 2026 project reports 4,536 pipelines in a three-week window, successful durations ranging from 6.4 minutes to 2.2 days on average for the largest <sup>[2]</sup>.

## 12.4 Failure triage at scale

Return to the opening scenario: 37 failures, of which one fact was reported 29 times. Triage is the process that turns *failures* into *facts*, and at scale it is a data-reduction problem before it is a debugging problem.

**Deduplication by signature is the first reduction.** Daily failure analysis begins with two questions — are these failures new, and if they look alike, are they really the same as one seen before? — which a first-failure analysis does not always answer, because the classification can be *over-generic*, mapping many bugs into one

group, or *over-specific*, splitting one bug across many [7]. Both are expensive, in opposite directions: over-generic buckets destroy real finds, as the opening scenario shows; over-specific buckets destroy the reduction, since 37 failures in 34 buckets is no better than 37 failures. A signature is therefore an artifact with a specification: derived from the failure rather than the test, excluding everything that varies run to run, built from the *first* error, since later errors are usually consequences.

```
signature := <ERROR_CLASS>:<COMPONENT>:<CHECK>
 from the FIRST error line, with these stripped:
 - timestamps, simulation time, run ids, seeds
 - hierarchical paths below the component boundary
 - numeric operands (addresses, IDs, data values)
example: SCB_DATA_MISMATCH:dma_scoreboard:beat_compare
counter-ex: UVM_ERROR @ 12480ns [dma_sb] exp=32'hDEAD_BEEF got=32'hDEAD_BEE0
 (over-specific: every seed produces a distinct signature)
```

The seed is stripped from the signature and kept in the manifest. Chapter 9’s rule that the seed travels with the failure is about the failure *report*; this is the dedup *key*, and a key carrying the seed makes every failure unique — the over-specific pathology by construction.

Automation goes further than string matching. Failure clustering can be learned from the assignments engineers already make: when an engineer assigns a failed run to a cluster, the system extracts that run’s properties as characteristics of the cluster and proposes it for runs that look similar — a capability one vendor describes as in progress [7]. Published results elsewhere are encouraging but unfinished: in one study reported by a 2023 survey, 616 features drawn from simulation-log metadata and message lines gave clustering scores well short of ideal, while supervised classification of root causes reached about 91% accuracy with a random-forest model [8]. Read that as a division of labor: automation proposes the bucket, a human confirms, and the confirmation is training data.

**The second reduction is classification into three kinds** — design bug, testbench bug, environment failure — whose ratio is itself a health metric. Testbench bugs are not noise — the environment is a design too, and it needs the same bug-tracking discipline (see Chapter 4). Environment failures — disk quota, licence starvation, a dead farm node — are the only category that may be discarded without debugging, and even they must be *counted*, because a rising count is an infrastructure problem masquerading as flakiness.

**The third reduction is not to debug the same thing twice.** Among the metrics a regression process should track is the count of errors *re-found* — wasted simulation [4], a number few teams instrument and one of the most actionable when they do, since a suite spending 40% of its night re-discovering three known-open bugs spends 40% of its night learning nothing. The cure is unglamorous: known-failure filters keyed on signature, expiring when the bug closes. The metric is what tells you the filters have gone stale.

**Debug economics.** That debug dominates a verification engineer’s time is established (see Chapter 7); this chapter’s contribution is the *per-failure* accounting that

makes it manageable. Instrument bug closure with three quantities — simulation time, engineer time, runs consumed — and triage becomes a ranked problem rather than a queue. Because instrumentation itself costs simulation resources, a regression can run with most metrics disabled and re-run only failing tests with the categories their failure type needs <sup>[4]</sup>. Three practices follow, described in one vendor’s regression-management flow: prioritize the highest-impact failure modes, identify the *shortest* test reproducing each, and rerun automatically to generate the debug data the engineer will need rather than making them ask <sup>[7]</sup>.

The organizational counterpart is a **triage rota** — one engineer per week owning the morning queue, who does not debug but classifies, deduplicates, assigns, and files whatever the buckets say about the machine. Debugging and triage compete for the same person and triage always loses, because debugging is more interesting.

## 12.5 Flakiness as a first-class enemy

---

A **flaky test** is one whose outcome varies between runs whose manifests are identical — same test, same seed, same revisions, same tool, same configuration. It is not a failing test and not a passing one; it is a test that has stopped being evidence. The term is borrowed from software testing, where the problem has a name and a decade of study; the hardware-verification literature has barely begun to name it.

Here is the judgment this section rests on: *a flaky test is more expensive than a failing one*. A failing test carries information — something is wrong, go look. A flaky test carries an instruction, and the instruction is **ignore red**. Teams learn it fast and generalize it, and the generalization is what destroys the regression’s value. The opening scenario ends with a real DMA bug closed as a duplicate of “the flaky one”: not carelessness, but a team behaving rationally on the training data its own regression gave it.

That thesis is judgment. Every mechanism below is not: nondeterminism has a small number of recurring sources, each documented, and most live in the environment rather than the design.

- **Sampling and scheduling races.** Synchronous interface signals must be sampled and driven through a clocking block (IEEE Std 1800-2023, *IEEE Standard for SystemVerilog* <sup>[9]</sup>), which avoids races between design and verification environment and lets one environment work against RTL and gate-level models unchanged. Sampling in the active region instead is where testbench flakes most often start: when the design’s update and the monitor’s sample fall in the same time step, which wins is not something you specified. Two *testbench* processes resuming on the same edge raise a related question with a different answer — see the worked example. The same family includes clock edges at time zero and environment instantiation, where a local variable prevents initialization races <sup>[10]</sup>.
- **Uninitialized state.** X-optimism — a simulation behavior in which a deterministic 0 or 1 propagates where silicon would have an unknown — can mask RTL design bugs, though not all of it is unwanted: asynchronous reset works *because*

of it <sup>[3]</sup> (see Chapter 13). Its regression-level symptom is a result that depends on what a register happened to hold.

- **Time-dependent checks.** A check written against wall-clock-like quantities rather than design events is nondeterministic by construction. The Ethernet MAC’s TSN queues are the cautionary case: a timestamp-precision check whose tolerance derives from the *host’s* measured latency instead of the design’s own clock passes on a quiet farm and fails on a busy one.
- **Provenance drift.** Random stability guarantees an unaffected instance the same sequence for the same seed even when other instances change <sup>[1]</sup> — but it is scoped to randomization-related *source* changes and says nothing about a different simulator build. A manifest recording the tool’s major version and not its patch level permits flakes.
- **Infrastructure collisions and runaways.** Two simulations of the same code with different seeds writing the same output filename lose the first run’s results, surfacing as a run that “failed for no reason” and passes when rerun alone. And a simulation waiting for a condition that never occurs runs until 64-bit simulation time reaches its maximum, which is to say forever, so a time bomb belongs in every simulation — but never as a test’s *normal* ending, or success and deadlock become indistinguishable <sup>[1]</sup>. A watchdog — required in every system-level environment, and worth the same discipline below that — must likewise halt a run in the absence of observed progress, since such runs block the tests behind them <sup>[10]</sup>.

The counter-discipline is to make instability *measurable*. One published flow does this for tests it cannot parallelize — an operating-system boot cannot be split across machines, so it is executed eight times, specifically to establish the test’s stability <sup>[2]</sup>. Generalize that into a **repeat-N stability check** on any test entering a gating tier. When a flake is confirmed, quarantine it explicitly: a named list, still running and reported but not gating, with an owner and a date. A quarantine list nobody empties is the same disease with better manners.

### Worked example: hunting a one-in-four-hundred race

Over twelve weeks — sixty nights of the reference SoC’s suite — the signature `SCB_DATA_MISMATCH:dma_scoreboard:beat_compare` appeared 35 times, always on a different seed. Each morning it was closed as “the flaky one”. The suite’s 900 nightly runs include 260 that instantiate the DMA environment, so the field rate is 35 failures in 15,600 DMA runs: about one in 450.

**Step 1 — refuse the bucket.** Check first that 35 failures are one thing. They were not: the signature was over-generic <sup>[7]</sup>, and three of the 35 carried a distinct first-error line that a stricter signature separated out — two a real, already-fixed midend bug, one a farm node that died mid-run. Thirty-two remained, sharing a first error and a component: one question instead of thirty-five, at a field rate near one in 490.

**Step 2 — buy statistics instead of waiting for them.** One in 490 is invisible at 260 runs a night. A weekend campaign ran 4,000 fresh seeds of the DMA test set —

$4,000 \times 11$  minutes = 733 CPU-hours, 30.6 hours of wall clock on the reference SoC's 24 slots — and produced 9 failures: one in 444, now a number rather than an impression.

**Step 3 — rerun from the manifest, and discover the manifest is wrong.** Rerunning the 9 failing manifests reproduced 6 and not the other 3, which with the same seed and the same revisions should have been impossible. It was: the farm ran two patch levels of the simulator and the manifest recorded only the major version. Pinning `tool` to the exact build made all 9 reproduce deterministically. This step diagnosed the *provenance*, not the race — but without it every later experiment would have been unrepeatable, and the team would have concluded the failure was “in the design, somewhere, sometimes”.

**Step 4 — bisect the observation path.** With reproducibility restored the race was findable, and it had two halves. The response monitor sampled the DMA backend's response channel from an `always @(posedge clk)` block rather than through a clocking block; worse, it *popped* the expected item and compared it there and then, in the same region the command monitor pushed it. Processes sensitive to one event are evaluated in an arbitrary order <sup>[9]</sup>, so which of the two ran first was the simulator's choice — which is exactly why two patch levels of it disagreed. It mattered only when a response arrived in the same cycle as the last beat of the previous descriptor, which needs the 16-entry backend FIFO to sit exactly at a boundary: a coincidence the random descriptor stream produces about once in four hundred and something runs.

```
// before: samples in the active region, and pops the expected item there too,
// so the pop and the command monitor's push are ordered by the simulator
always @(posedge clk)
 if (rvalid && rready)
 compare(exp_q.pop_front(), sample_response(rdata, rresp, rid));

// after: sample through the clocking block, and only enqueue — the scoreboard
// owns the comparison, so the order is stated rather than inherited
clocking mon_cb @(posedge clk);
 default input #1step;
 input rvalid, rready, rdata, rresp, rid;
endclocking
...
@(mon_cb);
if (mon_cb.rvalid && mon_cb.rready)
 obs_q.push_back(sample_response(mon_cb.rdata, mon_cb.rresp, mon_cb.rid));
```

Two changes, each curing one half. Every field the monitor records now comes from `mon_cb`, and a `#1step` input skew samples in the preponed region — the value present before the clock transition <sup>[9]</sup> — so what the monitor captures no longer depends on what the design does at that edge. And the monitor stops popping: it enqueues, and the scoreboard compares in its own process.

Note which change did which, because the clocking block is the seductive one. It defines *when* a signal is sampled; it does not order two testbench processes against each other. A reader who moves *both* monitors behind clocking blocks on the same



clock has them resuming together again with no defined relative order — and the bug is back. The enqueue-and-compare-elsewhere change is what closes it; the clocking block is what makes the evidence trustworthy.

**Step 5 — prove the fix honestly.** Re-running the 4,000-seed campaign produced zero failures. State what that establishes and no more: zero events in 4,000 trials bounds the true rate below roughly 1 in 1,300 with 95% confidence. It does not prove zero, and the difference between “bounded below 1 in 1,300” and “fixed” is the difference between an engineering claim and a hope.

The race itself was a small fix. What it cost was twelve weeks of a signature that trained the team to close DMA scoreboard mismatches without reading them — and at least one genuine failure closed as a duplicate.

## 12.6 Compute economics

The regression’s budget is not a policy but a physical quantity, and it is the constraint shaping every decision above. Adding CPU cores buys throughput, not evidence: large regressions consist of redundant simulations that do not improve coverage while still consuming simulator resources and lengthening turnaround <sup>[11]</sup>.

Two published numbers calibrate the scale. In a 2023 study, a commercial signal-processing unit’s verification required six months of simulation of nearly two million constrained-random tests on almost a thousand machines and EDA licences, at about two hours per test <sup>[12]</sup> — four million CPU-hours, approximately what a thousand machines produce running continuously for six months. The farm was saturated for the project’s duration; “run more” was not on the menu. Parallelism has its own ceiling: in a 2026 report, distributing a 1,738-test suite across an eight-board FPGA-emulation cluster cut it from about 5.2 hours to about 53 minutes — a 6× speed-up, not 8×, with near-linear gains only for the independent categories and none for tests that cannot be divided <sup>[2]</sup>. The ceiling shows up even where boards, not licences, are the constraint.

So the question is **what to run when you cannot run everything**.

**Rank.** Ranking chooses tests by their contribution to coverage improvement, producing a compressed suite of specific (test, seed) pairs and eliminating redundant runs <sup>[11]</sup>. The idea predates the tooling: coverage tools grade data sources by absolute contribution or rate of contribution over time, and the winning simulations *together with their initial seed values* form the regression suite, so running the most efficient first reaches the same coverage rating in less time <sup>[10]</sup> — start with the highest-contributing seeds and fill any remaining time with fresh random ones <sup>[1]</sup>. As a management instrument, ranking adjusts run *order* and *frequency*, not just membership <sup>[4]</sup>.

The reported compressions are large. In 2022 industrial results (posted in 2024), on an automotive microprocessor IP, 26 tests at 10 seeds each — 260 runs, 12 CPU-hours — compressed to 8 runs and 1.1 CPU-hours at 100% of the original coverage. On an industrial mixed-signal SoC, 5,124 runs spanning seven configurations compressed to 1,204 — a factor of 4.25, again at full coverage regain <sup>[11]</sup>. Both are Infin-



eon Technologies’ own projects — the paper reports three in all — and the two methods compared on them were classical seed ranking and Cadence Xcelium ML, which learns the correlation between randomization points and achieved coverage in earlier regressions and then emits an optimized pattern set; the automotive microprocessor IP is an ASIL-D safety product implemented in VHDL <sup>[11]</sup>. The paper identifies its projects by class rather than by part number, so that is as far as the attribution goes.

**Know what ranking costs.** A ranked regression selects existing test-and-seed pairs and simulates no new scenarios, so it is not recommended for exploring the random space or exposing new bugs <sup>[11]</sup>. The compression also goes stale: regenerate it whenever the design or testbench moves significantly. Ranking reproduces yesterday’s coverage cheaply; it cannot find tomorrow’s bug.

**Select, don’t just rank.** The complementary technique filters tests *before* simulating them, on the hypothesis that dissimilar tests hit dissimilar coverage events. On the same 2023 signal-processing unit — the same class of design as the radar front-end — neural-network novelty selectors beat random selection in every configuration, the best saving 49.37% of simulation to reach 99.5% coverage at a computational expense negligible against the simulation avoided <sup>[12]</sup>. Machine-learning-generated regressions go further and stay random: in the same 2022 results, optimized regressions regained 99% or more of the original coverage at roughly 3× compression, and in some configurations *more* than 100% by hitting bins the original had missed <sup>[11]</sup>. Ranking buys speed at the price of exploration; novelty selection tries to buy speed *from* exploration.

Two smaller levers: pruning the *coverage collection* rather than the tests trades a bounded loss of measurement for compute (see Chapter 7), and nightly runtime is itself a metric with an owner, since a spike in a block’s regression time is worth catching before it becomes archaeology <sup>[4]</sup>.

### Worked example: 40 runs, 900 runs, and a suite that no longer fits

**Week 6 — 40 runs.** Forty directed tests, one seed each, mean two minutes: 80 CPU-minutes on one workstation over lunch, one engineer reading every log. No tiers, no signatures, no manifests, and none needed. Every technique in this chapter would be overhead.

**Month 9 — 900 runs.** The suite that was 40 tests is now 180 distinct tests: 120 constrained-random at 7 seeds each (840 runs) plus 60 directed at one seed (60 runs). Mean run time is 11 minutes, so the nightly tier costs 165 CPU-hours; on 24 concurrent simulator licences that is 6.9 hours of wall clock inside a 10-hour night. Smoke is 22 fixed-seed runs of at most 90 seconds, one wave across the same 24 slots: under nine minutes end to end, most of it build. The weekend tier raises the random tests to 40 seeds each — 4,800 runs plus the 60 directed — for 891 CPU-hours, 37.1 hours on the same 24 slots inside a 48-hour window. Everything fits, with headroom that will be gone in four months.

**The flagship SoC — the same suite, a night that no longer contains it.** The reference SoC’s DMA suite carries forward directly (same three-stage architecture, same 4 KB legalization discipline), the register and peripheral suites carry forward, and the plan adds what eight channels, a coherent host cluster and a mesh NoC demand. The nightly *candidate* list is 6,400 runs:

**Table 12.2** — The flagship SoC’s nightly candidate list before any re-tiering: 6,400 runs in three groups, each with that group’s mean run time and the CPU-hours it costs. The total is the number that does not work — 4,409 CPU-hours on the farm’s 240 concurrent simulator licences overruns the night, and that overrun is the gap the re-tiering, the ranking and the pruning have to close.

Component	Runs	Mean	CPU-hours
System-level (OS boot, coherent traffic soak)	340	3.5 h	1,190
Constrained-random block/subsystem (600 tests × 8 seeds)	4,800	30 min	2,400
Directed and configuration runs	1,260	39 min	819
<b>Total</b>	<b>6,400</b>	<b>41 min</b>	<b>4,409</b>

The farm allows 240 concurrent simulator licences, so 4,409 CPU-hours is 18.4 hours of wall clock. The night is 10. The suite overruns by 8.4 hours — which means the morning summary arrives at 11:00 and the triage rota’s day is destroyed. Three decisions, in this order:

1. **Re-tier.** Three hundred of the 340 system-level runs move to the weekend; 40 stay as a nightly system smoke. Cost falls by 1,050 to **3,359 CPU-hours** (14.0 h wall). Still over.
2. **Rank the random block.** The 4,800 random runs compress to 1,150 — a factor of 4.17, in line with the 4.25 reported in 2022 on an industrial mixed-signal SoC [11] — for 575 CPU-hours. Cost falls by 1,825 to **1,534 CPU-hours** (6.4 h wall).
3. **Prune the directed set.** One hundred and eighty directed runs are retired under the hygiene rules below, leaving 1,080 runs at 702 CPU-hours. Cost falls to **1,417 CPU-hours** — 5.9 hours of wall clock, with four hours of headroom for the growth that will certainly come.

The weekend tier absorbs what moved and more: 300 system runs (1,050 CPU-hours), the *unranked* random set at 20 seeds instead of 8 — 12,000 runs, 6,000 CPU-hours — and the **retained** directed set (702), the 180 retirees having left both tiers for a parole list that runs on demand. Total 7,752 CPU-hours, 32.3 hours on 240 slots inside a 48-hour window.

The load-bearing decision is the one that looks like a detail: **the nightly tier is ranked and the weekend tier is not.** A ranked regression reproduces known coverage and explores nothing, so a project whose *only* regression is ranked stops finding bugs while its dashboard stays green [11] — the flat-curve pathology of Chapter 7, manufactured by the regression itself. Ranking buys the night; the unranked weekend buys back the exploration; and the ranking is regenerated whenever the design or testbench moves significantly, so it stays a compression of the current design rather than a fossil of an old one.

## Regression hygiene over time

Decision 3 is the one that needs a discipline behind it, because left alone a suite only grows: adding a test is virtuous and visible, deleting one looks like reducing verification, and nobody wants to be the engineer who removed the test that would have caught it. Three categories of dead weight are worth naming, each with a detection rule.

**Tests whose reason has expired.** Many tests exist because of a specific bug, and knowing when that bug was closed tells you when the test no longer needs to run <sup>[4]</sup> — which makes bug closure a hygiene event, not just a tracker transition. The rule is not “delete on close”, since a bug’s test is what protects against its recurrence; it is that closing a bug schedules a *decision* — keep permanently, demote to weekend, or retire — recorded with the closure.

**Tests that duplicate other tests.** Regressions consist of redundant simulations that do not improve coverage <sup>[11]</sup>, and ranking measures that redundancy for free: a test whose every seed is dropped by successive rankings contributes nothing the rest of the suite does not. Measuring redundancy and *acting* on it are separate acts, and only the second frees the night.

**Tests nobody can justify.** Every test should discharge something — a plan row, a coverage group, a closed bug (see Chapter 5). One discharging no row, contributing no unique coverage and finding no bug in six months is a retirement *candidate*, not a corpse: retirement goes through a one-line review, because the cost of wrongly deleting a test is asymmetric. The record is the artifact.

```
RETIRE dma_desc_legacy_mode (60 runs/night, 11 CPU-h)
Reason: the legacy descriptor format it exercises was removed from the RTL at
 rev 9f21ac4; no vplan row now depends on it
Unique coverage contributed: 0 bins over last 12 rankings
Bugs found: 1, closed 2025-11-04, fix protected by a_no_4kb_crossing
Reviewed by: DMA owner + verification lead Decision: retire; 2-month parole on-
demand
```

Two costs beyond runtime deserve the same discipline. Measurement is not free — functional metrics must be architected, written and maintained, and their simulation cost is real <sup>[4]</sup> — so an unread coverage group is dead weight exactly as an unread test is. And regression *data* accumulates faster than tests: snapshot what tracking needs and discard the raw results (see Chapter 4). A farm out of disk is indistinguishable, in the morning queue, from a design that has broken.

---

## IN PRACTICE

Nearly every team discovers, late, that its regression grew a *configuration* dimension nobody planned — one industrial mixed-signal SoC’s suite in 2022 spanned seven configurations distinguished by whether a CPU, a verification IP and a particular flash type were instantiated <sup>[11]</sup> — and configurations multiply the matrix in a way tests do not. Expect ownership of the machinery to be contested: infrastructure be-

longs to everyone and therefore to nobody unless a person is named. And expect the first serious ranking exercise to be uncomfortable, because it will show that much of the night is redundant, which reads as a criticism of whoever wrote those tests. It is not. Redundancy is the normal end state of a suite that grew honestly, one bug at a time.

---

## PITFALLS

1. **Green because nothing is asked.** A pass rate that rose after a constraint change may simply mean the suite is asking less; gate on coverage differentials, not only on pass/fail <sup>[5]</sup>.
2. **The manifest that omits the tool build.** A major version instead of the exact build makes a whole class of flake undiagnosable — and looks like a design bug for weeks.
3. **Over-generic signatures.** One bucket holding many distinct defects will eventually swallow a real find <sup>[7]</sup>; a signature that never splits is not a signature.
4. **Ranking the only regression you run.** A compressed suite reproduces known coverage and explores nothing new; keep an unranked tier, and regenerate the ranking when the design moves <sup>[11]</sup>.
5. **The time bomb as normal termination.** If tests routinely end on the watchdog, success and deadlock become indistinguishable <sup>[1]</sup> — and the regression stops discriminating exactly where it matters.
6. **Triage by whoever happens to care.** Without a rota the duty lands permanently on one engineer, and stops silently when they take a week off.

---

## SUMMARY

- A regression is a machine to be engineered, not a list to be accumulated: tiers are admission policies with runtime budgets, not schedules.
- The unit of work is the (test, seed) pair. One seed samples a distribution once <sup>[1]</sup>; seed budgets should follow unclosed coverage, not fairness between tests.
- Reproducibility must be *easy*, not merely possible <sup>[10]</sup> — a per-run manifest recording seed, revisions, exact tool build and configuration. An incomplete manifest is a defect in its own right.
- Per-commit gating works for hardware when the fast tier is genuinely fast and static checks sit in front of it <sup>[2]</sup>; gate on coverage differentials and simulation performance too <sup>[4, 5]</sup>.
- Triage is data reduction before it is debugging: signatures neither over-generic nor over-specific <sup>[7]</sup>, three-way classification, and a measured count of errors re-found <sup>[4]</sup>.
- Flakiness damages the team rather than the schedule, because it teaches people to ignore red. Hunt flakes with statistics, and state what a clean re-soak does and does not prove.

- Ranking and selection buy back the night at a stated price — a ranked regression explores nothing new <sup>[11]</sup> — so buy deliberately and keep an unranked tier.

---

## FURTHER READING

Corpus sources, in the order they matter here. <sup>[1]</sup> §7, *Simulation Management*, is the classic treatment: seed management and random stability (p. 364), then the regression section proper (pp. 365–369) — the nightly/weekend split, seed ranking, the time bomb, the tagged regression view. <sup>[10]</sup> supplies the reproducibility requirement, the watchdog rule and the environment-side race rules that generate most flakes; <sup>[9]</sup> is the authority on what the scheduler does and does not guarantee. <sup>[11]</sup> is the reference for ranking and machine-learning regression compression, with quantified industrial results and an unusually honest account of what compression costs — read knowing one of its authors works for the vendor of the tool it evaluates; <sup>[12]</sup> gives both the farm-scale calibration and the novelty-selection results. <sup>[7]</sup> and <sup>[4]</sup> describe the regression loop, failure clustering, test ranking and the metrics worth tracking, both vendor accounts with no population behind them; <sup>[5]</sup> covers coverage-based CI gating and coverage-collection pruning, with measurements. <sup>[2]</sup>, an arXiv preprint, is the most recent account of hardware CI tiering in this corpus; <sup>[6]</sup> reviews machine learning across simulation-based verification, including a section on test selection; and <sup>[8]</sup> surveys automated failure clustering and root-cause classification.

Not in the corpus and worth seeking out: the software-engineering literature on continuous integration, which transfers better than hardware engineers expect once the cost asymmetry between a unit test and a simulation is accounted for; and the software-testing literature on flaky tests, where the problem has a name and a decade of study.

---

## REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. **P25** S. Ahmed, A. Kropotov, R. I. Genovese, B. Homs, E. Merino, F. Urbani, et al., "Verification and Validation (V&V)-in-the-Loop for RISC-V Design: The Holistic Vision of BZL," arXiv preprint arXiv:2604.27013v1, April 2026.
3. **D23** S. Zhao, S. Yan, and Y. Feng, "Practical Approach Using a Formal App to Detect X-Optimism-Related RTL Bugs," Proc. DVCon U.S., 2014.
4. **D20** A. Meyer and H. Foster, "Metrics in SoC Verification: Not just for coverage anymore," Proc. DVCon U.S., 2012.
5. **D22** S. Hristozkov, J. Pallister, and R. Porter, "No Country For Old Men – A Modern Take on Metrics Driven Verification," Proc. DVCon Europe, 2021.
6. **P8** C. Bennett and K. Eder, "Review of Machine Learning for Micro-Electronic Design Verification," arXiv preprint arXiv:2503.11687v1, March 2025.
7. **D26** D. Zhang, "Smarter Verification Management with vManager Platform," Proc. DVCon China, 2021.
8. **D35** D. Yu, H. Foster, and T. Fitzpatrick, "A Survey of Machine Learning Applications in Functional Verification," Proc. DVCon U.S., 2023.
9. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.

10. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
11. **P7** D. N. Gadde, S. Simon, D. Lettnin, and T. Ziller, "Improving Simulation Regression Efficiency using a Machine Learning-based Method in Design Verification," Proc. DVCon Europe, 2022.
12. **P5** X. Zheng, K. Eder, and T. Blackmore, "Using Neural Networks for Novelty-based Test Selection to Accelerate Functional Coverage Closure," Proc. IEEE International Conference on Artificial Intelligence Testing (AITest), Athens, Greece, pp. 114-121, 2023.





PART IV

## Static and Formal

---

*Proving instead of sampling: from the static entry gate to model checking, the industrialised formal applications, and the hybrid flows that make one campaign of both.*

---

## 13. Static Verification

The reference SoC comes back from the lab with a defect nobody can reproduce. One board in forty hangs on wake-up from deep sleep, roughly once a day, always at the moment the always-on domain hands control back to the switchable system domain. RTL regression is green: eleven thousand seeds, every wake path the plan enumerated, the scoreboard silent throughout.

The design is correct at the level the simulation can see. The event unit encodes the reason for a wake-up in a three-bit cause code, produced on the 32 kHz always-on clock and consumed on the 200 MHz system clock. The encoding is sparse — four legal codes scattered across eight code points — and each of the three bits crosses the boundary through its own two-flop synchronizer. Three textbook synchronizers, one per bit, and a linter finds nothing to say about any of them. But when the code changes from `3'b010`, a GPIO wake, to `3'b100`, a comparator wake, two of the three bits change at once, and on the crossing edge each destination flop resolves independently. For exactly one system cycle the destination can therefore observe `3'b110` — a reserved code that the source never produced. The decoder has a branch for each legal cause and none for the rest, so for that cycle it holds its previous decode. The wake sequencer latches the decode as the cause code changes, captures the stale value, dispatches nothing new — and the handshake never completes.

Simulation could not have shown this. The source flops change at one instant of simulated time, the destination flops sample one instant later, and what they sample is exactly what the source drove: metastability is a behaviour of a flip-flop with real timing, not of RTL. Demonstrating this class of bug in simulation needs a gate-level model with timing, which does not exist yet at RTL sign-off, and the same bugs are hard to reproduce in the lab once silicon arrives <sup>[1]</sup>.

What could have found it — in seconds, before a testbench existed — is a structural analysis that walks the elaborated design, finds every path leaving one clock domain and entering another, and asks whether a multi-bit crossing is protected against its bits resolving independently. No stimulus, no reference model, no seed. That analysis is the first gate in Part IV, and this chapter is about what stands behind it.

---

### CHAPTER MAP

- §13.1 establishes what static analysis buys before a single simulation runs, and the precise boundary of what it can decide.
- §13.2 sets out what a linter actually finds, why a rule set turns into noise, and how to make one a policy instead.
- §13.3 develops clock-domain crossing — metastability and mean time between failures, and the standard synchronizer structures — and §13.4 separates the *structural* check from the *protocol* check, which answer different questions.

- §13.5 turns to reset-domain crossing: why it became a discipline of its own, and what it catches in a design with a single clock.
- §13.6 treats X-propagation: how RTL simulation lies optimistically and pessimistically at the same time, and what the gate-level, simulator and formal answers cost.

## 13.1 The entry gate

---

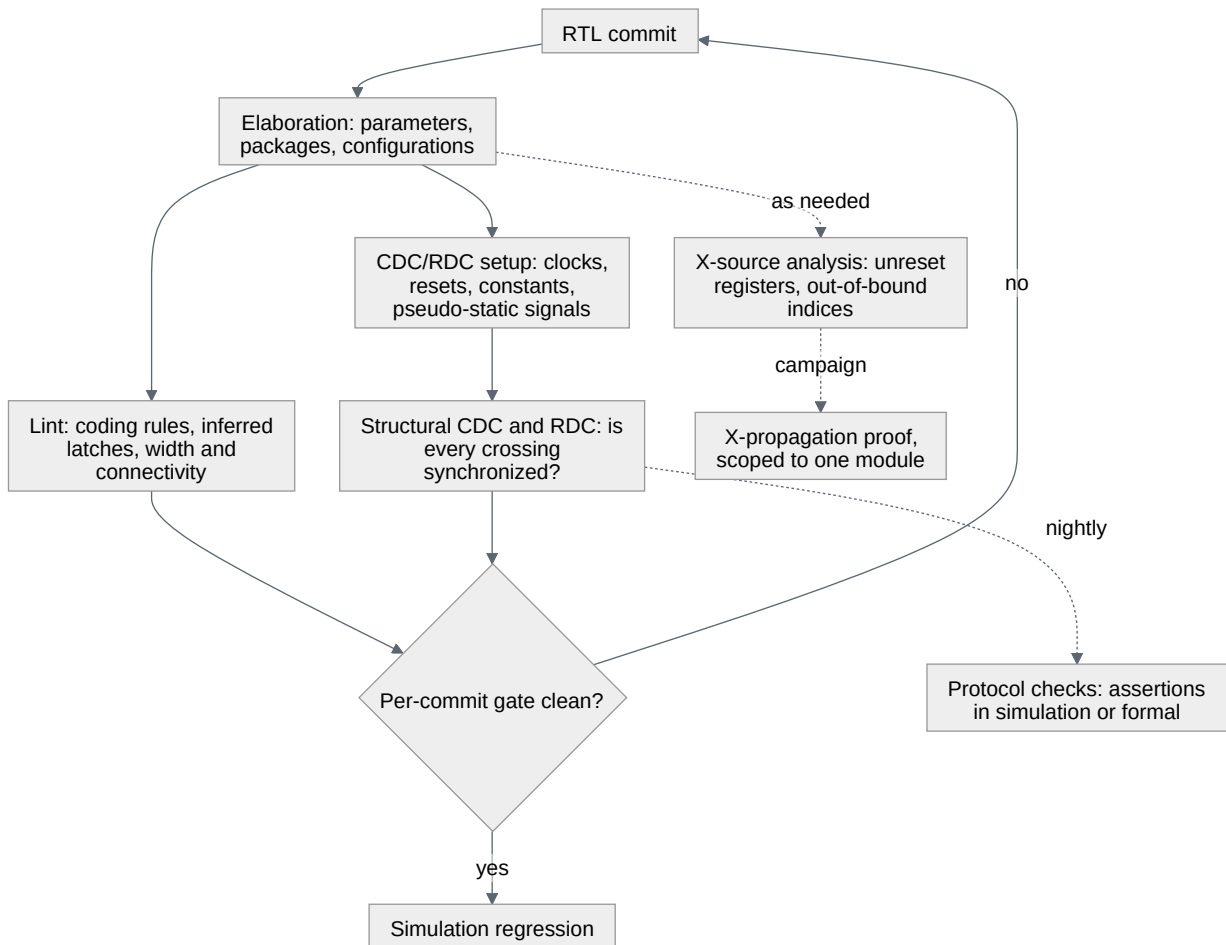
Static verification is analysis of the design description itself. It requires no stimulus and no statement of the expected output; the expectations are built into the checker [2]. That one sentence explains both its position in the flow and its limits.

Chapter 3 framed dynamic verification as bounded by controllability and observability. Static analysis has neither problem, because it has neither power. It drives nothing, so no condition is unreachable and no coverage question arises; it observes no behaviour, so nothing it reports is about what the design computes. It examines the shape of the code and the netlist that code implies, exhaustively, reporting every instance of a structural pattern it was taught to recognise.

That trade — total coverage of a structural question, total silence on functional ones — is what earns it the front of the flow. It runs on the first day RTL compiles, weeks before the environment in Chapter 4’s bring-up phase drives its first checked transaction, and the cheapest bug is the one its author catches the afternoon it was written [2]. It is deterministic: no seed, no run-to-run variation, an identical report on unchanged source — which is what makes it usable as a gate (see Chapter 12) and as an audited sign-off row with its own waiver file (see Chapter 7). And it reaches classes simulation reaches slowly or never; the opening scenario is the extreme case, and static clock-domain-crossing tools exist precisely because that class needed an answer earlier than gate-level timing simulation could give one [1].

What it cannot do is decide function. A static tool finds problems deducible from code structure, not problems in the algorithm or the data flow — a spell checker will find a misspelled word and will never notice that you wrote *with* where you meant *width* [2]. Every *structural* check in this chapter is a check on form, and a design that passes all of them can still compute entirely the wrong result. Sections 13.4 and 13.6 add the functional half, and it takes an engine that evaluates behaviour.

Within the static half the checks are not peers: what is fast and deterministic enough to run on every commit sits inside the gate, and what is slower or needs more setup belongs on a nightly or a campaign schedule. [Figure 13.1](#) places each one relative to that gate.



**Figure 13.1** — Where each static check sits relative to the per-commit gate: elaboration first, then lint and structural CDC and RDC inside the gate, whose two exits are the simulation regression and the commit sent back to the RTL. The dotted edges are the checks deliberately left outside it — protocol checks nightly, X-source analysis as needed, an X-propagation proof as a scoped campaign — because a gate that also carries the slow checks stops being fast enough to wait for. What the diagram does not show is the setup behind its CDC and RDC node, which is where the engineering lives and where §13.3 shows what one unreviewed line can hide.

### The setup is the hard part

The diagram hides the work. A structural CDC check is not a button; it is a *setup* followed by a button, and the setup is where the engineering lives. The tool must be told which clocks exist, which of them are asynchronous to which, which inputs are constant in the mode being analysed, which signals are pseudo-static — changed only while the receiver is held in reset or has its enable low — and which buses are gray-coded [3].

Timing constraints also declare clocks and exceptions, so it is tempting to point the CDC tool at the constraint file and start. Do not. The clock groups a timing tool needs are not the clock groups a CDC tool needs, and a path waived for timing analysis is not a path that may be waived for CDC analysis [3]. A false path in timing means *do not schedule this*; a false path in CDC means *there is no crossing here at all*, which is a much stronger claim and usually a wrong one.

The reference SoC makes the point concretely. It has three clock sources on paper — system at 200 MHz, peripheral at 50 MHz, always-on RTC at 32 kHz — but “three clocks” is not the same statement as “three clock domains”. A clock domain is the set of flip-flops driven by one clock or by clocks derived from a common source [3]. Had the 50 MHz peripheral clock been the system clock divided by four, the two would be edge-aligned and share a source, and declaring that boundary asynchronous would bury the report in hundreds of crossings that are not crossings. It is not: the peripheral clock has its own source, so the boundary carries a real crossing on every register interface in the APB subsystem, and declaring it synchronous would silently delete the entire check. Nothing in the RTL distinguishes the two cases. Somebody has to read the clocking specification and write the answer down.

**Scenario.** The reference SoC’s integration team wants a per-commit static gate.

**Approach.** Fix the gate’s contents once, in the plan: elaboration with the production parameter set, lint at the “must-fix” severity only, structural CDC and RDC against a checked-in setup file, and nothing else. Everything slower sits outside it: protocol assertions nightly, the X-propagation proof as a scoped campaign. The setup file is reviewed like RTL and lives beside it, and every constant or pseudo-static declaration in it names the mode it assumes. **Why.** A gate that also carries the slow checks stops being fast enough to wait for, and Chapter 12 has already shown what happens then. The second half is specific to static verification: the setup file is design intent written down, and Section 13.3 shows what one unreviewed line in it can hide.

## 13.2 Lint: from noise to policy

Lint’s ancestry is a C utility that reported questionable constructions the compiler accepted — mismatched argument types, a value used before it was set, a return value silently ignored [2]. RTL linting kept the ancestry and inherited the pathology.

What it finds falls into four families. *Synthesis-intent mismatches*: a combinational block that does not describe combinational logic — the language itself asks tools for this one, saying a procedure declared `always_comb` should be flagged if latched behaviour can be inferred from its body [4]. *Width and type errors*: a truncated assignment, a signed-unsigned comparison, an expression one bit wide where the author meant a vector. *Connectivity and structure*: floating input ports, dangling wires, a variable with no driver, a variable with several.

And *language constructs whose semantics differ from what the author probably meant*. `case` is the standard example, and the reason many groups ban it outright is worth stating precisely: it treats both `x` and `z` as do-not-care in any bit of *either* the case items *or* the case expression (IEEE Std 1800-2023, *IEEE Standard for SystemVerilog*) [4] — so a runtime unknown on the selector matches the first item whose remaining bits agree, and the design takes a branch nobody chose.

Here is the fragment behind the opening scenario, at the point where lint could have said something:



```

module wake_decode (
 input logic [2:0] cause,
 output logic [1:0] branch
);
 always_comb begin
 case (cause)
 3'd0 : branch = 2'd0; // sparse encoding, no default
 3'd1 : branch = 2'd1; // no wake pending
 3'd2 : branch = 2'd2; // timer
 3'd3 : branch = 2'd3; // GPIO
 3'd4 : branch = 2'd3; // comparator
 endcase
 end
endmodule

```

Four of the eight values of `cause` are decoded. For the other four — `3'd3`, `3'd5`, `3'd6`, `3'd7` — no assignment executes, so `branch` keeps the value it held: latched behaviour inside a procedure that declares itself combinational, and a latch in the netlist. Note what the linter sees. A synthesis-intent mismatch and a missing `default`, reported on the day the file is written. It does not know that `cause` crosses a clock boundary, that its bits can transiently disagree, or that `3'd6` is the transient the lab is hitting. Its report is structural, correct, and several levels of abstraction below the bug.

### Why teams drown

Linting errs on the side of caution: rather than let a real problem go unreported it reports potential problems where none exist, and engineers who spend a week chasing non-existent problems become engineers who stop reading the output, then engineers who abandon linting altogether <sup>[2]</sup>. This is not a defect a better tool fixes. It is intrinsic to a checker deciding from structure alone whether a construct is a mistake, and two multipliers make it worse.

The first is scale, and the GPU shows it purely. Four shader clusters are four instances of the same RTL, and a tile-based architecture replicates the per-tile datapath inside each. One questionable construct in a shared file — an unused parameter, a width deliberately truncated in a rate-matching stage — is reported once per instance. An issue count of eleven thousand across the shader complex may be forty distinct constructs, so any policy phrased as a count (“drive lint issues below 500”) measures elaboration depth rather than code quality. The unit that means something is the distinct rule violation at its source line.

The second multiplier is inherited code. A block bought or reused arrives with its own conventions and its own history of accepted deviations, so your rule set produces thousands of findings nobody on your team is authorised to fix. That is not noise in the same sense: it is somebody else’s signal.

### Making the rule set a policy

Three disciplines turn lint from noise into a gate, and only the first is about the tool.

**Classify rules by severity before the first run, not after.** Three classes are enough: violations that block the commit, violations waivable by a named owner with

a written reason, and findings that are informational and never gate anything. The classification lives in a checked-in file and is a *policy* document — it states what this team believes constitutes a defect, exactly the kind of statement Chapter 5 wanted written down rather than assumed. The middle class is where the discipline is really tested, and the corpus holds a cautionary case. A Broadcom power-management controller, the case Section 13.6 returns to, had passed extensive block-level UVM constrained-random simulation and SoC-level directed tests, and its lint analysis had been run with the errors and warnings waived by the designer; the block nonetheless still held an unreset flip-flop, which a formal run found before tape-out [5]. Nothing in that account says lint would have caught the flip-flop; it is not a lint result. What it does say is narrower and more useful: a waiver recorded by the code’s own author is a decision no later reader can audit — which is why the class above insists the owner be *named* and the reason *written*.

**Filter the output; never disable the check.** Suppressing a check in the source or on the command line looks equivalent and is not: it stays suppressed for an unpredictable length of time and hides problems that arrive with later files, whereas a filtered report still contains the finding and the filter itself can be audited. Make naming conventions carry the filtering, so it is mechanical rather than a judgement call — if intentional latches are the only signals whose names end in a reserved suffix, an inferred-latch report on such a signal is expected and every other one is a defect [2]. This is Chapter 7’s rule about waiver files, one level down.

**Lint as the code is written.** A first run on a finished block produces a report large enough to feel like a setback, arriving when its author has already moved on. Run it during writing and each finding is judged by the person still holding the context [2].

**Scenario.** The GPU team inherits a shader complex from a previous project and its lint report has 11,000 findings. **Approach.** Do not attempt closure. Take one snapshot of the report as a baseline, mark every finding in it as inherited, and gate only on the *delta*: a commit fails if it introduces a finding not in the baseline. In parallel, classify by rule rather than by instance, and pick the two or three rule classes with real failure modes behind them — inferred latches, `casex` on a decoded selector — to be retired against the baseline over the next quarter. **Why.** The gate’s job is to stop new debt, and the delta gate does that on day one at zero cleanup cost. Absolute closure on inherited code is a project in its own right and must be planned as one, with an owner and a schedule, not smuggled in as a quality initiative. Retiring rule *classes* rather than instances also means each cleanup commit has a stated failure mode behind it, which is what makes it reviewable.

### 13.3 Clock-domain crossing

A clock-domain crossing is a signal generated by logic on one clock and sampled by logic on another, where the two clocks have no fixed phase relationship. The fundamental problem is metastability. When the signal changes close enough to the destination edge to violate that flip-flop’s setup or hold time, the flop enters an unstable

state; it will resolve to a one or a zero after some delay, but which one cannot be predicted [6].

Two consequences are worth separating, because teams routinely conflate them. First, metastability itself is not observable in RTL simulation, in gate-level simulation, or in formal analysis: it is an attribute of a physical flip-flop, not of any model of one. What it produces in the real device is a family of *effects* — uncertainty about when a value changes, filtering of a pulse that simulation showed arriving, generation of a pulse that simulation showed being filtered, and several distinct timing scenarios where synchronized paths reconverge. Second, it cannot be eliminated, only given time to decay; that is the entire function of a synchronizer, which isolates the potentially metastable net so its unresolved state never reaches the destination domain [6].

### How long is long enough

The probability that a metastable condition has *not* decayed to a stable legal value in time for the next sampling event sets the failure rate, and through it the mean time between failures given by

$$\text{MTBF} = e^{(T_s/T_c)} / (T_w \cdot F_c \cdot F_d)$$

where  $T_s$  is the settling window available before the next sample,  $T_c$  the flip-flop's settling time constant,  $T_w$  the window of susceptibility around the destination edge,  $F_c$  the destination clock frequency and  $F_d$  the rate at which the crossing signal changes [6].

Read the shape, not the numbers. Two terms sit in the exponent — the settling window  $T_s$ , and  $T_c$  as its divisor — while only  $T_w$ ,  $F_c$  and  $F_d$  are linear. The first exponential lever is the time between the first flop's output settling and the second flop sampling it, which is why a two-flop synchronizer works at all, and why anything that erodes that window (a slow route, a scan multiplexer on the second flop's data path, clock skew that samples the second flop early) hurts far out of proportion to its apparent size. The second is the choice of flop: a metastability-hardened cell in the first stage damps a metastable condition faster — a smaller  $T_c$  — and narrows the susceptibility window as well. Note also what the expression does *not* say: it does not predict how often a metastable condition occurs. Those are relatively frequent; the failure — an unresolved value sampled and propagated onward — is catastrophic and extremely rare [6]. A design does not fail because metastability happened. It fails because settling did not finish in the window available, or because there was no synchronizer at all.

### The structures, and what each one assumes

There are four common answers, and each is a bargain with a different set of preconditions.

The **two-flop synchronizer** is the fundamental element for a single asynchronous signal and the building block of everything more complex. Three of its requirements

can be checked on the RTL: there must be no combinational logic on the crossing path that could be falsely sampled, the crossing signal must never carry a pulse too narrow for the destination to sample, and the data must be stable across more than two active destination edges so that nothing is filtered away <sup>[6]</sup>.

The **gray-coded parallel synchronizer** — one two-flop synchronizer per bit — extends this to a bus, and its precondition is absolute. It is safe only if exactly one bit changes at a time, because then at most one bit can go metastable and it resolves either to the old value or the new one, both legal codes. Give it a binary bus and it generates codes the source was not driving — illegal ones, and legal ones from the wrong moment <sup>[6]</sup>. That is the opening scenario stated as a rule: the reference SoC's event unit put one synchronizer per bit on a code where two bits change at once. Note where that leaves the setup file. Had anyone declared that cause bus gray-coded <sup>[3]</sup>, the structural check would have accepted one synchronizer per bit and reported nothing — the opening scenario passing a clean CDC run on the strength of one wrong line nobody reviewed.

Fixing metastability and fixing data coherency are independent problems. For one bit, fixing metastability is enough: the coin can land either way and both faces are legal. For several bits you must fix both, because now you need all the coins to land the same way <sup>[3]</sup>.

The **handshake synchronizer** solves the multi-bit case without constraining the encoding, by not sending the data through a synchronizer at all. Only the request and acknowledge controls are synchronized; the data bus is held stable in the source domain until the acknowledge comes back, and the destination captures it while it is guaranteed still <sup>[6]</sup>. It costs several round-trip latencies per transfer, which is why it is the answer for configuration and status rather than for streaming.

The **asynchronous FIFO** is the streaming answer: gray-coded read and write pointers crossing in both directions, with the payload passing through dual-ported memory rather than through any synchronizer. It buys bandwidth, and it buys throttling — the ability to absorb intermittent peaks in the arrival rate rather than stalling the source on every item <sup>[3]</sup>.

### Grounding: the reference SoC's three domains

Which structure you owe depends on the frequency ratio, and the reference SoC's ratios are extreme enough to make the reasoning visible.

Between the always-on RTC domain at 32 kHz and the system domain at 200 MHz the ratio is 6,250 to one: one 32 kHz period is 31.25  $\mu$ s, one system period is 5 ns. Going *up*, from slow to fast, a level generated on the RTC clock is stable for 6,250 system cycles, so no pulse can be filtered and a two-flop synchronizer per bit is sufficient — provided the bits do not have to agree with each other. Going *down*, from fast to slow, the pulse-width requirement bites: a system-domain pulse must be wider than two destination periods <sup>[7]</sup> — 62.5  $\mu$ s, 12,500 system cycles. No designer writes that, and nobody should: downward crossings on this SoC are handshakes or toggles, and a level pulse in that direction is a defect regardless of what the simulation shows.

The 200 MHz to 50 MHz boundary is milder — a four-to-one ratio, so pulse width is rarely the issue in either direction — but it is the widest surface, since every APB register interface sits on it. There the recurring question is coherency: a multi-bit status field that firmware polls must not be readable in a state it never held.

**Scenario.** The sensor hub — an always-on fusion block with a 32 kHz island and a DSP that is clock-gated off between bursts — must pass a configuration word from the island to the DSP. **Approach.** Handshake, with the request generated by the island and the acknowledge required before the word may change. Not a gray-coded parallel synchronizer, and not a pulse. **Why.** The “more than two active destination edges” rule is a duration only when the destination clock is running. Here it is not: the DSP clock is stopped for most of the time the island is awake, so the interval a level would have to survive is unbounded and cannot be written as a number of source cycles. A handshake’s correctness argument never mentions the destination frequency: the source holds the data until the destination tells it, on its own schedule, that it has taken it.

## 13.4 Structural checks and protocol checks

A team that believes it has done CDC verification because the structural report is clean has done half of it.

The structural check analyses the elaborated design: it identifies the clock domains, finds every signal crossing between them, flags illegal logic on those paths, and classifies each crossing into a synchronizer protocol group — an individual asynchronous bit, a handshake, an asynchronous FIFO — from which the properties that crossing owes follow <sup>[6]</sup>. What it decides is *structure*: whether a recognised synchronizer stands where one is required. It does not analyse functionality, so as long as the structure is valid the check passes <sup>[3]</sup>. A valid crossing is correct structure *and* correct function, and the second half needs a different instrument.

That instrument is an assertion (see Chapter 11), evaluated by a simulator or by a proof engine. The properties are the preconditions of the structure the classifier found: minimum pulse width and data stability for a two-flop crossing, one-bit-at-a-time for a gray-coded bus, request-acknowledge sequencing and source-side data stability for a handshake. The standard states the obligation for the simplest case

directly — a pulse driving a synchronizer input must be wide enough to be reliably sampled by the destination clock, and that requirement is to be discharged by formal or dynamic verification [7].

### Worked example: the pulse-width property, twice

The obvious way to write “the pulse is wide enough” is to sample the source signal on the destination clock and require it to hold:

```
// Wrong: samples the crossing signal on the clock that cannot see the problem.
property p_min_pulse_wrong;
 @(posedge dst_clk) $changed(src_sig) | => $stable(src_sig)[*2];
endproperty
```

Read it as the tool will. `$changed` is true at a destination edge when the sampled value differs from the previous sampled value; the non-overlapping implication then requires the value to be unchanged at each of the next two destination edges. So the property says: whenever the destination *notices* a change, the new value must survive two further destination samples.

The flaw is in the noticing. A source pulse narrower than a destination period can fall entirely between two destination edges. `$changed` is then never true, the antecedent never fires, the property passes having checked nothing — and the assertion reports a clean run for exactly the crossing it was written to protect, while silicon, where the sampling phase is not the one this simulation happened to use, may catch the pulse and go metastable [3].

The fix is to stop sampling on the clock that is blind to the failure and sample on the fastest clock in the system, scaling the stability requirement so it still expresses two destination periods:

```
// Right: sampled fast enough to see a pulse the destination could miss.
// N = ceil(2 * dst_clk period / fast_clk period)
module cdc_pulse_chk #(parameter int N = 2) (
 input logic fast_clk,
 input logic rst_n,
 input logic src_sig
);
 property p_min_pulse;
 @(posedge fast_clk) disable iff (!rst_n)
 $changed(src_sig) | => $stable(src_sig)[*N];
 endproperty
 a_min_pulse : assert property (p_min_pulse);
 c_min_pulse : cover property (@(posedge fast_clk) disable iff (!rst_n)
 $changed(src_sig));
endmodule
```

`N` is the number of fast-clock ticks that span two destination periods, rounded up [3]. Compute it for the downward crossing Section 13.3 said nobody should build — 200 MHz system clock as the fast clock, 32 kHz RTC as the destination — and `N` is 12,500. That is the point of computing it: the property states in one number the cost the level-pulse design was hiding, which is why the answer in that direction is a



handshake. If no clock in the design is fast enough, the property is bound to a virtual clock generated for the purpose.

A CDC protocol property is a claim about *time*, so the clock you sample it on is part of the claim and not an implementation detail. And the wrong version fails in the way Chapter 11 warned about: it passes vacuously, and only the cover on its antecedent would have exposed it.

### CDC reconvergence: signals that recombine after crossing

The hardest CDC question is not whether each crossing is synchronized but whether the design works *in the presence of* the synchronizers — including every clock relationship, the timing uncertainty each synchronizer introduces, and paths that converge after passing through more than one of them [6].

Two failure shapes live there. The first is loss of data coherency: several bits crossing through independent synchronizers, resolving independently, and recombining into a value the source was not driving. That is the opening scenario, and a gray code or a handshake eliminates it by construction. The second is glitching: several separately synchronized signals reconverging into combinational logic, where a one-cycle difference in when each arrives produces a momentary transition on the result that neither source ever intended [3].

The second shape is the one that survives review, because each crossing is individually correct. Suppose the peripheral domain raises two status flags, `fifo_empty` and `xfer_done`, each properly synchronized into the system domain, and a system-domain state machine advances on `fifo_empty && xfer_done`. Both flags are legal at all times in the source. But the two synchronizers resolve independently, so the conjunction can be asserted for one system cycle at a moment when the source never had both true — or, worse, can be *missed* for one cycle at a moment when the source did. Nothing in either crossing is structurally wrong. The bug is in the CDC reconvergence, and the cure is architectural: synchronize one signal and qualify the other with it, or carry both across a single handshake, so that only one crossing decision is ever made.

### Modelling the uncertainty

A synchronizer's output can legitimately arrive one cycle early, one cycle late, or on time, so the receiving logic must tolerate all three — and conventional simulation, which reproduces exactly one of them, cannot test that tolerance. The technique that closes the gap is metastability injection: instrumenting each crossing with a model that randomly advances, delays or passes through the sampled value.

Four criteria, set out in 2012, separate a trustworthy injection model from a convenient one, and the cheap models fail them. The delay must be *random* between the three legitimate outcomes — early, late, on time. Injection must be *independent* per receiving register, since two registers sampling the same value can resolve differently, which a model that merely jitters a primary clock cannot express. It must be *accurate*, injecting only when the data changes, is sampled, and violates setup or

hold; injecting at random moments manufactures failures silicon cannot produce. And it must be *complete*, covering registers with and without synchronizers, because a crossing can go metastable whether or not anyone synchronized it. Violate them and the model invents behaviour: replacing every two-flop synchronizer with a three-flop cell that randomly advances or delays can generate a sequence impossible in silicon, values skewed by two cycles [8]. The accurate models are slow, which is why this runs on an emulator rather than in the nightly regression (see Chapter 17).

## 13.5 Reset-domain crossing

A flip-flop with an asynchronous reset has a third timing requirement beside setup and hold: recovery time, the interval between reset release and the next clock edge. Violating it is no different in kind from violating setup or hold — the flop can go metastable [3]. That observation splits into two problems, and only the second is what the industry means by RDC.

The first is **reset de-assertion**. A reset generated in one place and released asynchronously with respect to a destination clock will, sooner or later, be released inside that flop's recovery window. The standard cure is the reset synchronizer: assert asynchronously so the design is safe the instant reset arrives, release synchronously so no flop sees the release inside its window. Whether a given reset has one is a purely structural question, which is why static analysis is sufficient to answer it [3].

The second is **reset assertion**, and this is reset-domain crossing proper. A register whose reset is asserted asynchronously clears its output at a moment bearing no relation to any clock edge — the path from the clear input to the output is not normally timing-closed. If another register, *in a different reset domain and therefore not being reset*, samples that output, it samples a signal changing at an arbitrary time and can go metastable [3].

Here is why RDC is a discipline of its own rather than a footnote to CDC. **The domains in question are reset domains, not clock domains.** Both registers above can be driven by the same clock. A design with exactly one clock, and therefore not one clock-domain crossing anywhere in it, can be full of reset-domain crossings — and a clean CDC report says nothing whatsoever about them. The discipline arrived later than CDC for the reason it became necessary: in practice resets multiplied, as power gating gave each domain its own, per-IP soft resets became a standard firmware affordance, and debug and watchdog resets were added on top. The cross-industry standardisation effort followed: its first draft covered CDC alone, and RDC entered the scope at the very next release, nine months later [3].

**Scenario.** The sensor hub's always-on island runs entirely on the 32 kHz clock. It has two resets: a power-on reset covering the whole island, and a soft reset that firmware pulses to restart the fusion pipeline after a sensor is reconfigured. The island's own CDC report is empty — internally one clock, no crossings. In the lab, reconfiguring a sensor occasionally produces a spurious wake interrupt. **Approach.** Run RDC analysis with the two resets declared as separate domains. Every path

from a fusion-pipeline register into the wake-decision logic is reported: the pipeline sits in the soft-reset domain, the wake decision in the power-on domain, and the wake-decision registers are live and clocking while the soft reset asserts.

**Why.** When firmware pulses the soft reset, the pipeline's `sample_valid` output clears asynchronously, at an instant unrelated to the 32 kHz edge. The wake-decision register samples exactly then and can resolve either way — or resolve late. Neither register is wrong, both resets are correct, and there is no clock-domain crossing to find. This is precisely the class CDC analysis is not looking for.

Three fixes exist, and choosing between them is a design decision rather than a verification one.

**Order the assertions.** If the destination reset is always already asserted when the source reset asserts, the destination register is sampling nothing and the crossing is harmless. That turns a structural obligation into a sequencing one, which is why the integration standard — the Accellera *Standard for IP Abstraction for Clock and Reset Domain Crossing Integration*, Version 1.0, 2026 — carries a reset assertion sequence — the order in which multiple resets shall be asserted — alongside a reset group, a set of resets between which no RDC violation exists at all [7].

**Qualify the path.** Insert a control that blocks the crossing path while the source reset is active, so the destination samples a held value rather than a changing one. The load-bearing condition is that the qualifying control must itself be synchronous with the destination domain [3] — a qualifier that arrives asynchronously has moved the problem rather than solved it.

**Synchronize the source reset to the destination clock.** Then the source register's output changes only on a clock edge and the crossing becomes an ordinary synchronous path. On the sensor hub's always-on island this is the cheapest of the three, because the 32 kHz clock never stops.

The ordering fix is checkable as an assertion, and worth writing even when the reset controller looks obviously correct:

```
module rdc_reset_order_chk (
 input logic clk,
 input logic rst_pipe_n, // fusion pipeline soft reset, active low
 input logic rst_por_n // always-on island power-on reset, active low
);
 // The pipeline reset may only assert while the island reset is already
 // asserted. `===` states the intent on its face; `==` would behave
 // identically here, since a property reads an unknown expression as false.
 property p_reset_order;
 @(posedge clk) $fell(rst_pipe_n) |-> (rst_por_n === 1'b0);
 endproperty
 a_reset_order : assert property (p_reset_order);
 // No `disable iff` here either, for the same reason.
 c_reset_order : cover property (@(posedge clk) $fell(rst_pipe_n));
endmodule
```

Three details are deliberate. The implication is overlapping: at the very tick where `rst_pipe_n` is first seen low, `rst_por_n` must *already* be low — write `|=>` instead and

the property tolerates the pipeline reset leading the island reset by a cycle, which is the exact ordering it exists to forbid. There is no `disable iff`, because a reset-ordering property that switches itself off during reset has nothing left to check. And sampling on `clk` makes this a sampled approximation of an asynchronous event; the same obligation is also written against the source reset's own falling edge [3].

At integration this becomes somebody else's problem. A team instantiating a purchased block cannot see its internal reset structure and should not have to; what it needs is a declaration — which ports carry a reset-domain control, with what blocking polarity, which resets share a group, in what order they must assert. Historically each vendor's tool produced that collateral in its own format and the models were not interchangeable [3]. A cross-vendor abstraction format for CDC and RDC collateral reached version 1.0 in March 2026, covering those attributes and requiring conforming tools to read and write it [7]. Who agreed it matters as much as what it says. The working group that produced it grew to 154 members from 24 companies — Arm, AMD, Apple, Google, Huawei, IBM, Infineon, Intel, Marvell, Microsoft, NVIDIA, NXP, Qualcomm, Renesas, Robert Bosch, Samsung and ST Microelectronics among them, alongside the major EDA vendors — over the three years between the working group's launch in January 2023 and the final release, with a first CDC draft appearing that October [3]. The standard's own account of why states it plainly: it addresses “aspects of clock domain crossing and reset domain crossing that are outside the expressiveness of typical HDL semantics” [7]. Read that as a participation fact, not a results fact: the industry judged the problem real and shared enough to standardize an answer, which says nothing about how well any tool implements it.

## 13.6 X-propagation

SystemVerilog carries four values: 0, 1, the unknown `x` and the high-impedance `z` [4]. Silicon carries two. Every simulation is therefore a translation between a four-valued model and a two-valued device, and it is lossy in both directions at once. This is the last of the entry-gate checks and the least well understood, because its failure mode is not a wrong answer but a *right-looking* one.

### The two lies

**X-optimism** — defined in Chapter 12 as simulation propagating a definite 0 or 1 where silicon would hold an unknown — follows from the language's own semantics, not from a simulator defect. An `if` takes the true branch only when its condition is a nonzero *known* value; the false branch is taken when the condition is 0, `x` or `z`. A condition nobody can predict therefore produces a decision nobody chose, and the run continues as though it had been made deliberately. `case`, `posedge` and `negedge` behave the same way [5]. Chapter 12 noted the flip side — asynchronous reset in RTL simulation works *because* of this — which is why the answer is never “make the simulator pessimistic everywhere”.

**X-pessimism** is the opposite error and the more visible one. With `a` unknown, both `a & ~a` and `a | ~a` evaluate to `x` in simulation, where silicon gives a definite 0 and

a definite 1. Worse, the arithmetic operators are all-or-nothing: if any operand bit is unknown the whole result is unknown, so `3'b000 + 3'b01x` is `3'bxxx` where `3'b01x` would have been accurate. Pessimism does not usually hide design bugs; it floods the waveform with unknowns that are expensive to trace and mostly artefacts [5].

Put together: an RTL simulation can pass a test silicon fails and fail a test silicon passes, for reasons that have nothing to do with the design.

### Where the unknowns come from

Four sources account for most of it: registers with no reset, out-of-bound array or bit-select accesses, explicit `x` assignments written as don't-cares or as error indications, and the corruption of a powered-down domain under a power-intent description [5]. The last belongs to Chapter 19. The first two are where the entry gate earns its place, because both are findable statically.

**Scenario.** The video codec's deblocking filter indexes a per-row descriptor array sized for the tile geometry of the profiles it originally shipped with. A new profile adds one row class; at the largest frame size the index reaches one entry past the end, for the last row of the frame only. The out-of-bound read returns `x`, and the filter-mode case that consumes it falls through to its default branch. The regression passes: frames complete, the checksum matches on every clip in the suite except the ones nobody added for the new profile. **Approach.** Before writing a single new test, run a structural property analysis for out-of-bound indexing across the block. It needs no testbench, only compilable RTL, and it reports the index and the condition under which it exceeds the array. **Why.** The alternative is to find it the way it has been found before: in silicon. A 2014 case study describes exactly this shape — an audio block extended from sixteen channels to seventeen, an index one past the end of a request vector, an unknown that X-optimism converted into a state machine that always returned to idle, a full passing regression, and a system that hung in the lab whenever the new channel was used [5].

**Scenario.** The crypto engine's key-valid flag is a control-register bit the designer left unreset — a common and usually harmless economy. The negative test that matters most for this block, "no ciphertext is produced before a key is loaded", passes on every seed. **Approach.** Run formal reset analysis before believing the test. It takes the reset vector and reports every register still unknown when the reset period ends, which is where this flag will appear. **Why.** In simulation the flag powers up unknown, so `if (key_valid)` takes the else branch and the engine refuses to encrypt — the test passes because of X-optimism, not because of the design. In silicon the flop powers up to a definite value, which on some corners and some dice is one, and the engine encrypts with whatever the key registers happen to hold. The cure is not a better test but a reset: control, clock and reset registers in particular should be reset, because the area, routing or timing gained by leaving them alone does not pay for the class of bug it hides [5].

### The three answers, and what each costs

**Gate level.** X-optimism largely disappears there, because `if`, `case` and the edge constructs have by then been synthesized into gates and flip-flop primitives that do not exhibit it. The cost is not only runtime: X-pessimism becomes *more* prevalent at gate level, and the resulting sea of unknowns has to be cleared by depositing or forcing known values, iteration after iteration, which is what makes the productivity poor rather than merely slow [5]. Chapter 12 gave the regression-level consequence — only a minority of the suite is ever regressed at gate level — so this answer arrives late and covers little.

**A simulator's X-propagation mode.** Simulators can be told to merge the results of both 0 and 1 in place of an unknown when executing `if`, `case` and the edge constructs, which converts optimism into an honest unknown at the point of decision [5]. It costs simulation performance, so in practice it is a named regression variant rather than a default — one 2026 project lists an `xprop` variant beside its DMA and JTAG variants in the same UVM regression environment [9].

**Formal X-propagation.** A formal application instruments the design with assertions asking whether an unknown can reach a chosen target — clocks and resets, primary outputs, the conditions of `if` and `case` statements — and proves or refutes them. Run blindly it hits the ordinary capacity wall: given a whole IP and a two-hour budget, the same application returned bounded proofs and no violations for a design already known to contain an X-optimism bug [5]. That is the failure's shape: not a wrong answer, an empty one.

What makes it work is attacking the sources rather than the propagation. Formal reset analysis takes the reset vector and reports which registers are still unknown when the reset period ends, catching unreset flops no structural check can see — a polarity mistake, or a clock not running while a synchronous reset asserts. Structural property analysis finds out-of-bound indexing far more cheaply than the propagation proof does. Only then, with every source found and judged intended or not, is the proof scoped to the module where a given source is produced and consumed, small enough to converge [5]. That is the shape of the case behind this section. On a Broadcom power-management controller for a quad-core application processor, formal reset analysis reported four flip-flops still unknown in the first clock cycle after reset deassertion, and every assertion failure the X-propagation app then produced traced back to X-optimism on just one of them — the very flip-flop the team had already suspected [5].



---

## IN PRACTICE

Static verification decays quietly, and it decays through its setup rather than through its checks. The CDC setup file accumulates declarations — this input is constant, that bus is gray-coded, this crossing is a false path — each true of the design on the day it was written, and nobody re-derives them. Six months later “the CDC report is clean” means “clean under assumptions last examined by someone who has left the project”. The only defence is to review the setup file as seriously as the RTL, at the same milestones.

Ownership is usually split. Designers run lint and own its findings, because the findings are about their code and cheapest to judge on the day of writing. The CDC and RDC setup belongs to an integration or methodology team, because it is a statement about the whole chip’s clocking and reset architecture that no block author can make. The verification engineer’s job is neither: it is to make sure the results are evidence — the waiver file reviewed rather than counted (see Chapter 7), the protocol assertions having actually run with non-vacuous attempts (see Chapter 11), the gate containing what the plan says it contains.

Hierarchy is the other practical reality. A flat structural check on a full SoC is expensive and produces a report dominated by crossings into blocks that are still black boxes. The reference SoC’s three domains give three boundaries to classify by hand; the flagship’s five give ten. The industrial flow is two-level: each block is closed at its own level and exports an abstraction model of its boundary alone — clocks, resets, which ports are synchronized inside — and the SoC run consumes those models instead of the RTL <sup>[3]</sup>. That is far better than an opaque box, and it is where the collateral must be trusted: a model generated under one set of parameters and configuration signals is valid only for that set, so a block instantiated twice in different modes needs two models — and on the flagship, whose DVFS moves clock ratios between operating points, one model per operating point.

The honest ordering, then. Lint every commit; structural CDC and RDC every commit once the setup is stable, and painfully before that; protocol assertions in the nightly and weekend tiers; metastability injection on the emulator. The formal X-propagation proof is a campaign — launched for a stated reason, scoped to a module, producing a written result — not a gate.

---

## PITFALLS

1. **A setup file that makes the check pass.** Symptom: a suspiciously short report; or hundreds of crossings on a boundary that is synchronous; or a report that shrank and nobody can say which crossings left it. Cure: write the CDC setup separately from the timing constraints, which answer a different question <sup>[3]</sup>, and treat a CDC false path as the strongest claim in the file — *no crossing exists here* — with a named owner and a written argument, exactly as for a coverage exclusion.

2. **Declaring CDC done when the structural report is clean.** Symptom: every crossing has a recognised synchronizer and the design still fails in the lab. Cure: run the protocol properties too (Section 13.4) — they are the other half.
3. **A crossing property sampled on the destination clock.** Symptom: a pulse-width assertion that has never fired on any seed. Cure: sample on the fastest clock in the system, or a virtual one, and scale the stability count accordingly.
4. **Reading a lint issue count as a quality metric.** Symptom: a number that moves when someone changes an instantiation count. Cure: count distinct rule violations at their source lines, and gate on the delta against a baseline for inherited code.
5. **Leaving control registers unreset because “it does not matter which value it starts at”.** Symptom: a test that passes for the wrong reason and a state machine that behaves differently in silicon. Cure: reset control, clock and reset registers; the area saved does not pay for the class of bug it hides [5].
6. **Treating RDC as covered by CDC.** Symptom: a single-clock block with a clean CDC report and intermittent failures around soft reset. Cure: run RDC with the reset domains declared, and check that the design’s resets have a defined assertion order.

---

## SUMMARY

- Static verification needs no stimulus and no expected output; it is exhaustive over structure and silent about function [2]. That is simultaneously why it goes first and why it can never be the whole story.
- A linter’s value is destroyed by its own false reports. Filter the output, never disable the check, encode intent in names so the filter can be mechanical, and run it while the code is being written [2].
- Metastability cannot be removed, only given time to decay; two terms of the MTBF relation are exponential — the settling window and the flop’s settling time constant — which is why anything that erodes the window matters far more than its size suggests [6].
- One bit needs metastability fixed; several bits need metastability *and* coherency fixed — which is why a gray code, a handshake or an asynchronous FIFO, and not three correct synchronizers, is the answer for a multi-bit crossing [3].
- The structural check asks whether a synchronizer is present; the protocol check asks whether its preconditions hold. Both are required, and the second is an assertion problem (see Chapter 11).
- RDC is not a special case of CDC: its domains are reset domains, so a design with a single clock and no clock-domain crossing at all can be full of reset-domain crossings [3].
- RTL simulation is optimistic about unknowns at every decision and pessimistic about them in every expression, so it can pass tests silicon fails and fail tests silicon passes [5]. Attack the sources — unreset registers, out-of-bound indices — before attacking the propagation.

## FURTHER READING

Within the corpus, [3] is the broadest single treatment of CDC and RDC together: synchronizer schemes, setup constraints and where they come from, structural analysis, the assertion layer with its clocking traps, and the hierarchical block-to-SoC flow. [7] is the normative document behind it — the attribute set for CDC and RDC collateral, the RDC clause covering reset groups, assertion sequences and blocking controls, and the SVA requirements for a closed-box crossing. For what happens to synchronizers *after* RTL sign-off — the sub-cycle constraint on the metastable net, why scan insertion on the second flip-flop erodes MTBF, how a parallel synchronizer fails when its bits are under-constrained — [6] is the reference, and the best corrective to the belief that CDC ends at RTL. [8] catalogues metastability injection models. On unknowns, [5] carries the definitions, the four X sources, the X-source-driven scoping and two industrial case studies. [2] remains the clearest short account of what linting is and is not, and [4] is definitive on the four-valued semantics that make the X problem exist, on `casex` and `casez`, and on the checks the language asks tools to perform for `always_comb` and `always_ff`. The 2024 industry data on asynchronous clock domains is in [1].

Outside the corpus: C. Cummings' SNUG papers on clock-domain-crossing design and on asynchronous FIFO design are the long-standing practitioner references for synchronizer and FIFO construction; M. Litterick, *Pragmatic Simulation-Based Verification of Clock Domain Crossing Signals and Jitter using SystemVerilog Assertions*, is the simulation-side companion to [6]; and M. Turpin, *The Dangers of Living with an X* (SNUG Boston, 2003) with S. Sutherland, *I'm Still In Love With My X!* (DVCon, 2013) are the two short papers to read next on X semantics and coding style.

## REFERENCES

1. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
2. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
3. **D36** D. Mills, B. Gascoyne, C. Choppali Sudarshan, D. Olopade, and D. K. Gupta, "Breakthrough in CDC-RDC Verification Defining a Standard for Interoperable Abstract Model," Proc. DVCon U.S., 2026.
4. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
5. **D23** S. Zhao, S. Yan, and Y. Feng, "Practical Approach Using a Formal App to Detect X-Optimism-Related RTL Bugs," Proc. DVCon U.S., 2014.
6. **D14** M. Litterick, "Full Flow Clock Domain Crossing - From Source to Si," Proc. DVCon U.S., 2016.
7. **S12** Accellera Systems Initiative, "Standard for IP Abstraction for Clock and Reset Domain Crossing Integration, Version 1.0," Accellera, March 2026.
8. **D34** A. Hari, S. Krishnamurthy, A. Jain, and Y. Badaya, "Verification of Clock Domain Crossing Jitter and Metastability Tolerance using Emulation," Proc. DVCon U.S., 2012.
9. **P25** S. Ahmed, A. Kropotov, R. I. Genovese, B. Homs, E. Merino, F. Urbani, et al., "Verification and Validation (V&V)-in-the-Loop for RISC-V Design: The Holistic Vision of BZL," arXiv preprint arXiv:2604.27013v1, April 2026.

---

# 14. Formal Property Verification

Six weeks of nightly regression on the reference SoC's crossbar, and every night is green. The bound protocol checker from Chapter 11 sits on all nine ports, its assertions have never fired, and the block is a week from its sign-off review. One number in the report should have stopped the review, and nobody has looked at it: on the four subordinate-side instances, the cover `c_w_overlap` — a second write address accepted at a subordinate while an earlier burst is still unfinished on the write-data channel — reads zero. Chapter 11 identified that cover as the precondition of exactly one hazard, and in six weeks no test has produced it.

An engineer takes that same checker file, its properties unchanged, and points a proof engine at one subordinate port instead of a simulator. Overnight the tool returns a counterexample eleven cycles long: two managers have writes in flight to the same subordinate, the subordinate back-pressures the write-data channel, and the crossbar interleaves their data beats into one stream. Beats from two bursts get counted as one, and the count no longer matches the length promised by the address at the head of the queue. It is a real bug, it is the canonical crossbar bug of this book, and no test in six weeks came within reach of it.

Nothing about the property changed. The question did. A simulator asks *did any run I performed break this rule?* A proof engine asks *can anything break this rule?* — and on blocks of the right size the second question is answerable in a night.

The same run returned three other things: two properties **proven**, one **undetermined**, and one proven only because an assumption nobody had reviewed forbade the very traffic that would have broken it. Those four results — falsified, proven, undetermined, and the green tick that is not one — are the chapter. The last is the one that will hurt you.

---

## CHAPTER MAP

- §14.1 states what a proof establishes, and what it is *over*: a model, a set of assumptions, sometimes a depth — three qualifiers, each of which can quietly empty the claim.
- §14.2 shows why an over-constrained formal run is the most dangerous green result in verification, and gives the discipline that keeps assumptions honest.
- §14.3 develops bounded versus unbounded proof — bug hunting by unrolling, proof by induction — and how to read a depth number instead of ignoring it.
- §14.4 treats abstraction: what is thrown away to make a proof converge, which reductions are sound, and how to avoid throwing away the bug; §14.5 sets out what to do with an undetermined property, in the order a practitioner actually does it.

- §14.6 states what a proof is evidence of at sign-off, what coverage can mean when there is no stimulus, and when formal is simply the wrong instrument.

## 14.1 What a proof is, and what it is over

---

Strip the tool away and a synchronous design is a finite transition system: a set of state variables, a set of initial states, and a relation saying which states can follow which. Model checking builds that finite model and checks whether the desired properties hold in it. The states reachable from the designated initial states are computed by fixed-point analysis — the *reachability analysis* — the property is checked against that set, and an error trace is produced if it fails. It is, in essence, an exhaustive state-space search which, because the state space is finite, is guaranteed to terminate <sup>[1]</sup>. Formal verification aims to establish that a design works correctly for **all** allowed inputs and **all** reachable states, by mathematical reasoning rather than by sampling <sup>[2]</sup>.

The whole value proposition is in the gap between two sentences. A green regression says: *among the traces I generated, none failed*. A proof says: *no trace fails*. The first is a statement about your testbench; the second is about your design.

**Four outcomes, not two.** Complete methods report that a property passes or fails. Capacity limits make incomplete methods common too, and those have three outcomes — passed, failed, or unknown; even a complete method reports unknown when it times out or exhausts memory <sup>[3]</sup>. So a formal report has a vocabulary a regression report does not:

- **Proven** — no reachable state and no legal input sequence violates the property. Full stop, subject to the three qualifiers below.
- **Falsified** — the tool produces a *counterexample* (CEX): a sequence of input stimuli leading to the failure, shown with the relevant internal signals and often with explicit don't-care values where the trace does not care what an input did <sup>[3]</sup>. This is the everyday payoff — a trace of incorrect behaviour, readable much like simulation output, that lets you identify the cause <sup>[2]</sup> — and unlike a regression failure it arrives with no irrelevant history attached, because the engine had no reason to generate any.
- **Bounded proof** — no violation up to  $n$  cycles. A statement about the near future only; §14.3 is about reading that number.
- **Undetermined** (inconclusive, unknown) — the tool gave up. §14.5 is about what to do next.

A `cover` behaves symmetrically. If the scenario can be hit, the tool returns a **witness**: a stimulus sequence reaching the coverage point while satisfying every stated assumption. If it cannot, the tool reports the point **unreachable under the specified assumptions** — and it may also fail to hit it simply because it ran out of memory or time, which is a third answer wearing the second one’s clothes [3]. Chapter 11 made the point in one line; here is where it earns its keep: an empty cover in simulation means “no test did this”, an unreachable cover in formal means “nothing can do this”, and only the second is a statement about the design.

### The three qualifiers

A proof is not a statement about your chip. It is a statement about a model, under assumptions, sometimes to a depth. Each qualifier is a place the claim leaks.

**The model.** The engine reasons about the RTL as elaborated, not about silicon, and verification at a high level of abstraction does not prove that the lower-level details are correct [2]. A proof over RTL says nothing about the synthesised netlist (Chapter 15’s equivalence checking closes that gap), nothing about timing, and nothing about the firmware that programs the block. It also says nothing about what you did not model: if a submodule was blackboxed or a memory replaced, the proof is over the replacement.

**The assumptions.** Verification generally requires assumptions about the environment the device operates in; if the real environment violates them, the device may fail despite successful verification. The redeeming feature — and it is a real one — is that formal methods force those assumptions to be written down [2]. §14.2 is entirely about this.

**The depth.** If the result is bounded, the property is guaranteed only within the bound [4].

Which is why the sentence “*the crossbar has been formally verified*” is not a claim at all. For such a claim to mean anything it must come with a description of what properties were verified and at what level of detail the design was modelled [2]. The useful form is longer and duller: *these five properties, on this RTL revision, under these five assumptions, proven unbounded except one which holds to depth 78*. That sentence is auditable — and it is auditable against this chapter, whose sign-off table and assumption register hold exactly those figures. The short one is a mood.

### The property Chapter 11 handed over

Chapter 11 ended with a property and a promise. Here is the property, unchanged:

```
property p_no_4kb_crossing;
 @(posedge clk) disable iff (!rst_n)
 (awvalid && awready && (awburst == 2'b01)) |->
 (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```



In simulation this is sampled: it is evaluated on the bursts the tests happened to emit, and it is silent about the ones they did not. Bound to the DMA backend’s write-address channel and handed to a proof engine, the identical text asks a different question — *is there any legal way to program this engine that makes it emit a crossing burst?* — and the engine answers by searching, not by sampling.

Now apply the three qualifiers, because they turn a green tick into a sentence you can defend at a review. The *model* is the midend legalizer and the AXI backend as elaborated — it contains the splitting arithmetic where the canonical bug lives, and it does not contain the register frontend if you blackboxed it. The *assumptions* are whatever you said about how descriptors arrive: assume a positive stride and you have deleted the negative-stride bug from the model rather than proved it absent. The *depth*, if the run was bounded, must be long enough for a descriptor to be programmed, legalized, split and emitted; a bound that never reaches the first accepted address proves the property over a stretch of silence.

## 14.2 Properties and assumptions

---

Constraints and assertions are two faces of the same coin. Both are formal, unambiguous statements about a design’s behaviour; assertions name the properties to be verified, constraints name the conditions required for a verification, and an assertion in one verification becomes a constraint in another <sup>[1]</sup>. Chapter 9 called the same object a generator, Chapter 11 called it an assertion, and here it is an assumption — one artifact, three hats.

The hats are not interchangeable. In simulation an `assume` is checked exactly like an assertion; in formal it is never checked, and instead deletes traces from the model, the assertions being proved over what survives (see Chapter 11). That difference is the whole of this section.

### Why green is the dangerous colour

Write an assumption stronger than reality and the engine will faithfully prove things about a design that does not exist. The assertion holds trivially, or vacuously, and the situation is dangerous because we may believe everything is checked when essentially nothing is verified <sup>[3]</sup>. The assertions here are SystemVerilog’s, specified in IEEE Std 1800-2023, *IEEE Standard for SystemVerilog* <sup>[5]</sup>; the vacuous evaluation is defined at IEEE 1800-2023 §16.14.8 “Nonvacuous evaluations”, which reads such a success as not matching the user intent and therefore as neither a pass nor a failure. From the constraint-solving side the verdict is the same: over-constraints stall progress in simulation and lead to vacuous proofs in formal verification <sup>[1]</sup>.

Note the asymmetry that makes this the chapter’s central hazard. A *missing* assumption lets the model do things the real environment never would, so the engine finds a violation that cannot happen — a spurious counterexample. Irritating, and it costs you an afternoon, but it announces itself in red. An *extra* assumption produces nothing to look at; the report is simply greener than before.

Over-constraint comes in two grades, and only one of them is loud.

**The loud grade: the empty model.** Add two assumptions that contradict each other — or one that contradicts the design’s own behaviour, such as requiring a signal to hold stable when an `always` block toggles it every cycle — and the constrained model contains no states at all. The intuition says every assertion should fail; the opposite happens, and all of them pass vacuously, because the proof’s hypothesis is that all assumptions hold and that hypothesis is itself false. Many tools detect basic cases of this, and the cheap prophylactic is a rule you can apply by eye: **every assumption should have a design input or a free variable in its support**, otherwise it is likely to collide with the design’s behaviour rather than constrain its environment [3].

**The quiet grade: the satisfiable narrowing.** This is the one that reaches sign-off. The assumptions are consistent, the model is far from empty, most properties still prove for the right reasons — and one clause has removed the exact traffic the interesting property was written for. Chapter 9 named this shape on the stimulus side: over-constraining whose quiet form stays satisfiable and silently deletes the feature under test. Formal makes it worse in one specific way. In simulation, deleting a scenario shows up as a coverage hole, because coverage is measured on what ran. In formal there is no stimulus to measure, so the deletion leaves no trace anywhere but in the assumption text. Overconstraining is usually not obvious and cannot always be discovered by automatic tools, which is why validating the assumptions is a task in its own right [3].

**Scenario (the crypto engine).** The inline AES-GCM engine on the memory path carries the two properties Chapter 11 wrote for it: a key may not be loaded while an operation is in flight, and ciphertext may not be presented under an unauthenticated key. Neither proof converges — the key ladder plus the GCM datapath is too much state — so an engineer encodes what the specification’s boot sequence says: key material arrives only after authentication has completed.

```
systemverilog m_auth_before_load : assume property (@(posedge clk) disable iff (!
rst_n) $rose(key_load) |-> auth_done);
```

Both properties now prove in minutes. **Approach.** Delete the assumption and re-run before believing either result. This one does not merely shrink the model: it deletes the entire class of traces in which key material enters the engine unauthenticated — the class `a_no_ct_unauth` exists to catch. The property was not so much proved as evacuated. **Why.** In a functional proof an assumption is a promise about a cooperative neighbour, and the worst case is a buggy neighbour. In a security proof the environment is an adversary who has read your assumption list, so every assumption is a concession, and each one owes an answer to *why can the attacker not do this?* Here there is none — an attacker able to drive `key_load` is the threat itself. The obligation belonged on the boot ROM’s proof, to be discharged, not on the engine’s, to be enjoyed.

## Discharging assumptions instead of trusting them

The mirror image of the hazard is also the cure. Because an assertion in one verification is a constraint in another, an assumption on a block’s inputs should be **proven as an assertion on whatever drives those inputs** [3]. That is the assume-guarantee paradigm, and its structural payoff is that properties of a system can be deduced from properties of its components without ever model-checking the complete model [2] — which matters, because the complete model is exactly the thing that will not converge.

On the crossbar this is mechanical rather than clever, and it reuses Chapter 11’s file — with one addition — rather than a new one. The five manager-side instances of the bound checker state the AXI manager rules the crossbar is entitled to expect from the core and the DMA. Those rules are AXI4’s, and the document that states them is the *AMBA® AXI and ACE Protocol Specification*, Arm IHI 0022H.c [6]. When you prove the crossbar, those same rules become its assumptions; when you prove the DMA engine, the same text is an obligation on the DMA’s own output. One file, two directives, selected by a parameter:

```
module xbar_port_checker #(
 parameter bit ASSUME_MODE = 1'b0, // 1 = constrain, 0 = check
 parameter int unsigned IDW = 4,
 parameter int unsigned AW = 32
) (
 input logic clk, rst_n,
 input logic awvalid, awready, wvalid, wready, wlast, bvalid, bready,
 input logic [IDW-1:0] awid, bid,
 input logic [AW-1:0] awaddr,
 input logic [7:0] awlen,
 input logic [2:0] awsize,
 input logic [1:0] awburst
);
 // Chapter 11's interface model and its other properties are omitted here;
 // the ASSUME_MODE parameter and the generate below are this chapter's
 // one addition to that file.
 property p_aw_stable;
 @(posedge clk) disable iff (!rst_n)
 (awvalid && !awready) | => awvalid &&
 $stable({awid, awaddr, awlen, awsize, awburst});
 endproperty

 if (ASSUME_MODE) begin : g_env
 m_aw_stable : assume property (p_aw_stable);
 end else begin : g_dut
 a_aw_stable : assert property (p_aw_stable);
 end
endmodule
```

The parameter is not decoration. It is the record of which side of the boundary you were standing on, and it makes “who guarantees this?” a question with a location rather than an opinion.

One thing in that file does have to change first, and it is not a property. Chapter 11’s interface model holds each accepted address in an unbounded queue — `logic [7:0] aw_len_q [$]` — which a simulator is happy with and a proof engine cannot encode:

§14.1’s guarantee that the search terminates was *because the state space is finite*. Cap the queue at the port’s outstanding-write capacity. That cap is not an assumption smuggled in at the door: it is a fact about the crossbar, proven on its own and handed back as a lemma (§14.5), and it reaches the sign-off package with a discharge like any other.

Sometimes you cannot discharge an assumption this way: the driving block may be too complex to prove, the signals may be driven from several blocks that would have to be pulled in together, or the driver may be third-party IP you have no model of. Then the assumptions should at least be checked in simulation — and checked again in simulation of the larger model the block is integrated into [3]. That is why the crossbar’s assumptions are also compiled into the SoC-level regression as ordinary assertions. An assumption that is nowhere checked, in any engine, is a comment.

Which gives the artifact this section is arguing for: an **assumption register**, reviewed like a waiver list (Chapter 7). One row per assumption, four columns — an ID, the text, who owns the behaviour it describes, and how it is discharged, including what breaks if it is wrong. §14.6 puts it in the package.

### 14.3 Bounded and unbounded proof

Two engines sit behind almost every result you will read, and they answer different questions. Knowing which one produced a green line is the difference between a guarantee and a very good night’s simulation.

**Bounded model checking** *unrolls* the sequential design: for each cycle of an interval of length  $k$  it makes a fresh copy of the combinational logic, each copy’s next-state outputs feeding the following copy’s state inputs, until the whole finite-time behaviour is one combinational circuit. That circuit and the negation of the property go to a satisfiability solver, which searches for a counterexample within the interval; if none is found you increase  $k$ , until a bug appears, the problem becomes intractable, or a preset upper bound is reached [4]. The one-line intuition: the effect is equivalent to an exhaustive simulation with the same bound on the number of cycles [1].

Two consequences follow. First, this is a *bug-hunting* engine: it is far more efficient at falsifying a property than at proving one, which is why bounded model checking is used mostly to find counterexamples rather than to establish correctness [4]. Second, a bounded run’s success is not a proof. Bounded methods check that there is no violation with a counterexample shorter than  $n$  cycles from an initial state — which need not be the reset state — so they do not guarantee that the assertion is correct, only that it cannot be violated *too soon*. Good tools say exactly that, reporting “the assertion has not been violated up to bound 50 clock cycles” rather than “the assertion passed” [3]. Do not paraphrase that into your own report.

There is a bound at which the distinction dissolves. If the bound is smaller than the **sequential depth** of the system — the longest shortest-path from an initial state to any reachable state — part of the reachable state space is never considered and the proof holds only for the interval. Push past the sequential depth and the bounded

result is a proof — of a safety property; an unbounded eventuality needs more than that, because a counterexample to it is a loop rather than a finite path. That is usually not available anyway: extending the interval that far is infeasible both because the solver’s cost grows and because computing the sequential depth is itself hard [4].

**Induction** is the other route to an unbounded answer, and bounded model checking extends to unbounded model checking precisely through it [1]. The move is to stop fixing the starting state: instead of unrolling from reset, unroll from an *arbitrary* state characterised by a predicate  $P$ . If every reset state satisfies  $P$ , and every interval that starts in a state satisfying  $P$  both upholds the property and ends in a state satisfying  $P$ , then  $P$  is an invariant and the property holds for arbitrarily long time frames rather than just the examined interval — good forever, from a computation no longer than a bounded one. The first condition is the one that gets left out of the telling, and it is what keeps the argument from being empty:  $P = \text{false}$  is closed under transitions too, and establishes nothing.

The price is a new failure mode, and it is the one that eats weeks. Such proofs are conservative: they never report a property to hold when it does not. But if  $P$  over-approximates the reachable state space, the engine starts intervals in states the design can never occupy and produces **spurious counterexamples** from them. The remedy is a *strengthened invariant* — additional assertions, found by inspecting the design or automatically, that rule those impossible starting states out [4].

In practice this is a readable skill rather than a theoretical one. When a counterexample arrives, look at cycle zero first and ask whether the design could be there at all. A FIFO whose occupancy counter reads 7 while its two pointers are equal; a one-hot state vector with two bits set; an outstanding-transaction count of 3 on a port that holds at most 2. Each observation becomes a helper assertion — proven on its own, then supplied to the engine as an invariant. That is where Chapter 11’s redundant-state invariants earn a second income: written to catch silent corruption in simulation, they turn out to be exactly the facts a proof engine needs to be told.

### Reading a depth number

Because a bound is only meaningful against the length of the behaviour you care about, the number in the report has to be compared with an arithmetic you do yourself. The rule of thumb is easy to state — for a pipelined design, a bound equal to or a little larger than the pipeline depth is usually enough [3] — and easy to misapply, because most interesting properties are not pipeline-shaped. Do the count.

Take one run on the DMA engine’s backend, bounded at 40 cycles, and two of Chapter 11’s properties that sit on it — one from the internal-invariant module, one from the port checker bound to the backend’s own AXI manager port.

- `a_no_overflow` guards a datapath FIFO of 16 entries. Filling it takes at least sixteen pushes with no intervening pop, so the interesting state is reachable well inside forty cycles, and the paired `c_full` cover confirms it with a witness. Here 40 is a generous bound.

- `a_w_burst_len` reconciles a burst's beat count against the length its address promised — and it can only do so at `WLAST`. The engine's maximum burst is 256 beats, so a maximum-length burst's `WLAST` cannot arrive until its 256th data beat has been handshaken. Inside a 40-cycle interval no burst longer than about forty beats can have completed at all. The property is proven for short bursts and completely silent about long ones.

Same block, same run, same number, two entirely different amounts of evidence. And one cycle either side of a bound can be the whole result: the crossbar's interleaving counterexample in this chapter's opening is eleven cycles long, so a run stopped at ten would have reported *not violated up to 10 cycles* — green, filed, and wrong.

Hence a cheap discipline that costs one extra directive per property: **before trusting a bounded pass, run the property's antecedent — or the interesting state it guards — as a cover under the same bound**. Every related cover point landing below the bound is what validates that the bound is not optimistic [7]; a cover with no witness inside it means the bounded proof is a statement about a stretch of silence.

**Scenario (the sensor hub).** The always-on sensor-fusion block lives on the 32 kHz island and wakes a DSP running at system speed. The obligation under proof is a wake-path latency: from an asynchronous sensor event, the DSP must have its first sample within a fixed number of *slow* ticks. Measured in fast-clock cycles, a handful of 32 kHz ticks is tens of thousands, and no bounded run reaches the interesting behaviour. **Approach.** Do not buy depth. Prove the obligation on the slow-domain state machine with the fast domain reduced to its handshake interface, and state the property on the slow clock so its natural unit is a tick. **Why.** Depth is not a property of the design but of the clock you measure it in, and a proof that will not converge in one domain often converges immediately in the other. What the reduction costs belongs in the record: synchroniser and metastability behaviour at the boundary is now outside the model, and stays the business of static clock-domain-crossing analysis (Chapter 13).

## 14.4 Abstraction: what you throw away

For a proof to complete at all, the model must be small enough for the tool to finish within the CPU and memory available — which usually means verifying at a relatively abstract level, or verifying relatively small subsystems separately, and doing it with an expert's insight into both the design and the engine [2]. Abstraction is therefore not an advanced topic in formal verification. It is the technique.

It is also the technique with a knife edge, and the edge has a name. Every reduction changes the set of behaviours the model can exhibit, and the direction of that change decides which results you may still believe:

- **Over-approximation** adds behaviours. The abstract model is simpler than the original and allows more traces; if the assertion is proven on it, it also holds on



the original. If the assertion fails, the counterexample may be *spurious* — impossible in the real design. Over-approximation is **sound**: a success is always accurate [3].

- **Under-approximation** removes behaviours. A failure is always real, so it is **safe** — but a pass means nothing, which is why tools that use it say “not violated up to bound 50 clock cycles” rather than “passed” [3].

The practical hazard of over-approximation is human rather than logical. Deciding by hand whether each counterexample is spurious is hard work when there are many, and a user who has waded through twenty impossible traces starts skimming — the twenty-first is real. The capacity you bought with the reduction is paid for in the analysis of spurious failures [3].

### The levers, and which way each one cuts

Tools prune most of the irrelevant model automatically — the logic outside a property’s cone of influence never enters the problem. Manual reduction is needed only for signals that affect the property or the assumptions indirectly, which is precisely why it takes detailed knowledge of the design [3]. Three manual levers cover most of what a practitioner reaches for, and their directions differ:

- **Free a signal** (a *cutpoint*) — disconnect it from its fan-in so it can take any value at any time. Over-approximation: sound, not safe. This is the right move when a property must hold for *every* value of some input, and it is worth seeing the counter-intuition: a property proven with a signal free is thereby proven for the correct values too, so the cut makes the result stronger, not weaker [8]. Blackboxing a submodule is the same lever applied to all its outputs at once.
- **Set an input to a constant** — under-approximation: safe, not sound. Useful for making a data path tractable: to verify propagation through a queue it can be enough to watch a single bit and tie the rest to zero.
- **Set an internal signal** — this may both forbid original behaviours and introduce new ones, so it is neither safe nor sound [3]. It has legitimate uses, and every one of them belongs in the record with a sentence explaining why.

For memories and wide arithmetic there is a fourth move: keep symbolic reasoning on the control logic and treat memory accesses or arithmetic operators as uninterpreted — a black box whose only property is that equal inputs give equal outputs [1].

A tile-based mobile GPU shows why this is not a compromise but the whole point. Its memory-ordering rules concern which writes a shader cluster may observe out of order, and the unified memory behind them holds megabytes; enumerating it is hopeless. Two reductions make it unnecessary, and they are justified differently. The values are the easy half: the rule does not depend on any stored value, so the memory can be uninterpreted and nothing the rule is about is lost. The addresses do the capacity work, and they owe a separate argument: the rule is a relation over a *pair* of locations, so a model carrying two symbolic addresses — a write to one ordered against a write to the other, as seen by a second cluster — states it exactly, while every other location stays free. Two is the smallest number that will do: with one ad-

dress the property is not weakened but deleted, since there is no second write to be ordered against. And the cut owes its own deletion, written as it is made: an ordering violation that appears only across a third address or a third cluster goes to system-level simulation.

### The canonical proof abstraction: two caches, one address

The coherent hub is the standard teaching case because its state space is a product of things you cannot bound: the MESI state of every line, in every cache, for every address. Chapter 11 left one of its invariants — at most one cache may hold a line Modified — with an explicit promise:

```
a_single_writer : assert property (@(posedge clk) disable iff (!rst_n)
 $onehot0(modified_vec));
```

The proof runs against a model cut down to two caches and one address — enough, because the invariant is per line. What survives that cut is more than it looks: the directory's transition rules for a line, every race between two requesters for it, the ordering of invalidations against data responses, and — since livelock lives on a single contended line — starvation of one requester by the other. That is a large fraction of what makes a coherence protocol worth proving, and none of it is reachable by simulation at any run length you can afford.

What does not survive is a list you must write down at the moment you make the cut, because nobody reconstructs it three months later:

- **Anything needing a second address.** The classic coherence deadlock — two agents acquiring two lines in opposite orders — is invisible by construction.
- **Anything needing capacity.** With one address there are no conflict evictions, so a writeback triggered by replacement rather than by a snoop is outside the model.
- **Anything needing a third sharer,** including the directory's own sharer-vector encoding and its overflow behaviour.

And a warning about the cut itself, because it is the most-copied abstraction in this book and the easiest to over-trust: cutting the hub's many caches down to two is **not** a sound abstraction in the sense above. Freeing and setting have a defined direction of error; deleting requesters has none in general. Reducing the requester count is neither a subset nor a superset of the original behaviour but a different parameterization of it — a narrower sharer vector, arbitration over a smaller set — so neither implication direction is available to you. The reduction is a modelling decision justified by a *symmetry argument* — that the hub treats its requesters interchangeably — and that argument is a claim about the RTL, not a theorem. If the arbitration inside the directory is fixed-priority by requester index, it is simply false, and the two-cache proof will never show the lowest-priority requester starving behind the others. So check the claim in the code before leaning on it, then record it where the other unproven premises live: the assumption register of §14.2. An abstraction argument is an assumption in every sense except the syntactic one.

That is the general answer to “how do I avoid throwing away the bug”. You cannot, entirely — an abstraction that kept everything would not converge. What you can do is name the bug classes it deletes, at the time you delete them, and give each one somewhere else to be caught. On the hub, the two-line deadlock goes to a separate, differently-abstracted proof; the eviction path goes to system-level simulation with a cache-thrashing traffic profile; the sharer-vector overflow goes to a directed test. Each becomes a plan row reading *not covered by proof P-COH-2 — covered by ...* (Chapter 5’s traceability, and Chapter 16’s division of labour). A proof whose deletions are enumerated is evidence; a proof whose deletions are implicit is a slogan.

## 14.5 Convergence: the undetermined property

Sooner or later a run comes back neither green nor red. The result is inconclusive because the tool timed out, ran out of memory, or used an algorithm that is incomplete by construction. Getting a conclusive answer means refining the verification model: a more aggressive or smarter abstraction, a smaller model, auxiliary assertions — **lemmas**, which may be easier to prove and, once proven, can be supplied as assumptions for the original property — or simply different tool options <sup>[3]</sup>.

That is the menu. The order you work through it is what separates an afternoon from a fortnight, and it is not the order the menu is written in.

**1. Ask whether the property is even true.** Before investing in invariants, run bounded and run deep. An undetermined property is sometimes a false one whose counterexample is simply further out than the last bound you tried, and finding it costs one overnight job instead of a week of strengthening.

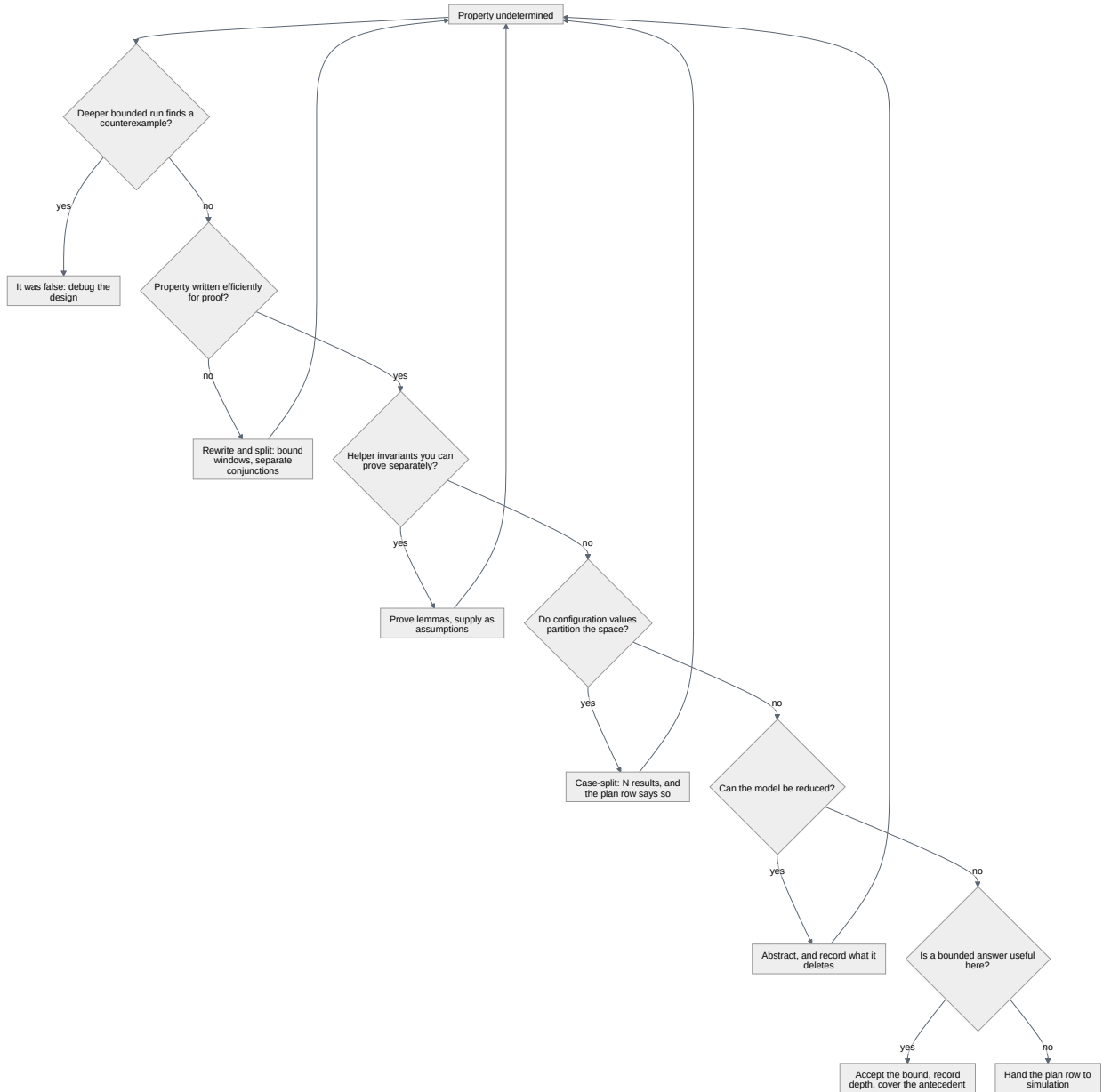
**2. Ask whether the property was written for this engine.** In simulation an inefficient assertion costs runtime; in formal it can be fatal, because a session with an inefficient assertion may return no conclusive result at all — and where one text must serve both engines, the formal-efficient form should win <sup>[3]</sup>. Unbounded eventualities, sprawling antecedents and wide equality chains are the usual offenders. Rewriting the property is free compared with everything below it.

**3. Split it.** A property asserting five things at once is five proofs sharing one worst case. Separate them and most will close; the report then names the one that did not, instead of hiding four successes behind it.

**4. Prove lemmas and feed them back.** This is §14.3’s invariant strengthening used deliberately rather than reactively: pick facts about internal state you believe and the engine does not know — a queue’s occupancy matches its pointers, a state vector is one-hot, an outstanding counter never exceeds the port’s capacity — prove each on its own, and hand them to the hard proof as assumptions. They are the only kind of assumption that costs nothing, because you have discharged them.

**5. Case-split**, proving the property separately under each value of a configuration register once you have shown those values partition the space —  $N$  results, and the plan row must say so. **6. Abstract**, last of the model-changing moves, because §14.4’s bill comes with it. **7. Accept a bounded result**, with the depth recorded and

its antecedent covered under the same bound. **8. Stop:** hand the plan row to simulation and close it there. The last is a legitimate outcome, not a defeat, and a team that cannot reach it will spend a whole schedule on one property. Four of them — rewrite, lemmas, case-split, abstract — change the problem instead of answering it, and return you to the top with a smaller one.



**Figure 14.1** — The order to work an undetermined property in, drawn as a decision tree: ask whether the property is false before strengthening anything, then whether it was written for a proof engine, then split it, then lemmas, then a case split, then abstraction — whose bill §14.4 states — and then a bounded answer, with handing the plan row to simulation as the last exit rather than a defeat. The four moves that loop back to the top, rewrite and lemmas and case split and abstraction, change the problem instead of answering it and return you to it smaller, which is why the tree is drawn in the order experience puts the menu rather than the order the menu is written in.

## When formal is the right instrument

Formal comes in two economies, and conflating them is why teams either over-invest or dismiss the technique after one bad week.

**Lightweight formal** asks only that you formulate adequate assumptions on the block's inputs — the trickiest and most effort-consuming step — after which running the tool is not very different from running a simulation. Run early, with small bounds, and it cleans up obvious bugs before the simulation environment even exists, buying time at the front of the schedule rather than the back [3]. Any competent verification engineer can do this; the belief that formal needs a specialist belongs to the other flow. The strongest published test of that claim inverted the usual staffing deliberately: at Intel, on a live graphics project, property verification was pushed out of the central formal group to the *design* engineers, 85 of whom were trained inside two weeks and then supported by onsite consultancy for a further four, and the drive found 82 interesting bugs across twelve weeks despite starting only after a major dynamic-verification milestone had already completed — that is, against simulation environments that were already stable and already green [9].

**Exhaustive formal** is a full-time job: model reduction and pruning, hand-written abstract models for parts of the design, checking that the specification is complete, iterative refinement, algorithm tuning. That cost is worth paying only where correctness is critical and simulation unreliable — arithmetic units such as multipliers and dividers, arbiters, coherency managers, branch predictors, critical controllers, bus protocols [3]. Read that list next to the reference SoC and it selects itself: the crossbar's ordering and routing rules, the DMA's legalization arithmetic, the event unit's interrupt priority resolution. Chapter 10 reached the same conclusion from the stimulus side — where the specification is a set of properties, a constrained-random generator samples what a proof concludes.

Knowing when to walk away is the more valuable half of the judgment.

**Scenario (the video codec).** An H.265-class encoder is put forward as a formal target: it is critical, it is hard to verify, and the team has proof capacity. **Approach.** Split it, and formalise only one half. The rate-distortion decision path is data-dependent control over a wide bit-exact datapath whose specification is a reference model — there is no compact property that says “this is the correct macroblock”, and any attempt to write one reproduces the reference model in SVA. That half belongs to golden-model comparison in simulation, and its arithmetic blocks to the equivalence checking of Chapter 15. The other half is full of good formal targets: a frame is never emitted before its last slice completes, the line buffer is never read past what has been written, a bitstream packet's length field always matches the bytes that follow. **Why.** Formal excels where the specification is a set of rules over a bounded amount of state, and fails where the specification is a transformation of data. Blocks are rarely one or the other; the useful skill is drawing the line inside them.

## 14.6 Formal sign-off

A proof reaches sign-off as evidence for a plan row, and the row has to say what the evidence is. A formal testbench reaches sign-off quality when it can answer three questions: is the list of end-to-end checkers complete, are there unintentional over-constraints, and have all checkers reached the required proof depth <sup>[7]</sup>? Those are §14.1’s three qualifiers — model, assumptions, depth — asked from the reviewer’s side rather than the engineer’s.

### Coverage when there is no stimulus

Coverage in simulation measures what your tests did. A proof has no tests, so “formal coverage” has to mean something else — and it turns out to mean four different things, each answering a different question <sup>[10]</sup>.

**Cone-of-influence coverage** takes the union of the cones of all asserted properties and reports the design space outside it. Purely structural and cheap, it answers *did anybody write a check about this logic at all?* — and the area outside every cone is where verification holes live. The honest response is either more properties or a reviewed waiver naming the plan item that covers that logic instead. The caveat that keeps it from being a closure metric: an assertion being fully proven does not mean the whole of its cone was verified.

**Input stimuli coverage** ignores the assertions and measures what the *constraints* let the engine reach, reporting code that is uncoverable only because of them — which turns §14.2’s silent hazard into a number. The shape of it is entirely ordinary: one line was uncoverable, and the cause was a single setup command in a tool script holding an enable low. It had been proven nowhere and checked in no simulation, and it would have survived any number of manual reviews <sup>[10]</sup>.

**Bounded proof coverage** measures how much of a property’s cone is reachable within the current bound, and supplies the stopping rule §14.3 left as judgment. If a bounded proof covers very little of its cone, the bound is not good enough to sign off on; and when two days of engine time between bound 10 and bound 11 adds no newly covered condition, that is the signal to stop buying depth and start abstracting <sup>[10]</sup>.

**Proof core coverage** is the sharpest and most expensive. Engines abstract internally, so the logic actually used to establish a proof — the proof core — can be much smaller than the cone, and a bug outside the core but inside the cone will not be caught by that assertion even though it is fully proven <sup>[10]</sup>. Practitioners warn that such vendor-specific metrics are smarter than simulation code coverage and correspondingly harder to interpret <sup>[8]</sup>. Run the analysis on your handful of highest-value properties, not on the suite.

### Grading the property set by breaking the design

All four models are structural: they say which logic was involved, not whether a check would have noticed a change to it. The empirical complement is **mutation** —



break the design deliberately, and some assertion should fail [8]. It is Chapter 8’s known-bad variant and Chapter 11’s mutation question, applied to a proof.

The most convincing published run of that experiment was a challenge at DAC in 2015: attendees inserted 73 functional bugs into a crossbar design that already had a formal testbench, and every one triggered a failure of one or more checkers. Read the footnote as carefully as the headline — two further changes failed nothing: one had no functional impact at all, a loop bound rewritten to save area, and the other had been inserted specifically to evade the methodology, which the authors classify as a non-bug on the grounds that a determined adversary can always do that [7]. That is the right scope for mutation: it grades a property set against the bugs designers make, not against an attacker. Chapter 23’s threat model is a different instrument.

### The strongest criterion, and what it costs

There is a criterion stronger than all of the above, and Chapter 3 promised it would be treated with respect. Instead of asking whether the proof touched the logic, ask whether the properties **pin the outputs down**. It is sufficient, for excluding verification gaps, to determine whether the conjunction of properties is satisfied by exactly one output trace for each input trace — that is, whether the engineer decided on a fixed output value at every point in time. If not, the verification must be improved [11].

Stated flat, that criterion is too strong for real interfaces, and the refinement is what makes it usable. Protocol specifications routinely leave signals free outside their valid windows, and those values may legitimately change with an area or power optimisation, so a verification that checks them is checking the wrong thing. The blur is made explicit through *determination conditions* — assumptions saying which input traces count as the same, commitments saying which output traces do — so completeness is judged only where the specification actually commits. The payoff is the strongest result in this chapter, and the qualifier is part of it: across a published list of commercial projects using complete verification and its less mature predecessors, almost all the verified circuits functioned properly after fabrication, and in the few projects where a fault was overlooked the escape was traced to human failure in applying the methodology — wrong constraints that masked errors, or an inaccurate description of expected output behaviour that accepted a real fault [11]. The cost is what Chapter 3 already named — a different way of writing properties, a dedicated methodology, and effort concentrated on selected modules.

That criterion also has an industrial service record, which is rarer for a sign-off argument than for a technique. Verification of this kind, with the completeness check run as an automatic step, was in regular productive use at Siemens and at Infineon from 1999 onwards; the published project list’s only named part is the Infineon TriCore2, a superscalar 32-bit automotive processor, alongside AHB master, slave, bridge and multilayer bus interfaces and an AXI interface, and on those projects experienced engineers verified 2,000 to 4,000 lines of RTL per person-month, with a recorded best of 8,000 [11]. The caveat is part of the record rather than a footnote to it: the author reports that almost all the circuits verified this way worked after fabrication,

and attributes the escapes that did occur to the methodology being misapplied rather than to the criterion being too weak.

### What goes in the package

The reviewer’s material, then, is not a screenshot of green ticks. It is:

1. **The property list with per-property results** — proven, falsified-and-fixed, or bounded with its depth stated as a number.
2. **The assumption register** of §14.2, with each row’s discharge status: proven on the driving block, checked in simulation, or accepted with a written argument.
3. **The required proof depth** for every bounded result, with its derivation. The method is a short arithmetic — latency analysis, micro-architectural analysis (how many transactions must pass to exercise the arbiter), corner-case analysis, plus margin. On an 8×8 crossbar it came to 13 cycles: one cycle of design latency, an input sequence long enough to push eight same-priority requests through the arbiter — the last of them visible at the target at cycle 11 — plus one cycle for back-pressure and one for a safety net. Because the engine reports the *shortest* path to failure, no input sequence beyond that depth represents a scenario not already covered inside it <sup>[7]</sup> — provided the corner-case enumeration behind it is complete, which is why the enumeration, not the arithmetic, is the part to review. Two cross-checks make the number defensible: every related cover witnessed below the bound, and a bound exceeding the longest counterexample the project has ever seen on that block <sup>[8]</sup>.
4. **Formal coverage evidence** of whichever of the four kinds you ran, with waivers for what is left uncovered.
5. **The abstraction deletion list** from §14.4, each entry pointing at the plan row that covers it elsewhere.
6. **The provenance**: RTL revision, property-file revision, tool build and engine settings. A proof is reproducible or it is an anecdote.

### Worked example: the crossbar’s formal row in the sign-off package

**Table 14.1** — The crossbar’s formal results in the shape a sign-off package presents them: one row per property binding, with the verdict, the proof depth where the result is bounded, the assumptions the row leans on by ID, and a note. The columns are chosen so that nothing can hide behind a green tick — the third row records a property that was false and fixed before it was proven, the fourth is the only bounded result and therefore owes a derivation of its depth, and the fifth was proven only under an assumption withdrawn at review, so the row leaves formal for a directed simulation sequence.

Property (binding)	Result	Depth	Assumptions	Note
a_aw_stable (4 subordinate-side)	proven	unbounded	A2	the crossbar’s own stability toward each subordinate
a_b_id_outstanding (4 subordinate-side)	proven	unbounded	A2, A4	

Property (binding)	Result	Depth	Assumptions	Note
<code>a_w_burst_len</code> (4 subordinate-side)	falsified → fixed → proven	unbounded	A2, A4	CEX 11 cycles; bug #217 (Chapter 4), this chapter’s opening
<code>a_grant_within_64</code> (arbitration, per manager port)	bounded pass	78	A1, A2, A3	window and depth derived below; antecedent cover at 12
<code>a_b_order</code> (response ordering, per manager port)	proven under A5; undetermined without it	—	A2, A5	A5 withdrawn at review; row handed to simulation as a directed multi-ID sequence

**Table 14.2** — The assumption register those results lean on: the five assumptions by ID, the party that owns each one, and how each was discharged. The discharge column is what the proofs are worth — A1 is discharged three different ways across the four subordinates, A4 was proven on the crossbar itself and supplied back as a lemma, A3 rests on simulation alone, and A5 was added during debug to make a property converge, was never reviewed, and is withdrawn at sign-off.

ID	Assumption	Owner	Discharge
A1	every subordinate accepts an address within 16 cycles — a project constraint, not a protocol requirement	subordinate owners	proven as an assertion on the SRAM subordinate; checked in SoC simulation for the APB bridge ( <i>AMBA® APB Protocol Specification</i> , Arm IHI 0024, Issue E, 2023, chapter 3, “Transfers”) <sup>[12]</sup> and the neural accelerator; the boot ROM’s port is always ready, discharged by construction
A2	the five managers obey the AXI manager rules (Arm IHI 0022H.c §A3.2.1 “Handshake process”, §A3.4.1 “Address structure”, §A5.2.2 “Write data ordering”) <sup>[6]</sup>	DV	the same bound checker with <code>ASSUME_MODE = 1</code> on the manager ports; the identical text is <i>asserted</i> on the DMA and core in their own proofs
A3	no manager stalls its own responses indefinitely	DV	<b>simulation only</b> — if false, the response buffers fill and A1 fails behind them, so <code>a_grant_within_64</code> does not hold; accepted, revisit if arbitration changes
A4	at most one write address outstanding per manager at each subordinate port	design	<b>proven on the crossbar itself</b> , then supplied back as a lemma; it bounds the checker’s address queue to five entries. Two managers with one write each still overlap, so the opening’s bug stays in the model
A5	each manager keeps at most one write outstanding per ID	DV	<b>none</b> — added during debug to make <code>a_b_order</code> converge, never reviewed. Withdrawn at sign-off

Row four is the only bounded result, so it owes item 3 a derivation. *The obligation*, first, because two readings of “grant” differ by the whole of A1: a write address offered at a manager port is granted by the target subordinate’s arbiter within 64 cycles. The address handshake that follows is a separate obligation, bounded by A1, and not what this property claims. *The contention model*, without which no number in the row is falsifiable: all five managers may want one subordinate at once, the arbiter is round-robin and sees a request in the cycle it is offered, and a grant is held until the subordinate accepts the address — sixteen cycles at worst, by A1. *The win-*

*dow* follows: round-robin serves each of the other four managers at most once ahead of this one, and each holds the target's address port for at most sixteen cycles, so the grant cannot fall later than  $4 \times 16 = 64$ . *The depth* follows from the window, not the reverse: the antecedent — five requests pending at one subordinate — is first witnessed by its cover at cycle 12, so the shortest counterexample this property admits is 76 cycles long, and the two cycles of margin above, one for back-pressure and one for a safety net, put the run at 78. There is no latency term, because the obligation ends at the grant, not at the subordinate's port. Chapter 11 wrote the unbounded shape of this rule — a request is eventually granted — which a proof engine decides; a sign-off row records the bounded restatement, because a bound is the part a reviewer can recompute.

Notice A1 doing real work there, and only there: it is why the wait is finite. Rows two and three are pure safety properties — a response nobody asked for, a wrong beat count — and no bound on how fast a subordinate accepts makes either more or less true, so A1 is not in them. What they need is A4, the lemma that keeps the checker's queue finite. An assume column is derived per row; one filled by copying is the same defect as an undeserved checkmark, a table to its left.

The second table is the one the sign-off review actually argues about, and A3 and A5 are why. A3 was a known concession, accepted with an argument — the discipline working. A5 arrived mid-debug to make a proof converge, was never reviewed, and was doing the proving: with one response of an ID in flight there is no pair left to order, so `a_b_order` proved a rule about traffic A5 had deleted. Chapter 7 stated the rule this chapter has been earning: a full proof under an over-constrained environment is a beautifully formatted blind spot, so the review reads the assume list, not the checkmarks.

---

## IN PRACTICE

Formal arrives in an organisation as two streams that must not be confused. Designers run lightweight proofs on their own blocks in the first weeks after the RTL compiles, before any simulation environment exists. A much smaller group runs exhaustive proofs on the short list of blocks that deserve them. The industry data matches that split: on IC/ASIC projects, adoption of formal property checking grew at 5.8 percent CAGR through 2024 while automatic formal applications — the packaged checks that need no property writing, Chapter 15's subject — grew at 8.7 percent [13]; on FPGA projects the combined figure is 7.5 percent CAGR [14]. The faster-growing half is the half that needs no expert, which tells you where the expertise still binds.

The messy parts are consistent across teams. Proofs go stale: RTL changes nightly, and a proof from six weeks ago is a claim about a design that no longer exists — so formal runs belong in continuous integration on a cadence, like a regression tier (Chapter 12), with a newly undetermined property treated as a failure rather than as weather. Assumptions accumulate during debug and are never removed; someone adds one on a Friday to make a proof converge and it is still there at sign-off, which

is what the assumption register and its named owner exist to prevent. And the most valuable habit sits upstream of all of it: one industrial effort found that the environment assumption blocking its proof could be eliminated by adding a small amount of logic to the arbitration unit, and concluded that keeping provability in mind during design yields shorter proofs and simpler designs [2]. Ask for that in design review, while it is still cheap.

---

## PITFALLS

1. **Reading a bounded pass as a proof.** Symptom: your report says “proven” where the tool said “not violated up to  $N$  cycles”. Cure: carry the depth into every report, and derive the depth the block actually needs.
2. **The assumption that assumed the property.** Symptom: a property that would not converge for days proves in seconds shortly after an assumption was added. Cure: delete that assumption and re-run; if the property collapses, the assumption was doing the proving.
3. **Waiving a counterexample.** Symptom: “we know our managers never do that”, and the failure is dismissed. Cure: a tool reports one counterexample, not all of them, and several design bugs can violate the same assertion — so convert the waiver into an explicit, reviewed assumption rather than an explanation, or the second bug stays hidden behind the first [8].
4. **Abstracting without a deletion list.** Symptom: a proof that now converges, and no record of what left the model. Cure: write the list at the moment of the cut, each entry naming where that behaviour is covered instead.
5. **Learning to skim spurious counterexamples.** Symptom: an over-approximated model produces impossible traces and the team starts dismissing failures by reflex. Cure: strengthen the invariant until the traces are real; the twenty-first one will not be.
6. **Proving what is easy to prove.** Symptom: a growing property count concentrated on decode and register logic, with the ordering and arbitration rules untouched. Cure: measure where the proofs are against where the plan said the risk was.

---

## SUMMARY

- A proof is a statement about a model, under assumptions, sometimes to a depth. “Formally verified” without those three qualifiers is not a claim [2].
- A formal report has four outcomes, not two, and a bounded pass is the one most often mistranslated: it says the property cannot be violated too soon [3].
- Assertions and constraints are one artifact with two directions. In formal an assumption is never checked; it deletes traces, and an over-constrained model produces vacuous proofs that look exactly like real ones [1].

- A missing assumption announces itself as a spurious counterexample; an extra one announces nothing at all. That asymmetry is why the assume list, not the checkmark list, is what a sign-off review reads.
- Abstraction is not an advanced topic; it is the technique. Over-approximation preserves proofs and can fabricate failures, under-approximation preserves failures and cannot prove anything [3].
- Convergence work has a cheapest-first order: check the property is true, check it was written for a proof engine, split it, prove lemmas, case-split, abstract, accept a bound, stop.
- Formal coverage means four measurable things — cone of influence, input stimuli, bounded proof and proof core — and none of them says a check would have caught a change. Only mutation does [8, 10].

---

## FURTHER READING

Within the corpus, [2] is the reference survey of what formal verification is and what a verification claim must state to mean anything, and [3] Chapter 20 is the most practical treatment available here of the working vocabulary — counterexample and witness, complete and incomplete methods, over- and under-approximation, pruning, the lightweight and exhaustive flows, assume-guarantee. [1] places model checking and bounded model checking among the other formal techniques and is the source for the constraint/assertion duality; [4] gives the clearest account of bounded versus unbounded proof, sequential depth, and why induction from an arbitrary starting state produces spurious counterexamples; [11] is the corpus's strongest completeness argument. For industrial practice, [7] is a sign-off methodology worked end to end on a crossbar, with a required-proof-depth derivation and a mutation experiment; [10] defines the four formal coverage models; [8] tours complexity management — cut-points, black-boxing, counter abstraction — and the traps around waivers and coverage metrics.

Outside the corpus: E. Seligman, T. Schubert and M. V. Achutha Kiran Kumar, *Formal Verification: An Essential Toolkit for Modern VLSI Design*, is the standard book-length introduction to the practice this chapter describes, and E. M. Clarke, O. Grumberg and D. Peled, *Model Checking*, remains the canonical text for the algorithms deliberately left inside the tool here.

---

## REFERENCES

1. **B6** J. Yuan, C. Pixley, and A. Aziz, "Constraint-Based Verification," Springer, 2006.
2. **P20** C. Kern and M. R. Greenstreet, "Formal Verification in Hardware Design: A Survey," ACM Transactions on Design Automation of Electronic Systems, vol. 4, no. 2, pp. 123–193, 1999.
3. **B4** E. Cerny, S. Dudani, J. Havlicek, and D. Korchemny, "SVA: The Power of Assertions in SystemVerilog," 2nd ed., Springer, 2015.
4. **T1** M. Schwarz, "Formal Hardware/Firmware Co-Verification of Optimized Embedded Systems," Ph.D. dissertation (Dr.-Ing.), Technische Universität Kaiserslautern, Germany, 2021.



5. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
6. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
7. **D31** I. Tripathi, A. Saxena, A. Verma, and P. Aggarwal, "The Process and Proof for Formal Sign-off: A Live Case Study," Proc. DVCon U.S., 2016.
8. **D13** J. Bromley and J. Sprott, "Formal Verification in the Real World," Proc. DVCon U.S., 2018.
9. **D18** M. V. Achutha Kiran Kumar, E. Seligman, A. Gupta, Ss Bindumadhava, and A. Bharadwaj, "Making Formal Property Verification Mainstream: An Intel Graphics Experience," Proc. DVCon U.S., 2017.
10. **D11** X. Feng, X. Chen, and A. Muchandikar, "Coverage Models for Formal Verification," Proc. DVCon U.S., 2017.
11. **T2** J. Bormann, "Complete Functional Verification," Ph.D. dissertation, University of Kaiserslautern, Germany, 2009 (English translation of the German dissertation 'Vollständige funktionale Verifikation', April 2009).
12. **S19** Arm Limited, "AMBA APB Protocol Specification, ARM IHI 0024, Issue E (ID022823)," Arm Limited, February 2023.
13. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
14. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.

---

# 15. Formal Apps and Equivalence

The reference SoC's top level is two thousand lines of instantiation and wiring, and it contains one mistake. The neural accelerator's error interrupt arrives at the event unit on input 5; the firmware header, generated from the architecture document, says input 6. Both blocks are correct, so every block-level environment passes. The system regression passes too, because no test both provokes an accelerator error and enables that particular interrupt: the error path is exercised by the accelerator's own tests, which read the status register, and the interrupt map by the event unit's tests, which inject events directly. The mistake surfaces in the lab, as a hang.

Nothing here is subtle. The intent is a table with a few hundred rows — this source, that destination, under this condition — and the implementation is structural wiring. Comparing the two is mechanical, and answering it by simulation is absurd: one directed test per row, each proving a single wire, none exhaustive over the conditions.

Meanwhile the team's one formal specialist is three weeks into the crossbar's ordering proof, choosing abstractions and defending assumptions (see Chapter 14). She is not available, and she is not needed: this question requires nobody to write a property. It requires a tool that already knows which property to write, and a table to write it from.

That is this chapter: formal packaged as a product rather than practised as a craft, and the equivalence flows that run on every project, quietly, over transformations property checking never looks at.

---

## CHAPTER MAP

- §15.1 establishes why the pre-packaged formal *app* succeeded where general property checking stalled — the app supplies the properties, so a team supplies only the design and a configuration — and §15.5 what changes when the properties it supplies are extracted from the design itself.
- §15.2 turns an integration table into a proof by connectivity checking, the highest-value, lowest-effort formal application on most projects.
- §15.3 discharges a register map against its machine-readable description, and states precisely which gap register checks do *not* close.
- §15.4 replaces a coverage waiver's argument with a proof of unreachability, and states why that verdict is only as trustworthy as the constraints behind it.
- §15.6 states what logic and sequential equivalence checking each prove and assume, and why LEC won full industrial acceptance while property checking was still a hope; §15.7, why arithmetic datapaths need a flow of their own.

## 15.1 The app: formal as a product

Chapter 14’s flow has a shape: someone reads the specification, decides what is worth proving, writes properties, states assumptions, defends them, fights convergence, interprets the result. Only one of those steps is the engine’s. The rest is specification work, and it is why formal property checking spent two decades as a specialist activity — high-effort, requiring skills and expertise most project teams did not have [1].

A formal app inverts the economics by shipping the properties. The user supplies the design and a configuration; the tool supplies everything else. The anatomy is the same for every app in the catalogue. An X-propagation app reads in the RTL, analyses the design, and automatically implements assertions checking for X occurrences on targets such as clocks, resets, control signals and output ports; the engineer’s remaining job is to judge whether each proved occurrence was expected [2]. Nobody wrote a property. Nobody stated an assumption. The specification was already in the tool.

The consequence is an adoption curve that looks nothing like property checking’s. Between 2012 and 2014, adoption of automatic formal applications grew by 62 percent against incremental growth for property checking, for exactly this reason: narrowly focused solutions that do not require specialised skills to adopt [1]. A decade later the gap persists rather than closes. Across IC/ASIC projects from 2016 to 2024, property checking adoption grew at 5.8 percent compound annually and automatic formal applications at 8.7 percent — a shift towards tools that make formal accessible to non-experts [3]. FPGA projects moved the same way over the same window, formal technologies together growing at 7.5 percent compound annually [4].

The catalogue built on this idea spans the whole flow rather than clustering at one phase [5]:

**Table 15.1** — The catalogue of formal apps: each app, the claim it discharges, and — the column that decides whether it pays — the machine-readable source its intent comes from. Read the right-hand column as the price list, because the properties come free only where that source already exists for some other reason; where it does not, creating and curating it is a project’s cost rather than a tool’s. The first four rows are this chapter’s business and the last two belong to the static gate.

App	What it proves	Where its intent comes from
Connectivity	Every declared connection exists, and no undeclared one does	An integration table (§15.2)
Register access	Reset values, access policies, decode	A register description (§15.3)
Unreachability	A coverage target admits no legal trace	The coverage database (§15.4)
Auto-generated properties	Interface and structural invariants extracted from the RTL	The RTL itself (§15.5)
X propagation	No unknown reaches a control point	The RTL’s reset and initialisation structure (Chapter 13)
Clock-domain crossing	Synchroniser structure and protocol correctness	The clock and reset declarations (Chapter 13)

The first four are this chapter’s business; the last two belong to Chapter 13’s static gate. The fourth is also the bridge back to Chapter 14 — it drafts what a specialist would otherwise write — and it is the one row whose economics do not resemble the others’, which is why §15.5 gives it a section of its own.

Notice what the right-hand column implies. An app does not remove the need for a specification; it relocates it. The properties come free only because the intent already exists somewhere machine-readable, and where that source does not exist, creating and curating it is the app’s real cost — a project’s cost, not a tool’s. Before promising a manager that connectivity checking is free: every hour saved on property writing reappears as intent curation.

Nor does convergence transfer. The engine underneath an app is Chapter 14’s engine, and a bounded result from an app is exactly as bounded as one from a hand-written proof. Run on a small module chosen with hindsight, an X-propagation app found its bug in minutes; run on the whole IP under a two-hour limit, the same app reported no violations and achieved only bounded proofs <sup>[2]</sup>. An app makes property writing free. It does not make the design smaller.

A safety app makes the same point with the numbers attached, and it is worth following through because the arithmetic is public. The design was an ECC block at NXP Semiconductors carrying a SECDED scheme — single-error correction, double-error detection — around an 8K memory. Run as an app on the whole thing, the fault campaign did not converge in even 12 hours, so the memory was blackboxed and its input ports promoted to observation points — an abstraction the authors are careful to characterise, since it can produce false negatives on the safe classification but not false positives. With it, the campaign cut 3,606 injected faults to 2,489 prime faults, proved 763 of those structurally unobservable and a further 232 and then 16 formally unobservable, and ended by identifying 512 faults that corrupt the stored data without ever raising the block’s error signals <sup>[6]</sup>. Read the shape rather than the totals: the app supplied the fault model, the pruning and the classification machinery, and a human still had to supply the abstraction that made any of it terminate — which is this section’s economics arriving in the least convenient place.

## 15.2 Connectivity checking: proving the wiring

---

Integration is where designs are assembled rather than designed, and its errors have a profile: individually trivial, individually invisible, collectively certain. None of them is a hard problem, and all of them survive block-level verification by construction, because every block is correct.

Formal is the right instrument here for reasons of shape, not power. Each connection is a tiny claim about a short path: this source reaches this destination, after this many cycles, when this condition holds. The cone of influence is a wire and perhaps a multiplexer, so proofs converge in seconds, thousands run in one job, and each answer is a proof rather than a sample. The reference SoC's interrupt map — several dozen sources into the event unit, some conditional on configuration — is discharged completely in a single run, where a simulation argument needs one directed test per source per condition and is still arguing about coverage at the end.

What the app generates from one row of the table looks like this:

```
// Intent row: src = nn_err_irq, dst = evt_in[6],
// enable = cfg_irq_en[6], latency = 1 cycle.
property p_conn_nn_err;
 @(posedge clk) disable iff (!rst_n)
 cfg_irq_en[6] | => (evt_in[6] == $past(nn_err_irq));
endproperty
a_conn_nn_err : assert property (p_conn_nn_err);
```

The shape recurs, so read it once carefully. The antecedent is the enabling condition sampled now; the consequent fires one tick later and compares the destination against the source *as it was when the enable was true* — which is what a one-cycle registered connection means, and `$past` is how you say it. It also says nothing about cycles where `cfg_irq_en[6]` is low, and an unconditional claim would be wrong: the point of the enable is that the connection is not always live. Two hundred rows of the connectivity table, generated and discharged in one run, is the entire flow.

One of the two hundred is the row above, and on the reference SoC it does not prove: the wiring reaches input 5. No test had to provoke an accelerator error while that interrupt was enabled — the proof does not care whether one ever would. Notice also what the generated text does not carry: a cover on `cfg_irq_en[6]`. Chapter 11's rule — every implication ships with a cover on its antecedent — is not discharged by an intent row. This row fails, so its own gap is moot; on the other 199, nothing in the generated text asks whether the enable is reachable, and a green result does not distinguish a proved connection from an enable no legal trace can set.

The dual claim matters as much, and it is the part teams forget: a table listing what must be connected says nothing about what must *not* be. On the reference SoC the interesting negative rows are the debug fabric's — the JTAG-reachable abstract-access path may reach the memory and the core's register file, and must not reach the event unit's retention registers. As a no-connection check this is cheaper still: a structural question about whether any path exists in the cone at all, which a tool answers without a proof engine getting involved.

**Scenario.** The crypto engine sits inline on the memory path, with a key ladder that derives working keys from fuses. The security requirement is that a working key reaches the AES core and nothing else: not a readable register, not the trace buffer, not a scan chain. **Approach.** Write the positive rows (key register to cipher core, per round) and, more importantly, the negative ones: no path from any key-

ladder output to any bus-readable address, to the debug data register, or to a scan-observable point. Run them as part of the same connectivity job, at every integration build. **Why.** This is the cheapest security check in the book and it catches the accident that matters — a debug hook, a test observation point, a “temporary” mux added during bring-up. But be exact about what it proves. A no-connection result is *structural*: no path exists. It does not say the key never influences an observable value along a path that does exist, which is an information-flow question and belongs to Chapter 23. Confusing the two is how a team believes it has proved non-leakage when it has proved non-adjacency.

One scaling note, because integration tables get large. Checking every path of a multi-port interconnect individually — the crossbar’s five manager ports against its four subordinate ports, twenty combinations and far more once each is qualified by burst type and identifier — has poor runtime and, by symmetry, minimal verification value. Make the port indices *symbolic* instead: pick an arbitrary manager M and an arbitrary subordinate S, constrain them only to be stable, and check the single path from M to S. Because the proof must hold for every value the arbitrary indices could take, one property covers all of them. It is not always faster — 162 seconds for the symbolic proof against 65 for the exhaustive per-path proofs together <sup>[5]</sup> — so measure rather than assume. But it is how a connectivity argument stays constant-sized as the interconnect grows.

### 15.3 Register checks: the map against its description

Chapter 10 built a register model and used it to drive and predict accesses. That model was generated, not written, and the thing it was generated from is the subject of this section: a register description language (Accellera SystemRDL 2.0 Register Description Language, Version 2.0, 2018), one file from which every view is generated <sup>[7]</sup>. The views are the ones every project already has and already lets drift — RTL decode and field logic, verification model, firmware header, documentation.

The register app closes the remaining edge: it proves the RTL implements the description. From that description it generates and proves reset, access-policy and functional checks for every field <sup>[5]</sup>, plus the implementation-level properties the description implies but does not state. Per register: the reset value is what the description says, for every reset; the decode maps exactly one address to it; a read returns what the policy says; a write does what the policy says *for every value written*, not for the values a test happened to pick; reserved fields ignore writes; unimplemented addresses behave as declared.

That enumeration is the argument for the app. A conventional register test walks the map front-door, register by register, pattern by pattern — write-read-verify sweep, reset check, bit-bash — which is a good test that scales linearly with the map, and why a large map’s register test is measured in hours. The app proves the same statements over all values and orderings in one job, at RTL rather than through the bus, so it converges even for registers a bus-level test reaches only through a long configuration sequence.



Take the canonical status register from Chapter 10 — the UART’s RX overrun flag:

```
class uart_status_reg extends uvm_reg;
 rand uvm_reg_field oe; // RX overrun error

 function new(string name = "uart_status_reg");
 super.new(name, 32, 0);
 endfunction

 virtual function void build();
 oe = uvm_reg_field::type_id::create("oe");
 // parent size lsb access volatile reset has_rst rand indiv
 oe.configure(this, 1, 3, "W1C", 1, 1'b0, 1, 0, 0);
 endfunction
endclass
```

Two of those arguments are exactly what a register app is for. The policy `"W1C"` — write one to clear — is a statement about every possible write value and both possible current values, of which a random register test exercises a handful. And `volatile` records that hardware can change the field without software’s involvement, which is why the check must read *a software write of 1 clears the field unless hardware sets it in the same cycle*, not *the field is 0 after the write*. A register description language makes this explicit — any hardware-writable field is inherently volatile, precisely because verification and test need to know <sup>[7]</sup> — so the generator derives the model’s `volatile` argument from the field’s hardware access rather than from anything anyone wrote down. An app that ignores the flag generates a check that fails on correct hardware, and the first thing a team does with a failing generated check is switch it off.

Now the honest limit, which is the one Chapter 3 named. The app proves *RTL against description*, and both the register model and the RTL check come from the same file, so an error in the description is invisible to both — a common-mode error, encoded once and read twice. Nothing here reviews whether bit 3 was the right bit. That review is a human reading the description against the architecture document, and the app’s value is that it removes everything else, leaving only the part that needs a person.

**Scenario.** The GPU’s four shader clusters carry identical register banks — performance counters, cache-invalidate triggers, per-cluster clock-gate overrides — instantiated four times with a parameterised base address. During a late change, cluster 3’s bank is hand-edited rather than regenerated, and one counter’s reset value is left at the previous revision’s. **Approach.** Regenerate all four banks from one description and run the register app across the whole map, all four instances, every field. **Why.** Replication is where copy-paste errors live, and a per-cluster register test finds this only if someone remembered to parameterise the test as well as the design — the same mistake in the same place. The app enumerates from the description, so it checks cluster 3 whether or not anyone thought about cluster 3. It is also where the cost comparison is decisive: four thousand fields is a long afternoon of bus traffic and a short formal job.

## 15.4 Unreachability: a proof instead of a waiver

Chapter 6 left coverage closure with an uncomfortable artifact. Some holes are genuine gaps in stimulus, and you close them. Others are *not holes at all* — no legal execution reaches the target, and the honest disposition is an exclusion, which Chapter 7 gave its written form: the hole, the reason, the risk argument, the owner, the expiry. The reason, almost always, is an engineer's claim that a state is unreachable.

That claim is a reachability question about the design, and reachability is what a model checker computes. The unreachability app takes the coverage targets and asks, for each, whether any legal trace reaches it; the unreachable ones come back with a proof, and the manual analysis they would otherwise have consumed is saved [5]. What was an opinion in a waiver becomes a result in a database. Dead RTL is excluded from coverage metrics by exactly this mechanism, manually or with formal tools, and excluding it is a precondition for any honest coverage percentage [8].

The catch is one word: *legal*. Reachability is computed relative to the constraints, and a wrong constraint makes reachable behaviour look impossible. One case is worth memorising because it is so ordinary. An input constraint hiding in a tool setup script — `enable_cnt == 0` — made a counter's increment line uncoverable; the line was reported as unreachable, and it was, under an assumption nobody had reviewed and that did not reflect the design's real environment [8]. An unreachability verdict inherits every assumption in the environment, including ones written in a scripting language by someone who has left the project. Treat the constraint set as part of the evidence, and review it as seriously as the waiver it replaces.

The second catch is structural rather than accidental: the app works on the *design* — RTL lines, branches, FSM states, cover properties bound into design scopes — so a functional coverage model computed by the testbench is invisible to it. Return to the canonical DMA coverage model, `cg_2d_4kb` from Chapter 5. Its cross `x_worst` has **72** combinations — 3 stride signs  $\times$  4 boundary relations  $\times$  3 burst lengths  $\times$  2 states of the other channel. **Six** of them, the whole `straddle  $\times$  max` column, are excluded: three stride signs by two other-channel states. They are excluded by *policy* rather than by any property of the design — Chapter 9 derives which policy. That leaves **66** cells to close.

Point an unreachability app at that cross and it sees nothing, because `txn.bnd_rel` lives in the monitor. To get a machine verdict you must restate the question where the design can answer it:

```
// Design-level restatement: can the midend ever emit a full-length
// burst out of a transaction it had to split at a 4 KB frontier?
// Signals are the midend's own outputs, not the monitor's per-row relation.
// midend_did_split is per emitted burst: set when this burst's parent
// row had to be split at a 4 KB frontier. midend_burst_len is a beat
// count, like Chapter 5's cp_len — not an AXI AWLEN, which is beats-1.
c_split_max_len : cover property (@(posedge clk) disable iff (!rst_n)
 midend_txn_done && midend_did_split && (midend_burst_len == 9'd256));
```

The comment's encoding note is the protocol's own: *AMBA® AXI and ACE Protocol Specification*, Arm IHI 0022H.c, §A3.4.1, where `Burst_Length = AxLEN[7:0] + 1` [9].

An engine answers that in seconds, and the answer is not the one the exclusion invites you to expect: the cover is **reachable**, and the engine hands back the trace that reaches it. Nothing in the design bounds a descriptor row at one maximum burst. The bound that empties the column lives in the constrained-random generator, and a design-level cover does not inherit it.

Be precise about what that does and does not overturn. The two questions are close relatives, not the same question: this cover asks about the midend's own split flag and burst length, while the six-cell figure is an argument about the monitor's per-row `bnd_rel` axis and how it pairs with a post-split length sample. The exclusion is not refuted. What is refuted is its *type*. It looked like a statement about the design and it is a statement about the stimulus — true while the generator stays as it is, false the morning someone widens it.

Which is why the app cannot discharge this exclusion for you, and the reason is structural rather than a gap you can engineer around. To return *unreachable* the tool would have to be told about the generator's budget, and that budget is not a property of the design: Chapter 9 labels its bounds engineering choices about solve time rather than statements about the DUT. Supply it as an assumption and you are back at the first catch, holding a verdict that inherits it. So the disposition stays a waiver — and here is what its last field is for. This exclusion has an expiry, and the expiry is an edit to the constraint file — which nobody counts as verification evidence until the morning it is.

The third catch generates the most paperwork: some targets are unreachable only in the configuration you are running. A parameter value can make a whole class of coverage impossible — a narrow bus cannot carry a wide transfer — and waiving that coverage by hand is messy and laborious, while aggregating coverage across parameterisations is a weak substitute [5]. Make the unreachability *conditional and generated* instead: derive coverage targets and their exclusions from the same configuration, so a parameter change regenerates both. An exclusion list maintained by hand against a parameterised design decays silently.

## 15.5 Auto-generated properties: a draft, not a specification

The fourth row of the catalogue inverts everything the first three just established, and the inversion is the most useful thing in the table. Point this app at the RTL and it extracts implementation-detail properties [5] — interface handshakes, one-hot encodings, occupancy bounds on a FIFO, the structural invariants a designer would state if anyone asked and nobody does. No table, no description, no configuration beyond a module name. Judged by the third column, it is the cheapest row there is: the intent source already exists, and it is the design.

Which is exactly what is wrong with it. Every other app reads intent from something that is *not* the design — a wiring table, a register description, a coverage model —

and the gap between the two is where the app finds bugs. This one closes the gap by construction. The limitation is structural rather than a matter of tool maturity: automatic property generation divides into dynamic mining from simulation traces and static analysis of a specification, and the mining family generates and evaluates its properties on the same RTL with no golden reference to appeal to, so flaws in the RTL yield flawed properties and the method has nothing with which to notice [10].

Follow that through and the cost model comes apart from the rest of the chapter. A mined property cannot fail on the RTL it was mined from — it proves, and it would have proved on a design with the opposite behaviour, because the behaviour is where it came from. Its bug-finding power against today's RTL is zero, and no amount of proof depth changes that. What it is worth is what a regression is worth: it converts the design's present behaviour into an obligation, so the next edit that changes that behaviour has to argue with something instead of sliding through.

That is a real asset and a poor substitute for the thing it resembles. The review question for each of three hundred generated properties is not *is it true* — it is, by construction — but *is it intended*, and answering that is specification work, property by property, done by the specialist the app was supposed to make unnecessary. An app that emits verdicts removes that work. An app that emits drafts relocates it, which is §15.1's sentence about intent curation arriving in its sharpest form: the generation is free and the review is the project. Budget the review, or do not run the app.

**Scenario.** The reference SoC's UART is inherited — silicon-proven, four years old, and its specification is a two-page register table plus the RTL itself. Bring-up needs a change detector around it before anyone touches the clock divider. **Approach.** Run the generation app, keep the several hundred properties it extracts, and adopt them as exactly that: a net that catches the next edit, never a statement of what the block should do. **Why.** Read the canonical UART escape against that set. The RX overrun flag is never cleared when software reads the status register in the same cycle a stop-bit error arrives — behaviour the RTL exhibits, so a mined property states it as an obligation, and the day someone fixes the bug the property fires as a regression. A generated set encodes the design's defects with exactly the fidelity it encodes its features. Worth having anyway, on a block whose specification is lost, provided the team knows which of the two it is holding and nobody quotes its pass rate as verification evidence.

## 15.6 Equivalence checking: the flow's quiet workhorse

Every technique in this book so far compares a design against a *specification*. Equivalence checking compares a design against another design. That sounds weaker and is worth more in practice, because the flow's most frequent transformations are ones nobody verifies any other way.

The framing that makes it click is Chapter 2's reconvergence model. Synthesis transforms RTL into a netlist; equivalence checking is an independent second path from the same origin, and the two reconverge at a mathematical proof that the transform-

ation preserved functionality <sup>[11]</sup>. No stimulus, no coverage, no oracle problem — only a question with a yes-or-no answer.

**Logic equivalence checking (LEC)** is the combinational form and the one that runs on every project. It compares two structurally similar models — most often RTL against the netlist synthesised from it, or netlist against netlist after a post-processing step such as scan-chain insertion, clock-tree synthesis or a manual edit <sup>[11]</sup>. It is an automatic activity requiring minimal human interaction, and its scalability to industrial-size circuits won it full industrial acceptance at a time when property checking was still a hope <sup>[12]</sup>.

Its assumption is the whole story, and half the engineers who run it could not state it. LEC does not reason about sequences. It matches *state points* — the registers of both models — one-to-one, then proves that for every matched pair and every output the combinational logic computing them is Boolean-equivalent. Given two sequential circuits with the same state encoding, combinational equivalence of their next-state and output functions is sufficient for their equivalence <sup>[13]</sup>. Everything follows. LEC is fast because Boolean equivalence of two similar cones is a well-solved problem; exhaustive because it never enumerates a trace; and it degenerates into a mapping problem when the state points do not match — reporting unmapped registers rather than a verdict, and comparing only the subset it could pair. Hence the first hour of a LEC run on a restructured design goes on mapping, and a design that renames or restructures its registers breaks the flow until someone supplies the correspondence.

The value proposition is blunt: equivalence checking keeps the synthesis tool honest. Synthesis tools are large software systems depending on the correctness of algorithms and library information, and history shows such systems to be error-prone. The canonical illustration is a design at Digital Equipment Corporation synthesised incorrectly because it used a synthetic arithmetic operator whose documentation stated correctness was not guaranteed above 48 bits — documentation the synthesis tool could not read. Equivalence checking found it quickly; gate-level simulation would have found it with great difficulty, if at all <sup>[11]</sup>. A functional bug in a wide arithmetic operator occupies a vanishing fraction of the input space and a whole equivalence class of the proof.

**Sequential equivalence checking (SEC)** drops the matched-state-point assumption and pays for it. Two machines are equivalent when every input sequence produces the same output sequence; deciding that means computing the reachable states of a product machine that runs both designs in lock-step on the same inputs and reports whether their outputs agree <sup>[13]</sup>. That is a model-checking problem, with model checking's costs, so SEC is aimed at bounded, well-chosen transformations rather than sprayed across a design — the ones LEC cannot see: rearchitected finite state machines and restructured data pipelines, where modern equivalence checkers handle sequential differences between two RTL models <sup>[11]</sup>. Two cases recur.

*Clock gating.* The power tool inserts gates on the neural accelerator’s 16-lane MAC datapath, and the gated design must be sequentially equivalent to the ungated one — same outputs, same timing, less energy. If the gating condition is wrong by one cycle, the datapath holds a stale value that surfaces only on a specific weight-stream boundary. SEC against the pre-gating RTL answers it once, for all inputs.

*Retiming.* The radar front-end’s FFT pipeline misses timing at the fast corner, and the fix moves a stage boundary: same arithmetic, one register deeper in one path. The function did not change and the register mapping entirely did, so LEC will not map the state points, and full functional re-verification of the DSP chain is days of simulation. SEC with a declared latency offset proves the two pipelines produce the same output sequence, and the change ships as a timing fix rather than a functional risk.

Table 15.2 holds both flows against the datapath form the next section adds, and gives each one’s assumption a row of its own, because the assumption is what decides whether a flow can run at all.

**Table 15.2** — The three equivalence flows side by side, one column each: what pair of models the flow compares, what it assumes before it starts, what it proves, and what it is blind to. Read the last row across all three columns — every one of them is blind to whether the reference itself is right, because equivalence checking is a relative argument that transfers your confidence onto another model exactly, and transfers your errors with the same fidelity.

	LEC	SEC	Datapath (§15.7)
Compares	RTL ↔ netlist, netlist ↔ netlist	RTL ↔ RTL across a sequential change	Algorithmic reference ↔ RTL
Assumes	Matched state points, same encoding	A declared input/output correspondence and latency relation	A declared operand and result mapping, bit-exact
Proves	Boolean equivalence of every cone	Equal output sequences for every input sequence	Equal result for every operand value
Blind to	Whether the reference itself is right	Whether the reference itself is right	Whether the reference itself is right

The last row holds across all three columns. Equivalence checking is a *relative* argument: it transfers your confidence in one model onto another exactly and without loss, and it transfers your errors with the same fidelity. Nothing here says the RTL is correct. It says the netlist is as correct as the RTL — a smaller claim, and the one the flow actually needs.

## 15.7 Datapath: where the input space wins outright

For control logic, simulation’s coverage argument is a reasonable approximation: the interesting behaviour lives in state sequences, the state space has structure, and a well-shaped coverage model samples it. For arithmetic the argument does not degrade — it collapses. The video codec’s inverse transform is the clean case. An 8×8 residual block is 64 coefficients; at 16 bits each, the input space of a *single* invocation is  $2^{1024}$  values. No coverage model samples that meaningfully, and no argument



from “we ran a billion random blocks” survives contact with the arithmetic. Worse, the failure mode is not a region of the space but a scatter: a rounding term applied at the wrong stage, a saturation clipping one bit late, an intermediate width one bit short — each wrong for a sparse, structured subset of operands that random stimulus meets with probability near zero. The codec’s compression standard specifies the inverse transform as exact integer arithmetic precisely so encoder and decoder cannot drift, which makes a one-bit mismatch in one coefficient a conformance failure rather than a quality issue.

What makes this tractable is that an arithmetic block’s specification is not prose. It is a mathematical relation between operands and result, and it can be written as one. The formal verification of floating-point hardware after the Pentium division episode established the pattern: a multiplier’s top-level specification is a formalisation of the arithmetic standard, stated as a relation between input operands and result in terms of arithmetic on integers, with significand and exponent vectors read as integer values rather than bit vectors. Proofs at that level are decomposed: over 120 properties covering all arithmetic operations of one industrial floating-point unit, each a property about a subunit (a loop invariant where the algorithm is iterative), with the irrelevant parts extracted away per property <sup>[13]</sup>. Two lessons carry forward: reason at word level, not bit level; and decompose, because the monolithic claim does not converge.

The industrial flow that grew from this compares an algorithmic reference — typically C or C++, bit-exact, often the code the architects used to develop the algorithm — against the RTL, with the operand and result correspondence declared by the user. That description of the flow is practitioner judgment: this book’s corpus does not document it, and no source here is being stretched to cover it.

What the corpus does supply is why it must be a *separate* flow rather than an application of §15.6’s checkers. A general equivalence checker relates an RTL model to a netlist or another RTL model because they are structurally very similar; a model written at a genuinely different level of abstraction uses a completely different modelling approach and is very difficult to correlate mathematically with the RTL <sup>[11]</sup>. Datapath tools escape that verdict only by narrowing scope until the correlation is mechanical: a pure function, a declared operand mapping, a declared latency, no side effects, no protocol. Give such a tool a C model with a state machine in it and you are back to writing properties by hand.

**Scenario.** The radar front-end’s CFAR detector accumulates FFT magnitudes in a fixed-point format the algorithm team chose from a floating-point precision study. Datapath verification proves the RTL bit-exact against the team’s fixed-point C model, on every operand. **Approach.** Ship it — and then run the *other* comparison, the one nobody asked for: the fixed-point C model against the floating-point model, over the operating envelope, with a declared error bound. **Why.** Bit-exactness against the fixed-point reference is a complete answer to “did we build the model correctly” and no answer at all to “is the model adequate”. A detector that saturates on a strong near-target loses a weak far one, and every stage of the equivalence argument passes while it happens. This is Chapter 3’s common-mode error in

its purest form — the proof tells you the RTL and the reference agree, and is silent on whether either is right — arriving in a place where the proof’s very completeness makes it easy to forget.

## IN PRACTICE

Formal apps enter a project sideways. Nobody plans them in the first vplan; someone runs a connectivity check during integration because a wire was wrong, it takes two days and finds nine more, and the next project has a connectivity table in its plan. Accelerate that trajectory by starting with the apps whose machine-readable intent already exists for another reason — the register description that generates the firmware header, the interrupt map that generates the documentation.

What the plan gains is two rows, in the shape Chapter 5 gave the DMA’s:

**Table 15.3** — The two verification-plan rows a connectivity app earns at SoC top level: the feature and its attributes, the pass/fail criterion, the method, and the metric that closes the row. There are two rather than one because “the connections exist” and “the forbidden ones do not” are two kinds of evidence with no single metric between them; note what the closure column has to spell out, since a connectivity property that ran out of budget is not a discharged row.

ID	Feature & attributes	Pass/fail criterion	Method	Closure metric
SOC-F01	Interrupt map: every row of the integration table — source, destination, enabling condition, latency — implemented as declared	Each declared connection holds for every legal value of its enabling condition	Connectivity app, SoC top level	Every generated property proven; none bounded, none waived
SOC-N01	No path from any key-ladder output to a bus-readable address, the debug data register, or a scan-observable point	No structural path exists in the cone	Connectivity app, no-connection mode	Every negative row proven; each exception dispositioned per Chapter 7

Two rows rather than one, for the reason Chapter 5 split DMA-N01: “the connections exist” and “the forbidden ones do not” are two kinds of evidence, and no single metric evaluates both. Note what the closure column has to spell out. *None bounded* is there because a connectivity property that ran out of budget is not a discharged row, and the default report will not distinguish the two for you.

The messy parts are consistent across teams. The intent table is a spreadsheet owned by whoever last edited it, and its first formal run produces dozens of failures that are table errors rather than design errors — genuinely useful, since the software team has been reading that table too, but it does not look like success on day one. Equivalence checking sits with the physical design or synthesis group rather than with verification, so the verification lead often cannot say whether last night’s netlist passed LEC; fix that, because a failing LEC is a functional regression regardless of whose queue it is in. And every app produces a list of items to disposition, so Chapter 7’s waiver discipline applies unchanged.

---

## PITFALLS

1. **Selling an app as free.** Symptom: connectivity checking is scheduled for one sprint and the intent table takes three weeks; or three hundred generated properties land and nobody costed the review. Cure: estimate the intent source *and* the disposition of the output, not the tool setup.
2. **Reading a bounded app result as a clean one.** Symptom: a run reports no violations, at a runtime limit, on a design that is too large. Cure: check the proof status of every generated property, and treat “bounded, no counterexample” as unfinished work.
3. **Trusting an unreachable verdict without its constraints.** Symptom: targets marked unreachable, and a setup script nobody has read since the project started. Cure: review the constraint set as exclusion evidence, and run over-constraint analysis before believing any hole is dead.
4. **Assuming the register app validates the register map.** Symptom: a field at the wrong offset in silicon, with a green formal report behind it. Cure: keep one human review of the description against the architecture document; the app proves conformance, not correctness.
5. **Expecting LEC to survive a restructuring.** Symptom: an equivalence run that will not map state points after an FSM re-encode or a retiming, and a team debugging the mapping instead of the change. Cure: recognise the sequential case and run SEC with a declared latency relation.
6. **Treating bit-exact equivalence as functional correctness.** Symptom: a datapath proven identical to a reference that is itself wrong, or precision-inadequate. Cure: verify the reference separately — against the standard, or against a higher-precision model with a stated error bound.

---

## SUMMARY

- The formal app works because it supplies the properties: you supply the design and a configuration, and the specification work that made property checking a specialist activity disappears into the tool [1, 2]. It disappears only where the intent already exists in machine-readable form; where it does not, curating that source is the flow’s real cost. And where the app takes its intent from the design under test, it does not disappear at all: a mined property has no golden reference to appeal to [10], so it proves by construction — a regression net rather than evidence.
- Connectivity checking is the highest-value, lowest-effort formal application on most projects, and its negative rows — what must *not* be connected — are cheaper than its positive ones and more often forgotten.
- Register checks discharge RTL against a register description exhaustively, over all values and orderings, and close nothing at all between the description and the architect’s intent.

- Formal turns a coverage waiver’s argument into a result, but the verdict inherits every assumption in the environment — including the one hiding in a setup script [8].
- LEC assumes matched state points and proves Boolean equivalence cone by cone; SEC drops that assumption, becomes a model-checking problem, and earns its cost on retiming and clock gating [11, 13].
- Equivalence is a relative argument: it transfers confidence from one model to another exactly, and transfers errors with the same fidelity. Nothing in it says the origin was right.
- Against an arithmetic datapath’s input space, simulation’s coverage argument does not degrade gracefully — it fails outright, which is why arithmetic gets a specification stated as a mathematical relation and a proof at word level [13].

---

## FURTHER READING

Within the corpus, [1] is the origin point for the automatic-formal-application category and its adoption argument, with current figures in [3, 4]. [5] is the best single overview of the app catalogue alongside the practical hurdles of formal in production, including the symmetry and specimen-value techniques that keep interconnect checks constant-sized, and [2] documents one app end to end — generation, proof, bounded-result limitations — on a real tape-out. For coverage-side formal, [8] defines the coverage models and carries the over-constraint case study every unreachability user should read before their first exclusion. On generated properties, [10] states the limitation that governs the RTL-mined variety and surveys where the alternatives take their intent from; Chapter 11 covers the specification-to-property direction. On equivalence, [11] gives the practitioner’s framing and the industrial anecdotes, [12] places it in the design flow and explains why it scaled first, and [13] is the reference treatment of the theory — state-machine equivalence via product machines, combinational equivalence under matched state encodings, and the hierarchical decomposition that made RTL-versus-transistor checking practical on a commercial microprocessor. [7] is definitive on register descriptions as a single generated source.

Outside the corpus: E. Seligman, T. Schubert and M. V. Achutha Kiran Kumar, *Formal Verification: An Essential Toolkit for Modern VLSI Design*, is the standard practitioner treatment of formal apps and their deployment. Equivalence-checking *algorithms* are the thin spot: relative to their industrial importance the published verification literature covers them lightly, and within the corpus [13] is the closest thing to a treatment.

---

## REFERENCES

1. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," Proc. 52nd Design Automation Conference (DAC '15), San Francisco, CA, 2015.
2. **D23** S. Zhao, S. Yan, and Y. Feng, "Practical Approach Using a Formal App to Detect X-Optimism-Related RTL Bugs," Proc. DVCon U.S., 2014.

3. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
4. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
5. **D13** J. Bromley and J. Sprott, "Formal Verification in the Real World," Proc. DVCon U.S., 2018.
6. **D12** N. Ahuja, M. Agarwal, and S. Jana, "Formal Assisted Fault Campaign for ISO26262 Certification," Proc. DVCon India, 2019.
7. **S2** Accellera Systems Initiative, "SystemRDL 2.0 Register Description Language," Accellera, January 2018.
8. **D11** X. Feng, X. Chen, and A. Muchandikar, "Coverage Models for Formal Verification," Proc. DVCon U.S., 2017.
9. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
10. **P11** W. Fang, M. Li, M. Li, Z. Yan, S. Liu, H. Zhang, and Z. Xie, "AssertLLM: Generating and Evaluating Hardware Verification Assertions from Design Specifications via Multi-LLMs," arXiv preprint arXiv:2402.00386v4, February 2026.
11. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
12. **B3** V. Bertacco, "Scalable Hardware Verification with Symbolic Simulation," Springer, 2006.
13. **P20** C. Kern and M. R. Greenstreet, "Formal Verification in Hardware Design: A Survey," ACM Transactions on Design Automation of Electronic Systems, vol. 4, no. 2, pp. 123–193, 1999.

---

## 16. Hybrid Flows

The flagship’s host cluster has been in verification for a quarter, and two engineers have been working on it without reading each other’s reports.

The formal report says: nine properties on the directory’s transition rules, proven unbounded on a model cut down to two caches and one address; an assumption register of five rows, four of them discharged on a neighbouring block. Among the nine is the invariant that at most one cache may hold a line Modified (see Chapter 14).

The simulation report says: four hundred runs of random traffic across four application cores, a functional cross of MESI state against requester against concurrent-snoop standing at 71%, a scoreboard that has been silent for three weeks, and the note that the remaining holes are all in the eviction column.

The lead’s dashboard merges both databases and prints one number. That number is not wrong about anything, and it is not a claim about anything either. Ask what its denominator is and nobody at the review can say. Ask the sharper question — does the proof of the single-writer invariant fill the cross bins that ask the same question? — and you get two confident, opposite answers from two competent engineers.

Neither of them is doing anything wrong. The flow is. There is one design, one plan and one sign-off, and the two engines have been run as two projects that meet for the first time at a dashboard. This chapter is about running them as one campaign: dividing work by the shape of the problem rather than by which team owns which tool, keeping one notion of legal traffic across both, and reporting the result in a form that claims exactly what was established and no more.

---

### CHAPTER MAP

- §16.1 assigns a plan row to an engine from properties of the problem: the shape of its state space, whether the interesting behaviour is a corner or a volume, what a witness costs, and where the criterion is observable.
- §16.2 follows one interface constraint as it becomes stimulus, a checker and a formal assumption, and the two detectors that keep those three from drifting apart.
- §16.3 establishes why merging simulation coverage with formal results is genuinely hard, what an interchange database can carry across engines, and what a unified report may and may not claim.
- §16.4 gives the decision procedure for choosing an engine per row — including the case where the answer is deliberately *both*, and the case where formal’s answer is cheaper but does not discharge the row.
- §16.5 states what travels between a formal engineer and a simulation engineer in each direction, and what has to travel with it.



## 16.1 Dividing the work

The question a lead is actually asked is never “formal or simulation for this block”. Blocks are not homogeneous: Chapter 14 split a video codec down the middle, and Chapter 15’s apps discharge integration questions independently of how the blocks either side of the wire were verified. The question is asked one plan row at a time, and Chapter 5 already put a field on the row for the answer. What follows is how to fill it in.

**The shape of the state space the criterion ranges over.** Chapter 14 stated the rule in one line — formal excels where the specification is a set of rules over a bounded amount of state, and fails where the specification is a transformation of data. The operational reading is grammatical. Read the row’s pass/fail criterion and look at how it quantifies. A criterion that begins *for every* — every burst lies inside one 4 KB frame, every grant follows a request, at most one cache holds this line Modified — is a claim over all traces, and an engine that samples can only ever provide evidence for it. A criterion that begins *for this traffic* — the destination buffer matches the source under a mixed read/write profile, the encoder’s output decodes bit-exactly on this stream — is a claim about a transformation, and its oracle is another executable (Chapter 8). The first kind of row is a formal row that simulation can corroborate. The second is a simulation row that formal cannot express without rewriting the reference model in temporal logic.

**Corner or volume.** A *corner* is a conjunction of individually rare conditions. Its cost in simulation is a product of probabilities, and no amount of seed grinding changes the arithmetic; its cost in formal is a search that does not care how rare the conjunction is. One published deployment documents the shape precisely: an arbiter starved a requester only when the output FIFO had filled so that both ports were unavailable, *and* all three clients then asserted request continuously, *and* the two ports subsequently became available on alternate cycles — input conditions the authors record as next to impossible to construct in simulation <sup>[1]</sup>. A *volume* is the opposite: a statement about aggregate behaviour over many transactions — sustained throughput, queue occupancy under load, fairness across a frame, energy per operation. There is nothing there to prove, because the criterion is statistical. Formal verification generalises and abstracts design behaviour, which is exactly why it complements a technique that can only ever stimulate a small portion of the design and may therefore miss errors surfacing under rare conditions <sup>[2]</sup>; it is also why it has nothing to say about how often something happens.

**Scenario (the GPU).** Two rows of the tile-based mobile GPU’s plan, on the same interface between a shader cluster and unified memory. Row one: *no shader cluster may observe a write reordered past a barrier it issued*. Row two: *at 90% memory-bus utilisation, the 99th-percentile shader stall waiting on a tile fetch stays under N cycles*. **Approach.** Row one goes to formal, on the two-symbolic-address memory model of Chapter 14 — the rule does not depend on any stored value, so memory can be uninterpreted; two addresses are the fewest that state an ordering relation at all. Row two goes to simulation with a traffic profile reprodu-

cing a real frame, and its closure metric is a distribution, not a bin. **Why.** The first is a corner: it needs a specific interleaving of a barrier, a write and a read from a different cluster, and its probability under random stimulus is the product of three small numbers. The second is a volume, and rewriting it as an all-traces property changes the question — a proof engine asked “can this stall exceed  $N$ ?” returns a worst case that exists, occurs once in a billion frames, and was never what the row was about. Two rows, one interface, two engines, and neither engine could have taken the other’s row.

**What a witness costs, in both directions.** A counterexample arrives near-minimal and carries no irrelevant history (Chapter 14); a simulation failure arrives at the end of a run with everything that preceded it attached, which is Chapter 8’s detection distance charged at debug time. The reverse asymmetry is less often priced: when the answer is *no violation*, simulation’s cost is predictable and formal’s is not. A few minutes of simulation gives a good estimate of clock cycles per second for the rest of the run, whereas if an assertion is slow to prove it is almost impossible to predict how much longer it will take <sup>[3]</sup>. So a lead may put a simulation row on the critical path and estimate it; a formal row there is an unbounded commitment unless the plan says in advance what happens when it does not converge — which is why Chapter 14’s convergence ladder ends with *stop, and hand the row to simulation* as a planned outcome.

**Where the criterion is observable.** Criteria statable over internal state — a queue’s occupancy against its pointers, a one-hot encoding — are cheap in both engines, which is why Chapter 11’s redundant-state invariants have a second career as formal lemmas. Criteria over a whole transaction’s payload are expressible for a proof engine and rarely tractable for one. Criteria over software behaviour belong to neither engine at this level (Chapter 18).

Read together, these four properties reassign work across a generation. On the reference SoC’s  $5 \times 4$  crossbar, the ordering and arbitration rules are small enough that end-to-end proofs close and simulation’s job is data transport at volume. On the flagship’s  $4 \times 4$  mesh, the same *class* of obligation — end-to-end ordering between two endpoints — spans sixteen routers, virtual channels and XY routing, and does not close as one proof; deadlock-freedom goes to formal on an abstracted model, ordering goes to formal on a symbolic endpoint pair, and everything about congestion, latency distribution and virtual-channel occupancy goes to simulation and emulation (Chapter 17). Same problem, two scales, and the split moves.

## 16.2 Assumption and constraint duality, as a flow

Chapter 9 named the duality: one constraint generates legal traffic on a block’s input, checks legality on the neighbour’s output, and is assumed in formal. Chapter 11 wrote it as an assertion, Chapter 14 as an assumption. What none of them owns is the operational problem, which is that stimulus legality lives in a `constraint` block inside a randomizable class while the checker and the assumption are temporal properties. Those are not one statement in two syntaxes — one is a predicate over a

generated object, the other over a signal history — so no tool diffs them and no compiler complains when they diverge. Teams discover the divergence at sign-off, or never.

### The drift is asymmetric

Two directions of drift, and only one of them announces itself.

**The generator is narrower than the assumption.** The formal environment permits traffic the tests never send. This is sound: the proof covers a superset of what simulation exercised, so nothing is claimed that is not established. It shows up honestly, as a coverage hole on the simulation side, and Chapter 6’s hole classification handles it.

**The assumption is stronger than the generator’s legal set.** The proof holds over a subset of the traffic the tests actually send — so simulation is exercising behaviour that the proof never covered, while both reports are green. Nothing on either side is empty. There is no hole, no failure, and no artifact anywhere that records the gap. This is Chapter 14’s quiet over-constraint arriving through a second door: not an assumption written too strong for the design’s environment, but one written accurately for an *older* environment that the stimulus has since outgrown.

### Two detectors

The cure is not to write the constraint once — you cannot, the languages differ. It is to make one of the three the source of truth and mechanically test the others against it. Take the *assumption* as the source, because it is the one whose failure is silent.

**Detector one: compile the assumption set into the simulation environment as assertions.** A published sign-off methodology names exactly this among the techniques for establishing that a formal environment has no unintentional over-constraints — instantiating the constraints in simulation, alongside cross-proof with neighbouring blocks [4]. The mechanics are Chapter 14’s `ASSUME_MODE` parameter pointed the other way. In formal, the parameterised checker constrains an input. In the block’s own simulation, the identical text is *asserted* — and because the signals it watches are driven by the testbench, that instance is checking your generator, not the design.

Here is that pattern at its smallest, on the sensor hub’s wake path. The always-on island raises `wake_req` and the DSP domain answers with `wake_ack`, both sampled on the 32 kHz clock (Chapter 14 proved the island’s wake latency on this clock rather than buying depth on the fast one):

```
// Sensor hub wake path: the always-on island requests, the DSP domain answers.
module wake_handshake_checker #(parameter bit ASSUME_MODE = 1'b0) (
 input logic clk_slow, rst_n,
 input logic wake_req, wake_ack
);
 // The requester may not withdraw a request that has not been acknowledged.
 property p_req_holds_until_ack;
 @(posedge clk_slow) disable iff (!rst_n)
 (wake_req && !wake_ack) |>= wake_req;
```

```

endproperty

// Has the environment ever come near the rule's edge?
c_req_waited : cover property (@(posedge clk_slow) disable iff (!rst_n)
 wake_req && !wake_ack);

if (ASSUME_MODE) begin : g_env
 m_req_holds_until_ack : assume property (p_req_holds_until_ack);
end else begin : g_chk
 a_req_holds_until_ack : assert property (p_req_holds_until_ack);
end
endmodule

```

Read the property before believing anything about it. It says only that an unacknowledged request survives into the next cycle; it says nothing about what happens after `wake_ack` rises, and it never constrains `wake_ack` at all. That is the whole rule, and it is enough for both jobs. When the DSP's wake controller is proved, `wake_req` is an input and the instance is built with `ASSUME_MODE = 1`. When the island is proved, `wake_req` is an output and the same text is asserted. And in the DSP's block-level simulation the assert-mode instance is bound at the DSP's scope, where `wake_req` comes from the testbench:

```

bind dsp_wake_ctrl wake_handshake_checker #(.ASSUME_MODE(1'b0)) u_wake_chk (
 .clk_slow(clk_32k), .rst_n(rst_n_32k),
 .wake_req(wake_req), .wake_ack(wake_ack)
);

```

If a sequence ever drops an unacknowledged request, that assertion fires in simulation and the message names the assumption that would have been silently violated in formal. The generator has drifted outside the proof's environment, and you find out on a nightly run rather than at sign-off.

**Detector two: cover the assumption's boundary.** Detector one catches the generator going *outside* the assumption; it says nothing about the other direction, the generator staying so far inside that the assumption was never tested. `c_req_waited` asks that: has any run ever produced an unacknowledged request at all? If it reads zero, `a_req_holds_until_ack` has never evaluated non-vacuously, and the claim that simulation corroborates the formal environment is empty. This is Chapter 11's rule that every implication ships with a cover on its antecedent, applied to a third artifact: **every assumption ships with a cover on the condition it restricts, and an assumption whose boundary is never approached in simulation is untested.** Chapter 11's caveat travels with it: where the restricted condition is ordinary traffic, a bare cover proves nothing, and the rule is discharged instead by a witness that implies the condition and isolates the interesting case.

Together the two detectors turn Chapter 14's assumption register from a reviewed document into a monitored one. Each row gains a column: the run in which its assertion form last passed, and the run in which its boundary cover was last hit.

**Scenario (the crypto engine).** Chapter 14 left the inline AES-GCM engine with a dangerous assumption — that key material arrives only after authentication completes — and the verdict that the obligation belonged on the boot ROM’s proof rather than on the engine’s. **Approach.** Move it, in three steps that are all book-keeping. The assumption’s text becomes an assertion in the boot ROM’s own formal run, where `key_load` is an output and the claim is discharged rather than enjoyed. The same text, in assert mode, is bound into the SoC-level simulation environment, so every firmware-driven boot sequence in the nightly regression re-checks it against real traffic. Its boundary cover — a key load attempted at all — goes on the same row, because an obligation nobody’s tests ever approach is not evidence. The engine’s assumption register row keeps the text and gains a discharge pointer. **Why.** In a security proof every assumption is a concession to an adversary who has read it, so “assumed on the engine, proven on its driver, re-checked in system simulation” is the difference between a proof and a promise. Note also which direction the flow runs: an artifact left over from a formal convergence problem became a permanent member of a simulation regression tier (Chapter 12). That is the ordinary case, not the exotic one.

### 16.3 Coverage unification

This is where the two engines’ vocabularies collide, and where most hybrid flows quietly overstate what they have.

A simulation bin holds a count: *this situation occurred, n times, in the runs I performed*. A proof holds a claim: *no legal trace violates this rule*. They are not two measurements of one quantity — one is a statement about your testbench and the other about your design, which is Chapter 14’s opening distinction arriving as an arithmetic problem. Average them and the resulting percentage has a denominator made of two incompatible units.

#### What the interchange standard actually provides

The mechanism is more subtle than “one database, one number”. The interchange standard — the Accellera Unified Coverage Interoperability Standard (UCIS), Version 1.0, 2012 — defines a single repository holding coverage data from all verification processes — simulation, static checking, functional formal verification, sequential equivalence checking, emulation — together with a standard access layer, so that data produced by many producers is semantically interpretable by many consumers. Its target flow is primary data collection in memory by the simulator or the formal tool, exported on completion, with those primary databases later combined by merge applications whose construction history is recorded in the database itself <sup>[5]</sup>.

What it carries for a formal run is exactly Chapter 14’s three qualifiers, made machine-readable. An assertion’s result is one of a small enumeration — no result, failure, proof, vacuous, inconclusive, assumption, or a merge conflict (UCIS 1.0 §8.19.3) — and a *radius* travels with it: a bounded proof’s depth or a counterexample’s length, recorded together with the name of the clock it is measured in (UCIS 1.0

§8.19.4–§8.19.5). Alongside sit the assumptions used, the restrictions, the initialisation sequence, initial-value abstractions and any cutpoints inserted to achieve the proof [5]. Read that list against §14.6’s sign-off package and it is the same list — carried in the schema rather than left to a report template, which is the schema conceding that a proof stripped of its qualifiers is not a result.

Three kinds of merge are distinguished (UCIS 1.0 §2.1.2), and only two of them are mechanical. A *temporal* merge combines runs of the same process on the same design. A *spatial* merge combines coverage from different parts of a design. A *heterogeneous* merge combines data from different verification processes — and the standard’s guidance is deliberately cautious rather than prohibitive: such items are more often semantically different than equivalent, so they should not be automatically merged, and an analysis tool that does have a merging algorithm should record the result as a *new* item carrying a semantic tag that says it is merged data. The merging algorithm itself is left unspecified (UCIS 1.0 §4.3.1) [5]. That is a licence with a condition attached. You may build a combined view; you may not let it masquerade as a primary measurement.

One more property of the database decides how much a merged number can ever mean. A metric is the rule assigning observed values to bins, and different metrics over the same data produce very different percentages: an eight-bit variable that took the values 17 and 96 scores 2/256 under a metric with one bin per value and 2/8 under an eight-bin auto-binning metric. Collection loses information that cannot be recovered afterwards [5]. Percentages are therefore properties of models, not of designs — Chapter 6 made that point about a single simulation model, and it compounds when the denominators come from two engines that never agreed on a partition.

## What a unified report may and may not claim

**Table 16.1** — What a cross-engine coverage report may and may not claim, sorted into three dispositions: quantities that genuinely add because both engines report hits against the same structural denominator, results that travel only as tagged items carrying their qualifiers, and pairs that must never be summed at all. The left column is where a hybrid flow earns its keep; the right column is where confidence gets manufactured, because writing a proof’s verdict into a functional bin looks like progress and blinds two metrics at once.

Merges in one unit	Travels as a tagged item, with its qualifiers	Must never be summed
Code coverage: lines, branches and conditions — formal reachability and simulation both report hits against the same structural denominator	An assertion’s result plus its radius, assumptions, abstractions and cutpoints	Functional coverage bins against proofs — a hit count and a proof answer different questions
Assertion coverage: the same property’s pass and vacuity counts from simulation runs	An unreachability verdict on a coverage target, with the constraint set that produced it	Two percentages whose metrics partition the space differently
Plan-row status: each row closed by <i>its</i> declared metric, whichever engine produced it	A merged view, provided it is recorded as a derived item and labelled as one	A proof’s “100%” into a functional bin the tests never filled



The left column is where hybrid flows earn their keep, and the strongest evidence for it is the least glamorous metric in the book. Formal reachability produces line, branch and condition coverage in precisely the units simulation uses, so those genuinely add. In the published sign-off case study of a multicast crossbar parameterised to eight clients and eight targets, coverage analysis run from the completed formal testbench alone reported all 288 lines and all 88 conditions covered, with the deepest cover point at depth 5 — comfortably inside that project’s derived required proof depth, which the authors take as evidence the depth is not optimistic <sup>[4]</sup>. A second deployment mandated line, statement and branch coverage at the end of every formal exercise and found real holes doing it <sup>[1]</sup>. Neither treats structural coverage from a proof engine as functional evidence; both use it for what it is, a check that the properties and the constraints between them actually reach the logic. Note that this is not one of Chapter 14’s four formal coverage models: it asks that question in simulation’s own units. The measured version of the same worry does come from those models, and it is the sharpest number in this chapter. When Oracle Labs ran cone-of-influence coverage across every register of a design of its own, 18% of the registers — 1,375 of them — turned out to lie in the cone of influence of no assertion at all <sup>[6]</sup>. Nothing had failed. Every proof in that testbench was still valid, and a fifth of the design’s state was simply outside anything the property set was asking about, which is exactly the residue the other engine has to be given rather than assumed to be covered.

The right column is where confidence gets manufactured, and writing a proof’s verdict into a functional bin is the failure worth naming, because it looks like progress and destroys two instruments at once. The bin’s job is to record what the stimulus did; overwrite it and Chapter 7’s coverage position can no longer corroborate a flat bug curve, while the coverage differential of Chapter 12 can no longer fail a commit that quietly made the suite ask less. You have not gained coverage. You have blinded two metrics to buy a number.

### The reconciliation

The honest answer is not a better merge algorithm. It is that **the unification layer is the verification plan, not the coverage database**. Chapter 5’s one-row-one-metric rule was written for exactly this: a row is discharged by its own declared evidence, and a row discharged by a proof closes on the proof, with its depth, its assumption list and its own formal coverage evidence attached (§14.6), while a row discharged by a covergroup closes on the covergroup. That is the honest form of *proven, so no coverage needed*: the row needs no functional **bin**, because the proof discharges what the bin was sampling towards — and it still owes the formal coverage models, which is why a proof never closes a row on its own. Nothing has to be averaged, because nothing was ever a fraction of the same whole. What the dashboard reports is rows closed out of rows planned, per engine and in total — a number with a denominator anyone at the review can name.

That leaves the coverage database to do the two things it does better than a plan ever could. The first is the structural coverage above. The second is the most useful cross-engine mechanism in the standard and the one teams most often leave

switched off: a coverage item can be flagged as *formally unreachable* with respect to a particular test, and the classification distinguishes a don't-care from a *predicted-unreachable* point precisely so that a warning is raised if a predicted-unreachable point is later covered [5]. Read what that buys. Chapter 15 turned a waiver's argument into a proof of unreachability; this turns the proof into a live tripwire. If a later RTL change, a relaxed constraint or a new test reaches a point the proof said was dead, the regression says so — instead of the exclusion sitting in a file, still true about a design that no longer exists.

**Scenario (the flagship's host cluster).** Return to the opening. The lead stops asking what the merged number means and rebuilds the report row by row. The directory transition rows close on proofs, each recorded as unbounded on the two-cache one-address abstraction. The rows that abstraction deleted — the two-address deadlock, conflict eviction and capacity writeback, the third sharer and the sharer-vector overflow — are simulation rows, and the middle pair is exactly the eviction column where the cross was empty. The cross is then re-scoped to the rows simulation owns — a reviewed coverage-model change with its justification recorded (Chapter 6) — and the structural coverage from both engines merges into one line/branch figure. **Approach.** Report two closure numbers and one merged structural number, never one blended percentage. **Why.** The 71% was never the problem; the problem was that nobody could say what the missing 29% was owed to. Once each hole belongs to a row and each row to an engine, the number stops being a temperature and becomes a work list.

## 16.4 Choosing an engine, row by row

A decision procedure a lead can apply in a plan review, in the order the questions actually break ties. Run it per row, not per block.

1. **Does the criterion quantify over all traces, or over a traffic profile?** (§16.1.) Profile criteria stay simulation rows however elegant the property would look.
2. **Is the interesting behaviour a corner or a volume?** (§16.1.) A conjunction of rare conditions goes to formal even when the block is large, because the abstraction that makes it converge usually preserves the conjunction.
3. **Can the block's environment even construct the witness?** If the block-level testbench cannot produce the scenario at all — Chapter 6's bin that needs corrupted authentication tags, or a coverage cross that needs contention no block bench can generate — the row is not a stimulus-tuning problem: it needs a testbench capability the block bench does not have (Chapter 6), formal, or a higher verification level.
4. **What does the row's model have to contain?** If the property needs the whole datapath, the memory and the firmware that programs them, the abstraction bill of §14.4 will be large and the deletion list long. Price it before committing.
5. **Who owns sign-off for this row?** Sometimes the answer is nobody on your project.

**6. What happens if it does not converge?** Answer this in the plan review, not in week nine.

Question five is the one that catches teams out, and it is not about completeness.

**Scenario (the USB controller).** The dual-role USB 3.x controller's link-training and low-power rows — the U0 through U3 transitions — are as good a formal target as exists: a state machine with published transition rules (*Universal Serial Bus 3.2 Specification*, Revision 1.1, sections 7.5.6 to 7.5.9) <sup>[7]</sup> over bounded state. A proof of them is cheap, fast and finds real bugs. **Approach.** Run it, and do not let it close the rows. Chapter 5 sends those rows to the compliance suite defined by the USB compliance methodology, whose item list is their closure metric; the proof enters the package as corroborating evidence and as a bug-finding instrument available months before the suite can run. **Why.** *Cheaper but not sufficient for sign-off* is not a criticism of formal here — the proof is stronger evidence than the suite for the rules it covers. It is a statement about who holds the authority to declare the row done, and on a compliance row that authority is external to your project. Substituting your own evidence for it is a plan change, not a method choice.

### Both, on the same feature, deliberately

The most misread outcome of this procedure is the row that gets two methods. It looks like waste and is usually the opposite, because the two engines fail differently.

The evidence is unusually clean. One industrial deployment — Intel's, on a live graphics project — began its formal effort *after* a major dynamic-verification milestone had completed, on designs whose simulation environments were already stable and running — precisely the situation in which a second method should find nothing — and reported 82 interesting bugs over twelve weeks. Its own taxonomy is the useful part: the easy finds were things like a state machine attempting two arcs at once, which in simulation would have surfaced as an X-propagation problem and been hard to trace; the medium ones were counter overflows and the arbiter starvation of §16.1; the hard ones needed a deep internal counter value coincident with a specific input sequence. The authors note that even the easiest of these would have been difficult for the running simulation environment to find <sup>[1]</sup>. Chapter 14's opening is the same story on this book's own crossbar: six weeks of green nightly regression, the hazard's precondition cover reading zero, and an eleven-cycle counterexample overnight.

So a deliberate double assignment is a risk decision, and it is written the way Chapter 5 writes any other: two rows, two metrics. The DMA's 4 KB legalization is the canonical pair in this book — the coverage row closes on its covergroup under constrained-random stimulus with a scoreboard, and the separate proof row closes on the boundary property proven on the backend standalone. Neither row's evidence substitutes for the other's, which is exactly why they are two rows.

## The contrast pair

The coherent hub as a standalone IP and the flagship's host cluster are the same protocol family at two scales, and the split moves with the scale rather than with the technique.

On the hub, block-level, the two-caches-one-address abstraction carries a large fraction of what makes coherence worth proving — the directory's transition rules, the races between two requesters, the ordering of invalidations against data responses, and starvation on a contended line — and Chapter 14 enumerated what it deletes. Simulation's job at that level is small: the deleted classes, plus data integrity through the datapath.

In the flagship, the same proofs run on the same abstraction, and everything they delete gets harder rather than easier. Conflict eviction now involves a four-slice L2 and a real replacement policy; the third and fourth sharers are real cores; and a new class appears that the hub never had — a mis-legalized DMA transfer whose second half writes a line a core holds Modified, the flagship descendant of this book's canonical DMA bug, invisible until the writeback and reachable only under a running software workload. The proof does not lose value at scale; the simulation half gains share. That is the general shape: as a system grows, formal's rows stay roughly where they were and simulation's multiply.

## 16.5 The handoff

---

Hybrid flows are made of artifacts crossing between two engineers. Each has a form, and each carries bookkeeping that must travel with it or the handoff decays into a conversation nobody remembers.

### Formal to simulation.

*A counterexample becomes a directed test.* The trace is near-minimal, so the test derived from it is short, and it is exactly the stimulus a random generator was demonstrably not producing. It joins a regression tier (Chapter 12) so the fix stays fixed. What must travel with it: the property it violated, so the test's failure condition is the assertion rather than a hand-written expectation, and a cover on the scenario, so a later constraint change that quietly stops reaching it is visible. A GPU memory-ordering counterexample makes the point — five cycles of barrier, write and cross-cluster read that the frame-driven traffic profile had never aligned. As a directed test it takes an afternoon and runs forever; as a debug session it takes a week and is forgotten.

*An unreachability proof becomes a waiver's justification.* Chapter 15 established the verdict and its main caveat, that reachability is computed relative to the constraints. What §16.3 adds is where the verdict then lives: not only in the waiver's reason field but as a flag on the coverage item, with the classification that raises a warning if the point is ever covered later. What must travel with it: the constraint set, as evidence, reviewed as seriously as the waiver it replaces.

*An assumption becomes a checker.* The §16.2 mechanism, and the most valuable of the three because it runs every night. What must travel with it: the assumption register row it discharges, and the boundary cover.

### **Simulation to formal.**

*A coverage hole becomes a reachability question.* A hole that has resisted three targeted tests is the classic candidate. Restate it as a cover property at design level — Chapter 15 showed the restatement and warned that a monitor-side functional axis is invisible to a design-level engine — and let a proof engine answer. Two of the three outcomes are decisive: a witness hands the stimulus engineer a recipe — provided the generator’s own constraints permit the trace it describes — and unreachable converts the hole into an exclusion with a proof behind it. Inconclusive leaves the hole exactly where it was, which is worth knowing before spending a week on it.

*A long failing trace becomes a property.* When a simulation failure is reproducible but the trace is thousands of cycles, formulating the violated rule as a property and running a bounded proof on the block often returns a counterexample an order of magnitude shorter. What must travel with it: the seed and run manifest of the original failure, because the short trace is a different execution and someone must confirm it is the same bug.

*A redundant-state invariant becomes a lemma.* Chapter 11’s invariants are exactly the facts a proof engine must be told to stop generating impossible starting states (Chapter 14), and they cost nothing to hand over because they are already written and already believed.

---

## **IN PRACTICE**

The two flows do not naturally run at the same cadence, and that is the first thing to fix. Chapter 14 argued for giving formal a regression tier of its own; the hybrid consequence is sharper, because a row’s evidence is only as fresh as the slower of the two flows that touch it. One published deployment kept its proofs and scripts in a repository and re-ran them on a fixed interval, reporting the trend to the owner <sup>[1]</sup>.

The messy parts are consistent. The two engineers report to different leads more often than not, and the assumption register is the artifact that falls between them; give it one owner. Dashboards are bought rather than built, and most will happily print a blended percentage — decide your project’s headline number before the tool decides it for you. Structural coverage from formal runs is usually available and usually switched off, because nobody asked for it. And on staffing: every handoff in §16.5 needs someone who can read both kinds of report, so a project with exactly one such person has a bottleneck rather than a hybrid flow.

---

## PITFALLS

1. **Assigning engines to blocks instead of rows.** Symptom: “the crossbar is a formal block” and its data-transport rows have no owner. Cure: the method field is on the row, and a block routinely needs both.
  2. **A proof written into a functional bin.** Symptom: coverage jumps when a proof closes. Cure: close the plan row on the proof; leave the bin measuring what the stimulus did, or the differential and fresh-seed metrics stop working.
  3. **A blended percentage with no nameable denominator.** Symptom: nobody at the review can say what the number is a fraction of. Cure: report rows closed per engine, plus one genuinely shared structural figure.
  4. **An assumption nobody’s tests ever approach.** Symptom: the assumption’s assertion form passes in every simulation, and its boundary cover reads zero. Cure: treat the empty cover as the failure; a rule the environment never nears is untested in both engines.
  5. **A counterexample debugged and not captured.** Symptom: the bug is fixed, the trace is in a chat thread, and no regression member exists. Cure: every counterexample that led to an RTL change leaves a directed test and a cover behind.
  6. **A stale unreachability exclusion.** Symptom: a coverage exclusion justified by a proof against an RTL revision three months old. Cure: flag it in the database as predicted-unreachable so a later hit warns, and re-run the proof on the same cadence as the regression that consumes it.
- 

## SUMMARY

- Assign engines per plan row, on properties of the problem: how the criterion quantifies, whether the behaviour is a corner or a volume, what a witness costs, and where the criterion is observable.
- Simulation’s cost when nothing fails is predictable; formal’s is not [3]. That is a scheduling fact, and it belongs in the plan rather than in week nine.
- One interface rule lives three lives, and the dangerous drift is silent: an assumption stronger than the generator’s legal set leaves both reports green and no artifact anywhere recording the gap.
- Two detectors close that gap — the assumption set compiled into simulation as assertions [4], and a cover on each assumption’s boundary.
- A hit count and a proof answer different questions, and the interchange standard says so in its own terms (UCIS 1.0 §2.1.2): heterogeneous data should not be automatically merged, and a merged result should be recorded as a new, tagged item [5].
- Unify at the verification plan, not at the coverage database: rows closed out of rows planned has a denominator anyone can name, and a blended percentage does not. The database still earns two cross-engine jobs — structural coverage in



one unit, and the formally-unreachable flag that warns when a supposedly dead point is later covered [5].

- A deliberate double assignment is not wasted effort. Formal deployed onto stable, already-simulated designs still finds bugs the running environment could not reach [1].

---

## FURTHER READING

Within the corpus, [5] is the primary source for §16.3's account of the interchange standard: the unified-repository model, the formal result enumeration and radius, the temporal/spatial/heterogeneous merge distinction with its caution about automatic merging, the metric-dependence of percentages, and the formally-unreachable API with its predicted-unreachable classification. [4] works a formal sign-off end to end and is the source for instantiating constraints in simulation as an over-constraint detector, and for structural coverage measured from a formal testbench. [1] is the best account here of formal deployed alongside an established simulation flow — its bug taxonomy by difficulty is the clearest published statement of what a second engine buys on already-verified logic. [3] is the practitioner's tour of formal in production, and the source for the predictability asymmetry between the two engines. [2] places simulation and formal on one spectrum and develops symbolic simulation as a genuine hybrid engine — a third point between the two this chapter treats.

Outside the corpus: E. Seligman, T. Schubert and M. V. Achutha Kiran Kumar, *Formal Verification: An Essential Toolkit for Modern VLSI Design*, treats the deployment questions of §16.4 at book length; and H. Foster, A. Krolnik and D. Lacey, *Assertion-Based Design*, is the standard treatment of the artifact this chapter keeps moving between engines — the property written once and read by both.

---

## REFERENCES

1. **D18** M. V. Achutha Kiran Kumar, E. Seligman, A. Gupta, Ss Bindumadhava, and A. Bharadwaj, "Making Formal Property Verification Mainstream: An Intel Graphics Experience," Proc. DVCon U.S., 2017.
2. **B3** V. Bertacco, "Scalable Hardware Verification with Symbolic Simulation," Springer, 2006.
3. **D13** J. Bromley and J. Sprott, "Formal Verification in the Real World," Proc. DVCon U.S., 2018.
4. **D31** I. Tripathi, A. Saxena, A. Verma, and P. Aggarwal, "The Process and Proof for Formal Sign-off: A Live Case Study," Proc. DVCon U.S., 2016.
5. **S3** Accellera Systems Initiative, "Unified Coverage Interoperability Standard (UCIS), Version 1.0," Accellera, June 2012.
6. **D11** X. Feng, X. Chen, and A. Muchandikar, "Coverage Models for Formal Verification," Proc. DVCon U.S., 2017.
7. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.

PART V

## Beyond RTL Simulation

---

*What to do when the simulator runs out of cycles: acceleration and emulation, prototypes running real firmware, verification after synthesis, and the first encounter with real silicon.*

---

## 17. Acceleration and Emulation

The flagship SoC’s full-chip integration milestone has one acceptance criterion, written eighteen months earlier and never disputed since: the four application cores boot the operating system to a shell prompt, with the mesh NoC carrying real traffic, the LPDDR4 subsystem out of PHY training, and the flagship’s DMA moving data under the host cluster’s coherence protocol. Everyone agrees it is the right criterion. It is, after all, the first moment at which the thing behaves like the product.

What breaks the milestone is arithmetic. The reference SoC had an equivalent gate — the core boots, the DMA moves a buffer, the UART prints — and it was a bare-metal image of a few tens of kilobytes and a couple of million cycles, comfortably inside the eleven-minute mean of that project’s nightly runs (see Chapter 12). The flagship’s boot is an operating system: page tables, drivers, an interrupt controller, a filesystem — several seconds of real execution, which at any application-class clock is billions of cycles. The verification lead runs the numbers and reports that the simulation farm cannot deliver that boot before tape-out. Not because the farm is too small. Because a boot is *one run*, and one run is serial: two hundred and forty simulator licences shorten a suite and do nothing whatsoever to shorten a single test.

This is the moment a project either buys into hardware-assisted verification or admits it will meet the operating system for the first time in the lab. The decision is expensive in both directions and is made badly more often than well — usually by a team that has mistaken a slow testbench for a workload that cannot be simulated. This chapter is about telling those two apart, and about what you get and give up when you move a design onto a machine that runs it a thousand times faster and lets you see almost none of it.

---

### CHAPTER MAP

- §17.1 states why simulation runs out of cycles, as arithmetic a reader can reproduce rather than as a truism about “big designs”.
- §17.2 establishes what an emulator, a prototyping platform and a simulator each cost — in compile time, in visibility and in debug ergonomics — and which of those costs actually shapes the workflow.
- §17.3 develops the two use models, in-circuit and virtual, what each buys, what each forecloses, and why the industry drifted toward the virtual one.
- §17.4 establishes why the transactor boundary, not the platform’s clock rate, decides throughput, with the arithmetic that says how much batching can possibly buy.
- §17.5 treats how a shared, scheduled, capital-intensive machine changes verification planning, and how to recognise the common case where the honest answer is a simulation problem, badly framed.

## 17.1 Why simulation runs out

Start with the ratio, because everything else in this chapter is a consequence of it. Simulation with RTL models is approximately a billion times slower than the target clock speed of the system it models. An activity taking a few seconds of execution on the target silicon — booting an operating system is the canonical example — takes several years on an RTL simulator, which precludes running software on top of an RTL model at all and confines RTL simulation to short directed or random tests. Map the same RTL onto a reconfigurable fabric or a purpose-built emulator and it runs about a hundred to a thousand times faster than the simulator, which puts that operating-system boot back inside a few hours <sup>[1]</sup>.

Read those as order-of-magnitude claims, because that is what they are: the speed-up band spans a factor of ten, and where you land in it depends on design size, on how much of the system is instantiated, and on how much instrumentation you compiled in. Nothing in the argument turns on the exact ratio. Whatever point you pick, the same conclusion falls out — the boot is *years* of one simulation and *hours* of one emulation run, and no reordering of the numbers moves it into a nightly window.

**Why the ratio exists, and why it stopped improving.** An event-driven simulator turns the design’s parallelism into its own serialism: every signal change is a software event, and every process sensitive to that signal is re-evaluated in sequence. A million gates switching in the same design cycle are a million pieces of work for one processor. For two decades the industry absorbed this by riding processor frequency, but as frequency scaling plateaued, simulation-based techniques stopped keeping pace with design complexity — most visibly on large designs that instantiate both software and embedded processor core models — which is why acceleration techniques are now essential rather than exotic <sup>[2]</sup>. The obvious way out does not exist either: an individual RTL simulation does not usefully exploit multiple cores <sup>[3]</sup>. The compute you can throw at one run has been flat for years.

**The multiplier nobody prices.** The arithmetic above is for a single execution of a single workload. Real verification does not consist of one execution of anything. Chapter 12’s flagship nightly *candidate* list is 6,400 simulations totalling 4,409 CPU-hours — and that list already had to be re-tiered, ranked and pruned before it fit a night at all. Add the cost of the instrumentation that makes a run informative and it gets worse before it gets better: in a Mentor Graphics team’s published measurement, introducing accurate metastability injection models into a design degrades simulation performance by three to four times, which is exactly why metastability-tolerance verification cannot live in a nightly regression on a large design <sup>[4]</sup> (see Chapter 13).

**The parallelism fallacy.** The most common wrong response to all of this is “we will add machines”. A farm’s width multiplies *runs*; it does not shorten a *run*. If your problem is a 4,800-run random suite, licences are the answer and this chapter is not. If your problem is one workload whose length is measured in billions of cycles, licences are irrelevant: cycle  $n$  of a boot depends on cycle  $n-1$ , so a single run has no parallel fraction to exploit and buying width does nothing to it. This distinction is the

single most useful triage question in the chapter, and it is worth asking before any conversation about capital equipment: *is my problem the number of runs, or the length of one run?*

**The workloads that have no simulation answer share a shape.** They are the ones whose value lies in their *length* rather than in their randomness — where the bug needs a long, correlated, self-consistent history to reach.

**Scenario.** A video codec’s rate-control loop adapts quantisation to a rolling estimate of channel occupancy. In simulation the team checks a few macroblocks against the bit-exact reference model per run, and everything matches. In the lab, encoded streams drift out of the target bitrate after a scene change followed by a long static passage. **Approach.** The scenario is put on the emulator: several hundred frames of a real sequence, driven through the same reference-model comparison, run end to end. **Why.** The failure is not a data corner case that more seeds would find. It is a control loop with hundreds of frames of memory, and the shortest stimulus that can express its bug is longer than a simulation of it can be. Randomising harder explores the wrong axis; only length reaches it.

The same shape covers an operating-system boot, a GPU driver whose interesting failures live in queue and residency management across many submissions rather than in any single shader, and a mesh NoC whose latency distributions and virtual-channel occupancy only become meaningful statistics over sustained traffic (see Chapter 16).

**The contrast pair, and its surprising half.** Set the two generations side by side on the *same* verification problem and the interesting result is what does not move.

**Table 17.1** — One verification problem per row, with the approach each generation of the example SoCs takes to it. The rows marked *unchanged* are the surprise: block-level constrained-random work carries forward intact to the larger design, and what the second generation adds is not a harder version of an old problem but a new class of workload — one long, coherent, software-driven execution — that no amount of simulation capacity delivers.

Problem	Reference SoC	Flagship SoC
DMA 4 KB legalization, 2D transfers, negative stride	Constrained-random simulation, multi-seed	<i>Unchanged</i> — constrained-random simulation, more seeds, eight channels
Register and peripheral suites	Simulation	<i>Unchanged</i> — carried forward
Boot to a working system	Bare-metal image, a couple of million cycles, one nightly run	Operating system; no simulation answer at any farm size
Interconnect congestion behaviour	Crossbar arbitration, proved and simulated at block level	Mesh NoC latency and occupancy statistics under sustained load

The block-level answer stays where it was. Chapter 12 already showed the flagship’s DMA suite carrying forward directly, and nothing about acceleration changes that: a suite of 4,800 independent runs is exactly the workload a licence farm is good at, and moving it to a machine that runs one image at a time would be a mistake (Sec-

tion 17.5 does that arithmetic). What appears at the second generation is a *new class* of workload — one long, coherent, software-driven execution — that the first generation never had and that no amount of simulation capacity delivers. Emulation is not “simulation, but faster for everything”. It is the answer to one specific shape of problem, and buying it for the other shape is how teams end up with an idle multi-million-dollar machine and an unfinished regression.

## 17.2 Three platforms, and what each takes away

In an emulation flow the design is *compiled* — synthesised and partitioned — onto a hardware execution fabric, then runs under host-provided stimulus. Which fabric gives the three deployment categories: one or more commodity FPGAs (prototyping); a vendor-built system with its own compilation, transactor and debug infrastructure (an emulator proper); or a split in which a virtual platform on the host runs the well-modelled parts while an emulator runs selected RTL at cycle accuracy, joined by transaction-level co-modelling channels (hybrid co-emulation). Prototypes and emulation systems alike operate at MHz-scale effective speeds on large SoCs, against a simulator’s hundred-to-thousand-times-slower rate (Section 17.1) [5]. A design capacity on the order of a billion gates is within reach of an emulator [4], and in the 2024 industry study emulation adoption rises significantly for designs above one billion gates [2].

The comparison that matters is not speed. It is the three things you hand over to get it, and Table 17.2 sets what each platform buys above what each takes away.

**Table 17.2** — A simulator, an emulator and an FPGA prototyping platform compared row by row: throughput and capacity are what the hardware platforms buy, and the rows beneath them are what each hands over to get them. The pair that reorganises how people work is build time against changing what you observe — on these platforms a signal that was not brought out before the build cannot be looked at without another build. §17.5 adds the cost in money.

	Simulator	Emulator	Prototyping platform
<b>Throughput</b>	baseline	$10^2$ – $10^3\times$ the simulator	the fastest pre-silicon platform
<b>Capacity</b>	bounded by host memory and patience	order of a billion gates	device capacity; above it, multi-FPGA partitioning
<b>Build time</b>	minutes	hours to days for a complex design	hours for a bitstream
<b>Visibility</b>	any internal signal, at any time	triggers, selective trace, save/restore	a few thousand signals, chosen <i>before</i> the build
<b>Changing what you observe</b>	edit and re-run	re-arm a trigger; recompile if the signal was never brought out	recompile the bitstream
<b>Parallel width</b>	licences (hundreds)	capacity $\div$ design size	capacity $\div$ design size, replicated per board



**Visibility.** In a simulator, virtually any internal design signal is observable at any time. On a reconfigurable fabric — the fastest of the pre-silicon platforms — observability is restricted to a few thousand internal signals, and you must decide which ones before the bitstream is generated; reconfiguring observability means recompiling, which might take several hours [1]. Emulators soften this with controlled observability mechanisms — triggers, selective trace, save/restore — but they do not remove the constraint, only move it: capturing every internal signal at full speed over billions of cycles is infeasible, so a subset of signals or trigger conditions is chosen a priori, and a failure arising from an unmonitored interaction sends you back through the same long compile [5].

**Turnaround, which is the cost that actually shapes the workflow.** Compile times of hours to days for complex designs, worsened by instrumentation, are the reported reality for large emulation flows [5]. Put that next to the simulation loop and the difference is not a percentage. In simulation you edit, rebuild in minutes and have an answer the same morning — the reference SoC’s smoke tier is under nine minutes end to end, most of it build (see Chapter 12) — so you can afford to be wrong about what to look at, ten times a day. On an emulator you may get *one* build per day, sometimes fewer. Every question you did not think to ask before the build costs a day.

That single fact reorganises how people work. Emulation engineers front-load: they instrument for the questions they expect, compile in the assertions and monitors rather than planning to add them later, and treat “which signals come out” as a review decision rather than a debug reflex. The amortisation runs the other way too, and it is why the model works at all: once built, an image is reused across many hours or days of execution, regressions and campaigns, so the build is paid once and divided by everything that runs on it [5]. A build supporting one run is indefensible; spread over two hundred it is a rounding error. Section 17.5 turns that into a decision rule.

**Debug ergonomics.** The habits a simulation-trained engineer brings are mostly unusable. A `$display` on every bus beat is a host interaction per cycle and destroys the throughput you paid for (Section 17.4). Waveform dumping is bounded by a finite trace buffer, not by disk. What replaces them is a different discipline: assertions compiled into the platform as part of the model, reporting violations immediately — logged, or stopping the run at the violation — and trigger-based debug, where large numbers of cycles run under trigger state machines built to recognise complex sequences of events. Placing an assertion near a suspected cause is also what makes a long run debuggable at all, since a failure on an accelerated platform routinely surfaces far from its cause and much later, exactly as injected metastability does [4]. This is where Chapter 11’s insistence that an assertion is *declarative* — it says what to check and leaves the mechanism to the tool, which is why the same three lines serve a simulator and an emulator alike — pays off most: an assertion needs no conversation with the host, so it is the one form of checking that crosses the boundary intact, and the one that keeps *detection distance* short (see Chapter 8) where nothing else can.

**Scenario.** A mobile GPU’s driver-level bring-up needs many thousands of frame submissions to shake out queue and residency management, but the application cores that run the driver are mature, well-modelled IP with no open verification questions. **Approach.** Hybrid co-emulation: the cores stay as fast host-side models, the GPU’s four shader clusters and its memory path go on the emulator, and the two exchange transactions. **Why.** Three wins at once — the RTL footprint on the scarce machine shrinks, mature models are used where they are mature, and cycle accuracy is preserved exactly where the open questions are <sup>[5]</sup>. The cost is that the boundary between virtual and RTL is now a modelling assumption you must be able to defend: any bug whose mechanism straddles it is invisible by construction.

One distinction is worth stating precisely, because this book’s vocabulary is stricter than the industry’s. An emulator runs a *model* of the design, so work done on it is verification, pre-silicon in every sense that matters. An FPGA prototype on a bench with real peripherals attached is closer to what this book calls **validation** — checking on real hardware rather than on a model — which is why prototyping and the software work it enables get their own treatment (see Chapter 18), and post-silicon another (see Chapter 20). The distinction is not pedantry: it decides whether a result is evidence about the design or evidence about a board.

### 17.3 Use models: in-circuit and virtual

Once the design is on the machine, something has to drive it, and there are exactly two answers. In **in-circuit emulation** (ICE), the emulated design is wired to real physical peripherals — a real host, a real link partner, a real piece of test equipment — so the environment is the world. In the **virtual** or **targetless** model, there is no physical target: the environment is a synthesised testbench on the platform plus software models on the host, and the design is driven through transactors.

The first thing to understand about ICE is that the two ends do not run at the same speed. The design is executing at MHz; a real link runs at its specified rate, which is typically orders of magnitude faster and, worse, has timeouts. Something has to sit between them, and that something is a *speed adapter* — infrastructure supplied alongside protocol transactors precisely for hardware interfaces such as USB, Ethernet and PCIe <sup>[3]</sup>. It buffers, throttles and re-times so the physical link stays inside its specification while the design crawls. That adapter is the quiet asterisk on every ICE fidelity claim: what your design sees is real *content* at manufactured *timing*.

**What ICE buys.** Three real things. Stimulus nobody could have written — a real driver stack, with its ordering quirks and its bugs, produces scenarios no constrained-random generator will. Interoperability against third parties you have no model of. And compliance procedures that exist only as instruments.

**Scenario.** A USB 3.x dual-role controller’s link-training and low-power rows — the U0 through U3 transitions — were assigned in the vplan to the USB compliance suite, with the suite’s own item list as the closure metric (see Chapter 5). Pre-silic-

on, that suite has to run against something. **Approach.** ICE: the emulated controller is connected through a speed adapter to the physical compliance instrument, and the suite is executed as the compliance methodology defines it — the tests are specified not by the USB specification itself (*Universal Serial Bus 3.2 Specification*, Revision 1.1) [6] but by a separate compliance methodology document. **Why.** There is no file of vectors to replay — the suite is a procedure driven by an instrument, with electrical and timing components as well as protocol ones, and reimplementing it as a virtual transactor would mean reimplementing the specification you were trying to check independently. **The caveat that belongs in the row.** The speed adapter sits between the design’s pins and the instrument, so what closes here is the *protocol-state* half of each row. Timing at the pins is the adapter’s, not the design’s, and those rows stay open until silicon (see Chapter 20). A row that records “compliance suite passed” without that split is claiming evidence it does not have.

**What ICE forecloses**, and this is the part that decided the industry’s direction. You lose repeatability: a failure produced by a real link partner has no seed and nothing to replay. Chapter 12’s entire apparatus — run manifests, deduplication by failure signature, the repeat-N stability check before a test may gate — assumes a run is a function of recorded inputs, and a failure you cannot reproduce is a failure you cannot triage. You lose scheduling: the rig is physical, local and singular, one bench and one set of cables, which collides head-on with the economics of Section 17.5. You lose unattended operation, because somebody has to plug the device in. And you lose save/restore, because you cannot rewind the world.

**The virtual model gives model time back.** In a virtual environment built on the Standard Co-Emulation Modeling Interface (SCE-MI) — supported across emulation vendors — the platform provides full simulation interoperability, including simulation-style clock generation *on the emulator* and accurate tracking of model simulation time inside it, to the point of being able to measure the distance in model time between a transmit clock edge and a receive clock edge [4]. That capability is the hinge. When the platform owns model time, you can stop it, resume it, replay it, and place an event at an exactly chosen instant — and everything the industry actually wanted follows: deterministic reruns, controlled clock relationships, injection at a specified cycle, and a debug environment that behaves like a simulator’s.

That last point is not a small one. Virtual emulation environments offer almost all of the powerful simulation-style debug features — interactive run control against model time, waveform display, full or selectively traced waveforms, source browsing — and virtual emulation is repeatable, which is what makes an elimination strategy possible: disable a subset of the injection points, rerun the identical scenario, and see whether the failure persists [4]. Try that on a bench with a real link partner. Add the fact that the same environment can serve both simulation and acceleration when it is built to the standard [3], and the migration is over-determined.

**Scenario.** A radar front-end’s FFT and CFAR pipelines are being verified for fixed-point precision: every intermediate result must match a bit-exact reference across

long chirp sequences, in three arithmetic configurations. Attaching the real RF front-end in ICE is technically possible. **Approach.** Don't. Capture one real chirp sequence in the lab, store the digitised samples, and replay them from the host through a virtual transactor — the same file, three times, once per configuration. **Why.** A bit-exact comparison requires the *same* input twice. A live front-end delivers a different realisation of noise on every run, so a mismatch between two configurations cannot be attributed to either of them, and a failure cannot be reproduced for debug. The realism ICE offers is worth nothing here, because the property under test is arithmetic, not environmental — and the one thing the real front-end was providing, a realistic signal, survives perfectly well as a recording.

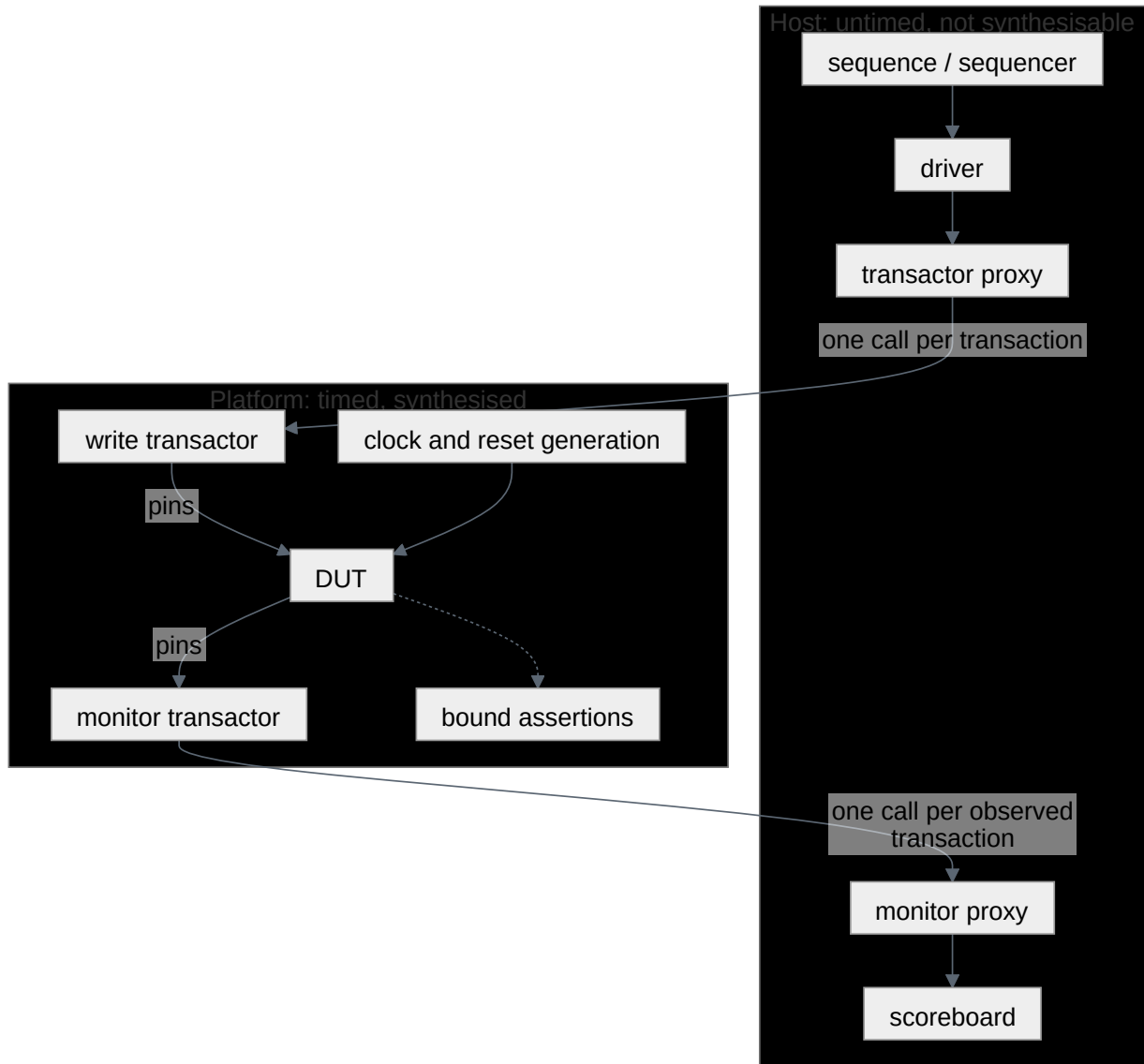
The rule that decides the use model, more reliably than any feature comparison: **use ICE when the environment is the thing you cannot model, and virtual for everything else.** A compliance instrument, a third-party host, a driver stack you do not own — those are environments, and the irreproducibility tax is worth paying for them. Noise, traffic patterns and workloads are *data*: recordable once, replayable forever, and paying that tax for nothing.

## 17.4 The transactor boundary

Chapter 8 introduced the transactor as an abstraction device: every physical-level operation of one interface encapsulated in one place, so everything above it speaks transactions. In acceleration it stops being a convenience and becomes a *physical cut*. Below it, the design is synthesised onto the platform and runs at the platform's clock rate; above it, code runs on a workstation. Between them is a link with latency — and where you place that boundary, not the platform's headline frequency, is what determines your throughput.

**The failure mode has a name.** Take a conventional environment in which the bus-functional behaviour lives in the UVM driver and monitor, reaching through a virtual interface to wiggle pins. Accelerate the DUT and nothing else changes: the design is on the platform, the protocol is still in the class, and every single pin event has to cross the link. This is *signal-level acceleration only*, and it is the bottleneck <sup>[3]</sup>. You have bought a machine that runs a billion gates at MHz and are feeding it one clock edge at a time from a workstation.

Chapter 8 stated the consequence in advance — an environment that reaches into signals from twenty places cannot be moved at all — and Chapter 10 stated the constraint that makes it true: a UVM component (IEEE Std 1800.2-2020, *IEEE Standard for Universal Verification Methodology* <sup>[7]</sup>) is not synthesisable, so it cannot cross to the other side. The question is therefore not whether to have a boundary but where to draw it, and the answer — drawn in [Figure 17.1](#) — is: at the transaction.



**Figure 17.1** — Where the transaction boundary cuts an accelerated environment: the upper box is untimed, not synthesisable and runs on the host, the lower box is timed, synthesised and running on the platform, and the cut carries exactly one call per transaction in each direction. What matters is what has no counterpart above the boundary — clock and reset generation, and the bound assertions, all sit below it, so nothing on the host is consulted per cycle.

Read the figure for what is *absent* from the host: clock and reset generation, and the assertions. Nothing above the boundary knows what a clock edge is, and nothing above it is consulted per cycle — the scoreboard is fed at transaction rate by a monitor transactor, and the assertions talk to nobody.

### The arithmetic of the boundary

Model it. Let  $f$  be the platform's clock rate, let  $N$  be the number of design cycles the platform advances between host interactions, and let  $L$  be the cost of one host interaction expressed in equivalent design cycles. Each period delivers  $N$  cycles of design progress and costs  $N + L$ , so the effective rate is

$$R(N) = f \cdot N / (N + L).$$

Pin-level driving is the case  $N = 1$ , giving  $f/(1 + L)$ . Batching to  $N$  cycles per interaction therefore speeds you up by

$$S(N) = R(N)/R(1) = N (1 + L) / (N + L),$$

which has two properties worth memorising. It **saturates at  $1 + L$** , no matter how large  $N$  becomes — batching can never buy you more than the round-trip cost itself. And it reaches exactly half of that ceiling at  $N = *L$ . *So the design rule is not “use transactions”; it is make your transaction granularity comparable to your round-trip cost\**, and stop optimising once it is.

$L$  is large, and it is the parameter you least control: it is a driver call, a link traversal, host scheduling and a return, measured against a clock period that at MHz-scale rates is of the order of a microsecond. Taking it as somewhere in the tens to the thousands of cycles, [Table 17.3](#) gives the fraction of the machine’s peak rate you actually get,  $N/(N + L)$ .

**Table 17.3** — The fraction of an accelerated platform’s peak rate a testbench actually obtains,  $N/(N + L)$ : rows are the design cycles  $N$  that one host interaction covers, columns three values for the round-trip cost  $L$  of that interaction expressed in equivalent design cycles. The top row is pin-level driving, which throws away nearly everything the machine was bought for; the rightmost column is the warning against complacency, since on a costly link even the longest burst the protocol encodes still leaves you at 11% of peak, and the remedy there is a coarser transaction rather than a faster link.

Cycles per host interaction	$L = 20$	$L = 200$	$L = 2000$
1 (pin-level)	4.8%	0.5%	0.05%
16 (a short burst)	44%	7.4%	0.8%
256 (the longest burst AXI encodes)	93%	56%	11%

Every column tells the same story with different force. Pin-level driving throws away between 95% and 99.95% of what you paid for, which is why “accelerate the DUT and leave the testbench alone” produces a very expensive simulator. And the rightmost column is the warning against complacency: on a platform with a costly link, even the longest burst the protocol can express leaves you at a tenth of peak. If your numbers look like that column, the answer is a *coarser* transaction — a descriptor, a page, a frame — not a faster link.

### Drawing the boundary in code

The mechanical change is small and entirely local. The timed protocol behaviour moves out of the class and into a synthesisable module — the transactor — which exports it as a task the host can call; a proxy on the host side imports that task; and the driver’s body becomes a single call. Below the boundary — here a write transactor for the flagship’s DMA, 40-bit physical addresses onto a 512-bit channel — one burst:

```
// ---- Below the boundary: synthesised onto the platform. One call = one burst.
module wr_xactor #(parameter int ADDR_W = 40, parameter int DATA_W = 512) (
 input logic clk,
 input logic rst_n,
```



```

output logic [ADDR_W-1:0] awaddr,
output logic [7:0] awlen,
output logic awvalid,
input logic awready,
output logic [DATA_W-1:0] wdata,
output logic wlast,
output logic wvalid,
input logic wready
);
logic [31:0] lfsr;

export "DPI-C" task do_write_burst;

task do_write_burst(input longint unsigned addr,
 input byte unsigned len,
 input int unsigned seed);
 lfsr = seed;
 @(posedge clk);
 while (!rst_n) @(posedge clk);
 awaddr <= ADDR_W'(addr);
 awlen <= len;
 awvalid <= 1'b1;
 do @(posedge clk); while (!awready);
 awvalid <= 1'b0;
 for (int i = 0; i <= int'(len); i++) begin
 lfsr = {lfsr[30:0], lfsr[31] ^ lfsr[21] ^ lfsr[1] ^ lfsr[0]};
 wdata <= {(DATA_W/32){lfsr}};
 wlast <= (i == int'(len));
 wvalid <= 1'b1;
 do @(posedge clk); while (!wready);
 end
 wvalid <= 1'b0;
 wlast <= 1'b0;
endtask
endmodule

// ---- Above the boundary: what the untimed testbench calls, once per burst.
interface wr_xactor_proxy;
 import "DPI-C" context task hdl_do_write_burst(input longint unsigned addr,
 input byte unsigned len,
 input int unsigned seed);
endinterface

```

Four things in that code are load-bearing. Three of them restate the standard acceleration guidelines [3]; the fourth is the lesson those guidelines leave implicit.

*The task is timed and the call is not.* Everything that waits on `clk` is inside the module; the proxy above declares no clock and the driver never mentions one. This is the rule that reads oddly to a simulation engineer and matters most: **do not use clocks to synchronise with the testbench** — synchronise on transactions and events. A driver that waits for an edge has re-created the pin-level case by another route.

*One call covers `len + 1` beats.* With `awlen` at its maximum of 255 that is a 256-beat burst — the  $N = 256$  row of the table — for a single host interaction. That row is the protocol's ceiling, not this channel's: a beat here is 512 bits, so 64 bytes, and 256 of them would move 16 KB and cross three 4 KB frontiers. The legalization discipline this DMA inherits caps a legal INCR burst at 64 beats, which is exactly one 4 KB frame. So a single call reaches somewhere between the table's second and third

rows, and the gap to the third row is not something a longer burst can close. That is the chapter's rule in its most concrete form: when one legal burst is not granularity enough, the next transaction up is a descriptor, not a bigger burst.

*The same source serves both engines.* The transactor is ordinary SystemVerilog that a simulator will execute directly, so one environment supports simulation and acceleration, and a bug found on either platform is reproducible on the other — for exactly as long as the two remain one source, which is harder to sustain than it sounds (see *In practice*).

*The payload is manufactured on the platform.* The host passes a 32-bit `seed`; the shift register produces the data. It would have been more natural to randomise the payload on the host and ship it, and that is exactly the instinct the boundary punishes: shipping a 512-bit beat once per beat is one host interaction per beat again, with the data volume on top. The pattern generalises far beyond payloads. When a metastability-injection methodology needed an uncorrelated random decision at every clock-domain-crossing point, the natural implementation — a stream of random values arriving from the workstation — was rejected for the logic needed to distribute the values and the runtime needed to transfer them, in favour of an on-platform shift-register generator whose bits are statically distributed among the injection points <sup>[4]</sup>. **Anything the environment needs every cycle becomes hardware.** That is the single most reliable predictor of how much of a testbench has to be rewritten.

### What the crossing costs a testbench

The platform accepts a superset of RTL — implicit state machines, some system tasks, shared variables, hierarchical names, named events are all typically compiled — but the common denominator is still *synthesisable plus DPI-C*, and the DPI itself is a restricted subset: the usable argument types are limited, nesting of imported and exported calls is bounded, and four-state values may be reduced to two-state at the boundary <sup>[3]</sup>. Two consequences deserve to be stated before somebody discovers them mid-build.

**Treat checking that reasons about X as something that does not travel.** The reduction to two-state is documented at the DPI boundary; in practice the prudent assumption is broader, because a fabric of real gates has no third value to propagate. If part of your checking depends on an unknown reaching an output — an uninitialised-register detector, a reset-coverage check — keep it where it works, in simulation (see Chapters 12 and 13). This is not an argument against acceleration; it is an argument for knowing which of your checks moved and which quietly did not.

**Checking splits by where its cost lands.** An assertion compiled into the platform costs logic and no interactions, so it scales to billions of cycles. A scoreboard on the host costs one interaction per transaction it must see, so it scales to whatever transaction rate the table above allows. An oracle that needs to see *every beat* costs one interaction per beat and does not scale at all — which means it has to be redesigned rather than ported.

**Scenario.** An inline AES-GCM crypto engine on the memory path is checked in simulation by a host-side scoreboard that compares every ciphertext beat against a reference model. The team wants a multi-hour soak with real firmware driving real traffic through it. **Approach.** Restructure the oracle rather than move it. A synthesisable digest accumulator on the platform hashes the ciphertext stream and the authentication tags; the host reads and compares one digest per checkpoint — say every few thousand blocks — instead of one value per beat. The per-beat comparison stays exactly where it belongs, as a short-vector check in the simulation tier. **Why.** The emulator’s product is *length*, and an oracle whose cost is proportional to beats converts length back into host interactions. Moving the per-cycle cost on-platform and leaving one host interaction per checkpoint is what makes the soak possible at all. **What it costs.** Detection distance (see Chapter 8): a digest mismatch says the stream diverged somewhere in the last checkpoint interval, not where. Checkpointing at intervals is the price of admission and also the cure — it bounds the bisection.

## 17.5 Economics: one machine, many claimants

Everything so far has been technical. The reason emulation is the hardest engine to introduce into a verification programme is not technical: it is that the resource is capital rather than seats. Acquisition and maintenance costs effectively restrict full-featured emulation to a limited number of large design houses, and even inside those, emulator slots are shared across multiple projects and teams and have to be scheduled and prioritised — with capacity typically reserved for tape-out-critical functional regressions, so that other campaigns get a subset of high-risk blocks or get deferred to later project stages [5]. A simulation licence pool degrades gracefully under contention. An emulator does not: it is granted or it is not.

The market figures put the scale in perspective. Hardware-assisted verification was estimated at USD 760.4 million in 2025, split roughly 61% emulation to 39% prototyping, and projected to reach USD 3.08 billion by 2035 at 15.0% compound growth; an independent estimate puts 2024 at USD 641.5 million growing to USD 1.5 billion by 2030 at 15.2% — against a total electronic-system-design industry that reported USD 5.6 billion of revenue in a single quarter of 2025 [5]. The segment is small and growing fast — what you expect of a technique moving from optional to necessary — and it is bought the way factories are bought, which is why the machine’s calendar is managed by someone senior and why “can I have three days on it” is a conversation about the project plan, not about tooling.

It is also not primarily *yours*. In the 2024 industry study the leading reason projects adopt emulation or prototyping is hardware/software co-design and verification, with post-silicon debug well down the list [2]. The machine is generally justified by software readiness and then shared with verification — the political fact behind the scheduling one.

### The rule that decides where a workload belongs

Take a body of  $N$  runs. On the farm, wall-clock cost is  $N \cdot s / W_{\text{sim}}$ , where  $s$  is the per-run simulation time and  $W_{\text{sim}}$  is your licence count. On hardware, it is  $B + N \cdot r / W_{\text{hw}}$ , where  $B$  is the build and  $r$  the per-run execution time. The term everyone forgets is  $W_{\text{hw}}$ , and it is not a purchasing decision: **hardware-assisted parallel width is capacity divided by design size**. A block-sized image replicates across boards cheaply — which is why an eight-board cluster can run a suite in parallel at all (see Chapter 12) — while a full-SoC image occupies the machine, and  $W_{\text{hw}} = 1$ .

Run the two cases the flagship actually presents.

**The OS boot.**  $N$  is a handful of runs per design revision;  $s$  is years (Section 17.1);  $r$  is a few hours <sup>[1]</sup>;  $B$  is hours to days. Simulation costs years per run and  $W_{\text{sim}}$  cannot help, because a licence pool shortens suites and not runs. Hardware costs a build plus an afternoon. The break-even is below one run — this workload does not need a business case, it needs a slot.

**The DMA legalization suite.** Chapter 12 gives the numbers: 4,800 constrained-random runs at a 30-minute mean, 2,400 CPU-hours, ten hours of wall clock on 240 licences. Suppose you moved it. Even at the optimistic end of the 100-1000× band — and that band is the platform's *raw* rate, before the boundary derating of Section 17.4, which only makes this worse —  $r$  is under two seconds, but  $W_{\text{hw}}$  is 1, so 4,800 runs cost about 2.4 hours of machine time — against simulation's ten hours, a real gain — and then you add  $B$ . And  $B$  is the killer, because it is paid **per design revision**, and a nightly suite runs per design revision. A build measured in hours, incurred every night, against ten hours on licences you already own, buys nothing. At the pessimistic end of the band the emulator loses on run time alone.

The general shape: **the machine wins when  $s$  is enormous and  $N$  is small; the farm wins when  $N$  is large and  $s$  is moderate**. Width beats speed as soon as the runs are independent — which is exactly why the contrast pair of Section 17.1 came out as it did.

### What earns time on the machine

The admission policy is a tier in Chapter 12's sense — a rule about what gets in, with a budget attached — but the currency is different. A tier spends CPU-hours, which are fungible. An emulation slot spends *machine days and build slots*, which are not. Three claims earn a slot, and one rule protects it:

1. **Workloads with no simulation answer:** boot, firmware, long soaks, sustained-traffic statistics on the mesh NoC (see Chapter 16).
2. **Campaigns whose value is iteration count under one image** — fault-injection sweeps and adversarial campaigns, where practical effectiveness depends on high iteration counts and repeatability, and where the build is amortised across the whole sweep <sup>[5]</sup>. Full-chip metastability-tolerance verification belongs here too: the accurate injection models are exactly the ones that cost 3-4× in simula-

tion (Section 17.1), which is why they run on hardware (see Chapter 13). That same team reports the degradation as tolerable on a small block and enough to make the traditional flow impractical as designs grow, which is the whole of the argument for moving the campaign onto hardware <sup>[4]</sup>. Its “often 1000X” figure for emulation over simulation, by contrast, is a general remark with no benchmark behind it, and this book does not use it — the 100-1000× band this section costs workloads against comes from <sup>[1]</sup> instead.

3. **Reproductions of failures found downstream.** Recreating a silicon failure on an acceleration platform gave one industrial team all the data a designer needed for root cause, precisely because the platform has the observability silicon lacks — though the failure could not be migrated as-is, since silicon runs more than six orders of magnitude faster and the exerciser had to be re-tuned to hit it <sup>[1]</sup>. Describing the scenario portably rather than per-platform is what makes that carrying cheap enough to be routine <sup>[8]</sup> (see Chapters 9 and 20).
4. **Nothing a farm can do.** This is the rule that has to be enforced, because it is the one that is always broken.

The planning consequence is one extra column, and it is not the obvious one. Chapter 5’s vplan row already carries a *method* field naming the engine that discharges it. An emulation row needs one thing more: **which image it runs on**. Because the image is the scheduling unit, emulation rows are batched by image rather than by priority — a low-priority row that fits an image already being built is nearly free, and a high-priority row needing its own instrumentation costs a build. Plan the images first, then assign rows to them; plan rows first and you discover, late, that the top-priority ones are spread across five builds you have no calendar for.

### When the honest answer is “this is a simulation problem”

Most requests for emulator time should be refused, and the three commonest reasons are all diagnosable in an afternoon.

*The testbench is what is slow.* Per-item object allocation, dynamic dispatch and a report server on the message path cost nothing on a ten-thousand-cycle block run and a great deal on a long one (see Chapter 10). Profile the run before costing a machine: if the design is a minority of the runtime, you have a software problem with a software fix.

*Every run boots from zero.* A workload that spends 90% of its cycles reaching the state under test does not need a faster engine; it needs a saved state, or a shorter path to that state.

*The stimulus is badly shaped.* The most expensive version of this failure is reaching for a system-level soak because nobody wants to write the block-level stimulus that would reach the same coverage directly. Emulator time is, in that case, being bought to avoid rewriting a test — and it is a poor trade, because the soak also finds the bug more slowly and localises it worse.

In every case the tell is Section 17.1’s triage question, asked out loud: *is my problem the number of runs, or the length of one run?* Only the second answer leads here.

---

## IN PRACTICE

The two environments drift, and nobody plans for it. Monitors and harnesses frequently have to be re-engineered to fit emulator timing and capacity constraints, which produces divergent simulation and emulation environments and subtle inconsistencies in observed behaviour — complicating debug and making it harder to reproduce an issue across platforms <sup>[5]</sup>. The cure is unglamorous and it is a standing commitment rather than a fix: one source for both engines, and a scheduled cross-platform run of the same scenario so that divergence is discovered on a Tuesday rather than during a tape-out escalation.

Expect the surrounding machinery to be less mature than the simulation equivalent. Regression management tooling routinely schedules thousands of simulations, analyses coverage and tracks closure; extending it to schedule long hardware campaigns is non-trivial, emulator jobs have to be co-optimised against functional runs, and campaigns requiring coordinated software stacks add orchestration complexity of their own <sup>[5]</sup>. Expect, too, that the build is owned by a small team who are not the people writing the tests, that a failed build costs a day with a queue behind it, and that “who gets the machine next week” is decided in a meeting you may not be in. None of this argues against the engine. It argues for arriving with your images planned and your instrumentation chosen.

---

## PITFALLS

1. **Accelerating the design and leaving the environment alone.** *Symptom:* an extremely expensive machine delivering a few thousand cycles per second. *Cure:* move the timed protocol into synthesisable transactors and batch to a granularity comparable to the round-trip cost (Section 17.4).
2. **Discovering the signal you need was never brought out.** *Symptom:* every interesting failure ends in “rebuild with more probes”, at a day each. *Cure:* choose observability in a review before the build, and compile assertions in rather than planning to add probes later.
3. **Buying capacity for a farm problem.** *Symptom:* the machine idles while the nightly overruns. *Cure:* answer “is it  $N$  or is it  $s$ ?” before writing the requisition, and profile the run to check the design is actually the majority of it.
4. **In-circuit by default.** *Symptom:* failures nobody can reproduce and a bench nobody else can use. *Cure:* virtual unless the environment itself is the unmodellable thing; record and replay anything that is only data.
5. **Porting the oracle instead of restructuring it.** *Symptom:* a per-beat scoreboard pins throughput at transaction rate and the soak never finishes. *Cure:* per-cycle checking on the platform, one host interaction per checkpoint.
6. **Treating a compliance pass as a closed row.** *Symptom:* an electrical requirement marked covered because a suite ran through a speed adapter. *Cure:* split the row — protocol state closes pre-silicon, timing at the pins does not.



---

## SUMMARY

- Simulation runs out for a reason you can compute: RTL simulation is roughly a billion times slower than the target clock, so seconds of silicon become years of simulation, and hardware-assisted platforms recover a hundred to a thousand times of that <sup>[1]</sup>. Read the band as a band; the conclusion is the same across all of it.
- One triage question separates the two worlds. *Is my problem the number of runs, or the length of one run?* A licence farm shortens suites and can never shorten a run.
- What you pay is visibility, turnaround and debug ergonomics — signals and triggers chosen before the build, one build a day and sometimes fewer, and checking planned in rather than added when curiosity strikes.
- Throughput lives at the transactor boundary, not in the platform's clock rate. Batching to  $N$  cycles per host interaction saturates at  $1 + L$  and reaches half of that at  $N = L$ , so match granularity to round-trip cost and stop.
- Anything the environment needs every cycle becomes hardware — payload generation, randomisation, per-beat checking. That rule predicts how much of a test-bench must be rewritten better than any other.
- Prefer virtual to in-circuit unless the environment is genuinely unmodellable, because model time under your control is what buys repeatability, replay and simulator-like debug.
- Plan images, then rows. The image is the scheduling unit on a shared capital machine, and a plan organised by row priority will not survive contact with the build calendar.

---

## FURTHER READING

Within the corpus, <sup>[1]</sup> is the anchor for this chapter's arithmetic — the billion-fold ratio, the hundred-to-thousand-fold recovery, the observability ladder from simulator to reconfigurable fabric to silicon, and the bitstream recompile — and its industrial case study is also the best account here of carrying a failure back from silicon onto an acceleration platform. <sup>[5]</sup> is the most complete recent survey of hardware-assisted verification as a *workflow*: platform taxonomy including hybrid co-emulation, the compile-versus-execution distinction and its amortisation, the observability and trace-buffer constraints, the divergence between simulation and emulation environments, and the market figures of Section 17.5; it is framed around security, but its platform and workflow analysis is general. <sup>[3]</sup> is the step-by-step conversion of a conventional UVM environment into an acceleration-ready one — the transactor, the exported task, the proxy, the guidelines — and worth reading in full before your first port. <sup>[4]</sup> is the clearest published account of what a virtual environment gives you that an in-circuit one does not, and the on-platform randomisation problem it solves is the general lesson in miniature. <sup>[2]</sup> supplies the 2024 adoption data by design size and the reasons projects give. <sup>[8]</sup> and <sup>[9]</sup> cover the stimulus side: describing scenarios once and retargeting them across simulation, emulation, prototyping and silicon.

Not in the corpus and worth seeking out: the Accellera Standard Co-Emulation Modeling Interface specification itself, which defines the boundary this chapter's Section 17.4 lives on and is short enough to read in an afternoon; and P. Rashinkar, P. Patterson and L. Singh, *System-on-a-Chip Verification: Methodology and Techniques*, whose treatment of emulation and rapid prototyping predates most of the current tooling and is still the clearest statement of why the use models are what they are.

---

## REFERENCES

1. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
2. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
3. **D33** A. Grove, "UVM hardware assisted acceleration with FPGA co-emulation," Proc. DVCon Europe, 2015.
4. **D34** A. Hari, S. Krishnamurthy, A. Jain, and Y. Badaya, "Verification of Clock Domain Crossing Jitter and Metastability Tolerance using Emulation," Proc. DVCon U.S., 2012.
5. **P14** T. Rahman, S. Saha, A. Y. Alhurubi, S. K. Saha, F. Farahmandi, and M. Tehranipoor, "Emulation-based System-on-Chip Security Verification: Challenges and Opportunities," arXiv preprint arXiv:2604.15073v1, April 2026.
6. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.
7. **S9** IEEE, "IEEE Std 1800.2-2020, IEEE Standard for Universal Verification Methodology Language Reference Manual (Revision of IEEE Std 1800.2-2017)," IEEE, 2020.
8. **D29** P. Gupta and M. Vax, "Test driving Portable Stimulus at AMD," Proc. DVCon U.S., 2019.
9. **S1** Accellera Systems Initiative, "Portable Test and Stimulus Standard, Version 3.0," Accellera, August 2024.

---

## 18. FPGA Prototyping and HW/SW Co-verification

The flagship SoC's prototype is four FPGA boards in a rack, a fan tray and a serial console. Nine months before tape-out it boots an operating system on the host cluster's four application cores, mounts a filesystem over the board's Ethernet port, and reaches a shell prompt. This is the first time the OS, the drivers, the boot ROM and the RTL have ever been in the same room, and it works — for about forty seconds. Then a benchmark that submits work to the compute cluster returns a result whose last cache line is stale. Run it again and it passes. Run it forty times and it fails twice.

What follows is the part nobody schedules. The verification team cannot reproduce it: their DMA and coherence tests pass, and the failing workload is hundreds of millions of cycles long — far beyond what their simulator will reach this quarter. The software team's driver is the same driver that has run for six months against a functional model without a single failure. Available evidence: one console, a few LEDs, and a trace buffer configured for a different question three bitstreams ago. Two teams spend nine days deciding whose bug it is. It is the flagship's DMA mis-legalizing a 2D descriptor at a 4 KB boundary — the reference SoC's canonical bug, inherited and now hidden one layer deeper by coherence, because the split's second half writes a line the host cluster holds Modified and the corruption stays invisible until the writeback. The bug was not hard. The nine days were: on a prototype, nobody could see it.

This chapter is about that instrument and the discipline that shortens those nine days. It is also where a terminological line matters. In this book **validation** means checking a design on real hardware rather than on a model — post-silicon on fabricated parts, *or in the lab on a prototype* (glossary; Chapter 20 states it in full). An FPGA prototype is therefore pre-silicon in *time* and validation in *kind*: it is the first platform on which real software runs against real hardware, and the first on which the debug ergonomics of the lab, rather than those of the simulator, decide how expensive a bug is.

---

### CHAPTER MAP

- §18.1 establishes how an FPGA prototype differs from an emulator in speed, capacity, turnaround and — decisively — visibility, and what that difference should change about the work sent to it.
- §18.2 states what a virtual platform can establish about firmware long before RTL is stable, and the specific class of thing it is structurally incapable of establishing.
- §18.3 places the boundary in a hybrid platform, and identifies which bugs the boundary deletes by construction.

- §18.4 sets out what running the real driver and the real boot sequence finds that a constrained-random environment does not — and what it cannot find, which is why it complements rather than replaces one.
- §18.5 triages a prototype failure across two organizations, and identifies the case where the bug belongs to neither of them because it belongs to the specification.

## 18.1 The prototype as a different instrument

An FPGA prototype maps the SoC's RTL onto one or more commodity FPGAs. It is widely used for early hardware/software integration because it gives hardware-speed execution of large RTL blocks, supports real I/O connectivity, and lets software be brought up on a cycle-accurate model of the design <sup>[1]</sup>. All three properties are doing work. *Hardware-speed* means megahertz: platforms of this kind run roughly a hundred to a thousand times faster than an RTL simulator, turning an operating-system boot from an impossibility into a few hours of runtime on a large SoC <sup>[2]</sup>. *Real I/O* means the console is a real UART, the network a real PHY, the storage a real device — so the driver being exercised is the driver, not a testbench's idea of one. *Cycle-accurate* means no abstraction sits between the software and the design it programs.

A 2026 report on one such flow gives the shape of what that buys. Its FPGA validation suite of 1,738 tests spans three classes — integration tests checking the RTL against the FPGA platform itself, bare-metal benchmark and litmus tests, and OS-level tests exercising the driver, the operating system and the software stack — and within it a full Linux boot of a dual-core design clocked at 20 MHz on the board completes in 115 seconds <sup>[3]</sup>. Two minutes, on a design far smaller than the flagship, for a boot RTL simulation would never finish: that is the whole argument for the platform.

### What you give up

The prototype's defining limitation is not speed or capacity. It is that you cannot see inside it. Observability on an FPGA is restricted to a few thousand internal signals, you must decide *which* signals before the bitstream is generated, and reconfiguring that choice means recompiling the bitstream — which may take several hours <sup>[2]</sup>. Compare that with the simulator you left behind, where every net is visible for free, after the fact, without rerunning anything (Chapter 3). The consequence for practice is direct: because internal probing and trace capacity are constrained, prototyping emphasises system-level execution and functional realism, while deep debug is performed selectively or deferred to a higher-visibility environment <sup>[1]</sup>.

*Deferred* is the word to notice: deferring debug elsewhere means reproducing the failure in simulation or on an emulator — exactly what the opening scenario's teams could not do, and why they spent nine days.

Turnaround compounds it. Synthesis, partitioning and compilation of a complex design can take many hours or even days, and instrumentation makes it worse:

probes are logic, and logic competes for the same fabric and routing resources as the design [1]. So the loop that costs seconds in simulation — add a probe, rerun, look — costs a working day here, and each iteration buys visibility only of the signals you guessed at the start of it. Toolchains push back with incremental build support, automatic partitioning, automatic multiplexing of signals and static and dynamic probes [4], but the constraint’s shape does not change: what you can see is decided before the run, and changing your mind is a compile.

### Capacity, partitioning and clock ratios

A large SoC does not fit in one device. Partitioning across several introduces machinery that is not in the design: bridges between partitions, serialiser/deserialiser links across the board, and synchronisation logic to hold the pieces together — at the cost of increased communication latency across the FPGA boundaries and higher integration complexity [3]. That latency is not neutral. A crossbar or NoC path costs materially more cycles across a partition boundary than inside one device, so the arbitration and back-pressure behaviour you observe on the prototype is not the behaviour the silicon will have. Performance measured on a partitioned prototype is a lower bound with an unknown gap, never a projection.

Clocking distorts in a subtler way, worth deriving because teams get caught by it. The system clock must drop by some factor to close timing in fabric. The peripheral domain drops with it, so the 4:1 ratio between the reference SoC’s 200 MHz and 50 MHz domains (Chapter 13) survives — divide both by the same factor and the relationship is preserved. The always-on domain does not cooperate: its 32 kHz reference is a real crystal on the board, and a crystal does not scale. So the 6,250:1 ratio between those two domains (200 MHz over 32 kHz) becomes 6,250 divided by the system slowdown factor, and every timing relationship crossing that boundary moves with it.

That is not abstract. The canonical wake-cause bug of this book lives exactly there: a 3-bit code crossing from the always-on island to the system domain through per-bit synchronizers, where a legal `3'b010 → 3'b100` transition is briefly observed as the reserved code `3'b110`. Whether the prototype ever shows it depends on how many fast-domain sampling edges fall inside the slow domain’s transition window — the very quantity you just changed. A prototype that never reproduces a crossing bug and one that reproduces it constantly can be the same design at two clock settings. Structural CDC analysis found this bug in seconds (Chapter 13); the prototype is not the instrument for it, in either direction.

### What the RTL must tolerate to be prototypable

Prototypability is a design property, planned rather than discovered. The RTL must be technology-independent enough to synthesise for a fabric it was not written for: no vendor-specific memory macros or standard-cell instantiations, no latch-based structures the FPGA has no equivalent for, no manually gated clocks the fabric’s clocking resources cannot honour, and no analog. Everything at the chip’s physical edge is substituted: bringing a design up on a board means interfacing the plat-

form’s own memory controllers, Ethernet, console UART, host DMA over PCIe, SPI for firmware boot and JTAG for debug, until the result is a complete software-bootable system [3].

Note what the substitution costs. The flagship’s LPDDR4 controller and PHY — whose training state machine is among the riskiest things on the chip — are exactly what cannot be prototyped, because the PHY is analog and its timing is the point; on the board they are replaced by the platform’s own memory controller. So the prototype will run the OS beautifully and tell you nothing whatever about memory training, a bring-up risk that must be discharged at the AMS boundary (Chapter 21) and in the lab after tape-out (Chapter 20). Write it in the plan as an explicit prototype-scope exclusion, the way Chapter 5 demands of every gap.

### Prototype or emulator?

Both platforms run RTL on hardware, and both reach megahertz-scale effective speeds on large SoCs, with the achievable frequency depending on design size, partitioning, instrumentation and the selected debug configuration [1]. The differences are the ones you buy an instrument for. An emulator is built for capacity and controlled observability — triggers, selective trace, save and restore — with compile and debug infrastructure sized to the design (Chapter 17). A prototype is built for speed and realism, and its observability is whatever you inserted before the build. Industry adoption follows that split: in the 2024 study, emulation adoption rises sharply for designs above a billion gates [5] — the capacity threshold at which a prototype stops being a platform and becomes a partitioning project.

The right mental model is not “cheap emulator”: **an emulator is a fast simulator with reduced visibility; a prototype is early silicon with more visibility than silicon** — one on each side of the verification/validation line. Work you would do in a simulator, if only it were faster, belongs on an emulator. Work you would do in the lab, if only you had a chip, belongs on the prototype.

### What earns a place on the prototype

That gives an admission test, and it has three conditions, all of which must hold:

1. **The workload needs more cycles than simulation can deliver.** Booting an OS, driving a full initialisation, encoding a thousand video frames, sustaining a network link for minutes. If it fits in a simulation run, run it there, where debug is cheaper and the coverage database already knows how to record it.
2. **It has an oracle that survives the visibility collapse.** Something must decide pass or fail without a waveform: an offline bit-exact diff, self-checking firmware, an in-fabric monitor that latches a violation into a readable register. A workload whose only oracle was “the engineer watched the waveform” has no oracle at all here (Chapter 3).
3. **Its failure is plausibly reproducible back in a higher-visibility environment.** Because that is where it will be debugged.



Condition 2 is the one teams skip, and condition 3 is the one that costs them. A workload that fails condition 2 produces a red LED and a week of ownership argument, which is Section 18.5's entire subject.

**Scenario.** The video codec's encoder passes its block-level environment: a bit-exact golden model checks every macroblock against the reference encoder, and the coverage model closes on prediction modes, quantiser steps and buffer-full conditions. What nobody has run is a *sequence*: a thousand consecutive frames of real content, where rate control adapts across scene changes and the DDR traffic pattern shifts under it. In simulation that is weeks. **Approach.** Put the encoder on the prototype, feed it a real clip from a file, write the bitstream out to storage, and diff the whole output against the reference encoder's offline — one comparison, after the run, on a workstation. **Why.** It satisfies all three admission conditions. The workload genuinely needs the cycles; the oracle is an offline bit-exact diff that needs no visibility at all; and when frame 738 differs, the frame number and the encoder state that produced it are enough to build a simulation testcase from the same input. That last property is what makes it worth doing — not the speed.

The counter-case is the discipline this chapter argues against. In the FPGA-target market segment — where the design ships in the fabric, the lab is hours away, and it is routinely used as the primary validation vehicle — many teams prioritise minimal pre-lab verification in order to reach the lab quickly, a strategy that worked for simpler devices and is increasingly impractical for modern SoC-class ones; in the 2024 Wilson Research Group FPGA study, only 13 percent of projects in that segment reported zero non-trivial bug escapes into production [6]. Prototypes make bugs *visible late*; they do not make them *findable*. A team that treats the prototype as its verification instrument has replaced a controllable, observable, reproducible environment with an uncontrollable, unobservable, weakly reproducible one, and gained only speed.

## 18.2 Virtual platforms

The prototype arrives late. It needs RTL that synthesises, a partition that closes, and a board — which means it is available months after the firmware team needed something to write code against. The instrument that fills that gap is a **virtual platform**: a software model of the whole system, fast enough to run the real firmware binary, available long before the RTL is stable.

Its position in the lifecycle is well documented. At the start of a project the only models available are high-level architectural specifications; abstract virtual models of the individual IPs come next, serving primarily as the framework against which software and firmware are developed and checked. These are highly abstract software models that preserve only the basic functionality of the hardware/software interfaces, and what they support is correspondingly *coarse-grained* checking of hardware/software use cases [2]. Every clause of that description bounds what the platform can be asked.

At the centre sits an instruction-accurate model of the core: it executes the target instruction set faithfully and updates architectural state correctly, with no commitment to how many cycles anything takes. Around it sit functional models of the memory map — a register file per peripheral, with the side effects the architecture document describes. That is enough to boot an OS and to debug software as if it were running on real hardware, with considerably more controllability and observability than debugging the same code over a JTAG port; the price is execution speed, and the compensation is that this may be the only option at all while the hardware is still under development <sup>[7]</sup>.

### What it can and cannot tell you

What a virtual platform establishes is substantial, and teams under-claim it as often as they over-claim it: that the initialisation sequence sets the registers it means to set in an order the hardware accepts; that the firmware’s view of the memory map agrees with the architecture’s; that handlers are wired to the sources they think they are; that device tree, linker script and boot ROM hand-off agree. On the flagship that is four cores’ worth of OS bring-up, a driver stack and a filesystem, debugged with full state inspection, none of it waiting for RTL.

What it cannot tell you follows directly from “preserve only the basic functionality of the hardware/software interfaces”. It has no arbitration, because there is nothing to arbitrate. It has no back-pressure, because a model that returns immediately never stalls. It has no latency, so two operations the design will overlap are, in the model, sequential. It has no coherence, because a single flat memory has nothing to be coherent with. The entire class of bugs that consists of *two things happening at once* is absent — not undetected, absent, which is worse, because the platform reports success with complete confidence.

The trap has two shapes, and both produce firmware that passes on the model and fails on hardware:

- **Assumptions the model never charged for.** A poll loop that reads a status register until a bit sets terminates on the first read against a model that completes instantly, so nobody notices it has no timeout and no memory barrier. A `udelay(10)` chosen because it felt safe is never tested against a device that takes longer.
- **Synchronisation the model made unnecessary.** Firmware that hands a buffer to a DMA engine without a cache maintenance operation works perfectly on a platform with no caches. The omission surfaces on the prototype, where it looks exactly like a hardware coherence bug — and is argued about as one.

A second failure is quieter: the virtual platform becomes the specification by default. When model and RTL disagree the reflex is to ask which is right, and neither is golden — both were written by people reading the same document, so a disagreement is as likely to be two misreadings as one bug, and where they agree wrongly you have a common-mode error that comparison cannot surface (Chapter 3). The structural fix is Chapter 15's: generate the model's register behaviour, the decode logic and the firmware header from one register description, so disagreements reduce to behaviour rather than layout.

**Scenario.** The GPU's driver stack is developed for eight months against a functional model: four shader clusters, a command processor, a unified memory. Submission, context switching, allocation and fault handling all work, and the driver team's suite is green. On the prototype, a compositing workload tears a frame roughly once every few thousand submissions. **Approach.** Do not start by suspecting the driver. Ask which of its assumptions the model was structurally unable to test — and the answer is the ordering between the command processor's descriptor read and the shader clusters' writes to the same memory, serialised by construction in the model and not in the design. **Why.** The model had no concurrency to get wrong, so it could neither confirm nor deny the ordering assumption; it never posed the question. That is the diagnostic signature: a bug not merely missed by the virtual platform but *unreachable* in it. When a prototype failure looks like a race, ask first what the software's development platform could not model — that is where the untested assumption was allowed to form.

### The accuracy/speed dial, and its middle setting

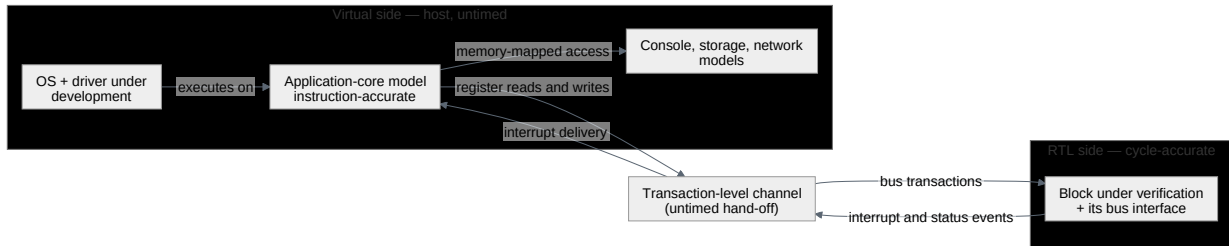
Virtual platforms are not one thing. Between the instruction-accurate model and the RTL sits a family of *approximately timed* models adding latency estimates, arbitration policies and bus delays — slower than the untimed model, far faster than RTL, useful for architectural exploration and buffer sizing.

Handle that middle setting with care, because it carries a hazard the untimed model does not. An untimed model is honestly silent about timing; nobody is tempted to trust it. An approximately timed model is not silent — it has *a* timing, plausible, quotable, and not the design's. Firmware tuned against it (a queue depth, a batch size, an interrupt coalescing threshold) is tuned against a fiction, and the numbers acquire an authority in review slides that the model never earned. The rule worth internalising: an approximately timed model answers *architecture* questions, whose answer is a ratio or a trend; it does not answer *verification* questions, whose answer is a verdict. When a decision turns on a verdict, the model must be the RTL.

## 18.3 Hybrid co-simulation

Most real hardware/software flows are neither purely virtual nor purely RTL. They are hybrids: what you are not verifying runs as fast models, what you *are* verifying runs as RTL, and the two are joined at a transaction-level boundary.

The industrial form partitions the system across two execution domains — a virtual platform on the host running high-level or fast instruction-set models, and an engine executing selected RTL blocks at cycle accuracy — connected through transaction-level co-modelling channels, so software workloads execute quickly while the hardware behaviour that matters stays at RTL fidelity <sup>[1]</sup>. The RTL side may be an emulator (Chapter 17) or, for smaller blocks and shorter workloads, an ordinary simulator; the architecture of the split is the same either way, and so are its consequences, which follow from the single channel [Figure 18.1](#) shows between the two sides: it carries transactions, never clock edges.



**Figure 18.1** — A hybrid co-simulation partition. On the virtual side the operating system and the driver under development execute untimed on the host, on an instruction-accurate application-core model alongside console, storage and network models; on the RTL side the block under verification and its bus interface run cycle-accurate. Everything between them crosses one untimed transaction-level channel — register reads and writes inward, interrupt and status events outward — and that hand-off is what deletes every bug living in the timing relationship between the two sides.

### Three reasons to split, and where the line goes

The published motivations are worth restating, because they are also the criteria for drawing the boundary. Hybrid co-modelling *reduces the RTL footprint* on the hardware engine by keeping well-modelled components virtual; exploits *model-maturity asymmetry*, leaning on mature virtual models — typically CPUs and standard peripherals — while keeping RTL fidelity for blocks whose behaviour is subtle or insufficiently modelled; and *balances throughput against fidelity*, accelerating end-to-end workloads while preserving cycle accuracy where timing, ordering and microarchitectural behaviour matter <sup>[1]</sup>.

Turn those into a placement rule. The boundary belongs where three things are true at once:

1. **It is already an architectural boundary with transaction semantics.** A memory-mapped register interface, a bus port, a mailbox. Something whose contract is written down as operations, because the channel can only carry operations. A boundary drawn mid-microarchitecture — across a pipeline, a credit counter, a handshake — has no transaction to send.
2. **The virtual side is mature.** You are trusting it, unverified, for the duration of every run. A model written last week to fill a hole in the partition is not a component you may lean on; it is a second design with no test plan.
3. **Everything whose timing you care about is on the RTL side.** This is the load-bearing one, and it is the subject of the next paragraph.

## What the boundary deletes

The channel between the domains is untimed. A transaction crosses it as a hand-off, not a waveform: the virtual core issues a write, the RTL side executes it in its own time, and the relation between “when the core issued it” and “when the block saw it” is an artifact of the co-modelling implementation, not a property of any system that will be built.

Therefore **every bug that lives in the timing relationship between the two sides is unreachable by construction**. If the core and the DMA engine contend for one memory port and only the DMA is RTL, the contention is gone — not modelled coarsely, gone. If a driver’s write races an interrupt raised in the same cycle, and the driver is virtual, the race cannot happen. That is not a defect in hybrid platforms; it is what they are. It is a defect only when nobody writes it down.

So write it down, on the plan row, in the same register of language Chapter 5 demands of every other exclusion: *“cross-boundary timing between the host cluster and the compute island is out of scope on this platform; covered by block-level full-RTL simulation and by the prototype’s system regression.”* A written exclusion is a coverage decision; an unwritten one is a hole with a green dashboard over it. Nor is the partition permanent: as a project matures the workflow is tightened by moving blocks from virtual to RTL, trading throughput for fidelity in a controlled manner <sup>[1]</sup> — the boundary is a scheduled artifact with a migration plan, not a one-time decision.

**Scenario.** The sensor hub is an always-on island with its own 32 kHz domain, a small DSP, and firmware that decides autonomously whether an event is worth waking the rest of the chip for. Its verification problem is hybrid by nature: the interesting behaviour spans the island’s RTL, the island’s firmware, and the host driver that configures it and receives its wake events. **Approach.** Run the island as RTL — small, new, and its wake-decision logic is the risk — and keep host cluster, OS and driver virtual, joined at the island’s register interface and its wake-event line. **Why.** The boundary is a real architectural one with transaction semantics, the virtual side is entirely stock, and the timing that matters (DSP pipeline, wake-decision register, interrupt assertion) is on the RTL side. Name what the split deletes: the *crossing* by which the wake-cause code reaches the system domain is now a channel hand-off, so the canonical multi-bit crossing bug — the reserved code 3'b110 observed for one cycle during a legal transition — is invisible here and stays the property of structural CDC analysis (Chapter 13). The platform verifies the *policy*; it says nothing about the *crossing*.

## Two generations, two answers

On the reference SoC, a hybrid platform is usually the wrong tool. The chip is small enough that full-RTL simulation of the whole thing is affordable, the firmware is a few thousand lines of bare-metal C, and a boot-to-main test runs in minutes. Building a co-modelling channel to avoid a cost you are not paying is engineering for its own sake — and it would trade away exactly what full-RTL simulation gives free at this

size: every interaction between the core, the two DMA channels and the memory, timed.

On the flagship, the calculus inverts. Full-SoC RTL simulation of an OS boot does not finish; the emulator is shared and scheduled; and for most of the project the mesh NoC and the LPDDR4 subsystem are still moving underneath everyone. A hybrid platform is how the driver team gets to run real driver code against the real compute cluster in month four rather than month eleven — with the NoC virtual, its timing out of scope, and both facts on the plan row. Same problem, two scales, opposite answers.

## 18.4 Firmware-driven verification

---

Firmware-driven verification uses the real boot sequence and the real device driver as stimulus. Not a testbench sequence that resembles what software will do — the binary that will ship.

The two ask different questions. A constrained-random environment asks *is every legal transaction handled correctly?* and answers by sampling the legal space widely. Firmware asks *does this sequence of operations, in this order, with these values, from this reset state, achieve the effect the software needs?* — and answers for exactly one trajectory. They are not competing methods but orthogonal projections of the same design, and firmware-driven verification earns its slot because its projection contains things the other structurally cannot. That this is mainstream rather than specialist work is measurable: hardware/software co-design and verification is the most-cited reason projects reach for emulation or FPGA prototyping, and in 2024, 84 percent of IC/ASIC projects incorporated one or more embedded processors [5].

### What it finds

**Initialisation ordering.** Hardware/software interaction is rarely a single access; it is a sequence that must follow rules and respect environmental constraints — and those access constraints and preconditions are frequently *not documented*, while being required nonetheless by the device hardware or by the bus systems between it and the core [7]. A testbench writer reads the register map and writes accesses to it; a driver writer reads the same map and produces the sequence the software actually needs — power-up order, clock enables, a PLL lock poll, a reset deassertion, a queue base address written before the queue is enabled and not after. That sequence is what the silicon will see for the rest of its life, and very often the only one anyone has tried.

**Register access races.** These live at the hardware/software interface, and are created by changes to that interface as often as they are latent in the design. In one published study, three variants of a driver differing only in their hardware abstraction layer and register layout were checked against each other; in one, a missing side-effect annotation introduced a race on the peripheral's input register that could make the device raise an interrupt incorrectly — and it had passed the simulation-



based verification previously applied to that design <sup>[7]</sup>. Nobody planted it. It arrived through a software-side optimisation of the interface, exactly the kind of change no hardware regression is watching.

**Error-recovery paths.** Every real driver contains recovery logic: a timeout, a retry with a shorter burst, a channel reset, a re-initialisation from scratch. Nobody writes directed tests for these, because the sequence *is* the driver's — not derivable from the register map, and a verification engineer inventing one invents a different sequence. Running the driver runs the recovery code, including the parts its author hoped never to run.

**Integration wiring.** The interrupt arriving on the wrong input, the address the header and the decode logic disagree about, the reset asserted for the wrong domain. Chapter 15's connectivity example is the archetype: block environments passed, the system regression passed, and the mismatch surfaced in the lab as a hang. Firmware is the first stimulus that traverses the whole path end to end with the software's own view of the map.

**Scenario.** The crypto engine sits inline on the memory path and is programmed through a key ladder: a sequence of writes that derives a working key from a root key, each stage gated by the previous one's completion. Its block environment covers every stage transition and every legal key length. On the prototype, a firmware image that re-keys between two secure sessions occasionally produces a session that decrypts to garbage — roughly once per few hundred re-keys, never reproducibly. **Approach.** Compare the driver's actual access trace against the engine's documented preconditions and find that there is no documented precondition covering *this* case: the driver writes the next key-slot register while the previous operation is still draining. **Why.** This is a known shape. In a formal check of an open-source SPI peripheral's access preconditions, any write to the device during an active transmission was found capable of producing divergent behaviour, and the conclusion drawn was that firmware must implement mechanisms to avoid generating such a pattern <sup>[7]</sup> — a requirement on the *software*, discovered in the *hardware*, and written down in neither. The engine is not wrong and the driver is not wrong. The interface contract is incomplete, which is Section 18.5's subject.

### What it cannot find

Firmware is one path. It uses the burst lengths the driver chooses, the descriptor shapes the driver builds, the interrupt configuration the product ships, and the buffer alignments the allocator happens to produce. It will never issue the legal transaction the driver has no reason to issue; it will never back-pressure an interface the way a hostile responder does; and it will never vary anything, because varying things is not what a driver is for.

So a firmware run's coverage is a thin, deep thread through a wide space — the complement of a constrained-random environment's, which is wide and shallow (Chapter 9). Neither substitutes for the other, and a project with only one has a predictable blind spot: firmware-only ships a design that works for the driver it grew up with and

breaks on the next; random-only ships a design where every transaction is legal and the initialisation sequence deadlocks.

There is a published route out of the first half of that blind spot, and it is worth naming because a team carried it all the way to silicon. At AMD, test intent written in the Portable Test and Stimulus Standard was compiled into UEFI and bare-metal tests that ran unchanged on an emulated AMD APU and then on post-silicon parts, reaching the hardware through an in-house runtime layer called uDREX that abstracts the BIOS and starts the test on every enabled thread; on the emulator the test and uDREX were backdoor-loaded into DRAM, and in the lab the team used the same descriptions to compose fresh power-management firmware tuning scenarios instead of the fixed set a shipping driver happens to exercise [8]. Notice what is *not* claimed there. The deck reports no bug count, no speedup and no effort saving, and it records a real cost of its own — test generation time that the authors put on record as having the potential to increase exponentially. What changes is authorship: a lab engineer or an architect can compose a system-level sequence in the afternoon it is needed, which is the variation a driver alone structurally cannot supply. Chapter 9 gives the language itself its own treatment.

A second limit is easy to miss. **Firmware is not an oracle for the hardware.** A driver checks that it achieved its own goal, not that the design behaved correctly. A hardware bug the driver's retry logic silently absorbs — a descriptor that fails and succeeds on the second attempt — is a passing boot with a defect in it, and nothing in the software reports it. Firmware-driven runs need their own checkers, which brings us to the artifact.

### Worked example: a property that has to survive the fabric

The reference SoC's DMA carries this book's canonical protocol property, and the flagship's DMA inherits it unchanged, because it inherits the same 4 KB legalization discipline (Arm IHI 0022H.c §A3.4.1) [9]:

```
property p_no_4kb_crossing;
 @(posedge clk) disable iff (!rst_n)
 (awvalid && awready && (awburst == 2'b01)) |->
 (16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) <= 16'd4096;
endproperty
a_no_4kb_crossing : assert property (p_no_4kb_crossing);
```

On the prototype that statement does not exist. Concurrent assertions are a simulation and proof construct; there is no assertion engine in the fabric, and where a tool-chain offers synthesisable assertion support, what it can report is bounded by the same visibility limits as everything else. To survive the trip the check must become logic — logic that reports its verdict through a register the driver can read.

```
// Prototype-side transliteration of a_no_4kb_crossing: the same expression,
// evaluated in fabric, latching evidence into registers the driver can read.
module dma_4kb_monitor #(
 parameter int unsigned ADDR_W = 40, // the flagship's physical address width
 parameter int unsigned CNT_W = 16
```

```

) (
 input logic clk,
 input logic rst_n,
 // write-address channel of one DMA manager port, observed passively
 input logic awvalid,
 input logic awready,
 input logic [ADDR_W-1:0] awaddr,
 input logic [7:0] awlen,
 input logic [2:0] awsize,
 input logic [1:0] awburst,
 // exposed through the prototype's debug register block
 output logic viol_seen,
 output logic [CNT_W-1:0] viol_count,
 output logic [ADDR_W-1:0] first_addr,
 output logic [7:0] first_len,
 output logic [2:0] first_size
);
 logic viol;

 // Identical to the property's consequent, negated: INCR bursts only,
 // 16-bit arithmetic, evaluated on an accepted address handshake.
 always_comb begin
 viol = awvalid && awready && (awburst == 2'b01) &&
 ((16'(awaddr[11:0]) + ((16'(awlen) + 16'd1) << awsize)) > 16'd4096);
 end

 always_ff @(posedge clk or negedge rst_n) begin
 if (!rst_n) begin
 viol_seen <= 1'b0;
 viol_count <= '0;
 first_addr <= '0;
 first_len <= '0;
 first_size <= '0;
 end else if (viol) begin
 if (!viol_seen) begin
 viol_seen <= 1'b1;
 first_addr <= awaddr;
 first_len <= awlen;
 first_size <= awsize;
 end
 if (viol_count != {CNT_W{1'b1}}) begin
 viol_count <= viol_count + 1'b1; // saturate rather than wrap
 end
 end
 end
endmodule

```

Read what the transliteration costs; the losses are the chapter's argument in miniature. You lose the waveform: when `viol_seen` comes back set you have one address, one length and one size, and no picture of what the interconnect was doing around them. You lose every violation after the first, which is why the counter is there — `viol_count` distinguishes *the driver is systematically emitting illegal descriptors* from *one corner case fired once*, and without it the sticky bit answers a much weaker question. And you pay in fabric: real logic, competing for the same resources and routing as the design, in the same currency that buys debug probes.

What you keep is the part that matters here. The check is evaluated at the instant of the violation, at full speed, on every burst of a workload no simulator will ever reach

— so the detection distance (Chapter 8) between the design’s mistake and the verdict is zero cycles, even though the distance between the verdict and *your knowledge of it* is however long until the driver next polls. That is condition 2 of Section 18.1’s admission test, satisfied concretely: an oracle that survives the visibility collapse. Build the monitors before the bitstream, because after it they cost a compile.

## 18.5 Who owns the bugs

Everything so far has been technical. This section is the organisational half — the half that actually breaks.

A failure on the prototype belongs to the hardware or to the software, and both teams arrive at triage with a green history and no evidence about the other: the hardware team’s block environments, assertions and coverage all pass; the software team’s driver has run for months against a virtual platform without a failure. Each team’s evidence is real, each is about its own artifact, and neither says anything about the *interface between them* — which is where the bug almost always is. The default outcome is a week of alternating hypotheses, and it is the default because nothing in either team’s normal practice produces what triage needs.

That information is producible, and far cheaper to arrange in advance than to reconstruct after. Four things make prototype triage tractable.

**1. A run you can repeat.** In simulation a run is a seed plus a design revision plus a tool build (Chapter 12), and repeating it is a command line. A prototype run has no seed: it has real time, real I/O, a real network peer, and a boot that consumes entropy. The nearest equivalent is a manifest pinning everything the bitstream and the boot depend on — RTL revision, bitstream hash, partition assignment, clock configuration, firmware and OS image hashes, boot arguments, board identity, input data. Without it, “it failed on Tuesday’s build” is not a fact anyone can act on.

Repetition then substitutes for replay. The 2026 flow of §18.1 hits exactly this limit: its OS boot is a single indivisible test that cannot be distributed across the boards the rest of the suite parallelises over, so it is executed eight times instead, to establish that the result is stable <sup>[3]</sup>. That is Chapter 12’s repeat-*N* stability check transplanted to a platform where determinism cannot be assumed — the honest answer to “was that intermittent, or did I imagine it?”

**2. One timeline.** Firmware writes a log; in-fabric monitors latch into registers; the trace buffer captures bus activity. If those three record time in three different ways, an engineer establishing whether the driver’s timeout preceded or followed the DMA’s error is guessing. A common timestamp source — a free-running counter readable by software and sampled by the monitors, at a resolution finer than the events you need to order — turns three records into one ordered narrative. The always-on domain’s 32 kHz reference is not that counter. Cheap before the build, impossible after it.

**3. A shared definition of “the same run”.** Divergence between platforms does real damage: monitors and harnesses re-engineered to meet a hardware engine’s

timing and capacity constraints leave divergent environments, and that divergence introduces subtle inconsistencies in observed behaviour, complicating debug and making an issue harder to reproduce across platforms <sup>[1]</sup>. The rule that follows: checks existing on both platforms must be the *same checks*, generated from one source rather than implemented twice. The 4 KB monitor of Section 18.4 and the assertion it was transliterated from belong to one description — maintained separately they will diverge, and that is the day the prototype and the simulator disagree about whether a burst was legal.

**4. A path back to visibility.** A prototype failure’s value is realised when it becomes a simulation testcase — the same escalation post-silicon bring-up depends on (Chapter 20), arriving one platform earlier. Build it deliberately: the manifest identifies the image, the monitor identifies the first offending transaction, and someone must be able to construct a stimulus that reproduces it where every net is visible. A team without that path debugs every prototype bug at the worst observability it will meet before the lab — the trade Section 18.1 warned against making by accident.

### When a firmware bug is really a specification bug

Now the judgment this chapter exists to teach. When a prototype failure lands, run the triage against the specification, not against the two implementations, and there are three outcomes rather than two:

- **The specification says X and the hardware does X.** The software is wrong. Fix the driver.
- **The specification says X and the hardware does Y.** The hardware is wrong. Fix the RTL, and add the test that should have caught it.
- **The specification does not say.** *Neither team owns it.* This is a specification defect, and treating it as a firmware bug is the single most common triage error in hardware/software co-verification.

The third case is not rare, and it is not a failure of diligence — it is structural. A peripheral’s access constraints are consequences of its internal architecture, and a designer who satisfies them naturally, without ever needing to state them, has no prompt to write them down. Formal checks of open-source peripherals bear this out at close range: one device’s software reset proved not to be implemented at all, a fact recorded in the HDL source and absent from the documentation <sup>[7]</sup>. Nobody lied; nobody checked.

The sharpest instance inverts the reflex entirely. An open-source UART’s baud-rate counters run continuously, and reprogramming the rate from a higher target value to a lower one while the counter sits between the two leaves it un-reset until it wraps — rendering the device unresponsive for up to  $2^{18}$  clock cycles, and **the software cannot avoid this, because nothing in the interface lets it infer the counter’s value** <sup>[7]</sup>. As a lab symptom — “the console stops responding after the driver changes baud rate” — that looks exactly like a driver bug, and a firmware team will spend days hunting the missing wait. The second clause is the load-bearing one: *no correct driver exists*. The defect is in the hardware, the omission is in the document,

and the only fix available to software is a workaround that must be specified before it can be written.

The pressure shows in the industry data: among the root causes of logical and functional flaws in the 2024 study, changing, incorrect or incomplete specifications remain a common concern of verification engineers and project managers, alongside design error [5].

**Scenario.** The radar front-end’s firmware computes per-channel calibration coefficients at start-up and writes them into the FFT chain’s correction registers. On the prototype, detections are consistently off: targets appear at roughly twice the expected amplitude, and the CFAR threshold logic behaves accordingly. The DSP team demonstrates the datapath is bit-exact against its reference model for every input it was given. The firmware team demonstrates the coefficients are correct, checked against the calibration procedure. **Approach.** Stop arguing about the two implementations and go to the register description. The correction register is 16 bits wide, and the document says only “signed coefficient”. The RTL treats it as Q2.14 — two integer bits, fourteen fractional; the firmware computes Q1.15. Both are self-consistent, both match the document, and the values differ by a factor of two. **Why.** This is the third outcome, in its purest form. There is no software bug and no hardware bug; there is a field whose interpretation was never written down, and two teams who each supplied the missing half from their own conventions. The fix is not a patch — it is an entry in the register description, from which the decode logic, the model and the firmware header are all generated (Chapter 15), plus a plan row asserting the interpretation so that the next 16-bit “signed coefficient” cannot repeat it.

The organisational consequence follows. A specification defect filed as a firmware bug is closed by a driver patch, and the document stays wrong for the next driver, the next derivative and the next customer. Route the third outcome somewhere it will survive: Chapter 5’s spec-hole log, which exists exactly for questions the specification cannot answer, plus three artifacts — the document change, a checker enforcing the newly stated rule, and a plan row that owns it. And give prototype triage a named owner with a foot in both camps, on a rotation drawn from both teams. The alternative is the default, and the default costs a week per incident.

---

## IN PRACTICE

Prototypes are built by two or three people who become the only ones able to operate them; fix that first, because if bringing up a bitstream is tacit knowledge the platform is unavailable to the teams it exists for. Automate the path from RTL revision to programmed board and treat that automation as a deliverable. Board access is contended, so the queue is a project resource with a policy — decide who preempts whom before the week it matters — and bitstream builds run overnight in a pipeline of their own, off the per-commit tier (Chapter 12) where no multi-hour job belongs.



The messy parts are consistent across projects. Probes get inserted for one investigation and left in, and nobody knows which are still connected to anything: keep a manifest of what each build instruments. Firmware and RTL versions drift, and half of “the prototype is broken” is an image mismatch, which a boot-time hash check eliminates for a day’s work. The prototype is also the demo machine, and will be requested for a customer visit in the same week the DMA bug is being chased — a management decision, not an engineering one. And the block left off the prototype because it could not be mapped, the analog PHY or the trimmed memory macro, is the one nobody thinks about again until bring-up; keep it on the plan as an explicit exclusion with a named alternative.

---

## PITFALLS

1. **Reading the prototype as a measuring instrument.** Symptom: coverage goals or sign-off criteria assigned to prototype runs, or latency numbers from a partitioned build quoted in an architecture review. Cure: cross-device links and reduced clocks make those latency figures an upper bound with an unknown gap, and closure belongs where the design is observable and stimulus controllable; the prototype validates integration and software.
2. **Debug instrumentation decided after the first failure.** Symptom: every investigation begins with a multi-hour rebuild to add probes. Cure: instrument for the questions you expect before the build, and budget the fabric for it.
3. **Firmware developed only against an untimed model.** Symptom: the driver has never met back-pressure, latency or concurrency, and meets all three first on a platform where you cannot see. Cure: run driver code against RTL early on a hybrid platform, even if only for the block whose timing it depends on.
4. **A hybrid partition whose deletions are undocumented.** Symptom: a green system-level campaign and an untested cross-boundary interaction nobody can name. Cure: every partition carries a written scope exclusion on the plan row, like any other coverage decision.
5. **Prototype runs with no manifest.** Symptom: “it fails on the old bitstream” and nobody can say which one that was. Cure: pin RTL revision, bitstream hash, image hashes, clock configuration and board identity per run, and repeat indivisible tests to establish stability.
6. **Filing a specification defect as a firmware bug.** Symptom: the driver gets a workaround, the document stays silent, and the next derivative repeats the failure. Cure: triage against the specification; when it does not answer, the outcome is a spec-hole entry, a document change and a checker — not a patch.

---

## SUMMARY

- A prototype is not a cheap emulator. An emulator is a fast simulator with reduced visibility; a prototype is early silicon with more visibility than silicon — and proto-

typing's practical trade-off is exactly that observability constraint, which pushes deep debug elsewhere <sup>[1]</sup>.

- Visibility on a prototype is decided before the run: a few thousand signals, chosen ahead of the bitstream, and changing the choice means a recompile measured in hours <sup>[2]</sup>.
- A workload earns a prototype slot when it needs the cycles, has an oracle that survives the visibility collapse, and can plausibly be reproduced back where things are visible. The second condition is the one teams skip and the third is the one that costs them.
- Virtual platforms preserve the basic functionality of the hardware/software interfaces and support coarse-grained checking of use cases <sup>[2]</sup> — which is precisely why the class of bugs made of two things happening at once is not merely missed there but unreachable.
- A hybrid platform's untimed boundary deletes every bug living in the timing relationship between its two sides. That is what the platform is, and it is a defect only when the deletion is not written on the plan row.
- Firmware-driven runs find what a testbench does not — initialisation order, interface races, recovery paths, integration wiring — and cannot find what a generator does, because a driver is one trajectory that never varies. They complement constrained-random stimulus; they do not replace it.
- Hardware/software access preconditions are often required by the device and absent from its documentation <sup>[7]</sup>. When a prototype failure lands and the specification does not answer, the bug belongs to the specification, and closing it as a driver patch guarantees the next team repeats it.

---

## FURTHER READING

**From the corpus.** <sup>[1]</sup> is the best single source for the platform taxonomy of §18.1 and §18.3 — prototyping, ASIC-class emulation and hybrid co-emulation, their trade-offs in capacity, speed and visibility, the compile-versus-execute distinction, and the divergent-environment problem that makes cross-platform reproduction hard. <sup>[2]</sup> places all of it on the pre-silicon-to-post-silicon observability gradient, and is the source for the virtual-model role and the bitstream-recompile cost of changing what you can see. <sup>[3]</sup> is the most concrete published account of an FPGA validation flow end to end: platform integration, multi-device partitioning, and a three-class suite from integration checks through bare-metal benchmarks to OS-level driver tests. <sup>[7]</sup> treats the hardware/firmware boundary as a formal object and supplies §18.5's central lesson on undocumented access preconditions. <sup>[5]</sup> and <sup>[6]</sup> give the industry framing. <sup>[4]</sup> works the acceleration-ready testbench question for readers crossing into Chapter 17's territory.

**Outside the corpus.** D. Amos, A. Lesea and R. Richter, *FPGA-Based Prototyping Methodology Manual*, is the standard practitioner's treatment of design-for-prototyping — partitioning, clocking and memory substitution at book length. P. Rashinkar, P. Paterson and L. Singh, *System-on-a-Chip Verification: Methodology and Techniques*,

contains the classic chapter-length account of hardware/software co-verification flows. F. Ghenassia (ed.), *Transaction-Level Modeling with SystemC*, is the reference for the modelling styles that virtual platforms and hybrid boundaries are built from.

---

## REFERENCES

1. **P14** T. Rahman, S. Saha, A. Y. Alhurubi, S. K. Saha, F. Farahmandi, and M. Tehranipoor, "Emulation-based System-on-Chip Security Verification: Challenges and Opportunities," arXiv preprint arXiv:2604.15073v1, April 2026.
2. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
3. **P25** S. Ahmed, A. Kropotov, R. I. Genovese, B. Homs, E. Merino, F. Urbani, et al., "Verification and Validation (V&V)-in-the-Loop for RISC-V Design: The Holistic Vision of BZL," arXiv preprint arXiv:2604.27013v1, April 2026.
4. **D33** A. Grove, "UVM hardware assisted acceleration with FPGA co-emulation," Proc. DVCon Europe, 2015.
5. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
6. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
7. **T1** M. Schwarz, "Formal Hardware/Firmware Co-Verification of Optimized Embedded Systems," Ph.D. dissertation (Dr.-Ing.), Technische Universität Kaiserslautern, Germany, 2021.
8. **D29** P. Gupta and M. Vax, "Test driving Portable Stimulus at AMD," Proc. DVCon U.S., 2019.
9. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.

---

## 19. Gate-Level, Timing and Power-Aware Verification

The reference SoC's netlist arrives on a Tuesday. By Wednesday evening the gate-level regression has produced two results, and the difference between them is the subject of this chapter.

The first result is a boot test that dies at time zero in a fog of unknowns. Nine hours of debug establish that the netlist's scan-enable pin was never driven by a testbench written against RTL, where no such pin existed; that a scoreboard comparison written with `!=` had been quietly reporting success on all-unknown data for two of those hours; and that a white-box checker on the DMA's internal descriptor pointer no longer has anything to bind to, because synthesis dissolved the register it watched. Nothing has been learned about the design. A great deal has been learned about the testbench.

The second result is the one that saves the tape-out. Run with the chip's power intent loaded, a sleep/wake test shows the event unit coming out of a power-down cycle with its interrupt-mask configuration destroyed — the canonical escape of Chapter 7, a bug that plain RTL simulation could not represent at all, because plain RTL simulation does not model losing power.

Same platform, same week, same engineers. One run was ritual; the other was the only place in the entire flow where that bug could have been caught. A chapter on verification after synthesis is really a chapter on telling those two apart, because gate-level simulation is expensive enough that running it out of habit is a decision with a price tag.

---

### CHAPTER MAP

- §19.1 establishes what gate-level simulation genuinely finds that RTL simulation cannot — and which of its traditional justifications are better discharged by other tools entirely.
- §19.2 distinguishes a real gate-level failure from a testbench that was never built to survive one.
- §19.3 states what SDF back-annotation and the timing checks actually assert, why static timing analysis and not simulation is the authority on timing, and what a gate-level run adds on top of it.
- §19.4 develops how power intent becomes a formal, checkable description, and why the bugs live in the *sequencing* rather than in any one domain.
- §19.5 budgets gate-level and power-aware runs: what earns the cost, and what to schedule earlier instead.

## 19.1 Why gate-level simulation still exists

Four arguments are made for gate-level simulation (GLS). They are not equally good, and you are owed the ranking rather than the list.

**“It checks that synthesis did not change the function.”** This is the oldest argument and the weakest one. Running the functional testbenches on the netlist verifies that the synthesis tool works properly — a task for which formal equivalence checking and static timing analysis are better technologies <sup>[1]</sup>. Chapter 15 gives the reason in full: equivalence checking is exhaustive over the transformation, needs no stimulus, and finds the class of synthesis defect — a wrong result in a wide arithmetic operator, say — that occupies a vanishing fraction of the input space and would take simulation forever to stumble into. If your project runs logic equivalence checking after synthesis, and it should, this argument for GLS has already been spent.

**“It checks timing.”** It does not, and Section 19.3 is about why. Static timing analysis examines every timing arc in the netlist structurally, across process corners, voltage and temperature, and on-chip variation <sup>[2]</sup>; a simulation exercises the paths one test happens to sensitise, at one corner, with one set of annotated delays. Using GLS as a timing check is a category error with a comforting appearance.

**“It finds X and reset problems the RTL hid.”** This one is real, and it is the strongest of the four. Chapter 13 established the mechanism: RTL simulation is optimistic about unknowns at every decision, because `if`, `case` and the edge constructs take a definite branch on an ambiguous condition. Those constructs no longer exist in a netlist — they have become gate primitives and flip-flop models that do not exhibit the optimism <sup>[3]</sup>. So a design that passed at RTL because an unknown was silently resolved will fail at gate level, at reset, where the unknowns live. The cost is that the same change makes pessimism worse, which is Section 19.2’s problem.

**“It exercises structures that are not in the RTL at all.”** Also real, and routinely underweighted. Scan chains, test-mode multiplexers, compression logic, self-test wrappers and the debug fabric’s boundary are inserted *after* RTL sign-off, so no RTL simulation has ever seen them and the obligation they carry — that every one is inert in functional mode — has no earlier place to be checked. This is also where a structure inserted for one purpose degrades another: scan insertion on the *second* flip-flop of a two-flop synchronizer adds multiplexer logic on the metastable path and erodes the settling window, degrading mean time between failures, while scan insertion on the first has no such effect <sup>[2]</sup>. That finding is worth reading twice, because it did not come from a simulation and could not have: it is a structural property of the implemented netlist.

### Two sources, one apparent contradiction

Two corpus sources appear to disagree outright, and resolving them gives the chapter’s thesis in miniature.

A class of metastability bugs cannot be demonstrated on an RTL model in simulation; reproducing them requires a gate-level model with timing, typically unavailable until late in the flow [4]. Yet when back-end processes interfere with synchronizer operation, the answer is *not* gate-level simulation on a back-annotated netlist but structural and timing analysis of the final implementation [2].

Both are right, about different questions. GLS with timing can *exhibit* one instance of the timing violation the class turns on — one path, one corner, one alignment of edges. It cannot *cover* the class, because whether a given alignment occurs in a given run is an accident of the stimulus. Sign-off needs the space; a structural analysis over the netlist gives that, where a simulation gives one sample. **Exhibiting a failure and signing off its absence are different jobs**, and GLS is only ever qualified for the first.

That generalises into the chapter’s rule. Every time a gate-level run is proposed, ask which of two things it is for: producing an instance of a phenomenon nothing else can produce, or discharging a plan row. The first is legitimate and occasionally irreplaceable. The second almost always belongs to a static tool that answers the same question exhaustively and in minutes.

**Scenario.** The flagship’s LPDDR4 subsystem includes a PHY training state machine that runs at link bring-up, and the team’s plan carries a row for it. The proposal on the table is to run the training sequence at gate level with back-annotated delays, on the argument that “training is a timing thing”. **Approach.** Split the row. The functional obligation — that the training state machine reaches a trained state from every starting condition and reports failure correctly when it cannot — is verified at RTL, where a thousand seeds are affordable. The timing obligation — that the trained delay settings satisfy setup and hold at every corner — is static timing analysis’s, and no simulation improves on it. What remains for gate level is exactly one thing: that the training sequence, running on the real netlist with real delays, does not violate a timing check that the constraints declared impossible. **Why.** The undivided row would have cost days of gate-level runtime and discharged neither obligation properly: too few seeds to be functional evidence, one corner to be timing evidence. The split costs less and produces two defensible arguments instead of one indefensible one.

The trajectory is one practitioners recognise, though no survey quantifies it: the gate-level tier keeps shrinking, squeezed from below by static tools that reach the same conclusions faster and from above by the cost of running a netlist. What survives is a narrow, high-value residue: unknowns at reset, structures that exist only after synthesis, power intent applied to a netlist, and a check on the timing constraints themselves. Everything else is habit, and habit at gate-level prices is expensive.



## 19.2 Reading a gate-level failure

Chapter 13 owns the semantics of unknowns and Chapter 12 owns the term: RTL simulation is *optimistic*, resolving an ambiguous condition into a definite branch. Synthesis removes the constructs that do that, so the optimism goes — and pessimism gets worse, because the netlist evaluates far more expressions and each one is all-or-nothing about unknown operands [3]. The practical consequence is that the first gate-level run of a project produces a large number of unknowns, of which a minority are design bugs. The engineering skill this section teaches is triage, and the first thing to internalise is that, in practice, **most first gate-level failures are not about the design at all.**

### Four families, and the tell for each

**Genuinely unreset state.** The tell: the unknown is present at time zero, survives reset de-assertion, and traces back to a flip-flop with no reset pin in the netlist. This is the class GLS exists for, and it is worth the run. It is also findable earlier and more completely by formal reset analysis, which reports every register still unknown when the reset period ends (see Chapter 13) — so a GLS run that discovers one is a GLS run that discovered a gap in the entry gate.

Not every unreset register is a defect, and this is where judgment enters.

**Scenario.** The radar front-end’s FFT pipeline holds several stages of butterfly registers, deliberately left unreset: they are pure datapath, the reset tree would be large, and every one of them is overwritten by the first frame that flows through. At gate level the first run floods the whole chain with unknowns for the first frames, and a junior engineer opens a bug. **Approach.** It is not a bug, but it is not obviously not a bug either, and the difference has to be *argued* rather than assumed. The argument has one premise: no control decision consumes any of those registers before the flush completes. Check it — the pipeline’s valid chain is reset, the CFAR threshold logic reads only post-flush samples, and no comparator output feeds a state machine during the flush window. Record the argument on the plan row, and pin it with an assertion that no downstream control signal is unknown once the valid chain has been high for the pipeline’s depth. **Why.** The unreset datapath is a legitimate area decision and it costs nothing in silicon. What it costs in verification is a permanent obligation to show that the unknown never reaches a decision — an obligation nobody discharges by looking at a waveform and deciding the X “looks like flush data”. The assertion turns the argument into something a later change can break loudly.

**Pins that exist only in the netlist.** Scan-enable, test-mode, scan inputs, BIST enables, the clock-multiplexer select for at-speed test. The tell is decisive: the unknown’s cone of influence starts at a top-level input with no RTL counterpart. This is not a design bug and not a testbench bug either, exactly — it is a missing piece of harness that nobody could have written earlier. Build it once, as a named functional-mode tie-off module, and review it as an artifact: tying `test_mode` to the wrong value

does not produce an error, it produces a netlist that passes tests while running in a mode nobody ships.

**Environment infrastructure that cannot survive a netlist.** White-box checkers are tied to a specific implementation and cannot be used in gate-level simulation [1] — synthesis dissolved the signal they watched, or renamed it, or replicated it into three. The same applies to any hierarchical reference: a bound checker’s path, a register model’s back-door path (see Chapter 10), a `force` on an internal net. The tell is that the failure appears in the environment’s own messages — an elaboration error, a null handle, a back-door read returning unknowns while the front door works. Chapter 11 gives the standard answer: bound checkers carry macro guards so they can be compiled out of a gate-level run, and the checks that must survive are the ones written against ports.

**Pessimism artifacts.** The tell is that the unknown appears mid-logic with no source: no unreset register upstream, no undriven pin, just an expression whose operands were partially unknown. These consume the most debug time and teach the least. The remedy is procedural — clear the real sources first, in order, and re-run; each one removed collapses a large fan-out of artifacts.

### The comparison that stops comparing

The third family has a silent cousin that deserves its own artifact: an environment that does not fail at all, but quietly stops checking. A scoreboard’s comparison operator decides whether an unknown is a failure or a pass.

```
// `!==' is case inequality: its result is always 1'b0 or 1'b1, so an
// unknown in `act` always reaches the report. `!=` is logical inequality
// -- 1'bx when the relation is ambiguous -- and `if (1'bx)` takes the
// false branch, scoring the transaction as correct. A 2-state variable
// holding that result would convert the x to 0 by a second route.
always_comb begin
 if ($isunknown(act))
 $error("unknown in sampled data: %0b", act);
 else if (act !== exp)
 $error("mismatch: got %0h, expected %0h", act, exp);
end
```

The language rule is exact and the exactness matters. For the logical equality and inequality operators, the result is `1'bx` only when unknown or high-impedance bits make the relation *ambiguous*; the case operators include those bits in the comparison and always yield a known result [5]. So `!=` does not simply “miss unknowns”. A value corrupted only in bits that already differ from the expectation is still reported, because the relation is not ambiguous. What escapes is the fully corrupted value, and the partially corrupted one whose unknowns sit precisely on the bits that would have carried the difference.

That selectivity is why the defect survives review: the checker works on most of the data most of the time, and fails exactly where the data is worst. It fails *silently* — the run is green, the coverage is collected, the plan row is discharged. Audit this before a testbench is ever pointed at a netlist. The cure is two lines: an explicit unknown

check on every sampled value, ahead of the comparison, so that “unknown” is its own verdict rather than a special case of “equal”.

### The order of operations

The three things a gate-level run can turn on — timing annotation, power intent, and the netlist’s stage — are independent, and teams routinely turn them on together and then debug their interaction. Do not. Bring the environment up against a zero-delay netlist with no power intent, on the shortest reset-and-boot test in the suite, and add nothing until that run is clean; every subsequent addition then has exactly one suspect. The first clean zero-delay boot is a milestone worth naming in the plan, because nothing else in this chapter can start until it exists.

## 19.3 SDF back-annotation and what a timing check asserts

RTL has no delays. A netlist has library delays, but they are estimates computed for an average output load, and in a real netlist every instance of the same cell drives a different number of inputs over wires of different lengths. Back-annotation replaces those estimates with per-instance values calculated from the physical netlist or the layout geometry, delivered in a Standard Delay Format file, so each gate instance — and each pin-to-pin connection — carries its own number <sup>[1]</sup>.

What SystemVerilog takes from the file is precisely bounded: specify path delays, specparam values, timing-check constraint values, and interconnect delays, with the values themselves coming from delay-calculation tools that use connectivity, technology and layout geometry <sup>[5]</sup>. Everything else in an SDF file is ignored.

### Three ways annotation lies quietly

**The un-annotated island.** Any timing value for which the SDF file provides no value is left unmodified, retaining its pre-annotation value, and the annotator is required only to warn about data it cannot annotate. The reading task chooses its scope from an argument, so a mis-scoped call — pointing at the wrong instance, or at a block whose hierarchy the SDF was written against differently — annotates nothing there and leaves an entire subsystem on library defaults, frequently zero. Nothing fails. The run is fast, clean and meaningless for that block. The defence is procedural: the annotation log, in which each individual annotation produces an entry <sup>[5]</sup>, is an artifact to review, its annotated-instance count compared against the netlist’s.

**The corner nobody chose.** Which member of each min/typ/max triple gets annotated is a string argument, whose default value leaves the choice to the simulator, and a separate argument applies scale factors to the triples <sup>[5]</sup>. So the operating corner of a gate-level run — the one property that determines whether the run says anything about setup or about hold — is a defaulted parameter of a system task, easily inherited from a script nobody has read in two years. A run whose corner nobody selected is not evidence about any corner. Name it in the run manifest (see Chapter 12) alongside the seed and the tool build.

**The cost that shapes the flow.** Netlists contain millions of gates and millions of pin-to-pin connections, each needing annotation, so the process is slow enough to be worth minimising: annotate once at compile time and run many test cases against the same compiled model, rather than repeating annotation per test <sup>[1]</sup>. This is not a micro-optimisation — it is the reason a gate-level tier is organised as a small number of long runs rather than a large number of short ones, and it interacts badly with per-test randomisation.

### What a timing check actually says

The system timing checks fall into two groups. One group defines a stability window with respect to a reference signal and checks when the data signal transitions relative to that window: `$setup`, `$hold`, `$setuphold`, `$recovery`, `$removal`, `$recrem`. The other measures the difference in time between two events, except `$nochange`, which involves three: `$skew`, `$timeskew`, `$fullskew`, `$width`, `$period`, `$nochange`. Evaluation turns on two events — a transition on the *timestamp* signal records the time, a transition on the *timecheck* signal triggers the evaluation — and `$setuphold` simply combines the setup and hold windows into a single check <sup>[5]</sup>.

None of that, by itself, does anything but print. The mechanism that makes a timing violation *functional* is the notifier:

```
// Abridged excerpt from a library flip-flop model. The rows handling
// unknown inputs are elided; in a real cell they outnumber these, and they
// are where gate-level pessimism comes from.
primitive dff_udp (q, clk, d, notifier);
 output q; reg q;
 input clk, d, notifier;
 table
 // clk d ntf : state : q
 r 0 ? : ? : 0 ;
 r 1 ? : ? : 1 ;
 f ? ? : ? : - ;
 ? * ? : ? : - ;
 ? ? * : ? : x ; // any notifier event drives the output to x
 endtable
endprimitive

module dff_with_timing_check (output wire q, input wire clk, input wire d);
 reg notifier;
 dff_udp u_ff (q, clk, d, notifier);
 specify
 // These limits are the library's defaults; SDF back-annotation
 // overwrites them per instance.
 specparam tSU = 120, tHLD = 40;
 $setuphold(posedge clk, d, tSU, tHLD, notifier);
 (posedge clk => (q += d)) = 90;
 endspecify
endmodule
```

Every timing check can take an optional notifier, a variable the check toggles whenever it detects a violation, so that the model can make its behaviour a function of violations — the canonical use being to drive the device's output to `x` <sup>[5]</sup>.

That one design decision explains most of what gate-level debug feels like. A timing violation is not a message you note and move past: it injects an unknown into the datapath, which propagates and produces a functional failure somewhere else entirely, potentially thousands of cycles later. So “twelve timing violations, all in one block, early in the run” is never benign — it is the *cause* of whatever the scoreboard reports at the end. Resolve timing-check output before debugging any functional mismatch in a back-annotated run, and resolve it in time order.

### Static timing analysis is the authority

Static timing analysis checks all legal timing parameters in a gate-level representation by analysing the netlist structurally to identify every possible timing arc, comparing them against a timing-constraints file that states the required timing; and it does so not against one set of parameters but across process variation, minimum and maximum voltage and temperature, and on-chip variation, with hold analysis critical at the best-case corner and setup at the worst <sup>[2]</sup>.

Set that beside what a gate-level simulation gives: the paths one test happened to sensitise, at one corner, under one annotation. It cannot report margin, cannot cover the arcs no stimulus reached, and cannot tell you a path passes at typical and fails slow unless someone ran it there. **STA answers the timing question exhaustively; GLS samples it.** Treating GLS as the timing sign-off is expensive twice over — once in runtime, once in false confidence.

### What gate-level simulation does add

The constraints file that STA reads is *manually generated* to match the device specification, and enhanced through synthesis, place-and-route and clock-tree synthesis as the netlist evolves <sup>[2]</sup>. Every exception in it — a false path, a multicycle path, a case-analysis setting — is a human claim about the design, and STA does not check those claims. It obeys them. A path declared false is a path STA stops analysing.

A back-annotated gate-level run makes no such assumption. It applies the real delay to the real path and lets the real timing check evaluate. If the exception was wrong, the check fires. That is the genuine, irreplaceable contribution of GLS with SDF: **not a check on the timing, but a check that the constraints described the design you actually built.**

**Scenario.** The neural accelerator’s weight-decode path is declared a multicycle path in the constraints, on the argument that a job takes several cycles to set up before the 16-lane MAC datapath consumes a weight. The declaration is correct — at 8-bit weights. At 2-bit weights the decoder produces four weights per fetch and the datapath runs one result per cycle, so the same path is single-cycle. STA reports timing met, because it was told not to look. **Approach.** One gate-level run of the accelerator’s quantization suite at the narrowest precision, back-annotated at the slow corner. \$setuphold violations appear on the decode registers within a few hundred cycles, the notifier drives unknowns into the MAC array, and the scoreboard mismatches. The fix is in the constraints file, not the netlist. **Why.** No

amount of RTL simulation could find this: at RTL the path has no delay. No amount of STA could find it either: STA was given the exception as a premise. The bug lives exactly in the seam between the two, and a mode-dependent exception is the commonest shape it takes — which is why the gate-level tests worth running are the ones that exercise *configurations the constraints made assumptions about*, not the ones that exercise the most logic.

## 19.4 Power-aware verification

Hardware description languages were built to capture what a design computes. They have no way to say how each element of it is powered, which is why a separate description — IEEE Std 1801-2024, *IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems* (UPF) — had to be invented once power gating and multiple supply voltages made the power architecture a functional matter rather than a physical one [6]. Power intent must live outside the golden design description because the same RTL is instantiated into different power architectures by different integrators [7], and because a description of the power architecture has to be readable by synthesis, place-and-route, static checkers and simulators alike.

The vocabulary is small and worth pinning precisely. A **power domain** is a collection of instances treated as a group for power-management purposes, typically sharing a primary supply set. **Isolation** provides defined behaviour for a signal whose driving logic is not active, implemented by an **isolation cell** that passes values normally and clamps its output to a specified value when its control asserts. A **level-shifter** translates a signal from one voltage swing to another. **Retention** is enhanced functionality on selected sequential elements so their values survive the power-down of the primary supply, and a **power state table** states the legal combinations of supply states [6]. Everything in this section is a statement about how those five interact in time.

### What a power-aware simulator does that a plain one does not

Loading power intent changes the simulator's value resolution. When a domain powers down, every logic node inside it is corrupted to unknown and stays unknown until the domain powers back up; an isolation cell on an output port stops that unknown reaching powered-on logic by driving the clamp value the power intent specifies; and a retention register is corrupted like everything else, except that the value saved before entry is restored on exit [3].

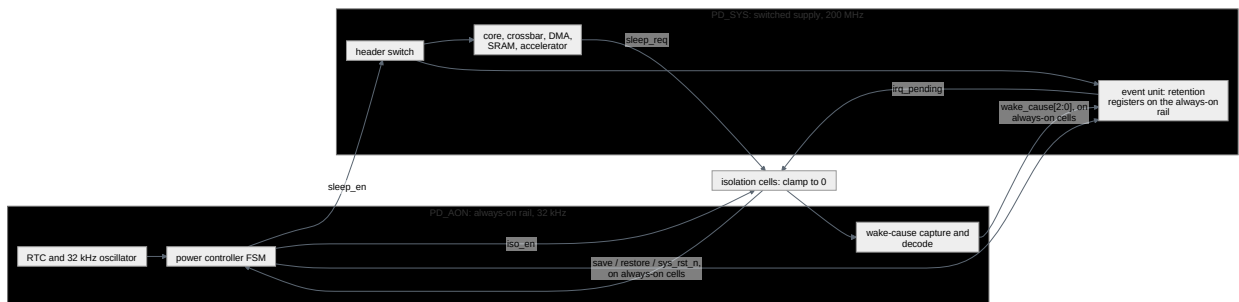
**The corruption is the checking mechanism.** This is the part that surprises people coming from functional simulation: nobody wrote those checks. Injecting unknowns into a powered-down domain makes an unprotected domain crossing *visible* — the unknown appears on the crossing, and the logic downstream of it starts misbehaving shortly after the driving domain goes down [7]. A missing isolation cell is not diagnosed by a rule; it is diagnosed by the failure it now causes.



The standard makes the level of corruption a declared property rather than a fixed behaviour. A **simstate** specifies the operational capability a supply state supports, from `NORMAL` — full switching with characterised timing — down through `CORRUPT_STATE_ON_CHANGE`, `CORRUPT_STATE_ON_ACTIVITY`, `CORRUPT_ON_CHANGE` and `CORRUPT_ON_ACTIVITY` to `CORRUPT`, where the supply cannot even hold existing state. The predefined power state `ON` carries simstate `NORMAL`; `OFF` and `ERROR` both carry `CORRUPT` (IEEE 1801-2024 §4.8) [6]. Read the ladder as a question you are being made to answer about every state you designed: does logic still switch here, does it hold without switching, or can it not even hold?

### The reference SoC: two domains, and everything that can go wrong between them

The reference SoC has two power domains — an always-on island carrying the real-time clock and the wake-up logic, and a switchable system domain holding everything else — and both run at one nominal voltage, so no level-shifters are required. That simplification is deliberate: it lets the two-domain case teach isolation, retention and sequencing without voltage, and leaves multi-voltage to the flagship. The crossings themselves are what remains, and Figure 19.1 has every one of them.



**Figure 19.1** — The reference SoC’s two power domains and every signal that crosses between them: an always-on island carrying the real-time clock, the wake-cause capture and the power-controller state machine, against a switched domain holding the core, the crossbar, the DMA, the SRAM, the accelerator and the event unit. The crossings are not symmetric — signals leaving the switched domain pass through isolation cells because their drivers vanish, while signals entering it need no clamp but do need a route built entirely of always-on cells, and so does the isolation enable itself.

Read the crossings in both directions, because they are not symmetric. Signals leaving the switched domain for the always-on island — the core’s sleep request, the event unit’s pending-interrupt flag — must pass through isolation cells (IEEE 1801-2024 §4.4.4), because their drivers vanish while the island keeps running and would otherwise feed unknowns into logic that is awake. Signals entering the switched domain from the island — the wake-cause code and the controller’s `save`, `restore` and `reset` — need no clamp, because their destination is unpowered and its value irrelevant; what they need instead is a *route* built entirely of always-on cells — and so does the isolation enable, whose cell stays powered precisely so it can clamp. A routing tool that inserts an ordinary buffer on one of those paths has broken it, and that is a real and documented bug class: one industrial design was found with ordinary buffers rather than always-on cells inserted on isolation-enable and wake-up signals, corrupting the wake-up path [7]. The event unit’s retention cells are powered from the always-on rail while their normal-mode storage sits on the switched supply, which is precisely why one survives the power-down and the other does not [6].

**The ordering is the specification.** The sequences below are not conventions; they are what the hardware requires, and every step exists because omitting it destroys something.

**Table 19.1** — The power-down and power-up sequences for the reference SoC’s switched domain, one column each, in the order the hardware requires. Three of the steps carry published failures, which is what to read the columns for: isolation must be asserted before the sleep enable and released after it, reset must already be asserted when the supply returns, and reset must be inactive at the restore edge — which is why step 3 of the power-up column has an order inside it.

Power-down	Power-up
1. Quiesce: no outstanding bus transactions	1. Assert the domain reset, while power is still off
2. Gate the domain clock	2. Close the header switch; wait for supply-good
3. Assert <code>save</code> : retention captures state	3. Release reset, then assert <code>restore</code>
4. Assert isolation: outputs clamp	4. Release isolation
5. Open the header switch: power off	5. Un-gate the clock

Three of those steps carry published failures. Isolation must be asserted before the sleep enable and de-asserted after it — swap either and the always-on island samples an unknown for the duration of the overlap. Reset must already be asserted when the supply returns: a design where reset was asserted *after* the sleep enable de-asserted showed the register output corrupted for exactly the window between the supply arriving and the reset landing. And reset must be inactive at the restore edge, which is why step 3 of the power-up column has an order inside it [7].

All three of those are published rather than deduced, and they come from a checklist Samsung Electronics published out of its own low-power practice — worth reading in full precisely because it refuses to generalise. Its isolation table alone runs to eleven numbered checks, enumerated as named failures: a missing isolation cell, a redundant isolation cell, a normal isolation cell sitting inside a block that switches off, an isolation cell whose output floats, an isolation control signal of the wrong polarity, an

isolation control signal tied off to a constant, and so on through cell type and cell name disagreeing with the declared power intent. The retention checks run the same way, down to a save or restore control taken from the wrong source or tied off to a constant [7]. Nobody derived that list. It is the residue of designs that failed, written down, and it belongs in the register §19.4 has been building.

### Worked example: the escape, and the assertion that would have caught it

The canonical event-unit bug of Chapters 3 and 7 lives on the boundary between steps 3 and 4 of the power-up column. Its full mechanism is this.

The event unit’s retention strategy carries a *restore-period condition*: a predicate that must hold throughout the restore window. Here it is “isolation still asserted”, and it is there for a reason — a half-restored register drives garbage, and while restore is in progress the clamp is the only thing keeping that garbage out of the always-on island. The standard is normative about what happens if the condition lapses: it shall be true throughout the entire restore period (IEEE 1801-2024 §6.53), otherwise the restore operation fails and the retention element is corrupted [6].

The reference SoC’s wake path has a fast-wake optimisation. Rather than waiting for the controller’s sequencer to reach its release state, the always-on wake logic drops the isolation enable in the cycle a wake event arrives, provided supply-good is already asserted — shaving 32 kHz cycles, an eternity, off wake latency. The qualification is what makes it look safe: the wake event that *starts* the power-up arrives while the supply is still off, so it cannot fire the bypass, and on every ordinary wake the sequencer releases isolation itself at step 4. The failure needs a *second* wake event — one that lands after supply-good, inside the restore window opened at step 3. That one is qualified, so the bypass fires and pulls the isolation enable low a cycle before the sequencer would have. The restore-period condition goes false mid-restore. The retention element is corrupted, and the event unit comes up with its interrupt-mask configuration destroyed. A wake event arriving in the same cycle as isolation release — exactly as Chapter 7’s post-mortem recorded it.

Plain RTL simulation cannot represent this: with no power-down, no corruption and no retention, the restore always “works”. Power-aware simulation can represent it, but still needs stimulus that hits the race. An assertion needs neither.

```
// Bound in the always-on domain. Every signal here lives in the always-on
// domain, so this checker keeps running while the switched domain has no
// power -- which is the whole point.
module pd_sys_sequence_chk (
 input wire clk_aon, // 32 kHz always-on clock
 input wire rst_aon_n, // always-on reset, active low
 input wire iso_en, // 1 = switched-domain outputs are clamped
 input wire sleep_en, // 1 = the domain's power switch is open
 input wire restore, // level-sensitive retention restore, active high
 input wire sys_rst_n // switched-domain reset, active low
);
// Power may be removed only from an already-isolated domain.
property p_iso_before_sleep;
 @(posedge clk_aon) disable iff (!rst_aon_n) $rose(sleep_en) |-> iso_en;
endproperty
```

```

a_iso_before_sleep : assert property (p_iso_before_sleep);

// Isolation may be released only once power is back.
property p_iso_after_power;
 @(posedge clk_aon) disable iff (!rst_aon_n) $fell(iso_en) |-> !sleep_en;
endproperty
a_iso_after_power : assert property (p_iso_after_power);

// The clamp must hold for the whole restore window: this is the
// retention strategy's restore-period condition, stated as an assertion.
property p_restore_isolated;
 @(posedge clk_aon) disable iff (!rst_aon_n) restore |-> iso_en;
endproperty
a_restore_isolated : assert property (p_restore_isolated);

// Reset must already be asserted when the supply returns.
property p_reset_before_power;
 @(posedge clk_aon) disable iff (!rst_aon_n) $fell(sleep_en) |-> !sys_rst_n;
endproperty
a_reset_before_power : assert property (p_reset_before_power);

// One cover per antecedent (Chapter 11's rule). Each is a rare, deliberately
// sequenced event, so an empty cover is a finding rather than a formality.
c_sleep_entry : cover property (@(posedge clk_aon) disable iff (!rst_aon_n)
 $rose(sleep_en));
c_iso_release : cover property (@(posedge clk_aon) disable iff (!rst_aon_n)
 $fell(iso_en));
c_restore : cover property (@(posedge clk_aon) disable iff (!rst_aon_n)
 restore);
c_supply_return : cover property (@(posedge clk_aon) disable iff (!rst_aon_n)
 $fell(sleep_en));

endmodule

```

Four assertions, all overlapping implications on the always-on clock, each with a cover on its antecedent, and all checking only always-on signals. That last point is what makes the checker work: it observes the power controller's *outputs*, never the switched domain, so it is alive and evaluating at exactly the moments the domain it protects is dead. `a_restore_isolated` fails on the canonical escape, and it fails on the cycle the fast-wake path fires rather than microseconds later when the event unit's configuration is next read.

Two further consequences follow. Because these assertions are about a small always-on finite state machine with a handful of inputs, they are an excellent formal target (see Chapter 14) — proved once, they hold for every wake-event arrival time, which is precisely the randomisation the escape's post-mortem said was missing. And because the assertions are written against controller outputs rather than against a netlist, they run unchanged at RTL, which answers the scheduling question the escape raised: power sequencing does not need to wait for gate level.

### The negative obligations: what must *not* survive

Retention is a default that has to be argued against, not only for.

**Scenario.** The crypto engine sits inside a switched power domain, and its key ladder holds an unwrapped AES key in ordinary registers. During power-intent cap-

ture, someone applies the domain’s retention strategy broadly — “retain the configuration registers” — and the key registers fall inside the filter. **Approach.** Two power-intent obligations, both negative, both stated as properties rather than as review comments. The key registers must *not* be in the retention strategy, so that powering the domain down destroys the key, which is the entire security value of powering it down. And the engine’s `key_valid` output must be clamped to 0 by its isolation strategy — the clamp value is a functional decision, and clamping it to 1 would tell the always-on island a key is loaded when the domain holding it has no power. **Why.** The first is invisible to every functional test, because a retained key makes the engine work *better*: it comes up ready and encrypts correctly. The check that catches it is a directed power-aware test — power down, power up, attempt an encryption without reloading the key, and require it to fail. The second is the general rule underneath: an isolation clamp value is not a don’t-care. Clamping a busy output to 1 hangs the interconnect; clamping a request line to 1 launches a transaction into a dead domain; clamping an error flag to 0 hides a fault. The clamp value belongs on a plan row with its rationale, and the review question is always “what does the receiving logic do with this value for as long as the domain is off?”

### The block that stays awake

**Scenario.** The sensor hub is the extreme case of the always-on citizen: a 32 kHz island with its own DSP, fusing sensor data continuously while the system it serves sleeps, and waking that system when a fusion threshold trips. Chapter 13 found a bug earlier along this same wake path — a reset-domain crossing *inside* the island, producing a spurious wake interrupt, diagnosed statically on the RTL. This is the next segment of the path failing a different way at a different stage, and the two are worth seeing together because the symptoms are nearly identical. **Approach.** Run power-aware simulation on the *post-layout* netlist, not only on the RTL and the pre-layout netlist. Low-power verification is a three-stage activity — RTL, pre-layout netlist, post-layout netlist — and each stage sees defects the previous one could not <sup>[7]</sup>. Here the post-layout run shows the wake request going unknown whenever the host’s switched domain is off: the system never wakes, and only a watchdog recovers it. **Why.** Place-and-route routed the wake request through a repeater physically placed inside the host’s switched domain, so a signal that starts and ends in always-on logic now passes through a cell with no power. That placement did not exist at RTL, did not exist in the pre-layout netlist, and is invisible to any functional test on either; the signal is logically correct at every stage before the one where it breaks. This is the strongest single argument for running power-aware simulation on a post-layout netlist at least once, and for treating the always-on route as an attribute checked structurally rather than assumed.

### Where inspection stops working

The reference SoC’s two domains give one switchable domain, on or off: two states, one power-down sequence, one power-up sequence, reviewable by three people around a whiteboard in an afternoon.

Now count the flagship. Its power intent declares six domains — always-on, root of trust, host cluster, compute island, NoC and memory, peripherals — with DVFS offering three operating points (0.6, 1.1 and 1.5 GHz) on the host cluster and the compute island, against a NoC held at 800 MHz. Give those two domains three operating points plus an off state, four states each, and the remaining three switchable domains on or off:  $4 \times 4 \times 2 \times 2 \times 2 = \mathbf{128 \text{ combinations}}$ . That is an upper bound on combinations, not a count of legal states — the power state table names the legal subset and most of the 128 are illegal by construction — but the bound is the point. Nobody reviews 128 combinations by inspection, and nobody enumerates the transitions between them at all: transitions, not states, are where the sequencing bugs live, and there are far more of the former.

And the flagship is small. A published industrial case from Samsung Electronics in 2010 — a 40-million-gate SoC with 30 power domains, eight of them software-controlled power-gating domains — carried a power-intent description running to more than 2,000 lines and more than 4,000 legal power states, and named power-state coverage as an explicit challenge with no satisfactory solution in the tooling of the day <sup>[7]</sup>. Two, 128, four thousand: the same discipline, three regimes, and the technique has to change at each step.

What changes is this. At two domains, the power state table is documentation and the sequences are checked by directed tests. At 128, the table becomes the machine-readable specification from which the checks are *generated*: one sequencing checker per domain, instantiated from the table rather than written by hand, and a coverage model whose axes are power state and transition rather than data. At four thousand, coverage of the state space stops being achievable and the argument moves to the controller: prove the power-management finite state machine only emits legal sequences, and verify the domains against those sequences rather than against the product space.

DVFS adds an orthogonal complication that the state count hides. An operating point is a state; a *transition* between operating points is a voltage ramp, and the interesting failures happen during the ramp rather than at either end. A level-shifter whose input voltage leaves the range it was characterised for corrupts its output — documented behaviour, reproduced in simulation with the input driven outside the cell's declared input voltage range <sup>[7]</sup>. Chapter 13 noted the companion effect on the clocking side: the flagship's clock ratios move with the operating point, so a CDC abstraction model is valid for one operating point only.

### Power-aware simulation has its own optimism

One closing turn, and it is the deepest thing in this chapter. Chapter 13 taught that RTL simulation lies optimistically about unknowns. Power-aware simulation lies optimistically one abstraction level up.

Simulation results reflect the implemented hardware only to the extent that the sim-state declared for each power state is correct. Declare a state as `CORRUPT_ON_ACTIVITY` when the silicon is really `CORRUPT_STATE_ON_CHANGE` and the results differ — and while that particular mismatch happens to be conservative, others



are optimistic, producing errors in the implemented design that the simulation showed working <sup>[6]</sup>. The power-aware platform is not an oracle. It is a model of the power architecture, written by the same people who wrote the power architecture, and it inherits their misunderstandings exactly the way a reference model inherits a misreading of the specification (see Chapter 3). A domain whose simstate is too generous is a domain whose corruption tests pass for the wrong reason — which is the failure mode of this entire chapter, one level up.

## 19.5 What to run, and when

---

Everything above is a capability question. This is the budget question, and the budget answers first.

### The arithmetic that ends the argument

Chapter 12 costed the reference SoC's nightly regression at month nine: 180 distinct tests, 900 runs, a mean of 11 minutes, 165 CPU-hours on 24 concurrent simulator licences, comfortably inside a ten-hour night. Now propose running those 180 tests at gate level with back-annotated delays.

Suppose the team measures a 40× slowdown. Eleven minutes becomes 7.3 hours; 180 runs is 1,320 CPU-hours; on the same 24 licences that is 55 hours of wall clock, which does not fit a weekend, let alone a night. The conclusion is insensitive to the ratio: at 10× the same 180 runs cost 330 CPU-hours, 13.8 hours on 24 licences, still outside the night, and the break-even — the slowdown at which the suite would just fit the ten-hour night — is around 7×. **“Run the suite at gate level” is arithmetically dead across the whole plausible range** — and prohibitive in practice for any reasonably sized SoC <sup>[3]</sup>, which is why the argument is worth having once, in numbers, rather than annually in opinions.

The ratio does matter for the other half of the question. At 40×, a gate-level tier of fourteen tests costs 102 CPU-hours, but fourteen runs into twenty-four licences is one wave: the wall clock is a single run's 7.3 hours, and ten slots stay free — which is why it fits the weekend tier without crowding what else runs there. The decision is not *whether* but *which fourteen*, and the ratio is the one figure here you must measure on your own design and tool rather than inherit from a book: it varies by an order of magnitude with netlist size, annotation scope and how much of the environment survives.

### Three axes, not one dial

Gate-level runs are usually discussed as though they were a single thing. They are three independent choices — timing annotation, power intent, netlist stage — and the useful combinations, set out in [Table 19.2](#), answer different questions at very different prices.

**Table 19.2** — The useful combinations of the three independent choices that make up a gate-level run — timing annotation, power intent and netlist stage — each paired with the question it answers and the tier it belongs in. The first row is the one teams get wrong most expensively: RTL plus power intent answers the power-sequencing question without a netlist at all, and belongs in the nightly tier from the day the power intent file exists.

Configuration	The question it answers	Where it belongs
RTL + power intent	Does the power sequence work? Is every domain crossing protected?	Nightly, from the day the power intent exists
Zero-delay netlist	Does the design leave reset with no unknowns? Are the test structures inert in functional mode?	Weekend, from first netlist
Zero-delay netlist + power intent	Does the <i>implemented</i> protection logic match the intent?	Weekend, once protection cells are inserted
SDF-annotated netlist, named corner	Do the timing constraints describe the design that was built?	Campaign, per corner
Post-layout netlist + power intent	Are the always-on routes really always-on?	Campaign, once per netlist drop

The first row is the one teams get wrong most expensively. Power-aware verification is not a gate-level activity that happens to use power intent; it is an RTL activity that *also* runs at gate level, and it should start the day the power intent file exists. Chapter 7’s escape post-mortem reached exactly this conclusion and wrote it into the sign-off checklist. The gate-level stage adds something specific and later: only the netlist contains the actual isolation, retention and always-on cells, so only there can you check that what was inserted matches what was specified — which is why power-aware simulators carry a mode intended for gate-level runs, where the multi-voltage cells are physically present rather than inferred from the intent [7].

### Which fourteen

Five criteria, in priority order. Applied honestly they name more candidates than fourteen — the second alone will name several — which is the point: the criteria rank, and the budget cuts. Take them in order until the hours run out, and record what fell below the line, because a tier whose contents nobody argued about is a tier nobody chose.

1. **Reset and boot.** Always. It is where unknowns live, it is short, and it is the run that must be clean before any other gate-level run means anything.
2. **One test per structure that does not exist at RTL.** Scan-enable inert in functional mode, test-mode multiplexers not selected, debug-fabric entry and exit, clock-gating enables. Each of these has never been simulated before and never will be again.
3. **Every power state transition, at least once.** At gate level this is about the inserted cells rather than about the sequence — the sequence was covered at RTL — so it needs no timing annotation.

4. **The configurations the timing constraints assumed something about.** Section 19.3's multicycle example: the tests worth annotating are the ones that exercise a mode the exceptions were written for, or written *around*.
5. **Whatever the last three escapes would have needed.** A gate-level tier is small enough that it can afford to be shaped by history, and old enough escapes are the only unbiased sample of what this team's flow misses.

What does *not* belong, whatever the budget: the constrained-random regression. Random seeds at gate level buy almost nothing per unit cost, because the value of randomness is in the volume of scenarios and volume is exactly what the slowdown removes. Ranking does not rescue it either — a ranked selection reproduces coverage already achieved and explores nothing new (see Chapter 12), so a ranked gate-level tier is an expensive way to re-derive what the RTL run already gave.

**Scenario.** The USB controller's plan carries rows for its link power states, U0 through U3 (*Universal Serial Bus 3.2 Specification*, Revision 1.1, sections 7.5.6 to 7.5.9) [8], and someone proposes running them power-aware at gate level on the grounds that they are "the power tests". **Approach.** Separate two state machines that share a vocabulary and nothing else. The link power states are a *protocol* obligation — the device negotiates them with a host, the spec mandates the transitions and their timing, and the evidence is a compliance suite at RTL. The controller's *power-domain* state is a power-intent obligation about supplies, isolation and retention. They interact at exactly one point: entering the deepest link state permits the integrator to power-gate the controller's domain, and the controller must then be quiesced, isolated and retained correctly. That interaction is the only row that earns a power-aware run, and it needs no timing annotation. **Why.** Conflating the two produces the worst of both: a slow platform running protocol tests that a fast one covers better, and a power-intent obligation nobody checked because "the U-state tests passed". The general rule is that a test earns a gate-level or power-aware slot by needing something the platform uniquely provides — never by having a plausible-sounding name.

## IN PRACTICE

Gate-level bring-up is owned by one person for a reason: it is a stretch of work in which nothing about the design is learned, and splitting it across the team multiplies the learning curve without shortening the schedule. Give it to someone who will also own the environment changes it forces, because those changes — tie-off harnesses, unknown checks, macro guards on white-box checkers — are permanent additions to the testbench, not workarounds.

The environment that runs at gate level unchanged is the one that was built for it from the start. Sampling and driving through clocking blocks lets a single environment work against both RTL and gate-level models (see Chapter 12), and every hierarchical reference is a place it will not. Teams that discover this at netlist arrival pay for it twice: once in the rewrite, and once in the confidence lost when the rewritten checkers disagree with the originals.

The waiver file for timing-check violations decays fastest of all. A violation waived as “this path is a false path” is a claim that must be traceable to the constraints file, and the two drift apart every time either is edited. Review them together or not at all.

And the schedule reality: the netlist arrives late, the first clean gate-level boot takes longer than anyone plans, and the tier that was supposed to run nightly runs twice before tape-out. Plan for two clean runs, not twenty, and choose the tests accordingly.

---

## PITFALLS

1. **Running the functional regression at gate level.** Symptom: a weekend tier that never completes, and a slowdown blamed on the farm. Cure: the arithmetic above, once, in writing; equivalence checking discharges the synthesis-correctness argument that motivated it (see Chapter 15).
2. **Debugging a functional mismatch before resolving timing-check output.** Symptom: days spent on a scoreboard error whose cause was a `$setuphold` violation two thousand cycles earlier. Cure: resolve timing violations in time order first — the notifier turns each one into an unknown that propagates <sup>[5]</sup>.
3. **Trusting an annotation nobody audited.** Symptom: a back-annotated run that is suspiciously fast and reports no violations at all; or nobody in the room can say whether it was min, typ or max. Cure: review the annotation log and compare its annotated-instance count against the netlist’s, and record the corner — a defaulted argument <sup>[5]</sup> — in the run manifest beside the seed and the tool build.
4. **Treating the power state table as documentation.** Symptom: sequencing checkers written by hand, one domain at a time, drifting from the table. Cure: generate the checkers and the coverage model from the table, so that a table edit breaks the checks rather than silently outdating them.
5. **Deferring power-aware verification until the netlist.** Symptom: a retention or isolation escape found after RTL sign-off, when the fix is an ECO. Cure: run power-aware at RTL from the day the power intent exists; the netlist stage checks the inserted cells, not the sequence.
6. **Retaining by default.** Symptom: a retention strategy whose filter is “the configuration registers”, applied without asking what must *not* survive. Cure: every retained register is a decision with a plan row, and secrets are the class where the default is wrong.

## SUMMARY

- Gate-level simulation's traditional justifications have mostly been taken over by better tools: running the functional testbenches on the netlist verifies the synthesis tool, a job formal equivalence checking and static timing analysis do better [1]. What survives is unknowns at reset, structures that exist only after synthesis, power intent on a netlist, and a check on the constraints themselves.
- Exhibiting a phenomenon and signing off its absence are different jobs. Simulating the metastability class needs a gate-level model with timing [4], and even then what appears is the timing violation on the synchronizer path: metastability itself is not observable in simulation, and only structural and timing analysis of the implementation covers the class [2].
- In practice, most first gate-level failures are testbench failures rather than design failures. Triage by tell: an unknown from an unreset flop, from a pin with no RTL counterpart, from a checker that cannot survive a netlist, or from pessimism with no source. The silent one is a scoreboard comparing with `!=`, whose result is `1'b x` when the relation is ambiguous, so `if (1'bx)` takes the false branch [5] — check for unknowns explicitly, ahead of comparing.
- Static timing analysis is the authority on timing; gate-level simulation is a sample of it. What GLS uniquely adds is a check on the *timing exceptions*, which STA obeys rather than verifies [2].
- Power-aware simulation checks by *corrupting*: a powered-down domain's nodes go unknown, and an unprotected crossing becomes visible as a failure downstream rather than as a rule violation [3].
- The bugs are in the sequencing, not the domains. Isolation before power-down and after power-up, reset before the supply returns, and a restore window whose conditions hold throughout — the standard is normative that a lapsed restore condition corrupts the retained value (IEEE 1801-2024 §6.53) [6].
- Power-aware simulation has its own optimism: results are only as accurate as the declared simstate, and an over-generous one produces tests that pass for the wrong reason [6].

## FURTHER READING

Within the corpus, [6] is normative for everything in Section 19.4: the definitions of domain, isolation, level-shifter and retention, the simstate ladder and its corruption semantics, and the retention strategy's save, restore and power-down conditions with the failure behaviour of each. [7] is its practical counterpart and the most useful short paper here — a power-intent qualification checklist, an eleven-item isolation check list, the sequencing rules between isolation and sleep enables, and an industrial case whose domain and state counts show where inspection stops. [5] is definitive on back-annotation and timing checks: what an SDF file may carry, what happens to values it omits, the two families of check, notifiers, and the min/typ/max and scale-factor arguments that decide which corner a run represents. [2] covers what happens to a design between RTL sign-off and silicon — static timing analysis and its corners,

the physical requirements of synchronizers, and why scan insertion on a synchronizer's second flip-flop erodes its mean time between failures. [3] carries the gate-level side of the unknown problem and the power-aware corruption semantics. [1] is the clearest short statement of what gate-level simulation is for, why SDF exists, and why the checks it is usually asked to perform belong elsewhere. The 2024 data on asynchronous clock domains, and the class of metastability bugs needing a gate-level model with timing, is in [4].

Outside the corpus: M. Keating, D. Flynn, R. Aitken, A. Gibbons and K. Shi, *Low Power Methodology Manual for System-on-Chip Design* (Springer, 2007) is the standard architectural treatment of the techniques this chapter verifies; S. Jadcherla, J. Bergeron, Y. Inoue and D. Flynn, *Verification Methodology Manual for Low Power* (Synopsys, 2009) is its verification-side companion and the origin of much of the sequencing discipline in Section 19.4. IEEE Std 1497 is the Standard Delay Format specification itself, referenced but not reproduced by the SystemVerilog standard. For static timing analysis as a discipline rather than as a neighbour of simulation, J. Bhasker and R. Chadha, *Static Timing Analysis for Nanometer Designs* (Springer, 2009) is the practitioner reference.

---

## REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
2. **D14** M. Litterick, "Full Flow Clock Domain Crossing - From Source to Si," Proc. DVCon U.S., 2016.
3. **D23** S. Zhao, S. Yan, and Y. Feng, "Practical Approach Using a Formal App to Detect X-Optimism-Related RTL Bugs," Proc. DVCon U.S., 2014.
4. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
5. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
6. **S10** IEEE, "IEEE Std 1801-2024, IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems (Revision of IEEE Std 1801-2018)," IEEE, 2024.
7. **D17** J. Liu, M.-S. Hong, B. H. Lee, J. Choi, H. Won, K.-M. Choi, H. Vardhan, and A. Kher, "Low Power Verification with UPF: Principle and Practice," Proc. DVCon U.S., 2010.
8. **S21** USB 3.0 Promoter Group, "Universal Serial Bus 3.2 Specification, Revision 1.1," USB 3.0 Promoter Group, June 2022.



---

## 20. Post-Silicon Validation

Two silicon bring-ups, eleven months apart.

The reference SoC's first parts arrive on a Tuesday. One goes onto a debug board with a JTAG probe, a scope on the 32 kHz crystal and a serial terminal. By Thursday it powers on, releases reset, boots from ROM and blinks a GPIO. By the following Friday the DMA has moved a 2D tile from SRAM to the neural accelerator and the UART is echoing at 115,200 baud. Two engineers, six weeks, one part on a bench. When something misbehaves they halt the core over the debug module, read the DMA's `STATUS` register through the abstract memory access, and — because the whole chip is 512 KB of SRAM, five crossbar managers and a handful of peripherals — they usually guess right on the second try.

Eleven months later the flagship's first parts arrive: the same company, the same DMA architecture scaled to eight channels, the same register discipline, one generation on. Now the bench is a rack, the part boots an operating system on four application cores, and the interesting workloads involve the LPDDR4 subsystem, the mesh NoC and a video decode running against live network traffic. The failure that costs the program three weeks looks like this. Under one particular mixture of decode and traffic, roughly once every forty minutes, the system stops. No exception, no error interrupt, no kernel log. When the lab halts it, the host cluster is idle, two NoC queues are full, the flagship's DMA reports no error on any channel, and every configuration register reads back exactly what firmware wrote. Forty minutes at target speed is of order  $10^{12}$  cycles — 2,400 seconds at any point on the flagship's 0.6–1.5 GHz DVFS band. The on-chip trace buffer holds about four thousand of them.

Nothing in the first bring-up prepared anyone for the second, and what eventually closed the second was not a debug technique. It was a decision taken fourteen months earlier, in a meeting nobody remembers, about which buses would get monitors, how deep the trace buffer would be, and whether those monitors would still be reachable when the compute island was power-gated. That is the uncomfortable centre of this chapter: **the observability you have in the lab is the observability you designed in, months before you knew what would break.**

---

### CHAPTER MAP

- §20.1 establishes why post-silicon is the exact inversion of pre-silicon — a part runs everything and reveals almost nothing — and what follows from that inversion.
- §20.2 treats design for debug as a pre-silicon decision with a hard deadline, and states how to budget it.
- §20.3 orders a bring-up plan, and establishes why the order is the argument.

- §20.4 develops why *localising* a post-silicon failure, not detecting it, is the expensive step, and gives the concrete techniques that shorten it.
- §20.5 states what every escape owes the pre-silicon process that missed it; §20.6 sets out how the fix decision is actually made when a respin is not on the table.

## 20.1 What changes when the design becomes a part

This book has reserved one word for this chapter. **Validation** is checking a design on real hardware rather than on a model: post-silicon on fabricated parts, or in the lab on a prototype (Chapter 18’s territory). It is *not* the software-engineering sense of “did we build the right thing?”, and it is not an umbrella term covering pre-silicon work. Everything before tape-out — simulation, formal, emulation, gate-level, power-aware — is **verification**, on a model; validation begins when the thing under test is a physical object. Guard the distinction, because the literature does not: much of it says “pre-silicon validation” where this book says pre-silicon verification, and then quotes umbrella cost figures that mix the two (Chapter 1 flags that scope trap where it uses those numbers).

Chapter 3 established the three links a bug must traverse to be found — activation, propagation, checking — and showed how controllability and observability bound the first two. Post-silicon inverts the budget on both sides of that chain, and the inversion is total:

**Table 20.1** — What changes when the design becomes a part: cycles available, internal visibility, reachable stimulus, the cost of a fix, and what only a physical part exhibits at all. The first two rows carry the chapter’s argument and are meant to be read together — pre-silicon you can see everything and run almost nothing; post-silicon you can run everything and see almost nothing.

	Pre-silicon (RTL simulation)	Post-silicon (a part)
Cycles available	~nothing. An OS boot that takes seconds on silicon takes years on an RTL simulator [1]	everything. Real workloads, real software stacks, indefinitely
Internal state visible	all of it, any signal, any cycle	a few hundred signals, for a few thousand cycles, chosen in advance
Stimulus reachable	anything the interface can express	whatever the running system and its configuration hooks can produce
What a fix costs	recompile	a new silicon stepping
What only this platform sees	nothing physical	power, temperature, electrical margin, real signal integrity, real peripherals

Read the first two rows together, because their product is the whole subject: **pre-silicon you can see everything and run almost nothing; post-silicon you can run everything and see almost nothing**. Every technique in this chapter is a way of buying back a little of the second row using the surplus in the first.

What the surplus buys is real, and it is not obtainable any other way. Post-silicon tests run at target clock speed, so they reach use cases nothing pre-silicon can: boot-

ing a full operating system in seconds, exercising power management and security features across many IPs at once, running under realistic on-field workloads. Because the vehicle is a physical object rather than a model, it also becomes possible to check non-functional characteristics — power consumption, physical stress, temperature tolerance, electrical noise margin — that no functional model represents at all. And the combinatorics of the *outside* world arrive for the first time: as of 2017, compatibility work on a large SoC routinely spanned more than a dozen operating system flavours, over a hundred peripherals and more than five hundred applications [1]. That space is not a pre-silicon oversight. It was never simulable.

The price is paid in three currencies, and the third is the one that changes how the work is organised.

**Observability.** Silicon exposes what design-for-debug hardware routes to a pin or a buffer, and nothing else — Chapter 3 traced that decay and Chapter 1 priced it. Section 20.2 is about spending the budget.

**Controllability.** In silicon, the control you have is exactly the number of configuration options the architecture defined [1]. You cannot force an internal FIFO to be nearly full; you set the registers that were provided, run the traffic, and hope. Where a testbench has a `force` statement, the lab has a defeature bit — if someone put one in.

**Fix latency, and therefore error sequentiality.** Pre-silicon, debugging is naturally sequential: find a bug, fix it, find the next. That mental model breaks in the lab, because fixing a hardware bug requires a new **stepping** — a fabrication cycle with corrected masks, what Chapter 1 calls a respin — and no program can afford one per bug. So when a bug is discovered — often *before* its root cause is known — someone must find a way to work around it so that validation can continue; and the workaround has to eliminate the bug’s effect without masking the other bugs still waiting to be found [1]. Post-silicon debug therefore runs in parallel, on a shared part, with a growing stack of workarounds underneath it. That is the single most distinctive fact about the activity, and Section 20.6 is where it comes due.

A fourth constraint is not technical: the calendar. Post-silicon runs between first silicon and the decision to start mass production, and that date is fixed years in advance by market forecasts — missing the window can cost the entire investment in the product. The decision is made on post-silicon evidence: the trend and type of bugs found, power and performance results, tolerance to noise and electrical variation, against production quality gates that in 2017 practice required an average defect level below about 50 parts per million [1]. This is why lab schedules are measured in weeks, and why every technique below is judged first on how much calendar it saves.

One consequence deserves stating before any technique, because it reorders everything: **the goal of post-silicon debug is not to root-cause the bug.** The goal is to narrow an observed failure down to an error scenario that can be investigated in a pre-silicon environment [1] — a stimulus short enough to simulate, on a scope small enough to load, with the failure still present. Root-causing then happens

where you can see. Judge every technique in this chapter by how far it moves a failure toward that hand-off, not by how much it explains.

**Scenario.** The flagship’s video codec passes every pre-silicon check — a bit-exact golden reference decodes the same streams the RTL does, and the regression carries a corpus of conformance bitstreams. In the lab, a customer’s live stream produces one corrupted macroblock roughly every two hours of playback. **Approach.** The lab does not try to explain the corruption. It reduces: capture the offending stream to memory, replay it from DRAM instead of from the network, bisect it to a 40-frame segment that still fails, then to a single frame, then hand the frame plus the decoder’s register configuration to the pre-silicon team, who reproduce it in RTL simulation. **Why.** The oracle already exists and is golden — the bit-exact reference model of Chapter 8 runs on the captured stream on a workstation, so the lab can decide *corrupt or not* without any internal visibility at all. What the lab supplies is not a diagnosis but a reproducer, and each bisection step is worth more than any amount of staring at the trace buffer. The escape itself is a coverage story: the failing frame combined a slice-boundary condition with a reference-list update the conformance corpus never paired, which is where Section 20.5 picks it up.

## 20.2 Design for debug: the budget you spend before you know what breaks

**Design for debug (DfD)** is on-die instrumentation whose only purpose is to make silicon observable and controllable during validation. It is the most transferable lesson in this chapter, and the reason is a deadline: DfD is specified, sized, routed and taped out with everything else, months before the first failure exists. Missing observability is typically discovered only in the lab, as a failure nobody can root-cause — and adding it then requires a silicon spin, which is exactly what the lab is trying to avoid <sup>[1]</sup>. There is no incremental path: an FPGA prototype (Chapter 18) at least lets you re-select signals and rebuild the bitstream at the cost of hours, and silicon offers no equivalent at any price.

So DfD is a budget allocated under uncertainty, and it has a shape. Every observability mechanism ultimately writes into a fixed store or out through a fixed port, which gives one governing inequality: **signals × cycles ≤ buffer**. Take the flagship’s 64 KB trace buffer. Sixty-four kilobytes is 524,288 bits, so 128 traced signals buys 4,096 cycles of history; 256 signals buys 2,048; 512 signals buys 1,024. Nothing else in the design behaves this way — you are not choosing *how much* to observe but trading width against depth on a fixed rectangle, decided at tape-out. The industry shape has been stable for a long time: a representative buffer is 128 signals × 2,048 cycles, and a design with millions of signals typically traces a few hundred of them for a few thousand cycles <sup>[1]</sup>. A decade of published post-silicon work uses the same order of magnitude — trace buffers that record roughly a thousand cycles of history, or a longer history at the cost of recording fewer signals <sup>[2]</sup>. Hold that rectangle in mind through Section 20.4, where it meets a run  $10^{12}$  cycles long.

Why a buffer at all, rather than streaming off-chip? Because the debug port is orders of magnitude slower than the logic it watches — a JTAG-class interface runs at megahertz while the design runs far faster — so continuous export is arithmetically impossible and an internal buffer is forced <sup>[1]</sup>. Triggers, qualified capture and everything else downstream exist because that rectangle is small and cannot be widened at runtime.

The instruments themselves come in a small number of categories, and it is worth being precise about what each one actually gives you:

**Table 20.2** — On-die design-for-debug instruments: what each yields, what it costs, and the design stage at which it is fixed. Trace and scan are complementary rather than alternatives — a short film of a few signals against a still photograph of nearly every flop — and no row of the last column falls later than tape-out, which is what makes this a budget allocated before the first failure exists.

Instrument	What it yields	What it costs	Decided by
Scan chains, reused for debug	full internal state at one stopped moment	already present for manufacturing test; needs a debug-usable stop	DfT/DfD architecture
Signal trace into an on-chip buffer	a few hundred signals over a few thousand cycles, continuous	buffer area and power; routing congestion to the buffer	signal selection, at RTL freeze
Processor trace	the instruction flow of a core, compressed	modest per core; bandwidth to transport	core integration
Bus / protocol monitors with triggers	transaction-level history at a chosen interface, on a condition	monitor logic + share of the same buffer	interconnect architecture
Hardware checkers in silicon	a stop close to the bug's origin	area, power, and timing pressure — the reason there are few	design, per block
Irritators and non-functional modes	<i>controllability</i> : forced corner conditions without the stimulus	area and power, plus the risk of a mode nobody ever exercises	design, per block
Debug fabric and access ports	reachability of all the above from outside	routing, and one port per power domain if you want it to survive gating	SoC integration

Two of these deserve emphasis because teams under-use them. **Scan chains** were built to find manufacturing defects, and they are a mature, already-paid-for architecture that also gives post-silicon observability of internal state <sup>[1]</sup> — a full snapshot rather than a window, complementary in exactly the way that matters: trace gives you a short film of a few signals, scan a still photograph of nearly every flop. Between them they answer different questions, and a lab that planned only one of the two will find itself asking the other.

**On-chip checkers** are the silicon descendants of Chapter 11's assertions, and their value is not detection but *proximity*: when a hardware checker fires, the part stops near the origin of the bug, so the trace buffer still holds relevant history and the debug logic contains usable hints. In one large processor program, failures caught by

hardware checkers were markedly easier to debug for exactly this reason, while their use stayed limited by area, power and timing cost <sup>[1]</sup>. That is the whole trade in one sentence: a checker in silicon shortens Chapter 8's *detection distance* by orders of magnitude, and you can afford only a handful.

Controllability instruments are the mirror image. Chapter 3's grey-box prescription — add provisions to force error and exception conditions that would otherwise be prohibitively slow to provoke <sup>[3]</sup> — becomes, in silicon, an **irritator**: logic embedded in the design that triggers microarchitectural events at random, initialised at power-on and firing as the part executes. It is the on-die counterpart of Chapter 7's test-bench irritator, and it earns its area by reaching tough corners without the stimulus that would normally produce them — flushing a pipeline at random, or injecting an invalidation as if it came from another chip <sup>[1]</sup>. Non-functional modes do the same job statically: a cache mode in which almost every access forces a cast-out exists solely to stress the level below it.

### Choosing which signals to trace

Given a fixed rectangle, which signals go in it? The research answer is *state restoration*: traced values imply untraced ones, forwards through a gate and backwards through it, so the quality of a signal set is measured by the **state restoration ratio** — restored plus traced states over traced states. On a small worked circuit, tracing 2 signals over 5 cycles yields 10 traced states and 16 restored ones, a ratio of 2.6 <sup>[1]</sup>. A large literature optimises that number by structural analysis, simulation or learning.

Now the honest part: there is **no known industrial report of the restoration ratio actually being used** to select trace signals <sup>[1]</sup>. It takes no account of what the design *does*, so the same block in two products gets the same answer for two different sets of lab needs; it ignores architectural and physical constraints that rule signals out; and it assumes tracing is the only instrument, when trace works alongside scan, monitors and checkers. Selection still rests mainly on designer experience — which is permission to do the engineering rather than the optimisation. For each block, ask what failure you would most fear and what you would need to see to separate its two likeliest causes, and trace *that*.

### The two conflicts nobody schedules

Observability collides with two other architectures, and both collisions are structural rather than accidental.

**Power management.** Debug data has to travel, and its route runs through logic that power management may switch off. A block can be perfectly instrumented and still invisible because the path carrying its signals to the buffer crosses an IP that is powered down at the moment of interest — and that IP need have no functional relationship to the one under debug <sup>[1]</sup>. The reflexive fix, disabling power management during debug, destroys the ability to debug the power sequences themselves, which are among the hardest things in the part. The architectural fix is the one the flagship took: a debug access port per power domain, so the debug fabric is not one thing that dies with the first gated island.



**Scenario.** A sensor hub’s always-on 32 kHz island fuses accelerometer and pressure samples and wakes the DSP when a fused threshold trips. In the lab, roughly once a day the wake does not happen; the system stays asleep until the next unrelated event. **Approach.** The team wants a trace of the island’s threshold logic across the wake decision — but the DfD route from the always-on island to the trace buffer passes through the DSP island, which is power-gated precisely while the bug is armed. Two things make the debug possible: a debug access port living inside the always-on domain, and a small always-on capture of the four signals that matter (threshold comparator, event FIFO not-empty, wake request, isolation-release acknowledge) sampled at 32 kHz. **Why.** The lesson is not “add more trace”; it is that observability of a domain must be *powered by* that domain. A trace path that depends on a neighbour’s power state is observability you own only when you do not need it — and this book’s canonical retention bug, where a wake event races the release of isolation, lives exactly in that blind spot (Chapter 19).

**Security.** DfD is, by construction, a back door into the part’s internal state, and much of it stays present on-field so deployed systems can be diagnosed. That makes it an obvious path to the assets it sits beside — keys, firmware, debug modes — and published system compromises have used post-silicon observability features to get in [1]. The industry’s answer, and the flagship’s, is authenticated debug unlock through the root of trust with the trace fabric’s reach reduced once the part leaves the lab; the knee-jerk alternative of cutting DfD buys security by making validation long, hard or impossible. Chapter 23 treats the unlock path as an attack surface. Here it is enough that the crypto engine’s key ladder and the trace fabric are neighbours, and the neighbourhood needs a threat model before tape-out.

Both conflicts are resolved in a specification review a year before the lab exists, by people who will not be in it. **Post-silicon readiness** — DfD architecture, debug software, debug boards, post-silicon test plans — is a pre-silicon workstream that starts at product-functionality planning and runs the whole length of the design cycle [1]. Treat it like the verification plan of Chapter 5: it has owners, rows and review gates, or it does not exist.

## 20.3 Bring-up: the first hours and days

Chapter 4 used **bring-up** for the pre-silicon phase in which environment and design first exchange checked transactions. Silicon bring-up is the same idea one platform later, and two disciplines carry over: drive toward demonstrations rather than toward infrastructure, and self-check from the very first transaction. One difference changes everything downstream. In the testbench you can add a probe when you need one. Here you cannot, so the *order* of the checks stops being a convenience and becomes the argument.

The first activity is **power-on debug**, and it is the one moment when none of Section 20.2’s machinery is available: if the part does not power on, the on-chip instrumentation is not running either, and visibility into the internals is extremely limited or

simply zero. What remains is a custom debug board offering fine-grained external control, plus brainstorming. The standard method is **defeaturing** — strip the configuration back to a bare-bones system, no power management, no security, no complex firmware boot, that can power on reliably — then **refeature** incrementally. A stable power-on recipe typically takes days to a week from the moment silicon reaches the lab; getting the complex features back on can take several weeks, and it gets harder the more configurable the design is <sup>[1]</sup>. Pause on that last clause, because configurability is normally a virtue: every mode you add to help the lab also enlarges the space the lab must search to find a configuration that boots.

Once the part powers on, the ordering principle takes over, and it has one rule: **each check must be decidable using only what the checks before it have established**. A bring-up step whose verdict depends on machinery not yet proven is not a check — it is a coin flip with two failure modes and no way to tell them apart. That is the entire reason the canonical order is what it is. Power precedes clocks because a PLL cannot lock on a rail that is not up. Clocks precede reset release because a reset synchroniser needs edges. Reset precedes the debug port because the port is logic like everything else. And the debug port precedes every check whose oracle is a register read — which, after that point, is nearly all of them, since reading back a known reset value is the cheapest oracle silicon offers.

A bring-up plan is therefore a verification artifact and deserves Chapter 5’s treatment: each row states one check, the evidence that closes it, and — the field a pre-silicon vplan does not need — the oracle actually available at that moment. One row, one metric, for the same reason as always: a step closed by “it seems to be running” is closed by fatigue.

**Worked example — bring-up plan excerpt, the flagship’s first week.** Ten rows of a plan that runs to about ninety. The third column is the one that makes it a *bring-up* plan.

#	Check	Oracle available here	A pass rules out	Exit criterion
B01	Rails come up in sequence; quiescent current in band	bench instruments only	shorts, missing rail, wrong sequence	all rails in spec, current in band, 5 consecutive power cycles
B02	Always-on PLL locks; divided clock present on a pin	scope, external reference	oscillator, PLL configuration	measured frequency within tolerance
B03	Reset released; JTAG identification register reads back	a constant known from the spec	chain wiring, TAP state machine	value matches, 20 consecutive reads
B04	Debug fabric reaches the always-on domain’s registers	reset values from the register spec	debug decode, address map at this port	all reset values match
B05	One host core executes a three-instruction loop from boot ROM, writing a heartbeat register	the heartbeat toggles	core reset, ROM read path, first bus hop	heartbeat observed for 60 s without stall

#	Check	Oracle available here	A pass rules out	Exit criterion
B06	L2: write-read-verify a walking pattern across all four slices	data comparison	slice enable, L2 address decode	pattern verified in every slice
B07	LPDDR4 PHY training converges; write-read-verify over the channel	data comparison + training status	PHY configuration, board routing, gross SI faults	training converges on 5 cold boots; margin recorded
B08	First timer interrupt taken and acknowledged on one core	a handler-incremented counter	interrupt wiring, exception entry, vectoring	counter advances at the expected rate
B09	DMA channel 0: 1D copy, L2 to L2, one NoC hop	data comparison + completion interrupt	descriptor decode, one NoC route, one coherent port	copy verified; completion interrupt observed
B10	OS boots to a shell on one core, then on four	console output	MMU, caches, coherence, driver stack	shell reached on 20 consecutive boots

**Why this order.** B05 is the first row whose oracle lives *inside* the part, and it is deliberately the smallest one available: three instructions, one of them a store, because a heartbeat that stops tells you the host core stopped, whereas a failing OS boot tells you almost nothing. B08 follows B06 rather than preceding it, because an interrupt handler needs a stack and a stack needs memory B06 has already proven. B09 depends on B06 and B08 at once, which is why it is late — its failure would otherwise be unattributable. And B10 is one row, not ten, precisely because everything under it has been proven: when the shell fails to appear after B01–B09 pass, the search space is the software stack, and that is a very different week.

The contrast with the previous generation is the point of running both. On the reference SoC, this plan is a page, the whole order can be improvised on a bench by one engineer, and a wrong guess costs an afternoon: the part has one core, one memory, three clock domains and two power domains, and a person can hold all of it in their head. On the flagship, the same sequence has to be executed across five asynchronous clock domains and six power domains, with one debug access port per power domain and a bring-up team that is not the design team. The plan stops being a checklist and becomes a contract — because the person running B07 at 2 a.m. is not the person who knows why B06 comes first.

**Scenario.** The flagship’s LPDDR4 PHY training completes on eleven of the first twelve parts and, on the twelfth, converges to a setting that fails a memory test an hour later. **Approach.** The lab does not debug the training algorithm. It runs training a hundred times per part across the temperature range, logs the margin the training reports at each step, and finds that the failing part converges at the edge of the acceptance window while the others sit centred. The fix is a firmware change to the acceptance criterion plus a retrain-on-error path; the RTL is untouched. **Why.** This failure is *only* observable post-silicon, and not because of complexity: pre-silicon there is no signal integrity to train against. Every model of the PHY, from RTL to a real-number model of the analog path (Chapter 21), trains

against an idealisation, and the distribution across parts and temperature does not exist until there are parts. Note also what made the debug tractable — the training state machine reports its per-step margin through a status register. That register is design for debug, decided at tape-out; without it the only observables would have been “training passed” and, an hour later, “the memory test failed”.

Two supporting workstreams decide whether the first week goes well, and both are pre-silicon deliverables. **Post-silicon test plans** are typically more elaborate than pre-silicon ones, because they target system-level use cases that could not be exercised before silicon existed; their development starts alongside design planning, before a detailed microarchitecture exists, so the first version is written against architectural specifications and refined as features land <sup>[1]</sup>. That is Chapter 5’s extraction discipline applied a year earlier and one level up.

**Debug software** is the systematically under-planned half: the instrumented system software that replaces a general-purpose OS with something small enough to reason about, the tools that configure triggers and query registers, the transport that gets data off-chip, and the analysis tools that turn raw signal streams back into transactions. All of it is written against a design whose features are still changing, with a hazard worth naming out loud — in post-silicon work it is not uncommon to root-cause an observed failure to the debug software rather than to the part <sup>[1]</sup>. Chapter 4’s rule that the environment is a design too, whose bugs are tracked like any other, applies here with more force, not less.

Does the preparation pay? On a large processor bring-up reported in 2017, roughly half of all post-silicon bugs were found in the first three months — and much of the first two of those months went on hardware stability and screening parts, not on finding logic bugs <sup>[1]</sup>. A bring-up that reaches its bug-finding phase quickly is one that was planned; a bring-up still fighting power-on in month three is spending the calendar it needed for Section 20.4.

## 20.4 Triage and localisation: the expensive step

Detection, in the lab, is often free. A hang is a hang; a hardware checker fires; a customer’s stream shows a corrupt macroblock; a **multipass consistency check** — the same test run several times, with architected registers and part of memory compared at the end of each pass against a saved reference pass — reports a mismatch millions, and sometimes billions, of cycles after the error that caused it <sup>[1]</sup>. What costs the calendar is **localisation** — identifying the sequence of inputs that activates the bug and the block it lives in. The effort of localising bugs from observed system failures dominates the overall cost of post-silicon validation and debug, and a single difficult logic bug has been reported to take days, weeks or months of manual work <sup>[2]</sup>. It is worth being precise about *why*, because the reason is arithmetic rather than attitude.

The quantity that governs everything is **error detection latency**: the time between the moment a test activates a bug and creates an error, and the moment that error

becomes an observable failure — a crash, a timeout, a deadlock, a wrong result. For difficult bugs this latency runs to millions or even billions of clock cycles [2]. Now put that next to Section 20.2’s rectangle. The opening scenario’s run is of order  $10^{12}$  cycles; the flagship’s trace buffer, at 128 signals wide, holds 4,096. Divide: the trace covers roughly one part in 250 million of the execution. And the placement is not free either — even a perfect trigger that captures the last 4,096 cycles before the hang captures the *symptom*, while the activation sat millions of cycles earlier, outside the window by construction. This is the shape of the problem in one line:

A checker tells you the answer is wrong. The trace buffer shows you four thousand cycles of a forty-minute film, and the interesting scene is not in them.

Which brings back the thesis of Section 20.1 in operational form. The job is not to explain the bug in the lab. The job is to produce a **pre-silicon reproducer**: a stimulus short enough to simulate, on a design scope small enough to load, that still fails. Every technique below is measured against that hand-off.

### The pipeline, and what each stage owes

Industrial practice separates the path from failure to fix into named stages, and the names are worth adopting because they force a distinction teams otherwise blur [1]:

**Table 20.3** — The industrial path from a failed test run to a resolved bug, one row per named stage, with the artifact each stage has to hand on. Note the third row: *sighting disposition* means assigning a debug team and a plan, not the recorded fix / waive-as-errata / defer decision, which arrives much later and is §20.6’s subject.

Stage	What happens	Exit
Test execution	run it; on failure, the obvious sanity checks — part seated, supplies connected, switches as the test expects	resolved, or a <b>pre-sighting</b>
Pre-sighting analysis	make it repeatable: rerun under varied software, hardware, system and environmental conditions until a stable recipe exists	a <b>sighting</b>
Sighting disposition	assign a debug team; agree a plan to track, work around and address it	an owned plan
Bug resolution	work around it so validation continues; group with related failures; localise; root-cause; prove the fix	a <b>bug</b> , resolved

Two vocabulary notes. *Sighting disposition* means assigning a team and a plan; it is not Chapter 7's **disposition**, the recorded fix / waive-as-errata / defer decision, which arrives much later and is Section 20.6's subject. And Chapter 12's **triage** discipline — deduplicate by signature, classify, assign, never debug the same fact twice — applies unchanged in intent and much harder in practice. Two failures with one root cause can present as a crash in one test and a hang in another, and grouping them *before* either is analysed is exactly what an aggressive schedule cannot afford to get wrong <sup>[1]</sup>. The first classification question is one no pre-silicon triage rota ever asks: is this a silicon issue at all, or a logic error? Repeating the test on a different part, or a different core of the same part, is the cheapest discriminator there is — a failure that follows the part is a manufacturing or marginality problem, one that follows the design is yours.

### Reproduction, and the problem of non-determinism

Most localisation methods assume you can make the failure happen again, and post-silicon that assumption is under attack from two directions.

The first is ordinary system non-determinism: interrupts, I/O activity, interactions between cores, operating-system context switches <sup>[2]</sup>. Returning a complex part to an identical error-free state and re-executing the same stimulus is not something a running system naturally supports. The second is physical and has no pre-silicon analogue. A logical error can be masked by a glitch or a fluctuating supply, and reproducing it may require a particular thermal range; finding a recipe — a tuned combination of physical, functional and non-functional parameters — that brings the bug back is itself a challenge <sup>[1]</sup>. The sharpest instance comes from clock-domain crossing: a whole class of metastability bugs cannot be demonstrated on an RTL model by simulation at all, and is generally difficult to reproduce and find in the lab either (2024) <sup>[4]</sup>. That is why Chapter 13 puts structural clock-domain analysis before tape-out rather than trusting the bench.

Against this, post-silicon adopts a **weakened standard of reproducibility**, and it is a good trade: because silicon is fast, an error that appears reliably once in a few executions still counts as reproducible. A one-in-five failure rate is intolerable in a nightly regression, where the same failure on an identical manifest is Chapter 12's flake; in the lab it is a working recipe, because five runs cost minutes. Better still, where the platform provides one, is a **cycle-reproducible mode** — a special hardware mode in which re-running the same test produces exactly the same execution <sup>[1]</sup>. That single property is what makes the next technique legal.

**Scenario.** A radar front-end runs an FMCW chirp chain whose CFAR detection threshold is calibrated at power-on against a reference target. In the field, roughly one unit in two hundred reports spurious detections after about twenty minutes of operation. Nothing reproduces on the bench. **Approach.** The lab stops trying to reproduce the *failure* and starts characterising the *recipe*: parts run in a thermal chamber across the qualification range while the calibration coefficients are logged every chirp frame. The spurious detections appear only above roughly 85 °C, and only on parts whose initial calibration landed in the lower third of the coef-



ficient distribution — two conditions no bench at room temperature produces together. With the recipe in hand the failure reproduces on demand, and the fixed-point saturation bug in the recalibration path localises in a day. **Why.** Two lessons, and the second is the one teams miss. This is a calibration path only real operation exercises: pre-silicon it was verified against a model of the analog chain, and models do not drift with temperature. And “cannot reproduce” almost never means the bug is non-deterministic — it usually means the recipe has a parameter nobody has varied yet. Temperature, supply voltage, part-to-part variation and elapsed time are parameters. Vary them deliberately, log them always, and the search becomes an experiment instead of a vigil.

### Move 1: binary search in time

Given cycle reproducibility and a programmable trace, you can bisect. Run the test, terminate the trace at cycle N, look; if the state is still good, run again terminating later; if it is already bad, run again terminating earlier. Each run halves the interval containing the transition from good to bad. Published practice uses the same mechanism — re-running the same test with different trace-array configurations, terminating at different cycle counts, and aggregating the results of several runs into one effective trace far longer than the buffer <sup>[1]</sup>. This is what *backspacing* means: not a hardware time machine, but many forward runs stitched into a backwards view.

Cost it honestly, because the number is the argument for everything in Section 20.2. To narrow a  $10^{12}$ -cycle run down to one 4,096-cycle window takes  $\log_2(10^{12} / 4096) \approx 28$  halvings. At forty minutes a run that is about nineteen hours of pure execution, before any time spent reconfiguring the trace between runs. If the failure only reproduces one run in three, multiply by three: over two days. And the search is two-dimensional, not one: bisection tells you *when*, but the window is only useful if it contains the right signals, so a change of hypothesis part-way through means re-selecting the trace set and repeating steps you have already paid for. A trace buffer twice as deep does not halve this — it removes one halving out of twenty-eight. What actually collapses the cost is a *trigger*: a condition, programmed into a bus monitor, that starts or stops capture on a hypothesised cause rather than on the failure itself, converting a blind search into a single run when the hypothesis holds.

### Move 2: binary search in configuration

The other axis is the design’s own configurability, and it pays best early. If the failure needs four active cores, does it need three? If it needs the compute island powered, does it survive with power management off? Changing the hardware setup — disabling cores, varying the number of active hardware threads — is standard practice for establishing what the failure actually requires <sup>[1]</sup>. Each answer removes a subsystem from the search space at a cost of one run.

**Scenario.** The GPU’s four shader clusters share a unified memory path. In the lab, a compute workload produces wrong results roughly once per thousand dispatches — never with a single cluster enabled, sometimes with two, reliably with four. **Ap-**

**proach.** Configuration bisection first: with clusters {0,1} the failure appears; with {0,2}, {1,3} and {2,3} it does not; with {0,1} and the DMA idle it disappears. Five runs have reduced “somewhere in the GPU and its memory path” to “an interaction between two specific clusters and concurrent DMA traffic” — and pointed at the memory path rather than the shader cores, because the pair that fails together is the pair sharing a port into unified memory. Only then does the trace budget get spent, on that port, with a trigger on the arbiter’s grant sequence. **Why.** Configuration bisection is cheap, coarse and criminally under-used. It costs one run per question and it answers the question that decides where the expensive instruments point. Doing it in the other order — trace first, hypothesis later — is how a week disappears.

### Move 3: shorten the latency instead of searching it

The most leveraged idea in post-silicon test generation attacks the latency itself rather than the search. **Quick error detection (QED)** transforms an existing test into one with a much shorter interval between activation and observable failure — the canonical transformation inserts a read immediately after each write, so a write that stores a wrong value is caught at once instead of thousands of cycles later when something happens to read it [1]. The transformations are software-only: duplicate the instruction stream into a reserved half of the register and memory space and compare periodically, or proactively load and check shared locations. Nothing is added to the silicon [2].

Layering formal analysis on top turns detection into localisation. Bounded model checking over the space of QED-compatible tests searches for the *shortest* instruction sequence that activates and detects the bug, yielding a minimal reproducer and a short list of candidate blocks at once. On a 500-million-transistor open-source multicore SoC — the OpenSPARC T2 — in 2015, this localised all 92 injected bug scenarios in under seven hours, with traces up to six orders of magnitude shorter than the original end-result-checking tests; the motivating deadlock, whose activation sat millions of cycles before the failure, reduced to a three-instruction trace found in two and a half hours [2]. An industrial application narrowed post-silicon localisation automatically to about twenty flip-flops out of a million, and nine instructions out of billions [5]. The same flow has a named industrial deployment: on an ASIL-rated Infineon automotive microcontroller core — 60 instructions, 1.8K flip-flops, 70K gates, tracked across 16 versions — Symbolic QED found every bug Infineon’s own flow had found, plus 7% more that were specification errors, at 2 person-days of effort against 6 person-months for a reused design [5]. Hold on to the gate count, because it is the limit of what that comparison establishes.

None of this is free — it needs a programmable core in the system, a way to run transformed tests, and a formal engine that can hold a reduced model. The claim is about *where* the leverage sits: reducing the distance between cause and symptom beats searching that distance, every time you can arrange it.

**Move 4: move the failure to a platform that can see**

The endgame of every localisation is a hand-off, and two destinations are usual.

**Emulation and acceleration** (Chapter 17) offer far more visibility than silicon at far less speed, and the transfer is not automatic: because silicon runs so much faster, a test that fails there cannot simply be replayed on an accelerator — it must first be tuned so that it fails *early*, within a run the slower platform can complete <sup>[1]</sup>. That tuning is itself a localisation exercise, and it is usually where the pre-silicon reproducer is born.

**Formal property verification** is the other destination, and it is underrated for this job because of a plausible objection: post-silicon failures are isolated to large sub-system boundaries, and formal does not scale there. The published counter-method inverts the framing — do not take the sub-system as the proof scope. Pick the smallest block boundary at which you can still observe the property you want to check; over-constrain everything that did not participate in the failure, including configuration registers pinned to the values dumped from the failing part; leave the signals of interest under-constrained so the tool can still find manifestations you did not predict; and instead of abstracting a long boot sequence, load the design’s state from a simulation waveform at a timestamp after initialisation and start the proof there.

The reported results are the argument. A configuration-register corruption was reproduced in a 25,000-gate scope in three person-days; temporary over-constraints produced five further manifestations of the same bug; five person-days proved the fix; and, because the environment outlived the emergency, eight more functional bugs turned up when it was reused on later projects. A second case chose a 1.5-million-gate scope over a 4-million-gate one purely so that basic covers would hit, and reproduced its data-corruption bug in about four weeks and three hundred person-hours <sup>[6]</sup>. Both cases are Intel’s own report, and the paper identifies its two designs only by function — a bridge block and a scheduler — inside SoCs it does not name <sup>[6]</sup>.

That last clause is the strategic point. A formal environment built to reproduce one post-silicon failure is the cheapest verification asset a program ever acquires: the panic pays for the setup and the next project inherits it. Chapter 16’s division of labour between simulation and formal has a post-silicon corollary — **the plan row a silicon escape indicts is, by revealed preference, the row that deserves formal sign-off in the next generation.**

## 20.5 Escapes and the feedback loop

---

Every bug found post-silicon is, by this book’s definition, an **escape**: it crossed a sign-off boundary undetected (Chapter 1). That makes the lab’s bug list an unfiltered report card on the pre-silicon process, written by reality and arriving too late to argue with. Turning it into changes upstream is the single most valuable activity in this chapter, and the one most reliably skipped — it becomes possible exactly when the team is most exhausted and most needed elsewhere.

Chapter 7 supplies the machinery: escape analysis, blameless, five written questions — mechanism, why stimulus never created it, why no checker caught it — or none could have seen it, why the metrics said we were done, what changes now. None of that changes post-silicon. What changes is which question bites. Pre-silicon escapes are usually stimulus escapes; post-silicon escapes concentrate in the fourth question, because the dashboards were almost always green. The plan was discharged, the coverage model was closed, the regression was clean, and the bug shipped anyway. An escape analysis that cannot explain a green dashboard has not finished.

Chapter 3’s chain gives the classification that makes the answer actionable. Every escape breaks one of three links first, and each break has a different upstream fix:

**Table 20.4** — Classifying a silicon escape backwards, by which of the three links — activation, propagation, checking — broke first, and what each break owes the process that missed it. The classification earns its keep because the three owe different things: a plan row and a coverage cross, a platform change, or a checker plus a test proving the checker fires.

Link that broke	What it means	The upstream change it owes
<b>Activation</b>	the stimulus never created the condition	a new plan row and a coverage cross that would have demanded it (Chapters 5, 6)
<b>Propagation</b>	the condition happened but could not reach any observable — usually because the platform did not model the mechanism	a platform change: a level, an engine, or an earlier scheduling of one (Chapters 13, 19)
<b>Checking</b>	it happened and was visible, and nothing looked	a new checker, plus a test that proves the checker fires (Chapter 5’s companion-row rule)

There is a fourth break that this book has been careful about and that post-silicon exposes brutally: a wrong **assumption**. A formal `assume` that does not hold in the real system deletes traces silently (Chapter 11), and no coverage number anywhere reports the deletion. When an escape’s mechanism was inside the deleted set, the change owed upstream is not a checker or a bin — it is a written assumption, promoted into a checked obligation on whoever is supposed to guarantee it.

### Worked example — the escape record for the video codec corruption of Section 20.1. Five questions, five written answers, and what each one bought.

1. **Mechanism.** A slice boundary falling inside a macroblock row, combined with a reference-list update on the same frame, causes the line-buffer read pointer to advance once too far; the affected macroblock reads a neighbour’s reconstructed samples.
2. **Why did stimulus never create it?** The conformance corpus contains streams with unusual slice geometry and streams with frequent reference-list updates. It contains none with both, because the corpus is organised by feature and each stream was authored to exercise one.
3. **Why did no checker catch it?** The checker was fine. The bit-exact reference model would have flagged this instantly — on a stream that contained it.

4. **Why did the metrics say we were done?** The coverage model had a coverpoint for slice geometry and a coverpoint for reference-list update, both at 100%. It had no cross. This is Chapter 6's oldest lesson arriving through the most expensive possible channel: two closed coverpoints are not a closed cross, and a coverage model built one feature at a time inherits the corpus's blind spot instead of correcting it.
5. **What changes.** (a) A cross of slice geometry  $\times$  reference-list state added to the codec's coverage model, with the excluded cells argued rather than assumed. (b) Two synthesised streams added to the regression, generated to hit the new cells. (c) A plan-row family for *stream-syntax combinations* rather than stream-syntax features, so the next codec's plan cannot be written one axis at a time. (d) A standing rule for any block with a bit-exact golden model: when the oracle is free, the constraint is stimulus diversity, so budget for a stream generator instead of a stream library.

**Why it is worth the week it costs.** Only (a) and (b) fix this bug. (c) and (d) are the ones that pay, because they change what the *next* plan looks like — and a post-mortem that produces only (a) and (b) has repaired a hole without repairing the hole-making process.

Two organisational habits distinguish teams that actually close this loop.

**They set their own escape bar, above the industry's.** One large processor program counted any bug found by an operating-system-based test or by real application software as an escape from its *validation strategy*, on the grounds that the bare-metal exercisers were supposed to have found it first <sup>[1]</sup>. That is a strictly internal standard, deliberately harsher than “did it reach a customer”, and it works because it makes the exercisers accountable for their own coverage instead of letting the OS act as an unplanned safety net. The program was IBM's POWER8 — 12 cores on a chip of over 5 billion transistors — and bare-metal exercisers were its primary post-silicon test vehicle, which is what makes that bar coherent rather than merely harsh <sup>[1]</sup>.

**They measure post-silicon coverage one platform down.** You cannot instrument silicon with covergroups — the area, timing and power costs of synthesising coverage monitors into the part are prohibitive. The published answer is to synthesise those monitors into the *accelerator* model instead, run the same exercisers there, and use the resulting coverage to decide which exerciser shifts deserve scarce silicon time <sup>[1]</sup>. This is the practical form of a **unified plan**: one plan source, with each row assigned to the platform that will discharge it, and a common test-template language so a test written for one platform can be narrowed and re-run on another. That last property is what makes the hand-off of Section 20.4 possible at all — when a bug appears on silicon, narrowing the same template until it hits in simulation is what turns a lab failure into a debuggable one.

Read the escape *count* correctly, too. On the best-documented public program, about 1 percent of all bugs were found in post-silicon validation (2017) <sup>[1]</sup>. One percent sounds like a rounding error and is not: those bugs consumed weeks each in a phase measured in weeks, and any one of them could have forced a stepping. The reading

is two-sided. The pre-silicon process caught ninety-nine out of a hundred, which is the argument for everything in Chapters 5 through 19; the hundredth cost more than the ninety-nine, which is the argument for this section. That program is IBM's POWER8, and the 1% is a statement about one chip's own bug population rather than an industry rate <sup>[1]</sup>.

The feedback loop also runs *forward*, across generations, and this is where the family narrative earns its keep. The reference SoC's canonical DMA escape — a 2D transfer with negative stride mis-legalised at the 4 KB boundary, corrupting data only when the second channel was active — produced a plan row, a coverage cross and a regression test. The flagship inherits all three along with the DMA architecture. It also inherits a descendant of the bug: the same mis-legalisation, now under coherence, where the split's second half writes a line the host cluster holds Modified, so the corruption stays invisible until the writeback. Inheriting the assets is not the same as inheriting the coverage. The old cross does not mention coherence state; the old test does not run with a cache holding the line dirty. **“We already found that one” is a claim the next generation must earn**, and the way it is earned is by re-deriving the coverage model against the new mechanism rather than re-running the old suite and reading it as green.

## 20.6 Silicon debug meets the fix

Section 20.1 called error sequentiality the most distinctive fact about post-silicon work. Here is what it does to the order of operations. Pre-silicon, the sequence is *find* → *root-cause* → *fix* → *verify*. Post-silicon it is:

**work around** → **localise** → **root-cause** → **propose a fix** → **prove the fix pre-silicon** → **decide what to do with it**.

The workaround comes first, before anyone knows what the bug is, because no program can afford a silicon stepping per bug and the lab cannot stop while one is investigated. And the workaround carries the constraint Section 20.1 named: it must eliminate the bug's effect *without masking the other bugs still waiting to be found*. A workaround that disables a whole subsystem is not a workaround; it is a decision to stop validating that subsystem, taken by accident.

What makes a workaround available at all is a pre-silicon property, and it is the twin of everything in Section 20.2. Mitigation post-tape-out is overwhelmingly a matter of patching or reconfiguring behaviour through software or firmware — and the ability to patch depends directly on how much configurability and controllability were built into the design beforehand <sup>[1]</sup>. Design for debug buys you *sight*; design for configurability buys you *a way out*. The same architecture review decides both, at the same time, for the same reason: neither can be added later.

Then the last step, which is the one the rest of the organisation is waiting for.



## The disposition, under a constraint that has changed

Chapter 7 gave a project three ways to dispose of an open bug: fix it before tape-out, waive it as known errata with a named software workaround and a documentation owner, or defer it with an argument. After tape-out the first option is gone in that form, and what replaces it is more expensive. The triad becomes:

**Table 20.5** — The three forward-looking options for an open bug once the masks exist — a new stepping, a shipped firmware workaround, a documented erratum — with the conditions that favour each, its cost, and the obligation it carries. The second and third rows are a spectrum rather than two alternatives: what separates them is who is obliged to apply the workaround.

Option	When it wins	What it costs	What it obliges
<b>New stepping</b> (revised masks)	no workaround exists, or the workaround's cost is unacceptable; safety- or security-relevant behaviour; the bug blocks a committed use case	refabrication, requalification, and the calendar — usually the dominant term	the fix proven pre-silicon <i>before</i> the mask change, plus regression of everything the change touches
<b>Firmware or software workaround,</b> shipped	a configuration or code path can avoid the condition reliably	performance or power, permanently; a constraint every future software release must honour	proof that the workaround covers <i>every</i> path to the condition, and an owner for that proof
<b>Errata</b> — documented, not worked around in the product	the condition lies outside the product's committed use cases, or the customer can reasonably avoid it	support burden and a permanent public record	a named workaround in the document, a documentation owner, and Chapter 7's recorded decision with a real expiry

Notice that the second and third rows are not alternatives so much as a spectrum: an errata entry usually *contains* a workaround, and the difference is who is obliged to apply it. Notice also what has not changed. Chapter 7's five waiver fields — the hole, the reason, the risk argument, an owner, an expiry — apply verbatim to an erratum. An erratum without a named workaround and an owner is a waiver with the fields left blank, and it will be rediscovered by a customer.

The inputs to the decision are more concrete than the framing suggests. On one industrial program, bug severity was set by three things together — the bug's impact on functional behaviour, the *performance cost of working around it* where a workaround existed, and the complexity of the fix — and severity could only be determined once the root cause was found and a fix proposed; the same program's lab engineers nonetheless guessed severity correctly and early from limited data, and scheduled accordingly, so that high-severity bugs were root-caused in about 6 days on average against roughly 10 for the medium ones (2017) <sup>[1]</sup>. Both halves matter. Severity is not a property of the symptom — a hang looks catastrophic and may be one register bit away from a free firmware workaround, while a one-bit corruption looks minor and may have no workaround at all — and yet an experienced lab triages by expected consequence long before it can prove consequence. That guess is one of the highest-leverage judgements in the phase.

**Proving the fix is not optional, and it belongs pre-silicon.** A fix must be shown to correct the original error and to introduce no new one <sup>[1]</sup>, and the environment built in Section 20.4 to reproduce the bug is exactly the instrument for it. The published recommendation is unambiguous: reproduce every post-silicon control-logic issue in a formal environment and check its fix with formal technology before the design goes for a re-spin <sup>[6]</sup>. The economics are hard to argue with — the environment already exists, a proof over the fixed logic costs person-days, and the alternative is discovering in the next stepping that the fix was incomplete.

**Scenario.** The flagship’s inline AES-GCM engine on the memory path has a fault: under a particular back-to-back key-slot switch, one 16-byte block is encrypted with the previous key. It is found in the lab, in month two, on a workload nobody expected to run. **Approach.** Triage in a day: it is a functional bug, follows the design not the part, and reproduces with a two-transaction sequence once the sequence is known. Workaround: firmware serialises key-slot switches by draining the pipeline, at a measured cost to throughput on the affected path. Then the decision, which takes a week and is not a verification decision. Because the failure produces ciphertext under the *wrong* key, it is a confidentiality failure, not a performance one; the drain-based workaround is enforceable only in the driver the vendor ships, and any customer writing their own would reintroduce it. The disposition is a stepping, with the firmware workaround shipped in the interim and the erratum published so that customer software cannot inherit the hazard silently. **Why.** Two things decided this and neither was severity-as-symptom. First, the consequence class: a workaround that depends on every future software author doing the right thing is not a workaround for a security property (Chapter 23). Second, discoverability — once a defect of this kind is known outside the company, the time available to mitigate collapses, because it can be exploited rather than merely encountered <sup>[1]</sup>. That asymmetry is why security bugs are dispositioned differently from functional ones of identical technical severity.

One last, unglamorous point. This decision is not made by the verification team and should not be: it belongs to a room holding information verification does not have — committed customers, launch dates, the qualification plan, the cost of a mask revision, the competitive window. What verification owes that room is not an opinion about respinning. It is three things stated precisely: **what the bug does, what conditions are necessary and sufficient to hit it, and what the proposed workaround does and does not cover.** A team that supplies those three is why the decision is good. A team that supplies a severity label and a shrug has outsourced the only part of the judgement it was uniquely qualified to make.

## IN PRACTICE

Parts are scarce and bench time is scheduled like a beam line. Debug competes for the same silicon with characterisation, with the software team that needs a platform, and with whoever promised a demonstration — so the technique that wins is often the one needing fewer part-hours, not the one that is technically best. The lab team

is usually not the verification team either, and the hand-off is lossy both ways: the lab receives a design it did not verify, and verification receives a failure description written by someone who has never seen the coverage model.

Three habits fail quietly and often. Escape analysis gets scheduled *after* launch, which means after the team disperses, which means never. Defeature modes and irritators ship unchecked because nothing in the pre-silicon plan owns them — and a bug inside a defeature mode costs a week of chasing a ghost that does not exist in the functional design. And a firmware workaround closes the ticket, so the coverage hole that let the bug through is never filed and the next generation inherits it with the IP.

The context does not help: in 2024, 14 percent of IC/ASIC projects reached first-silicon success and 75 percent finished behind their original schedule <sup>[4]</sup>. The lab inherits schedule debt it did not create, and it is the last place in the flow with anywhere to pass it.

---

## PITFALLS

1. **Sizing DfD from an area budget rather than from debug scenarios.** *Symptom:* “we allocated our usual 20 percent for debug.” *Cure:* size it from the failures you would most fear per block — what would you need to see to separate their two likeliest causes — then negotiate the area against that list.
2. **Spending the trace budget before bisecting the configuration.** *Symptom:* a week of trace captures and no hypothesis. *Cure:* configuration bisection first; it costs one run per question and it decides where the trace should point.
3. **Treating “cannot reproduce” as a property of the bug.** *Symptom:* an intermittent filed as non-deterministic and parked. *Cure:* treat it as a recipe with an unvaried parameter — temperature, voltage, part, elapsed time, active feature set — and vary them deliberately.
4. **A workaround that masks other bugs.** *Symptom:* a subsystem disabled to unblock the lab, and its bug queue never grows again. *Cure:* every workaround records what it stops exercising, and that record is a validation gap with an owner.
5. **Closing an escape with a fix and nothing else.** *Symptom:* the bug tracker closes; the plan, the coverage model and the checklist are untouched. *Cure:* no post-silicon bug closes without a named upstream change — a plan row, a cross, a checker, or a written assumption.
6. **Dispositioning by symptom.** *Symptom:* a hang is S1 because hangs look bad, a corruption is S2 because it is one bit. *Cure:* severity is impact × workaround cost × fix complexity, and it is not knowable until the root cause and a proposed fix exist — and a clock-domain waiver deferred to “we’ll see it in the lab” is the same error one stage earlier, since metastability is precisely what a bench does not reproduce.

---

## SUMMARY

- **Validation** is checking real hardware, not a model. Post-silicon inverts the pre-silicon budget completely: you can run everything and see almost nothing.
- The observability you have in the lab is the observability you designed in. DfD obeys **signals × cycles ≤ buffer**, and that rectangle is fixed at tape-out — 64 KB buys 128 signals for 4,096 cycles, and no amount of lab ingenuity widens it.
- Bring-up is a verification artifact. Order the checks so that each one is decidable using only what the checks before it established, and give every row a single exit criterion.
- Detection is cheap; **localisation** is the expensive step, because error detection latency for difficult bugs runs to millions or billions of cycles against a trace window of a few thousand [2]. Bisect in configuration before you bisect in time, and shorten the latency before you search it.
- The deliverable of post-silicon debug is not an explanation. It is a **pre-silicon re-producer** — short enough to simulate, small enough to load, still failing [1].
- Every escape owes an upstream change: a plan row, a coverage cross, a checker, or a written assumption. A post-mortem that produces only a fix has repaired a hole without repairing the hole-making process.
- After tape-out the disposition triad becomes stepping / shipped workaround / errata, and which one you can reach depends on configurability designed in a year earlier.

---

## FURTHER READING

**From the corpus.** The post-silicon tutorial [1] is the anchor for this chapter and the best single overview of the field: the pre-silicon-to-field continuum of checking activities, the pre-sighting/sighting vocabulary, trace signal selection and state restoration, the readiness workstream, and a detailed account of one large processor bring-up. The symbolic quick-error-detection paper [2] is the sharpest treatment of error detection latency and automated localisation, with the bug scenarios and trace-length data behind this chapter's numbers; the accompanying industrial deck [5] adds the effort comparisons. For formal in the post-silicon phase, [6] is a step-by-step method with two worked industrial cases, and it is the source for the constraint strategy — over-constrain what did not participate, under-constrain what did — and for loading a proof's initial state from a simulation waveform. The 2024 trend reports supply the industry context: [4] for first-silicon success, schedule slip and why metastability is the bug class least likely to be caught on a bench; [7] for what happens to teams that plan to reach the lab quickly and do their checking there — a strategy that worked for simpler devices and is increasingly impractical for SoC-class FPGAs.

**Outside the corpus.** P. Mishra and F. Farahmandi (eds.), *Post-Silicon Validation and Debug* (2019), is the book-length treatment of everything this chapter compresses. For bring-up as an industrial practice, A. Nahir et al., "Post-silicon validation of the

IBM POWER8 processor” (DAC 2014), and M. Riley et al., “Debug of the CELL processor: moving the lab into silicon” (ITC 2006), are the clearest published accounts of a real lab. F. M. De Paula, A. J. Hu and A. Nahir, “nuTAB-BackSpace” (CAV 2012), is the primary source on reconstructing history backwards from a non-deterministic system, and S.-B. Park, T. Hong and S. Mitra, “Post-silicon bug localization in processors using instruction footprint recording and analysis” (IEEE TCAD, 2009), is the standard reference for on-chip localisation instrumentation.

---

## REFERENCES

1. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
2. **P9** D. Lin, E. Singh, C. Barrett, and S. Mitra, "A Structured Approach to Post-Silicon Validation and Debug Using Symbolic Quick Error Detection," Proc. IEEE International Test Conference (ITC), Anaheim, CA, pp. 1-10, 2015.
3. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.
4. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
5. **D24** S. Mitra, E. Singh, and K. Devarajegowda, "QED & Symbolic QED: Pre-silicon Verification, Post-silicon Validation, Industrial Results," Proc. DVCon Europe, 2019.
6. **D3** A. Jain, A. Gupta, M. V. Achutha Kiran Kumar, Ss Bindumadhava, S. S. Kolar, and S. Gadey NV, "Never too late with formal: Stepwise guide for applying formal verification in post-silicon phase to avoid re-spins," Proc. DVCon U.S., 2022.
7. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.

PART VI

## Specialized Domains

---

*The obligations a market imposes: where the digital abstraction meets the continuous world, what changes when a standard is watching, and how to verify against an adversary.*



---

## 21. Analog and Mixed-Signal Verification

The reference SoC carries its analog front-end on-die: a 12-bit SAR ADC, a PLL, and a temperature sensor, all under the control of a peripheral-domain register block the digital team owns. The first checker is written the way this team has written every checker for two years — apply a known input, compute the expected code, compare:

```
assert (adc_code == expected_code);
```

It fails on nearly every run. Not by much: one code, occasionally two. Three engineers spend a week looking for the bug, and there is no bug. One least-significant bit against the front-end's 1.8 V reference is  $1.8 / 4096 \approx 439 \mu\text{V}$ , and 439  $\mu\text{V}$  is smaller than the amount by which the model, the schematic and the eventual silicon are *designed* to disagree. The comparison was never going to hold.

The instinct that follows is correct; the way it is usually carried out is where mixed-signal verification goes wrong. Someone changes the check to `±2 LSB`, the regression goes green, and the number is never written down anywhere but that line of code. Six months later a run fails at 3 LSB and nobody can say whether that is a bug, a modelling artifact, or an unlucky seed — because nobody knows where the 2 came from. A debugging convenience has quietly become the specification.

This chapter is about that moment. The digital abstraction has stopped, and everything the team built on top of it — the oracle, the checkers, the coverage model, the pass/fail rule that decides whether a regression passed — has to be rebuilt on a foundation where *correct* is a band rather than an equality. It is also about a boundary: the digital verification engineer is, in almost every organisation, not the person verifying the ADC. Naming what they *do* own, precisely, is worth more than a superficial tour of analog verification. The two normative documents behind this chapter are Accellera's *Verilog-AMS Language Reference Manual* (Accellera Std VAMS-2023) <sup>[1]</sup> and its *Universal Verification Methodology for Mixed-Signal Standard* (UVM-MS), Version 1.0, January 2025 <sup>[2]</sup>.

---

### CHAPTER MAP

- §21.1 establishes why the oracle is the first thing that breaks at the analog boundary, and what a tolerance-based verdict costs downstream in checkers, coverage and regression pass/fail.
- §21.2 states what real number modelling captures, what it silently discards, and why it is the technique that made mixed-signal regression practical at all.
- §21.3 treats when to reach for full AMS co-simulation, what it costs, and how to choose the handful of scenarios that earn it.

- §21.4 develops how behavioural models are correlated against the circuit, when they go stale, and who owns them — an analog model is an assumption with a waveform.
- §21.5 draws the boundary where the digital team’s responsibility actually starts and stops: the interface contract, the control and calibration sequences, and the failure handling.

## 21.1 Where the digital abstraction stops

---

Two assumptions underpin every technique in the preceding twenty chapters, and both are built so deeply into the tools that most engineers have never had to notice them. The first is that a signal takes a value from a small discrete set, so comparison is exact and a bin either contains a value or does not. The second is that time advances in discrete steps, so “the value at the clock edge” is a well-defined thing to sample.

An ADC, a PLL, a voltage regulator and a high-speed PHY honour neither. Checking one means measuring amplitude, gain, frequency, phase or similar quantities rather than reading a logic value <sup>[3]</sup> — continuous properties, none of them reducible to a bit without discarding the thing you were trying to check. A PLL is not “locked” the way a state machine is in a state; it is within some frequency error of its target, and has been for long enough that you are willing to call it settled.

What breaks first is the **oracle**. Chapter 3 established that every practical oracle is partial — it checks a projection of correctness rather than correctness itself — and the analog boundary is the sharpest instance of that principle in the book. Here the projection is explicit and numeric: you are not checking that the output is right, you are checking that it lies inside a band, for a specified time, under specified conditions. The verdict has three parts where a digital verdict has one:

- **A band.** Absolute ( $\pm 10$  mV) or relative ( $\pm 2\%$ ), and the choice matters: a relative tolerance on a quantity that legitimately approaches zero is a trap, and an absolute tolerance on a quantity that spans three decades is meaningless over most of its range.
- **A window.** Analog outputs settle. A comparison made before the settling time has elapsed is checking a transient; one made at an arbitrary time is checking whatever the model’s step size happened to produce. Every analog check needs a stated moment or interval at which it is valid.
- **A condition set.** Supply voltage, temperature, process corner, input frequency, load. A tolerance without the conditions under which it holds is not a specification.

Everything downstream inherits that structure. The checker becomes a band comparison instead of an equality. Coverage becomes harder in a way that is easy to underestimate: a coverage bin on a real value that names a single point can fail to be hit at all, because floating-point representation has limited precision —  $0.1 + 0.2$  yields  $0.30000000000000004$ , which does not fall in a bin covering `{0.3}` — and the

language standard names this explicitly as a problem for analog specifications that carry tolerance ranges. The remedy is in the same standard: a tolerance token attached to a single real value defines an implicit range, so `{1.8 +/- 0.05}` and `{1.8 + %- 2.0}` are bins that a real measurement can actually land in [4].

And regression pass/fail inherits all of it. A digital regression's verdict is a property of the design; a mixed-signal regression's verdict is a property of the design *and* of every tolerance somebody typed into a checker. Scattered through the testbench as literals, those numbers leave the team unable to answer the two questions that matter in a bring-up crisis — *where did this tolerance come from* and *who approved it*.

So the tolerances belong in the verification plan, as the closure criterion of the row, not in the code. A digital plan row closes when a coverpoint is hit or an assertion is proven; a mixed-signal row closes when a measured quantity sits inside a stated band under stated conditions, with a named source for the band. That traceability is exactly what analog verification has historically lacked, where practice has leaned on interactive simulation and visual inspection of waveforms by the same analog team that designed the circuit [3].

**Scenario.** The reference SoC's PLL must be within 100 ppm of its target after lock. **Approach.** The plan row states the band (100 ppm), the window (measured over 1024 output cycles beginning 20  $\mu$ s after the lock request), the conditions (nominal supply, three temperature points, three process corners), and the source of the band — the tightest frequency accuracy any consumer of the system clock demands. The checker reads all four from a package, not from literals. **Why.** When a run fails at 140 ppm, the conversation is about which requirement is violated and whether the band or the design should move. Without the row, the conversation is about whether 100 was ever the right number, and it has no ending.

## 21.2 Real number modelling

There is a pragmatic middle ground between simulating an analog block properly and pretending it is a black box that returns whatever the testbench needs. **Real number modelling** (RNM) describes the block's behaviour with real-valued signals inside the ordinary event-driven simulator: a voltage is a `real`, a net carries a number rather than a logic value, and the model computes its output directly from its inputs and internal state.

The distinction from analog modelling is structural, not cosmetic. An analog model describes the current-versus-voltage relationship between nodes, and the solver finds all node voltages and branch currents by Newton-Raphson iteration with matrix inversion, choosing each timestep from accuracy criteria. A real model has no matrix solution at all: the output is computed directly, the model itself decides when to perform each internal computation, and there is no continuous-time operation — only sampled, clocked or event-driven updates. What follows is the whole value proposition. There is no analog solver and therefore no convergence failure [5], and the model runs in an event-driven framework that is inherently faster than the analog solvers it replaces [3].

The substrate is ordinary HDL. Verilog-AMS provides `wreal` nets; SystemVerilog provides user-defined net types with a resolution function, so a net carrying a structure of voltage and current can define what happens when two things drive it. Be blunt about the state of standardisation, though: there is no standard for how discrete modelling of analog nets should be done, so published examples are one method among several rather than a normative pattern [2].

### What it buys

Three measurements, from two independent sources, each against its own baseline:

**Table 21.1** — Three published real number modelling results, each measured against its own baseline: the first row against a SPICE netlist, the other two against Verilog-AMS models of the same blocks. The second column is why the ratios cannot be chained into a ladder — different designs, different references, different years — and the first row's accuracy figure is the one to notice, because what it traded away the verdict never depended on.

Design	Compared against	Result	Year
A major analog block with filtering and fault detection	SPICE netlist	24× faster, 0.3% gain in accuracy at 100 MHz	2011
PLL loop filter	Verilog-AMS	≈90× faster (47 s vs 1 h 8 min 32 s)	2015
Delta-sigma ADC	Verilog-AMS	230× faster (0.4 s vs 92.5 s), ENOB 11.515 vs 11.72	2015

The first row shows why the numbers matter rather than just how large they are. Its 0.3% inaccuracy was far better than required, because the tolerances on the checkers were in the range of 5% — the accuracy traded away for the speedup was accuracy the verdict never depended on [3]. That is the design rule for a real number model: model to the tolerance the plan row states, and not one digit finer. The block behind that row is real and named: the proof-of-concept targeted a current design from LSI Corporation, and the block picked for it was a major analog block driver with filtering and fault detection [3]. One block in one company's design is the whole of that evidence, which is why the ratio is quoted here with its baseline attached and never on its own.

The other two rows come from a different comparison entirely — SystemVerilog real models against Verilog-AMS models of the same blocks, not against SPICE [5]. The three ratios measure separate things. They do not chain into a ladder, and any composite figure built by multiplying them would be a new claim with nothing behind it.

### What it silently discards

Speed is bought with abstraction, and it is worth knowing exactly what left the building.

**Electrical relationships.** A real net carries a value, not a Kirchhoff relation. Loading, drive strength, input impedance and current draw are simply absent unless the modeller writes them in, and in SystemVerilog that means a user-defined net type with a resolution function that does the arithmetic explicitly. A shared bias line feeding two loads behaves in a real model exactly as it would with one load, which is the point at which a whole class of real circuit failures becomes invisible.

**Continuous time.** The model updates on events. Between updates the modelled quantity is a held value, and any transient shorter than the update interval does not exist. Glitches, ringing, overshoot on a supply ramp and the settling behaviour that a tolerance window is supposed to be checking are all things a real model shows only if the modeller decided in advance to represent them.

**Noise, variation and margin.** Thermal noise, jitter, offset, mismatch and the spread across process corners are not free consequences of a real model the way they are consequences of a circuit. Each one is a feature somebody wrote, with parameters somebody chose.

None of this argues against RNM: the alternative to abstraction here is not simulating everything, it is simulating nothing. It argues for knowing which of your checks depend on the discarded physics, because those checks are checking the model rather than the design.

**Scenario.** The radar front-end's receive chain runs from mixer output through a variable-gain amplifier to the ADC, feeding the FFT and CFAR detection pipelines. Chapter 17 replayed a recorded chirp sequence through it, because the property under test there was bit-exact arithmetic across three configurations and that demands the identical input every time. **Approach.** For a different property, a real number model of the analog path lets the testbench *generate* the chirp: sweep the return amplitude across the full input range, inject a controlled noise floor, drive the amplifier into and out of saturation, and run it as a constrained-random regression. **Why.** Detection behaviour at low signal-to-noise ratio is a question about coverage of conditions, not about reproducibility, and a recorded capture set contains exactly the SNR values that happened to be recorded. The two stimulus strategies answer different questions and the project needs both. **The limit.** The model's noise floor is a parameter, not a measurement, so a CFAR threshold tuned against it has been tuned against a cleaner world than the silicon. That number needs a source, and §21.4 is about where the source comes from.

## 21.3 AMS co-simulation

When the physics matters, the alternative is to run both engines at once: the digital event-driven simulator coupled to an analog time-domain solver, one simulation with two solvers exchanging values at their shared boundary. This is what buys back everything §21.2 discarded — real electrical relationships, real transients, the actual convergence behaviour of the actual circuit.

The coupling is where the cost lives, and it repays understanding mechanically rather than as a vague warning about speed. Where a continuous discipline meets a discrete one, connect modules are automatically inserted to join them, selected by the disciplines and directions of the ports involved [1]. And every time the digital side wants an analog value — say, on a periodic sampling clock — the simulator must ask the analog solver to interpolate one for that digital time point. That request is a *blocking* one: control passes to the analog solver and does not return until the value is available [2]. A sampling clock chosen without thought does not merely add work; it repeatedly stalls the digital engine on the slowest component in the system.

The magnitude of the gap is visible in §21.2's own table read backwards. If a real number model of a delta-sigma converter finishes in 0.4 seconds where the Verilog-AMS version takes 92.5, then at anything like that ratio an hour of real-model regression becomes a multi-day proposition with analog solvers in the loop, and a job that had to finish overnight does not finish at all. This is not a tuning problem. It is the reason AMS co-simulation is a *campaign* — a purpose-built run launched for a stated reason, producing a written result, and then stopping — rather than a regression tier with a nightly budget.

Which makes scenario selection the actual skill, under a rule that survives contact with a schedule: **co-simulate what real number modelling threw away, and nothing else.**

- **Model correlation runs.** The subject of §21.4, and the most valuable use of the analog solver, because it is what makes every subsequent real-model run mean something.
- **Failure modes that live in loading, current and impedance.** Precisely the class that a real model cannot express and therefore cannot fail on.
- **Power-up, power-down and supply ramps.** Sequencing across a boundary where the analog side has a genuine DC operating point to find and the digital side's reset release depends on it.
- **One end-to-end signal-path scenario per configuration.** Not for bug-finding, but as evidence that the composed system behaves as the real models claim.

**Scenario.** The sensor hub's always-on island shares one bias reference between its temperature channel and its pressure channel, and firmware may enable them independently. **Approach.** The real model of the bias generator delivers its nominal output regardless of how many channels are enabled, so a real-model regression will never show a problem. One co-simulated scenario enables both channels and checks the reference under the doubled load. **Why.** The failure is a droop in the



shared reference when the second channel switches on, which shifts the first channel's conversion by more than its stated tolerance — a loading failure by construction, and therefore invisible to any abstraction that does not carry current. **What it costs.** One scenario, re-run when the bias generator's schematic changes, not nightly.

### The flagship's PHY, and the limit of the boundary

Chapters 18 and 20 both deferred the LPDDR4 PHY's training to this chapter, so it is owed a straight answer. The AMS boundary can discharge the *algorithm*: that the training state machine converges against a channel model, terminates rather than loops when a step fails, has reachable and correct retry and fallback paths, and respects the protocol's timing obligations throughout. That is substantial, genuinely risky work, and real models plus a few co-simulated runs reach it.

It cannot discharge what Chapter 20 names: the *distribution* of training outcomes across parts and temperature. Every pre-silicon channel model trains against an idealisation of signal integrity, and the spread separating a part that converges at the centre of its acceptance window from one that converges at the edge does not exist until there are parts. The plan carries both rows; the second closes in the lab. Writing that division down is the point — discovering it during bring-up is not.

## 21.4 Behavioural models and the trust problem

Everything in the two preceding sections rests on a model, and a model is a claim about the silicon. **A model nobody has correlated is a lie you regress against nightly** — an expensive one, because it produces green results, at speed, in volume, for months.

### The reference is inverted

In digital work the specification is the reference and the implementation is checked against it. That is what makes a *golden* model golden in this book's vocabulary — it is golden **because it is the specification**, and Chapter 8's cleanest case is the one where a standard ships the executable model outright, so that checking collapses to comparison. Synthesis then produces the gates, and equivalence checking closes the gap between RTL and gates mechanically.

In analog, the direction reverses. At the block level people do write specifications, but the schematic is treated as the reference — golden in a different sense, golden because it is what gets fabricated. There is no analog synthesis, so the schematic is a creative artifact rather than a derived one, and there is no equivalence checking to confirm that a hand-written model matches the schematic that will become silicon. Chip-level verification therefore assumes correlated models, and without them chip-level verification is suspect [6].

That inversion relocates the whole problem. Digital's risk is a misinterpreted specification, which is why the discipline puts two independent pairs of eyes between

specification and RTL. Analog specifications are usually interpreted correctly and are quite simple; the problems are creating the implementation and creating the model. So analog’s “second pair of eyes” is not an assertion in a testbench — it is the comparison of two independently created representations of the same block, the schematic and the model [6].

### What model correlation actually is

Not a review, not a side-by-side glance at two waveforms, and not a claim in a design document. **Model correlation** is a procedure with an artifact:

1. **One self-checking testbench**, written against the block’s specification.
2. **Run twice** — once against the behavioural model, once against the schematic at transistor level.
3. **Compared per port, against a declared tolerance**. In a published programmable-gain-amplifier example, the specification states that the output voltage of the model and of the schematic must differ by less than 10 mV under the same stimulus, and that the supply current on the supply pin must match to within 1 mA [6].

The third point is where most teams fall short, and it is worth stating plainly: **the correlation tolerance is part of the specification, per port, before anyone runs anything**. A match criterion invented after seeing the mismatch is not a criterion.

One place analog is genuinely *easier* than digital makes this procedure affordable. Analog blocks generally have little state, and that amount keeps falling as design style pushes digital logic out to the digital designers. For functionality at the analog block level, all states can typically be simulated exhaustively even at transistor level, giving 100% coverage for the digital inputs [6]. The combinatorial explosion that forces constrained-random stimulus on a digital block is simply not there. The modes are enumerable; enumerate them.

### When a model goes stale

The correlation result is a statement about one revision of one schematic. It expires. The trigger that must be written into the flow is explicit in the practice: the process is repeated as the schematics and/or specifications are updated [6]. Three further triggers are worth naming, because they are the ones teams miss:

- **A new operating condition**. A model correlated at nominal supply says nothing about behaviour at the corners of that supply, an entire power-management campaign can run against a model that was never correlated outside its nominal box.
- **A parameter change with no schematic change**. Someone retunes a bias current or a trim default. The schematic revision moves; the model, which hard-codes the old value, does not.

- **A model edit made during debug.** The most dangerous of all, because it happens under pressure, it makes a failing test pass, and it is exactly the kind of change nobody re-correlates.

This is why correlation belongs in the flow as a **campaign** with named triggers rather than as a regression tier: it runs the schematic at transistor level, so §21.3's economics apply in full, and its written result stands until one of the triggers fires. Running it nightly is neither affordable nor useful; running it never is how a project arrives at tape-out with a chip-level regression that has been checking nothing.

### Who owns it, and what it costs

Model creation is not a rounding error in the schedule. One consultancy's experience across many mixed-signal chips puts the creation of correlated analog functional models at as much as 80% of the effort involved in getting analog blocks into chip-level verification, and the analog verification effort as a whole at typically 20% of the design effort — rising to 30% or 40% for a team new to it. The same source puts the proportion of analog designers skilled at *reading* models at around 20%, with fewer able to write them [6]. These are experiential figures rather than survey data, and they carry no year; treat them as the shape of the problem, not as a budget line.

One project's measured data point sits alongside them. In a proof-of-concept run in about three months of calendar time — half of it verification planning — creating and checking the real number model of the block took an expert analog modeller two weeks, on top of three weeks and two weeks for the two implementation phases [3]. That was 2011, one block, one team. It is enough to establish that a real number model is a multi-week specialist deliverable rather than an afternoon's work, and not enough to establish anything else.

Ownership follows from that skill distribution rather than from an org chart. The model is written by an analog designer or a specialist modeller and consumed by the digital verification team — so the person who can tell whether it is right is not the person who notices when it is wrong. That gap needs an artifact.

### The model register

Chapter 14 argued that an unproven premise which nobody has written down will still be unproven at sign-off, and gave it an artifact: the assumption register, one row per assumption, reviewed like a waiver list. An analog model is an assumption with a waveform, and it earns the same treatment.

**Table 21.2** — A model register entry laid out one field per row, filled in for the analog front-end's ADC: what the model abstracts away, what it was correlated against, under which criteria, and at which conditions. The last two fields are what make the register worth maintaining — a date lets a correlation claim visibly go stale, and the consequence field answers the only question a reviewer needs to ask about a model.

Field	Example
Model ID	MDL-AFE-ADC-01
Block	Analog front-end, 12-bit SAR ADC

Field	Example
Abstraction	Real number model, event-driven, no supply-current behaviour
Correlated against	Schematic rev. 7, transistor level
Correlation criteria	Output code within 1% of full scale ( $\approx 41$ LSB); conversion time within 5%
Conditions covered	Nominal supply, 3 temperature points; corners <b>not</b> correlated
Date and owner	Named analog engineer, with date
If wrong	Every ADC-path result in the chip-level regression, and every plan row that closes on an ADC measurement

The last two rows are what make it worth maintaining. A date turns “the model is correlated” into a claim that can visibly go stale, and the consequence field lets a reviewer at sign-off ask the only question that matters about a model: if this is wrong, what else is wrong with it?

## 21.5 What the digital team owns

The most useful thing this chapter can do is draw a boundary, because the alternative — a digital verification engineer half-learning analog verification — produces neither competence. You are almost certainly not verifying the analog block. Its performance, noise, linearity and corner behaviour belong to the analog team, who have tools and instincts you do not. What you own is everything the digital domain does *to* and *about* that block, and it is bug-rich territory: the RTL that talks to the analog does calibration, control loops and power-up sequences, and is a documented source of chip bugs in its own right, alongside interface faults such as missing level shifters and unconnected signals <sup>[6]</sup>.

Four things, concretely.

**The interface contract.** Every control line’s meaning, every status line’s timing, the register map, and the wiring. At the analog subsystem level the wires include supply lines, current bias lines, voltage references and digital control lines, and all of them have to be connected correctly; at chip level there are thousands running from the digital domain into the analog <sup>[6]</sup>. This is a connectivity problem, and Chapter 15 already owns the technique for it; the analog-specific part is only that a swapped or inverted control line here produces a plausible-looking analog output rather than an obvious failure, so the digital-side check is the one that will catch it.

**The control and calibration sequences.** The algorithm that drives the analog block into specification: trim searches, offset cancellation, lock acquisition, gain ranging. Pure digital logic operating on analog feedback, and where the interesting bugs are — it is the only part of the system whose correctness depends on both a state machine and a tolerance.

**The failure handling.** What the design does when the analog reports out-of-range, fails to converge, or saturates. Chapter 5 called this class a must-not-happen feature, and it is systematically under-verified at the analog boundary because the failure conditions are ones the model must be *told* to produce.

**The tolerances themselves.** Not the physics behind them, but their provenance: every band in the plan traceable to a specification, and the model register kept current.

**Scenario.** The sensor hub's temperature channel runs a trim search at wake-up: the always-on controller steps a trim code, waits for the reading to settle, and keeps the code whose reading is closest to the on-die reference. **Approach.** Verify the search, not the sensor. Does it terminate for every starting code, including the two ends of the range? What does it do when two codes tie? What happens when the reading never settles because the sensor is broken — does it converge on a wrong code, or report failure? Run all of it against a real number model with injectable offset, drift and non-monotonicity. **Why.** The analog team owns whether the sensor is accurate; the digital team owns whether the search finds the right code, in bounded time, and says so honestly when it cannot. **The trap.** A search that always converges in the model may be relying on monotonic behaviour the real sensor does not guarantee — which is a modelling assumption, and belongs in the register with the model that carries it.

**Scenario.** The flash controller reads a page and ECC reports too many errors to correct. Its read-retry state machine walks a table of reference-voltage offsets, re-reading at each until the page decodes. **Approach.** The digital team verifies that state machine — the table is applied in order, the retry budget is respected, a mid-retry power loss leaves the array readable, and exhausting the table reports an uncorrectable error rather than silently returning the last attempt. **Why.** NAND reads are threshold-voltage comparisons, so the analog quantity being manipulated is a reference level; but the correctness question is entirely digital. **What is different here.** The oracle is a *probability*, not a band: with ECC downstream, a marginal read is not wrong, it is more likely to need correction. The plan row closes on a bit-error-rate distribution across the retry table, which is a third currency for a closure criterion after equality and tolerance — and one reason to name the currency explicitly in every mixed-signal plan row.

The radar front-end supplies the third obligation in its purest form. Chapter 5 put an arithmetic must-not-happen entry on its FFT — no stage shall saturate without raising its saturation flag — and the analog boundary adds a second, upstream one: the receive chain flags an input driven beyond the amplifier's range, and the detection pipeline must discard that frame rather than treat a clipped signal as a target. The two flags are different mechanisms and both belong on the negative list. The upstream one is the more likely to go unverified, because raising it means driving the real number model into a region nobody thinks to visit — which is precisely the argument for stimulus that sweeps amplitude rather than replaying captures.

**Worked artifact: tolerances with one owner**

The chapter's opening failure was a tolerance invented in a checker. The cure is mechanical — every band lives in one package, named, with a comment saying what it is, and every checker and coverage bin quotes it. These are IEEE 1800-2023 constructs, and the tolerance tokens in particular need a tool configured for that language version:

```
package afe_spec_pkg;
 // Bands for the analog front-end. One place, one owner, quoted everywhere.
 localparam real V_REF = 1.8; // ADC reference, volts
 localparam real V_MIDSCALE = V_REF / 2.0; // mid-scale of the input span, volts
 localparam real V_BIN_STEP = 0.05; // coverage bin width, volts
 localparam real V_BIN_TOL = 0.005; // mid-scale bin half-width, volts
 localparam real V_CORR_ABS = 0.01 * V_REF; // model-vs-schematic band, volts: 1%
 // of full scale, from MDL-AFE-ADC-01
endpackage

module afe_probe (input logic sample_done, input bit corr_run);
 import afe_spec_pkg::*;

 real v_in; // stimulus, applied identically to model and schematic
 real v_md1; // real number model output, volts
 real v_ckt; // transistor-level output, volts; driven, and corr_run high, in
 // correlation runs only

 covergroup cg_afe_input @(posedge sample_done);
 cp_v_in : coverpoint v_in {
 type_option.real_interval = V_BIN_STEP;
 bins span[] = {[0.0 : V_REF]};
 bins midscale = {[V_MIDSCALE +/- V_BIN_TOL]};
 }
 endgroup

 cg_afe_input cg_i = new();

 always @(posedge sample_done)
 if (corr_run && !(v_md1 inside {[v_ckt +/- V_CORR_ABS]}))
 $error("AFE model outside its correlation band");
endmodule
```

Three details carry the argument. `type_option.real_interval` sets the width of the automatically created bins, so `span[]` divides the input range into equal intervals rather than demanding exact values. `midscale` uses an absolute tolerance token: its total width is smaller than the real interval, so it stays a single bin, and a measurement near half-scale lands in it instead of falling between two points floating-point arithmetic will never produce exactly. The correlation check uses the *absolute* token, discharging §21.1's rule: a band taken as a percentage of the measured value shrinks with the signal and collapses near zero, whereas the same percentage of the fixed full-scale reference is a constant band across the whole range. Underneath all three, none of those numbers is a bare literal: each is a `localparam` with a comment, and the correlation band names the register row behind it — which is the entire point. The `corr_run` qualifier keeps the nightly regression and the correlation campaigns apart in one file, so a nightly run never compares `v_md1` against an undriven `v_ckt`.



---

## IN PRACTICE

**Two populations, not one.** A nightly regression built entirely on real number models, and correlation campaigns fired by the triggers of §21.4. Mixing them produces a nightly job that is too slow to be nightly and a correlation result nobody can locate.

**Randomise the supply, and cover it.** The highest-value analog coverage habit, and it transfers directly from a published fractional-N PLL case: with a nominal supply of 2.5 V, the testbench randomises it across  $\pm 7\%$  — deliberately wider than the  $\pm 5\%$  legal band — and a covergroup carries three bins, an under-voltage region below 2.375 V, a healthy band from 2.375 V to 2.625 V, and an over-voltage region above [5]. The numbers belong to that design; the habit is what transfers. In that example twenty runs produce a report that *shows* whether the supply ever left its healthy band — and a team that has never asked the question is usually surprised by the answer, typically not before a supply-related failure turns up in the lab.

**Split the model's ownership from the model's accounting.** The analog engineer writes and correlates the model; the verification team owns its row in the register and its expiry. Neither role works alone: the writer does not see the chip-level runs that depend on it, and the accountant cannot judge whether the physics is right.

---

## PITFALLS

1. **A tolerance invented in a checker.** *Symptom:* a `±2 LSB` literal nobody can source, and arguments about whether a marginal failure is a bug. *Cure:* every band is a named constant traceable to a plan row and a specification.
2. **Comparing before the output has settled.** *Symptom:* intermittent failures that move when the model's timestep changes. *Cure:* every analog check states the moment or interval at which it is valid, and the check is qualified by it.
3. **Trusting a real number model for something electrical.** *Symptom:* a loading, droop or current-draw bug found in the lab that no simulation could have shown. *Cure:* enumerate which checks depend on discarded physics, and cover those in a co-simulation campaign.
4. **A model correlated once, at nominal, and never again.** *Symptom:* months of green chip-level regressions against a model that stopped matching the schematic three revisions ago. *Cure:* the model register, with a date, an owner, the conditions covered, and named re-correlation triggers.
5. **Co-simulating on a fast periodic sampling clock.** *Symptom:* a co-simulation far slower than its analog content justifies. *Cure:* remember that each query is a blocking request into the analog solver, and sample on change where the modeling style allows it.
6. **Verifying the analog block instead of the digital control of it.** *Symptom:* effort spent reproducing the analog team's characterisation, while the calibration sequence's failure paths are untested. *Cure:* the boundary of §21.5, written into the plan as an explicit scope statement.

---

## SUMMARY

- At the analog boundary the oracle stops being an equality and becomes a **band, a window, and a condition set**. Everything downstream — checkers, coverage, regression pass/fail — is rebuilt on that.
- **Real number modelling** keeps digital simulation speed while representing continuous quantities, and it is what made mixed-signal regression practical. It discards electrical relationships, continuous time, and noise; know which of your checks depended on them.
- Published speedups are real and are **not composable**: they measure different baselines, different designs, and different years. Quote them with their baseline or not at all.
- **AMS co-simulation is a campaign, not a tier**. Spend it on model correlation, on failures that live in loading and current, on supply sequencing, and on one end-to-end run per configuration.
- **An analog model is an assumption with a waveform**. Correlate it against the schematic with per-port tolerances declared in advance, record it in a model register with a date and an owner, and re-correlate on named triggers.
- The digital team owns the **interface contract, the control and calibration sequences, the failure handling, and the provenance of every tolerance** — not the analog block. Naming that boundary in the plan is worth more than blurring it.

---

## FURTHER READING

From the corpus: <sup>[3]</sup> for metric-driven verification applied end to end to an analog block, including the effort data and the coverage-to-plan mapping; <sup>[6]</sup> for the trust problem, the specification-driven approach to model correlation, and the effort economics; <sup>[5]</sup> for real-versus-analog modelling, the runtime comparisons, and worked supply-coverage and real-randomisation examples; <sup>[2]</sup> for the mixed-signal UVM bridge architecture and the push, push-sync and pull coupling patterns; <sup>[1]</sup> for the continuous-discrete boundary and connect-module semantics; <sup>[4]</sup> for real cover-points, tolerance tokens and real-valued randomisation.

Not in the corpus, and worth owning if this is your area: Kenneth S. Kundert and Olaf Zinke, *The Designer's Guide to Verilog-AMS* (Kluwer Academic Publishers, 2004), still the clearest treatment of behavioural modelling discipline; and Peter J. Ashenden, Gregory D. Peterson and Darrell A. Teegarden, *The System Designer's Guide to VHDL-AMS* (Morgan Kaufmann, 2002), for the same material from the VHDL side.

---

## REFERENCES

1. **S5** Accellera Systems Initiative, "Verilog-AMS Language Reference Manual, Accellera Std VAMS-2023," Accellera, February 2024.

2. **S13** Accellera Systems Initiative, "Universal Verification Methodology for Mixed-Signal Standard (UVM-MS), Version 1.0," Accellera, January 2025.
3. **D19** N. Khan, Y. Kashai, and H. Fang, "Metric Driven Verification of Mixed-Signal Designs," Proc. DVCon U.S., 2011.
4. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
5. **D30** J. Brennan, T. Ziller, K. Fotouhi, and A. Osman, "The How To's of Advanced Mixed-Signal Verification," Proc. DVCon Europe, 2015.
6. **D27** H. Chang and K. Kundert, "Specification Driven Analog and Mixed-Signal Verification," Proc. DVCon U.S., 2015.

---

## 22. Safety Verification

Eleven weeks before the flagship-A's tape-out, the verification evidence is complete by every conventional measure: a verification plan whose rows are all discharged, a coverage database at closure, a green regression, a bug curve that flattened a month ago. The review ahead is a functional-safety audit, and the first question an assessor from the customer's safety organisation asks is about none of it.

"Your architecture says the tightly-coupled memory is protected by single-error-correcting, double-error-detecting code, and that a double-bit error is reported to the safety island inside the fault-tolerant time interval. Show me the faults you injected into that path. Show me the ones the mechanism did not catch. And show me your argument for why the faults you did not inject did not need injecting."

Nobody on the team can answer, because nobody was asked to. The environment has never corrupted a memory bit on purpose; the error-correcting code was verified the way every other block was — by checking that correct data goes in and correct data comes out. That is a perfectly good functional argument, and it answers a different question. The assessor is not asking whether the design works. She is asking whether the design *notices when its silicon breaks*, how much of the breaking it notices, and how you know.

That question is the whole of this chapter. It changes what you build, what you measure, what you keep, and — more than any of those — what you must be able to say at the end, in a document that outlives everyone who wrote it.

---

### CHAPTER MAP

- §22.1 states what a safety standard actually obliges a verification engineer to do, day to day, beyond what a competent team already does.
- §22.2 establishes why systematic and random hardware faults are two different problems, and why only one of them is what the rest of this book has been about.
- §22.2 then develops how failure analysis becomes a verification input: failure modes, effects, safety mechanisms, and the diagnostic coverage the project is on the hook to demonstrate.
- §22.3 treats how fault injection turns that analysis into evidence — how a fault list is pruned honestly, and why "not detected" and "not dangerous" are different verdicts.
- §22.4 sets out how to verify a safety mechanism, including the half teams forget: that it does *not* fire when it should not; §22.5 states what tool qualification and an audit-proof evidence trail demand.

## 22.1 What changes when a standard is watching

Safety-critical development is no longer a niche. In 2024, 44 percent of IC/ASIC projects reported working to at least one safety-critical development standard or guideline, and among projects that do, the median spends between a quarter and half of total project time on functional-safety activities <sup>[1]</sup>. On FPGA projects the adoption figure is higher still, 63 percent in the same year <sup>[2]</sup>. If you have not worked under a standard yet, the odds are you will.

The standard that dominates the automotive world is ISO 26262, which defines functional safety as the absence of unreasonable risk arising from hazards caused by malfunctioning behaviour of electrical and electronic systems <sup>[3]</sup>. That definition, and the rest of the vocabulary this chapter uses, belongs to Part 1 of the series: ISO 26262-1:2018, *Road vehicles — Functional safety — Part 1: Vocabulary*. The corresponding regime for airborne electronic hardware, DO-254 (RTCA, Inc., *Design Assurance Guidance for Airborne Electronic Hardware*, published by EUROCAE as ED-80), appears in the same 2024 survey among the standards teams comply with <sup>[1]</sup>. Engineers who have worked under both describe it as speaking of design assurance levels rather than integrity levels, with its centre of gravity closer to requirements-based verification than to random-fault metrics. This chapter's concrete detail comes from the automotive side, because that is where the published engineering practice is; the obligations below bite the same way in both.

There are three of them, and one consequence that outranks all three.

**Process obligations.** The standard prescribes *how* work is organised, not only what is produced: activities have prescribed inputs and outputs, reviews have prescribed independence, changes have prescribed impact analysis. A verification engineer feels this as friction with a specific shape — you can no longer quietly delete a test that has become useless, quietly re-scope a coverage model, or quietly upgrade a tool. Each is now a change to a controlled work product, and needs a record of who decided and on what basis.

**Evidence obligations.** Every claim must be supported by an artifact somebody else can inspect — a higher bar than “the regression is green”. Green is a state; evidence is a *retained* state, tied to a specific design revision and tool configuration, with the raw results kept and not just the summary. The shift is from *we ran it* to *we can show, later, what we ran and what came back*.

**Traceability obligations.** Every safety requirement is linked forward to the thing that discharges it, and every test back to the requirement that justifies it. Chapter 5 already asked for traceability as professional hygiene — permanent IDs from specification to feature to test to result. A safety project takes that from hygiene to obligation and adds the direction most teams never build: *backwards*. Given a test, name the requirement it serves. A test that traces to nothing is not merely wasteful; it is an unexplained artifact in a controlled repository, and an assessor will ask about it.

And the consequence: **the argument itself becomes a deliverable**. In an ordinary project the reasoning connecting your evidence to “this is good enough to ship” lives in the heads of the people at the sign-off table and evaporates the day they move on. On a safety project that reasoning is written down, reviewed and shipped — a safety case, whose claims your evidence supports and whose gaps are argued explicitly. Nothing else here changes engineering judgment as much: you are no longer producing results, you are producing *the premises of somebody else’s argument*, and you will be asked whether each premise says what the argument needs it to say.

This is visible in what teams themselves report as hard. Among the biggest functional-safety challenges named by 2024 IC/ASIC participants, safety architecture and design and safety requirement definition, management and traceability lead the list — ahead of fault list generation and injection, which is the part everyone expects to be the hard part <sup>[1]</sup>. The technique is not the bottleneck. The bookkeeping around the technique is.

### Where your target comes from

One piece of vocabulary is worth getting right early, because it governs how much of the rest applies to you. An **ASIL** — automotive safety integrity level, A through D, with D the most demanding — is not a property of your block. It is arrived at by assessing the risk of a potential *hazard*, weighing three judgments: how severe the harm would be, how often the vehicle is exposed to the situation, and how well a driver could control the outcome <sup>[4]</sup>. A headlamp that fails on a lit motorway and one that fails on an unlit country road are the same electronics and different hazards <sup>[3]</sup>.

The integrity level then travels down — hazard, safety goal, system requirements, requirements on your part, requirements on your block — and by the time it reaches you it is a *number in a requirements document*. The honest thing to notice is that you almost never get to argue about it. You inherit a target, and a set of mechanisms someone else chose to meet it.

**Scenario.** The reference SoC has no error-correcting code anywhere. Its 512 KB of tightly-coupled SRAM is fast, four-banked, and completely undefended: a particle strike that flips a bit in a weight buffer produces a wrong answer with no indication that anything happened. That was a defensible choice for a low-power edge part whose worst failure is a wrong inference. **Approach.** When the same architecture is pulled toward an automotive application, the answer is not to bolt detection onto the first-generation part. It is a derivative — the flagship-A — with SECDED protection on the tightly-coupled memory, the L2 slices and the LPDDR path, a dual-core delayed-lockstep safety island, and monitored reset and clock. **Why.** Detection is architecture, not a verification add-on. There is no fault campaign you can run on the reference SoC that will improve its diagnostic coverage, because it has no mechanism whose coverage could be measured. This is the first judgment call of a safety project and it happens before verification starts: if the failure analysis says a failure mode needs covering and no mechanism covers it, the finding is a *design* finding, and the later you raise it the more expensive it is.



One more structural fact about the flagship-A, because it recurs: the safety island is built to ASIL D inside a system built to ASIL B. Mixing integrity levels inside one design is ASIL decomposition, the subject of ISO 26262-9:2018, *Road vehicles — Functional safety — Part 9: Automotive safety integrity level (ASIL)-oriented and safety-oriented analyses*. Different parts of one chip carry different targets, and the boundary between them is itself something you must verify — a fault in the ASIL B region must not defeat the ASIL D region’s mechanisms. That is the reason the safety island exists as an island. Parts of that shape are documented in public: one account describes an ASIL-rated custom automotive microcontroller core at Infineon — 60 instructions, 1.8K flip-flops, 70K gates — verified across 16 versions over 5 years of feature updates, architecture upgrades and bug fixes, and shipping in products as different as a 3D camera, an airbag and a pressure monitor [5]. The scale is worth registering: an ASIL-rated core can be small, and what makes it demanding is the obligation attached to it rather than its gate count.

## 22.2 Failure analysis as a verification input

---

Here is the distinction that organises everything else in this chapter. A **systematic fault** is a design bug: a mistake made during development, present in every part ever manufactured, and always permanent. A **random hardware fault** is a physical defect: a latch-up, a marginal via, an aging transistor, a particle strike — arising during manufacturing or in operation, and either permanent or transient [6].

Almost everything in Chapters 1 through 21 is aimed at systematic faults. Constrained-random stimulus, formal proof, coverage closure, regression engineering, gate-level simulation, post-silicon bring-up: all of it exists to find design bugs before they ship. A safety standard demands that work too, with more process around it — but it does not stop there, because a design can be *perfectly correct* and still kill somebody when a particle flips a bit in its configuration register.

You cannot verify a random fault away. It is a physics problem, and the physics will happen at some rate whatever you do. What you can do is show that when it happens, the design *notices* and does something safe. That is why safety verification feels alien: for the first time you are not hunting a bug, you are measuring a mechanism’s effectiveness against a population of defects that are nobody’s fault.

### FMEDA: the document you are handed

The analysis that produces your work list is a failure mode, effects and diagnostic analysis — **FMEDA**. The part of the series that carries the hardware metrics an FMEDA feeds is ISO 26262-5:2018, *Road vehicles — Functional safety — Part 5: Product development at the hardware level*, and the semiconductor-specific guidance on applying the series to an integrated circuit is ISO 26262-11:2018, *Road vehicles — Functional safety — Part 11: Guidelines on application of ISO 26262 to semiconductors*, which is new in 2018. It works element by element, and for each element asks a fixed set of questions: in what manner can this element fail; what is the consequence if it does; how likely and how severe is that consequence; and

which safety mechanism handles it [6]. A typical answer names a mechanism from a small, stable family — error-correcting code on a memory, a dual-core lockstep pair (two hardware cores running the same instruction stream, watched by a comparator), a software test library, a triply-redundant structure with a majority voter.

Two things about this document matter enormously to a verification engineer and are almost never explained.

First, **the FMEDA is written before your evidence exists.** It is produced by a safety engineer, early, from design data and technology data, and it carries an *estimated* diagnostic coverage for each mechanism — the fraction of that failure mode’s faults the mechanism is claimed to catch. Fault injection is what converts that estimate into a measurement [6]. If your measurement comes in below the estimate, the architecture’s metrics (ISO 26262-5:2018) move, and if they move far enough the part misses its target. This is why fault campaigns start late and end badly: they are the last chance to discover that an architectural assumption was optimistic.

Second, **the granularity of the FMEDA’s failure modes fixes the shape of your fault list.** A failure mode written as “the FIFO indicates its flags incorrectly, so data is overwritten or lost, and the mechanism is redundant flag logic” tells you exactly where to inject and which signal is the alarm [6]. One written as “the block malfunctions” tells you nothing and cannot be measured. Argue with the granularity while arguing is cheap; you cannot re-cut it in the last four weeks.

### The classification you must produce

Fault campaigns exist to sort a design’s faults into classes the standard recognises (ISO 26262-5:2018, with the defined terms themselves in ISO 26262-1:2018). The names are worth learning precisely, because ordinary English gets them wrong [3]:

- A **safe fault** cannot violate the safety goal — either it cannot affect safety-related logic at all, or its effect is one the system tolerates.
- A **single-point fault** violates the safety goal and no safety mechanism is present to catch it. This is the class that must be nearly empty.
- A **residual fault** violates the safety goal in a place where a mechanism *is* present, but this particular fault is outside what the mechanism covers. It is the honest measure of how good your mechanism is not.
- A **latent fault** cannot violate the safety goal on its own, and is neither detected by a mechanism nor perceived [6]. It waits, silently, for a second fault to arrive and combine with it. A dead comparator in a lockstep pair is the canonical example: nothing goes wrong until the day the two cores disagree and nobody is listening.
- A **multi-point fault** is a combination that violates the safety goal, and is classified by whether the mechanism detects it, whether it is merely perceived, or whether it stays latent [6].

These classes feed the architectural metrics: the **single-point fault metric** (SPFM), the **latent fault metric** (LFM), and the probabilistic metric for random hardware

failures (PMHF). Each carries a threshold per integrity level, tabulated in ISO 26262-5. The values reported for ASIL D are SPFM at or above 99 percent, LFM at or above 90 percent, and PMHF below 10 FIT; for ASIL C, 97 and 80 percent; for ASIL B, 90 and 60 percent, with PMHF below 100 FIT at both of those levels <sup>[4]</sup> — figures taken from that paper, not read off the standard, which is not in this book's corpus. The standard also specifies the formulas behind those metrics, where the fraction of safe faults is established by structural analysis and formal proof, while the diagnostic coverage terms — the ones that make or break the metric — are measured by fault injection <sup>[6]</sup>.

### The unit trap: faults are counted, metrics are weighted

Here is the item that trips up every engineer arriving from functional verification, and that nothing in the toolflow warns you about. Your campaign counts faults: it injects a number of them and reports how many were detected, and the temptation to divide one by the other and call the quotient your diagnostic coverage is overwhelming.

The architectural metrics are not fault-count ratios. They are ratios of **failure rates**. Each failure mode of each design element carries a base failure rate computed from design data and technology data, and the metric formulas combine those rates — the safe fraction, the residual fraction, the multi-point fractions — into a single number for the element and then for the part <sup>[6]</sup>. Your campaign's detected fraction is an input applied to a weighted quantity, not the quantity itself.

The practical consequence: two campaigns with identical fault counts can produce different metrics, and a fault list that over-samples a large but low-rate structure while under-sampling a small critical one misleads you in a direction no coverage report shows. Agree the fault list with the safety engineer *per failure mode*, and keep the per-mode results separate all the way to the report. A single aggregate percentage across a whole block cannot be composed with anything.

**Scenario.** The brake controller MCU is small enough to enumerate honestly: a lockstep core pair, its ECC-protected memories, a watchdog, and a safety island holding the comparator and the alarm aggregation. Its FMEDA fits in one spreadsheet, and every failure mode maps to a mechanism a verification engineer can name. **Approach.** The verification lead walks the FMEDA row by row with the safety engineer *before* writing any injection infrastructure, and produces one artifact per row: where faults are injected, which signal is the observation point (where a violation of the safety goal becomes visible), and which signal is the detection point (where the mechanism raises its alarm). **Why.** On a part this size the walk takes two days and removes almost all the late surprises. On the flagship-A the same walk is a multi-week activity across a dozen sub-teams, and it is the reason large safety programmes staff a safety architect whose full-time job is keeping the FMEDA and the campaign definition in agreement. The technique does not change with scale; the coordination cost does, and it dominates.

## 22.3 Fault injection and fault simulation

The mechanics state in a paragraph, which is part of why people underestimate the activity. You make a second copy of the design with one hypothetical fault inserted — the *faulty machine*, against the unmodified *good machine* — and run a workload on both. At designated strobe times you compare designated signals, and if a signal differs between the two runs, the fault has reached it <sup>[6]</sup>.

Two kinds of signal are designated, and confusing them is a common beginner error:

- **Observation points** are where the failure becomes visible to the outside world — the data output that would now be wrong. A fault seen here is *observed*.
- **Detection points** are where the safety mechanism raises its alarm — the error flag, the comparator mismatch, the interrupt to the safety island. A fault seen here is *detected*.

In the published ECC campaign this chapter draws on, the observation point is the read-data port and the detection points are the single-error, multi-error and any-error signals on the block's boundary <sup>[4]</sup>. That choice is not incidental bookkeeping; it is the definition of what you are measuring.

One distinction before going on, because this book already has a neighbouring term. **Error injection**, as Chapter 2 introduces it and Chapters 3, 8 and 10 practise it, drives erroneous *stimulus* at an interface to check that the design detects and recovers. **Fault injection** corrupts the design's own *internal state* — a net, a flip-flop — to model a physical defect no interface can produce. The two share infrastructure and answer different questions, and a safety assessor means only the second.

### The four outcomes, and the one that matters

Cross the two axes and you get four verdicts. A fault neither observed nor detected did nothing visible in this run. A fault observed and detected is one the mechanism caught: it corrupted something, and the alarm went off. A fault detected but not observed triggered the alarm without corrupting anything — not dangerous, if noisy. And a fault **observed but not detected** corrupted the design's output while the mechanism stayed silent <sup>[4]</sup>. That last class is the dangerous one, and it is the whole point of the campaign.

That brings us to the distinction that separates a competent safety engineer from a dangerous one. **A fault that is not detected is not the same as a fault that is not dangerous**. A fault that was never observed in your campaign has two possible explanations, and your campaign cannot tell them apart: either it is genuinely safe — it cannot affect the safety goal under any stimulus — or it is dangerous and your workload simply failed to excite it <sup>[6]</sup>. Both look identical in the results database. Both are a row that says “not observed”.

Which explanation you choose is the largest single lever on your reported metrics, and the place where fault campaigns become dishonest: classifying every un-excited fault as safe will get you to any number you like, and an assessor will ask for exactly the argument you did not make.

The honest positions are two, distinguished by *which question the evidence answers*:

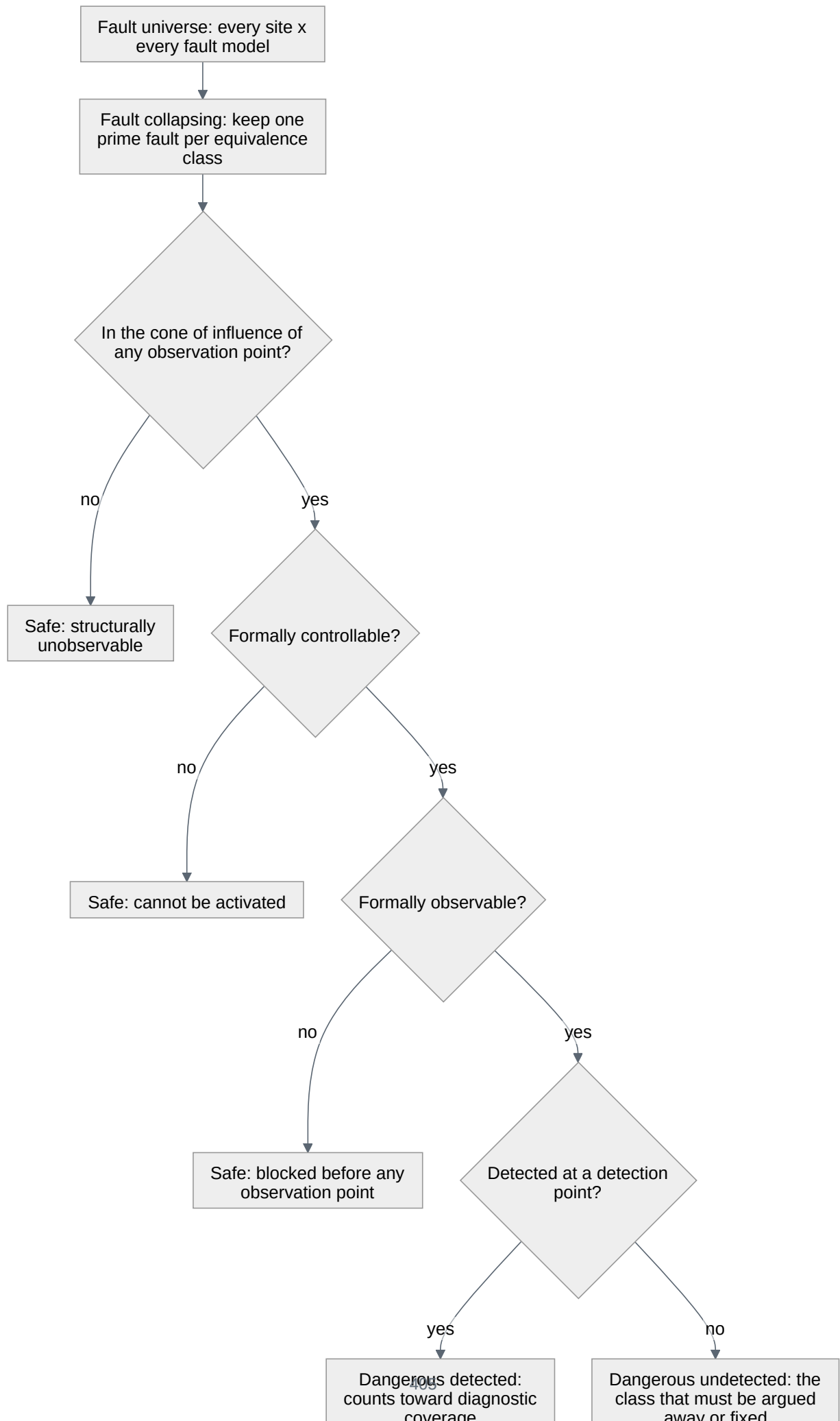
- **Structural and formal arguments answer “under any stimulus”.** A fault outside the cone of influence of every observation point cannot reach an observation point under any workload whatsoever — that is a property of the netlist, established statically, and it scales from block to system <sup>[4]</sup>. Likewise, a fault that a proof engine shows can never cause a difference at the observation point is untestable by construction, not merely unstimulated <sup>[6]</sup>. These are real safe faults and they may be excluded without apology.
- **Simulation answers “under this stimulus”.** A fault your workload did not excite is a fault about which you have learned nothing. It is not evidence of safety; it is absence of evidence, and it must be resolved by better stimulus, by proof, or by a recorded engineering judgment with a name attached.

Keep those two apart every time you prune, and the pruning is defensible. Blur them and the campaign is theatre.

### Why the fault list is enormous, and how it is honestly reduced

Every net, port and sequential element is a fault site, and each site carries at least two permanent fault models — held at zero and held at one — plus transient ones. Permanent defects are conventionally injected as stuck-at faults, transient upsets as a single event upset: a bit flipped once and then left to propagate or die <sup>[3]</sup>. On a large block the raw count runs to millions.

The reduction sequence is standard, and each step answers a specific question:





**Figure 22.1** — The reduction sequence that takes a fault list from every site crossed with every fault model down to the faults a campaign actually simulates: collapsing, then structural pruning, then formal controllability and observability analysis. Each diamond is a question whose answer holds under any stimulus, and the order is deliberate — cheap arguments first, so the expensive engine only sees what the cheap ones could not settle.

**Collapsing** comes first: faults are sorted into equivalence classes, a *prime* fault represents each class, a *collapsed* fault is one producing the same observable behaviour as its prime, and only primes are simulated [6]. **Structural pruning** follows, removing faults outside the cone of influence, and then formal controllability and observability analysis removes faults that cannot be activated or cannot propagate. What survives is simulated.

The published ECC campaign gives the shape concretely. A design with faults at ports, flip-flops and wires starts from 3,606 faults; collapsing removes 973 and testability analysis another 144, leaving 2,489 to analyse. Structural cone-of-influence analysis marks 763 of those unobservable, leaving 1,726. Formal observability analysis then clears a further 248 — 232 in one phase and 16 in a second — leaving 1,478 that genuinely reach the read-data port [4].

Nothing in that chain from 3,606 to 1,478 is a guess. A collapsed fault is dropped because an equivalent prime fault stays in the list to represent it; every other fault is dropped by an argument that holds under any stimulus whatsoever. That is what makes the reduction defensible, and it is why the order matters: cheap arguments first, so the expensive engine only sees what the cheap ones could not settle.

### The plateau, and what to do at it

Simulation-based campaigns follow a characteristic curve. Progress is close to linear at first, then flattens, and eventually additional runs classify almost nothing. At that point you are left with a residue of faults that were never observed, and the answer is not “assume they are safe” — it is “analyse them”. Doing that by hand is the activity nobody budgets for: reviewing every not-observed fault to establish that it went unseen because of the design’s structure and not because of the environment’s incompleteness, at a cost measured in engineer-days to engineer-weeks [4].

This is where formal earns its place in a safety flow, and the argument is precisely the one made above: a proof answers “under any stimulus”, so it can convert a not-observed fault into a proven-safe fault, which simulation can never do. The published campaign runs the whole classification formally on a block small enough to take it — and the block was only *almost* small enough. The 8K memory on its periphery would not converge in twelve hours until the memory was black-boxed with its input ports promoted to observation points, and then abstracted to a pair of back-to-back flip-flops under a constraint fixing the read/write ordering [4].

Note the direction of that abstraction’s error, because it is what makes it usable: promoting the memory inputs to observation points can only make a fault look *more* observable than it is, so a fault the abstraction calls safe (the safe-fault class of ISO 26262-5:2018) is genuinely safe [4]. An abstraction whose error runs the other way — one that can call a dangerous fault safe — is not an abstraction, it is a bug in your

evidence. Chapter 14’s discipline on sound abstraction applies here with an auditor attached.

The campaign’s final classification, over the 1,726 faults that entered detection analysis: 225 neither observed nor detected, 23 detected without being observed, 966 both observed and detected — and 512 observed but not detected, faults that corrupt read data while the ECC error signals stay quiet [4]. Those 512 are the result. Everything else in the campaign existed to isolate them. The block behind those numbers is real and named: an NXP Semiconductors ECC block with single-error correction and double-error detection, wrapped around an 8K memory [4]. The 512 belong to that block, and a different memory geometry behind a different mechanism would produce a different number.

**Scenario.** The radar front-end is a fixed-point DSP chain — FFT pipelines feeding a constant-false-alarm-rate detector — and it is the sensing front of an automotive safety function. The naive campaign injects stuck-at faults across the FFT datapath, strobes the detection list at the output, and reports a discouraging diagnostic coverage: a great many faults change the output and no mechanism says a word. **Approach.** Look at what “changes the output” means here. A single-bit fault deep in a twiddle-factor multiplier perturbs a bin’s magnitude by a fraction of a least-significant bit. The detector’s threshold sits far above that perturbation, so the detection list comes out identical: the fault changes the raw spectrum and changes nothing at the interface the safety requirement is written against. The observation point belongs at that interface — the detection list handed to the fusion stage — not at the spectrum. **Why.** The observation point is a claim about where “violating the safety goal” happens, and it is the highest-leverage decision in the whole campaign. Setting it at the boundary the requirement names is correct; moving it outward afterwards because the numbers were disappointing is the classic dishonest prune, and the difference between the two is a written rationale, agreed before the campaign runs and countersigned by the safety engineer. Note also what this does *not* license. A perturbation that hides under the threshold on one frame need not hide on the next, so the margin argument must hold across the operating envelope. That makes it a stimulus obligation, not a one-line exclusion — and while the numerical and mixed-signal side of the radar chain is Chapter 21’s subject, the safety argument built on top of it belongs here.

### Time is part of the verdict

One more axis, easy to miss because ordinary functional verification has no equivalent. A safety mechanism must detect and react *within a budgeted time interval measured from the moment the fault occurs*, and must be running continually while the device operates; the fault-tolerant time interval and the fault reaction time interval are both things a fault injection campaign exists to establish [3]. So a campaign does not only classify detected against undetected: it classifies detected-in-time against detected-late, and a late detection is not partial credit. It is an undetected fault with a misleading log message.

This has a concrete implication for how you run the campaign. The simulation cannot stop at the first difference; it has to keep running long enough to see whether the alarm arrives inside the interval, which is why fault simulators expose a separate time budget for what happens after the first divergence [3]. Set that budget from the FMEDA's timing requirement, not from what makes the run finish quickly.

## 22.4 Safety mechanisms and their verification

The mechanisms themselves come from a short list, and none of them is exotic. A failure analysis will typically name error-correcting code, a dual-core lockstep pair with a comparator, a software test library, or hardware redundancy with a majority voter [6]; add the watchdog and built-in self-test run at start-up or periodically, and you have most of what you will meet.

Two vocabulary notes before going further, because both words are already in use in this book with different meanings. **Hardware redundancy** here means duplicated or triplicated *silicon* — two cores computing the same thing, three copies of a register voted bit by bit. That is not the sense of Chapter 2, where redundancy is a second engineer's independent reading of the specification. And **lockstep** here is the hardware pair glossed above — not Chapter 3's sense, where an instruction-set simulator runs in lockstep with the RTL as a reference model. Same word, different mechanisms, and mixing them produces sentences that sound right and mean nothing.

They share one property teams consistently underestimate: **a safety mechanism is a design, and it has bugs**. It was written by the same people, under the same schedule, from the same specification as the logic it protects. It gets less verification attention than the functional path because it does nothing in the nominal case, and it integrates last because it depends on everything else. A safety mechanism is therefore, structurally, the least-verified block on the part — and the one whose failure the metrics assume away.

So verify each mechanism against three questions, not one.

**Does it fire when it should?** This is the fault campaign — the question everybody asks, and the easiest of the three, because the campaign infrastructure answers it as a by-product.

**Does it fire in time?** Covered above: the budgeted interval is part of the requirement, and a mechanism that reports a double-bit error long after the corrupted data has been consumed has not detected anything useful.

**Does it stay silent when nothing is wrong?** This is the question teams forget, and it is a first-class verification obligation. A safety mechanism with false positives is a reliability defect, and often a worse one than the fault it guards against, because it fires on healthy parts in the field at a rate the fault never would. A mechanism that trips spuriously does not degrade gracefully: it takes the system to a safe state, which in a vehicle means a warning lamp, a limp mode, or a dealership visit. That is a product returned for a defect the part does not have, and it is the direct consequence of verification that only ever asked the first question.

The asymmetry explains why the second half gets skipped. Proving a mechanism fires means injecting a fault — deliberate, scripted, easy to demonstrate. Proving it stays silent means running the design’s *entire legal operating space* with the alarm never asserting, an obligation with no natural stopping point. It is the shape of any must-not-happen requirement (Chapter 5) and is discharged the same way: an always-on assertion in every regression run, plus a demonstration that the assertion can actually fail.

**Scenario.** The sensor hub is an always-on part: a 32 kHz island and a DSP, fusing accelerometer and gyroscope streams while the system it serves sleeps. In its safety derivative it also carries a clock monitor — logic that watches the system clock against the always-on reference and raises an alarm if that clock stops, slows or runs fast, on the theory that a stopped clock is the one failure the rest of the chip cannot report about itself. **Approach.** The fault campaign is straightforward: kill the clock, halve it, double it, check the alarm. The verification that matters is the other side. The domain being watched legitimately changes frequency on a low-power transition, legitimately stops during power-down, and legitimately restarts with a ramp — and every one of those is a healthy event that looks exactly like the failure the monitor exists to catch. The obligation is a cross of every legal power and clock transition against the monitor’s alarm, with the alarm asserted as an error. **Why.** The failure mode this catches is not a missing detection. It is a mechanism that reports a fault on a working part every time the system enters its low-power state — and it is the hardest kind of bug to find late, because it does not reproduce on a bench that never exercises the low-power sequences. Chapter 19’s power-state work and this cross are the same stimulus written for two purposes, which is a good argument for building it once and sampling it twice.

**Scenario.** The GPU renders the instrument cluster, and the cluster displays tell-tales — the warning symbols whose absence is itself a hazard. The safety mechanism is frozen-frame detection: logic that watches the display pipeline and raises an alarm if the rendered content stops changing, on the theory that a hung GPU showing a stale frame is indistinguishable, to the driver, from a working one showing good news. **Approach.** Firing correctly is easy to verify: stall the pipeline and watch the alarm. Not firing is the hard half, because *a correct instrument cluster is legitimately static most of the time*. A parked vehicle with no faults displays an unchanging image for as long as the ignition is on. The mechanism cannot be a simple frame-comparison; it needs an injected, known-varying region — a counter or a pattern the renderer is required to update every frame — so that “unchanged” becomes an unambiguous claim about the *pipeline* rather than an ambiguous claim

about the *content*. **Why.** This is the general pattern behind every false-positive problem in this section: the alarm condition must be a proposition that is false exactly when the design is healthy, never a heuristic that happens to correlate with health. When you find yourself verifying a heuristic, the finding belongs back with the architect — early.

### A worked artifact: the non-firing obligation

Here is the assertion form for the third question, on the flagship-A's ECC path. The mechanism corrects single-bit errors and flags them for the error counters, and reports double-bit errors to the safety island as a fault. The obligation is that neither flag asserts while nothing is wrong.

```
// In the checker bound to the ECC wrapper on the flagship-A's memory.
// inject_active is the campaign's own control: high only while a fault
// is deliberately present. Every other cycle is a claim about health.
property p_no_spurious_ecc_alarm;
 @(posedge clk) disable iff (!rst_n)
 !inject_active |-> !(ecc_err_single || ecc_err_double);
endproperty
a_no_spurious_ecc_alarm : assert property (p_no_spurious_ecc_alarm);

// The mechanism must also be alive, not merely quiet: a mechanism that
// can no longer fire is a latent fault, and silence is what it looks like.
c_ecc_double_reported : cover property (
 @(posedge clk) disable iff (!rst_n)
 inject_active && ecc_err_double
);
```

Two things about this pair. The assertion is worthless without the cover: an ECC wrapper whose error outputs never assert — a design bug tied them off, or a fault killed the mechanism — passes `a_no_spurious_ecc_alarm` on every run of every test, forever. Silence is exactly what a dead mechanism looks like, and a dead mechanism is the latent fault of the classification above. The cover makes the silence mean something: it is Chapter 8's known-bad-variant argument, and Chapter 6's point about vacuous passes, applied to a mechanism instead of a checker.

And `inject_active` is doing real work. Without that qualifier the assertion fires on every fault campaign run, so the team disables it during campaigns, so it protects nothing during exactly the runs where the ECC path is most exercised. Gate it on the injection control and it holds in campaign runs too — everywhere the campaign is not currently corrupting something.

### Verifying a lockstep pair

Lockstep deserves its own paragraph because its verification is not what it appears to be. Two cores, one comparator: the obvious test injects a fault into one core and checks that the comparator flags a mismatch. That test passes on a design with a serious defect.

What makes a lockstep pair effective is that the two cores fail *independently*. On the flagship-A the pair is delayed: the shadow core runs some cycles behind, so a disturbance hitting both at the same instant hits them at different points in their execution and produces a mismatch instead of two identical wrong answers. That delay, and the separation behind it, is the mechanism. So the questions are: does the delay pipeline preserve the comparison across every stall, flush and interrupt; do the two cores share any resource — a clock buffer, a reset synchroniser, a power rail, a memory port — whose single failure would corrupt both identically; and does the comparator's own failure get caught by something else? The third is the latent-fault question again, and its usual answer is a periodic self-test that provokes a mismatch on purpose and confirms the alarm.

None of that is found by injecting a fault into one core. It is found by asking what a *single* physical event could do to *both* halves at once — the analysis of dependent failures, a discipline of its own that a verification engineer contributes to rather than owns. What you owe it is the list of shared resources your netlist actually contains, which nobody else can produce.

## 22.5 Tool qualification and the evidence trail

---

A certification body cares which simulator you used for a reason that is easy to state and unwelcome to think about: **your evidence is tool output**. The coverage database, the fault classification, the proof result — every artifact supporting the safety case was produced by software that could, in principle, be wrong. If the tool is wrong the evidence is wrong, and the argument built on it is unsound while looking exactly as sound as before.

Tool qualification is the response (ISO 26262-8:2018, *Road vehicles — Functional safety — Part 8: Supporting processes*). Stripped of vocabulary, it is an argument with two halves: how much damage could this tool do if it malfunctioned — introduce an error into the safety-related work product, or fail to detect one that is there — and how likely is such a malfunction to be caught by something else in your flow? The more damage and the less independent checking, the more evidence the tool needs behind it, and that evidence comes from the vendor as a qualification package, from your own confirmation testing, or from a demonstrated history of use.

What matters operationally is a property of that argument that surprises people the first time: **qualification is scoped** — to a tool *version*, to a set of *use cases*, and often to a *configuration* (ISO 26262-8:2018). It is not a badge the product wears. Three consequences follow, and they are the ones that change a verification engineer's week.

- **Version pinning becomes a safety obligation.** The exact tool build is part of the evidence, not part of the environment. Chapter 12's run manifest — seed, design revision, testbench revision, exact tool build, configuration, host — stops being good practice and becomes the atom of the evidence trail.



- **A mid-project upgrade is a re-argument, not an IT ticket.** When the vendor ships a fix you need, the question is not whether it installs but what evidence produced under the old version must be re-established. Teams that have not thought this through discover it in month nine.
- **Using a qualified tool outside its qualified use case buys nothing.** A simulator qualified for the use case you are exercising is evidence; the same simulator driving a flow the qualification never contemplated is a tool you happen to trust.

Where qualification is unavailable or too narrow, the fallback is diversity in the flow — a second, independent check of the same property by a different engine, so that a tool error would have to occur twice and identically to survive. That is also, not coincidentally, good verification practice.

### Qualifying the flow, not just the tool

There is a second and quite separate question, and it is easy to conflate with the first: is your *verification environment* capable of finding the bugs it claims to look for? A safety assessment asks for evidence that the flows themselves are robust, and the technique that supplies it is to inject systematic faults into the design deliberately — at architecture, system and register-transfer level — and measure verification completeness from what the environment detects <sup>[6]</sup>.

This is the industrial form of the known-bad variant from Chapter 8: instead of one deliberately broken copy of the design kept in the repository, a systematic population of small mutations, each run against the environment, each classified as detected or not. The undetected ones are the interesting output — they are places where a real bug of the same shape would also have gone unnoticed, and they point at a missing checker or an unsampled signal rather than at a missing test.

Keep the two apart when you write them down. Tool qualification asks whether the *software you ran* can be trusted; flow qualification asks whether the *environment you built* can fail. An assessor will ask about both, and answering one with the other is a recognisable and unconvincing move.

### The chain that must survive the audit

The traceability chain a safety project is asked for runs further in both directions than the one Chapter 5 builds, and it must remain navigable years after everyone has moved on:

vehicle-level hazard → safety goal → safety requirement → the design feature that implements it → the failure modes that feature can exhibit → the safety mechanism that covers them → the plan row that discharges the mechanism's verification → the specific runs that produced the evidence → the archived results → the report that quotes them.

That chain crosses three parts of the series: Part 3, which holds the concept phase and the hazard analysis that assigns the ASIL; ISO 26262-4:2018, *Road vehicles — Functional safety — Part 4: Product development at the system level*; and ISO 26262-5:2018 at the hardware level.

Two joints break most often. The first is between the failure mode and the plan row: the FMEDA and the verification plan are maintained by different people in different tools, and they drift the moment the design changes. The second is between the report and the archived results, because reports are written from summaries and summaries outlive the raw data. Guard both by making the plan row carry the failure-mode identifier as a field, and by refusing to let a report quote any number that is not reproducible from a retained manifest.

Here is what a plan row with a certification obligation looks like in this book's five-column format, on the flagship-A's ECC-protected memory:

**Table 22.1** — Three plan rows, in this book's five-column format, for one safety mechanism on the flagship-A's ECC-protected memory: that the mechanism fires within its budgeted interval, that it raises no alarm on healthy traffic, and that the classification's residue is argued rather than assumed. Three rows because there are three obligations, each of which an assessor will ask to see discharged on its own.

ID	Feature & attributes	Pass/fail criterion	Method	Closure metric
FA-SM04a	SECCDED ECC on the flagship-A's tightly-coupled memory detects double-bit errors. FMEDA failure mode TCDM-FM03. Attributes: error type {single-bit, double-bit, address decode} × access {read, write} × requester {cluster core, DMA} × concurrent bank activity {idle, contended}	Every injected double-bit error raises <code>ecc_err_double</code> to the safety island within the fault reaction time budgeted for TCDM-FM03	Fault injection campaign, permanent and transient models, on the gate-level netlist	Fault classification report for TCDM-FM03 complete: every fault in a class, dangerous-undetected list itemised and dispositioned
FA-SM04b	The mechanism raises no alarm on healthy traffic, including low-power entry and exit	<code>a_no_spurious_ecc_alarm</code> holds in every run in which injection is inactive	Constrained-random sim (always-on check) + power-aware sim	Check enabled and passing in 100% of regression runs
FA-SM04c	The classification's residue is argued, not assumed	Every fault left not observed carries either a structural or formal argument that it cannot violate the safety goal, or a named engineering judgment	Formal observability analysis, then review	Zero faults in state not observed, unargued at sign-off

Three rows because there are three obligations, and the one-row-one-metric rule of Chapter 5 does not relax for safety — if anything it tightens, since each row is the thing an assessor will ask to see discharged on its own. The cover directive from the artifact above needs no row of its own: row FA-SM04a's campaign hits it the first time an injected double-bit error is detected, and a campaign in which it is never hit has already failed that row. And note what row FA-SM04c is — the honest name for the plateau. It does not promise that every fault is safe. It promises that no fault is *silently* assumed safe.

And when a hole survives — because it always does — it goes into a waiver with the same five fields this book has used since Chapter 7: the hole, the reason, the risk argument, an owner, an expiry. A safety project changes one thing about that record, and it is not a field: who signs it. The safety engineer, not the verification lead, owns the residual risk — the verification lead owns the accuracy of the statement, the safety engineer owns the decision to accept it.

### Where this is engineering, and where it is not

Both halves of this deserve saying out loud, because engineers on safety programmes say them to each other constantly and rarely in writing.

Genuine engineering, unambiguously: the fault campaign, the observation-point and detection-point definitions, the abstraction that makes a formal classification converge, the false-positive verification of every mechanism, the dependent-failure analysis of a lockstep pair, and the argument separating provably-safe faults from merely-unstimulated ones. This work finds real defects that ordinary functional verification does not look for, and the discipline it imposes — say precisely what you measured and against what — makes teams better at everything else. The best-run safety programmes are visibly better-run projects.

Documentation that engineers resent, also unambiguously: re-typing the same information into a third tool because the traceability database will not import from the plan; review records whose value is the signature and not the review; templates with sections that do not apply and cannot be deleted; the periodic re-issue of documents whose content has not changed. None of this makes anything safer. It makes the evidence *auditable*, which is a real requirement — but it is a documentation requirement being met by engineers because nobody has automated it.

Keep the distinction sharp, for a practical reason. Teams that resent all of it — the fault campaign included — do bad safety work; teams that pretend all of it is engineering burn out. The productive stance is to defend the first list fiercely, automate the second relentlessly, and say which is which when the schedule slips. The highest-leverage investment on a safety programme is generating the traceability artifacts from the plan and the run database instead of maintaining them by hand — a tooling project, not a safety activity, which is exactly why it never gets funded until the second programme.

---

### IN PRACTICE

Real programmes are less tidy than the sections above. A few things teams do:

**They start the campaign infrastructure before the FMEDA is final.** The injection harness, the strobe definitions, the good-machine reference run and the results database take weeks to build and barely depend on which faults you eventually inject. Waiting for a signed FMEDA is how campaigns end up on the critical path.

**They run the campaign on the netlist, and pay for it.** Fault models are defined over physical structures, so the gate-level campaign is the one that matches the fail-

ure-rate data — and it is far slower than register-transfer simulation and lands late. Most teams run an early register-transfer campaign to shake out the mechanism logic and find the architectural surprises, then repeat on the netlist for the evidence. Budget for two campaigns, not one.

**They negotiate the workload.** Diagnostic coverage is measured under a workload, and one that leaves half the design idle classifies half the design's faults as not observed. Choosing it — long enough to activate the mechanisms, short enough that thousands of faulty-machine runs fit the compute budget — is a real engineering decision with a written rationale, and one of the first things an assessor reads.

**They keep the campaign out of the nightly regression.** A fault campaign is a campaign in Chapter 12's sense — launched for a stated reason, owned, producing a written result, then stopping. The nightly suite carries the mechanisms' always-on checks instead.

---

## PITFALLS

1. **Counting faults where the metric weighs failure rates.** *Symptom:* one aggregate diagnostic-coverage percentage for a whole block, which the safety engineer cannot use. *Cure:* classify and report per failure mode and let the FMEDA apply the base failure rates; never compute an architectural metric by dividing fault counts.
2. **Un-excited faults recorded as safe.** *Symptom:* diagnostic coverage that improved sharply in the last two weeks with no change to the design or the workload. *Cure:* a fault is safe only when a structural or formal argument shows it cannot violate the safety goal under any stimulus; everything else is unresolved and needs a name on it.
3. **Verifying only that the mechanism fires.** *Symptom:* a green campaign, and field returns from healthy parts. *Cure:* an always-on assertion that the alarm stays silent outside injection, exercised across every legal power, clock and reset transition, plus a cover proving the alarm can still fire.
4. **A mechanism that has silently died.** *Symptom:* the non-firing assertion passes on every run for a year. *Cure:* pair every silence assertion with a cover directive on the alarm's positive case, and run the mechanism's own self-test in the gating tier.
5. **Moving the observation point to improve the number.** *Symptom:* a classification that changed after someone "corrected the strobe list". *Cure:* the observation point is a claim about where the safety goal is violated — set it from the requirement, have the safety engineer countersign it, and change it only with a written rationale.
6. **Tool versions treated as environment rather than evidence.** *Symptom:* nobody can say which build produced last quarter's fault report. *Cure:* the run manifest is the unit of evidence; pin the build, archive the manifest with the results, and treat an upgrade as a change to a controlled work product.

---

## SUMMARY

- Safety verification does not replace functional verification; it adds a second target. Design bugs are systematic faults and always permanent; random hardware faults are physical defects arising in manufacturing or operation, permanent or transient, and cannot be verified away — only detected and handled [6].
- The FMEDA is a verification input, not a document to file: it fixes your failure modes, injection points, alarms and the diagnostic coverage you must demonstrate. Read it early enough that its granularity is still negotiable.
- Fault classification, not fault count, is the deliverable: safe, single-point, residual, latent and multi-point are defined classes with per-level metric thresholds — SPFM at or above 99 percent and LFM at or above 90 percent for the most demanding level [4].
- “Not observed” is not a verdict. Structural and formal arguments answer *under any stimulus* and license exclusion; simulation answers *under this stimulus* and licenses nothing. The residue left when the campaign plateaus is analysed, argued or fixed.
- A safety mechanism is the least-verified block on the part and the one the metrics assume works. Verify that it fires, that it fires in time, and — the half that gets skipped — that it stays silent on a healthy design.
- Your evidence is tool output, so the tool is part of the evidence. Qualification is scoped to a version and a use case, making version pinning a safety obligation and a mid-project upgrade a re-argument.
- The argument is the deliverable. Chapter 7’s sign-off asked whether the evidence justified shipping; a safety project asks you to write that reasoning down, hand it to someone who was not there, and have it hold up years later.

## FURTHER READING

**From the corpus.** [6] is the best single overview in the corpus of how functional safety verification is structured around ISO 26262: the systematic-versus-random split, the FMEDA process and its columns, the fault classification tree, the metric formulas and their failure-rate weighting, the seven-step fault campaign flow, and — on the evidence side — mutation-based qualification of the verification flow itself for Part 8 and 9 assessments. [4] is the case study this chapter's numbers come from: a formal-assisted fault campaign on a single-error-correcting, double-error-detecting block, with the full reduction chain from the raw fault list to the dangerous-undetected residue, and a candid account of the manual-analysis cost that motivates the approach. [3] covers the fault-injection side in more detail: permanent and transient fault models, failure and detection strobes, static pruning by cone of influence, the mapping from analysis statuses to the standard's fault classes, and the timing dimension of detection. The industry adoption and effort figures are from the 2024 Wilson Research Group studies [1, 2]. Chapter 14 covers the abstraction discipline formal fault classification depends on; Chapter 12 covers the run manifest the evidence trail is built from; Chapter 21 takes up the analog and mixed-signal side of the radar chain.

**Not in the corpus.** The standards themselves are the primary references and should be read directly rather than paraphrased: the ISO 26262 series, *Road vehicles — Functional safety*, particularly Part 3 on the concept phase, where the hazard analysis and risk assessment that assigns an ASIL is defined; ISO 26262-5:2018, *Product development at the hardware level*, which carries the architectural metrics and their target values; ISO 26262-8:2018 on supporting processes including tool qualification; ISO 26262-9:2018 on integrity-level-oriented analyses; and ISO 26262-11:2018 on the application of the standard to semiconductors. Those descriptions are of each part's scope; only Part 5's title is quoted. For airborne electronic hardware, RTCA DO-254 / EUROCAE ED-80, *Design Assurance Guidance for Airborne Electronic Hardware*. The IEC 61508 series is the general industrial functional-safety standard from which several domain standards derive, and the one most IC/ASIC projects reported working to in the 2024 survey [1].

## REFERENCES

1. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
2. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
3. **D2** Sesha Sai Kumar C V, A. Mouallem, and J. Mazzawi, "Fault Injection Analysis for Automotive Safety and Security," Proc. DVCon Europe, 2022.
4. **D12** N. Ahuja, M. Agarwal, and S. Jana, "Formal Assisted Fault Campaign for ISO26262 Certification," Proc. DVCon India, 2019.
5. **D24** S. Mitra, E. Singh, and K. Devarajegowda, "QED & Symbolic QED: Pre-silicon Verification, Post-silicon Validation, Industrial Results," Proc. DVCon Europe, 2019.



6. **D32** J. Richter, "Unified Functional Safety Verification Platform for ISO 26262-Compliant Automotive," Proc. DVCon Europe, 2019.

---

## 23. Security Verification

The flagship's root of trust is signed off twice, and the second review is the one this chapter is about.

The first review is functional, and it goes well. The inline AES-GCM engine on the memory path has a reference model behind it and a scoreboard that has compared ciphertext beat by beat for nine weeks. Every plan row is discharged. Chapter 15's connectivity job has run at every integration build, positive rows and negative ones together, and it proves what the security requirement asked for: no structural path from any key-ladder output to a bus-readable address, to the debug data register, or to a scan-observable point. The bug curve is flat and the fresh-seed yield is near zero. By every instrument Chapter 7 gave the team, this block is done.

The second review is a security review, and it opens with a question the verification plan does not contain: *what would an attacker want from this block, what are they able to do, and where do those two meet?* Within an hour the room has produced obligations with no rows behind them. Nothing in the plan says what the engine's completion latency may and may not depend on. Nothing says what becomes of a working key when the key ladder's domain goes down in the middle of an operation. Nothing says whether the debug port — once opened by the authenticated unlock path that everyone trusts — can reach the key ladder, as opposed to merely not being *wired* to it. And nothing anywhere in the plan names an adversary, which is why none of those questions had a reason to be asked.

None of that is a failure of the functional plan. The functional plan answered its own question completely: the engine does what the specification says it does. The security question is a different question — can the engine be made to do something the specification never mentioned? — and no quantity of the first kind of evidence accumulates into the second.

The gap can be very wide. A published information-flow study took an AES encryption block from a public repository, unmodified and fully functional, encrypting correctly across thousands of test vectors, and checked one rule against it: the key must not flow to the data output. The rule fails. The key, exclusive-ORed with the data, reaches the output pin as plaintext <sup>[1]</sup>. Every functional test that block will ever run continues to pass.

---

### CHAPTER MAP

- §23.1 establishes why an adversary, rather than a specification, sets the scope — and why security cannot be closed the way functional coverage is closed.
- §23.2 writes the threat model as a versioned engineering artifact — assets, adversary capabilities, trust boundaries — and turns its rows into checkable claims.

- §23.3 establishes why confidentiality and integrity are properties over *information flow* rather than over values, why formal is the right shape of answer for them, and where that shape stops fitting.
- §23.4 states what an information-flow argument does and does not establish, and why its exception clauses carry the whole weight of the result.
- §23.5 establishes why side channels live below the abstraction RTL verification works at and what pre-silicon evidence can honestly claim about them; §23.6 states how to tell a security property that can be verified from one that can only be argued.

## 23.1 Why an adversary changes everything

---

Every technique in the preceding twenty-two chapters shares one premise: there is a specification, and the job is to establish that the design implements it. The plan enumerates features from the specification (Chapter 5). The coverage model partitions the specification’s input space (Chapter 6). The oracle encodes the specification’s expected response (Chapter 3). Even the negative work is specification-derived — a *must-not-happen* feature is a requirement the specification states, and an `illegal_bins` declaration is a rule someone wrote down.

Security verification has no such anchor. The property is not “the design does X”; it is “the design cannot be made to do Y”, where Y is chosen by someone who has read the specification, read the datasheet, and possibly read the RTL, and who is looking specifically for the behaviours nobody wrote down. Four consequences follow, and they are structural rather than a matter of degree.

**The properties are negative, and negative properties do not compose from positive evidence.** “This key never reaches that bus” is a claim about the complement of the design’s intended behaviour. You cannot establish it by demonstrating intended behaviour, however much of it you demonstrate — which is exactly what the opening AES result shows, and why a block can pass every vector and still fail the one rule that matters. Chapter 3’s distinction between the specification’s stated behaviour and everything else is usually a footnote about the oracle’s projection; here it is the entire subject.

**Rarity stops being a defence.** Risk-driven verification (Chapter 3) allocates effort by likelihood times cost, and likelihood is computed against a stimulus distribution the team controls: a constrained-random generator draws from a distribution, so a scenario needing four improbable conditions at once is genuinely improbable. An adversary is not a distribution. An adversary is an optimiser with a budget, who will spend a month arranging exactly the four conditions your generator would only ever have stumbled into — and who, having arranged them once, can arrange them on demand. The entire argument from unlikeliness, which is load-bearing for functional prioritisation, is void inside the threat model’s scope.

**The failure is often functionally silent.** A confidentiality breach need not corrupt any output, and an integrity violation may hold only briefly before something masks

it, so detecting one usually means correlating several kinds of evidence rather than watching for a wrong value <sup>[2]</sup>. Put that in Chapter 3’s vocabulary and it is precise: the bug is *activated* and it *propagates* — propagation to an observable point is not the detection mechanism here, it is the failure — but nothing *checks*, because no checker was ever written for a value that was correct.

**Coverage does not close.** There is no enumeration of “everything an attacker might try”, so there is no denominator, so there is no percentage. Functional metrics — line, toggle, FSM — correlate only loosely with an attacker’s search space, and the field has no settled, tool-friendly definition of security coverage; it is one of the most consistently reported gaps in the area <sup>[2]</sup>. This is not a temporary tooling shortfall: it is what happens when the space being covered is defined by an opponent rather than by a document.

### What this costs, stated plainly

A chapter that promised a security-coverage number would be lying, so here is the honest version. Security verification produces evidence for *named* claims against a *named* adversary. It does not produce a completeness argument. The output of the process is a threat model, a set of properties discharged against it, a set of properties *not* discharged with their reasons, and a residual-risk statement someone signs — the same shape as Chapter 7’s sign-off, with one difference that matters: the residual is not merely unquantified, it is unquantifiable, because you cannot count what you have not enumerated.

That is uncomfortable, and it is why the discipline leans so hard on structure: if you cannot bound the space, be explicit about which part you looked at, and make each look exhaustive within its scope. Every practice in this chapter follows from that trade.

It also explains why the obvious approaches under-deliver, each in its own way. Architecture review is manual and rarely reaches the implementation as built. Red-team exercises are essential for the finished product and find nothing before silicon exists, and their results are hard to measure. Formal methods belong inside a larger strategy and do not scale to a large design running hardware and software together. Directed tests take substantial effort to write and mostly re-confirm vulnerabilities already known <sup>[1]</sup>. None is a bad technique. Each simply has its strength somewhere other than where an adversary’s advantage lies, so a plan built on any one alone inherits precisely its blind spot.

### The one thing that transfers

Here is the encouraging part, and the reason to put a verification engineer with no security background on this work. Everything downstream of the threat model is verification: properties, assumptions, stimulus, coverage, waivers, sign-off. Once someone has written “the working key must not be observable at the debug port under adversary capability C3”, the question of how to establish that is a question this book has spent twenty-two chapters on. The specialist knowledge is concentrated almost entirely in one artifact.

So write that artifact first, and write it as an engineer.

## 23.2 The threat model as the plan's root

A threat model has three parts, and each is useless without the other two.

**Assets** are the things worth protecting. Asset identification comes first in account after account of the discipline, because it determines what must be protected before threats, properties and tests can be defined at all — and where it is left late, the fixes land in the far more expensive post-silicon stage [3]. On the flagship the obvious assets are cryptographic: the device root key burned into the OTP fuses, the working keys the ladder derives from it, the entropy the TRNG produces. The non-obvious ones matter more, because they are the ones a functional plan never lists. The boot measurement is an asset. The firewall and address-decode configuration that enforces every other protection is an asset — if an adversary can rewrite the enforcement, no other lock needs breaking. And an asset need not be secret at all: the flash controller's anti-rollback counter is public by design, and everything depends on nobody being able to *decrement* it, so what it needs is an integrity property, not a confidentiality one. Sorting assets by which of the two they need is the fastest way to stop a security plan collapsing into “encrypt more things”.

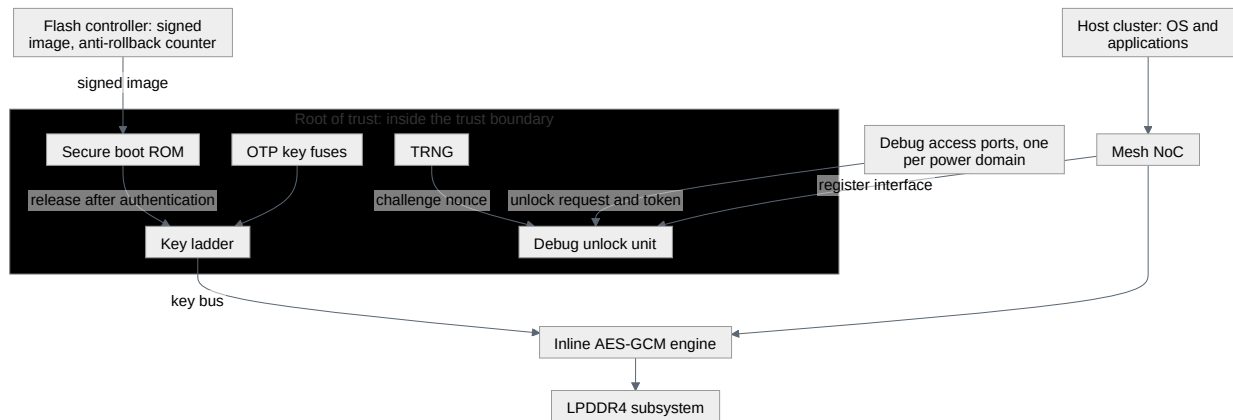
**Adversary capabilities** say what the opponent can do. Write capabilities, not attacks: attacks are an open set and capabilities are an enumerable one, and only an enumerable list can be closed over. Four tiers cover most SoCs, and the flagship's model uses them:

- **C1** — unprivileged software running on the host cluster: it can issue loads and stores, program what it is allowed to program, and time itself.
- **C2** — privileged software on the host cluster: everything in C1, plus configuration of the interconnect, the memory management unit, the power controller and the debug fabric.
- **C3** — physical access to the package pins: probing the debug interface, measuring supply current and electromagnetic emission, perturbing clock and supply.
- **C4** — invasive physical access, up to decapsulation and direct probing of the die.

The flagship declares C4 **out of scope**, and the reason is written down rather than assumed: countering it is a packaging, sensor and countermeasure-design problem whose evidence does not come from RTL verification, and pretending otherwise would put rows in a plan that no pre-silicon activity could ever discharge. An out-of-scope declaration with a reason is engineering. An out-of-scope declaration without one is where the next escape lives.

**Trust boundaries** are the surfaces where the capability tier changes. They are the reason the first two lists become checkable: a property is always “asset A does not cross boundary B under capability C”. Modern SoCs make this hard precisely because they stack heterogeneous IP, complex interconnect and several privilege boundaries into one attack surface that is large and awkward to analyse [3] — which

is an argument for drawing the boundaries explicitly, not for hoping they are obvious.



**Figure 23.1** — The flagship’s root of trust, asserting one thing: which blocks sit inside the trust boundary — fuses, key ladder, secure boot ROM, TRNG, debug unlock unit — while everything drawn outside it is reachable, in some mode, by an adversary at C2 or above. The inline AES-GCM engine is outside the box deliberately: it holds key material and sits on a path carrying untrusted traffic, and boxing such a block on one side would tell a lie about it.

The figure asserts one thing: which blocks sit inside the trust boundary. Everything drawn outside it is reachable, in some mode, by an adversary at C2 or above. The inline engine is deliberately outside the box and is the interesting case, because it is the one component that genuinely straddles: it holds key material from the ladder and it sits on a memory path that carries untrusted traffic and terminates in untrusted DRAM. A figure that boxes such a block on one side has already told a lie about it. Roots of trust of this shape are real and public in outline: a published diagram of the Rambus CryptoManager RT-360 shows a RISC-V CPU beside an OTP management core, a key transport core, a TRNG management core, an SRAM interface behind a memory protection unit, and AES, hash and public-key engines on a bus fabric — and the failure classes a security-verification vendor names against it are architectural rather than cryptographic: assets mismanaged during a secure boot, a memory protection unit misconfigured, keys or configuration registers not cleared, debug modes left enabled [1]. No measurement travels with that list and none is claimed for it here; what it buys is that those surfaces belong to a shipping part rather than to this book’s example.

## The rows

The three lists cross into a table, and the table is what the rest of the chapter discharges. Six rows, each naming an asset, a capability, a boundary and its obligation:

**Table 23.1** — The flagship’s threat model as rows: each crosses one asset with the adversary capability it must withstand and the boundary where that capability meets it, then states the obligation. Read the last column first. Three rows



name a single technique, two name a pair, and TM-06 names none — which is the table doing its job rather than a defect in it.

ID	Asset	Capability	Boundary	Obligation	Discharged by
TM-01	Device root key (fuses)	C1	Root-of-trust register interface	No sequence of register accesses makes fuse content observable outside the ladder	Information flow, RTL (§23.4)
TM-02	Working key	C2	Memory path through the engine	Key material influences no value on the memory path except through the cipher	Information flow + Chapter 15's negative connectivity rows
TM-03	Working key	C3	Debug access port into the root-of-trust domain	A debug request addressing the key region is never granted, unlocked or not	Formal, on the always-on control logic (§23.3)
TM-04	Working key	C2	Power-domain edge of the root of trust	Powering the domain down destroys the key; leaving the unlocked state requests a wipe	Formal + power-aware simulation (Chapter 19)
TM-05	Anti-rollback counter	C2	Flash controller register interface	No command sequence decreases the counter	Formal, on the counter's control logic
TM-06	Working key	C3	Engine's physical emissions	No key material is recoverable from the engine's power signature	<b>Argued, not verified</b> — see §23.5

Read the last column before the others. Three rows name a single technique that produces evidence, two name a pair, and TM-06 names none, because at RTL there is no honest way to discharge it. That row is not a defect in the table; it is the table doing its job. A threat model that contains only rows you know how to close is a threat model that was written backwards from the tools you own.

### It is an artifact, and artifacts go stale

Treat the threat model exactly as Chapter 5 treats the verification plan: a versioned document, under review, with traceability from each row to the evidence that discharges it, and with a named owner. That traceability is not bureaucracy: it is the only mechanism that notices when a design change invalidates a row.

Design changes invalidate rows constantly, and almost never visibly. A performance counter is added and made readable from user mode; a boundary just moved. A new DMA channel is given an address range for convenience; a boundary just moved. During bring-up someone adds the “temporary” multiplexer Chapter 15 warns about, and the negative connectivity row that discharged TM-02 is now false while still showing green in yesterday's report. The discipline that catches these is the same one Chapter 7 applies to waivers: every row carries an owner and an expiry, and the review that re-reads the threat model is scheduled, not spontaneous.

One more scoping habit, learned the hard way by anyone who has reviewed an IP-level security report. A finding at a module boundary is conditional on integration. A published module-scoped study of an accelerator's configuration port makes the

point in its own words: it establishes that the module, taken in isolation, applies no local privilege check and therefore depends on trusted integration or upstream filtering, and it explicitly declines to claim that any particular deployment is exploitable — whether the weakness becomes a system-level one depends on integration and on whether an untrusted requester can reach that path at all [3]. State your findings the same way. “This block does not enforce X” and “this SoC is vulnerable to Y” are different claims with different evidence, and conflating them is how a security report gets ignored by the people who could act on it. That accelerator is one a reader can open: the study takes the `NV_NVDLA_csb_master` IP of NVIDIA’s NVDLA at its public `nvd1av1` snapshot, and enumerates the port’s assets against named weakness identifiers as the first step of a security plan [3].

### 23.3 Security properties and how they differ

---

Two shapes carry most of the load, and both are statements about *flow* rather than about values. Integrity is violated when an unauthorised agent — software or physical — can modify the state of a target register or memory: untrusted sources must not flow to trusted state. Confidentiality is violated when an unauthorised agent can read the state of a target register or memory: secret sources must not flow to observable destinations [1]. Every row of the threat-model table is one or the other, which is why sorting assets by which they need was worth the effort.

Now compare that with a functional property. Chapter 11’s canonical 4 KB-boundary property (Arm IHI 0022H.c §A3.4.1) [4] is a claim about *values at a point*: given an accepted INCR write address, this arithmetic relation holds. It is decidable by looking at one execution at that point. A confidentiality property is not decidable that way at all. Its natural statement quantifies over *pairs* of executions — change the secret and nothing the adversary can see may change — and a concurrent assertion, by the definition Chapter 11 gave it, evaluates sampled values from one execution. That is a mismatch of shape rather than a limitation you can write around with a cleverer sequence, and it is why information-flow analysis exists as a separate technique rather than as a style of assertion (§23.4).

#### Why formal fits, and exactly how far

For everything that *is* expressible as an invariant, formal is the right shape of answer, and the reason is the negative property. “This never happens, by any path” is precisely what a proof engine returns and precisely what no amount of simulation returns. Formal’s strength here is exhaustiveness within scope — when a proof succeeds it rules out an entire class of violations for the modelled design under the stated assumptions — and in security work it is used most effectively on localised, precisely stateable things: access-control invariants, privilege-transition correctness, protocol safety [2]. Three of the six rows name formal for at least part of their evidence, and TM-03 is the purest case.

**Scenario (the debug unlock path).** TM-03 and TM-04 both concern the working key and both cross a boundary the flagship’s own instrumentation creates: post-silicon observability is a designed-in back door to the state beside it — keys, firmware, debug modes — much of it still present on-field, and published system compromises have gone in through exactly those features [5]. The unlock unit that guards it is a small always-on finite state machine. **Approach.** Three assertions and three covers, all reading always-on signals only.

```
``systemverilog // Root-of-trust debug port: obligations TM-03 and TM-04. // All
signals below are in the always-on domain. a_unlock_needs_token : assert property
(@(posedge clk) disable iff (!rst_n) $rose(dbg_unlock) |-> token_verified);

a_key_region_locked : assert property (@(posedge clk) disable iff (!rst_n) (dbg_req
&& dbg_addr_in_key_region) |-> !dbg_gnt);

a_relock_wipes : assert property (@(posedge clk) disable iff (!rst_n)
$fell(dbg_unlock) |=> wipe_req);

c_unlock_granted : cover property (@(posedge clk) disable iff (!rst_n)
$rose(dbg_unlock));

c_key_region_probed : cover property (@(posedge clk) disable iff (!rst_n) dbg_req
&& dbg_addr_in_key_region);

c_relock_seen : cover property (@(posedge clk) disable iff (!rst_n)
$fell(dbg_unlock)); ``
```

**Why.** None of the six looks back more than one cycle or forward more than one, and all of them read a handful of always-on signals, so a proof engine settles them in seconds and the result holds for every arrival time of an unlock request — which is the randomisation no directed test was ever going to supply.

Read those six precisely, because the reading is the lesson.

`a_unlock_needs_token` constrains one cycle: the one on which `dbg_unlock` rises, and only to the extent that `token_verified` was asserted then. It says nothing about *which* token, and nothing about whether the verification that produced that signal is itself sound. This is Chapter 11’s reading of `a_no_ct_unauth` transplanted to another asset — an invariant names a state, and provenance is a different invariant over different state — and the obligation to establish the verifier belongs on the verifier, discharged there rather than enjoyed here (Chapter 14).

`a_key_region_locked` deliberately does not mention `dbg_unlock`. Guarding a security property with the very state the adversary is working to reach would make it vacuous exactly where it is needed. Note also which signal it constrains: the *grant*, not the request. The request belongs to the adversary; only the grant belongs to the design, and a property written over an adversary-controlled signal is not a property, it is a wish.

`a_relock_wipes` asserts that a wipe is *requested* one cycle after the unlocked state is left. It does not assert that the wipe completes: it asserts the request, which is all TM-04 asks of it. The row’s other half — that a power-down destroys the key outright

— is what its second technique, Chapter 19’s power-aware test, establishes. That is why the row names two.

The three covers are not optional. An assertion about denial proves nothing on a run that never made the request, an assertion about relocking proves nothing on a run that never relocked, and `c_key_region_probed` demands something a functional environment will not produce: a request no cooperative requester would make. Chapter 9’s constraints encode what the interface makes *legal* and what the testbench steers toward, and neither covers traffic whose only purpose is to be refused. An adversary is under no such restraint. Reusing the functional environment unchanged is therefore not a saving; it is a silent deletion of the entire threat model, and it is the most common way a security campaign runs green for a month while checking nothing.

### Where the fit ends

The same reasoning that makes formal ideal for the unlock unit disqualifies it for the block next door, and it is worth being blunt, because enthusiasm runs the other way. Chapter 14 already ran the experiment: the crypto engine’s two key-handling properties do not converge, because the key ladder plus the GCM datapath is more state than the engine will take. Nothing about calling them *security* properties changes that. Formal at SoC scale runs into state explosion, heavy abstraction effort, and the difficulty of modelling realistic software and environment assumptions; it is indispensable for targeted reasoning and rarely sufficient on its own for a full system [2].

So the divergence is between the shape of answer you want and the capacity you have, and it is what routes the rest of the table. TM-03 and TM-05 are small control logic and stay in formal, and so does the first half of TM-04. TM-01 and TM-02 are statements about a datapath’s dependence on a secret, so they go to information flow. Anything that only manifests after a firmware stack has booted goes to a platform that can boot one. And TM-06 goes nowhere, for reasons §23.5 owes you.

One rule survives the routing, and it comes out of Chapter 14’s crypto-engine scenario. In a functional proof an assumption is a promise about a cooperative neighbour; in a security proof the environment is the adversary, so every assumption is a concession. Make that operational: the assumption list is a column of the threat model, and each entry must name either the capability tier that cannot produce the excluded behaviour, or the property elsewhere that discharges it. An assumption that maps to neither is a hole with a green tick on top.

## 23.4 Information flow and its verification

Information-flow tracking attaches labels to data — secret or public, trusted or untrusted — and follows how those labels propagate through the design, raising a violation when a sensitive value reaches an untrusted sink or an untrusted input influences a control signal [2]. The label is usually called taint, and the mental model is worth stating exactly: you are not tracking the *value*, you are tracking *dependence*. Anything whose value could differ because the secret differs is tainted, whether it holds a copy of the secret, a function of it, or merely a bit that was set because of it.

A rule in this style has four parts, and writing them out is most of the work:

**Table 23.2** — The four parts of an information-flow rule, with the crypto engine’s confidentiality rule written out against each. The fourth part is where the argument lives: without an exception the rule fails immediately and correctly, because a cipher’s output depends on its key, and with one the rule is exactly as strong as the exception is narrow.

Part	What it names	On the crypto engine
Source set	the asset, as the signals that hold it	every bit of the key ladder’s working-key output
Sink set	what the adversary can observe	bus-readable registers, the debug data register, the trace fabric’s input
Activation	when tracking starts	from the cycle a key is loaded, rather than from reset
Exception	the flow that is permitted	the cipher core’s ciphertext output

Tools express these differently, and the rule languages are proprietary; the shape is what transfers. The published vocabulary covers the same four ideas — a no-flow relation between a source and a destination, a `when` clause that starts tracking on a condition, a disjunct that excuses the flow while some condition holds, and an *ignoring* clause that declares one destination secure and stops the analysis there <sup>[1]</sup>.

### The exception clause carries the whole argument

Look again at the fourth row. A confidentiality rule for the engine reads, in words: *the working key must not flow to any output, ignoring the cipher’s ciphertext output*. Without the exception the rule fails immediately and correctly — the ciphertext depends on the key; that is what a cipher is. With the exception, the rule becomes checkable and also becomes exactly as strong as the exception is narrow.

This is Chapter 14’s over-constraint hazard in security clothing, and it is worse in one specific way. An over-constraining assumption looks like a modelling shortcut, and a reviewer who reads assumption lists will challenge it. An exception clause looks like *domain knowledge* — of course the ciphertext depends on the key — so it reads as competence rather than as a concession, and reviewers wave it through. Widen it by one signal and you have declared a whole port secure. The discipline is the same as for assumptions: every exception is a row, with a written argument for why the flow it permits is safe, and with the same expiry and owner as any waiver (Chapter 7). “The ciphertext output is declassified because the cipher is trusted” is an argument. “We added `busy` to the ignore list because the rule kept failing” is a defect with a comment on it.

### Non-adjacency is not non-leakage

Chapter 15 proved the negative connectivity rows for this exact key ladder — no structural path to a bus-readable address, to the debug data register, or to a scan-observable point — and was explicit that this is a *structural* result: no path exists. Information flow answers the other question, and the difference is the whole reason this section exists. The key bus from the ladder to the cipher core is a path that exists **by design**. Nobody is going to prove it away. The question is whether anything

reachable downstream of that path depends on the key other than through the cipher's intended transformation, and no cone-of-influence check can answer it, because the cone is supposed to be there.

That is precisely the failure in the opening AES result. Structurally, key and datapath are adjacent — they must be. Functionally, a value carrying the key survived to the output pin. A team holding a green connectivity report and calling it non-leakage has proved non-adjacency and filed it under the wrong heading.

### The unintended paths are the point

Static analysis builds dependence graphs from the RTL or the netlist and reasons about possible flows without running anything, which makes it a good early screen and an over-approximating one: it reports flows the design will never actually perform. Dynamic tracking propagates labels at runtime, so it sees the paths an execution really took and catches trigger-dependent behaviour, at the cost of instrumentation and runtime. Both meet the same wall, and it is the honest limit of the technique: implicit flows and microarchitectural effects are hard to model accurately, and policy precision cuts both ways — over-approximate and you drown in false positives, prune aggressively and you lose real violations [2].

Take that trade seriously: it decides whether anybody reads your reports. A screen returning four hundred alerts of which nearly all are benign will be ignored within two weeks, and the instinct — loosen the policy — is the move that starts deleting real findings. Scope by threat-model row instead: one rule per row, so every alert arrives already attached to an asset, a capability and a boundary, and triage becomes “does this flow cross TM-02's boundary?” rather than “is this bad?”. The rows also give you somewhere to record a benign flow permanently, which is the only thing that stops the same false positive being re-triaged every release.

**Scenario (the GPU).** Four shader clusters share a unified memory, and contexts belonging to different applications time-slice it. The asset is one context's data; the sink is anything the next context can read; the boundary is the context switch. **Approach.** State it as a flow rule with an activation condition — track everything written by context A, starting at the switch into A, and require that none of it reaches anything readable after the switch out of A — and pair it with a structural inventory of every piece of state that survives a switch: register-file banks, line buffers, scratchpad, cache ways, the descriptor queues. **Why.** The failure here is never a wire. It is *residue*: state that a functional test cannot see because the next context overwrites it before reading, and that a taint rule sees immediately because it is tracking dependence rather than values. Note also what makes this rule statable at all — the activation condition. Without it the rule is “context A's data never reaches memory”, which is false by construction, since writing to memory is the point.

The GPU case also shows where flow coverage sits, and it is not where teams first look. The interesting covers are on the *sources*, not the sinks: did this run ever put a secret in the source register, did it ever switch context with live data resident, did it



ever load a key at all. A flow rule evaluated over a run in which no source was ever tainted is Chapter 6’s vacuous pass wearing a different hat — the report is green, nothing was tracked, and the difference is invisible unless someone covered the source.

## 23.5 Side channels

This is the strongest example in the book of a property that lives *below* the abstraction you verify at, and the sentence to hold onto is this: an RTL model represents logic values at clock ticks and represents nothing else. It does not model supply current badly, or approximately, or with optimistic assumptions the way it models unknowns. It does not model supply current at all. A property about how much charge the part draws is not *hard* to check on an RTL model; it is not *expressible* on one, and no amount of stimulus, coverage or proof depth changes that.

So the first job is to sort the channels by whether the leaking quantity exists in your model, because the answer decides whether you are doing verification or something else.

**Timing is inside the model, and this is the good news.** A cycle count is a signal-level fact: if the number of cycles between an operation’s start and its completion depends on key material, that dependence is present in the RTL and can be reasoned about there. Two shapes handle it. Where the specification says the operation has a fixed latency, the claim is single-execution and mechanical:

```
// LAT is the engine's declared operation latency, in cycles.
a_fixed_latency : assert property (@(posedge clk) disable iff (!rst_n)
 $rose(op_start) |-> ##LAT op_done);

c_op_start : cover property (@(posedge clk) disable iff (!rst_n)
 $rose(op_start));
```

Where the latency legitimately varies — with message length, with back-pressure from the memory path — `a_fixed_latency` is false and the claim becomes a flow rule instead: no key bit may flow to the completion signal. Which is to say `op_done` is just another sink for §23.4’s machinery — a timing side channel is precisely the case where execution latency depends on secret data or on privileged state <sup>[2]</sup>, and analysing one is a listed application of information-flow tooling <sup>[1]</sup>. The unification is worth pausing on: a timing channel is not a new category of problem, it is a confidentiality property whose sink happens to be a control signal rather than a data bus.

**Power and electromagnetic emission are outside the model, and this is the bad news.** Nothing in the RTL holds them. What pre-silicon work can do is reason over *proxies*: switching activity, performance counters and timing distributions exported from a run and post-processed as stand-ins for power and timing behaviour, which is enough to rank structures by risk and to compare a countermeasure against its absence <sup>[2]</sup>. Be precise about what a proxy licenses. Switching activity correlates with dynamic power; it is not power, it carries none of the analogue detail that a

measurement carries, and a countermeasure that improves the proxy has been shown to improve the proxy. Ranking and comparison are real value. A leakage *verdict* is not available at this abstraction, and a report that implies one is worse than none, because it retires a question that is not closed.

This is the same structural problem Chapter 21 has with analog behaviour — a property that lives in the continuous world while the model is discrete — and the same resolution applies: model the quantity explicitly at some cost and fidelity, or move the question to a platform where the quantity is physically present. Here the second platform is fabricated silicon with measurement equipment attached, which puts the closing evidence in Chapter 20’s territory and on Chapter 20’s calendar.

### The capability tier is a design decision

One design choice deserves its own paragraph, because it can invalidate a threat model without touching a single security block. Physical side channels and fault injection are C3 capabilities: nominally they require access to the package. But software-controllable mechanisms for scaling voltage and frequency and for monitoring power consumption are ordinary features of modern chipsets, and they let an attacker mount fault-injection and side-channel attacks without physical access to the device [6]. The flagship has DVFS with three operating points on the host cluster and the compute island, and something has to select among them. If that selector is reachable from C1 or C2, a whole row’s capability tier has quietly changed, and every property justified by “the adversary would need physical access” is now unsupported.

That is a threat-model question, not a crypto question, and it is exactly the kind a verification engineer is placed to ask — not how a power analysis works, but which side of a boundary that selector’s enable sits on.

Fault injection is the same asymmetry running in the other direction: the adversary perturbs rather than observes, deliberately inducing conditions — glitched supplies and clocks, faults engineered so that security-critical instructions are skipped — that a functional model would classify as impossible [6]. The machinery for injecting faults and measuring what survives is Chapter 22’s, built there for safety; what security adds is that the fault distribution is chosen by an opponent rather than by physics, so a campaign weighted by natural failure rates is answering a different question. Borrow the tooling, rewrite the weighting.

### What TM-06 gets

Return to the row the table left open: *no key material is recoverable from the engine’s power signature*, against a C3 adversary. Its nearest neighbour is closable and it is not, and the pair is worth seeing together. The timing claim — that no key bit reaches `op_done` — is a flow rule with a sink you can point at; the honest move on finding it in review is to open TM-07 for it, not to widen a row governing a different boundary. The power claim has no signal to name at all, so it gets an argument rather than a proof, and an argument has a shape too: a named countermeasure recorded in the design review; a proxy comparison showing the countermeasure moves

the proxy in the expected direction, with its own limitations written down; and a post-silicon leakage assessment on the plan with a date and an owner. That is Chapter 7’s conditional sign-off, applied honestly: evidence now, named evidence later, and a residual risk that somebody puts their name against. What it is not is a green tick, and the distinction between a row like this one and a row like TM-03 is the single most useful thing a verification engineer can hold clearly in a security review.

## 23.6 Where security verification lives in the flow

The cheapest security work in a project is not verification. It is the hour in architecture definition when someone lists the assets and draws the boundaries, before the interconnect topology is fixed and while an address decode is still a decision rather than a fact. In scheduling terms: an asset nobody names early gets protected late, and the protections still available late are the expensive, awkward ones. The second-cheapest is Chapter 15’s integration table with its negative rows in it from the first build, so that a “temporary” multiplexer fails a job the day it lands rather than surviving to tape-out.

After that, each question goes where it can be answered, and [Table 23.3](#) sets out which platform that is: forced by the question, not chosen from what the project happens to own.

**Table 23.3** — Six security questions, each routed to the platform on which it can be answered, from a proof over a small control block to measurement on a fabricated part. The third column is what keeps this from being a menu: the answers are not interchangeable, an adjacency result and a flow result answer different questions, and the last row’s question exists nowhere but in silicon.

Question	Where it is answered	What the answer is worth
Does this small control block enforce an invariant?	formal	exhaustive within the scope and the assumptions
Is there any structural path from this asset to that sink?	connectivity check (Chapter 15)	cheap, and about adjacency only
Can this asset influence that sink at all?	static information flow	an early screen, over-approximating: expect false positives
Does it influence it along the paths a real workload takes?	dynamic information flow, in simulation or on an emulator	grounded in execution, silent about paths not taken
Does the policy still hold once firmware boots and drivers run?	emulation	realistic and long, with limited visibility
Does the fabricated part leak through a physical channel?	silicon and measurement	the only place the question exists

### What needs a platform that can boot

Some security behaviour has no meaning below the system level. Privilege transitions, the interaction of a DMA engine with a memory protection configuration, a

debug unlock sequence driven by real boot firmware, the moment a driver hands a buffer across a boundary — these need a running software stack, interrupts and peripheral concurrency, and they need executions long enough to reach them, which is exactly what emulation supplies and simulation does not [2].

The most useful pattern joining this section to §23.3 is property-guided monitor generation. Prove the property formally where it is tractable — a small subsystem, an access-control invariant — then translate the same property into a synthesisable checker and run it on the emulator under full firmware workloads, so that a proof of the rule and a search for the sequences that stress it are the same statement checked two ways [2]. `a_key_region_locked` is a candidate: proved on the unlock unit in seconds, and worth carrying into a boot campaign because what a proof cannot tell you is whether real firmware ever puts the system into the state the property protects.

Now the constraint, stated where the recommendation is made. Emulator capacity is normally reserved for tape-out-critical functional regressions, and security workloads are not usually first-class citizens in flows built for coverage closure, so campaigns get scoped down to a few high-risk blocks or pushed to a later stage [2]. Plan accordingly: a security campaign is a **campaign** in Chapter 12’s exact sense — launched for a stated reason, owned by someone, producing a written result, then stopping — and it must compete for slots explicitly, with a named block list and a named window. A security activity described as ongoing background work will lose every scheduling argument it is never present for.

One further caution comes with the platform. Security monitors and adversarial harnesses often have to be re-engineered to fit an emulator’s timing and capacity, which leaves the simulation and emulation environments genuinely different and makes a failure seen on one hard to reproduce on the other [2].

### Adversarial stimulus, and where it actually pays

Stimulus that asks “can an adversary drive this design into an unsafe state?” rather than “is this design functionally correct?” is a different generation problem, and its value depends almost entirely on three things: the quality of the generated sequences, the security oracles that decide what counts as a violation, and whether an interesting failure can be reproduced and triaged at all [2]. In practice the third is the one teams under-budget, and a finding nobody can reproduce is not a finding.

It also pays very unevenly across IPs, and knowing where it pays is the practical skill.

**Scenario (the video codec).** A decoder is the purest attack surface on the SoC: its input is a bitstream that arrives from the network or from a file, so it is attacker-supplied by construction, and its control flow is data-dependent by design — that is what a variable-length entropy-coded format is. **Approach.** Two changes to the block’s existing environment. First, replace the oracle. Chapter 3’s definitional oracle — the standard’s bit-exact reference decoder — is worth nothing here, because on a malformed stream the reference has no defined output to compare

against; the oracle for hostile input is a set of invariants that must hold whatever arrives: every line-buffer and DDR write stays inside its allocated region, the decoder makes forward progress or raises an error within a bounded number of cycles, and no malformed stream reaches a state only a firmware reset can leave. Second, generate the input by mutating well-formed streams rather than by constraining a transaction class. **Why.** Mutation-based generation earns its keep here precisely because the input is a structured byte stream that a mutation leaves *almost* legal — which is the interesting region. Apply the same technique to a bus interface and most mutations produce traffic the protocol forbids outright, which the interface will reject and which tells you nothing.

### Telling a verifiable property from an arguable one

The test is short enough to apply in a review. **Can you name the signals?** A verifiable security property can be written down entirely as signals in a model you own — a source set, a sink set, an activation condition and an exception if it is a flow rule; an antecedent and a consequent if it is an invariant. TM-03 passes on the second form: `dbg_req`, `dbg_addr_in_key_region`, `dbg_gnt`. The timing claim §23.5 sets beside it passes on the first: `op_start` and `op_done` are signals, and “no key bit reaches the second” is an ordinary flow rule. TM-06 fails where that neighbour passes, because its subject is a physical quantity with no representation in the model: no signal list exists, and nothing will ever settle it. Between those poles sit the properties whose signals exist but whose exception is doing suspicious work — and that is where a reviewer should spend the meeting.

Which is also the answer to the question this chapter opened with, about what someone without a security background contributes. Not attacks, and not cryptography. Three questions asked persistently: *who can write this register?*, *what happens to this state across this transition?*, and *what exactly does this exception permit?* Every one of those is a question about design behaviour, and none of them requires knowing how a power analysis works. The specialists supply the threat model. Everything after it is the job this book has been describing all along.

---

## IN PRACTICE

Security is on almost every project and owned by almost nobody. In 2024, 83 percent of IC/ASIC projects incorporated security features — hardware modules holding encryption keys, content-protection keys, passwords, biometric references — and those features add new requirements and complexity to the verification process <sup>[7]</sup>. Having the feature and having verified it against an adversary are separated, on most projects, by an unstaffed gap.

The failure mode is chronological. A threat model that arrives after RTL freeze does not produce plan rows; it produces findings, and a finding against frozen RTL is a waiver argument with a deadline attached. Teams that get this right treat the threat model as a month-one deliverable with the same standing as the verification plan, precisely so that its rows can still change a design.

Two further frictions are worth expecting. Information-flow tooling is specialist, its rule languages are proprietary, licences are scarce, and the knowledge concentrates in one or two people whose departure resets the capability — so write the *rules* against threat-model rows, in a form a successor can read, not as tool artifacts. And security bugs do not fit the severity ladder. Chapter 20’s asymmetry is the reason: once a defect of this kind is known outside the company, the time available to mitigate collapses, because it can be exploited rather than merely encountered. Most teams end up with a parallel classification and a separate, access-controlled tracker, which is inconvenient, occasionally absurd, and correct.

---

## PITFALLS

1. **Reusing the functional environment unchanged.** *Symptom:* the security regression is green from the first night and stays green. *Cure:* an adversary is bound neither by the protocol nor by what the testbench chooses to generate; the environment must be able to emit the forbidden request, and Chapter 11’s rule applies here in full — every implication ships with a cover on its antecedent, the positive obligations as much as the denials.
2. **Reading a connectivity result as non-leakage.** *Symptom:* “we proved the key cannot reach the bus”, on the strength of a no-connection job. *Cure:* that result is about adjacency; the flow question concerns paths that exist by design, and those are the paths the key actually uses.
3. **An exception clause that grew.** *Symptom:* the ignore list has six entries and no comments. *Cure:* one written argument per exception, with an owner and an expiry, reviewed exactly like a waiver — a widened exception is indistinguishable from a passing property.
4. **A threat model written backwards from the tools.** *Symptom:* every row closes. *Cure:* suspect it immediately; a model scoped to the evidence you already know how to produce has hidden its residual risk rather than retired it, and the row you cannot close is the one that most needs a named plan.
5. **A security assumption enjoyed rather than discharged.** *Symptom:* a proof converges after an `assume` that the boot sequence is followed. *Cure:* every assumption names the capability tier that cannot produce the excluded behaviour, or the property elsewhere that establishes it.
6. **Reporting a module-scope finding as a system claim.** *Symptom:* “this accelerator is vulnerable.” *Cure:* state reachability and integration explicitly; a block enforcing nothing locally may be perfectly safe behind a filter, or may not be, and that difference is evidence somebody has to produce.

---

## SUMMARY

- Security verification is scoped by a **threat model** — assets, adversary capabilities, trust boundaries — not by a specification. Without one, “is it secure?” is un-



answerable; with one, it becomes a list of checkable claims and a list of honest gaps.

- The properties are negative and the failures are often functionally silent: the design computes the right answer and leaks anyway <sup>[2]</sup>. No quantity of functional pass evidence accumulates into a security result.
- There is no security-coverage percentage, and there will not be one soon. The deliverable is named claims, named gaps, and a residual risk somebody signs.
- Rarity is not a defence inside the threat model's scope. A constrained-random generator is a distribution; an adversary is an optimiser who will arrange your improbable conjunction deliberately and then repeat it.
- A no-connection result proves **non-adjacency**. Only an information-flow argument speaks to the paths that exist by design — and its **exception clauses** carry the whole weight of the result, which is why each one needs a written argument.
- Formal is the right shape of answer for a negative property and runs out of capacity exactly where assets are densest. Route small control logic to proofs, datapath dependence to information flow, and firmware-dependent rows to a platform that can boot one.
- Side channels sort by whether the leaking quantity exists in your model. Timing does, so it is a sink like any other; power and emission do not, so pre-silicon work yields ranking and comparison of countermeasures — never a leakage verdict.

---

## FURTHER READING

**From the corpus.** <sup>[2]</sup> is the anchor for this chapter and the fullest recent survey of the area as a *workflow*: the technique taxonomy and its bottlenecks, static versus dynamic information-flow tracking, the campaign-driver and observability dimensions, the coverage gap, the emulator-contention and environment-divergence problems, and formal-assisted monitor generation. Note that it uses “validation” as an umbrella term where this book says verification. <sup>[3]</sup> organises the same territory by workflow stage — asset identification, threat modelling, test-plan generation, simulation and formal artifacts, countermeasures — and its worked module-scoped analysis is a model of how to state a finding without overclaiming it. <sup>[1]</sup> is the clearest short statement of confidentiality and integrity as flow relations, with the rule shapes including the exception clause, and it is the source of this chapter's opening result. <sup>[6]</sup> extends fault campaigns into adversarial territory and supplies the weakness taxonomy for glitching, instruction skips and software-reachable voltage and frequency control. <sup>[5]</sup> explains why design-for-debug is itself an attack surface and why security defects are dispositioned differently from functional ones. <sup>[7]</sup> gives the 2024 adoption figure.

**Outside the corpus.** MITRE's Common Weakness Enumeration, and in particular its hardware design view, is the standard vocabulary for naming a weakness class, and a threat-model row that cites a CWE identifier is easier for a security reviewer to check than one that does not. For the artifact this chapter says to write first, Accellera's *Security Annotation for Electronic Design Integration* (SA-EDI) standard and the IEEE P3164 working group's *Asset Identification for Electronic Design IP*

supply a principled vocabulary for assets. For the channel this chapter can only rank: P. Kocher, J. Jaffe and B. Jun, “Differential Power Analysis” (CRYPTO ‘99), and S. Mangard, E. Oswald and T. Popp, *Power Analysis Attacks* (Springer, 2007), with G. Goodwill et al., “A testing methodology for side-channel resistance validation” (NIST, 2011) for the measurement side. Finally, the OpenTitan project publishes an open-source root of trust together with its security specifications; reading a real one end to end is the fastest way to see what a reviewed threat model looks like.

---

## REFERENCES

1. **D28** J. Oberg, "System-Level Security Verification Starts with the Hardware Root of Trust," Proc. DVCon U.S., 2019.
2. **P14** T. Rahman, S. Saha, A. Y. Alhurubi, S. K. Saha, F. Farahmandi, and M. Tehranipoor, "Emulation-based System-on-Chip Security Verification: Challenges and Opportunities," arXiv preprint arXiv:2604.15073v1, April 2026.
3. **P13** K. T. Hasan, M. A. Hasan, N. Alam, M. T. Islam, U. Das, and F. Farahmandi, "AI-Assisted Hardware Security Verification: A Survey and AI Accelerator Case Study," Proc. IEEE VLSI Test Symposium (VTS), Napa, CA, pp. 1–7, 2026.
4. **S18** Arm Limited, "AMBA AXI and ACE Protocol Specification, ARM IHI 0022H.c (ID012621)," Arm Limited, January 2021.
5. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," IEEE Design & Test, vol. 34, no. 3, 2017.
6. **D2** Sesha Sai Kumar C V, A. Mouallem, and J. Mazzawi, "Fault Injection Analysis for Automotive Safety and Security," Proc. DVCon Europe, 2022.
7. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.



PART VII

# The Human and the Machine

---

*The people, the process, and the machine: how verification teams are built, what automation is actually delivering, and where the discipline is going.*

---

## 24. Teams, Process and Ramp-up

Week 41 of the reference SoC. A system-level test running a quantized network on the neural accelerator, with both DMA channels streaming, produces one wrong output tensor in roughly four hundred inferences. Two days of debug find the mechanism: the accelerator’s weight prefetcher wraps its buffer pointer one entry early when the weight stream ends mid-beat under DMA contention. It reproduces only at 2-bit weight precision, only with a degenerate tensor geometry, and only when the other channel is moving traffic. The block signed off in week 33 at 100% statement and branch coverage.

The post-mortem convenes three people. The accelerator’s verifier says memory contention is a system-level concern: her environment drove the accelerator’s ports directly, as the plan said, and quantization corners were her rows — she closed all of them. The SoC integration verifier says quantization corners are a block-level concern: his job was connectivity and system traffic, and he had no visibility into weight-buffer geometry. The DMA verifier was not invited, because the DMA was not at fault. Everyone in the room did their job. Every artifact in the project had a named owner: the block environment, the SoC environment, the DMA environment, the coverage model, the vplan. And the outcome that the escape actually falsified — *is the accelerator demonstrably correct while something else is competing for memory?* — had no owner at all.

This chapter is about that gap and about what teams use to close it: how roles are drawn, how reviews are run, how progress is talked about, how people are brought into the discipline, and what process can and cannot do. It has fewer hard technical claims than any other chapter in this book, which makes it more exposed to confident folklore, not less. Where the corpus supports a claim you will see a marker; where it does not, the text says whose read it is.

---

### CHAPTER MAP

- §24.1 establishes why a role defined by the artifact it owns (“owns the test-bench”) leaks, and what defining a role by outcome changes.
- §24.2 treats what each review rite is actually for, the mechanics that make it productive, and the specific ways each degenerates into theatre.
- §24.3 develops how Chapter 4’s maturity stages change the *shape* of a status conversation — from confidence to evidence — and how a stage claim gets inflated.
- §24.4 sets out what a designer brings to verification that transfers, what actively works against them, and what a structured ramp-up looks like week by week.
- §24.5 states what checklists, templates and coding standards genuinely buy, and how to recognize the point past which process substitutes for thought.

## 24.1 Roles, and what they actually own

A verification organization has fewer distinct roles than its job titles suggest. Someone answers for a block's features from extraction to sign-off: the **block verifier**. Someone answers for the machinery everyone else runs on — base environment, register generator flow, regression infrastructure, coverage merge: the **methodology owner**, whose customers are the other verifiers. Someone answers for behaviour that exists only once parts are connected: the **integration verifier**. Someone reads trends and negotiates: the **lead**. Around them are the designer, the architect and the firmware engineer, none of whom are verification staff and all of whom produce verification evidence.

The interesting question is not what to call these people. It is what each is answerable for, and the default answer — the one that arrives without anyone deciding it — is the *artifact*. She owns the block environment. He owns the SoC environment. That one owns the coverage model. Artifact ownership is administratively convenient, it maps cleanly onto repository directories, and it is the source of the chapter's opening escape.

The alternative is to state ownership as an **outcome**: not “owns the accelerator test-bench” but “owns whether the accelerator is demonstrably correct under every condition the system can present to it”. This is not a rhetorical upgrade of the same sentence. The two formulations disagree about who is at fault when the environment is excellent and the answer is still unknown, because artifact ownership is discharged by the artifact being good and outcome ownership only by evidence about the design.

The book's own vocabulary already contains the idea one level down. An RTL designer's responsibility is not to write RTL — that is only a means to an end — but to produce a working design, which is why the entire design team, not the verification team alone, must contribute to the verification plan and answer for its completeness <sup>[1]</sup>. The same move applied to verification roles turns “my rows are closed” from a defence into an admission: closed rows are an artifact statement, and the outcome is a claim about the design.

**Where the boundaries actually fail.** Not in the middle of a block, where the owner is obvious, but at three seams.

The first is **block-to-system**. At integration, each block generally arrives from its own team, and the SoC verification team must debug and analyse the assembled system without knowledge of the IP internals — each block is a black box to that team, which has neither the time nor the background to understand the inner workings of several complex IPs at once <sup>[2]</sup>. That is a structural fact, not a staffing failure, so the seam cannot be closed by asking the integration verifier to learn every block. It closes by naming an owner for each *cross-block outcome* — “the accelerator under memory contention”, “the flagship's DMA under a coherent writeback” — and putting that owner on the block side, where the internals are understood, with an obligation to state what system conditions the block environment did *not* present.



The second is **design-to-verification**, which is not a wall at all. Design engineers spent, on average, 49% of their time on verification activities in 2024 [3]. Half a designer's working life is already inside this discipline: white-box assertions, smoke tests, debugging failures alongside the verifier. A boundary drawn as though the designer hands over an artifact and departs produces the familiar pathology in which design-side checks are written but never regressed, because they belong to no suite.

The third is **feature-to-nobody**. Every plan has rows whose evidence is expected to arrive from somewhere else — a protocol checked by purchased verification IP, a corner covered by the emulation run, a mode “the software team exercises”. Chapter 5 has them reviewed face to face with the designers, and Chapter 14 gives the formal ones a register with an owner and a discharge column. The point here is that column: an assumption is a row whose owner is a *sentence*. Give it a person or delete it.

**Scenario.** The video codec's checking rests on a bit-exact C model. The design team wrote it first, to explore the entropy-coding trade-offs, and it is excellent — faster and better tested than anything the verification team would have produced in the time. So verification adopts it as its checking reference and compares the RTL against it frame by frame. Ownership by artifact says this is efficient: design owns the model, verification owns the comparison, nobody duplicates work. **Approach.** Keep the model, but be precise about what it is — a transfer function written by the design team, not a golden model — and move the *outcome*, “the codec's output is what the standard requires”, to the verification side. Discharge it with evidence the C model cannot produce: conformance streams from the standards body decoded against both, an independent reader written from the specification for the syntax layer, and a written record of every place the C model was tuned to match the RTL rather than the specification. **Why.** A model written by the design team encodes the design team's reading of the specification, so it is not the independent second interpretation a reference model has to be (Chapter 3). Comparing RTL against it reconverges on that reading, and the comparison can stay green on a misread clause forever. The artifact is fine; what was wrong is that adopting it silently transferred an outcome nobody re-assigned.

One test separates the two kinds of ownership, and it is the author's read rather than a published rule: ask an owner to state what would have to be true for their answer to be *wrong*. An artifact owner describes a bug in the artifact; an outcome owner describes a condition of the design they have not yet reached. A project that cannot get the second answer for a feature does not have an owner for it — it has a directory.

## 24.2 The rites of review

Chapter 4 puts the reviews on the calendar and names who sits in each; Chapter 5 works the plan review's internal procedure — weakness analysis, row by row — and Chapter 7 works the sign-off review's evidence package. None says what makes a re-

view *work as an institution*, or how each one dies. Start with what each is for, because the rites have genuinely different purposes and confusing them is a common way to run a bad one.

The **plan review** exists to test *completeness*, a property no single perspective can check. That is why its roster deliberately reaches outside the project team, and why experienced design, verification and software engineers are all named as contributors: the reviews are designed to establish that the identified functional verification requirements are complete <sup>[4]</sup>. Completeness is the review question that gets harder the more expert the room becomes, because expertise is a map of what you already thought of.

The **code review** — of the testbench, not the design — exists to test *fidelity between intent and implementation*. Its object is source code produced by one engineer and read by one or more peers, and its declared purpose is to find problems no automated tool can: discrepancies between intent and implementation, and qualitative properties like maintainability and the relevance of comments and assertions. The goal is explicitly not to ridicule the author <sup>[1]</sup>.

There is a second, sharper reason it matters in verification specifically, and it has no equivalent on the design side. A testbench routinely acquires temporary structures that bypass large sections of a testcase to speed up debugging of a critical part, and nothing fails when those are left in — so the environment quietly stops implementing the tests it claims to contain. Peer review of the testbench against the plan, together with review of the simulation log to confirm the run followed the specification it claims to execute, is the redundancy that catches it <sup>[1]</sup>. Chapter 8's known-bad variant is the mechanical counterpart; the review is the human one.

The **coverage-model review** exists to test whether the *measurement* is honest, and it needs its own slot because a coverage model is a reflection of the plan and there is no automated way to establish that either is complete. A functional coverage model should therefore be put through the same development and review process as the plan itself <sup>[4]</sup>. It carries a roster constraint worth naming: the block's designer must be in the room, because whether a cell is reachable *by construction* is a question about the RTL. Skip the coverage-model review and the project acquires a green number that certifies stimulus rather than behaviour.

The **sign-off review** exists to *judge assembled evidence and accept what is left over*; Chapters 4 and 7 work its content. Its distinctive failure is that it arrives with no room to act on what it finds — by then, a hole discovered is a risk to be accepted, not a defect to be fixed — which is why the three reviews above have to be real.

**What makes a review productive.** Four mechanics, of which only the last is drawn from the corpus; the first three are the author's read, offered because they are cheap and their absence is diagnostic.

The artifact is circulated *before* the meeting, and the meeting opens by asking whether people read it — a review where the author narrates the document aloud has already failed, because narration is a presentation and the reviewers are an audience. Someone chairs who is not the author, so the author can answer questions

instead of managing the clock. Every finding leaves as an action with a name and a date, closed before the next gate rather than at it. And the review record is attached to the object it judged: review records linked directly to plan objects make the history retrievable during audits and quality reviews [5], which is the difference between a review that accumulates into institutional memory and one that evaporates into a meeting-notes folder.

There is a fifth mechanic that is about tone rather than procedure, and one practitioner tutorial states it about as bluntly as it can be stated: demand clear justification of how each specification point is verified, accept that the justification may be complex and needs thoughtful review — and *do not tolerate, or encourage, hand-waving* [6]. Reviews fail politely far more often than they fail rudely.

**How each rite degenerates.** The plan review degenerates when everyone present has a stake in the plan being adequate: it becomes a signing ceremony whose output is a date rather than a list of holes. The code review degenerates into style enforcement — the part a tool should already be doing — and stops asking the question that needs a human: does this environment implement the plan? The coverage-model review degenerates when the only attendees are the person who wrote the model and the lead who wants it closed. And every rite degenerates the same way under schedule pressure: the review happens *after* the decision it was meant to inform, so its findings can only be recorded as risks.

A second failure mode is subtler and worth naming because it is invisible from inside the meeting. Where a review has to span two levels of abstraction — a formal property set against a coverage model, say — the mismatch makes the review process much harder, a deep dive into individual items is often required, and the time for it has to be allowed for [6]. A review scheduled as a one-hour slot for something that needs a half-day deep dive does not produce a bad judgement; it produces no judgement, delivered on time.

**Scenario.** The crossbar's coverage-model review, week 9. The proposed model includes a cross intended to stress ID handling at a subordinate port: two managers presenting the *same* manager-side ID with writes in flight to one subordinate. It is a plausible-sounding hazard and it was written in good faith. The designer in the room kills it in a sentence: the interconnect appends source-identifying bits toward the subordinate, so the subordinate-side ID is 7 bits — the 4-bit manager ID plus the 3 bits needed to distinguish 5 manager ports — and two managers using one manager-side ID therefore arrive with *different* IDs. The cell can never be hit, by construction. **Approach.** The row is not deleted, it is *replaced*: the real precondition for the canonical write-interleaving hazard at a subordinate port is a second write address accepted while an earlier burst is still unfinished on the write-data channel, and that is what the cross must sample. The review minutes record the replacement and the reason for it, so nobody re-proposes the original at the next generation. **Why.** An unreachable cover point does not fail loudly. It sits at zero, absorbs stimulus effort during closure when time is shortest, and is eventually excluded by an engineer who no longer remembers whether it was impossible or merely unreached — while the hazard it was meant to represent stays unmodelled.

This is precisely the class of error no tool catches: the model compiles, the bin is legal, and only somebody who knows how the interconnect handles IDs can see that the question is unaskable.

The second scenario is the one Chapter 4 warns about in a single line, worked out in full.

**Scenario.** The coherent hub’s plan review. The hub’s own rows are dense and well argued: MESI state transitions, directory occupancy, the livelock and starvation cases. In the assumptions section sits one line — *“IO-coherent port behaviour under snoop-in is covered by the accelerator teams.”* Two of the three accelerator plans contain the mirror-image sentence: *“coherency is handled by the hub.”* Nobody wrote a row. **Approach.** The reviewer’s rule, the **counterparty rule**, is that an assumption naming a team is not closed until someone from that team has read it *in this review* and named the row that discharges it. Here that produces two new rows on the hub’s plan (snoop-in during an in-flight accelerator write; snoop-in to a line the accelerator has just released) and one on the plan of each accelerator team that wrote the mirror sentence. **Why.** Both sentences are individually reasonable and jointly fatal. Unspoken and undocumented assumptions are how design features fall through the cracks, which is why every assumption has to be reviewed with the designers face to face, usually over several rounds before alignment is reached [5]. The same discipline is what the counterparty rule extends: a review that reads an assumption without whoever is named in it has confirmed nothing except that the sentence is grammatical.

### 24.3 Maturity stages as a shared language

Chapter 4 defines the maturity stages — bring-up complete, verification mature, verified — and the discipline of written, demonstrable exit checklists that goes with them. This section takes them as given and asks a narrower question: what do they change about the *conversations* a team has?

Three conversations, and they change in different ways.

**“Are you on track?”** Without stages, this question is answered with a percentage and a tone of voice, and the person asking cannot audit either — Chapter 4 explains why a coverage trajectory is unreadable from outside. The organizational consequence is what matters here. A verification effort whose plan carries too little detail communicates poorly with the other teams, leaves them confused about what has actually been verified, costs the verification team their trust, and pushes everyone into using proxies for progress — coverage percentages, test counts [7]. Stages replace the proxy with an artifact: a stage claim is auditable in an hour by someone who does not work on the block, because every line of a stage checklist is a demonstration, not an estimate. The conversation stops being about whether the lead is an optimist.

**“Can I build on your block?”** The consuming team’s question needs a *comparable* answer across blocks written by different people, which is what stages supply: stage 2 means the same thing for the crossbar and the DMA and the core, and that is what makes a table of blocks against stages meaningful. The record then works as a shared contract rather than a per-team document: where a management platform holds one plan centrally and keeps every team’s view of it in sync automatically, that single plan becomes the verification contract across sites, and the document-merge exercise that otherwise substitutes for agreement disappears <sup>[8]</sup>.

**“Who needs help?”** This is the conversation stages enable that percentages actively prevent, and on the author’s read it is the one that pays for the whole apparatus. A block reporting “stage 2, four checklist lines open, two of them blocked on the formal proof of the ordering rules” is asking for a specific intervention; a block reporting “85%” is asking for nothing, because there is nothing in the number a lead can act on. Stages also make it socially possible to say *not yet* without saying *we are behind* — the stage is a state, not a verdict — and teams that cannot say “not yet” say “nearly” instead, for months.

**Scenario.** The sensor hub arrives from a partner team, delivered as a pre-verified block for integration into the reference SoC’s always-on island. The delivery note says “verification mature”. The integration lead does not argue about the claim; he asks for the stage-2 checklist with its tick marks. It comes back with the stimulus, checker and regression lines ticked and three lines blank: retention behaviour across the hub’s ultra-low-power modes, asynchronous wake events crossing into the system domain, and the functional coverage model for low-power mode transitions. **Approach.** The block is accepted at stage 2 *for the functions the checklist covers*, and the three open lines become rows on the SoC plan with the partner team named as owner and a date. Integration proceeds; the low-power sequencing tests are gated on those rows, not on goodwill. **Why.** The claim “verification mature” was not false, it was unqualified. What the checklist did was convert a trust question — do we believe this team? — into an evidence question that both sides can read the same way, and it did so without anyone having to allege anything. That is the whole social function of the vocabulary.

The pathology to watch for is **stage inflation**, and its signature distinguishes it from ordinary optimism: the stage arrives *without* its checklist. A stage claimed rather than demonstrated has become a status word, and a status word can be assigned by whoever wants it assigned — which is generally not the team. Chapter 4 names the process countermeasure; the conversational one belongs to every engineer, not just leads. Never report a stage without the open lines: “stage 2, three lines open” is a complete sentence, and “stage 2” is a slogan.

One caution from Chapter 4 bears repeating in a status meeting: a stage describes the maturity of the *verification*, not the cleanliness of the design. A block at stage 2 with fifty open bugs is reporting that the machinery which finds bugs is working — which is exactly what stage 2 claims.

## 24.4 Onboarding: designer → verifier

Verification engineers often arrive from design, or oscillate between the two roles over a career, and the verification mindset that results is very different from a design mindset — one usually reached only through years of experience, mentoring, and lessons learned the hard way <sup>[9]</sup>. That characterisation belongs to Verilab, a verification consultancy that frames the mindset as a set of concrete choices rather than an attitude — the first being whether the environment’s duty is to replicate the real world or to stress the design beyond it <sup>[9]</sup>. Two things follow, and most ramp-up programmes get both wrong. The transition is *slow*, so a two-day tool course is not a ramp-up. And it is *taught*, not read, so a wiki is not a ramp-up either.

Chapter 2 describes the destination — the mindset itself. This section is about the journey: what the newcomer already has, what they must unlearn, and what a deliberate twelve weeks looks like.

### What transfers, and it is a lot

A designer converting to verification brings assets that are expensive to acquire the other way round, and a ramp-up that does not use them wastes its best material. They can read RTL at speed. This is not a nicety: it decides how fast a failure gets localized, whether a white-box assertion is written at the right signal, and whether a proposed coverage exclusion is credible. They have micro-architectural intuition about where designs are fragile — which counters wrap, which FIFOs have a subtle almost-full, which handshakes have a reset hole — and that intuition is a bug-hunting instrument the specification cannot supply. They know how the block was *built*, and often which parts the design team itself was uneasy about. And they have standing with the design team, which matters more than it should when a verifier has to say that a failure is a design bug.

There is an organizational asset here too, and one industry taxonomy names its absence directly: under its reuse gap it lists both an IP domain knowledge gap and the failure to leverage, at one level, knowledge developed about a block at another <sup>[10]</sup>. That taxonomy is the work of Oracle Labs engineers, presented in 2015: it sorts the verification gap by tractability — inherent, transient and self-induced — and names its contributors as the complexity, skills, schedule, quality, metrics, reuse and integrity gaps, of which the self-induced part is the part a team can act on <sup>[10]</sup>. A designer moving into verification of the same block family is, on the author’s read, the cheapest available closure of that gap — provided the ramp-up asks them to use the knowledge rather than to forget it.

### What works against them

These are reflexes, not deficiencies, and each was correct in the previous job. They matter because several of them make a testbench *look* healthier while making it check less.

- **Explaining a failure instead of reproducing it.** A designer’s productive first move when something breaks is a hypothesis about the mechanism, because they



own the mechanism. A verifier's first obligation is a failure that can be replayed on demand — the seed, the manifest (Chapter 12). Hypothesis-first produces a diagnosis with no artifact behind it, and often enough the diagnosis turns out to describe a different bug from the one that fired. This is the author's read of the most-repeated correction of a designer's first months, and the ramp's first rule follows: a failure is not understood until it reproduces.

- **Interoperating rather than stressing.** The design reflex is to build the most robust component possible; applied to a verification component it inverts the purpose, because the more tolerant the testbench, the less it checks. The recognizable form is clock snooping — the testbench uses the design's internal clock instead of generating its own — and engineers coming from a design mindset fall into it because it yields fewer failing tests and faster environment bring-up, at the cost of masking clocking bugs [9].
- **Treating “invalid” as a stimulus filter.** “A transfer with that parameter is invalid, so there is no need to test it” is usually true about intent and irrelevant to verification: what matters is whether the scenario can occur, and if it can, the recovery from it is a feature [9]. Chapter 2 develops this; on a ramp-up it is worth making a rule the newcomer states out loud at their first plan review.
- **Building a testbench that recovers.** A design must survive an error and carry on. A testbench is closer to a mousetrap: once it has caught something, effort spent making it recover and keep checking is effort spent in the wrong place [9]. Newcomers build elaborate error recovery into a monitor because that is what a good component does.
- **Debugging in waveforms.** Almost all of a design's state is visible in waves, which is why designers live there. A verification component's work happens in zero time and in dynamic data structures, so the log file, not the waveform, is where it gets debugged — and a testbench that cannot be debugged from its log has a messaging problem, not a tooling problem [9].
- **Staying zoomed in.** Design work is naturally scoped to a module. Verification requires switching deliberately between zoomed-in tasks — reading the spec, writing the plan, coding, debugging — and zoomed-out ones: which features to focus on, how to tease bugs out, where effort buys the most checking and coverage. Verification also continues past tape-out, so what is done before, after, and not at all has to be prioritized consciously [9].

### A structured ramp-up

The shape below is the author's, but its load-bearing components are not.

**Give it real time, and protect the early decisions.** Engineers need time to experiment and gain experience, because poor early decisions frozen in place by schedule pressure can be hugely counterproductive [6]. A newcomer's first architectural choice will be wrong; the question is whether the project can still afford to change it when they realise.

**Pair them, and make the pair explicit.** One published industrial deployment that took formal property verification mainstream across a large graphics design organiz-

ation built a two-tier support structure rather than relying on a central team of experts. It identified a **champion** per cluster — the first point of contact and local expert for that group’s questions — with escalation to a central expertise team when a query exceeded the champion, and those champions continued to own the activity across projects <sup>[11]</sup>. The organisation is Intel, and the deployment’s scale is what makes it worth naming: 85 engineers — design engineers rather than formal specialists, the paper’s whole point being that the tools had become deployable outside a central formal team — were trained in a span of two weeks and then supported by onsite consultancy for another four, and the drive found 82 interesting bugs in twelve weeks despite beginning only after a major simulation milestone had already been completed <sup>[11]</sup>. Those are the figures the paper opens with; its later pages give slightly different totals, which is the reason to hold the opening ones rather than an average of all three. That is the durable shape of mentoring: a named local person, a named escalation path, and continuity. The same tutorial that asks for experimentation time asks for the other half of it — encourage teamwork and discussion, because this work is not intuitive and a single engineer can easily become stalled, and find the internal expertise and make sure it is *accessible* <sup>[6]</sup>. An expert nobody is allowed to interrupt is not a resource.

**Scaffold, do not lecture.** The same deployment paired its training with self-help material aimed squarely at novices: frequently asked questions, cheat sheets per application, and worked step-by-step exercises, all reachable without asking a person <sup>[11]</sup>. A curated library does the same job for code: novices learn to write assertions and coverage points by reading existing verified checkers and adapting them to their own problem <sup>[4]</sup>. Reading known-good artifacts is faster than reading documentation about them. Note where the material sat in that deployment, though — alongside dedicated training and the champion network, not instead of them. Scaffolding shortens the questions a mentor has to answer; it does not remove the mentor, which is why a wiki on its own is still not a ramp-up.

**Point the existing artifacts at the newcomer.** Two of them are already onboarding tools if the team lets them be. Enabling new teammates rapidly is a stated benefit of having a verification plan at all <sup>[7]</sup> — the plan is the document that says what the effort is *for*. And the house comment standard should already assume its reader is a junior engineer who knows the language but not the design, and who must maintain and modify it without the original designer’s help <sup>[1]</sup>. A team whose comments meet that standard has an onboarding programme it did not know it had.

**Worked example — the DMA ramp-up ladder.** Four assignments, in order, each with an exit criterion stated as a demonstration rather than a duration. The order is the point: it moves from *reproducing* through *extending* and *owning* to *breaking*, and the newcomer never writes new infrastructure before they have had to debug someone else’s.

```
Ramp-up: <name>, joining DMA verification from RTL design
Mentor: <name> | Escalation: methodology owner | Review: fortnightly
```

```
[1] Reproduce (weeks 1-2)
```

```
Task: replay 3 archived failing seeds from their run manifests;
```

write one bug report per failure at MECHANISM level.  
 Done when: all 3 reproduce on demand, and the mentor cannot tell the new reports from the originals.  
 Purpose: kills the explain-before-reproduce reflex.

[2] Extend a checker (weeks 2-4)

Task: add the per-channel error-report cross-check to the scoreboard - predict from stimulus which transfers must fail; treat the DMA's own error report as an OUTPUT to be checked, never as ground truth.  
 Done when: peer code review passed; the check fires on a deliberately broken error-report path (Ch. 8's known-bad-variant discipline) and is silent on 200 clean seeds.  
 Purpose: independence of judgment, at the smallest scale.

[3] Own an outcome (weeks 4-8)

Task: DMA-F06a (2D legalization at the 4 KB boundary) end to end: implement cg\_2d\_4kb, drive it to closure, and defend the exclusion at the coverage-closure review.  
 Done when: the 6 (., straddle, max, .) cells are recorded as a WAIVER naming the generator's c\_solver\_budget as the reason - not written off as geometry - with owner and expiry.  
 Purpose: the difference between impossible and merely decided.

[4] Break it on purpose (weeks 8-12)

Task: the MENTOR injects 3 undisclosed bugs into a private RTL copy; run the full block regression; report which were caught, by what, and how long each took to localize.  
 Done when: written result reviewed with the mentor; any bug the environment missed becomes a plan row.  
 Purpose: the environment is a design, and it is under test too.

Assignment 3 is the one that teaches judgment rather than technique, and it is deliberately given four weeks. The six excluded cells of the canonical 2D coverage cross are dead because the stimulus generator's solver budget caps a row at 2048 bytes — 256 beats on the 64-bit path — so a straddling row splits into pieces none of which can itself be 256. That is a team decision about solve time, not a property of the design, and the entire lesson is that the newcomer must be able to tell those two apart and file the first kind as a waiver that expires. Assignment 4 borrows Chapter 8's known-bad-variant discipline and inverts the usual ramp-up ending: instead of finishing with a new component nobody has audited, it finishes with a measurement of the environment.

Two things this ladder deliberately does not do. It does not open with a course on the methodology library, because a newcomer with no failure to chase has no reason to remember any of it. And it does not open with new code: the first artifact they produce is a bug report, because the report, not the testbench, is what the rest of the project consumes.

The reverse move deserves a sentence. A team that rotates people through both roles is buying the liaison position of Chapter 2 as a permanent capability rather than as an accident of hiring — the author's read, not a measured result, but the cheapest experiment on this list.

## 24.5 Process quality, and its limits

Process is the accumulated answer to “how do we not make that mistake again”, and a team without it re-derives its conventions once per project. Three instruments carry most of the weight.

**Templates** buy consistency across authors. A plan template is called crucial precisely because a large project has many individual plan authors feeding into one guide, and the template gives each the same prescriptive path through the same creation process <sup>[5]</sup>. The real product is not the section headings but the fact that ten plans can be read by one reviewer without ten context switches. The logic scales down to bug reports and up to sign-off packages.

**Coding standards** buy something narrower than teams usually claim. One published guideline hierarchy separates rules — which must be followed, because breaking them costs the productivity the methodology exists to provide — from recommendations, where the *detail is often explicitly unimportant* (a naming convention is the example given) and can be customized, but where adherence within a team is strongly recommended so that the implementation stays consistent and portable <sup>[4]</sup>. Read that carefully: for a large class of standards, the content is arbitrary and the agreement is everything. Teams that argue about the content have misidentified what they are buying. And one standard genuinely does have content: writing comments for a reader who knows the language but not the design and must maintain it without the original designer’s help <sup>[1]</sup> is a testable requirement, not a convention.

**Reusable libraries and checklists** buy a lower entry barrier and a memory. In the mainstream-adoption deployment of Section 24.4, the RTL was reviewed up front and cookie-cutter property templates were produced for the modules that recur — arbiters, state machines, standard interfaces — and packaged so that a novice met a filled-in starting point rather than a blank file <sup>[11]</sup>. Chapter 7 makes the corresponding point about sign-off checklists: they are the sediment of past post-mortems, each odd-looking line a former escape.

Process buys one more thing that is easy to miss, and the same deployment states it flatly: unless the practice is integrated into mainstream deliverables, it will not be exercised despite its benefits, which is why a set of check-points analogous to the existing milestones was introduced <sup>[11]</sup>. A technique that is not on somebody’s deliverable list does not happen. That is a strong argument for process as such, and it is an argument about calendars, not about quality.

**What it costs.** Adherence and discipline are the price, and they are charged continuously rather than once <sup>[5]</sup>. Wholesale adoption of a full methodology may simply be unaffordable for a real project with real people and schedules — an argument for adopting individual elements incrementally, not for adopting none <sup>[4]</sup>. A house-built flow multiplies the bill: the team now owns and must maintain the infrastructure, and switching methodology introduces a learning curve for experienced engineers with limited exposure to the new environment <sup>[12]</sup>. Nor is the alternative free: best-known methods simply not being in use is estimated to cost on the order of 20% of productivity on its own <sup>[10]</sup>.

### What a checklist cannot do

The line this chapter has to draw is not between good process and bad process. It is between a checklist that makes a *claim auditable* and a checklist that is asked to make a *judgement*. The first is what Section 24.3's stage checklists do, and it works. The second cannot work, for four reasons worth stating plainly.

**It cannot decide whether its own line is true.** “Functional coverage model implemented and reporting in nightly regression” is objectively true or false; anyone can go and look. “Coverage is adequate for this feature” is a judgement wearing a tick-box, and the tick tells you only that somebody was willing to make it. The rule that follows: every line of a gating checklist must name an artifact a skeptic could regenerate, and a line that cannot be written that way belongs in a review discussion instead. As one practitioner tutorial puts it about establishing completeness, tools can supply some quality metrics but there is no substitute for smart engineers thinking it through [6].

**It cannot see what is missing from the plan.** Every line audits against a question somebody already thought to ask. The neural accelerator's escape in this chapter's opening would have passed a stage-3 checklist unblemished, because no line asked about the block under memory contention. This is why Chapter 7's escape analysis exists and why its output is *new rows*: the reliable mechanism for extending a checklist is to be hurt in a place it did not cover.

**It cannot make a review honest.** It converts a judgement into a tick, which is useful for recall and useless against motive: if the room wants the gate passed, the checklist supplies a format for passing it. The listed contributors to why teams under-verify include fear-driven, miscommunication-driven and ulterior-motive-driven verification, alongside a plain lack of introspection — and that self-induced portion is the part a team can actually attack, by asking whether it is creating the gap itself [10].

**It cannot narrow the spread that matters most.** Individual productivity across engineers ranges up to tenfold [10]. Process compresses variance in the mechanical parts of the work — how a bug is filed, what a plan contains, when a regression runs — and does approximately nothing to the parts where that spread lives: which corner someone thinks to chase, whether a failing seed gets minimized or shelved, whether an odd waveform is investigated at 5 p.m. on a Friday.

There is a point past which process stops paying, and it has a recognizable shape: the artifact's existence becomes the deliverable. One published workshop describes the failure directly as *too much detail too soon* — a good-faith effort that becomes labour-intensive because every item on the feature list must be planned, every change requires a detailed plan update and every finished task requires another, with granularity so fine that tasks are measured in hours; and the result communicates *worse*, not better, because it becomes hard to tell what is finished and what is not [7]. When maintaining the process consumes the attention that the process was created to direct, the process is the problem.

**Scenario.** The radar front-end's fixed-point precision sign-off. The house checklist carries a line — *"quantization error characterized against the reference model across the operating range"* — inherited from an earlier project and ticked on every project since. The engineer ticks it: the characterization ran, the report exists, the maximum observed error is recorded. Nobody asks the question the line cannot contain, which is whether that error is *acceptable* for the CFAR stage downstream of it, whose false-alarm rate is what the customer actually buys. **Approach.** Split the line in two. One line stays mechanical and auditable: the characterization ran across the declared range, and here is the report. The second is not a checklist line at all — it is a named review item, with the DSP architect present, asking whether the measured error meets a tolerance that was written down *before* the run. **Why.** Mixing the two kinds of obligation in one box is how a checklist launders a judgement nobody made. The mechanical half was always true; the half that mattered had no owner, no threshold, and no record of ever having been considered.

## IN PRACTICE

The messy parts, which no process document admits to.

- **Roles are assigned by availability.** The "integration verifier" is frequently whoever was free when integration started, and the outcome map is drawn afterwards to match the staffing. Teams that do better write the outcome list first, staff against it, and treat an unowned outcome as an open issue rather than an organizational detail.
- **Review invitations expand until the meeting is useless.** Twelve people cannot read a plan together. The compromise most teams converge on is two meetings — a small working review that produces the findings, and a wider one that walks them and closes the actions — rather than one that does neither.
- **The champion model works until the champion is promoted.** The published version has champions continuing to own the activity across projects <sup>[11]</sup>; the version most teams live with loses the local expert to a lead role and finds the expertise never left that person's head. The cheap insurance is that every champion's answers become entries in the self-help material, which is the part that survives a reorganization.
- **Onboarding is deferred because it costs the mentor more than the newcomer.** A twelve-week ramp-up spends perhaps two weeks of a senior engineer who is always on the critical path. Teams that ramp people successfully book that time as an explicit deliverable; teams that do not discover in month four that the newcomer has been quietly assigned only work nobody has to explain.
- **The talent question is not local.** The 2024 industry study raises concern about the talent gap in design and verification engineering as one of its headline themes <sup>[3]</sup>. A team that cannot ramp people is competing for the small pool that arrives ready.



- **A coding standard nobody enforces mechanically decays within two projects.** The standards that survive are the ones a linter or a continuous-integration job checks; the rest become folklore new hires learn by having a patch rejected.

---

## PITFALLS

1. **Ownership by artifact.** *Symptom:* after an escape, everyone can prove they did their job. *Cure:* state each owner's charge as an outcome about the design, and give every assumption a person rather than a sentence.
2. **The review with no pre-read.** *Symptom:* the author narrates the document while the reviewers meet it for the first time. *Cure:* circulate the artifact in advance, open by asking who read it, and reschedule rather than proceed.
3. **Onboarding by documentation.** *Symptom:* "read the wiki and shout if you're stuck." *Cure:* a named local expert with a named escalation path <sup>[11]</sup>, plus an assignment ladder whose first deliverable is a reproduced failure.
4. **The stage as a status word.** *Symptom:* a stage reported without its open checklist lines, and assigned by whoever needs it. *Cure:* never report a stage without the open lines; treat the transition as a reviewed event.
5. **The judgement laundered into a tick.** *Symptom:* a checklist line containing the word *adequate*, *sufficient* or *acceptable*. *Cure:* split it — the auditable half stays on the checklist, the judgement becomes a named review item with a threshold agreed in advance.
6. **The single accessible expert.** *Symptom:* one person answers every hard question, and their queue is the project's real critical path. *Cure:* champions per team with an escalation path, and every answer written into the self-help material <sup>[6, 11]</sup>.

---

## SUMMARY

- Define roles by outcome, not by artifact. "Owns the testbench" is discharged by a good testbench; "owns whether this feature is demonstrably correct" is discharged only by evidence about the design — and only the second survives an escape post-mortem.
- The three seams where ownership fails are block-to-system, design-to-verification, and features whose owner is an assumption. The first is structural: at integration each block is a black box to the team that must debug the system <sup>[2]</sup>.
- Each review rite has a distinct purpose — completeness (plan), intent-versus-implementation (code), honest measurement (coverage model) — and each has a distinct way of dying. Peer review of the testbench is the redundancy that catches the bypassed testcase nothing fails on <sup>[1]</sup>. Review records linked to the plan objects under review are what keep a review's history retrievable <sup>[5]</sup>.
- Maturity stages change status conversations from confidence to evidence, and make "not yet" sayable. Never report a stage without its open checklist lines.

- Designers arrive with RTL fluency, micro-architectural instinct and credibility; the reflexes that work against them — interoperating instead of stressing, treating “invalid” as a stimulus filter, building a testbench that recovers — all make an environment look healthier while checking less <sup>[9]</sup>. Add one this book supplies: explaining a failure before reproducing it.
- A ramp-up is mentored, scaffolded and sequenced: reproduce, extend, own, break. Its first deliverable should be a bug report, not new infrastructure.
- Process buys consistency, a lower entry barrier and a calendar slot — and a practice absent from the deliverable list does not happen <sup>[11]</sup>. It cannot supply judgement, see what the plan omitted, or make a review honest; nor can it narrow the up-to-tenfold spread in individual productivity <sup>[10]</sup>.

---

## FURTHER READING

**From the corpus.** <sup>[9]</sup> is the indispensable read for anyone converting from design and the source of most of Section 24.4’s reflexes; Chapter 2 develops the mindset it describes. <sup>[11]</sup> is this corpus’s best account of taking a technique mainstream inside a large organization — champions, escalation, self-help scaffolding, cookie-cutter libraries, and the observation that a practice absent from mainstream deliverables is not exercised. <sup>[6]</sup>’s planning-and-completion slides are unusually candid about people: experimentation time, teamwork against stalling, accessible internal expertise, and the refusal to accept hand-waving in a review. <sup>[1]</sup> carries Section 24.2’s code-review material (Chapters 2 and 3) and the comment standard aimed at a junior maintainer (Appendix A); <sup>[4]</sup> supplies the requirements-review roster, the guideline hierarchy behind Section 24.5, the novice’s route in through existing verified checkers, and the rule that a coverage model is reviewed like a plan. <sup>[5]</sup> and <sup>[7]</sup> are the planning workshops behind the template, assumption-review and over-detail material; <sup>[8]</sup> shows the plan working as a contract across distributed teams. <sup>[2]</sup> is the source for the integration team’s black-box problem, <sup>[10]</sup> for the skills, quality, reuse and integrity gaps, and <sup>[3]</sup> for the 2024 workforce and talent-gap data.

**Not in the corpus** (pointers, not evidence). The OpenTitan project publishes its per-block verification stage checklists openly, and the OpenHW Group publishes its verification process documents; both are accessible worked examples of the artifacts this chapter describes, and neither is citable here. Weinberg, *The Psychology of Computer Programming* (1971), on why independent review works on people rather than only on code. Wile, Goss & Roesner, *Comprehensive Functional Verification* (Morgan Kaufmann, 2005), treats the verification engineer’s role and its ramp at industrial scale. Fagan’s 1976 *IBM Systems Journal* paper on design and code inspections is the origin of the formal software inspection, whose separation of roles and defect-recording discipline Section 24.2’s mechanics informally reconstruct.

---

## REFERENCES

1. **B2** J. Bergeron, "Writing Testbenches using SystemVerilog," Springer, 2006.

2. **D20** A. Meyer and H. Foster, "Metrics in SoC Verification: Not just for coverage anymore," Proc. DVCon U.S., 2012.
3. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
4. **B1** J. Bergeron, E. Cerny, A. Hunter, and A. Nightingale, "Verification Methodology Manual for SystemVerilog," Springer, 2006.
5. **D9** B. Ehlers, C. Vargas, and P. Carzola, "Best Practices in Verification Planning," Proc. DVCon U.S., 2013.
6. **D13** J. Bromley and J. Sprott, "Formal Verification in the Real World," Proc. DVCon U.S., 2018.
7. **D7** P. Marriott, J. Vance, and J. McNeal, "Proven Strategies for Better Verification Planning," DVCon 2022 Workshop, 2022.
8. **D26** D. Zhang, "Smarter Verification Management with vManager Platform," Proc. DVCon China, 2021.
9. **D21** J. Montesano and M. Litterick, "Verification Mind Games: how to think like a verifier," DVCon, 2014.
10. **D16** R. Narayan and T. Symons, "I created the Verification Gap," Proc. DVCon U.S., 2015.
11. **D18** M. V. Achutha Kiran Kumar, E. Seligman, A. Gupta, Ss Bindumadhava, and A. Bharadwaj, "Making Formal Property Verification Mainstream: An Intel Graphics Experience," Proc. DVCon U.S., 2017.
12. **D22** S. Hristozkov, J. Pallister, and R. Porter, "No Country For Old Men – A Modern Take on Metrics Driven Verification," Proc. DVCon Europe, 2021.

---

## 25. AI and ML in Verification

Two proposals reach the verification lead in the same week, and both arrive with a number.

The first comes from the team that owns the video codec’s nightly regression. They have trained a classifier on eighteen months of failure logs; it sorts each morning’s failures into buckets and names a probable cause for each. On a held-back month of history it agreed with the human triage engineer 88% of the time. The proposal is to put it in front of the triage rota.

The second comes from a pilot on the crypto engine. A language model was given the engine’s specification chapter and produced forty SystemVerilog assertions in about twenty minutes. Thirty-eight compile. Thirty-six pass a formal proof against the current RTL. The proposal is to adopt the flow for the remaining blocks.

Both numbers are real. Only one of them is evidence about the thing being proposed.

The first is measured against a baseline the team owns: yesterday’s triage decisions, made by a person, written down, and available to disagree with. It can be re-measured next quarter on next quarter’s failures, and if agreement falls the team will see it fall. The second measures agreement between a generated property and the design it was generated to check. An assertion that agrees with the RTL is precisely what an assertion must not be judged by — Chapter 11 spends a section on why — and a property whose antecedent this design never satisfies passes on it without ever having been evaluated. Thirty-six passing proofs is a statement about compilation and consistency. It is not a statement about intent, and intent is the only thing a specification-derived property exists to capture.

That asymmetry is this chapter’s subject. Two risks attend a chapter like this one — that it ages into embarrassment, and that an engineer who has been promised a great deal and shown less reads it cynically and stops — and both are managed the same way: by separating what has been *demonstrated* from what is *claimed*, and by dating everything. A vendor demonstration is not evidence. A published result on a benchmark is evidence for that benchmark, under that benchmark’s definition of success, which is often the most interesting part of the paper. Where a result comes from an industrial deployment, its scope and its caveats are the finding rather than a footnote to it.

The field’s own reviewers say as much. A 2025 review of machine learning for micro-electronic design verification observes that although these techniques have been proposed since the late 1990s, many never reached mainstream adoption, and names three reasons that are about evidence rather than capability: withheld code and data make results hard to reproduce, techniques are evaluated on simple designs against random stimulus and little else, and research rarely asks whether a method generalises past the one application it was tried on <sup>[1]</sup>.

---

## CHAPTER MAP

- §25.1 identifies which machine-learning techniques have a track record in verification, what each one actually optimises, and what it costs to keep working once deployed.
- §25.2 states how to read a capability claim about generated verification artifacts: what was measured, on what, against what baseline, and what the paper meant by *correct*.
- §25.3 establishes why a generated artifact is plausible by construction and why that makes a wrong one more expensive than none.
- §25.4 states what a review of machine-written code must check that a review of human-written code need not, and where a person must remain in the loop — expressed as named decision points with owners rather than as reassurance.
- §25.5 states what to record so that a sign-off resting partly on generated evidence is still defensible two years later, and how to decide what to adopt, what to pilot and what to wait on.

## 25.1 Classical machine learning, where the results are real

---

Three applications have a genuine track record: steering stimulus toward uncovered behaviour, sorting regression failures, and choosing which tests to run. They are the least glamorous techniques in this chapter and the most deployed, and their published results span two decades rather than two years. The value in reading them now is not the numbers. It is the shape of the numbers: each technique optimises something narrower than what you want, and each one decays.

### Coverage-directed generation: what it optimises

Chapter 6 places learning-based coverage-directed generation in the automation spectrum and states the structural point — every such technique learns the mapping from generator directives to achieved coverage and inverts it, which means it optimises the derivative rather than the goal. This chapter asks when it transfers.

The founding result is worth reading in full because it is unusually honest about cost. In 2003 a Bayesian network was trained to direct stimulus at the storage control element of a commercial mainframe. The coverage model had five attributes with a cross product of 13,888 combinations, of which 1,968 were legal coverage tasks. Training used 160 test-cases, each taking more than thirty minutes to run, and covered 1,745 of them; 223 remained. The trained network covered all 223 within 250 further test-cases, where an expert-written directive file covered two-thirds of them after more than 400 [2].

Read the second half of that sentence before the first. The baseline was a human expert's tuned directive file, not uniform random stimulus — which is rare, and is what makes the result mean something. The 2025 review quoted above notes that random is the most common baseline in this literature and cautions that beating random says

nothing about whether a technique generalises [1]. The cost follows from the training figures above: more than eighty hours of simulation before the model produced its first useful directive, on top of building a coverage model precise enough to learn from.

Nine years later, a review of the whole literature found a limitation common to every approach it surveyed — none could consistently cover all the tasks in a coverage model — and named an adoption barrier that has aged well: the average verification engineer is not an expert in these techniques and cannot tune one, so a method needing customisation will not be supported, and an approach fit for industry must produce results engineers can easily interpret [3].

The modern instances behave the same way at a different scale. Chapter 6 records the 2023 deep reinforcement-learning result on a compression encoder, where an agent reached 133 of 136 hard functional bins; uniform-random stimulus plateaued at 28. The operational detail that chapter does not need is this: closure required merging the coverage of two separate training runs, because neither alone hit every bin — a 500-episode run reached 133, a 750-episode run reached 127, and the second run’s bins were not a subset of the first’s [4]. The technique bought the hard bins. It did not buy closure, and a plan that had promised closure would have been wrong.

The ceiling reappears when a language model is put inside the loop. A 2025 survey describes a 2024 system given the coverage report and the design under test and asked to propose stimulus; it beats random testing on simple and medium designs and degrades above roughly sixty-four states, which the survey reads as an argument for combining language models with structural approaches rather than replacing them [5].

Put that against a real target. The GPU’s shader clusters are the natural place to want this: four clusters, a scheduler, and a coverage model whose interesting bins are combinations of occupancy, divergence and memory-request pattern that nobody wants to enumerate by hand. The fit is good because the space is large, the legal directives are many, and a bin is cheap to check. It is a poor fit for the crypto engine’s key ladder, where the interesting states are few, reachable by a directed test in an afternoon, and defined by a security argument that no coverage bin expresses.

### **Failure triage and clustering**

Chapter 12 owns the mechanism: signatures, buckets, the rota, and the three reductions that make a morning’s failures tractable. What machine learning adds there is worth separating into two halves that behave very differently.

A 2023 survey of machine-learning applications in functional verification reports both halves of a 2018 study working from one feature set — 616 features drawn from semi-structured simulation logs. Unsupervised clustering of failures scored 0.543 and 0.593 on adjusted mutual information across three algorithms, against an ideal of 1.0. Supervised classification of the root cause, on the same feature set, reached 90.7% accuracy and an F1 score of 0.913 with a random forest [6].



That gap is the practical lesson, and it points the wrong way for anyone hoping to buy their way out of triage. Supervised classification is strong and needs a labelled history: somebody must already have looked at hundreds of failures and written down what caused each. Unsupervised grouping needs no labels and is weak. The technique therefore works best where a team already has years of curated triage decisions, and worst on a new project or a block's first tape-out — which is where triage hurts most. The same survey ladders the problem into three — cluster failures by cause, classify the cause, suggest the fix — and reports that most research addressed the first two, with no results available on the third <sup>[6]</sup>.

The economics follow. That survey also reports a typical machine-learning project spending up to 70% of the team's effort on preparing data <sup>[6]</sup>. On the video codec's nightly, that effort is not hypothetical: the failure population is dominated by golden-model mismatches whose useful label is a frame region and a coding tool, and neither is in the log unless somebody puts it there. Chapter 12's point — that the triage engineer's confirmation *is* the training data — is what makes the labelling affordable, and only if that confirmation is recorded in a form a model can read.

### Regression compression and test selection

Chapter 12 carries the published figures for ranked regressions and novelty-based selection with their baselines. This section adds only what each optimises, because that is what decides where it belongs.

A ranking model optimises time-to-first-failure over a known suite: it reorders existing test-and-seed pairs so the runs most likely to fail go first. A selection model optimises coverage per simulation-second, discarding runs whose contribution it predicts is redundant. Neither optimises the discovery of new behaviour, and the industrial study behind the ranking figures says so in its own words — a ranked regression simulates no new scenarios and is not recommended for exploring the random space or exposing new bugs <sup>[7]</sup>.

That is a constraint on where the technique can sit, not a caveat to file away. A ranked or compressed regression is a *change-detection* instrument: it answers “did this commit break something the suite already knew how to catch?” quickly and cheaply. It is not a bug-finding instrument, and a program that replaces its full random regression with a compressed one has quietly converted its main source of new bugs into a fast regression check. The honest deployment keeps both — a compressed suite per commit, and a full sweep with fresh seeds on a slower cadence, whose yield Chapter 7 tracks as its own signal.

### The stale model

Every technique in this section shares a decay mode, and it is why so many published results do not survive contact with a program.

A model trained on one design's coverage response, one quarter's failure logs, or one release's regression outcomes is a claim that tomorrow resembles yesterday. The 2025 review names this directly: an example that was novel becomes unremark-

able once more examples have been seen, so the model needs regular retraining to stay relevant — and worse, once deployed, the learner shapes the examples it will later be retrained on, which can suppress exploration and make performance *decrease* over time. The same review observes that research usually frames verification as a one-time learning problem while industrial designs change continuously, and warns that if substantial effort goes into crafting fitness or reward functions, the technique has merely shifted effort from writing tests to tuning models [1].

Two consequences for anyone deploying one. Retraining is a named, owned, scheduled task with a cost, not a maintenance detail: a triage classifier 90% accurate at tape-out and 60% accurate two derivatives later is worse than none, because the rota has stopped checking it. Give it a measured agreement rate, re-measure on a fixed cadence against human decisions, and publish the number where the people relying on it can see it.

And there is a distinction in severity that the word *model* hides. A stimulus model that goes stale mis-steers: you get worse coverage per run, and the coverage report tells you. A generated checker that goes stale is a bad oracle — it decides whether an observed response is correct — and a bad oracle does not announce itself in any report. That difference is why the rest of this chapter is mostly about generated checkers.

## 25.2 Large language models, with the evidence

---

Three capabilities are claimed for language models in verification: generating assertions and properties, scaffolding testbench code, and drafting verification plans. The corpus supports statements about all three, but not statements of the same strength, and the differences matter more than the headline figures. Every claim below carries its year, the artifact it was measured on, and — the part usually left out of the summary — what the study counted as success.

### Assertions and properties from a specification

Chapter 11 records the headline result: a 2024 framework that reads a full natural-language specification and emits SystemVerilog assertions reported 89% of 56 generated properties correct both syntactically and functionally on a single serial-bus design, where *correct* means the property passed a formal proof against known-good RTL, after human engineers had removed the generations referencing signals the tool could not map [8]. Two things that chapter does not need are load-bearing here.

The first is the ablation. Given the same specification files, the underlying general-purpose model on its own produced 71 candidate properties, of which 23 were syntactically correct and 6 passed the proof — 8% by the same measure that gave the full framework 89% [8]. The engineering around the model — splitting the specification, mapping signal names, retrieving the right passage per property — is doing most of the work, and it is the part that does not transfer for free to a specification written in a different house style.

The second is scope. Of the three property classes the framework generates, connectivity properties could be produced only for control signals, because the connectivity of data signals depends on internal RTL structure the specification does not describe; and function properties only where the specification detailed the register behaviour [8]. The reachable subset of a specification is narrower than the specification, and a generator will not tell you which rows of your plan it left untouched.

A 2025 study attacks a different target and shows why comparisons across this field are so hard to make. Its subject is protocol formal specifications — documents with timing diagrams — and its flow is two-stage: generate candidate properties from a grammar, check each against the timing diagram with a SAT solver, then use a language model to filter the survivors against the textual description. The SAT stage retains between 5% and 22% of the candidates in every case, which is the clearest available measure of how aggressively a mechanical filter can thin a candidate set before anyone reads it. Across six open-source on-chip protocols, the extracted properties covered 53 of 58 manually derived target properties — for every protocol but one, which is excluded from that count [9].

That paper also runs the specification-driven framework of the previous paragraphs on one of its own protocols, and the two results must not be read as a curve. They differ in task and in artifact class: the first framework was built to consume a full design specification and was measured on one design under its own pass criterion; the second target is a protocol formal specification with timing diagrams, and the head-to-head re-ran that framework itself, reporting it under two underlying models. On that protocol the two-stage flow covered all 13 manually derived properties with no syntax errors, while under one of them the specification-driven framework produced 38 properties of which 3 were judged correct and 2 of the 13 targets covered, with 16 syntax errors and 11 references to signals that do not exist [9]. The conclusion is not that one framework is better than another by some ratio. It is that a framework built for one artifact class does not transfer to another, and that the eleven non-existent signals track the model rather than the framework: under the other model it produces none.

That study's taxonomy of outcomes is worth borrowing wholesale, because it is the vocabulary a reviewer needs: a generated property may be *correct* (unique or redundant), *incorrect*, *unsure*, or *hallucinated* — invented rather than drawn from the set it was given to filter. One protocol's transaction section produced four hallucinated properties [9].

Its authors' limits are stated plainly: the method depends on unambiguous timing diagrams; on the complex protocols the target set does not cover the entire formal specification; models remain poor at reading figures, often getting a signal's value in a given cycle wrong; and experienced engineers are still better at inferring properties where the document has no text to work from [9].

### **Agentic flows: a plan, a proof, and a loop**

The corpus contains one industrial report of an end-to-end agentic formal flow, from 2025: a lead agent drafts a verification plan in natural language, a dispatcher deleg-

ates assertion generation, critic agents evaluate the results, properties are proven in a commercial model checker, counterexamples are analysed, and formal coverage is assessed for sign-off. Its headline is that the flow does not guarantee a result on every run and works most of the time, at an overall efficacy of around 40% <sup>[10]</sup>.

Before reading that as good or bad, read what was counted. The study's first-named indicator is a *success rate*, defined as the number of runs that completed end-to-end formal verification out of the total runs <sup>[10]</sup> — a measure of whether the pipeline finished, not of whether the properties it produced were the right ones.

On this flow those are different questions. The authors list vacuous passes in formal properties among the failure modes they observed, alongside syntactically incorrect or semantically irrelevant assertions, assertions referencing signals absent from the specification, and truncation of large RTL against the model's context limit <sup>[10]</sup>.

The benchmark's scale matters too: fifteen designs across three difficulty tiers, where the *advanced* tier holds a Booth multiplier, a pipelined adder, a small RISC-V core, a PID controller and a round-robin arbiter <sup>[10]</sup>. These are block-scale exercises, and a 40% figure should be read as 40% on designs of that size. The report is Infin-eon's and the flow has a name, Saarthi; the designs in its benchmark are not Infin-eon products, which is exactly the distinction between a company reporting a capability and a company reporting a deployment <sup>[10]</sup>.

The result is also a function of which model was used, by an amount that should govern how it is quoted. The same flow was run with three general-purpose language models. On the easiest tier they completed 17.78%, 43.19% and 48.77% of runs end to end; on the hardest, 4.33%, 9.00% and 41.43%. The two models within six points of each other on the easiest tier were the 9.00% and the 41.43% on the hardest <sup>[10]</sup>. That is not a property of the technique but of the technique paired with one model, and Section 25.5 makes the pairing part of what gets recorded.

The flow's most transferable idea is not in its numbers. When two agents loop on generating and correcting an assertion without converging, it stops them at a threshold of five iterations and asks a human for the correct assertion <sup>[10]</sup> — a designed hand-off point rather than a fallback.

## Debug and root cause

Debug is where verification time actually goes — Chapter 7 counts debug-time share among its five instruments, and calls it the one that goes missing most often — which makes counterexample debugging the highest-value target in this list. A 2025 system attacks it structurally: it converts the failure trace into a causal graph, scans the graph for suspicious nodes, explores them agentically into a causal narrative, and proposes an RTL fix. On a benchmark of 38 real debugging challenges with human-verified ground-truth fixes, the full system scored 0.956 on best-hypothesis quality, and its proposed fixes resolved the failure 71.1% of the time on the first attempt and 86.8% within five <sup>[11]</sup>. The system is NVIDIA's and is called FVDebug. The benchmark is where those two figures were earned; the paper's other case studies are proprietary designs whose details are masked, so nothing in the pair licenses a claim about what the tool does on a production part <sup>[11]</sup>.

Read this paper closely for its two families of metric and its explicitness about their difference. Hypothesis quality is scored by a language model comparing the hypothesis against the ground truth; Pass@k is measured mechanically, by applying the proposed fix to the RTL and re-running the proof in a model checker <sup>[11]</sup>.

The two do not track each other, and the paper's own baseline table shows it. One unstructured baseline scored 0.783 on best-hypothesis quality and resolved 65.8% of failures within five attempts; another scored 0.474 — the worst hypothesis quality in the table — and resolved 81.6% <sup>[11]</sup>. A model-judged score measures how convincing an explanation reads. It is not a proxy for whether the fix works, and where a study reports only that kind, you have been told how plausible the output is.

Which is what happens on the harder designs. Taken to two counterexamples from a larger open-source application-class processor — one a load-store unit failure spanning 1,918 lines of RTL across seven files — best-hypothesis quality falls to 0.713 and no Pass@k is reported at all: the authors state that automated fix generation for such multi-line issues remains future work <sup>[11]</sup>. That is a candid result. It says the causal-graph machinery still finds its way around a graph of 170 nodes, and says nothing yet about fixes at that scale. That processor is CVA6, and both failures were ones the AutoSVA project had found first, which is what makes this the checkable half of the paper: the RTL and the counterexamples are public <sup>[11]</sup>.

## Where the corpus is thin

On testbench scaffolding and verification-plan drafting — the capabilities most often demonstrated and least often measured — this corpus contains no study that scores the artifact itself; the flows that draft a plan are graded only by downstream outcomes <sup>[10, 12]</sup>. The nearest adjacent evidence is negative: the 2023 survey of machine learning in functional verification found that code summarisation in design verification had not been reported in any literature at that point, and that no research had been published on coding assistance for this domain <sup>[6]</sup>.

The obstacles a 2025 survey of language models in electronic design automation names are structural rather than incidental: models struggle with long inputs con-

taining long-range dependencies, which is what a testbench and a specification both are; and hardware description languages and tool-control scripts are scarce in pre-training data compared with mainstream software languages, so the domain gets less of the fluency that makes these models useful elsewhere. That survey concludes that scalability on modern designs and the role of human oversight in verification remain the significant hurdles [5].

The practitioner’s answer is judgment rather than a citation: generated scaffolding pays where a mechanical check independent of the generator can confirm it. A register block whose fields are already machine-readable can have its checker generated and then diffed against the field description — the diff is the evidence, and it does not care how the file was produced. A constraint block encoding a legalisation rule cannot be checked that way, and its review costs what it costs.

### The evidence table

Each row of Table 25.1 pairs one reported capability with the conditions that produced it. The fourth column is the one to read first; it is where most summaries of this field go wrong.

**Table 25.1** — What language models have been reported to do in this field, each capability paired with the artifact it was measured on, the year, and the study’s own definition of success. The fourth column is the one to read first. Two rows describe one system twice on purpose, because one of its numbers was awarded by a language model and the other by a model checker.

Capability	Reported result and year	Measured on	What counted as success	Stated limits
Assertions from a full specification document	89% of 56 properties correct, 2024	One serial-bus design, 23 signals	Passing a formal proof against known-good RTL	Generations referencing unmappable signals were removed by human engineers before scoring; connectivity properties limited to control signals [8]
The same task without the surrounding framework	6 of 71 candidates passed, 2024	The same design and specification	The same proof criterion	Reported by the framework’s own authors as the baseline for their contribution [8]
Properties from a protocol specification with timing diagrams	53 of 58 target properties covered, 2025	Five of six open-source on-chip protocols; the sixth is outside that count	Covering a manually derived target property, after a SAT check and a model-based filter	The SAT stage retains 5–22% of candidates; on the complex protocols the target set is not the whole formal specification [9]



Capability	Reported result and year	Measured on	What counted as success	Stated limits
End-to-end agentic formal verification	17.78–48.77% of runs completed on the easiest tier, 4.33–41.43% on the hardest, 2025	15 block-scale designs, three difficulty tiers, three models	A run that completed the flow end to end, not a property that was correct	Authors summarise overall efficacy at around 40%; observed failure modes include vacuous passes, irrelevant assertions and context truncation on large RTL [10]
Root-cause hypotheses for a counterexample	0.956 best-hypothesis quality, 2025	38 debugging challenges with human-verified fixes, one model	Similarity to the ground-truth cause, scored by a language model	The score is model-judged, and does not order configurations the same way the mechanical measure does [11]
Fixes for a counterexample	71.1% first attempt, 86.8% within five, 2025	The same 38 challenges, the same model	The fix applied to the RTL and the proof re-run	Not reported on the two application-class processor failures; the authors call fix generation at that scale future work [11]
Testbench scaffolding and plan drafting	No measurement of artifact quality in this corpus	—	—	The corpus flows that draft a plan are graded only by downstream outcomes [10, 12]

Two rows describe the same paper’s system twice, on purpose. A row that merged them — *root-cause analysis, 0.956 quality and 71.1% fix rate* — would be true in both halves and wrong as a whole, because it would hide that one number was awarded by a language model and the other by a model checker.

## 25.3 Plausible by construction

A generated artifact is plausible by construction. That is not a criticism of the technology but a description of what it optimises: a model trained to continue text produces the continuation most consistent with the text it was given, and a property that reads like one written by a competent engineer is exactly what it is built to emit. Correctness is not the objective function. Plausibility is, and correctness is a frequent by-product.

Consider one sentence from the crypto engine’s specification and the property a generator returns for it.

```
// From: "While the engine is holding a request that the memory path has
// not accepted, the key slot selector must not change."
property p_keysel_stable_gen;
 @(posedge clk) disable iff (!rst_n)
 (req_valid && !req_ready) |-> $stable(key_sel);
endproperty
a_keysel_stable_gen : assert property (p_keysel_stable_gen);
```

Read it as a reviewer would: the signals are the right signals, the guard is present, the handshake condition is the one the sentence describes, and the operator is the one the word *stable* suggests. It is wrong, and it is wrong in two directions from one missing delay.

`$stable` compares the sampled value at the current tick against the previous tick, so at the first cycle in which the request is held — call it the cycle the request rises — the property demands that `key_sel` already had its current value one cycle *earlier*, before the request existed. A design that legally presents a fresh request with a new key slot in the cycle immediately after a previous transfer completes will fail a rule the specification never stated. In the other direction, on the last held cycle the property says nothing about `key_sel` in the following cycle, the one in which the request is finally accepted — which is the cycle where a change would do actual damage. The obligation the sentence expresses lands one tick later than the property places it:

```
property p_keysel_stable;
 @(posedge clk) disable iff (!rst_n)
 (req_valid && !req_ready) |-> ##1 $stable(key_sel);
endproperty
a_keysel_stable : assert property (p_keysel_stable);
c_keysel_held : cover property (@(posedge clk) disable iff (!rst_n)
 (req_valid && !req_ready)[*2]);
```

The cover is the antecedent's, strengthened: it asks for a two-cycle hold rather than one, because a hold of a single cycle exercises only the accepting beat, and it is the longer hold that shows the property constraining `key_sel` while the request is still waiting.

This is not an invented failure mode. A 2025 study of property generation from protocol documents opens with the same defect on a generic valid/ready interface — a stability requirement rendered without its one-cycle delay — and notes that earlier machine-learning work made exactly that mistake [9].

Now ask what a formal proof would say about the first version. It depends entirely on whether this pipeline happens to hold `key_sel` steady for a cycle before a request rises. If a register upstream loads the selector one cycle early, the wrong property is proven, on RTL that is behaving correctly, and under a pass criterion of *proved against known-good RTL* it is scored correct and enters the suite. The verdict a proof engine returns on today's RTL is a verdict about today's RTL. It is not a verdict about the property.

Chapter 11 already names three ways an assertion can be worthless: it never triggers, it restates the design, or it cannot distinguish a correct design from an incorrect one. A generator produces all three fluently, and the second is structural rather than accidental. Where an assertion is mined from the design's own traces, generation and evaluation both happen against the same RTL with no independent reference, so a flaw in the RTL becomes a flaw in the assertion that the method has no way to detect — and a method that reads the specification *and* the RTL together in-

herits the same weakness [8]. A property derived from the design cannot find a bug in that design; it can only restate whatever the design does, bug included.

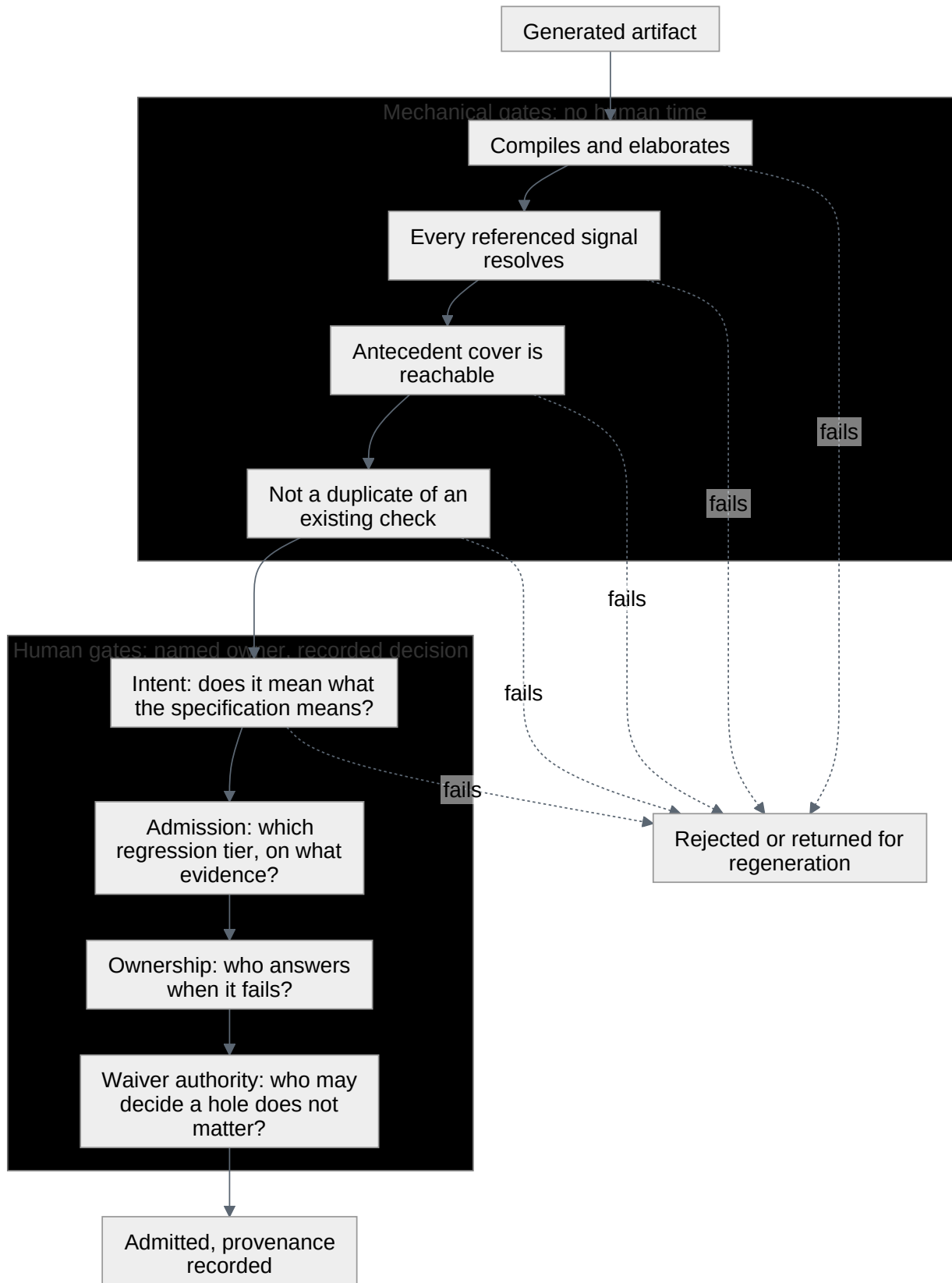
A fourth failure mode belongs to generated artifacts specifically, and it changes how they must be reviewed. A human writing forty properties makes idiosyncratic mistakes — a typo here, a misread sentence there — uncorrelated across the batch, so sampling five of the forty is defensible: the sample estimates a rate. A generator’s mistakes are systematic. One misreading of one recurring specification pattern — *while X holds, Y must not change* — reproduces identically in every property derived from that pattern, and a sample of five either catches the whole class or misses it entirely. Sampling estimates nothing. This is practitioner judgment rather than a published result, and it is the most consequential difference in review practice this chapter contains.

The cost of a wrong-but-plausible artifact is not the cost of finding it; it is the cost to everything around it. A verification suite has collective authority: engineers believe a passing regression because they believe the checkers in it were written by someone who understood the design. One property that looks careful and encodes the wrong rule spends that credit, and the next one spends more. When the reviewer’s scepticism runs out, the outcome is not that generated properties get rejected — it is that all properties get skimmed. An empty slot in a plan is visible and gets filled; a wrong property in a suite is invisible and gets trusted, which is why the gates below sit before admission rather than after.

## 25.4 The human gates

---

“Keep a human in the loop” is advice with no operational content. A loop has places in it; the useful question is which ones a person occupies, what they are deciding there, and what happens when nobody has been named to decide. [Figure 25.1](#) names four decision points that need a person, and separates them from the checks a machine can settle on its own. Each has a question, an owner, the evidence that owner needs, and the specific damage of leaving it unowned.



**Figure 25.1** — The gates a generated artifact passes before it is admitted to a suite: four mechanical checks that cost no human attention, then four human gates, each with a named owner and a recorded decision. A failure at any mechanical gate, or at the intent gate, sends the artifact back rejected or for regeneration. The ordering is what makes the human gates affordable, since no artifact reaches a person until the machine has finished with it.

The order in the figure is the point: everything a machine can decide is decided before a person spends attention, and no human gate is reached by an artifact that has not survived the mechanical ones. That is not a courtesy to reviewers; it is what makes the human gates affordable at all.

**Gate 1 — intent.** *Does this artifact mean what the specification means?* The owner is whoever owns the plan row it discharges, not whoever ran the generator, and Chapter 5’s traceability makes that assignment unambiguous. They need two things: the specification sentence the artifact came from, side by side with the artifact, and a demonstration that the artifact can fail — the cover on its antecedent, a mutation, or a deliberately broken variant. A passing proof is not evidence at this gate; Section 25.3 is entirely about why. Left unowned, a wrong rule enters the suite and immediately inherits the suite’s authority.

**Gate 2 — admission.** *Which tier does this run in, and how does it get promoted?* Chapter 12 puts a repeat-N stability check on any test entering a gating tier, to establish that it does not fail spuriously. Generated checks want the same discipline for a different reason: not “does it fail at random?” but “does it fail on things that are not bugs?”. A generated property earns a gating tier by running clean across a window of known-good history, and that window is the evidence. Left unowned, the first false failure on a Monday morning teaches everyone to add a plusarg.

**Gate 3 — ownership on breakage.** *Who answers when this fails at 07:40?* Named at merge, in the file, not discovered at the first failure. The hazard is specific to generated code and easy to underrate: a component nobody wrote is a component nobody can reason about under time pressure. The engineer on the rota can read it but cannot recall what it was *for*, because no hour was ever spent deciding. The remedy is to require that the accepting engineer be able to explain the artifact without regenerating it — and to record that they did.

**Gate 4 — exclusion and waiver authority.** *Who may decide a hole does not matter?* A coverage exclusion proposed by a model is still a waiver in Chapter 7’s sense, carrying the same five fields: the hole, the reason, the risk argument, an owner, an expiry. Nothing about its origin changes who signs it. This gate exists because exclusion is where an optimising system will always push — to a coverage metric, excluding a bin is indistinguishable from covering it.

A fifth place a person belongs is a bound rather than a gate. Where two automated stages iterate against each other — generate, criticise, regenerate — the loop needs a cap, which is what the five-iteration threshold above provides. A 2026 survey of agentic design automation names the failure a cap prevents: *deadlooping*, an agent oscillating between states without converging, alongside context loss, where an agent optimises a local sub-goal having forgotten a global constraint, and tool hallucination <sup>[13]</sup>. A loop without a cap does not fail loudly. It consumes budget and returns something.

### What this review checks that an ordinary review does not

A review of machine-written verification code is not a stricter code review. Three of its checks carry weight they do not carry when the author is a person.

- **Every referenced signal exists and is the one intended.** Generated properties can name signals that are not in the design — the head-to-head in Section 25.2 counted eleven such references in a batch of thirty-eight [9], and the specification-driven framework there needed human engineers to remove unmappable generations before it could be scored at all [8]. Elaboration catches the non-existent ones. It does not catch the *wrong-but-existing* signal, which is the common case in a design with families of similarly named ports.
- **The artifact is in a position to fail.** Vacuous passes are a named, observed failure mode of generated properties in industrial use [10], which promotes Chapter 11's cover-with-every-implication rule from good habit to admission criterion.
- **The batch, not the sample.** Because a generator's errors are correlated, a review covers every artifact in a batch or rejects the batch.

That last item sets the real constraint on adoption, and it inverts the intuition the technology invites. The limit on throughput is not how fast artifacts can be generated but how fast they can be read. A flow producing two hundred properties in an afternoon has saved nobody a week unless someone reviews two hundred properties; if nobody does, it has not produced verification but a suite with an unknown number of wrong rules in it and a passing regression. Adopt where the mechanical filter is strong and the batch is small enough to review whole — a design constraint on the flow, not an attitude toward the technology.

## 25.5 Governance

Two years after tape-out, a field failure triggers the escape analysis of Chapter 7. Somebody opens the sign-off package and asks how a particular behaviour was covered. The answer points at a property. The next question is where that property came from, and if the honest answer is *a model wrote it and someone approved it*, the review has nowhere to go: it cannot re-derive the property, cannot tell whether the reviewer checked it against the specification or against the RTL, and cannot tell whether the specification it came from is the revision that shipped. Governance is the set of records that keeps that conversation possible.

### The provenance record

Attach to every generated artifact, in the repository and not in a wiki, a record with these fields:

- **What produced it.** The system, its version, and the identity and version of the model underneath. Section 25.2 showed the same flow varying from 4.33% to 41.43% end-to-end success on one benchmark tier depending on which model was underneath [10]. Model identity is not a footnote; it is a parameter of the result.



- **What it was given.** The exact input — which specification document, which revision, which section — and the prompt or template version. Without the revision, a later reviewer cannot tell whether the property predates the change that made it wrong.
- **The sampling settings, and how many runs were kept.** See below.
- **Which mechanical checks it passed, and their versions.** Compilation, signal resolution, antecedent reachability, duplicate detection.
- **Who accepted it at each human gate, and against what text.** Not a signature on a batch: a name against an artifact, and the specification sentence they compared it to.
- **Whether it was edited after generation, and by whom.** A hand-edited generated property is a hand-written property with a misleading history, and the distinction matters when the same generator is re-run on the next revision.

The last two are the ones teams skip and the ones a defensible sign-off needs. A record of what a machine produced is an audit trail of the machine. A record of what a person accepted is an audit trail of the decision, and the decision is what is being reviewed.

### Reproducibility is not seeding

Chapter 9 makes the seed part of a run’s identity and is precise about what it does and does not reproduce. Generated artifacts sit outside that discipline entirely, and the vocabulary invites a false analogy. A language model’s sampling parameters are not a seed in Chapter 9’s sense; identical inputs can yield different outputs across runs, and one research group’s response is instructive — issue the same prompt three times and take the union of the resulting property sets, precisely to make the result robust enough to report [9]. An industrial report notes the same axis from the other side: the sampling temperature that works varies by design, and a wrong setting yields output either overly deterministic or too random [10].

So record the sampling settings and how many runs were kept, because “we generated this” is not reproducible while “we took the union of three runs at these settings” is at least describable. Do not treat regeneration as a way to re-check an artifact: a second run agreeing with the first is not corroboration, since both come from the same process with the same blind spots. And version the artifact rather than the recipe — the artifact is what shipped.

### Data governance

The industrial obstacle here is not usually accuracy. A 2023 survey put the industrial concerns under three headings — model generalisation, model scalability and data governance — and spelled the third out: an engineer who cannot send her data outside her organisation, and has no machine-learning infrastructure inside it, cannot use a model that requires training on her data [6]. The constraint has sharpened rather than softened. A 2026 survey of agentic design automation names data privacy and intellectual-property exposure in cloud-based inference as a first-order

trust problem, and lists the mitigations: local deployment, federated approaches, and differential privacy <sup>[13]</sup>.

For a verification lead this is a procurement question with an engineering answer. Before a pilot starts, establish what leaves the building — specification, RTL, failure logs, coverage database — and get it in writing from whoever owns the design’s confidentiality, not from the team running the pilot. It is the one governance failure that cannot be corrected afterwards.

### Deciding where to adopt

Three questions decide it, and they fit in a fifteen-minute meeting.

*Is there a check on the artifact that is independent of what produced it?* A diff against a machine-readable source, a proof against a reference the artifact was not derived from, a fix applied and the failure re-run. If the only evidence is the generator’s confidence, or a proof passed against the very design the artifact came from, there is no check.

*Is there a baseline you already own and can re-measure on your own data?* Yesterday’s triage decisions, last quarter’s regression runtime, the current suite’s coverage. A vendor benchmark is not a baseline; your own last quarter is.

*Does acceptance require a judgment about intent that no run can settle?* If so, the flow’s throughput is that judgment’s throughput, and it should be budgeted before the pilot rather than discovered during it.

**Adopt now** where the first two hold and the third does not. Regression ranking and selection qualify: the baseline is your own runtime, the check is your own coverage database — with Section 25.1’s constraint carried in the same breath, that a compressed suite is a change-detection instrument and the full fresh-seed sweep has to survive alongside it. Supervised triage classification qualifies where a labelled failure history already exists, with a published agreement rate re-measured on a schedule. Generated scaffolding qualifies wherever a mechanical diff against a machine-readable source decides correctness.

**Pilot, in a non-gating tier**, property generation from a specification: on a block whose specification is unusually precise, with the reviewing capacity budgeted first, and with a statistic of your own — what fraction of generated properties survived the intent gate — recorded from the first batch. Counterexample debug assistance is the other good pilot, for a reason specific to its shape: its output is a hypothesis a human was going to form anyway, and its proposed fix can be applied and the proof re-run, which is a check the assistant does not control <sup>[11]</sup>.

**Wait** on flows whose sign-off artifact is the flow’s own report. The 2026 survey is blunt about where the field stands: reinforcement-learning systems have reached industrial tape-out, and no language-model-driven autonomous agent has yet demonstrated a complete, human-free tape-out of an industrial chip <sup>[13]</sup>. That survey also names the reading error to avoid — the *unit-test fallacy*, in which high pass rates on module-level benchmarks are read as evidence about system-level capability <sup>[13]</sup>.

Every headline figure in Section 25.2 was measured on a block, a protocol or a curated benchmark. None was measured on an SoC.

The same survey supplies vocabulary for saying where a proposal sits, on a scale running from tool-assisted design through prediction-oriented copilots to agentic orchestration, where an agent drives the task and a human monitors, and finally to autonomy in which a human supplies only intent; its near-term expectation from a 2026 vantage is agents functioning as strong assistants under human oversight rather than autonomous flows <sup>[13]</sup>. Use the scale to name what is actually being proposed, because proposals routinely describe one level and cost another.

One structural idea there is worth adopting ahead of the tooling. Rather than trusting the generator, require the artifact to arrive with a mechanically checkable argument for itself — the survey calls this proof-carrying, pairing it with a reasoning trace linking the intent an agent was given to the action it took, so an engineer audits the decision rather than the output <sup>[13]</sup>. A generated property arriving with its antecedent cover and the specification sentence it was derived from is a small instance of exactly that, available today without new tooling.

### What ages, and how to tell

Every capability figure in this chapter is dated in the text, and the dates run from 2003 to 2026. The classical results have been stable for two decades and will most likely still hold in five years: what a coverage-directed generator optimises, and that models decay, are structural facts rather than technology facts. The language-model figures are the perishable ones.

They will not perish uniformly, and the useful signal is not that the numbers rise — expect them to rise. Ask instead whether three things have changed: whether results are reported on designs larger than a block, whether the pass criterion has moved from *a proof passed on known-good RTL* to something that distinguishes a correct rule from a plausible one, and whether the mechanically checked metric is reported alongside the model-judged one rather than instead of it. Until those change, a higher percentage is a higher percentage on the same benchmark, which is the one kind of progress that costs nothing to produce.

---

### IN PRACTICE

The first thing a team adopting any of this needs is not a tool but a place to put the number. Pick the one statistic that says whether the technique still works — agreement with human triage, coverage retained after compression, fraction of generated properties surviving the intent gate — and put it on the same dashboard as the bug curve, re-measured on a fixed cadence. A technique with no number on a dashboard has no way to be retired, and the ones needing retirement are the ones people have stopped checking.

The messy parts are predictable. Enthusiasm arrives from outside the team with a demonstration and a deadline, and the useful response is a pilot with a review budget rather than a debate about the technology. Generated artifacts get hand-ed-

ited and the provenance record silently stops being true, which is why the *edited by* field exists. A coverage model changes and the reward function nobody owns quietly stops steering. And the first generated batch overruns its estimated review effort, because the reviewers are learning what this generator gets wrong — time well spent, and exactly the number to record for the second batch.

---

## PITFALLS

1. **Reading a pass rate without its pass criterion.** Symptom: a proposal quotes a percentage and cannot say, when asked, what the study counted as correct. Cure: find the definition before the number; in this field it is usually one sentence and usually decisive.
2. **Chaining results from different baselines.** Symptom: a slide composing one paper's speedup with another's accuracy into a single trend. Cure: state each figure with the benchmark, the baseline and the underlying model that produced it, and refuse to compose them.
3. **Judging a generated property by whether it proves.** Symptom: "thirty-six of forty passed formal" offered as evidence of quality. Cure: a passing proof against the design the property was derived from is consistency, not intent; require an antecedent cover and a specification sentence instead.
4. **Sampling a generated batch.** Symptom: "we spot-checked five and they were fine", or two hundred generated properties and nobody who has read them all. Cure: review the batch or reject it — a generator's errors are correlated, so a sample estimates nothing — and budget the reviewing capacity before the pilot, because it sets the flow's throughput.
5. **Replacing the random sweep with a compressed one.** Symptom: regression runtime halves and the bug discovery rate quietly follows. Cure: keep the compressed suite for change detection and the full fresh-seed sweep for finding bugs; they are different instruments.
6. **The model nobody retrains.** Symptom: a triage classifier that was excellent at tape-out and is silently wrong two derivatives later. Cure: an owner, a cadence, and a published agreement rate that anyone can see falling.

---

## SUMMARY

- Classical machine learning has real results in coverage-directed generation, failure classification and regression selection, spanning two decades; each optimises something narrower than the goal, and each needs re-measuring once deployed [1, 6].
- The headline property-generation result in this corpus — 89% of 56 properties on one design in 2024 — defines *correct* as passing a formal proof against known-good RTL after unmappable generations were removed by hand, and reading that definition changes what the number means [8].

- A generated artifact is plausible by construction: plausibility is what the technology optimises, and correctness is a by-product. A property derived from the design cannot disagree with the design [8].
- Where a study reports a model-judged quality score and a mechanically checked outcome for the same system, the two do not order the alternatives the same way — so a study reporting only the first has told you how convincing its output reads [11].
- Human gates are decision points with owners, not sentiment: intent, admission to a tier, ownership on breakage, and waiver authority — with every mechanical check run before any of them, so the human attention goes where only a human can go.
- A generator’s errors are correlated across a batch, which makes sampling an invalid review strategy and makes reviewing capacity, not generating capacity, the limit on adoption.
- What makes a sign-off defensible two years later is the record of what a person accepted and against what text — not the record of what the machine produced.
- As of a 2026 survey of agentic design automation, no language-model-driven autonomous agent had demonstrated a complete, human-free tape-out of an industrial chip [13] — and every capability figure in this chapter was measured on a block, a protocol or a curated benchmark rather than on an SoC.

---

## FURTHER READING

Within the corpus, [6] is the best entry point to classical machine learning in functional verification as of 2023 — applications, the clustering-versus-classification asymmetry, and a candid section on industrial obstacles — and [1] is its necessary companion, a 2025 review of why so little of this literature reached deployment. For coverage-directed generation, [2] is the founding result and still the clearest statement of the cost side, [3] reviews the field at its 2012 maturity, and [14] surveys directed test generation broadly in 2024, referring its readers back to [3] for the machine-learning material. On generated properties, read [8] and [9] together and in that order, watching how each defines success; [10] is the corpus’s one industrial agentic-formal deployment report and is unusually candid about its failure modes; [11] is the model for how these results should be reported, with a model-judged metric and a mechanically checked one side by side. For the wider landscape, [5] surveys language models across design automation as of 2025 and [13] supplies the autonomy scale, the failure taxonomy and the governance vocabulary used in Section 25.5. [12] applies the same evidentiary discipline to security verification and is worth reading beside Chapter 23.

Outside the corpus, this is the one subject in this book where the proceedings turn over faster than a book can follow. The reading discipline is the one in Section 25.2’s table: find the benchmark, the pass criterion and the baseline before you read the percentage. A paper that does not state all three has told you nothing you can act on.

## REFERENCES

1. **P8** C. Bennett and K. Eder, "Review of Machine Learning for Micro-Electronic Design Verification," arXiv preprint arXiv:2503.11687v1, March 2025.
2. **P16** S. Fine and A. Ziv, "Coverage Directed Test Generation for Functional Verification using Bayesian Networks," Proc. 40th Design Automation Conference (DAC '03), Anaheim, CA, pp. 286–291, June 2003.
3. **P19** C. Ioannides and K. I. Eder, "Coverage-Directed Test Generation Automated by Machine Learning – A Review," ACM Transactions on Design Automation of Electronic Systems, vol. 17, no. 1, Article 7, 2012.
4. **D1** E. Ohana, "Closing Functional Coverage With Deep Reinforcement Learning: A Compression Encoder Example," Proc. DVCon U.S., 2023.
5. **P24** J. Pan, G. Zhou, C.-C. Chang, I. Jacobson, J. Hu, and Y. Chen, "A Survey of Research in Large Language Models for Electronic Design Automation," arXiv preprint arXiv:2501.09655v1, January 2025.
6. **D35** D. Yu, H. Foster, and T. Fitzpatrick, "A Survey of Machine Learning Applications in Functional Verification," Proc. DVCon U.S., 2023.
7. **P7** D. N. Gadde, S. Simon, D. Lettnin, and T. Ziller, "Improving Simulation Regression Efficiency using a Machine Learning-based Method in Design Verification," Proc. DVCon Europe, 2022.
8. **P11** W. Fang, M. Li, M. Li, Z. Yan, S. Liu, H. Zhang, and Z. Xie, "AssertLLM: Generating and Evaluating Hardware Verification Assertions from Design Specifications via Multi-LLMs," arXiv preprint arXiv:2402.00386v4, February 2026.
9. **P12** Y.-A. Shih, A. Lin, A. Gupta, and S. Malik, "FLAG: Formal and LLM-assisted SVA Generation for Formal Specifications of On-Chip Communication Protocols," arXiv preprint arXiv:2504.17226v1, April 2025.
10. **D25** A. Kumar, D. N. Gadde, K. K. Radhakrishna, and D. Lettnin, "Saarthi: The First AI Formal Verification Engineer," Proc. DVCon, 2025.
11. **P22** Y. Bai, G. Bany Hamad, C.-T. Ho, S. Suhaib, and H. Ren, "FVDebug: An LLM-Driven Debugging Assistant for Automated Root Cause Analysis of Formal Verification Failures," arXiv preprint arXiv:2510.15906v1, September 2025.
12. **P13** K. T. Hasan, M. A. Hasan, N. Alam, M. T. Islam, U. Das, and F. Farahmandi, "AI-Assisted Hardware Security Verification: A Survey and AI Accelerator Case Study," Proc. IEEE VLSI Test Symposium (VTS), Napa, CA, pp. 1–7, 2026.
13. **P23** Z. Zang, Y. Song, A. Wang, B. W.-K. Ling, Q. Sun, Z. Lei, F. Yang, C. Zhuo, and J. Luo, "The Dawn of Agentic EDA: A Survey of Autonomous Digital Chip Design," arXiv preprint arXiv:2512.23189v2, February 2026.
14. **P10** A. Jayasena and P. Mishra, "Directed Test Generation for Hardware Validation: A Survey," ACM Computing Surveys, vol. 56, no. 5, Article 132, 2024.



---

## 26. The Road Ahead

A generation after the night that opened this book, the same DMA architecture goes to tape-out again. This time it is the flagship SoC's eight-channel engine: 40-bit physical addressing, descriptor chaining, the same frontend/midend/backend split and the same 4 KB legalization discipline inherited from the reference SoC. The verification program around it is unrecognisable. The 4 KB rule exists as a formal property rather than only as a scoreboard check hoping to meet the right descriptor. The stimulus is a scenario space retargeted from simulation to the prototype instead of a constraint block written for one environment. Regressions gate on every commit, coverage merges across platforms, and machine time is no longer what this block's team argues about.

The bug that arrives in week 31 is SOC-1284's descendant. The midend mis-legalizes a negative-stride row across a 4 KB frontier again — a different module, wider addresses, the same arithmetic mistake — and this time the second half of the split writes a line the host cluster holds Modified. The scoreboard that caught the original sees nothing, because the corrupted bytes never reach memory during the check; they surface at a writeback, later, somewhere else. The team finds it eventually, from a firmware symptom on the prototype: weeks of debug on a platform where the corrupted line is not visible, for a bug whose ancestor a scoreboard caught in one night.

Now ask what actually decided the outcome, both times. In 1284, two hundred and fourteen directed tests passed honestly and a constrained-random run stumbled into the combination — and the escape analysis recorded that the functional coverage model had no cross of the three dimensions, so the hole was invisible to the team's own metrics. In the descendant, no engine was wrong: the proof was sound, and it was a proof about the *previous* generation's backend, inherited into the new plan as a row already discharged. Nobody wrote the row saying that a data-integrity check crossing into a coherent domain is a different claim needing different evidence. Both times the binding constraint was the same, and it was not compute, not stimulus, and not checking. It was knowing what to ask.

That is the argument of this chapter, and it is the one thing in it worth carrying.

---

### CHAPTER MAP

- §26.1 traces why verification's constraint has migrated across four generations of tooling, what evidence supports each stage, and why the migration is per-level rather than industry-wide.
- §26.2 states what shift-left and shift-right genuinely buy, what each cannot buy, and what happens to the middle of the flow when both pull at once.
- §26.3 identifies which properties of the verification problem survive any improvement in tooling — and how to tell those from the ones that merely feel permanent.

- §26.4 names what the discipline has become as a profession, and the four things it still handles badly.

## 26.1 The bottleneck moves

---

The historical argument in this section is ours rather than the corpus's. The evidence attached to each stage is the corpus's, each measurement belongs to its own stage, and none of it composes: figures taken in different years, from different populations, by different instruments do not make a trend line, and drawing one would be the same mistake as multiplying two speed-ups measured against different baselines. What the stages share is a shape, not a scale.

A constraint binds when it is what you would spend a doubled budget on. Verification's has occupied four positions in turn.

**Cycles.** The founding constraint was raw simulation throughput. RTL simulation runs approximately a billion times slower than the target clock, which is why an operating-system boot — seconds of silicon — takes years in a simulator, and why hardware-assisted platforms exist to recover a hundred to a thousand times of that <sup>[1]</sup> (Chapter 17 works the economics). This constraint has not disappeared; it has been *relocated by level*. For a block environment on something the size of the reference SoC's DMA, cycles have not been the binding constraint for a long time — the nightly suite fits in a night. For a firmware run on the flagship, they still are, and the mechanism is on the record as of the 2024 industry data: as processor frequency scaling plateaus, simulation-based techniques struggle to keep pace with growing complexity, which is what makes acceleration essential for many designs rather than a luxury <sup>[2]</sup>. Read that as the discipline's first lesson about its own constraints: they do not lift, they move down the hierarchy and wait for you at the next level up.

**Stimulus.** Once you could afford cycles, the question became what to run in them. Hand-enumerated tests stop scaling in the low hundreds (Chapter 9), and the answer — replacing enumeration with description, so a solver generates legal-but-unlikely traffic by the thousand — arrived with the constraint-based verification languages of the late 1990s and became the industry default. The tell that this constraint dissolved is in the diagnosis attached to the technique's own limits. In 2014, with constrained-random adoption seen to be levelling off, the reason offered was scaling: the technique works well at the IP block or subsystem level and does not scale to the entire SoC integration level <sup>[3]</sup>. That is not a complaint about generating stimulus. It is a complaint about a level of the design at which deciding *what* to generate has become the hard part. The adoption picture since has been the opposite of a plateau — a decade later, constrained-random simulation was the one exception to a general stabilisation of simulation-technique adoption <sup>[2]</sup> — which is the right shape for a technique that solved its problem and is still spreading to the teams that have not yet had it.

**Checking.** Generating traffic you cannot grade is a waveform generator, not a test-bench. The self-checking discipline — the verdict computed rather than eyeballed (Chapter 3), the environment architected so that stimulus, driving, monitoring,

checking and coverage are five separable responsibilities (Chapter 8), and assertions carrying the rules that are local enough to state (Chapter 11) — is the generation of work that dissolved this one. Its completion is visible in the adoption data as standardisation: SystemVerilog Assertions is the predominant assertion language and UVM the predominant methodology for building IC/ASIC testbenches [2]. The video codec shows what “dissolved” means and where it stops. A standards body that publishes a bit-exact reference decoder has handed you the oracle: the expected pixels are not a matter of opinion, and checking collapses to a comparison. What it has not handed you is rate control, latency, or what the encoder does when its line buffer back-pressures — questions the standard is silent on, and checkers somebody has to decide to write. Chapter 3 draws the general rule from this case and from the instruction-set simulator: a definitional oracle is definitional about exactly one projection of correctness.

**Knowing what to check.** This is the constraint that binds now, and the evidence for it is indirect by nature, because a question nobody asked leaves no trace in any tool’s output.

Start with the industry outcome. First-silicon success stood at 14 percent in 2024 — the lowest value recorded in twenty years, and the low point of a declining trend running from 2012 [2]. A decade earlier it was about 30 percent [3]. On the FPGA side — a different population, measured by a different metric, and not a point on the same curve — 87 percent of projects reported non-trivial bugs escaping into production in 2024 [4]. The two figures do not combine into one, and they do not need to. What they jointly rule out is the comfortable story that better engines produce better outcomes on their own. The engines got better. Adoption of the mature techniques climbed. Both effectiveness metrics are bad.

Then look at what the study says is growing. Not gates: *requirements*. The 2024 report attributes complexity growth to the emergence of new layers of design requirements beyond basic functionality — clocking, security, safety and hardware-software interaction requirements, which did not exist years ago [2]. Each layer is a category of question somebody must think to ask before any engine can answer it, and each arrived faster than the practice of asking it. The root-cause data points the same way: design errors lead, and problems arising from changing, incorrect or incomplete specifications remain a common concern of verification engineers and project managers [2]. A specification defect is precisely a case where the checking machinery was pointed at the wrong claim.

The effort data draws the same line. The 2024 study reads the spread in verification time as a reuse effect: projects spending the least time typically reuse pre-verified IP, while those spending the most carry high proportions of newly developed IP [2]. In practice, reused IP is not cheaper because its logic is simpler, but because someone has already worked out what to ask of it.

Three further signals are worth reading together. Debug — understanding what a failure means, which is the human half of the loop — took more of a verification engineer’s time than any other activity in 2024 and is named the discipline’s major bottleneck [2]. The fastest-growing corner of formal is the *applications* — in the 2024

data, automatic formal applications growing at 8.7 percent compound annual rate against 5.8 percent for property checking <sup>[2]</sup> — and an application is precisely a flow that supplies its own properties from a register description or a connectivity table, which relocates the knowing-what-to-check step for the narrow classes where the intent already exists (Chapter 15). And the completeness question has a rigorous answer whose existence makes the point: a property set cannot tell you whether it is enough, so Bormann’s criterion asks instead whether the properties allow unique determination of one output trace for each input trace — whether they pin the design down — with almost all the circuits verified to that criterion on a published project list functioning properly after fabrication, and the few overlooked faults traced to misapplication of the methodology rather than to the criterion <sup>[5]</sup> (Chapter 14 states its refinement in full). It is expensive, it is real, and it is a purpose-built instrument for the one question the rest of the toolchain cannot ask itself.

The reference SoC supplies the small version. The neural accelerator’s weight prefetcher wraps its buffer pointer one entry early when the weight stream ends mid-beat under DMA contention. It escaped 100 percent code coverage entirely: the line was executed, the branch was taken, and nothing about the structural metric could represent the conjunction. Only a functional cross of weight precision  $\times$  tensor width class  $\times$  contention demands the scenario — and that cross exists only if somebody decided those were the three axes. No engine chooses axes.

If you are deciding where to put a career, put it where the constraint is. That answer is per-level, not universal: on a block, it is almost always the model of what must be true; on a full-chip firmware workload, it may still be cycles. The skill that compounds across both is turning a specification into claims precise enough to point a machine at.

## 26.2 Shift-left and shift-right

**Shift-left** means verification starting earlier and running against something other than finished RTL. Static analysis moves the structural questions to the earliest point at which they can be asked: clock-domain crossing tools exist to identify domain issues directly on an RTL model at earlier stages of the design flow, because the class of metastability bug they target cannot be demonstrated on an RTL model in simulation at all and is hard to reproduce in the lab <sup>[2]</sup> (Chapter 13). Virtual platforms let firmware run against functional models of the memory map before the hardware exists, offering far more controllability and observability than debugging the same code over a debug port, and sometimes being the only option available while the hardware is still under development <sup>[6]</sup> (Chapter 18). Portable stimulus describes a scenario space once and retargets it across platforms, so the modelling work precedes any particular environment (Chapter 9). Continuous integration binds the whole loop to the commit rather than to the milestone; one 2026 research-programme account reports that running UVM-based verification and FPGA-based checks inside CI increased productivity by making the verification flow transparent from RTL development <sup>[7]</sup>.

What shift-left genuinely buys is not earlier bug-finding. It is earlier *specification* finding. The most valuable thing that happens when you write an assertion against a document rather than against a design is that the document turns out not to say what everyone assumed — Chapter 11 makes it a takeaway: writing an assertion is the cheapest way to discover that a specification sentence is ambiguous. Set that beside the root-cause picture of §26.1, where specification problems are a persistent theme, and it is clear where the leverage sits.

The sensor hub shows the shape and its limit. Its always-on 32 kHz island, its wake sources and its retention list are architectural decisions, and in practice they are made and reviewed long before the RTL that implements them: which domains exist, what retains, which supply-state combinations are legal — power intent, in the form IEEE Std 1801-2024 (UPF) standardises [8]. That is real work, done early, and it is not simulation of anything. What it cannot reach is the reference SoC’s canonical event-unit escape — a retention register losing its configuration when a wake event races the release of isolation — because that failure is a property of the implementation’s sequencing, and it surfaced in power-aware gate-level simulation after RTL sign-off (Chapter 19). Shift-left moves the *claims* earlier. It does not move the evidence with them, and confusing the two is how a project arrives at week 40 with a beautiful early plan and no discharged rows.

**Shift-right** means the design continues to be checked after tape-out. **Validation** — checking real hardware rather than a model, the distinction Chapter 20 states in full — is not a fallback for what verification missed; it is a planned stage with its own instruments, and the industry pays for it in silicon: in some modern SoCs over 20 percent of the design real estate, alongside a significant component of the CAD-flow effort, is devoted to silicon validation [1]. The flagship carries the standard kit — per-core processor trace, eight bus monitors with programmable triggers, a 64 KB on-chip trace buffer, a debug access port per power domain.

What shift-right buys is coverage of the one thing no pre-silicon platform has: the real device, at real speed, running real software for real durations. What it cannot buy is observability, and the constraint is arithmetic rather than cultural. Every mechanism writes into a fixed store, so **signals × cycles ≤ buffer**: 64 KB buys 128 traced signals for 4,096 cycles, or 256 signals for 2,048, and the rectangle is fixed at tape-out (Chapter 20). No amount of lab ingenuity widens it, and the deliverable of a post-silicon debug is therefore not an explanation but a pre-silicon reproducer — short enough to simulate, small enough to load, still failing. Shift-right, done properly, is a machine for moving problems *back* into the middle of the flow.

The observation this section closes on is judgment rather than a finding: a discipline pulling in both directions is one whose middle is under strain. The block-level RTL environment — the subject of this book’s middle parts — is the only place in the flow that has both full observability and a real design. Shift-left compresses it from the front by moving claims onto models that cannot carry the evidence; shift-right compresses it from the back by promising that whatever is left will be caught on a platform where the cost of finding anything is set by the observability you no longer have. Both movements are correct in their own terms. The failure mode of doing

both enthusiastically is a project with an excellent early plan, an excellent lab, and a thin layer of actual verification between them — the layer where evidence is cheapest to produce and easiest to trust.

**Scenario.** Late in the reference SoC project, the SoC integration team proposes cutting the crossbar’s block-level constrained-random regression by half, on the grounds that its protocol properties are formally proven and its integration behaviour will be covered in the lab. **Approach.** The verification lead accepts the first argument and refuses the second: the proven properties genuinely do retire the protocol rows, so those runs no longer earn their place and go. The rows they do not retire are the ones about *arbitration under sustained back-pressure across two managers* — proven only up to a bound, and reachable in the lab but not diagnosable there inside the trace buffer. Those seeds stay. **Why.** The two ends of the flow retired one class of evidence and neither of them retired the other. Cutting by argument row by row is a different act from cutting by percentage, and only the first one can tell you which half you kept.

### 26.3 What does not change

The useful test for this section is a single question asked of each candidate: *would this still be true if every tool got a thousand times better?* Properties of the problem survive it. Properties of the tooling do not, and a closing chapter that mixes them ages badly, because a reader ten years out will find half of it quaint and will not know which half.

Six things pass.

**The state space is exponential in the number of state bits, and abstraction is the only reply.** One DMA channel’s 192 bits of architecturally visible state make  $2^{192}$  — about  $10^{57}$  — configurations, of which a simulation farm running since the Big Bang would visit a vanishing fraction; Chapter 3 does that arithmetic. A thousand-fold faster engine buys you ten bits. The coherent hub makes the consequence concrete and permanent: the MESI state of every line in every cache is a product space no tool will ever enumerate, so the proof runs against a model cut down to two caches and one address — small enough to converge, large enough to contain the livelock and starvation cases that make coherence worth proving. That is not a work-around awaiting better capacity. It is the technique, and it will still be the technique when the capacity is a thousand times larger, because the space grows faster than any engine.

**Simulation shows the presence of bugs, never their absence.** Testing yields probabilistic assurance, not proof (Chapter 3). This is a statement about sampling, and no improvement in sampling rate converts a sample into a census.

**Every practical oracle is partial.** A reference model, a scoreboard, an assertion — each checks a projection of correctness rather than correctness, and the craft is combining them so their blind spots do not overlap (Chapter 3). The radar front-end



shows the durable version of the problem inside one design. Its FFT and detection pipelines are bit-exact arithmetic, so their oracle is an equality; one stage upstream, on the receive chain from mixer through variable-gain amplifier to ADC, the oracle for the same signal becomes a band, a window and a condition set, and every part of the environment below it — checker, coverage, pass/fail — is rebuilt on that (Chapter 21). No engine converts a band into an equality. Somebody decides the band, and that decision is part of the specification whether or not anyone wrote it there.

**A proof is a statement about a model, under assumptions, sometimes to a depth.** “Formally verified” without those three qualifiers is not a claim (Chapter 14). This is the durable form of what people often state as “formal doesn’t scale” — which is a tooling fact and will date. The qualifiers will not: a model is always an abstraction of something, assumptions are always claims about an environment, and a bound is always a bound.

**A plan is a set of claims, and coverage is a sample of a model you wrote.** Recording that a situation occurred and judging whether the design behaved correctly in it are two separate aspects of an environment: they draw on the same monitors, and their fidelities are traded off against one another <sup>[9]</sup>. So coverage measures sampling, not correctness (Chapter 6), and “100 percent” is a statement about your model’s own extent that says nothing about its fidelity. This is the property behind SOC-1284’s blind spot and the accelerator’s, and it is invariant under the kinds of automation Chapter 25 takes up: a drafted plan is still a set of claims somebody’s judgment shaped, and it still cannot see what it cannot represent.

**Activation, propagation and checking are three independent conditions, and controllability and observability bound the first two.** A bug found requires all three; each of them can fail alone (Chapter 3). This is why observability is designed in a year before it is needed, and why it is a rectangle rather than a dial.

Now three that feel durable and are not, because sorting them is what makes the list above worth trusting. *Debug dominates verification time* is a measured share in 2014 and again in 2024 <sup>[2, 3]</sup> — a robust empirical regularity of the current practice, not a property of the problem, and the most plausible thing on this page for automation to change — which would move the constraint this chapter named fourth rather than retire it, because an engine that explains a failure in seconds still does not decide which questions were worth asking. *Formal is capacity-limited to small blocks* is a statement about engines. *Simulation is a billion times slower than silicon* is a measurement of today’s tools rather than a law, and treating it as one is how a chapter becomes a period piece. Add one more that is repeated more often than any of them and rests on nothing: *70 percent of a project’s overall effort is spent in verification*. Harry Foster, the author of the study series that supplies most of the real numbers on this page, names that figure himself as an example of the subjective and unsubstantiated claims the field makes about itself, and points instead at the lineage that does carry measurement — the Collett International Research studies of 2002 and 2004, the later of which had 201 eligible participants, and then the Wilson Research Group studies, with 1,886 in 2014 <sup>[3]</sup>. A figure worth repeating has a sample size attached to it.

**Worked example — a durability audit of four vplan rows.** Take the DMA rows of Chapter 5 and ask, of each, which part is a property of the problem and which is contingent on today’s tooling. It is a short exercise and the cheapest way to find out which of your practices you are defending on merit.

Row	Durable	Contingent
<b>DMA-F06a</b> — 2D legalization cross, closed by cg_2d_4kb at 100%	That the cross is a <i>chosen</i> set of equivalence classes, and that “100%” measures its extent, not its fidelity	That the cross has four axes and 72 cells; that a solver budget forecloses the maximum-length straddle; that a human wrote the bins
<b>DMA-F06b</b> — p_no_4kb_crossing proven unbounded on the backend	That the proof is about a model, under assumptions, at a binding point somebody chose	That it converges unboundedly on this block at this size; that the property text is hand-written
<b>DMA-N01a</b> — scoreboard flags any write outside the legalized window	That an end-to-end oracle pays a detection distance, and that it checks one projection	That it is cheap enough to leave enabled in every regression run
<b>DMA-N01b</b> — injection test proving the check fires	That a checker you have never seen fail is not evidence	That the proof of firing is a directed test rather than a generated one

Read the right-hand column as the part of your methodology that has an expiry date, and the left as the part that outlives it. Note where this chapter’s opening bug landed: the left column of F06b — *the proof is about a model*, and the model was the previous generation’s backend.

## 26.4 Verification as a profession

Chapter 1’s first pitfall is treating verification as a phase that starts when the RTL is done. Whether it is a phase or a profession is not really an open question any more, and the answer is visible in four independent places.

**Headcount.** Demand for verification engineers has outgrown demand for design engineers since at least 2007 — a 12.5 percent compound annual rate against 3.7 percent through 2014, by which point the mean peak number of verification engineers on a project already exceeded the design count [3]. The growth rates have since converged, but from that inverted baseline, and in some market segments — processors among them — a five-to-one ratio of verification to design engineers is not unusual [2].

**Boundary.** The distinction is no longer organisational. In 2024, design engineers spent on average 49 percent of their time on verification activities [2]. Nearly half of the design headcount is, functionally, verification effort, which means the profession is defined by the activity rather than by the org chart. Chapter 24 takes up how such teams are built and how verifiers are trained.

**Apparatus.** The discipline has its own standards — an assertion language (IEEE Std 1800-2023) [10], a testbench methodology (IEEE Std 1800.2-2020) [11], a stimulus de-

scription language (Accellera PSS 3.0) <sup>[12]</sup>, a coverage interchange format (Accellera UCIS 1.0) <sup>[13]</sup>, a power-intent format (IEEE Std 1801-2024) <sup>[8]</sup>, and as of March 2026 a standard for describing an IP’s clock- and reset-domain-crossing interface so that different teams’ tools can exchange collateral about it <sup>[14]</sup>. What a field standardises tells you where its work actually is. That list is not about running simulations faster; it is about making claims portable between people. The newest of them is worth naming for who agreed to it: the Accellera working group behind it counted 154 members from 24 companies as of September 2025 <sup>[15]</sup>, and the standard’s own list of eligible voting companies at the time of standardization runs Agnisys, Arm, Blue Pearl Solutions, Google, IBM, Infineon, Intel, Marvell, Microsoft, NVIDIA, Qualcomm, Renesas, Siemens EDA, STMicroelectronics and Synopsys <sup>[14]</sup>. Three years of competitors in one room is not how an industry spends its time on a problem it thinks is cosmetic.

**Theory.** State-space arguments, the oracle problem, coverage as sampling, proofs qualified by model and assumption and depth: a body of reasoning that is genuinely its own, and that a good verification engineer uses daily whether or not they can name it.

For someone choosing the discipline, the honest pitch is narrow and strong. The product is judgment, and judgment is what the tooling has been least able to absorb through four generations of trying. The work is intellectually severe in a specific way: you spend your time reasoning about what could be true rather than about what is, which suits some minds and frustrates others. It also carries an unusual professional obligation — Chapter 2’s independence of judgment, which comes down to a willingness to say “no, this is not the way to do it” to a designer with more years on the block, or to a schedule that wants the block closed. That is a duty rather than a personality trait, and the day it is inconvenient is the day it counts.

Four things the field still handles badly. Saying so is judgment, not pessimism; every one of them is a place where an individual can be visibly better than the average.

**It does not learn from its escapes.** Escape analysis, where it happens at all, is usually a post-mortem slide rather than a change to the next plan and the next sign-off checklist (Chapter 7 gives the five-question form, Chapter 20 the post-silicon version). The mechanism for compounding organisational knowledge exists, is cheap, and is skipped.

**It cannot estimate.** Debug takes more of a verification engineer’s time than anything else, is unpredictable, and varies significantly between projects — which makes planning a project’s effort and schedule from the previous project’s data a stated management problem, not a solved one <sup>[2]</sup>. In practice, teams estimate that way regardless. Three quarters of IC/ASIC projects were behind their original schedule in 2024, against a historical average of about two thirds <sup>[2]</sup>.

**It rarely measures its own environments.** The book’s instruments all measure the design. Almost nothing routinely measures whether the environment could still fail: mutation is the one technique that answers “would this check have caught a change?” (Chapter 14), and in practice it runs on a small fraction of blocks. A check-

er nobody has seen fail is a decoration, and that is true at the scale of a whole environment, not just of a single assertion.

**It has no common language for “how verified is this?” across domains.** A functional sign-off aggregates coverage and plan rows. A safety argument aggregates fault classification against per-level metric thresholds (Chapter 22). A security result is a list of named claims and named gaps with no percentage available at all, and there will not be one soon (Chapter 23). These three are not commensurable, they arrive at the same tape-out review, and the discipline has no accepted way to state a combined position. That is an open problem, not a solved one presented badly.

---

## IN PRACTICE

Teams that navigate this well tend to do three things nobody gets promoted for. They re-derive inherited rows rather than inheriting them: a plan row marked *verified in the previous generation* is re-opened and its assumptions re-read against the new integration, which is exactly the step the flagship’s DMA program skipped. They keep an explicit register of what the current program is *not* verifying — the gaps, not just the waivers — because a named gap is reviewable and an unnamed one is a surprise. And they spend a fraction of every program deliberately on the middle: block-level environments on the newest logic, funded against the pull of both a keen early-modelling effort and a well-equipped lab. The messy part is that none of the three has a metric attached, so all three lose their budget arguments to things that do.

---

## PITFALLS

1. **Reading the bottleneck as industry-wide.** *Symptom:* a methodology decision justified by “the industry has moved past simulation throughput” on a block whose regression does not fit in a night. *Cure:* the constraint is per-level and per-block; establish which one binds *here* before reallocating anything.
2. **Treating shift-left as evidence rather than as claims arriving early.** *Symptom:* a plan whose rows were written in month two and whose closure column is still empty in month nine. *Cure:* every row named early gets a platform and a date named with it; an early claim with no scheduled evidence is a wish.
3. **Letting the two ends hollow out the middle.** *Symptom:* strong architectural modelling, a well-equipped lab, and block environments staffed by whoever is free. *Cure:* cut middle-of-the-flow work row by row with a retirement argument, never by percentage.
4. **Confusing a durable property with a tooling limit.** *Symptom:* a design rule of thumb inherited from a capacity constraint that lifted two tool generations ago, still shaping how blocks are partitioned. *Cure:* the thousandfold test — if a better engine changes it, it is a budget decision and belongs in the plan where it can be revisited.

5. **Skilling only at the ends.** *Symptom:* a team fluent in architectural models and lab bring-up that cannot debug a scoreboard mismatch at RTL. *Cure:* the middle is where both other skills are eventually cashed in; a pre-silicon reproducer is useless to a team that cannot run one.

---

## SUMMARY

- Verification’s binding constraint has migrated — cycles, then stimulus, then checking, now knowing what to check — and each generation of tooling dissolved one constraint and exposed the next. The migration is per-level, not uniform: cycles still bind full-chip firmware runs.
- The measured outcomes rule out the comfortable story. IC/ASIC first-silicon success fell to 14 percent in 2024 from about 30 percent a decade earlier [2, 3]; separately, 87 percent of FPGA projects reported non-trivial bugs escaping into production in 2024 [4]. Adoption of the mature techniques climbed and then saturated across the same span. Better engines alone do not produce better outcomes.
- Shift-left buys specification defects found early; shift-right buys the real device at real speed. Neither buys what the other one does, and a flow that pursues both without defending its middle ends up with claims at one end, cost at the other, and thin evidence between.
- What survives any improvement in tooling: exponential state spaces and abstraction as the reply; presence but never absence; partial oracles; proofs qualified by model, assumptions and depth; a plan as a set of claims and coverage as a sample of a model you wrote; and activation, propagation and checking as three separate conditions.
- What does not survive, and should not be taught as if it did: debug’s share of effort, formal’s capacity limits, and the simulation-to-silicon speed ratio. All three describe a practice and its tools, not the problem.
- The discipline is a profession — its own headcount, its own standards, its own theory, and an activity that took 49 percent of design engineers’ time in 2024 as well [2]. It still fails to learn from its escapes, cannot estimate, rarely measures its own environments, and has no common language for combining a functional, a safety and a security position at one review.

---

## FURTHER READING

**From the corpus.** The two 2024 Wilson Research Group trend reports are the closest thing the field has to a self-portrait, and are short enough to read whole: IC/ASIC [2] and FPGA [4], with Foster’s DAC 2015 paper [3] supplying the decade-earlier baseline and the methodology behind the series. Read them for the requirement-layer framing and the adoption curves, not only the headline percentages. Bormann’s dissertation [5] is the corpus’s most serious attempt to answer “have I written enough properties?”, and repays the effort even if you never adopt the methodology. Mishra et al. [1] remains the best account of what happens to a bug’s cost and to

your visibility as the project moves right. The 2026 agentic-EDA survey <sup>[16]</sup> is the corpus's most explicit published projection about where automation is going, and should be read as its authors' expectation rather than as a finding: it proposes an autonomy taxonomy modelled on the levels used for self-driving cars, offers a near-, mid- and long-term roadmap against them, notes that as of its writing no LLM-driven autonomous agent had demonstrated a complete human-free tape-out of an industrial chip, and argues that trust should move from the generating model to a mechanically verifiable proof — which, read from this book's side, makes the property somebody writes *more* load-bearing, not less. The Accellera clock- and reset-domain-crossing interface standard <sup>[14]</sup>, published March 2026, is worth a skim as an example of what the discipline standardises when its problem is making claims portable between teams.

**Not in the corpus** (general references only): Wile, Goss and Roesner, *Comprehensive Functional Verification* (Morgan Kaufmann, 2005), still the best full-lifecycle treatment; and the DVCon and DAC proceedings, where this discipline argues with itself in public and where the survey series is presented.

---

One idea to carry out of twenty-six chapters.

Verification does not produce a correct design. Chapter 3 gave the reasons — the state space cannot be enumerated, and every practical oracle checks a projection — and Chapter 14 showed how far a purpose-built completeness methodology can push against them, for selected modules, at a price. What verification produces is a *true sentence about a design*: this is what we established, by this method, under these assumptions, and here is what we did not establish. Sign-off is the moment that sentence is spoken aloud and somebody's name goes under it.

Everything in this book is machinery for making that sentence mean more. A plan is the sentence written down in advance. Coverage is evidence that the sentence's scope was actually visited. A scoreboard is the part that could have made it false. An assertion is a clause of it, checked continuously. A waiver is the sentence being honest about its own edges. A proof is a clause with no exceptions inside a named model. An escape analysis is the sentence being repaired after it turned out to be too generous.

SOC-1284 is what a false sentence looks like from the inside. Two hundred and fourteen directed tests passed, honestly; the dashboard was green; the coverage report was accurate about everything it measured. The sentence all of that supported was *the DMA's 2D legalization is verified*, which was not true, and no instrument in the project could say so. Eleven days of margin and one constrained-random seed were all that stood between that sentence and a customer discovering it.

So the question to bring to every review, every plan, every dashboard, and every green regression for the rest of your career is not *are we done?* It is: **what exactly are we claiming, and what would have to be true for that claim to be false?** If you can answer the second half, you are verifying. If you cannot, you are counting.



## REFERENCES

1. **P21** P. Mishra, S. Ray, R. Morad, and A. Ziv, "Post-Silicon Validation in the SoC Era: A Tutorial Introduction," *IEEE Design & Test*, vol. 34, no. 3, 2017.
2. **R2** H. D. Foster, "2024 Wilson Research Group IC/ASIC Functional Verification Trend Report," Siemens EDA white paper, 2024.
3. **P17** H. D. Foster, "Trends in Functional Verification: A 2014 Industry Study," *Proc. 52nd Design Automation Conference (DAC '15)*, San Francisco, CA, 2015.
4. **R1** H. D. Foster, "2024 Wilson Research Group FPGA Functional Verification Trend Report," Siemens EDA white paper, 2024.
5. **T2** J. Bormann, "Complete Functional Verification," Ph.D. dissertation, University of Kaiserslautern, Germany, 2009 (English translation of the German dissertation 'Vollständige funktionale Verifikation', April 2009).
6. **T1** M. Schwarz, "Formal Hardware/Firmware Co-Verification of Optimized Embedded Systems," Ph.D. dissertation (Dr.-Ing.), Technische Universität Kaiserslautern, Germany, 2021.
7. **P25** S. Ahmed, A. Kropotov, R. I. Genovese, B. Homs, E. Merino, F. Urbani, et al., "Verification and Validation (V&V)-in-the-Loop for RISC-V Design: The Holistic Vision of BZL," *arXiv preprint arXiv:2604.27013v1*, April 2026.
8. **S10** IEEE, "IEEE Std 1801-2024, IEEE Standard for Design and Verification of Low-Power, Energy-Aware Electronic Systems (Revision of IEEE Std 1801-2018)," IEEE, 2024.
9. **B5** A. Piziali, "Functional Verification Coverage Measurement and Analysis," Springer, 2004.
10. **S8** IEEE, "IEEE Std 1800-2023, IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language (Revision of IEEE Std 1800-2017)," IEEE, 2023.
11. **S9** IEEE, "IEEE Std 1800.2-2020, IEEE Standard for Universal Verification Methodology Language Reference Manual (Revision of IEEE Std 1800.2-2017)," IEEE, 2020.
12. **S1** Accellera Systems Initiative, "Portable Test and Stimulus Standard, Version 3.0," Accellera, August 2024.
13. **S3** Accellera Systems Initiative, "Unified Coverage Interoperability Standard (UCIS), Version 1.0," Accellera, June 2012.
14. **S12** Accellera Systems Initiative, "Standard for IP Abstraction for Clock and Reset Domain Crossing Integration, Version 1.0," Accellera, March 2026.
15. **D36** D. Mills, B. Gascoyne, C. Choppali Sudarshan, D. Olopade, and D. K. Gupta, "Breakthrough in CDC-RDC Verification Defining a Standard for Interoperable Abstract Model," *Proc. DVCon U.S.*, 2026.
16. **P23** Z. Zang, Y. Song, A. Wang, B. W.-K. Ling, Q. Sun, Z. Lei, F. Yang, C. Zhuo, and J. Luo, "The Dawn of Agentic EDA: A Survey of Autonomous Digital Chip Design," *arXiv preprint arXiv: 2512.23189v2*, February 2026.

---

## Appendix A — Glossary

Terms appear broadly in the order the book introduces them, except that entries the book deliberately contrasts are kept side by side — the pairs where one word carries a different sense in design and in verification are the reason this glossary exists, and separating them would hide exactly what the reader came to check. The *Introduced* column names the chapter that owns the definition, which is not always the chapter of first mention: a term is often used in passing well before the chapter that pins it down.

Term	Definition (one line)	Introduced
<b>DUT (design under test)</b>	The design being verified by the testbench.	ch02
<b>reconvergence model</b>	Verification as reconciliation of a transformation and an independent second path sharing a common origin.	ch02
<b>verification component (VC)</b>	Reusable testbench-side model of an interface, with a driver to stimulate and a monitor to observe.	ch02
<b>scoreboard</b>	The data structure holding the data expected to be received, filled by a predictor and consulted when an output is observed; loosely, the whole self-checking structure around it.	ch02
<b>redundancy (verification sense)</b>	Independent second interpretation of the specification by a different person, used as the error-catching mechanism.	ch02
<b>poka-yoke</b>	Mistake-proofing a human process by reducing it to foolproof steps.	ch02
<b>validity pruning</b>	Discarding stimulus as “can’t happen” before checking whether the interface can express it (anti-pattern).	ch02
<b>white-box assertion</b>	Assertion checking internal design signals rather than interface behavior.	ch02
<b>zoom-out thinking</b>	Periodic step back from task-level work to reassess project-level verification priorities.	ch02
<b>error injection</b>	Deliberately driving erroneous stimulus to verify detection/recovery behavior.	ch02
<b>functional verification</b>	Pre-fabrication establishment that a design implements its specification.	ch01
<b>validation</b>	Checking a design on real hardware rather than on a model — post-silicon on fabricated parts, or in the lab on a prototype. The load-bearing contrast is hardware vs model, not fabricated vs FPGA. NOT the Boehm “did we build the right thing?” sense, and not the umbrella term covering pre-silicon work.	ch20
<b>bug escape</b>	Design flaw surviving verification into a later stage.	ch01
<b>respin</b>	New fabrication cycle with corrected masks forced by an escape.	ch01
<b>first-silicon success</b>	First fabricated silicon is production-worthy.	ch01
<b>tape-out</b>	Freezing and releasing the design database for fabrication.	ch01

Term	Definition (one line)	Introduced
<b>constrained-random</b>	Automatic generation of legal-but-unlikely stimulus under constraints.	ch01
<b>observability</b>	How much internal design state is visible during debug.	ch01
<b>design-for-debug (DfD)</b>	On-die instrumentation for post-silicon observability.	ch01
<b>verification gap</b>	Shortfall between what must be verified and what can be (inherent/transient/self-induced).	ch01
<b>errata sheet</b>	Published catalog of post-silicon bugs with workarounds.	ch01
<b>NRE</b>	Non-recurring engineering cost.	ch01
<b>verification lifecycle</b>	The phased process from plan to sign-off, with feedback loops, run once per verification level.	ch04
<b>milestone</b>	A phase-gating event defined as a checkable demonstration, not a state estimate.	ch04
<b>definition of done</b>	Per-deliverable exit criterion stated as an action (“passes in regression”), never a percentage.	ch04
<b>maturity stage</b>	One of a small set of named, checklist-audited levels of verification maturity shared across blocks.	ch04
<b>bring-up</b>	The phase in which environment and DUT first exchange checked transactions.	ch04
<b>coverage closure</b>	Converging every plan item to covered-or-consciously-excluded status.	ch04
<b>sign-off review</b>	The formal review that judges evidence against the plan and accepts residual risk.	ch04
<b>first-time success</b>	The plan-defined set of features that must work in first silicon.	ch04
<b>executable verification plan</b>	Machine-readable plan whose items link to the coverage/checks/results that discharge them.	ch04
<b>regression suite</b>	The test set re-run at fixed cadence to guarantee backward compatibility.	ch04
<b>the crunch</b>	The end-of-project concentration of the hardest verification work against an immovable date.	ch04
<b>escape</b>	A bug that crosses a sign-off boundary undetected — found on the wrong side of the boundary that was supposed to catch it. Ordinary-English uses of “escape” (“the obvious escape”, “escape routes”) collide with this term of art and must be reworded.	ch01
<b>state space</b>	The set of all possible configurations of a design’s state elements; $2^n$ for $n$ state bits.	ch03
<b>state-space explosion</b>	The exponential growth of state count with design size that makes exhaustive analysis intractable.	ch03
<b>controllability</b>	The ability to steer a design into a given internal condition using only drivable interfaces.	ch03
<b>oracle</b>	The mechanism that decides whether an observed response is correct.	ch03

Term	Definition (one line)	Introduced
<b>partial oracle</b>	A checker that verifies one projection of correctness rather than correctness itself.	ch03
<b>reference model</b>	An independent executable of the specification, taken as golden, run on the same stimulus and compared with the design.	ch03
<b>self-checking testbench</b>	An environment that computes pass/fail itself instead of relying on human waveform inspection.	ch03
<b>common-mode error</b>	The same specification misreading encoded in both design and oracle, making a bug structurally invisible.	ch03
<b>equivalence class</b>	A set of input values assumed to exercise identical design behavior, used to shrink the test space.	ch03
<b>risk-driven verification</b>	Allocating verification effort by bug likelihood × escape cost instead of pursuing completeness.	ch03
<b>grey-box verification</b>	Black-box verification augmented with design hooks for controllability/observability.	ch03
<b>feature extraction</b>	Enumerating from spec, interviews and architecture everything that must be shown to work.	ch05
<b>must-not-happen feature</b>	Negative requirement enumerating an error the environment must be able to detect.	ch05
<b>attribute</b>	A parameter or dimension of a behavior whose value set defines its verifiable space.	ch05
<b>one-row-one-metric rule</b>	Each vplan row carries exactly one measurable closure criterion; two evidences = two rows.	ch05
<b>traceability</b>	Permanent-ID linkage spec ↔ feature ↔ test/property/coverage ↔ result.	ch05
<b>weakness analysis</b>	Row-by-row review challenge on correctness, precision, completeness.	ch05
<b>spec-hole log</b>	Tracked list of questions extraction raised that the specification cannot answer, filed as spec defects.	ch05
<b>metrics-driven verification (MDV)</b>	Managing verification on quantitative measurements collected continuously from the process rather than on status estimates from people.	ch07
<b>bug curve</b>	Bug discoveries (or open/closed counts) plotted over time; the discipline's main convergence instrument.	ch07
<b>waiver</b>	Written record accepting a specific evidence hole: hole, reason, risk argument, owner, expiry.	ch07
<b>irritator</b>	Background traffic generator run concurrently with the main test, whose only job is to perturb the DUT. Two senses: the testbench component (ch07) and the on-die version running on real silicon (ch20). Both translate the same way, but the Italian text must not imply a testbench when the subject is a fabricated part.	ch07, ch20
<b>S1/S2/S3</b>	Bug severity classes: S1 blocks tape-out; S2 fix-or-waive with an argument; S3 documentation or cosmetic.	ch07
<b>disposition</b>	The recorded decision on an open item (fix / waive-as-errata / defer with justification).	ch07

Term	Definition (one line)	Introduced
<b>conditional sign-off</b>	Sign-off taken now but contingent on named evidence arriving by a named date.	ch07
<b>escape analysis</b>	Blameless structured post-mortem of an escape that feeds changes into the next plan and checklist.	ch07
<b>fresh-seed yield</b>	Failures per unit of newly-seeded stimulus; discriminates a clean flat bug curve from a saturated one.	ch07
<b>debug-time share</b>	Fraction of engineering time spent in debug, tracked as a project health metric.	ch07
<b>covergroup</b>	SystemVerilog container encapsulating a coverage model specification.	ch06
<b>coverpoint</b>	One observed variable/expression partitioned into bins.	ch06
<b>bins</b>	Counters over value sets — executable equivalence classes.	ch06
<b>cross coverage</b>	Cartesian combinations of coverpoints.	ch06
<b>shaped cross</b>	Cross with the reachable space stated via justified exclusions.	ch06
<b>toggle coverage</b>	Per-bit 0→1 activity metric.	ch06
<b>path coverage</b>	Coverage of decision sequences, not single branches.	ch06
<b>coverage model</b>	Multi-dimensional region defined by attributes and their values.	ch06
<b>fidelity</b>	Degree to which a coverage model captures actual behavioral requirements.	ch06
<b>bug footprint</b>	Region of the coverage space a bug occupies.	ch06
<b>coverage hole</b>	Required stimulus/behavior not yet observed.	ch06
<b>valid/invalid hole</b>	Genuine stimulus gap vs a coverage-model bug.	ch06
<b>CDG</b>	Coverage-directed test generation via feedback from coverage results to stimulus.	ch06
<b>ignore_bins vs illegal_bins</b>	“Not my job” exclusion vs “must never happen” (runtime error).	ch06
<b>UCIS</b>	Unified Coverage Interoperability Standard — cross-tool coverage interchange.	ch06
<b>vacuous pass</b>	Success of an implication whose antecedent never became true, so nothing was checked — neither a pass nor a failure.	ch06
<b>transactor (bus-functional model)</b>	Component converting between pins and transactions: every physical-level operation of one interface encapsulated in one place, so everything above it speaks transactions.	ch08
<b>transaction level</b>	Abstraction whose unit is a whole operation — injected into the running simulation, terminated later by an observed result — rather than the design’s clock cycles.	ch08
<b>operational vs observational communication</b>	Point-to-point blocking hand-off, where back-pressure is meaningful, versus broadcast non-blocking publication, where it must never be.	ch08
<b>analysis port</b>	Broadcast, non-blocking publication of an observed transaction to any number of subscribers, none of which may block the producer or modify what it receives.	ch08

Term	Definition (one line)	Introduced
<b>predictor</b>	Component computing the expected response from a monitored request — never from what the driver intended to send.	ch08
<b>transfer function</b>	Testbench model reproducing the design’s data transformation to predict its output: written by you, not golden, and as capable of misreading the specification as the design is.	ch08
<b>golden model</b>	Model trusted by construction because it is the specification, stated at the specification’s own level of abstraction; golden is not the same as complete.	ch08
<b>autonomous monitor</b>	Monitor whose observing thread starts at construction and runs continuously, so a late testbench never back-pressures the design.	ch08
<b>protocol monitor vs functional monitor</b>	Passive component judging whether traffic on an interface is legal, versus one that only reconstructs and publishes what the design did.	ch08
<b>responder</b>	Agent that answers requests the design initiates; because its reply is under testbench control it is a driver, not a monitor.	ch08
<b>stream key</b>	The tuple naming one ordered stream in a scoreboard: one dimension per independent source of concurrency, and computable from what the observer sees.	ch08
<b>orphan response</b>	A response from the design that matches no outstanding expectation.	ch08
<b>leftover</b>	An expectation still queued at end of test, never answered — the failure that produces no mismatch, only a queue that never empties.	ch08
<b>data tagging</b>	Encoding the expected destination and transformation inside the payload, so each output monitor can decide correctness on its own.	ch08
<b>detection distance</b>	The time and state between a mistake and the check that catches it; what an end-to-end oracle pays and an assertion does not.	ch08
<b>clocking block</b>	Declaration fixing the moment interface signals are sampled or driven, so the testbench cannot race the design it watches.	ch08
<b>modport</b>	Restricted view of an interface; an inputs-only clocking modport makes a monitor’s passivity a compile-time property rather than a review item.	ch08
<b>checking contract</b>	The written list of verdicts an environment will produce, each with a named owner — and the list of verdicts it will not.	ch08
<b>known-bad variant</b>	Deliberately broken copy of the design kept in the repository and run regularly, to prove the environment can still fail.	ch08
<b>directed test</b>	Stimulus whose content and order are written out by hand, doing the same thing on every run.	ch09
<b>decay (of a directed test)</b>	A test that keeps passing while no longer exercising what it was written for, because the design grew past it.	ch09
<b>environment vs directive constraint</b>	Constraints stating what the interface makes legal, which must be obeyed, versus constraints layered above them to steer a run toward chosen scenarios.	ch09



Term	Definition (one line)	Introduced
<b>over-constraining</b>	Narrowing stimulus below what the design must accept; the quiet form stays satisfiable and silently deletes the feature under test.	ch09
<b>soft constraint</b>	Preference rather than requirement: discarded when it cannot hold alongside the active hard constraints, so a default can be overridden instead of fought.	ch09
<b>distribution constraint (dist)</b>	Weights over values and ranges that move probability mass without changing the legal set — and silently forbid whatever the list omits.	ch09
<b>variable ordering (solve ... before)</b>	Ordering the solver's choices so a corner case becomes frequent, changing probabilities without changing the legal set.	ch09
<b>seed</b>	The value fixing a run's random draws; replaying it reproduces the stimulus only while generator, sources and tool build are unchanged.	ch09
<b>run</b>	The unit of work: one execution with a specific seed and a specific set of source and tool revisions, producing its own messages and coverage.	ch09
<b>random stability</b>	Localization of random generation per object and thread, so existing work keeps its sequence — provided new objects, threads and draws are appended, never inserted.	ch09
<b>scenario (stimulus)</b>	A sequence of stimulus interesting to the design and unlikely to arise from individually constrained-random items.	ch09
<b>multi-stream (virtual) generator</b>	One randomized object holding a sub-descriptor per stream, because constraints cannot be expressed across separate generators.	ch09
<b>generator/monitor duality</b>	One constraint used three ways: to generate legal traffic, to check a neighbour's output, and to assume in formal.	ch09
<b>portable stimulus</b>	Stimulus described once as a scenario space and retargeted to simulation, emulation, prototype and silicon.	ch09
<b>PSS (Portable Test and Stimulus Standard)</b>	The standard defining that single declarative representation of scenario spaces, taking SystemVerilog as its constraint and coverage reference.	ch09
<b>action (PSS)</b>	Unit of behaviour: atomic when it maps to one operation of the system, compound when it encapsulates a flow of others.	ch09
<b>activity (PSS)</b>	The flow of sub-actions a compound action encapsulates, stating scheduling relations rather than a schedule.	ch09
<b>pool (PSS)</b>	Sized store of resources or objects that actions claim; its depth is a rule the generator must satisfy, not a queue to wait in.	ch09
<b>inference (PSS)</b>	The tool completing a partial statement of intent with the actions it requires — and nothing it does not.	ch09
<b>test realization</b>	Mapping abstract actions onto target code for one platform: portable describes the model, not the effort.	ch09
<b>UVM (Universal Verification Methodology)</b>	The standardized base class library and API set for building modular, reusable, configurable verification components.	ch10
<b>agent</b>	The reusable unit of an environment: sequencer, driver and monitor for exactly one interface.	ch10

Term	Definition (one line)	Introduced
<b>active vs passive agent</b>	Active instantiation emulates a device and drives it; passive builds only the monitor and observes — the switch that carries a block environment into a system one.	ch10
<b>factory</b>	Indirection through which components and objects are constructed, so a type can be replaced without editing the code that instantiates it.	ch10
<b>type vs instance override</b>	Replacing a requested type everywhere, or only at one instance path; instance wins over type, and both must be registered before the parent builds its children.	ch10
<b>configuration database</b>	Store of typed values set against hierarchical path patterns, letting an integrator configure an environment without knowing its implementation.	ch10
<b>phasing</b>	The standard ordered steps every component runs together — build top-down, connect bottom-up, run concurrently — so independently written environments can be combined.	ch10
<b>objection</b>	A raised claim that an activity must finish before its phase may end; a task phase lasts only while at least one is raised.	ch10
<b>drain time</b>	Grace period after the last objection drops, so transactions still in flight reach the checkers.	ch10
<b>sequence</b>	Transient object generating stimulus, deliberately outside the component hierarchy so scenarios are not welded into the environment.	ch10
<b>sequence item</b>	The transaction object a sequence produces and a driver consumes.	ch10
<b>sequencer</b>	Component holding pending sequence requests and choosing which one the driver receives each time the driver asks.	ch10
<b>arbitration mode</b>	Policy by which a sequencer chooses among pending requests; only the strict modes grant the highest-priority one first.	ch10
<b>virtual sequence / virtual sequencer</b>	Sequence coordinating several interfaces at once, through a sequencer that only holds references to sub-sequencers and drives nothing itself.	ch10
<b>TLM (transaction-level modelling)</b>	Communication between components in whole transactions over standard interface handles, rather than through signals.	ch10
<b>RAL (register abstraction layer)</b>	Object model of the design's memory-mapped registers and memories — blocks, registers, fields — commonly called the register model.	ch10
<b>mirror vs desired value</b>	What the model believes the design currently holds, versus a value set in the model alone and pushed to the design later.	ch10
<b>front-door vs back-door access</b>	Access driving real bus cycles over the real path, versus one reaching the simulation constructs directly by hierarchical path.	ch10
<b>peek / poke</b>	Back-door sample or deposit that bypasses a field's behaviour entirely, unlike back-door read and write, which mimic the front door's side effects.	ch10
<b>field access policy</b>	Declared behaviour of a register field on read and write (read-only, write-one-to-clear, and the rest), from which the model predicts the mirror.	ch10

Term	Definition (one line)	Introduced
<b>volatile (register field)</b>	A field the design can change unobserved, so its mirrored value may not be trusted without a fresh read.	ch10
<b>implicit vs explicit prediction</b>	Updating the mirror only from accesses the model itself issued, versus updating it from a predictor fed by the bus monitor, which sees every access.	ch10
<b>register description language</b>	Machine-readable single source for a register map — addresses, fields and their behaviour — from which model, decode logic and headers are generated.	ch10
<b>first/second/third-order reuse</b>	One environment across many testcases; its components in a system-level environment; those components in a different environment for a different design.	ch10
<b>verification IP (VIP)</b>	A verification component supplied ready-made for a standard protocol, typically purchased rather than written.	ch10
<b>assertion</b>	A declarative statement of a property the design must hold, evaluated by a tool, whose falsehood indicates an error.	ch11
<b>SVA (SystemVerilog Assertions)</b>	The four-layer notation — Boolean, sequence, property, statement — in which temporal design rules are written once and read by simulator and proof engine alike.	ch11
<b>concurrent assertion</b>	Clocked, temporal assertion evaluated at clock ticks over sampled values, checked on every run that executes it.	ch11
<b>immediate assertion</b>	Procedural assertion testing a non-temporal expression when control flow reaches it, where x or z counts as failure.	ch11
<b>sampled value</b>	The value a concurrent assertion sees: the one held before the clock edge, which is why an assertion cannot race the design it watches.	ch11
<b>evaluation attempt</b>	One evaluation of a property, started afresh at every tick of its clock and running to its own verdict, overlapping the others in flight.	ch11
<b>implication (overlapping, non-overlapping)</b>	Property operator making an obligation conditional, starting the consequent in the same tick or the next; an attempt whose antecedent is false succeeds having checked nothing.	ch11
<b>disable iff</b>	Reset guard discarding any attempt in flight while its expression is true, ending it as neither pass nor failure; at most one per assertion, guarding the whole property.	ch11
<b>assumption (assume)</b>	A property claimed of the environment rather than of the design: checked like an assertion in simulation, never checked in formal, where it deletes traces instead.	ch11
<b>cover property</b>	Directive asking whether a scenario occurred: a count of matches in simulation, a reachability question answered with a witness in formal.	ch11
<b>restrict</b>	Directive constraining formal computation only; simulators ignore it.	ch11
<b>bind</b>	Construct instantiating a checker into a design scope without modifying the design's source.	ch11
<b>bound checker</b>	Module carrying its own model of an interface — counters, queues — bound to a port when the rules outgrow what a single property can state.	ch11

Term	Definition (one line)	Introduced
<b>liveness property</b>	A claim that something must eventually happen: it can never fail in simulation, where the attempt merely stays open, but a proof engine decides it.	ch11
<b>redundant-state invariant</b>	Assertion that two independently maintained representations of one fact agree — where silent corruption is most often caught.	ch11
<b>tier</b>	An admission policy with a runtime budget attached — what earns a place in a given regression run, not when that run is scheduled.	ch12
<b>smoke tier</b>	The per-commit tier: fixed seeds, bounded runtime and zero tolerance for a known-failing member, proving only that the build is alive.	ch12
<b>campaign</b>	Purpose-built run on no clock: launched for a stated reason, owned by someone, producing a written result, then stopping.	ch12
<b>(test, seed) matrix</b>	What a random regression actually runs — tests as rows, seeds as columns, each cell a run with its own outcome.	ch12
<b>run manifest</b>	Per-run record of everything needed to repeat it: seed, design and testbench revisions, exact tool build, configuration, host and result.	ch12
<b>failure signature</b>	Deduplication key built from the first error with everything run-to-run stripped out; over-generic it swallows real finds, over-specific it reduces nothing.	ch12
<b>triage</b>	Turning failures into facts before debugging any: deduplicate by signature, classify as design bug, testbench bug or environment failure, then assign.	ch12
<b>errors re-found</b>	Count of failures that re-discover an already-known open bug — the measure of how much of a night was spent learning nothing.	ch12
<b>flaky test (flake)</b>	Test whose outcome varies between runs with identical manifests; it has stopped being evidence, and it teaches the team to ignore red.	ch12
<b>quarantine</b>	Named list of confirmed flaky tests, still run and reported but not gating, each with an owner and a date.	ch12
<b>repeat-N stability check</b>	Running a test N times unchanged before admitting it to a gating tier, to establish that it is stable.	ch12
<b>X-optimism</b>	Simulation propagating a definite 0 or 1 where silicon would hold an unknown, which can mask design bugs — though asynchronous reset works because of it.	ch12
<b>time bomb</b>	Timeout ending a run that waits for something that never comes; useful only as an abnormal ending, since a suite routinely ending on it cannot tell success from deadlock.	ch12
<b>ranking</b>	Selecting the (test, seed) pairs that contribute coverage and dropping the rest — a compressed suite at equal coverage, exploring nothing new.	ch12
<b>test selection</b>	Filtering tests before simulating them, on the hypothesis that dissimilar tests hit dissimilar coverage.	ch12
<b>power domain</b>	A collection of instances treated as a group for power-management purposes, typically sharing a primary supply set.	ch19

Term	Definition (one line)	Introduced
<b>isolation</b>	Defined behaviour for a signal whose driving logic is not active.	ch19
<b>isolation cell</b>	The cell implementing isolation: it passes values normally and clamps its output to a specified value when its control asserts. Its own cell stays powered — that is what lets it clamp.	ch19
<b>level-shifter</b>	A cell translating a signal from one voltage swing to another.	ch19
<b>retention</b>	Enhanced functionality on selected sequential elements so their values survive the power-down of the primary supply.	ch19
<b>power state table (PST)</b>	A statement of the legal combinations of supply states.	ch19
<b>simstate</b>	The operational capability a supply state supports, from <b>NORMAL</b> (full switching with characterised timing) down to <b>CORRUPT</b> (the supply cannot even hold existing state). Makes the level of corruption a declared property rather than a fixed behaviour.	ch19
<b>notifier</b>	The mechanism that makes a timing violation <i>functional</i> rather than merely printed: a timing check drives a notifier signal, which the cell model uses to corrupt its output.	ch19
<b>SDF back-annotation</b>	Loading implementation delays into a simulation from a Standard Delay Format file. SystemVerilog takes only the timing constructs from that file and ignores the rest.	ch19
<b>model correlation</b>	Checking a behavioural model against the schematic it abstracts: one self-checking testbench run against both, compared per port against tolerances declared in advance. Two words deliberately — bare “correlation” appears across ten chapters in the ordinary English sense.	ch21
<b>correlation tolerance</b>	The per-port band, part of the specification and declared before any run, within which a model and the schematic must agree.	ch21
<b>model register</b>	The table of every behavioural model a project regresses against: abstraction, circuit revision correlated against, tolerances, conditions covered, date, owner, and what breaks if it is wrong. Sibling of ch14’s assumption register.	ch21
<b>fault injection</b>	<b>Safety sense:</b> forcing a fault into internal design state to see whether a safety mechanism detects it. NOT ch02’s <i>error injection</i> , which perturbs stimulus at the interface. The Italian must keep the two apart.	ch22
<b>hardware redundancy</b>	Replicated hardware whose outputs are compared or voted. NOT ch02’s <i>redundancy</i> , which is two independent readings of a specification. Same word, unrelated mechanisms.	ch22
<b>lockstep</b>	<b>Safety sense:</b> a pair of hardware cores executing identically with a comparator between them. NOT ch03’s sense of stepping an instruction-set simulator against RTL.	ch22
<b>stepping</b>	A fabrication cycle with corrected masks — what Chapter 1 calls a <b>respin</b> , in the vocabulary of the lab. The two words denote the same event; <i>stepping</i> is how a bring-up team names it.	ch20
<b>coverage differential</b>	Bin-level comparison of two regressions, used to fail a commit that quietly made the suite ask less.	ch12
<b>systematic fault</b>	A design bug: a mistake made during development, present in every part ever manufactured, and always permanent. Contrast <i>random hardware fault</i> .	ch22

Term	Definition (one line)	Introduced
<b>random hardware fault</b>	A physical defect arising during manufacturing or in operation, permanent or transient. It cannot be verified away — only detected and handled.	ch22
<b>FMEDA</b>	Failure mode, effects and diagnostic analysis: element-by-element analysis asking how each element can fail, with what consequence and likelihood, and which safety mechanism handles it.	ch22
<b>diagnostic coverage</b>	The fraction of a failure mode's faults a mechanism is claimed to catch — estimated in the FMEDA, measured by fault injection.	ch22
<b>safe fault</b>	A fault that cannot violate the safety goal, either because it cannot reach safety-related logic or because its effect is tolerated.	ch22
<b>single-point fault</b>	A fault that violates the safety goal with no safety mechanism present to catch it.	ch22
<b>residual fault</b>	A fault that violates the safety goal where a mechanism <i>is</i> present, but which falls outside what that mechanism covers.	ch22
<b>latent fault</b>	A fault that cannot violate the safety goal alone and is neither detected nor perceived; it waits for a second fault.	ch22
<b>multi-point fault</b>	A combination of faults that together violate the safety goal, classified by whether the mechanism detects it, merely perceives it, or misses it.	ch22
<b>ASIL</b>	Automotive safety integrity level, A to D with D most demanding. Assigned from a hazard's severity, exposure and controllability — <b>a property of the hazard, never of your block.</b>	ch22
<b>SPFM / LFM / PMHF</b>	Single-point fault metric, latent fault metric, probabilistic metric for random hardware failures: the architectural metrics, each carrying a threshold per integrity level.	ch22
<b>observation point</b>	Where a fault becomes visible to the outside world. A fault seen here is <i>observed</i> . Distinct from ch01's <i>observability</i> , which is the general property.	ch22
<b>detection point</b>	Where the safety mechanism raises its alarm. A fault seen here is <i>detected</i> — which is not the same as observed.	ch22
<b>good machine / faulty machine</b>	The unmodified design and a copy carrying one hypothetical injected fault, compared at designated strobe times. <b>ch22 deliberately writes “good”, never “golden”</b> — do not normalise it to ch08's bound <i>golden model</i> sense.	ch22
<b>stuck-at fault</b>	The conventional model of a permanent defect: a node held at zero or at one.	ch22
<b>single event upset (SEU)</b>	The transient fault model: one bit flipped once, then left to propagate or die out.	ch22
<b>prime fault / collapsed fault</b>	A prime fault represents an equivalence class; a collapsed fault produces the same observable behaviour as its prime, so only primes are simulated.	ch22
<b>cone of influence (COI)</b>	The netlist region that can reach an observation point. A fault outside it cannot reach one under any workload whatsoever.	ch22



Term	Definition (one line)	Introduced
<b>fault-tolerant time interval</b>	The budgeted interval, measured from the moment a fault occurs, within which a mechanism must detect and react. Paired with the <i>fault reaction time interval</i> .	ch22
<b>safety mechanism</b>	The design element that handles a failure mode — and itself a design, with bugs of its own.	ch22
<b>safe state</b>	The state a mechanism takes the system to when it fires: in a vehicle, a warning lamp, a limp mode, or a trip to the dealership.	ch22
<b>safety case</b>	The written argument shipped as a deliverable, whose claims the evidence supports and whose gaps are argued explicitly.	ch22
<b>tool qualification</b>	The argument that a tool's output can be trusted as evidence, scoped to a version, a set of use cases, and often a configuration.	ch22
<b>flow qualification</b>	Evidence that the verification <i>environment</i> can fail: systematic faults are injected and what the environment detects is measured.	ch22
<b>dependent failure analysis</b>	Analysis of what a single physical event could do to both halves of a redundant pair at once.	ch22
<b>shift-left</b>	Moving a check earlier than the stage that traditionally performed it, so a defect is found before the artifact it would have damaged exists.	ch26
<b>shift-right</b>	Moving a check <b>after tape-out</b> , onto the fabricated part — the lab and the field, not a pre-silicon platform. An emulator runs a model, so emulation is never shift-right however late it happens.	ch26
<b>stage inflation</b>	A maturity stage claimed without its exit checklist, turning a demonstrable state into an assignable status word.	ch24
<b>counterparty rule</b>	An assumption naming another team is not closed until someone from that team has read it in review and named the row that discharges it.	ch24
<b>champion</b>	A named local expert for a technique inside one team: first point of contact, with an escalation path to central expertise and continuity across projects. Borrowed from the adoption-programme literature.	ch24
<b>own an outcome</b>	Accountability stated as a claim about the design rather than about an artifact. Contrast <i>artifact ownership</i> , and note it is <b>not</b> a <i>definition of done</i> , which is an exit criterion rather than a unit of accountability.	ch24